PHP — Banco de dados (MySQLi, PDO)

Manual oficial do PHP (php.net) — English

Fonte oficial: https://www.php.net/manual/
Gerado para o projeto doc-frog

MySQL Drivers and Plugins

PHP offers several MySQL drivers and plugins for accessing and handling MySQL.

The differences and functionality of the MySQL extensions are described within the overview of this section.

The extensions listed support the MySQL protocol. Examples of compatible database servers are » MariaDB Server, » MySQL Server, » Percona Server for MySQL, and » TiDB.

Overview of the MySQL PHP drivers

Introduction

There are several PHP APIs for accessing the MySQL database. Users can choose between the mysqli or PDO_MySQL extensions.

This guide explains the terminology used to describe each API, information about choosing which API to use, and also information to help choose which MySQL library to use with the API.

Terminology overview

This section provides an introduction to the options available to you when developing a PHP application that needs to interact with a MySQL database.

What is an API?

An Application Programming Interface, or API, defines the classes, methods, functions and variables that your application will need to call in order to carry out its desired task. In the case of PHP applications that need to communicate with databases the necessary APIs are usually exposed via PHP extensions.

APIs can be procedural or object-oriented. With a procedural API you call functions to carry out tasks, with the object-oriented API you instantiate classes and then call methods on the resulting objects. Of the two, the latter is usually the preferred interface, as it is more modern and leads to better organized code.

When writing PHP applications that need to connect to the MySQL server there are several API options available. This document discusses what is available and how to select the best solution for your application.

What is a Connector?

In the MySQL documentation, the term *connector* refers to a piece of software that allows your application to connect to the MySQL database server. MySQL provides connectors for a variety of languages, including PHP.

If your PHP application needs to communicate with a database server you will need to write PHP code to perform such activities as connecting to the database server, querying the database and other database-related functions. Software is required to provide the API that your PHP application will use, and also handle the communication between your application and the database server, possibly using other intermediate libraries where necessary. This software is known generically as a connector, as it allows your application to *connect* to a database server.

What is a Driver?

A driver is a piece of software designed to communicate with a specific type of database server. The driver may also call a library, such as the MySQL Client Library or the MySQL Native Driver. These libraries implement the

low-level protocol used to communicate with the MySQL database server.

By way of an example, the PHP Data Objects (PDO) database abstraction layer may use one of several database-specific drivers. One of the drivers it has available is the PDO MYSQL driver, which allows it to interface with the MySQL server.

Sometimes people use the terms connector and driver interchangeably, which can be confusing. In the MySQL-related documentation the term “driver” is reserved for software that provides the database-specific part of a connector package.

What is an Extension?

In the PHP documentation, you will come across another term - *extension*. The PHP code consists of a core, with optional extensions to the core functionality. PHP's MySQL-related extension, `mysqli`, is implemented using the PHP extension framework.

An extension typically exposes an API to the PHP programmer, to allow its facilities to be used programmatically. However, some extensions which use the PHP extension framework do not expose an API to the PHP programmer.

The PDO MySQL driver extension, for example, does not expose an API to the PHP programmer, but provides an interface to the PDO layer above it.

The terms API and extension should not be taken to mean the same thing, as an extension may not necessarily expose an API to the programmer.

Choosing an API

PHP offers different APIs to connect to MySQL. Below we show the APIs provided by the mysqli and PDO extensions. Each code snippet creates a connection to a MySQL server running on "example.com" using the username "user" and the password "password". And a query is run to greet the user.

Example #1 Comparing the MySQL APIs

```php
<?php
// mysqli
$mysqli = new mysqli("example.com", "user", "password", "database");
$result = $mysqli->query("SELECT 'Hello, dear MySQL user!' AS _message FROM DUAL");
$row = $result->fetch_assoc();
echo htmlentities($row['_message']);

// PDO
$pdo = new PDO('mysql:host=example.com;dbname=database', 'user', 'password');
$statement = $pdo->query("SELECT 'Hello, dear MySQL user!' AS _message FROM DUAL");
$row = $statement->fetch(PDO::FETCH_ASSOC);
echo htmlentities($row['_message']);
```

Example #2 Comparing prepared statements

```php
<?php
// mysqli
$mysqli = new mysqli("example.com", "user", "password", "database");
$statement = $mysqli->prepare("SELECT District FROM City WHERE Name=?");
$statement->execute(["Amersfoort"]);
$result = $statement->get_result();
$row = $result->fetch_assoc();
```

```php
echo htmlentities($row['District']);

// PDO
$pdo = new PDO('mysql:host=example.com;dbname=database', 'user', 'password');
$statement = $pdo->prepare("SELECT District FROM City WHERE Name=?");
$statement->execute(["Amersfoort"]);
$row = $statement->fetch(PDO::FETCH_ASSOC);
echo htmlentities($row['District']);
```

Feature comparison

The overall performance of both extensions is considered to be about the same. Although the performance of the extension contributes only a fraction of the total run time of a PHP web request. Often, the impact is as low as 0.1%.

	ext/mysqli	PDO_MySQL
PHP version introduced	5.0	5.1
Included with PHP 7.x and 8.x	Yes	Yes
Development status	Active	Active
Lifecycle	Active	Active
Recommended for new projects	Yes	Yes
OOP Interface	Yes	Yes
Procedural Interface	Yes	No
API supports non-blocking, asynchronous queries with mysqlnd	Yes	No
Persistent Connections	Yes	Yes
API supports Charsets	Yes	Yes
API supports server-side Prepared Statements	Yes	Yes
API supports client-side Prepared Statements	No	Yes
API supports Stored Procedures	Yes	Yes
API supports Multiple Statements	Yes	Most
API supports Transactions	Yes	Yes
Transactions can be controlled with SQL	Yes	Yes
Supports all MySQL 5.1+ functionality	Yes	Most

Choosing a library

The mysqli and PDO_MySQL PHP extensions are lightweight wrappers on top of a C client library. The extensions can either use the mysqlnd library or the `libmysqlclient` library. Choosing a library is a compile time decision.

The mysqlnd library is part of the PHP distribution. It offers features like lazy connections and query caching, features that are not available with libmysqlclient, so using the built-in mysqlnd library is highly recommended. See the mysqlnd documentation for additional details, and a listing of features and functionality that it offers.

Example #1 Configure commands for using mysqlnd or libmysqlclient

```
// Recommended, compiles with mysqlnd
$ ./configure --with-mysqli=mysqlnd --with-pdo-mysql=mysqlnd
```

```
// Alternatively recommended, compiles with mysqlnd
$ ./configure --with-mysqli --with-pdo-mysql

// Not recommended, compiles with libmysqlclient
$ ./configure --with-mysqli=/path/to/mysql_config --with-pdo-mysql=/path/to/mysql_config
```

Library feature comparison

It is recommended to use the mysqlnd library instead of the MySQL Client Server library (libmysqlclient). Both libraries are supported and constantly being improved.

	MySQL native driver (mysqlnd)	**MySQL client server library (`libmysqlclient`)**
Part of the PHP distribution	Yes	No
PHP version introduced	5.3.0	N/A
License	PHP License 3.01	Dual-License
Development status	Active	Active
Lifecycle	No end announced	No end announced
Compile default (for all MySQL extensions)	Yes	No
Compression protocol support	Yes	Yes
SSL support	Yes	Yes
Named pipe support	Yes	Yes
Non-blocking, asynchronous queries	Yes	No
Performance statistics	Yes	No
LOAD LOCAL INFILE respects the open_basedir directive	Yes	No
Uses PHP's native memory management system (e.g., follows PHP memory limits)	Yes	No
Return numeric column as double (COM_QUERY)	Yes	No
Return numeric column as string (COM_QUERY)	Yes	Yes
Plugin API	Yes	Limited
Automatic reconnect	No	Optional

Concepts

Table of Contents

These concepts are specific to the MySQL drivers for PHP.

Buffered and Unbuffered queries

Queries are using the buffered mode by default. This means that query results are immediately transferred from the MySQL Server to PHP and then are kept in the memory of the PHP process. This allows additional operations like counting the number of rows, and moving (seeking) the current result pointer. It also allows issuing further queries on the same connection while working on the result set. The downside of the buffered mode is that larger result sets might require quite a lot memory. The memory will be kept occupied till all references to the

result set are unset or the result set was explicitly freed, which will automatically happen during request end at the latest. The terminology "store result" is also used for buffered mode, as the whole result set is stored at once.

> **Note** :
>
> When using libmysqlclient as library PHP's memory limit won't count the memory used for result sets unless the data is fetched into PHP variables. With mysqlnd the memory accounted for will include the full result set.

Unbuffered MySQL queries execute the query and then wait for the data from the MySQL server to be fetched. This uses less memory on the PHP-side, but can increase the load on the server. Unless the full result set was fetched from the server no further queries can be sent over the same connection. Unbuffered queries can also be referred to as "use result". Once all rows in the result set are fetched, the result set is gone and it cannot be iterated again.

Following these characteristics, unbuffered queries should be used only when a large result set is expected that will be processed sequentially. Unbuffered queries contain a number of pitfalls that makes it more difficult to use them, e.g. the number of rows in the result set is unknown until the last row is fetched. Buffered queries are the easier and more flexible way to process result sets.

Because buffered queries are the default, the examples below will demonstrate how to execute unbuffered queries with each API.

Example #1 Unbuffered query example: mysqli

```
<?php
$mysqli  = new mysqli("localhost", "my_user", "my_password", "world");
$unbufferedResult = $mysqli->query("SELECT Name FROM City", MYSQLI_USE_RESULT);

foreach ($unbufferedResult as $row) {
    echo $row['Name'] . PHP_EOL;
}
?>
```

Example #2 Unbuffered query example: pdo_mysql

```
<?php
$pdo = new PDO("mysql:host=localhost;dbname=world", 'my_user', 'my_password');
$pdo->setAttribute(PDO::MYSQL_ATTR_USE_BUFFERED_QUERY, false);

$unbufferedResult = $pdo->query("SELECT Name FROM City");
foreach ($unbufferedResult as $row) {
    echo $row['Name'] . PHP_EOL;
}
?>
```

Character sets

Ideally a proper character set will be set at the server level, and doing this is described within the » Character Set Configuration section of the MySQL Server manual. Alternatively, each MySQL API offers a method to set the character set at runtime.

> **Caution**

> **The character set and character escaping**
>
> The character set should be understood and defined, as it has an affect on every action, and includes security implications. For example, the escaping mechanism (e.g., mysqli_real_escape_string() for mysqli and PDO::quote() for PDO_MySQL) will adhere to this setting. It is important to realize that these functions will not use the character set that is defined with a query, so for example the following will not have an effect on them:

Example #1 Problems with setting the character set with SQL

```php
<?php

$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

// Will NOT affect $mysqli->real_escape_string();
$mysqli->query("SET NAMES utf8mb4");

// Will NOT affect $mysqli->real_escape_string();
$mysqli->query("SET CHARACTER SET utf8mb4");

// But, this will affect $mysqli->real_escape_string();
$mysqli->set_charset('utf8mb4');

// But, this will NOT affect it (UTF-8 vs utf8mb4) -- don't use dashes here
$mysqli->set_charset('UTF-8');
?>
```

Below are examples that demonstrate how to properly alter the character set at runtime using each API.

> **Note** : **Possible UTF-8 confusion**
>
> Because character set names in MySQL do not contain dashes, the string "utf8" is valid in MySQL to set the character set to UTF-8 (up to 3 byte UTF-8 Unicode Encoding). The string "UTF-8" is not valid, as using "UTF-8" will fail to change the character set and will throw an error.

Example #2 Setting the character set example: mysqli

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

echo 'Initial character set: ' . $mysqli->character_set_name() . "\n";

if (!$mysqli->set_charset('utf8mb4')) {
    printf("Error loading character set utf8mb4: %s\n", $mysqli->error);
    exit;
}

echo 'Your current character set is: ' . $mysqli->character_set_name() . "\n";
?>
```

Example #3 Setting the character set example: pdo_mysql

```php
<?php
$pdo = new PDO("mysql:host=localhost;dbname=world;charset=utf8mb4", 'my_user', 'my_pass');
```

```
?>
```

MySQLi

Introduction

The `mysqli` extension allows you to access the functionality provided by MySQL 4.1 and above. More information about the MySQL Database server can be found at » http://www.mysql.com/

An overview of software available for using MySQL from PHP can be found at Overview

Documentation for MySQL can be found at » http://dev.mysql.com/doc/.

Parts of this documentation were included from the MySQL manual with permission of Oracle Corporation.

Examples use either the » world or » sakila database, which are freely available.

Overview

This section provides an introduction to the options available to you when developing a PHP application that needs to interact with a MySQL database.

What is an API?

An Application Programming Interface, or API, defines the classes, methods, functions and variables that your application will need to call in order to carry out its desired task. In the case of PHP applications that need to communicate with databases the necessary APIs are usually exposed via PHP extensions.

APIs can be procedural or object-oriented. With a procedural API, you call functions to carry out tasks, with the object-oriented API you instantiate classes and then call methods on the resulting objects. Of the two, the latter is usually the preferred interface, as it is more modern and leads to better-organized code.

When writing PHP applications that need to connect to the MySQL server there are several API options available. This document discusses what is available and how to select the best solution for your application.

What is a Connector?

In the MySQL documentation, the term *connector* refers to a piece of software that allows your application to connect to the MySQL database server. MySQL provides connectors for a variety of languages, including PHP.

If your PHP application needs to communicate with a database server you will need to write PHP code to perform such activities as connecting to the database server, querying the database, and other database-related functions. Software is required to provide the API that your PHP application will use, and also handle the communication between your application and the database server, possibly using other intermediate libraries where necessary. This software is known generically as a connector, as it allows your application to *connect* to a database server.

What is a Driver?

A driver is a piece of software designed to communicate with a specific type of database server. The driver may also call a library, such as the MySQL Client Library or the MySQL Native Driver. These libraries implement the low-level protocol used to communicate with the MySQL database server.

By way of an example, the PHP Data Objects (PDO) database abstraction layer may use one of several database-specific drivers. One of the drivers it has available is the PDO MYSQL driver, which allows it to interface with the

MySQL server.

Sometimes people use the terms connector and driver interchangeably, which can be confusing. In the MySQL-related documentation the term “driver” is reserved for software that provides the database-specific part of a connector package.

What is an Extension?

In the PHP documentation you will come across another term - *extension*. The PHP code consists of a core, with optional extensions to the core functionality. PHP's MySQL-related extensions, such as the mysqli extension, and the PDO MySQL driver extension, are implemented using the PHP extension framework.

An extension typically exposes an API to the PHP programmer, to allow its facilities to be used programmatically. However, some extensions which use the PHP extension framework do not expose an API to the PHP programmer.

The PDO MySQL driver extension, for example, does not expose an API to the PHP programmer, but provides an interface to the PDO layer above it.

The terms API and extension should not be taken to mean the same thing, as an extension may not necessarily expose an API to the programmer.

What are the main PHP API offerings for using MySQL?

There are two main API options when considering connecting to a MySQL database server:

- PHP's mysqli Extension
- PHP Data Objects (PDO)

Each has its own advantages and disadvantages. The following discussion aims to give a brief introduction to the key aspects of each API.

What is PHP's mysqli Extension?

The `mysqli` extension, or as it is sometimes known, the MySQL *improved* extension, was developed to take advantage of new features found in MySQL systems versions 4.1.3 and newer. The `mysqli` extension is included with PHP versions 5 and later.

The `mysqli` extension has a number of benefits, the key enhancements over the `mysql` extension being:

- Object-oriented interface
- Support for Prepared Statements
- Support for Multiple Statements
- Support for Transactions
- Enhanced debugging capabilities

As well as the object-oriented interface the extension also provides a procedural interface.

The `mysqli` extension is built using the PHP extension framework, its source code is located in the directory *ext/mysqli*.

For further information on the `mysqli` extension, see MySQLi.

What is PDO?

PHP Data Objects, or PDO, is a database abstraction layer specifically for PHP applications. PDO provides a consistent API for your PHP application regardless of the type of database server your application will connect to. In theory, if you are using the PDO API, you could switch the database server you used, from say Firebird to MySQL, and only need to make minor changes to your PHP code.

Other examples of database abstraction layers include JDBC for Java applications and DBI for Perl.

While PDO has its advantages, such as a clean, simple, portable API, its main disadvantage is that it doesn't allow you to use all of the advanced features that are available in the latest versions of MySQL server. For example, PDO does not allow you to use MySQL's support for Multiple Statements.

PDO is implemented using the PHP extension framework, its source code is located in the directory *ext/pdo*.

For further information on PDO, see the PDO.

What is the PDO MYSQL driver?

The PDO MYSQL driver is not an API as such, at least from the PHP programmer's perspective. In fact, the PDO MYSQL driver sits in the layer below PDO itself and provides MySQL-specific functionality. The programmer still calls the PDO API, but PDO uses the PDO MYSQL driver to carry out communication with the MySQL server.

The PDO MYSQL driver is one of several available PDO drivers. Other PDO drivers available include those for the Firebird and PostgreSQL database servers.

The PDO MYSQL driver is implemented using the PHP extension framework. Its source code is located in the directory *ext/pdo_mysql*. It does not expose an API to the PHP programmer.

For further information on the PDO MYSQL driver, see MySQL PDO Driver.

What is PHP's MySQL Native Driver?

In order to communicate with the MySQL database server, `mysqli` and the PDO MYSQL driver each use a low-level library that implements the required protocol. In the past, the only available library was the MySQL Client Library, otherwise known as `libmysqlclient`.

However, the interface presented by `libmysqlclient` was not optimized for communication with PHP applications, as `libmysqlclient` was originally designed with C applications in mind. For this reason, the MySQL Native Driver, `mysqlnd`, was developed as an alternative to `libmysqlclient` for PHP applications.

Both the `mysqli` extension and the PDO MySQL driver can each be individually configured to use either `libmysqlclient` or `mysqlnd`. As `mysqlnd` is designed specifically to be utilised in the PHP system it has numerous memory and speed enhancements over `libmysqlclient`. You are strongly encouraged to take advantage of these improvements.

The MySQL Native Driver is implemented using the PHP extension framework. The source code is located in *ext/mysqlnd*. It does not expose an API to the PHP programmer.

> **Note** :
>
> There are different internal implementations within `libmysqlclient` and `mysqlnd` for handling columns of type `MYSQL_TYPE_GEOMETRY`. Generally speaking, `mysqlnd` will allocate significantly less memory. For example, if there is a `POINT` column in a result set, `libmysqlclient` may pre-allocate up to 4GB of RAM although less than 50

> bytes are needed for holding a `POINT` column in memory. Memory allocation is much lower, less than 50 bytes, if using `mysqlnd`.

Comparison of Features

The following table compares the functionality of the main methods of connecting to MySQL from PHP:

Comparison of MySQL API options for PHP

	PHP's mysqli Extension	PDO (Using PDO MySQL Driver and MySQL Native Driver)
PHP version introduced	5.0	5.0
MySQL development status	Active development	Active development
API supports Charsets	Yes	Yes
API supports server-side Prepared Statements	Yes	Yes
API supports client-side Prepared Statements	No	Yes
API supports Stored Procedures	Yes	Yes
API supports Multiple Statements	Yes	Most
Supports all MySQL 4.1+ functionality	Yes	Most

Quick start guide

Table of Contents

This quick start guide will help with choosing and gaining familiarity with the PHP MySQL API.

This quick start gives an overview on the mysqli extension. Code examples are provided for all major aspects of the API. Database concepts are explained to the degree needed for presenting concepts specific to MySQL.

Required: A familiarity with the PHP programming language, the SQL language, and basic knowledge of the MySQL server.

Dual procedural and object-oriented interface

The mysqli extension features a dual interface. It supports the procedural and object-oriented programming paradigm.

Users migrating from the old mysql extension may prefer the procedural interface. The procedural interface is similar to that of the old mysql extension. In many cases, the function names differ only by prefix. Some mysqli functions take a connection handle as their first argument, whereas matching functions in the old mysql interface take it as an optional last argument.

Example #1 Easy migration from the old mysql extension

```
<?php
$mysqli = mysqli_connect("example.com", "user", "password", "database");
$result = mysqli_query($mysqli, "SELECT 'Please do not use the deprecated mysql extension for new
development. ' AS _msg FROM DUAL");
```

```php
$row = mysqli_fetch_assoc($result);
echo $row['_msg'];

$mysql = mysql_connect("example.com", "user", "password");
mysql_select_db("test");
$result = mysql_query("SELECT 'Use the mysqli extension instead.' AS _msg FROM DUAL", $mysql);
$row = mysql_fetch_assoc($result);
echo $row['_msg'];
```

The above example will output:

```
Please do not use the deprecated mysql extension for new development. Use the mysqli extension
instead.
```

The object-oriented interface

In addition to the classical procedural interface, users can choose to use the object-oriented interface. The documentation is organized using the object-oriented interface. The object-oriented interface shows functions grouped by their purpose, making it easier to get started. The reference section gives examples for both syntax variants.

There are no significant performance differences between the two interfaces. Users can base their choice on personal preference.

Example #2 Object-oriented and procedural interface

```php
<?php

$mysqli = mysqli_connect("example.com", "user", "password", "database");

$result = mysqli_query($mysqli, "SELECT 'A world full of ' AS _msg FROM DUAL");
$row = mysqli_fetch_assoc($result);
echo $row['_msg'];

$mysqli = new mysqli("example.com", "user", "password", "database");

$result = $mysqli->query("SELECT 'choices to please everybody.' AS _msg FROM DUAL");
$row = $result->fetch_assoc();
echo $row['_msg'];
```

The above example will output:

```
A world full of choices to please everybody.
```

The object-oriented interface is used for the quickstart because the reference section is organized that way.

Mixing styles

It is possible to switch between styles at any time. Mixing both styles is not recommended for code clarity and coding style reasons.

Example #3 Bad coding style

```php
<?php

$mysqli = new mysqli("example.com", "user", "password", "database");

$result = mysqli_query($mysqli, "SELECT 'Possible but bad style.' AS _msg FROM DUAL");

if ($row = $result->fetch_assoc()) {
    echo $row['_msg'];
}
```

The above example will output:

```
Possible but bad style.
```

See also

- mysqli::__construct()
- mysqli::query()
- mysqli_result::fetch_assoc()
- $mysqli::connect_errno
- $mysqli::connect_error
- $mysqli::errno
- $mysqli::error

Connections

The MySQL server supports the use of different transport layers for connections. Connections use TCP/IP, Unix domain sockets or Windows named pipes.

The hostname `localhost` has a special meaning. It is bound to the use of Unix domain sockets. To open a TCP/IP connection to the localhost, `127.0.0.1` must be used instead of the hostname `localhost`.

Example #1 Special meaning of localhost

```php
<?php

$mysqli = new mysqli("localhost", "user", "password", "database");

echo $mysqli->host_info . "\n";

$mysqli = new mysqli("127.0.0.1", "user", "password", "database", 3306);

echo $mysqli->host_info . "\n";
```

The above example will output:

```
Localhost via UNIX socket
127.0.0.1 via TCP/IP
```

Connection parameter defaults

Depending on the connection function used, assorted parameters can be omitted. If a parameter is not provided, then the extension attempts to use the default values that are set in the PHP configuration file.

Example #2 Setting defaults

```
mysqli.default_host=192.168.2.27
mysqli.default_user=root
mysqli.default_pw=""
mysqli.default_port=3306
mysqli.default_socket=/tmp/mysql.sock
```

The resulting parameter values are then passed to the client library that is used by the extension. If the client library detects empty or unset parameters, then it may default to the library built-in values.

Built-in connection library defaults

If the host value is unset or empty, then the client library will default to a Unix socket connection on `localhost`. If socket is unset or empty, and a Unix socket connection is requested, then a connection to the default socket on `/tmp/mysql.sock` is attempted.

On Windows systems, the host name `.` is interpreted by the client library as an attempt to open a Windows named pipe based connection. In this case the socket parameter is interpreted as the pipe name. If not given or empty, then the socket (pipe name) defaults to `\\.\pipe\MySQL`.

If neither a Unix domain socket-based nor a Windows named pipe-based connection is to be established and the port parameter value is unset, the library will default to port `3306`.

The mysqlnd library and the MySQL Client Library (libmysqlclient) implement the same logic for determining defaults.

Connection options

Connection options are available to, for example, set init commands which are executed upon connect, or for requesting use of a certain charset. Connection options must be set before a network connection is established.

For setting a connection option, the connect operation has to be performed in three steps: creating a connection handle with mysqli_init() or mysqli::__construct(), setting the requested options using mysqli::options(), and establishing the network connection with mysqli::real_connect().

Connection pooling

The mysqli extension supports persistent database connections, which are a special kind of pooled connections. By default, every database connection opened by a script is either explicitly closed by the user during runtime or released automatically at the end of the script. A persistent connection is not. Instead it is put into a pool for later reuse, if a connection to the same server using the same username, password, socket, port and default database is opened. Reuse saves connection overhead.

Every PHP process is using its own mysqli connection pool. Depending on the web server deployment model, a PHP process may serve one or multiple requests. Therefore, a pooled connection may be used by one or more scripts subsequently.

Persistent connection

If an unused persistent connection for a given combination of host, username, password, socket, port and default database cannot be found in the connection pool, then mysqli opens a new connection. The use of persistent connections can be enabled and disabled using the PHP directive mysqli.allow_persistent. The total number of connections opened by a script can be limited with mysqli.max_links. The maximum number of persistent connections per PHP process can be restricted with mysqli.max_persistent. Please note that the web server may spawn many PHP processes.

A common complaint about persistent connections is that their state is not reset before reuse. For example, open and unfinished transactions are not automatically rolled back. But also, authorization changes which happened in the time between putting the connection into the pool and reusing it are not reflected. This may be seen as an unwanted side-effect. On the contrary, the name `persistent` may be understood as a promise that the state is persisted.

The mysqli extension supports both interpretations of a persistent connection: state persisted, and state reset before reuse. The default is reset. Before a persistent connection is reused, the mysqli extension implicitly calls mysqli::change_user() to reset the state. The persistent connection appears to the user as if it was just opened. No artifacts from previous usages are visible.

The mysqli::change_user() call is an expensive operation. For best performance, users may want to recompile the extension with the compile flag **`MYSQLI_NO_CHANGE_USER_ON_PCONNECT`** being set.

It is left to the user to choose between safe behavior and best performance. Both are valid optimization goals. For ease of use, the safe behavior has been made the default at the expense of maximum performance.

See also

- mysqli::__construct()
- mysqli_init()
- mysqli::options()
- mysqli::real_connect()
- mysqli::change_user()
- $mysqli::host_info
- MySQLi Configuration Options
- Persistent Database Connections

Executing statements

Statements can be executed with the mysqli::query(), mysqli::real_query() and mysqli::multi_query(). The mysqli::query() function is the most common, and combines the executing statement with a buffered fetch of its result set, if any, in one call. Calling mysqli::query() is identical to calling mysqli::real_query() followed by mysqli::store_result().

Example #1 Executing queries

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");
```

```
$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");
```

Buffered result sets

After statement execution, results can be either retrieved all at once or read row by row from the server. Client-side result set buffering allows the server to free resources associated with the statement's results as early as possible. Generally speaking, clients are slow consuming result sets. Therefore, it is recommended to use buffered result sets. mysqli::query() combines statement execution and result set buffering.

PHP applications can navigate freely through buffered results. Navigation is fast because the result sets are held in client memory. Please, keep in mind that it is often easier to scale by client than it is to scale the server.

Example #2 Navigation through buffered results

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");
$mysqli->query("INSERT INTO test(id) VALUES (1), (2), (3)");

$result = $mysqli->query("SELECT id FROM test ORDER BY id ASC");

echo "Reverse order...\n";
for ($row_no = $result->num_rows - 1; $row_no >= 0; $row_no--) {
    $result->data_seek($row_no);
    $row = $result->fetch_assoc();
    echo " id = " . $row['id'] . "\n";
}

echo "Result set order...\n";
foreach ($result as $row) {
    echo " id = " . $row['id'] . "\n";
}
```

The above example will output:

```
Reverse order...
 id = 3
 id = 2
 id = 1
Result set order...
 id = 1
 id = 2
 id = 3
```

Unbuffered result sets

If client memory is a short resource and freeing server resources as early as possible to keep server load low is not needed, unbuffered results can be used. Scrolling through unbuffered results is not possible before all rows have been read.

Example #3 Navigation through unbuffered results

```php
<?php

$mysqli->real_query("SELECT id FROM test ORDER BY id ASC");
$result = $mysqli->use_result();

echo "Result set order...\n";
foreach ($result as $row) {
    echo " id = " . $row['id'] . "\n";
}
```

Result set values data types

The mysqli::query(), mysqli::real_query() and mysqli::multi_query() functions are used to execute non-prepared statements. At the level of the MySQL Client Server Protocol, the command `COM_QUERY` and the text protocol are used for statement execution. With the text protocol, the MySQL server converts all data of a result sets into strings before sending. This conversion is done regardless of the SQL result set column data type. The mysql client libraries receive all column values as strings. No further client-side casting is done to convert columns back to their native types. Instead, all values are provided as PHP strings.

Example #4 Text protocol returns strings by default

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label CHAR(1))");
$mysqli->query("INSERT INTO test(id, label) VALUES (1, 'a')");

$result = $mysqli->query("SELECT id, label FROM test WHERE id = 1");
$row = $result->fetch_assoc();

printf("id = %s (%s)\n", $row['id'], gettype($row['id']));
printf("label = %s (%s)\n", $row['label'], gettype($row['label']));
```

The above example will output:

```
id = 1 (string)
label = a (string)
```

It is possible to convert integer and float columns back to PHP numbers by setting the `MYSQLI_OPT_INT_AND_FLOAT_NATIVE` connection option, if using the mysqlnd library. If set, the mysqlnd library will check the result set meta data column types and convert numeric SQL columns to PHP numbers, if the PHP data type value range allows for it. This way, for example, SQL INT columns are returned as integers.

Example #5 Native data types with mysqlnd and connection option

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = new mysqli();
$mysqli->options(MYSQLI_OPT_INT_AND_FLOAT_NATIVE, 1);
$mysqli->real_connect("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label CHAR(1))");
$mysqli->query("INSERT INTO test(id, label) VALUES (1, 'a')");

$result = $mysqli->query("SELECT id, label FROM test WHERE id = 1");
$row = $result->fetch_assoc();

printf("id = %s (%s)\n", $row['id'], gettype($row['id']));
printf("label = %s (%s)\n", $row['label'], gettype($row['label']));
```

The above example will output:

```
id = 1 (integer)
label = a (string)
```

See also

- mysqli::__construct()
- mysqli::options()
- mysqli::real_connect()
- mysqli::query()
- mysqli::multi_query()
- mysqli::use_result()
- mysqli::store_result()

Prepared Statements

The MySQL database supports prepared statements. A prepared statement or a parameterized statement is used to execute the same statement repeatedly with high efficiency and protect against SQL injections.

Basic workflow

The prepared statement execution consists of two stages: prepare and execute. At the prepare stage a statement template is sent to the database server. The server performs a syntax check and initializes server internal resources for later use.

The MySQL server supports using anonymous, positional placeholder with `?`.

Prepare is followed by execute. During execute the client binds parameter values and sends them to the server. The server executes the statement with the bound values using the previously created internal resources.

Example #1 Prepared statement

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Non-prepared statement */
$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label TEXT)");

/* Prepared statement, stage 1: prepare */
$stmt = $mysqli->prepare("INSERT INTO test(id, label) VALUES (?, ?)");

/* Prepared statement, stage 2: bind and execute */
$id = 1;
$label = 'PHP';
$stmt->bind_param("is", $id, $label); // "is" means that $id is bound as an integer and $label as a
string

$stmt->execute();
```

Repeated execution

A prepared statement can be executed repeatedly. Upon every execution the current value of the bound variable is evaluated and sent to the server. The statement is not parsed again. The statement template is not transferred to the server again.

Example #2 INSERT prepared once, executed multiple times

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Non-prepared statement */
$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label TEXT)");

/* Prepared statement, stage 1: prepare */
$stmt = $mysqli->prepare("INSERT INTO test(id, label) VALUES (?, ?)");

/* Prepared statement, stage 2: bind and execute */
$stmt->bind_param("is", $id, $label); // "is" means that $id is bound as an integer and $label as a
string

$data = [
    1 => 'PHP',
    2 => 'Java',
    3 => 'C++'
];
foreach ($data as $id => $label) {
    $stmt->execute();
}

$result = $mysqli->query('SELECT id, label FROM test');
var_dump($result->fetch_all(MYSQLI_ASSOC));
```

The above example will output:

```
array(3) {
  [0]=>
  array(2) {
    ["id"]=>
    string(1) "1"
    ["label"]=>
    string(3) "PHP"
  }
  [1]=>
  array(2) {
    ["id"]=>
    string(1) "2"
    ["label"]=>
    string(4) "Java"
  }
  [2]=>
  array(2) {
    ["id"]=>
    string(1) "3"
    ["label"]=>
    string(3) "C++"
  }
}
```

Every prepared statement occupies server resources. Statements should be closed explicitly immediately after use. If not done explicitly, the statement will be closed when the statement handle is freed by PHP.

Using a prepared statement is not always the most efficient way of executing a statement. A prepared statement executed only once causes more client-server round-trips than a non-prepared statement. This is why the SELECT is not run as a prepared statement above.

Also, consider the use of the MySQL multi-INSERT SQL syntax for INSERTs. For the example, multi-INSERT requires fewer round-trips between the server and client than the prepared statement shown above.

Example #3 Less round trips using multi-INSERT SQL

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");

$values = [1, 2, 3, 4];

$stmt = $mysqli->prepare("INSERT INTO test(id) VALUES (?), (?), (?), (?)");
$stmt->bind_param('iiii', ...$values);
$stmt->execute();
```

Result set values data types

The MySQL Client Server Protocol defines a different data transfer protocol for prepared statements and non-prepared statements. Prepared statements are using the so called binary protocol. The MySQL server sends result set data "as is" in binary format. Results are not serialized into strings before sending. Client libraries receive binary data and try to convert the values into appropriate PHP data types. For example, results from an SQL `INT` column will be provided as PHP integer variables.

Example #4 Native datatypes

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Non-prepared statement */
$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label TEXT)");
$mysqli->query("INSERT INTO test(id, label) VALUES (1, 'PHP')");

$stmt = $mysqli->prepare("SELECT id, label FROM test WHERE id = 1");
$stmt->execute();
$result = $stmt->get_result();
$row = $result->fetch_assoc();

printf("id = %s (%s)\n", $row['id'], gettype($row['id']));
printf("label = %s (%s)\n", $row['label'], gettype($row['label']));
```

The above example will output:

```
id = 1 (integer)
label = PHP (string)
```

This behavior differs from non-prepared statements. By default, non-prepared statements return all results as strings. This default can be changed using a connection option. If the connection option is used, there are no differences.

Fetching results using bound variables

Results from prepared statements can either be retrieved by binding output variables, or by requesting a mysqli_result object.

Output variables must be bound after statement execution. One variable must be bound for every column of the statements result set.

Example #5 Output variable binding

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Non-prepared statement */
$mysqli->query("DROP TABLE IF EXISTS test");
```

```php
$mysqli->query("CREATE TABLE test(id INT, label TEXT)");
$mysqli->query("INSERT INTO test(id, label) VALUES (1, 'PHP')");

$stmt = $mysqli->prepare("SELECT id, label FROM test WHERE id = 1");
$stmt->execute();

$stmt->bind_result($out_id, $out_label);

while ($stmt->fetch()) {
    printf("id = %s (%s), label = %s (%s)\n", $out_id, gettype($out_id), $out_label,
gettype($out_label));
}
```

The above example will output:

```
id = 1 (integer), label = PHP (string)
```

Prepared statements return unbuffered result sets by default. The results of the statement are not implicitly fetched and transferred from the server to the client for client-side buffering. The result set takes server resources until all results have been fetched by the client. Thus it is recommended to consume results timely. If a client fails to fetch all results or the client closes the statement before having fetched all data, the data has to be fetched implicitly by `mysqli`.

It is also possible to buffer the results of a prepared statement using mysqli_stmt::store_result().

Fetching results using mysqli_result interface

Instead of using bound results, results can also be retrieved through the mysqli_result interface. mysqli_stmt::get_result() returns a buffered result set.

Example #6 Using mysqli_result to fetch results

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Non-prepared statement */
$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label TEXT)");
$mysqli->query("INSERT INTO test(id, label) VALUES (1, 'PHP')");

$stmt = $mysqli->prepare("SELECT id, label FROM test WHERE id = 1");
$stmt->execute();

$result = $stmt->get_result();

var_dump($result->fetch_all(MYSQLI_ASSOC));
```

The above example will output:

```
array(1) {
  [0]=>
  array(2) {
    ["id"]=>
    int(1)
    ["label"]=>
    string(3) "PHP"
  }
}
```

Using the mysqli_result interface offers the additional benefit of flexible client-side result set navigation.

Example #7 Buffered result set for flexible read out

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Non-prepared statement */
$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT, label TEXT)");
$mysqli->query("INSERT INTO test(id, label) VALUES (1, 'PHP'), (2, 'Java'), (3, 'C++')");

$stmt = $mysqli->prepare("SELECT id, label FROM test");
$stmt->execute();

$result = $stmt->get_result();

for ($row_no = $result->num_rows - 1; $row_no >= 0; $row_no--) {
    $result->data_seek($row_no);
    var_dump($result->fetch_assoc());
}
```

The above example will output:

```
array(2) {
  ["id"]=>
  int(3)
  ["label"]=>
  string(3) "C++"
}
array(2) {
  ["id"]=>
  int(2)
  ["label"]=>
  string(4) "Java"
}
array(2) {
  ["id"]=>
  int(1)
  ["label"]=>
```

```
  string(3) "PHP"
}
```

Escaping and SQL injection

Bound variables are sent to the server separately from the query and thus cannot interfere with it. The server uses these values directly at the point of execution, after the statement template is parsed. Bound parameters do not need to be escaped as they are never substituted into the query string directly. A hint must be provided to the server for the type of bound variable, to create an appropriate conversion. See the mysqli_stmt::bind_param() function for more information.

Such a separation is sometimes considered the only security feature to prevent SQL injection, but the same degree of security can be achieved with non-prepared statements, if all the values are formatted correctly. It should be noted that correct formatting is not the same as escaping and involves more logic than simple escaping. Thus, prepared statements are simply a more convenient and less error-prone approach to this element of database security.

Client-side prepared statement emulation

The API does not include support for client-side prepared statement emulation.

See also

- mysqli::__construct()
- mysqli::query()
- mysqli::prepare()
- mysqli_stmt::prepare()
- mysqli_stmt::execute()
- mysqli_stmt::bind_param()
- mysqli_stmt::bind_result()

Stored Procedures

The MySQL database supports stored procedures. A stored procedure is a subroutine stored in the database catalog. Applications can call and execute the stored procedure. The `CALL` SQL statement is used to execute a stored procedure.

Parameter

Stored procedures can have `IN`, `INOUT` and `OUT` parameters, depending on the MySQL version. The mysqli interface has no special notion for the different kinds of parameters.

IN parameter

Input parameters are provided with the `CALL` statement. Please, make sure values are escaped correctly.

Example #1 Calling a stored procedure

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
```

```php
$mysqli->query("CREATE TABLE test(id INT)");

$mysqli->query("DROP PROCEDURE IF EXISTS p");
$mysqli->query("CREATE PROCEDURE p(IN id_val INT) BEGIN INSERT INTO test(id) VALUES(id_val); END;");

$mysqli->query("CALL p(1)");

$result = $mysqli->query("SELECT id FROM test");

var_dump($result->fetch_assoc());
```

The above example will output:

```
array(1) {
  ["id"]=>
  string(1) "1"
}
```

INOUT/OUT parameter

The values of `INOUT`/`OUT` parameters are accessed using session variables.

Example #2 Using session variables

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP PROCEDURE IF EXISTS p");
$mysqli->query('CREATE PROCEDURE p(OUT msg VARCHAR(50)) BEGIN SELECT "Hi!" INTO msg; END;');

$mysqli->query("SET @msg = ''");
$mysqli->query("CALL p(@msg)");

$result = $mysqli->query("SELECT @msg as _p_out");

$row = $result->fetch_assoc();
echo $row['_p_out'];
```

The above example will output:

```
Hi!
```

Application and framework developers may be able to provide a more convenient API using a mix of session variables and database catalog inspection. However, please note the possible performance impact of a custom solution based on catalog inspection.

Handling result sets

Stored procedures can return result sets. Result sets returned from a stored procedure cannot be fetched correctly using mysqli::query(). The mysqli::query() function combines statement execution and fetching the first result set into a buffered result set, if any. However, there are additional stored procedure result sets hidden from the user which cause mysqli::query() to fail returning the user expected result sets.

Result sets returned from a stored procedure are fetched using mysqli::real_query() or mysqli::multi_query(). Both functions allow fetching any number of result sets returned by a statement, such as CALL. Failing to fetch all result sets returned by a stored procedure causes an error.

Example #3 Fetching results from stored procedures

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");
$mysqli->query("INSERT INTO test(id) VALUES (1), (2), (3)");

$mysqli->query("DROP PROCEDURE IF EXISTS p");
$mysqli->query('CREATE PROCEDURE p() READS SQL DATA BEGIN SELECT id FROM test; SELECT id + 1 FROM test;
END;');

$mysqli->multi_query("CALL p()");

do {
    if ($result = $mysqli->store_result()) {
        printf("---\n");
        var_dump($result->fetch_all());
        $result->free();
    }
} while ($mysqli->next_result());
```

The above example will output:

```
---
array(3) {
  [0]=>
  array(1) {
    [0]=>
    string(1) "1"
  }
  [1]=>
  array(1) {
    [0]=>
    string(1) "2"
  }
  [2]=>
  array(1) {
    [0]=>
    string(1) "3"
  }
```

```
}
---
array(3) {
  [0]=>
  array(1) {
    [0]=>
    string(1) "2"
  }
  [1]=>
  array(1) {
    [0]=>
    string(1) "3"
  }
  [2]=>
  array(1) {
    [0]=>
    string(1) "4"
  }
}
```

Use of prepared statements

No special handling is required when using the prepared statement interface for fetching results from the same stored procedure as above. The prepared statement and non-prepared statement interfaces are similar. Please note, that not every MYSQL server version may support preparing the CALL SQL statement.

Example #4 Stored Procedures and Prepared Statements

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");
$mysqli->query("INSERT INTO test(id) VALUES (1), (2), (3)");

$mysqli->query("DROP PROCEDURE IF EXISTS p");
$mysqli->query('CREATE PROCEDURE p() READS SQL DATA BEGIN SELECT id FROM test; SELECT id + 1 FROM test;
END;');

$stmt = $mysqli->prepare("CALL p()");

$stmt->execute();

do {
    if ($result = $stmt->get_result()) {
        printf("---\n");
        var_dump($result->fetch_all());
        $result->free();
    }
} while ($stmt->next_result());
```

The above example will output:

```
---
array(3) {
  [0]=>
  array(1) {
    [0]=>
    int(1)
  }
  [1]=>
  array(1) {
    [0]=>
    int(2)
  }
  [2]=>
  array(1) {
    [0]=>
    int(3)
  }
}
---
array(3) {
  [0]=>
  array(1) {
    [0]=>
    int(2)
  }
  [1]=>
  array(1) {
    [0]=>
    int(3)
  }
  [2]=>
  array(1) {
    [0]=>
    int(4)
  }
}
```

Of course, use of the bind API for fetching is supported as well.

Example #5 Stored Procedures and Prepared Statements using bind API

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");
$mysqli->query("INSERT INTO test(id) VALUES (1), (2), (3)");

$mysqli->query("DROP PROCEDURE IF EXISTS p");
$mysqli->query('CREATE PROCEDURE p() READS SQL DATA BEGIN SELECT id FROM test; SELECT id + 1 FROM test;
END;');
```

```php
$stmt = $mysqli->prepare("CALL p()");

$stmt->execute();

do {
    if ($stmt->store_result()) {
        $stmt->bind_result($id_out);
        while ($stmt->fetch()) {
            echo "id = $id_out\n";
        }
    }
} while ($stmt->next_result());
```

The above example will output:

```
id = 1
id = 2
id = 3
id = 2
id = 3
id = 4
```

See also

- mysqli::query()
- mysqli::multi_query()
- mysqli::next_result()
- mysqli::more_results()

Multiple Statements

MySQL optionally allows having multiple statements in one statement string, but it requires special handling.

Multiple statements or multi queries must be executed with mysqli::multi_query(). The individual statements of the statement string are separated by semicolon. Then, all result sets returned by the executed statements must be fetched.

The MySQL server allows having statements that do return result sets and statements that do not return result sets in one multiple statement.

Example #1 Multiple Statements

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$mysqli->query("DROP TABLE IF EXISTS test");
$mysqli->query("CREATE TABLE test(id INT)");

$sql = "SELECT COUNT(*) AS _num FROM test;
        INSERT INTO test(id) VALUES (1);
```

```
        SELECT COUNT(*) AS _num FROM test; ";

$mysqli->multi_query($sql);

do {
    if ($result = $mysqli->store_result()) {
        var_dump($result->fetch_all(MYSQLI_ASSOC));
        $result->free();
    }
} while ($mysqli->next_result());
```

The above example will output:

```
array(1) {
  [0]=>
  array(1) {
    ["_num"]=>
    string(1) "0"
  }
}
array(1) {
  [0]=>
  array(1) {
    ["_num"]=>
    string(1) "1"
  }
}
```

Security considerations

The API functions mysqli::query() and mysqli::real_query() do not set a connection flag necessary for activating multi queries in the server. An extra API call is used for multiple statements to reduce the damage of accidental SQL injection attacks. An attacker may try to add statements such as `; DROP DATABASE mysql` or `; SELECT SLEEP(999)`. If the attacker succeeds in adding SQL to the statement string but mysqli::multi_query() is not used, the server will not execute the injected and malicious SQL statement.

Example #2 SQL Injection

```
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");
$result = $mysqli->query("SELECT 1; DROP TABLE mysql.user");
?>
```

The above example will output:

```
PHP Fatal error:  Uncaught mysqli_sql_exception: You have an error in your SQL syntax;
check the manual that corresponds to your MySQL server version for the right syntax to
use near 'DROP TABLE mysql.user' at line 1
```

Prepared statements

Use of the multiple statement with prepared statements is not supported.

See also

- mysqli::query()
- mysqli::multi_query()
- mysqli::next_result()
- mysqli::more_results()

API support for transactions

The MySQL server supports transactions depending on the storage engine used. Since MySQL 5.5, the default storage engine is InnoDB. InnoDB has full ACID transaction support.

Transactions can either be controlled using SQL or API calls. It is recommended to use API calls for enabling and disabling the `autocommit` mode and for committing and rolling back transactions.

Example #1 Setting `autocommit` mode with SQL and through the API

```php
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

/* Recommended: using API to control transactional settings */
$mysqli->autocommit(false);

/* Won't be monitored and recognized by the replication and the load balancing plugin */
$mysqli->query('SET AUTOCOMMIT = 0');
```

Optional feature packages, such as the replication and load balancing plugin, can easily monitor API calls. The replication plugin offers transaction aware load balancing, if transactions are controlled with API calls. Transaction aware load balancing is not available if SQL statements are used for setting `autocommit` mode, committing or rolling back a transaction.

Example #2 Commit and rollback

```php
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");
$mysqli->autocommit(false);

$mysqli->query("INSERT INTO test(id) VALUES (1)");
$mysqli->rollback();

$mysqli->query("INSERT INTO test(id) VALUES (2)");
$mysqli->commit();
```

Please note, that the MySQL server cannot roll back all statements. Some statements cause an implicit commit.

See also

- mysqli::autocommit()

- mysqli::begin_transaction()
- mysqli::commit()
- mysqli::rollback()

Metadata

A MySQL result set contains metadata. The metadata describes the columns found in the result set. All metadata sent by MySQL is accessible through the `mysqli` interface. The extension performs no or negligible changes to the information it receives. Differences between MySQL server versions are not aligned.

Meta data is access through the mysqli_result interface.

Example #1 Accessing result set meta data

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$result = $mysqli->query("SELECT 1 AS _one, 'Hello' AS _two FROM DUAL");
var_dump($result->fetch_fields());
```

The above example will output:

```
array(2) {
  [0]=>
  object(stdClass)#3 (13) {
    ["name"]=>
    string(4) "_one"
    ["orgname"]=>
    string(0) ""
    ["table"]=>
    string(0) ""
    ["orgtable"]=>
    string(0) ""
    ["def"]=>
    string(0) ""
    ["db"]=>
    string(0) ""
    ["catalog"]=>
    string(3) "def"
    ["max_length"]=>
    int(1)
    ["length"]=>
    int(1)
    ["charsetnr"]=>
    int(63)
    ["flags"]=>
    int(32897)
    ["type"]=>
    int(8)
    ["decimals"]=>
    int(0)
  }
  [1]=>
```

```
  object(stdClass)#4 (13) {
    ["name"]=>
    string(4) "_two"
    ["orgname"]=>
    string(0) ""
    ["table"]=>
    string(0) ""
    ["orgtable"]=>
    string(0) ""
    ["def"]=>
    string(0) ""
    ["db"]=>
    string(0) ""
    ["catalog"]=>
    string(3) "def"
    ["max_length"]=>
    int(5)
    ["length"]=>
    int(5)
    ["charsetnr"]=>
    int(8)
    ["flags"]=>
    int(1)
    ["type"]=>
    int(253)
    ["decimals"]=>
    int(31)
  }
}
```

Prepared statements

Meta data of result sets created using prepared statements are accessed the same way. A suitable mysqli_result handle is returned by mysqli_stmt::result_metadata().

Example #2 Prepared statements metadata

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("example.com", "user", "password", "database");

$stmt = $mysqli->prepare("SELECT 1 AS _one, 'Hello' AS _two FROM DUAL");
$stmt->execute();
$result = $stmt->result_metadata();
var_dump($result->fetch_fields());
```

See also

- mysqli::query()
- mysqli_result::fetch_fields()

Installing/Configuring

Table of Contents

Requirements

In order to have these functions available, you must compile PHP with support for the mysqli extension.

MySQL 8

When running a PHP version before 7.1.16, or PHP 7.2 before 7.2.4, set MySQL 8 Server's default password plugin to *mysql_native_password* or else you will see errors similar to *The server requested authentication method unknown to the client [caching_sha2_password]* even when *caching_sha2_password* is not used.

This is because MySQL 8 defaults to caching_sha2_password, a plugin that is not recognized by the older PHP (mysqlnd) releases. Instead, change it by setting `default_authentication_plugin=mysql_native_password` in *my.cnf*. The *caching_sha2_password* plugin is fully supported as of PHP 7.4.4. For older releases, the mysql_xdevapi extension does support it.

Installation

The `mysqli` extension was introduced with PHP version 5.0.0. The MySQL Native Driver was included in PHP version 5.3.0.

Installation on Linux

The common Unix distributions include binary versions of PHP that can be installed. Although these binary versions are typically built with support for the MySQL extensions, the extension libraries themselves may need to be installed using an additional package. Check the package manager that comes with your chosen distribution for availability.

For example, on Ubuntu the `php5-mysql` package installs the ext/mysql, ext/mysqli, and pdo_mysql PHP extensions. On CentOS, the `php-mysql` package also installs these three PHP extensions.

Alternatively, you can compile this extension yourself. Building PHP from source allows you to specify the MySQL extensions you want to use, as well as your choice of client library for each extension.

The MySQL Native Driver is the recommended client library option, and as of PHP 8.2.0 the only option, as it results in improved performance and gives access to features not available when using the MySQL Client Library. Refer to What is PHP's MySQL Native Driver? for a brief overview of the advantages of MySQL Native Driver.

The `/path/to/mysql_config` represents the location of the `mysql_config` program that comes with MySQL Server.

mysqli compile time support matrix

PHP Version	Default	Configure Options: mysqlnd	Configure Options: `libmysqlclient`	Changelog
8.2.0 and later	mysqlnd	**--with-mysqli**	libmysqlclient is not supported	mysqlnd is the only option
5.4.0 through 8.1.x	mysqlnd	**--with-mysqli**	**--with-mysqli=/path/to/mysql_config**	mysqlnd is the default
5.3.x	libmysqlclient	**--with-mysqli=mysqlnd**	**--with-mysqli=/path/to/mysql_config**	mysqlnd is supported
5.0.x, 5.1.x, 5.2.x	libmysqlclient	Not Available	**--with-mysqli=/path/to/mysql_config**	mysqlnd is not supported

Installation on Windows Systems

On Windows, *php_mysqli.dll* DLL must be enabled in *php.ini*.

As with enabling any PHP extension (such as *php_mysqli.dll*), the PHP directive extension_dir should be set to the directory where the PHP extensions are located. See also the Manual Windows Installation Instructions. An example `extension_dir` value is *c:\php\ext*.

> **Note** :
>
> If when starting the web server an error similar to the following occurs: `"Unable to load dynamic library './php_mysqli.dll'"`, this is because *php_mysqli.dll* cannot be found by the system.

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

MySQLi Configuration Options

Name	Default	Changeable	Changelog
mysqli.allow_local_infile	"0"	INI_SYSTEM	Before PHP 7.2.16 and 7.3.3 the default was "1".
mysqli.local_infile_directory		INI_SYSTEM	Available as of PHP 8.1.0.
mysqli.allow_persistent	"1"	INI_SYSTEM	
mysqli.max_persistent	"-1"	INI_SYSTEM	
mysqli.max_links	"-1"	INI_SYSTEM	
mysqli.default_port	"3306"	INI_ALL	
mysqli.default_socket	NULL	INI_ALL	
mysqli.default_host	NULL	INI_ALL	
mysqli.default_user	NULL	INI_ALL	
mysqli.default_pw	NULL	INI_ALL	
mysqli.reconnect	"0"	INI_SYSTEM	Removed as of PHP 8.2.0
mysqli.rollback_on_cached_plink	"0"	INI_SYSTEM	

For further details and definitions of the preceding INI_* constants, see the chapter on configuration changes.

Here's a short explanation of the configuration directives.

`mysqli.allow_local_infile` bool

Allow accessing, from PHP's perspective, local files with LOAD DATA statements

`mysqli.local_infile_directory` string

Allows restricting LOCAL DATA loading to files located in this designated directory.

`mysqli.allow_persistent` bool

Enable the ability to create persistent connections using mysqli_connect().

`mysqli.max_persistent` int

The maximum number of persistent connections that can be made. Set to 0 for unlimited.

`mysqli.max_links` int

The maximum number of connections per process.

`mysqli.default_port` int

The default TCP port number to use when connecting to the database server if no other port is specified. If no default is specified, the port will be obtained from the `MYSQL_TCP_PORT` environment variable, the `mysql-tcp` entry in */etc/services* or the compile-time `MYSQL_PORT` constant, in that order. Win32 will only use the `MYSQL_PORT` constant.

`mysqli.default_socket` string

The default socket name to use when connecting to a local database server if no other socket name is specified.

`mysqli.default_host` string

The default server host to use when connecting to the database server if no other host is specified.

`mysqli.default_user` string

The default user name to use when connecting to the database server if no other name is specified.

`mysqli.default_pw` string

The default password to use when connecting to the database server if no other password is specified.

`mysqli.reconnect` int

Automatically reconnect if the connection was lost.

> **Note** :
>
> This *php.ini* setting had been ignored by the mysqlnd driver, and was removed as of PHP 8.2.0.

`mysqli.rollback_on_cached_plink` bool

If this option is enabled, closing a persistent connection will rollback any pending transactions of this connection before it is put back into the persistent connection pool. Otherwise, pending transactions will be rolled back only when the connection is reused, or when it is actually closed.

Users cannot set `MYSQL_OPT_READ_TIMEOUT` through an API call or runtime configuration setting. Note that if it were possible there would be differences between how `libmysqlclient` and streams would interpret the value of `MYSQL_OPT_READ_TIMEOUT`.

The mysqli Extension and Persistent Connections

The idea behind persistent connections is that a connection between a client process and a database can be reused by a client process, rather than being created and destroyed multiple times. This reduces the overhead of creating fresh connections every time one is required, as unused connections are cached and ready to be reused.

Unlike the mysql extension, mysqli does not provide a separate function for opening persistent connections. To open a persistent connection you must prepend `p:` to the hostname when connecting.

The problem with persistent connections is that they can be left in unpredictable states by clients. For example, a table lock might be activated before a client terminates unexpectedly. A new client process reusing this persistent connection will get the connection "as is". Any cleanup would need to be done by the new client process before it could make good use of the persistent connection, increasing the burden on the programmer.

The persistent connection of the `mysqli` extension however provides built-in cleanup handling code. The cleanup carried out by `mysqli` includes:

- Rollback active transactions
- Close and drop temporary tables
- Unlock tables
- Reset session variables
- Close prepared statements (always happens with PHP)
- Close handler
- Release locks acquired with **GET_LOCK()**

This ensures that persistent connections are in a clean state on return from the connection pool, before the client process uses them.

The `mysqli` extension does this cleanup by automatically calling the C-API function `mysql_change_user()`.

The automatic cleanup feature has advantages and disadvantages though. The advantage is that the programmer no longer needs to worry about adding cleanup code, as it is called automatically. However, the disadvantage is that the code could *potentially* be a little slower, as the code to perform the cleanup needs to run each time a connection is returned from the connection pool.

It is possible to switch off the automatic cleanup code, by compiling PHP with **`MYSQLI_NO_CHANGE_USER_ON_PCONNECT`** defined.

> **Note** :
>
> The `mysqli` extension supports persistent connections when using either MySQL Native Driver or MySQL Client Library.

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

`MYSQLI_READ_DEFAULT_GROUP` (int)

Read options from the named group from *my.cnf* or the file specified with **`MYSQLI_READ_DEFAULT_FILE`**.

`MYSQLI_READ_DEFAULT_FILE` (int)

Read options from the named option file instead of from *my.cnf*.

`MYSQLI_OPT_CAN_HANDLE_EXPIRED_PASSWORDS` (int)

Indicates to the server that the client can handle sandbox mode for expired passwords. Can be used with mysqli_options().

MYSQLI_OPT_LOAD_DATA_LOCAL_DIR (int)

If enabled, this option specifies the directory from which client-side `LOCAL` data loading is permitted in `LOAD DATA LOCAL` statements.

MYSQLI_OPT_CONNECT_TIMEOUT (int)

Connect timeout in seconds.

MYSQLI_OPT_READ_TIMEOUT (int)

Command execution result timeout in seconds. Available as of PHP 7.2.0.

MYSQLI_OPT_LOCAL_INFILE (int)

Enables command `LOAD LOCAL INFILE`.

MYSQLI_OPT_INT_AND_FLOAT_NATIVE (int)

Convert integer and float columns back to PHP numbers. Only valid for mysqlnd.

MYSQLI_OPT_NET_CMD_BUFFER_SIZE (int)

The size of the internal command/network buffer. Only valid for mysqlnd.

MYSQLI_OPT_NET_READ_BUFFER_SIZE (int)

Maximum read chunk size in bytes when reading the body of a MySQL command packet. Only valid for mysqlnd.

MYSQLI_OPT_SSL_VERIFY_SERVER_CERT (int)

MYSQLI_INIT_COMMAND (int)

Command to execute when connecting to MySQL server. Will automatically be re-executed when reconnecting.

MYSQLI_CLIENT_CAN_HANDLE_EXPIRED_PASSWORDS (int)

Indicates to the server that the client can handle sandbox mode for expired passwords. Can be used with mysqli_real_connect().

MYSQLI_CLIENT_FOUND_ROWS (int)

Return number of matched rows, not the number of affected rows.

MYSQLI_CLIENT_SSL_VERIFY_SERVER_CERT (int)

Verify server certificate.

MYSQLI_CLIENT_SSL (int)

Use SSL (encrypted protocol). This option should not be set by application programs; it is set internally in the MySQL client library.

MYSQLI_CLIENT_COMPRESS (int)

Use compression protocol.

MYSQLI_CLIENT_INTERACTIVE (int)

Allow `interactive_timeout` seconds (instead of `wait_timeout` seconds) of inactivity before closing the connection. The client's session `wait_timeout` variable will be set to the value of the session `interactive_timeout` variable.

MYSQLI_CLIENT_IGNORE_SPACE (int)

Allow spaces after function names. Makes all functions names reserved words.

MYSQLI_CLIENT_NO_SCHEMA (int)

Don't allow the `db_name.tbl_name.col_name` syntax.

MYSQLI_CLIENT_MULTI_QUERIES

Allows multiple semicolon-delimited queries in a single mysqli_query() call.

MYSQLI_STORE_RESULT (int)

For using buffered result sets. It has a value of `0`.

MYSQLI_STORE_RESULT_COPY_DATA (int)

As of PHP 8.1, this constant no longer has any effect. Before PHP 8.1, this constant is used to copy results from the internal `mysqlnd` buffer into the PHP variables fetched. By default, `mysqlnd` will use a reference logic to avoid copying and duplicating results held in memory. For certain result sets, for example, result sets with many small rows, the copy approach can reduce the overall memory usage because PHP variables holding results may be released earlier. Available with `mysqlnd` only. Deprecated as of PHP 8.4.0.

MYSQLI_USE_RESULT (int)

For using unbuffered result sets. It has a value of `1`.

MYSQLI_ASSOC (int)

Columns are returned into the array having the fieldname as the array index.

MYSQLI_NUM (int)

Columns are returned into the array having an enumerated index.

MYSQLI_BOTH (int)

Columns are returned into the array having both a numerical index and the fieldname as the associative index.

MYSQLI_NOT_NULL_FLAG (int)

Indicates that a field is defined as `NOT NULL`.

MYSQLI_PRI_KEY_FLAG (int)

Field is part of a primary index.

MYSQLI_UNIQUE_KEY_FLAG (int)

Field is part of a unique index.

MYSQLI_MULTIPLE_KEY_FLAG (int)

Field is part of an index.

MYSQLI_BLOB_FLAG (int)

Field is defined as `BLOB`.

MYSQLI_UNSIGNED_FLAG (int)

Field is defined as `UNSIGNED`.

MYSQLI_ZEROFILL_FLAG (int)

Field is defined as `ZEROFILL`.

MYSQLI_AUTO_INCREMENT_FLAG (int)

Field is defined as `AUTO_INCREMENT`.

MYSQLI_TIMESTAMP_FLAG (int)

Field is defined as `TIMESTAMP`.

MYSQLI_SET_FLAG (int)

Field is defined as `SET`.

MYSQLI_NUM_FLAG (int)

Field is defined as `NUMERIC`.

MYSQLI_PART_KEY_FLAG (int)

Field is part of a multi-index.

MYSQLI_GROUP_FLAG (int)

Field is part of `GROUP BY`.

MYSQLI_NO_DEFAULT_VALUE_FLAG (int)

A column has no `DEFAULT` clause in its definition. This does not apply to `NULL` or to `AUTO_INCREMENT` columns because such columns have a default value of `NULL` and an implied default value respectively.

MYSQLI_TYPE_DECIMAL (int)

Field is defined as `DECIMAL`.

MYSQLI_TYPE_NEWDECIMAL (int)

Precision math `DECIMAL` or `NUMERIC` field.

MYSQLI_TYPE_BIT (int)

Field is defined as `BIT`.

MYSQLI_TYPE_TINY (int)

Field is defined as `TINYINT`.

MYSQLI_TYPE_SHORT (int)

Field is defined as `SMALLINT`.

MYSQLI_TYPE_LONG (int)

Field is defined as `INT`.

MYSQLI_TYPE_FLOAT (int)

Field is defined as `FLOAT`.

MYSQLI_TYPE_DOUBLE (int)

Field is defined as `DOUBLE`.

MYSQLI_TYPE_NULL (int)

Field is defined as `DEFAULT NULL`.

MYSQLI_TYPE_TIMESTAMP (int)

Field is defined as `TIMESTAMP`.

MYSQLI_TYPE_LONGLONG (int)

Field is defined as `BIGINT`.

MYSQLI_TYPE_INT24 (int)

Field is defined as `MEDIUMINT`.

MYSQLI_TYPE_DATE (int)

Field is defined as `DATE`.

MYSQLI_TYPE_TIME (int)

Field is defined as `TIME`.

MYSQLI_TYPE_DATETIME (int)

Field is defined as `DATETIME`.

MYSQLI_TYPE_YEAR (int)

Field is defined as `YEAR`.

MYSQLI_TYPE_NEWDATE (int)

Field is defined as `DATE`.

MYSQLI_TYPE_INTERVAL (int)

Alias of **MYSQLI_TYPE_ENUM**. Removed as of PHP 8.4.0.

MYSQLI_TYPE_ENUM (int)

Field is defined as ENUM.

MYSQLI_TYPE_SET (int)

Field is defined as SET.

MYSQLI_TYPE_TINY_BLOB (int)

Field is defined as TINYBLOB.

MYSQLI_TYPE_MEDIUM_BLOB (int)

Field is defined as MEDIUMBLOB.

MYSQLI_TYPE_LONG_BLOB (int)

Field is defined as LONGBLOB.

MYSQLI_TYPE_BLOB (int)

Field is defined as BLOB.

MYSQLI_TYPE_VAR_STRING (int)

Field is defined as VARCHAR.

MYSQLI_TYPE_STRING (int)

Field is defined as CHAR or BINARY.

MYSQLI_TYPE_CHAR (int)

Field is defined as TINYINT. For CHAR, see MYSQLI_TYPE_STRING.

MYSQLI_TYPE_GEOMETRY (int)

Field is defined as GEOMETRY.

MYSQLI_TYPE_JSON (int)

Field is defined as JSON. Only valid for mysqlnd and MySQL 5.7.8 and up.

MYSQLI_TYPE_VECTOR (int)

Field is defined as VECTOR. Available as of PHP 8.4.0, for MySQL 9.0 and later.

MYSQLI_NEED_DATA

More data available for bind variable.

MYSQLI_ENUM_FLAG (int)

Field is defined as ENUM.

MYSQLI_BINARY_FLAG (int)

Field is defined as BINARY.

MYSQLI_CURSOR_TYPE_FOR_UPDATE (int)

Removed as of PHP 8.4.0.

MYSQLI_CURSOR_TYPE_NO_CURSOR (int)

MYSQLI_CURSOR_TYPE_READ_ONLY (int)

MYSQLI_CURSOR_TYPE_SCROLLABLE (int)

Removed as of PHP 8.4.0.

MYSQLI_STMT_ATTR_CURSOR_TYPE (int)

MYSQLI_STMT_ATTR_PREFETCH_ROWS (int)

Removed as of PHP 8.4.0.

MYSQLI_STMT_ATTR_UPDATE_MAX_LENGTH (int)

MYSQLI_SET_CHARSET_NAME (int)

MYSQLI_REPORT_INDEX (int)

Report if no index or bad index was used in a query.

MYSQLI_REPORT_ERROR (int)

Report errors from mysqli function calls.

MYSQLI_REPORT_STRICT (int)

Throw a `mysqli_sql_exception` for errors instead of warnings.

MYSQLI_REPORT_ALL (int)

Set all options on (report all).

MYSQLI_REPORT_OFF (int)

Turns reporting off.

MYSQLI_DEBUG_TRACE_ENABLED (int)

Is set to 1 if mysqli_debug() functionality is enabled.

MYSQLI_SERVER_PUBLIC_KEY (int)

MYSQLI_REFRESH_GRANT (int)

Refreshes the grant tables. Deprecated as of PHP 8.4.0.

MYSQLI_REFRESH_LOG (int)

Flushes the logs, like executing the `FLUSH LOGS` SQL statement. Deprecated as of PHP 8.4.0.

MYSQLI_REFRESH_TABLES (int)

Flushes the table cache, like executing the `FLUSH TABLES` SQL statement. Deprecated as of PHP 8.4.0.

MYSQLI_REFRESH_HOSTS (int)

Flushes the host cache, like executing the `FLUSH HOSTS` SQL statement. Deprecated as of PHP 8.4.0.

MYSQLI_REFRESH_REPLICA (int)

Alias of **MYSQLI_REFRESH_SLAVE** constant. Available as of PHP 8.1.0. Deprecated as of PHP 8.4.0.

MYSQLI_REFRESH_STATUS (int)

Reset the status variables, like executing the `FLUSH STATUS` SQL statement. Deprecated as of PHP 8.4.0.

MYSQLI_REFRESH_THREADS (int)

Flushes the thread cache. Deprecated as of PHP 8.4.0.

MYSQLI_REFRESH_SLAVE (int)

On a slave replication server: resets the master server information, and restarts the slave. Like executing the `RESET SLAVE` SQL statement. Deprecated as of PHP 8.4.0.

MYSQLI_REFRESH_MASTER (int)

On a master replication server: removes the binary log files listed in the binary log index, and truncates the index file. Like executing the `RESET MASTER` SQL statement. Deprecated as of PHP 8.4.0.

MYSQLI_REFRESH_BACKUP_LOG (int)

Closes and reopens the backup log files. Deprecated as of PHP 8.4.0.

MYSQLI_TRANS_COR_AND_CHAIN (int)

Appends "AND CHAIN" to mysqli_commit() or mysqli_rollback().

MYSQLI_TRANS_COR_AND_NO_CHAIN (int)

Appends "AND NO CHAIN" to mysqli_commit() or mysqli_rollback().

MYSQLI_TRANS_COR_RELEASE (int)

Appends "RELEASE" to mysqli_commit() or mysqli_rollback().

MYSQLI_TRANS_COR_NO_RELEASE (int)

Appends "NO RELEASE" to mysqli_commit() or mysqli_rollback().

MYSQLI_TRANS_START_READ_ONLY (int)

Start the transaction as "START TRANSACTION READ ONLY" with mysqli_begin_transaction().

MYSQLI_TRANS_START_READ_WRITE (int)

Start the transaction as "START TRANSACTION READ WRITE" with mysqli_begin_transaction().

MYSQLI_TRANS_START_WITH_CONSISTENT_SNAPSHOT (int)

Start the transaction as "START TRANSACTION WITH CONSISTENT SNAPSHOT".

MYSQLI_CLIENT_SSL_DONT_VERIFY_SERVER_CERT (int)

Requires MySQL 5.6.5 and up.

`MYSQLI_IS_MARIADB` (bool)

Whether the mysqli extension has been built against a MariaDB client library. Available as of PHP 8.1.2.

`MYSQLI_ASYNC` (int)

The query is performed asynchronously and no result set is immediately returned. Available with `mysqlnd` only.

`MYSQLI_ON_UPDATE_NOW_FLAG` (int)

If a field is updated it will get the current time value.

mysqli

(PHP 5, PHP 7, PHP 8)

Introduction

Represents a connection between PHP and a MySQL database.

Class synopsis

```
class mysqli {
/* Properties */
public readonly int|string $affected_rows;
public readonly string $client_info;
public readonly int $client_version;
public readonly int $connect_errno;
public readonly ?string $connect_error;
public readonly int $errno;
public readonly string $error;
public readonly array $error_list;
public readonly int $field_count;
public readonly string $host_info;
public readonly ?string $info;
public readonly int|string $insert_id;
public readonly string $server_info;
public readonly int $server_version;
public readonly string $sqlstate;
public readonly int $protocol_version;
public readonly int $thread_id;
public readonly int $warning_count;
/* Methods */
public function __construct(
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null
)
```

```
public function autocommit(bool $enable): bool
public function begin_transaction(int $flags = 0, ?string $name = null): bool
public function change_user(string $username, #[\SensitiveParameter] string $password, ?string $database): bool
public function character_set_name(): string
public function close(): true
public function commit(int $flags = 0, ?string $name = null): bool
public function connect(
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null
): bool
public function debug(string $options): true
public function dump_debug_info(): bool
public function execute_query(string $query, ?array $params = null): mysqli_result|bool
public function get_charset(): ?object
#[\Deprecated(
    since: '8.1',
    message: 'use mysqli_get_client_info() instead',
)]
public function get_client_info(): string
public function get_connection_stats(): array
public function get_server_info(): string
public function get_warnings(): mysqli_warning|false
#[\Deprecated(
    since: '8.1',
    message: 'replace calls to parent::init() with parent::__construct()',
)]
public function init(): ?bool
#[\Deprecated(
    since: '8.4',
    message: 'use KILL CONNECTION/QUERY SQL statement instead',
)]
public function kill(int $process_id): bool
public function more_results(): bool
public function multi_query(string $query): bool
public function next_result(): bool
public function options(int $option, string|int $value): bool
#[\Deprecated(
    since: '8.4',
    message: 'because the reconnect feature has been removed in PHP 8.2 and this method is now redundant',
)]
public function ping(): bool
public static function poll(
    ?array &$read,
    ?array &$error,
    array &$reject,
    int $seconds,
```

```
    int $microseconds = 0
): int|false
public function prepare(string $query): mysqli_stmt|false
public function query(string $query, int $result_mode = MYSQLI_STORE_RESULT): mysqli_result|bool
public function real_connect(
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null,
    int $flags = 0
): bool
public function real_escape_string(string $string): string
public function real_query(string $query): bool
public function reap_async_query(): mysqli_result|bool
#[\Deprecated(
    since: '8.4',
    message: 'use FLUSH SQL statement instead',
)]
public function refresh(int $flags): bool
public function release_savepoint(string $name): bool
public function rollback(int $flags = 0, ?string $name = null): bool
public function savepoint(string $name): bool
public function select_db(string $database): bool
public function set_charset(string $charset): bool
public function ssl_set(
    ?string $key,
    ?string $certificate,
    ?string $ca_certificate,
    ?string $ca_path,
    ?string $cipher_algos
): true
public function stat(): string|false
public function stmt_init(): mysqli_stmt|false
public function store_result(int $mode = 0): mysqli_result|false
public function thread_safe(): bool
public function use_result(): mysqli_result|false
}
```

Table of Contents

mysqli::$affected_rows

mysqli::$affected_rows

mysqli_affected_rows

(PHP 5, PHP 7, PHP 8)

mysqli::$affected_rows -- mysqli_affected_rows — Gets the number of affected rows in a previous MySQL operation

Description

Object-oriented style

int|string *$mysqli->affected_rows*;

Procedural style

function **mysqli_affected_rows**(mysqli `$mysql`): int|string

Returns the number of rows affected by the last `INSERT`, `UPDATE`, `REPLACE` or `DELETE` query. Works like mysqli_num_rows() for `SELECT` statements.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

An integer greater than zero indicates the number of rows affected or retrieved. Zero indicates that no records were updated for an `UPDATE` statement, no rows matched the `WHERE` clause in the query or that no query has yet been executed. `-1` indicates that the query returned an error or that **mysqli_affected_rows()** was called for an unbuffered `SELECT` query.

> **Note** :
>
> If the number of affected rows is greater than the maximum int value (`PHP_INT_MAX`), the number of affected rows will be returned as a string.

Examples

Example #1 *$mysqli->affected_rows* example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* Insert rows */
$mysqli->query("CREATE TABLE Language SELECT * from CountryLanguage");
printf("Affected rows (INSERT): %d\n", $mysqli->affected_rows);

$mysqli->query("ALTER TABLE Language ADD Status int default 0");

/* update rows */
$mysqli->query("UPDATE Language SET Status=1 WHERE Percentage > 50");
printf("Affected rows (UPDATE): %d\n", $mysqli->affected_rows);

/* delete rows */
$mysqli->query("DELETE FROM Language WHERE Percentage < 50");
printf("Affected rows (DELETE): %d\n", $mysqli->affected_rows);
```

```php
/* select all rows */
$result = $mysqli->query("SELECT CountryCode FROM Language");
printf("Affected rows (SELECT): %d\n", $mysqli->affected_rows);

/* Delete table Language */
$mysqli->query("DROP TABLE Language");
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* Insert rows */
mysqli_query($link, "CREATE TABLE Language SELECT * from CountryLanguage");
printf("Affected rows (INSERT): %d\n", mysqli_affected_rows($link));

mysqli_query($link, "ALTER TABLE Language ADD Status int default 0");

/* update rows */
mysqli_query($link, "UPDATE Language SET Status=1 WHERE Percentage > 50");
printf("Affected rows (UPDATE): %d\n", mysqli_affected_rows($link));

/* delete rows */
mysqli_query($link, "DELETE FROM Language WHERE Percentage < 50");
printf("Affected rows (DELETE): %d\n", mysqli_affected_rows($link));

/* select all rows */
$result = mysqli_query($link, "SELECT CountryCode FROM Language");
printf("Affected rows (SELECT): %d\n", mysqli_affected_rows($link));

/* Delete table Language */
mysqli_query($link, "DROP TABLE Language");
```

The above examples will output:

```
Affected rows (INSERT): 984
Affected rows (UPDATE): 168
Affected rows (DELETE): 815
Affected rows (SELECT): 169
```

See Also

- mysqli_num_rows() - Gets the number of rows in the result set
- mysqli_info() - Retrieves information about the most recently executed query

mysqli::autocommit

mysqli::autocommit

mysqli_autocommit

(PHP 5, PHP 7, PHP 8)

mysqli::autocommit -- mysqli_autocommit — Turns on or off auto-committing database modifications

Description

Object-oriented style

public function **mysqli::autocommit**(bool `$enable`): bool

Procedural style

function **mysqli_autocommit**(mysqli `$mysql`, bool `$enable`): bool

Turns on or off auto-commit mode on queries for the database connection.

To determine the current state of autocommit use the SQL command `SELECT @@autocommit`.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`enable`

Whether to turn on auto-commit or not.

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

Examples

Example #1 mysqli::autocommit() example

Object-oriented style

```
<?php

/* Tell mysqli to throw an exception if an error occurs */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* The table engine has to support transactions */
$mysqli->query("CREATE TABLE IF NOT EXISTS language (
    Code text NOT NULL,
    Speakers int(11) NOT NULL
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;");
```

```php
/* Turn autocommit off */
$mysqli->autocommit(false);

$result = $mysqli->query("SELECT @@autocommit");
$row = $result->fetch_row();
printf("Autocommit is %s\n", $row[0]);

try {
    /* Prepare insert statement */
    $stmt = $mysqli->prepare('INSERT INTO language(Code, Speakers) VALUES (?,?)');
    $stmt->bind_param('ss', $language_code, $native_speakers);

    /* Insert some values */
    $language_code = 'DE';
    $native_speakers = 50_123_456;
    $stmt->execute();
    $language_code = 'FR';
    $native_speakers = 40_546_321;
    $stmt->execute();

    /* Commit the data in the database. This doesn't set autocommit=true */
    $mysqli->commit();
    print "Committed 2 rows in the database\n";

    $result = $mysqli->query("SELECT @@autocommit");
    $row = $result->fetch_row();
    printf("Autocommit is %s\n", $row[0]);

    /* Try to insert more values */
    $language_code = 'PL';
    $native_speakers = 30_555_444;
    $stmt->execute();
    $language_code = 'DK';
    $native_speakers = 5_222_444;
    $stmt->execute();

    /* Setting autocommit=true will trigger a commit */
    $mysqli->autocommit(true);

    print "Committed 2 row in the database\n";
} catch (mysqli_sql_exception $exception) {
    $mysqli->rollback();

    throw $exception;
}
```

Procedural style

```php
<?php

/* Tell mysqli to throw an exception if an error occurs */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

/* The table engine has to support transactions */
mysqli_query($mysqli, "CREATE TABLE IF NOT EXISTS language (
    Code text NOT NULL,
    Speakers int(11) NOT NULL
```

```
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;");

/* Turn autocommit off */
mysqli_autocommit($mysqli, false);

$result = mysqli_query($mysqli, "SELECT @@autocommit");
$row = mysqli_fetch_row($result);
printf("Autocommit is %s\n", $row[0]);

try {
    /* Prepare insert statement */
    $stmt = mysqli_prepare($mysqli, 'INSERT INTO language(Code, Speakers) VALUES (?,?)');
    mysqli_stmt_bind_param($stmt, 'ss', $language_code, $native_speakers);

    /* Insert some values */
    $language_code = 'DE';
    $native_speakers = 50_123_456;
    mysqli_stmt_execute($stmt);
    $language_code = 'FR';
    $native_speakers = 40_546_321;
    mysqli_stmt_execute($stmt);

    /* Commit the data in the database. This doesn't set autocommit=true */
    mysqli_commit($mysqli);
    print "Committed 2 rows in the database\n";

    $result = mysqli_query($mysqli, "SELECT @@autocommit");
    $row = mysqli_fetch_row($result);
    printf("Autocommit is %s\n", $row[0]);

    /* Try to insert more values */
    $language_code = 'PL';
    $native_speakers = 30_555_444;
    mysqli_stmt_execute($stmt);
    $language_code = 'DK';
    $native_speakers = 5_222_444;
    mysqli_stmt_execute($stmt);

    /* Setting autocommit=true will trigger a commit */
    mysqli_autocommit($mysqli, true);

    print "Committed 2 row in the database\n";
} catch (mysqli_sql_exception $exception) {
    mysqli_rollback($mysqli);

    throw $exception;
}
```

The above examples will output:

```
Autocommit is 0
Committed 2 rows in the database
Autocommit is 0
Committed 2 row in the database
Autocommit is 0
Committed 2 rows in the database
```

```
Autocommit is 0
Committed 2 row in the database
```

Notes

> **Note** :
>
> This function does not work with non transactional table types (like MyISAM or ISAM).

See Also

- mysqli_begin_transaction() - Starts a transaction
- mysqli_commit() - Commits the current transaction
- mysqli_rollback() - Rolls back current transaction

mysqli::begin_transaction

mysqli::begin_transaction

mysqli_begin_transaction

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

mysqli::begin_transaction -- mysqli_begin_transaction — Starts a transaction

Description

Object-oriented style

public function **mysqli::begin_transaction**(int `$flags` = 0, ?string `$name` = **`null`**): bool

Procedural style:

function **mysqli_begin_transaction**(mysqli `$mysql`, int `$flags` = 0, ?string `$name` = **`null`**): bool

Begins a transaction. Requires the InnoDB engine (it is enabled by default). For additional details about how MySQL transactions work, see » http://dev.mysql.com/doc/mysql/en/commit.html.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`flags`

Valid flags are:

- **`MYSQLI_TRANS_START_READ_ONLY`**: Start the transaction as "START TRANSACTION READ ONLY". Requires MySQL 5.6 and above.
- **`MYSQLI_TRANS_START_READ_WRITE`**: Start the transaction as "START TRANSACTION READ WRITE". Requires MySQL 5.6 and above.

- `MYSQLI_TRANS_START_WITH_CONSISTENT_SNAPSHOT`: Start the transaction as "START TRANSACTION WITH CONSISTENT SNAPSHOT".

`name`

Savepoint name for the transaction.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.0.0	`name` is now nullable.

Examples

Example #1 **mysqli::begin_transaction()** example

Object-oriented style

```
<?php

/* Tell mysqli to throw an exception if an error occurs */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* The table engine has to support transactions */
$mysqli->query("CREATE TABLE IF NOT EXISTS language (
    Code text NOT NULL,
    Speakers int(11) NOT NULL
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;");

/* Start transaction */
$mysqli->begin_transaction();

try {
    /* Insert some values */
    $mysqli->query("INSERT INTO language(Code, Speakers) VALUES ('DE', 42000123)");

    /* Try to insert invalid values */
    $language_code = 'FR';
    $native_speakers = 'Unknown';
    $stmt = $mysqli->prepare('INSERT INTO language(Code, Speakers) VALUES (?,?)');
    $stmt->bind_param('ss', $language_code, $native_speakers);
    $stmt->execute();

    /* If code reaches this point without errors then commit the data in the database */
    $mysqli->commit();
} catch (mysqli_sql_exception $exception) {
    $mysqli->rollback();

    throw $exception;
}
```

Procedural style

```php
<?php

/* Tell mysqli to throw an exception if an error occurs */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

/* The table engine has to support transactions */
mysqli_query($mysqli, "CREATE TABLE IF NOT EXISTS language (
    Code text NOT NULL,
    Speakers int(11) NOT NULL
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;");

/* Start transaction */
mysqli_begin_transaction($mysqli);

try {
    /* Insert some values */
    mysqli_query($mysqli, "INSERT INTO language(Code, Speakers) VALUES ('DE', 42000123)");

    /* Try to insert invalid values */
    $language_code = 'FR';
    $native_speakers = 'Unknown';
    $stmt = mysqli_prepare($mysqli, 'INSERT INTO language(Code, Speakers) VALUES (?,?)');
    mysqli_stmt_bind_param($stmt, 'ss', $language_code, $native_speakers);
    mysqli_stmt_execute($stmt);

    /* If code reaches this point without errors then commit the data in the database */
    mysqli_commit($mysqli);
} catch (mysqli_sql_exception $exception) {
    mysqli_rollback($mysqli);

    throw $exception;
}
```

Notes

> **Note** :
>
> This function does not work with non transactional table types (like MyISAM or ISAM).

See Also

- mysqli_autocommit() - Turns on or off auto-committing database modifications
- mysqli_commit() - Commits the current transaction
- mysqli_rollback() - Rolls back current transaction

mysqli::change_user

mysqli::change_user

mysqli_change_user

(PHP 5, PHP 7, PHP 8)

mysqli::change_user -- mysqli_change_user — Changes the user of the database connection

Description

Object-oriented style

public function **mysqli::change_user**(string `$username`, #[\SensitiveParameter] string `$password`, ?string `$database`): bool

Procedural style

function **mysqli_change_user**(
  mysqli `$mysql`,
  string `$username`,
  #[\SensitiveParameter] string `$password`,
  ?string `$database`
): bool

Attempts to connect to the specified database using the provided credentials.

In comparison to mysqli::connect(), this method will not disconnect the current connection if the new connection cannot be opened.

In order to successfully change users, a valid `username` and `password` must be provided, and that user must have sufficient permissions to access the desired database. If for any reason authorization fails, the current user authentication will remain.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`username`

The MySQL user name.

`password`

The MySQL password.

`database`

The database name. If **`null`** or an empty string is passed, the connection to the server will be opened with no default database.

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

Examples

Example #1 Resetting the connection session

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "test");

$mysqli->query("SET @a:=1");

$mysqli->change_user("my_user", "my_password", "world");

$result = $mysqli->query("SELECT DATABASE()");
$row = $result->fetch_row();
printf("Default database: %s\n", $row[0]);

$result = $mysqli->query("SELECT @a");
$row = $result->fetch_row();
if ($row[0] === null) {
    printf("Value of variable a is NULL\n");
}
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "test");

mysqli_query($link, "SET @a:=1");

mysqli_change_user($link, "my_user", "my_password", "world");

$result = mysqli_query($link, "SELECT DATABASE()");
$row = mysqli_fetch_row($result);
printf("Default database: %s\n", $row[0]);

$result = mysqli_query($link, "SELECT @a");
$row = mysqli_fetch_row($result);
if ($row[0] === null) {
    printf("Value of variable a is NULL\n");
}
```

The above examples will output:

```
Default database: world
Value of variable a is NULL
```

Example #2 If `database` is `null` the connection is opened without selecting any default database

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "test");

$mysqli->change_user("my_user", "my_password", null);

$result = $mysqli->query("SELECT DATABASE()");
$row = $result->fetch_row();
printf("Default database: %s\n", $row[0]);
```

The above examples will output:

```
Default database:
```

Notes

> **Note** :
>
> Using this command will always cause the current database connection to behave as if it was a completely new database connection, regardless of if the operation was completed successfully. This reset includes performing a rollback on any active transactions, closing all temporary tables, and unlocking all locked tables.

See Also

- mysqli_connect() - Open a new connection to the MySQL server
- mysqli_select_db() - Selects the default database for database queries

mysqli::character_set_name

mysqli::character_set_name

mysqli_character_set_name

(PHP 5, PHP 7, PHP 8)

mysqli::character_set_name -- mysqli_character_set_name — Returns the current character set of the database connection

Description

Object-oriented style

public function **mysqli::character_set_name**(): string

Procedural style

function **mysqli_character_set_name**(mysqli `$mysql`): string

Returns the current character set of the database connection.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

The current character set of the connection

Examples

Example #1 mysqli::character_set_name() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* Set the default character set */
$mysqli->set_charset('utf8mb4');

/* Print current character set */
$charset = $mysqli->character_set_name();
printf("Current character set is %s\n", $charset);
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

/* Set the default character set */
mysqli_set_charset($mysqli, 'utf8mb4');

/* Print current character set */
$charset = mysqli_character_set_name($mysqli);
printf("Current character set is %s\n", $charset);
```

The above examples will output:

```
Current character set is utf8mb4
```

See Also

- mysqli_set_charset() - Sets the client character set

- mysqli_real_escape_string() - Escapes special characters in a string for use in an SQL statement, taking into account the current charset of the connection

mysqli::close

mysqli::close

mysqli_close

(PHP 5, PHP 7, PHP 8)

mysqli::close -- mysqli_close — Closes a previously opened database connection

Description

Object-oriented style

public function **mysqli::close**(): true

Procedural style

function **mysqli_close**(mysqli `$mysql`): true

Closes a previously opened database connection.

Open non-persistent MySQL connections and result sets are automatically closed when their objects are destroyed. Explicitly closing open connections and freeing result sets is optional. However, it's a good idea to close the connection as soon as the script finishes performing all of its database operations, if it still has a lot of processing to do after getting the results.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Always returns `true`.

Changelog

Version	Description
8.0.0	This function now always returns `true`. Previously it returned `false` on failure.

Examples

Example #1 mysqli::close() example

Object-oriented style

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$result = $mysqli->query("SELECT Name, CountryCode FROM City ORDER BY ID LIMIT 3");
```

```
/* Close the connection as soon as it's no longer needed */
$mysqli->close();

foreach ($result as $row) {
    /* Processing of the data retrieved from the database */
}
```

Procedural style

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$result = mysqli_query($mysqli, "SELECT Name, CountryCode FROM City ORDER BY ID LIMIT 3");

/* Close the connection as soon as it's no longer needed */
mysqli_close($mysqli);

foreach ($result as $row) {
    /* Processing of the data retrieved from the database */
}
```

Notes

> **Note** :
>
> **mysqli_close()** will not close persistent connections. For additional details, see the manual page on persistent connections.

See Also

- mysqli::__construct() - Open a new connection to the MySQL server
- mysqli_init() - Initializes MySQLi and returns an object for use with mysqli_real_connect()
- mysqli_real_connect() - Opens a connection to the MySQL server
- mysqli_free_result() - Frees the memory associated with a result

mysqli::commit

mysqli::commit

mysqli_commit

(PHP 5, PHP 7, PHP 8)

mysqli::commit -- mysqli_commit — Commits the current transaction

Description

Object-oriented style

public function **mysqli::commit**(int `$flags` = 0, ?string `$name` = `null`): bool

Procedural style

function **mysqli_commit**(mysqli `$mysql`, int `$flags` = 0, ?string `$name` = `null`): bool

Commits the current transaction for the database connection.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`flags`

A bitmask of `MYSQLI_TRANS_COR_*` constants.

`name`

If provided then `COMMIT/*name*/` is executed.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

Changelog

Version	Description
8.0.0	`name` is now nullable.

Examples

See the mysqli::begin_transaction() example.

Notes

> **Note** :
>
> This function does not work with non transactional table types (like MyISAM or ISAM).

See Also

- mysqli_autocommit() - Turns on or off auto-committing database modifications
- mysqli_begin_transaction() - Starts a transaction
- mysqli_rollback() - Rolls back current transaction
- mysqli_savepoint() - Set a named transaction savepoint

mysqli::$connect_errno

mysqli::$connect_errno

mysqli_connect_errno

(PHP 5, PHP 7, PHP 8)

mysqli::$connect_errno -- mysqli_connect_errno — Returns the error code from last connect call

Description

Object-oriented style

int *$mysqli->connect_errno*;

Procedural style

function **mysqli_connect_errno**(): int

Returns the error code from the last connection attempt.

Parameters

This function has no parameters.

Return Values

An error code for the last connection attempt, if it failed. Zero means no error occurred.

Returns the last connection error code regardless of the instance on which it is called.

Examples

Example #1 *$mysqli->connect_errno* example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_OFF);
/* @ is used to suppress warnings */
$mysqli = @new mysqli('localhost', 'fake_user', 'wrong_password', 'does_not_exist');
if ($mysqli->connect_errno) {
    /* Use your preferred error logging method here */
    error_log('Connection error: ' . $mysqli->connect_errno);
}
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_OFF);
/* @ is used to suppress warnings */
$link = @mysqli_connect('localhost', 'fake_user', 'wrong_password', 'does_not_exist');
if (!$link) {
    /* Use your preferred error logging method here */
    error_log('Connection error: ' . mysqli_connect_errno());
}
```

See Also

- mysqli_connect() - Open a new connection to the MySQL server
- mysqli_connect_error() - Returns a description of the last connection error

- mysqli_errno() - Returns the error code for the most recent function call
- mysqli_error() - Returns a string description of the last error
- mysqli_sqlstate() - Returns the SQLSTATE error from previous MySQL operation

mysqli::$connect_error

mysqli::$connect_error

mysqli_connect_error

(PHP 5, PHP 7, PHP 8)

mysqli::$connect_error -- mysqli_connect_error — Returns a description of the last connection error

Description

Object-oriented style

?string *$mysqli->connect_error*;

Procedural style

function **mysqli_connect_error**(): ?string

Returns the error message from the last connection attempt.

Parameters

This function has no parameters.

Return Values

A string that describes the error. `null` is returned if no error occurred.

Returns the last connection error regardless of the instance on which it is called.

Examples

Example #1 *$mysqli->connect_error* example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_OFF);
/* @ is used to suppress warnings */
$mysqli = @new mysqli('localhost', 'fake_user', 'wrong_password', 'does_not_exist');
if ($mysqli->connect_error) {
    /* Use your preferred error logging method here */
    error_log('Connection error: ' . $mysqli->connect_error);
}
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_OFF);
/* @ is used to suppress warnings */
```

```php
$link = @mysqli_connect('localhost', 'fake_user', 'wrong_password', 'does_not_exist');
if (!$link) {
    /* Use your preferred error logging method here */
    error_log('Connection error: ' . mysqli_connect_error());
}
```

See Also

- mysqli_connect() - Open a new connection to the MySQL server
- mysqli_connect_errno() - Returns the error code from last connect call
- mysqli_errno() - Returns the error code for the most recent function call
- mysqli_error() - Returns a string description of the last error
- mysqli_sqlstate() - Returns the SQLSTATE error from previous MySQL operation

mysqli::__construct

mysqli::__construct

mysqli::connect

mysqli_connect

(PHP 5, PHP 7, PHP 8)

mysqli::__construct -- mysqli::connect -- mysqli_connect — Open a new connection to the MySQL server

Description

Object-oriented style

```
public function mysqli::__construct(
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null
)
public function mysqli::connect(
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null
): bool
```

Procedural style

```
function mysqli_connect(
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
```

```
    ?int $port = null,
    ?string $socket = null
): mysqli|false
```

Opens a connection to the MySQL Server.

Parameters

`hostname`

> Can be either a host name or an IP address. When passing `null`, the value is retrieved from mysqli.default_host. When possible, pipes will be used instead of the TCP/IP protocol. The TCP/IP protocol is used if a host name and port number are provided together e.g. `localhost:3308`.
>
> Prepending host by `p:` opens a persistent connection. mysqli_change_user() is automatically called on connections opened from the connection pool.

`username`

> The MySQL username or `null` to assume the username based on the mysqli.default_user ini option.

`password`

> The MySQL password or `null` to assume the password based on the mysqli.default_pw ini option.

`database`

> The default database to be used when performing queries or `null`.

`port`

> The port number to attempt to connect to the MySQL server or `null` to assume the port based on the mysqli.default_port ini option.

`socket`

> The socket or named pipe that should be used or `null` to assume the socket based on the mysqli.default_socket ini option.
>
> > **Note** :
> >
> > Specifying the `socket` parameter will not explicitly determine the type of connection to be used when connecting to the MySQL server. How the connection is made to the MySQL database is determined by the `hostname` parameter.

Return Values

mysqli::__construct() always returns an object which represents the connection to a MySQL Server, regardless of it being successful or not.

mysqli_connect() returns an object which represents the connection to a MySQL Server, or `false` on failure.

mysqli::connect() returns `true` on success or `false` on failure. Prior to PHP 8.1.0, returns `null` on success.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

As of PHP 8.5.0, calling the constructor on an already-constructed object throws an Error.

Changelog

Version	Description
8.5.0	Calling the constructor on an already-constructed object now throws an Error.
8.1.0	**mysqli::connect()** now returns `true` instead of `null` on success.
7.4.0	All parameters are now nullable.

Examples

Example #1 mysqli::__construct() example

Object-oriented style

```php
<?php

/* You should enable error reporting for mysqli before attempting to make a connection */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = new mysqli('localhost', 'my_user', 'my_password', 'my_db');

/* Set the desired charset after establishing a connection */
$mysqli->set_charset('utf8mb4');

printf("Success... %s\n", $mysqli->host_info);
```

Procedural style

```php
<?php

/* You should enable error reporting for mysqli before attempting to make a connection */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$mysqli = mysqli_connect('localhost', 'my_user', 'my_password', 'my_db');

/* Set the desired charset after establishing a connection */
mysqli_set_charset($mysqli, 'utf8mb4');

printf("Success... %s\n", mysqli_get_host_info($mysqli));
```

The above examples will output something similar to:

```
Success... localhost via TCP/IP
```

Example #2 Extending mysqli class

```php
<?php

class FooMysqli extends mysqli {
    public function __construct($host, $user, $pass, $db, $port, $socket, $charset) {
        mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
        parent::__construct($host, $user, $pass, $db, $port, $socket);
        $this->set_charset($charset);
    }
}

$db = new FooMysqli('localhost', 'my_user', 'my_password', 'my_db', 3306, null, 'utf8mb4');
```

Example #3 Manual error handling

If error reporting is disabled, the developer is responsible for checking and handling failures

Object-oriented style

```php
<?php

error_reporting(0);
mysqli_report(MYSQLI_REPORT_OFF);
$mysqli = new mysqli('localhost', 'my_user', 'my_password', 'my_db');
if ($mysqli->connect_errno) {
    throw new RuntimeException('mysqli connection error: ' . $mysqli->connect_error);
}

/* Set the desired charset after establishing a connection */
$mysqli->set_charset('utf8mb4');
if ($mysqli->errno) {
    throw new RuntimeException('mysqli error: ' . $mysqli->error);
}
```

Procedural style

```php
<?php

error_reporting(0);
mysqli_report(MYSQLI_REPORT_OFF);
$mysqli = mysqli_connect('localhost', 'my_user', 'my_password', 'my_db');
if (mysqli_connect_errno()) {
    throw new RuntimeException('mysqli connection error: ' . mysqli_connect_error());
}

/* Set the desired charset after establishing a connection */
mysqli_set_charset($mysqli, 'utf8mb4');
if (mysqli_errno($mysqli)) {
    throw new RuntimeException('mysqli error: ' . mysqli_error($mysqli));
}
```

Notes

> **Note** :
>
> MySQLnd always assumes the server default charset. This charset is sent during connection hand-shake/authentication, which mysqlnd will use.

> Libmysqlclient uses the default charset set in the *my.cnf* or by an explicit call to mysqli_options() prior to calling mysqli_real_connect(), but after mysqli_init().

> **Note** :
>
> Object-oriented style only: If the connection fails, an object is still returned. To check whether the connection failed, use either the mysqli_connect_error() function or the mysqli->connect_error property as in the preceding examples.

> **Note** :
>
> If it is necessary to set options, such as the connection timeout, mysqli_real_connect() must be used instead.

> **Note** :
>
> Calling the constructor with no parameters is the same as calling mysqli_init().

> **Note** :
>
> Error "Can't create TCP/IP socket (10106)" usually means that the variables_order configure directive doesn't contain character `E`. On Windows, if the environment is not copied the `SYSTEMROOT` environment variable won't be available and PHP will have problems loading Winsock.

See Also

- mysqli_real_connect() - Opens a connection to the MySQL server
- mysqli_options() - Set options
- mysqli_connect_errno() - Returns the error code from last connect call
- mysqli_connect_error() - Returns a description of the last connection error
- mysqli_close() - Closes a previously opened database connection

mysqli::debug

mysqli::debug

mysqli_debug

(PHP 5, PHP 7, PHP 8)

mysqli::debug -- mysqli_debug — Performs debugging operations

Description

Object-oriented style

public function **mysqli::debug**(string `$options`): true

Procedural style

function **mysqli_debug**(string `$options`): true

Performs debugging operations using the Fred Fish debugging library.

Parameters

`options`

A string representing the debugging operation to perform

The debug control string is a sequence of colon separated fields as follows:

```
<field_1>:<field_2>:<field_N>
```

Each field consists of a mandatory flag character followed by an optional `,` and comma separated list of modifiers: `flag[,modifier,modifier,...,modifier]`

Recognized Flag Characters

`options` character	Description
O	`MYSQLND_DEBUG_FLUSH`
A/a	`MYSQLND_DEBUG_APPEND`
F	`MYSQLND_DEBUG_DUMP_FILE`
i	`MYSQLND_DEBUG_DUMP_PID`
L	`MYSQLND_DEBUG_DUMP_LINE`
m	`MYSQLND_DEBUG_TRACE_MEMORY_CALLS`
n	`MYSQLND_DEBUG_DUMP_LEVEL`
o	output to file
T	`MYSQLND_DEBUG_DUMP_TIME`
t	`MYSQLND_DEBUG_DUMP_TRACE`
x	`MYSQLND_DEBUG_PROFILE_CALLS`

Return Values

Always returns `true`.

Changelog

Version	Description
8.0.0	This function now always returns `true`. Previously it returned `false` on failure.

Examples

Example #1 Generating a Trace File

```
<?php

/* Create a trace file in '/tmp/client.trace' on the local (client) machine: */
mysqli_debug("d:t:o,/tmp/client.trace");

?>
```

Notes

> **Note** :
>
> To use the **mysqli_debug()** function you must compile the MySQL client library to support debugging.

See Also

- mysqli_dump_debug_info() - Dump debugging information into the log
- mysqli_report() - Sets mysqli error reporting mode

mysqli::dump_debug_info

mysqli::dump_debug_info

mysqli_dump_debug_info

(PHP 5, PHP 7, PHP 8)

mysqli::dump_debug_info -- mysqli_dump_debug_info — Dump debugging information into the log

Description

Object-oriented style

public function **mysqli::dump_debug_info**(): bool

Procedural style

function **mysqli_dump_debug_info**(mysqli `$mysql`): bool

This function is designed to be executed by a user with the SUPER privilege and is used to dump debugging information into the log for the MySQL Server relating to the connection.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Returns **`true`** on success or **`false`** on failure.

See Also

- mysqli_debug() - Performs debugging operations

mysqli::$errno

mysqli::$errno

mysqli_errno

(PHP 5, PHP 7, PHP 8)

mysqli::$errno -- mysqli_errno — Returns the error code for the most recent function call

Description

Object-oriented style

int *$mysqli->errno*;

Procedural style

function **mysqli_errno**(mysqli `$mysql`): int

Returns the last error code for the most recent MySQLi function call that can succeed or fail.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

An error code value for the last call, if it failed. `0` means no error occurred.

Examples

Example #1 *$mysqli->errno* example

Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if ($mysqli->connect_errno) {
    printf("Connect failed: %s\n", $mysqli->connect_error);
    exit();
}

if (!$mysqli->query("SET a=1")) {
    printf("Errorcode: %d\n", $mysqli->errno);
}

/* close connection */
$mysqli->close();
?>
```

Procedural style

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}
```

```php
if (!mysqli_query($link, "SET a=1")) {
    printf("Errorcode: %d\n", mysqli_errno($link));
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Errorcode: 1193
```

See Also

- mysqli_connect_errno() - Returns the error code from last connect call
- mysqli_connect_error() - Returns a description of the last connection error
- mysqli_error() - Returns a string description of the last error
- mysqli_sqlstate() - Returns the SQLSTATE error from previous MySQL operation

mysqli::$error

mysqli::$error

mysqli_error

(PHP 5, PHP 7, PHP 8)

mysqli::$error -- mysqli_error — Returns a string description of the last error

Description

Object-oriented style

string *$mysqli->error*;

Procedural style

function **mysqli_error**(mysqli `$mysql`): string

Returns the last error message for the most recent MySQLi function call that can succeed or fail.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

A string that describes the error. An empty string if no error occurred.

Examples

Example #1 *$mysqli->error* example

Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if ($mysqli->connect_errno) {
    printf("Connect failed: %s\n", $mysqli->connect_error);
    exit();
}

if (!$mysqli->query("SET a=1")) {
    printf("Error message: %s\n", $mysqli->error);
}

/* close connection */
$mysqli->close();
?>
```

Procedural style

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

if (!mysqli_query($link, "SET a=1")) {
    printf("Error message: %s\n", mysqli_error($link));
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Error message: Unknown system variable 'a'
```

See Also

- mysqli_connect_errno() - Returns the error code from last connect call
- mysqli_connect_error() - Returns a description of the last connection error
- mysqli_errno() - Returns the error code for the most recent function call
- mysqli_sqlstate() - Returns the SQLSTATE error from previous MySQL operation

mysqli::$error_list

mysqli::$error_list

mysqli_error_list

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

mysqli::$error_list -- mysqli_error_list — Returns a list of errors from the last command executed

Description

Object-oriented style

array *$mysqli->error_list*;

Procedural style

function **mysqli_error_list**(mysqli `$mysql`): array

Returns an array of errors for the most recent MySQLi function call that can succeed or fail.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

A list of errors, each as an associative array containing the errno, error, and sqlstate.

Examples

Example #1 *$mysqli->error_list* example

Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "nobody", "");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

if (!$mysqli->query("SET a=1")) {
    print_r($mysqli->error_list);
}

/* close connection */
$mysqli->close();
?>
```

Procedural style

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

if (!mysqli_query($link, "SET a=1")) {
    print_r(mysqli_error_list($link));
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Array
(
    [0] => Array
        (
            [errno] => 1193
            [sqlstate] => HY000
            [error] => Unknown system variable 'a'
        )

)
```

See Also

- mysqli_connect_errno() - Returns the error code from last connect call
- mysqli_connect_error() - Returns a description of the last connection error
- mysqli_error() - Returns a string description of the last error
- mysqli_sqlstate() - Returns the SQLSTATE error from previous MySQL operation

mysqli::escape_string

mysqli::escape_string

mysqli_escape_string

(PHP 5, PHP 7, PHP 8)

mysqli::escape_string -- mysqli_escape_string — Alias of mysqli_real_escape_string()

Description

This function is an alias of: mysqli_real_escape_string().

mysqli::execute_query

mysqli::execute_query

mysqli_execute_query

(PHP 8 >= 8.2.0)

mysqli::execute_query -- mysqli_execute_query — Prepares, binds parameters, and executes SQL statement

Description

Object-oriented style

public function **mysqli::execute_query**(string `$query`, ?array `$params = null`): mysqli_result|bool

Procedural style

function **mysqli_execute_query**(mysqli `$mysql`, string `$query`, ?array `$params = null`): mysqli_result|bool

Prepares the SQL query, binds parameters, and executes it. The **mysqli::execute_query()** method is a shortcut for mysqli::prepare(), mysqli_stmt::bind_param(), mysqli_stmt::execute(), and mysqli_stmt::get_result().

The statement template can contain zero or more question mark (`?`) parameter markers—also called placeholders. The parameter values must be provided as an array using `params` parameter.

A prepared statement is created under the hood but it's never exposed outside of the function. It's impossible to access properties of the statement as one would do with the mysqli_stmt object. Due to this limitation, the status information is copied to the mysqli object and is available using its methods, e.g. mysqli_affected_rows() or mysqli_error().

> **Note** :
>
> In the case where a statement is passed to **mysqli_execute_query()** that is longer than `max_allowed_packet` of the server, the returned error codes are different depending on the operating system. The behavior is as follows:
>
> - On Linux returns an error code of 1153. The error message means “got a packet bigger than `max_allowed_packet` bytes”.
> - On Windows returns an error code 2006. This error message means “server has gone away”.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`query`

The query, as a string. It must consist of a single SQL statement.

The SQL statement may contain zero or more parameter markers represented by question mark (`?`) characters at the appropriate positions.

> **Note** :
>
> The markers are legal only in certain places in SQL statements. For example, they are permitted in the `VALUES()` list of an `INSERT` statement (to specify column values for a row), or in a comparison with a column in a `WHERE` clause to specify a comparison value. However, they are not permitted for identifiers (such as table or column names).

`params`

An optional list array with as many elements as there are bound parameters in the SQL statement being executed. Each value is treated as a string.

Return Values

Returns **`false`** on failure. For successful queries which produce a result set, such as `SELECT, SHOW, DESCRIBE` or `EXPLAIN`, returns a mysqli_result object. For other successful queries, returns **`true`**.

Examples

Example #1 mysqli::execute_query() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli('localhost', 'my_user', 'my_password', 'world');

$query = 'SELECT Name, District FROM City WHERE CountryCode=? ORDER BY Name LIMIT 5';
$result = $mysqli->execute_query($query, ['DEU']);
foreach ($result as $row) {
    printf("%s (%s)\n", $row["Name"], $row["District"]);
}
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = 'SELECT Name, District FROM City WHERE CountryCode=? ORDER BY Name LIMIT 5';
$result = mysqli_execute_query($link, $query, ['DEU']);
foreach ($result as $row) {
    printf("%s (%s)\n", $row["Name"], $row["District"]);
}
```

The above examples will output something similar to:

```
Aachen (Nordrhein-Westfalen)
Augsburg (Baijeri)
Bergisch Gladbach (Nordrhein-Westfalen)
```

```
Berlin (Berliini)
Bielefeld (Nordrhein-Westfalen)
```

See Also

- mysqli_prepare() - Prepares an SQL statement for execution
- mysqli_stmt_execute() - Executes a prepared statement
- mysqli_stmt_bind_param() - Binds variables to a prepared statement as parameters
- mysqli_stmt_get_result() - Gets a result set from a prepared statement as a mysqli_result object

mysqli::$field_count

mysqli::$field_count

mysqli_field_count

(PHP 5, PHP 7, PHP 8)

mysqli::$field_count -- mysqli_field_count — Returns the number of columns for the most recent query

Description

Object-oriented style

int *$mysqli->field_count*;

Procedural style

function **mysqli_field_count**(mysqli `$mysql`): int

Returns the number of columns for the most recent query on the connection represented by the `mysql` parameter. This function can be useful when using the mysqli_store_result() function to determine if the query should have produced a non-empty result set or not without knowing the nature of the query.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

An integer representing the number of fields in a result set.

Examples

Example #1 *$mysqli->field_count* example

Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "test");

$mysqli->query( "DROP TABLE IF EXISTS friends");
```

```
$mysqli->query( "CREATE TABLE friends (id int, name varchar(20))");

$mysqli->query( "INSERT INTO friends VALUES (1,'Hartmut'), (2, 'Ulf')");


$mysqli->real_query("SELECT * FROM friends");

if ($mysqli->field_count) {
    /* this was a select/show or describe query */
    $result = $mysqli->store_result();

    /* process resultset */
    $row = $result->fetch_row();

    /* free resultset */
    $result->close();
}

/* close connection */
$mysqli->close();
?>
```

Procedural style

```
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "test");

mysqli_query($link, "DROP TABLE IF EXISTS friends");
mysqli_query($link, "CREATE TABLE friends (id int, name varchar(20))");

mysqli_query($link, "INSERT INTO friends VALUES (1,'Hartmut'), (2, 'Ulf')");

mysqli_real_query($link, "SELECT * FROM friends");

if (mysqli_field_count($link)) {
    /* this was a select/show or describe query */
    $result = mysqli_store_result($link);

    /* process resultset */
    $row = mysqli_fetch_row($result);

    /* free resultset */
    mysqli_free_result($result);
}

/* close connection */
mysqli_close($link);
?>
```

mysqli::get_charset

mysqli::get_charset

mysqli_get_charset

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

mysqli::get_charset -- mysqli_get_charset — Returns a character set object

Description

Object-oriented style

public function **mysqli::get_charset**(): ?object

Procedural style

function **mysqli_get_charset**(mysqli `$mysql`): ?object

Returns a character set object providing several properties of the current active character set.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

The function returns a character set object with the following properties:

`charset`

Character set name

`collation`

Collation name

`dir`

Directory the character set description was fetched from or "" for built-in character sets

`min_length`

Minimum character length in bytes

`max_length`

Maximum character length in bytes

`number`

Internal character set number

`state`

As of PHP 8.2.0, it is always `1`

Examples

Example #1 mysqli::get_charset() example

Object-oriented style

```
<?php
  $db = mysqli_init();
  $db->real_connect("localhost","root","","test");
```

```
  $db->set_charset('latin1');
  var_dump($db->get_charset());
?>
```

Procedural style

```
<?php
  $db = mysqli_init();
  mysqli_real_connect($db, "localhost","root","","test");
  mysqli_set_charset($db, 'latin1');
  var_dump(mysqli_get_charset($db));
?>
```

The above examples will output:

```
object(stdClass)#2 (7) {
  ["charset"]=>
  string(6) "latin1"
  ["collation"]=>
  string(17) "latin1_swedish_ci"
  ["dir"]=>
  string(0) ""
  ["min_length"]=>
  int(1)
  ["max_length"]=>
  int(1)
  ["number"]=>
  int(8)
  ["state"]=>
  int(1)
}
```

See Also

- mysqli_character_set_name() - Returns the current character set of the database connection
- mysqli_set_charset() - Sets the client character set

mysqli::$client_info

mysqli::$client_info

mysqli::get_client_info

mysqli_get_client_info

(PHP 5, PHP 7, PHP 8)

mysqli::$client_info -- mysqli::get_client_info -- mysqli_get_client_info — Get MySQL client info

Description

Object-oriented style

```
string $mysqli->client_info;
#[\Deprecated(
    since: '8.1',
    message: 'use mysqli_get_client_info() instead',
)]
public function mysqli::get_client_info(): string
```

Procedural style

```
function mysqli_get_client_info(?mysqli $mysql = null): string
```

Returns a string that represents the MySQL client library version.

Parameters

This function has no parameters.

Return Values

A string that represents the MySQL client library version.

Changelog

Version	Description
8.1.0	Calling **mysqli_get_client_info()** with the `mysql` argument has been deprecated. This function never required a parameter, but incorrectly allowed it as an optional parameter.
8.1.0	The object-oriented style **mysqli::get_client_info()** has been deprecated.

Examples

Example #1 mysqli_get_client_info

```php
<?php

/* We don't need a connection to determine
   the version of mysql client library */

printf("Client library version: %s\n", mysqli_get_client_info());
?>
```

See Also

- mysqli_get_client_version() - Returns the MySQL client version as an integer
- mysqli_get_server_info() - Returns the version of the MySQL server
- mysqli_get_server_version() - Returns the version of the MySQL server as an integer

mysqli::$client_version

mysqli::$client_version

mysqli_get_client_version

(PHP 5, PHP 7, PHP 8)

mysqli::$client_version -- mysqli_get_client_version — Returns the MySQL client version as an integer

Description

Object-oriented style

int *$mysqli->client_version*;

Procedural style

function **mysqli_get_client_version**(): int

Returns client version number as an integer.

Parameters

This function has no parameters.

Return Values

A number that represents the MySQL client library version in format: `main_version*10000 + minor_version *100 + sub_version`. For example, 4.1.0 is returned as 40100.

This is useful to quickly determine the version of the client library to know if some capability exists.

Examples

Example #1 mysqli_get_client_version

```php
<?php

/* We don't need a connection to determine
   the version of mysql client library */

printf("Client library version: %d\n", mysqli_get_client_version());
?>
```

See Also

- mysqli_get_client_info() - Get MySQL client info
- mysqli_get_server_info() - Returns the version of the MySQL server
- mysqli_get_server_version() - Returns the version of the MySQL server as an integer

mysqli::get_connection_stats

mysqli::get_connection_stats

mysqli_get_connection_stats

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli::get_connection_stats -- mysqli_get_connection_stats — Returns statistics about the client connection

Description

Object-oriented style

public function **mysqli::get_connection_stats**(): array

Procedural style

function **mysqli_get_connection_stats**(mysqli `$mysql`): array

Returns statistics about the client connection.

> **Note** :
>
> Available only with mysqlnd.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Returns an array with connection stats.

Examples

Example #1 A **mysqli_get_connection_stats()** example

```
<?php
$link = mysqli_connect();
print_r(mysqli_get_connection_stats($link));
?>
```

The above example will output something similar to:

```
Array
(
    [bytes_sent] => 43
    [bytes_received] => 80
    [packets_sent] => 1
    [packets_received] => 2
    [protocol_overhead_in] => 8
    [protocol_overhead_out] => 4
    [bytes_received_ok_packet] => 11
    [bytes_received_eof_packet] => 0
    [bytes_received_rset_header_packet] => 0
    [bytes_received_rset_field_meta_packet] => 0
    [bytes_received_rset_row_packet] => 0
    [bytes_received_prepare_response_packet] => 0
    [bytes_received_change_user_packet] => 0
    [packets_sent_command] => 0
    [packets_received_ok] => 1
    [packets_received_eof] => 0
    [packets_received_rset_header] => 0
    [packets_received_rset_field_meta] => 0
    [packets_received_rset_row] => 0
```

```
    [packets_received_prepare_response] => 0
    [packets_received_change_user] => 0
    [result_set_queries] => 0
    [non_result_set_queries] => 0
    [no_index_used] => 0
    [bad_index_used] => 0
    [slow_queries] => 0
    [buffered_sets] => 0
    [unbuffered_sets] => 0
    [ps_buffered_sets] => 0
    [ps_unbuffered_sets] => 0
    [flushed_normal_sets] => 0
    [flushed_ps_sets] => 0
    [ps_prepared_never_executed] => 0
    [ps_prepared_once_executed] => 0
    [rows_fetched_from_server_normal] => 0
    [rows_fetched_from_server_ps] => 0
    [rows_buffered_from_client_normal] => 0
    [rows_buffered_from_client_ps] => 0
    [rows_fetched_from_client_normal_buffered] => 0
    [rows_fetched_from_client_normal_unbuffered] => 0
    [rows_fetched_from_client_ps_buffered] => 0
    [rows_fetched_from_client_ps_unbuffered] => 0
    [rows_fetched_from_client_ps_cursor] => 0
    [rows_skipped_normal] => 0
    [rows_skipped_ps] => 0
    [copy_on_write_saved] => 0
    [copy_on_write_performed] => 0
    [command_buffer_too_small] => 0
    [connect_success] => 1
    [connect_failure] => 0
    [connection_reused] => 0
    [reconnect] => 0
    [pconnect_success] => 0
    [active_connections] => 1
    [active_persistent_connections] => 0
    [explicit_close] => 0
    [implicit_close] => 0
    [disconnect_close] => 0
    [in_middle_of_command_close] => 0
    [explicit_free_result] => 0
    [implicit_free_result] => 0
    [explicit_stmt_close] => 0
    [implicit_stmt_close] => 0
    [mem_emalloc_count] => 0
    [mem_emalloc_ammount] => 0
    [mem_ecalloc_count] => 0
    [mem_ecalloc_ammount] => 0
    [mem_erealloc_count] => 0
    [mem_erealloc_ammount] => 0
    [mem_efree_count] => 0
    [mem_malloc_count] => 0
    [mem_malloc_ammount] => 0
    [mem_calloc_count] => 0
    [mem_calloc_ammount] => 0
    [mem_realloc_count] => 0
    [mem_realloc_ammount] => 0
    [mem_free_count] => 0
    [proto_text_fetched_null] => 0
```

```
    [proto_text_fetched_bit] => 0
    [proto_text_fetched_tinyint] => 0
    [proto_text_fetched_short] => 0
    [proto_text_fetched_int24] => 0
    [proto_text_fetched_int] => 0
    [proto_text_fetched_bigint] => 0
    [proto_text_fetched_decimal] => 0
    [proto_text_fetched_float] => 0
    [proto_text_fetched_double] => 0
    [proto_text_fetched_date] => 0
    [proto_text_fetched_year] => 0
    [proto_text_fetched_time] => 0
    [proto_text_fetched_datetime] => 0
    [proto_text_fetched_timestamp] => 0
    [proto_text_fetched_string] => 0
    [proto_text_fetched_blob] => 0
    [proto_text_fetched_enum] => 0
    [proto_text_fetched_set] => 0
    [proto_text_fetched_geometry] => 0
    [proto_text_fetched_other] => 0
    [proto_binary_fetched_null] => 0
    [proto_binary_fetched_bit] => 0
    [proto_binary_fetched_tinyint] => 0
    [proto_binary_fetched_short] => 0
    [proto_binary_fetched_int24] => 0
    [proto_binary_fetched_int] => 0
    [proto_binary_fetched_bigint] => 0
    [proto_binary_fetched_decimal] => 0
    [proto_binary_fetched_float] => 0
    [proto_binary_fetched_double] => 0
    [proto_binary_fetched_date] => 0
    [proto_binary_fetched_year] => 0
    [proto_binary_fetched_time] => 0
    [proto_binary_fetched_datetime] => 0
    [proto_binary_fetched_timestamp] => 0
    [proto_binary_fetched_string] => 0
    [proto_binary_fetched_blob] => 0
    [proto_binary_fetched_enum] => 0
    [proto_binary_fetched_set] => 0
    [proto_binary_fetched_geometry] => 0
    [proto_binary_fetched_other] => 0
)
```

See Also

- Stats description

mysqli::$host_info

mysqli::$host_info

mysqli_get_host_info

(PHP 5, PHP 7, PHP 8)

mysqli::$host_info -- mysqli_get_host_info — Returns a string representing the type of connection used

Description

Object-oriented style

string *$mysqli->host_info*;

Procedural style

function **mysqli_get_host_info**(mysqli `$mysql`): string

Returns a string describing the connection represented by the `mysql` parameter (including the server host name).

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

A character string representing the server hostname and the connection type.

Examples

Example #1 *$mysqli->host_info* example

Object-oriented style

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* print host information */
printf("Host info: %s\n", $mysqli->host_info);
```

Procedural style

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* print host information */
printf("Host info: %s\n", mysqli_get_host_info($link));
```

The above examples will output:

```
Host info: Localhost via UNIX socket
```

See Also

- mysqli_get_proto_info() - Returns the version of the MySQL protocol used

mysqli::$protocol_version

mysqli::$protocol_version

mysqli_get_proto_info

(PHP 5, PHP 7, PHP 8)

mysqli::$protocol_version -- mysqli_get_proto_info — Returns the version of the MySQL protocol used

Description

Object-oriented style

int *$mysqli->protocol_version*;

Procedural style

function **mysqli_get_proto_info**(mysqli `$mysql`): int

Returns an integer representing the MySQL protocol version used by the connection represented by the `mysql` parameter.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Returns an integer representing the protocol version.

Examples

Example #1 *$mysqli->protocol_version* example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password");

/* print protocol version */
printf("Protocol version: %d\n", $mysqli->protocol_version);
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password");
```

```
/* print protocol version */
printf("Protocol version: %d\n", mysqli_get_proto_info($link));
```

The above examples will output:

```
Protocol version: 10
```

See Also

- mysqli_get_host_info() - Returns a string representing the type of connection used

mysqli::$server_info

mysqli::$server_info

mysqli::get_server_info

mysqli_get_server_info

(PHP 5, PHP 7, PHP 8)

mysqli::$server_info -- mysqli::get_server_info -- mysqli_get_server_info — Returns the version of the MySQL server

Description

Object-oriented style

string *$mysqli->server_info*;
public function **mysqli::get_server_info**(): string

Procedural style

function **mysqli_get_server_info**(mysqli `$mysql`): string

Returns a string representing the version of the MySQL server that the MySQLi extension is connected to.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

A character string representing the server version.

Examples

Example #1 *$mysqli->server_info* example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password");

/* print server version */
printf("Server version: %s\n", $mysqli->server_info);
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password");

/* print server version */
printf("Server version: %s\n", mysqli_get_server_info($link));
```

The above examples will output something similar to:

```
Server version: 8.0.21
```

See Also

- mysqli_get_client_info() - Get MySQL client info
- mysqli_get_client_version() - Returns the MySQL client version as an integer
- mysqli_get_server_version() - Returns the version of the MySQL server as an integer

mysqli::$server_version

mysqli::$server_version

mysqli_get_server_version

(PHP 5, PHP 7, PHP 8)

mysqli::$server_version -- mysqli_get_server_version — Returns the version of the MySQL server as an integer

Description

Object-oriented style

int *$mysqli->server_version*;

Procedural style

function **mysqli_get_server_version**(mysqli `$mysql`): int

The **mysqli_get_server_version()** function returns the version of the server connected to (represented by the `mysql` parameter) as an integer.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

An integer representing the server version.

The form of this version number is `main_version * 10000 + minor_version * 100 + sub_version` (i.e. version 4.1.0 is 40100).

Examples

Example #1 *$mysqli->server_version* example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password");

/* print server version */
printf("Server version: %d\n", $mysqli->server_version);
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password");

/* print server version */
printf("Server version: %d\n", mysqli_get_server_version($link));
```

The above examples will output something similar to:

```
Server version: 80021
```

See Also

- mysqli_get_client_info() - Get MySQL client info
- mysqli_get_client_version() - Returns the MySQL client version as an integer
- mysqli_get_server_info() - Returns the version of the MySQL server

mysqli::get_warnings

mysqli::get_warnings

mysqli_get_warnings

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

mysqli::get_warnings -- mysqli_get_warnings — Get result of SHOW WARNINGS

Description

Object-oriented style

public function **mysqli::get_warnings**(): mysqli_warning|false

Procedural style

function **mysqli_get_warnings**(mysqli `$mysql`): mysqli_warning|false

Returns a singly linked list comprised of mysqli_warning or **`false`** if there are no warnings. Each object in the list corresponds to a single line from the result of `SHOW WARNINGS`. Calling mysqli_warning::next() will refill the object with the values from the next row.

> **Note** :
>
> To retrieve warning messages, it is recommended to use the SQL command `SHOW WARNINGS [limit row_count]` instead of this function.

> **Warning**
>
> The linked list cannot be rewound or retrieved again.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Returns a singly linked list comprised of mysqli_warning or **`false`** if there are no warnings.

Examples

Example #1 Traversing the linked list to fetch all warnings

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$mysqli->query("SELECT 1/0, CAST('NULL' AS UNSIGNED)");

if ($mysqli->warning_count > 0) {
    $warning = $mysqli->get_warnings();
    if ($warning !== false) {
```

```
        do {
            printf("Error number: %s\n", $warning->errno);
            printf("Message: %s\n", $warning->message);
        } while ($warning->next());
    }
}
```

Procedural style

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "user", "password", "test");

mysqli_query($link, "SELECT 1/0, CAST('NULL' AS UNSIGNED)");

if (mysqli_warning_count($link) > 0) {
    $warning = mysqli_get_warnings($link);
    if ($warning !== false) {
        do {
            printf("Error number: %s\n", $warning->errno);
            printf("Message: %s\n", $warning->message);
        } while ($warning->next());
    }
}
```

The above examples will output:

```
Error number: 1365
Message: Division by 0
Error number: 1292
Message: Truncated incorrect INTEGER value: 'NULL'
```

mysqli::$info

mysqli::$info

mysqli_info

(PHP 5, PHP 7, PHP 8)

mysqli::$info -- mysqli_info — Retrieves information about the most recently executed query

Description

Object-oriented style

?string *$mysqli->info*;

Procedural style

function **mysqli_info**(mysqli `$mysql`): ?string

The **mysqli_info()** function returns a string providing information about the last query executed. The nature of this string is provided below:

Possible mysqli_info return values

Query type	Example result string
INSERT INTO...SELECT...	Records: 100 Duplicates: 0 Warnings: 0
INSERT INTO...VALUES (...),(...),(...)	Records: 3 Duplicates: 0 Warnings: 0
LOAD DATA INFILE ...	Records: 1 Deleted: 0 Skipped: 0 Warnings: 0
ALTER TABLE ...	Records: 3 Duplicates: 0 Warnings: 0
UPDATE ...	Rows matched: 40 Changed: 40 Warnings: 0

> **Note** :
>
> Queries which do not fall into one of the preceding formats are not supported. In these situations, **mysqli_info()** will return an empty string.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

A character string representing additional information about the most recently executed query.

Examples

Example #1 *$mysqli->info* example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$mysqli->query("CREATE TEMPORARY TABLE t1 LIKE City");

/* INSERT INTO ... SELECT */
$mysqli->query("INSERT INTO t1 SELECT * FROM City ORDER BY ID LIMIT 150");
printf("%s\n", $mysqli->info);
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

mysqli_query($link, "CREATE TEMPORARY TABLE t1 LIKE City");

/* INSERT INTO ... SELECT */
```

```
mysqli_query($link, "INSERT INTO t1 SELECT * FROM City ORDER BY ID LIMIT 150");
printf("%s\n", mysqli_info($link));
```

The above examples will output:

```
Records: 150  Duplicates: 0  Warnings: 0
```

See Also

- mysqli_affected_rows() - Gets the number of affected rows in a previous MySQL operation
- mysqli_warning_count() - Returns the number of warnings generated by the most recently executed query
- mysqli_num_rows() - Gets the number of rows in the result set

mysqli::init

mysqli::init

mysqli_init

(PHP 5, PHP 7, PHP 8)

mysqli::init -- mysqli_init — Initializes MySQLi and returns an object for use with mysqli_real_connect()

Description

Object-oriented style

```
#[\Deprecated(
    since: '8.1',
    message: 'replace calls to parent::init() with parent::__construct()',
)]
public function mysqli::init(): ?bool
```

Procedural style

```
function mysqli_init(): mysqli|false
```

Allocates or initializes a MYSQL object suitable for mysqli_options() and mysqli_real_connect().

> **Note** :
>
> Any subsequent call to any mysqli function (except mysqli_options() and mysqli_ssl_set()) will fail until mysqli_real_connect() is called.

Parameters

This function has no parameters.

Return Values

mysqli::init() returns `null` on success, or `false` on failure. **mysqli_init()** returns an object on success, or `false` on failure.

Changelog

Version	Description
8.1.0	The object-oriented style **mysqli::init()** method has been deprecated. Replace calls to **parent::init()** with **parent::__construct()**.

Examples

See mysqli_real_connect().

See Also

- mysqli_options() - Set options
- mysqli_close() - Closes a previously opened database connection
- mysqli_real_connect() - Opens a connection to the MySQL server
- mysqli_connect() - Open a new connection to the MySQL server

mysqli::$insert_id

mysqli::$insert_id

mysqli_insert_id

(PHP 5, PHP 7, PHP 8)

mysqli::$insert_id -- mysqli_insert_id — Returns the value generated for an AUTO_INCREMENT column by the last query

Description

Object-oriented style

int|string *$mysqli->insert_id*;

Procedural style

function **mysqli_insert_id**(mysqli `$mysql`): int|string

Returns the ID generated by an `INSERT` or `UPDATE` query on a table with a column having the `AUTO_INCREMENT` attribute. In the case of a multiple-row `INSERT` statement, it returns the first automatically generated value that was successfully inserted.

Performing an `INSERT` or `UPDATE` statement using the `LAST_INSERT_ID()` MySQL function will also modify the value returned by **mysqli_insert_id()**. If `LAST_INSERT_ID(expr)` was used to generate the value of `AUTO_INCREMENT`, it returns the value of the last `expr` instead of the generated `AUTO_INCREMENT` value.

Returns `0` if the previous statement did not change an `AUTO_INCREMENT` value. **mysqli_insert_id()** must be called immediately after the statement that generated the value.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

The value of the AUTO_INCREMENT field that was updated by the previous query. Returns zero if there was no previous query on the connection or if the query did not update an AUTO_INCREMENT value.

Only statements issued using the current connection affect the return value. The value is not affected by statements issued using other connections or clients.

> **Note** :
>
> If the number is greater than the maximum int value, it will be returned as a string.

Examples

Example #1 *$mysqli->insert_id* example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$mysqli->query("CREATE TABLE myCity LIKE City");

$query = "INSERT INTO myCity VALUES (NULL, 'Stuttgart', 'DEU', 'Stuttgart', 617000)";
$mysqli->query($query);

printf("New record has ID %d.\n", $mysqli->insert_id);

/* drop table */
$mysqli->query("DROP TABLE myCity");
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

mysqli_query($link, "CREATE TABLE myCity LIKE City");

$query = "INSERT INTO myCity VALUES (NULL, 'Stuttgart', 'DEU', 'Stuttgart', 617000)";
mysqli_query($link, $query);

printf("New record has ID %d.\n", mysqli_insert_id($link));

/* drop table */
mysqli_query($link, "DROP TABLE myCity");
```

The above examples will output:

```
```

```
New record has ID 1.
```

mysqli::kill

mysqli::kill

mysqli_kill

(PHP 5, PHP 7, PHP 8)

mysqli::kill -- mysqli_kill — Asks the server to kill a MySQL thread

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.4.0. Relying on this function is highly discouraged.

Description

Object-oriented style

```
#[\Deprecated(
    since: '8.4',
    message: 'use KILL CONNECTION/QUERY SQL statement instead',
)]
public function mysqli::kill(int $process_id): bool
```

Procedural style

```
#[\Deprecated(
    since: '8.4',
    message: 'use KILL CONNECTION/QUERY SQL statement instead',
)]
function mysqli_kill(mysqli $mysql, int $process_id): bool
```

This function is used to ask the server to kill a MySQL thread specified by the `process_id` parameter. This value must be retrieved by calling the mysqli_thread_id() function.

To stop a running query you should use the SQL command `KILL QUERY processid`.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

If mysqli error reporting is enabled (**MYSQLI_REPORT_ERROR**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **MYSQLI_REPORT_STRICT**, a mysqli_sql_exception is thrown instead.

Changelog

Version	Description
8.4.0	Both **mysqli::kill()** and **mysqli_kill()** are now deprecated. Use the KILL SQL command instead.

Examples

Example #1 mysqli::kill() example

Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

/* determine our thread id */
$thread_id = $mysqli->thread_id;

/* Kill connection */
$mysqli->kill($thread_id);

/* This should produce an error */
if (!$mysqli->query("CREATE TABLE myCity LIKE City")) {
    printf("Error: %s\n", $mysqli->error);
    exit;
}

/* close connection */
$mysqli->close();
?>
```

Procedural style

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

/* determine our thread id */
$thread_id = mysqli_thread_id($link);

/* Kill connection */
mysqli_kill($link, $thread_id);

/* This should produce an error */
```

```php
if (!mysqli_query($link, "CREATE TABLE myCity LIKE City")) {
    printf("Error: %s\n", mysqli_error($link));
    exit;
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Error: MySQL server has gone away
```

See Also

- mysqli_thread_id() - Returns the thread ID for the current connection

mysqli::more_results

mysqli::more_results

mysqli_more_results

(PHP 5, PHP 7, PHP 8)

mysqli::more_results -- mysqli_more_results — Check if there are any more query results from a multi query

Description

Object-oriented style

public function **mysqli::more_results**(): bool

Procedural style

function **mysqli_more_results**(mysqli `$mysql`): bool

Indicates if one or more result sets are available from a previous call to mysqli_multi_query().

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Returns **`true`** if one or more result sets (including errors) are available from a previous call to mysqli_multi_query(), otherwise **`false`**.

Examples

See mysqli_multi_query().

See Also

- mysqli_multi_query() - Performs one or more queries on the database
- mysqli_next_result() - Prepare next result from multi_query
- mysqli_store_result() - Transfers a result set from the last query
- mysqli_use_result() - Initiate a result set retrieval

mysqli::multi_query

mysqli::multi_query

mysqli_multi_query

(PHP 5, PHP 7, PHP 8)

mysqli::multi_query -- mysqli_multi_query — Performs one or more queries on the database

Description

Object-oriented style

public function **mysqli::multi_query**(string `$query`): bool

Procedural style

function **mysqli_multi_query**(mysqli `$mysql`, string `$query`): bool

Executes one or multiple queries which are concatenated by a semicolon.

> **Warning**
>
> **Security warning: SQL injection**
>
> If the query contains any variable input then parameterized prepared statements should be used instead. Alternatively, the data must be properly formatted and all strings must be escaped using the mysqli_real_escape_string() function.

Queries are sent in a single call to the database and processed sequentially. **mysqli_multi_query()** waits for the first query to complete before returning control to PHP. In the meantime, the MySQL server will continue processing the remaining queries asynchronously from PHP and make the results available for fetching.

It is recommended to use do-while to process multiple queries. The connection will be busy until all queries have completed and their results are fetched to PHP. No other statement can be issued on the same connection until all queries are processed. To proceed to the next query in the sequence, use mysqli_next_result(). If the next result is not ready yet, mysqli will wait for the response from the MySQL server. To check if there are more results, use mysqli_more_results().

For queries which produce a result set, such as `SELECT`, `SHOW`, `DESCRIBE` or `EXPLAIN`, mysqli_use_result() or mysqli_store_result() can be used to retrieve the result set. For queries which do not produce a result set, the same functions can be used to retrieve information such as the number of affected rows.

> **Tip**

> Executing CALL statements for stored procedures can produce multiple result sets. If the stored procedure contains SELECT statements, the result sets are returned in the order that they are produced as the procedure executes. In general, the caller cannot know how many result sets a procedure will return and must be prepared to retrieve multiple results. The final result from the procedure is a status result that includes no result set. The status indicates whether the procedure succeeded or an error occurred.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`query`

A string containing the queries to be executed. Multiple queries must be separated by a semicolon.

Return Values

Returns **`false`** if the first statement failed. To retrieve subsequent errors from other statements you have to call mysqli_next_result() first.

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

Examples

Example #1 **mysqli::multi_query()** example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT CURRENT_USER();";
$query .= "SELECT Name FROM City ORDER BY ID LIMIT 20, 5";

/* execute multi query */
$mysqli->multi_query($query);
do {
    /* store the result set in PHP */
    if ($result = $mysqli->store_result()) {
        while ($row = $result->fetch_row()) {
            printf("%s\n", $row[0]);
        }
    }
    /* print divider */
    if ($mysqli->more_results()) {
        printf("-----------------\n");
    }
} while ($mysqli->next_result());
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT CURRENT_USER();";
$query .= "SELECT Name FROM City ORDER BY ID LIMIT 20, 5";

/* execute multi query */
mysqli_multi_query($link, $query);
do {
    /* store the result set in PHP */
    if ($result = mysqli_store_result($link)) {
        while ($row = mysqli_fetch_row($result)) {
            printf("%s\n", $row[0]);
        }
    }
    /* print divider */
    if (mysqli_more_results($link)) {
        printf("-----------------\n");
    }
} while (mysqli_next_result($link));
```

The above examples will output something similar to:

```
my_user@localhost
-----------------
Amersfoort
Maastricht
Dordrecht
Leiden
Haarlemmermeer
```

See Also

- mysqli_query() - Performs a query on the database
- mysqli_use_result() - Initiate a result set retrieval
- mysqli_store_result() - Transfers a result set from the last query
- mysqli_next_result() - Prepare next result from multi_query
- mysqli_more_results() - Check if there are any more query results from a multi query

mysqli::next_result

mysqli::next_result

mysqli_next_result

(PHP 5, PHP 7, PHP 8)

mysqli::next_result -- mysqli_next_result — Prepare next result from multi_query

Description

Object-oriented style

public function **mysqli::next_result**(): bool

Procedural style

function **mysqli_next_result**(mysqli `$mysql`): bool

Prepares next result set from a previous call to mysqli_multi_query() which can be retrieved by mysqli_store_result() or mysqli_use_result().

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Returns **`true`** on success or **`false`** on failure. Also returns **`false`** if the next statement resulted in an error, unlike mysqli_more_results().

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

Examples

See mysqli_multi_query().

See Also

- mysqli_multi_query() - Performs one or more queries on the database
- mysqli_more_results() - Check if there are any more query results from a multi query
- mysqli_store_result() - Transfers a result set from the last query
- mysqli_use_result() - Initiate a result set retrieval

mysqli::options

mysqli::options

mysqli_options

(PHP 5, PHP 7, PHP 8)

mysqli::options -- mysqli_options — Set options

Description

Object-oriented style

public function **mysqli::options**(int `$option`, string|int `$value`): bool

Procedural style

function **mysqli_options**(mysqli `$mysql`, int `$option`, string|int `$value`): bool

Used to set extra connect options and affect behavior for a connection.

This function may be called multiple times to set several options.

mysqli_options() should be called after mysqli_init() and before mysqli_real_connect().

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`option`

The option that you want to set. It can be one of the following values:

Valid options

Name	Description
`MYSQLI_OPT_CONNECT_TIMEOUT`	Connection timeout in seconds
`MYSQLI_OPT_READ_TIMEOUT`	Command execution result timeout in seconds. Available as of PHP 7.2.0.
`MYSQLI_OPT_LOCAL_INFILE`	Enable/disable use of `LOAD LOCAL INFILE`
`MYSQLI_OPT_LOAD_DATA_LOCAL_DIR`	Directory to be used for `LOAD DATA LOCAL INFILE`.
`MYSQLI_INIT_COMMAND`	Command to execute after establishing a connection to the MySQL server
`MYSQLI_SET_CHARSET_NAME`	The charset to be set as default.
`MYSQLI_READ_DEFAULT_FILE`	Read options from named option file instead of *my.cnf* Not supported by mysqlnd.
`MYSQLI_READ_DEFAULT_GROUP`	Read options from the named group from *my.cnf* or the file specified with **`MYSQL_READ_DEFAULT_FILE`**. Not supported by mysqlnd.
`MYSQLI_SERVER_PUBLIC_KEY`	RSA public key file used with the SHA-256 based authentication.
`MYSQLI_OPT_NET_CMD_BUFFER_SIZE`	The size of the internal command/network buffer. Only valid for mysqlnd.
`MYSQLI_OPT_NET_READ_BUFFER_SIZE`	Maximum read chunk size in bytes when reading the body of a MySQL command packet. Only valid for mysqlnd.
`MYSQLI_OPT_INT_AND_FLOAT_NATIVE`	Convert integer and float columns back to PHP numbers when using non-prepared statements. Only valid for mysqlnd.
`MYSQLI_OPT_SSL_VERIFY_SERVER_CERT`	Whether to verify server certificate or not.
`MYSQLI_OPT_CAN_HANDLE_EXPIRED_PASSWORDS`	Whether the client should accept expired passwords.
`1`	Whether to use network communication compression.

`value`

The value for the option.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

Examples

See mysqli_real_connect().

Notes

> **Note** :
>
> MySQLnd always assumes the server default charset. This charset is sent during connection hand-shake/authentication, which mysqlnd will use.
>
> Libmysqlclient uses the default charset set in the *my.cnf* or by an explicit call to **mysqli_options()** prior to calling mysqli_real_connect(), but after mysqli_init().

See Also

- mysqli_init() - Initializes MySQLi and returns an object for use with mysqli_real_connect()
- mysqli_real_connect() - Opens a connection to the MySQL server

mysqli::ping

mysqli::ping

mysqli_ping

(PHP 5, PHP 7, PHP 8)

mysqli::ping -- mysqli_ping — Pings a server connection, or tries to reconnect if the connection has gone down

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.4.0. Relying on this function is highly discouraged.

Description

Object-oriented style

```
#[\Deprecated(
    since: '8.4',
    message: 'because the reconnect feature has been removed in PHP 8.2 and this method is now redundant',
)]
public function mysqli::ping(): bool
```

Procedural style

```
#[\Deprecated(
    since: '8.4',
    message: 'because the reconnect feature has been removed in PHP 8.2 and this function is now redundant',
)]
function mysqli_ping(mysqli $mysql): bool
```

Checks whether the connection to the server is working. If it has gone down and global option mysqli.reconnect is enabled, an automatic reconnection is attempted.

> **Note** :
>
> The *php.ini* setting mysqli.reconnect is ignored by the mysqlnd driver, so automatic reconnection is never attempted.

This function can be used by clients that remain idle for a long while, to check whether the server has closed the connection and reconnect if necessary.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

Changelog

Version	Description
8.4.0	Both **mysqli::ping()** and **mysqli_ping()** are now deprecated. As of PHP 8.2.0, the `reconnect` feature is no longer available, making this function obsolete.

Examples

Example #1 mysqli::ping() example

Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if ($mysqli->connect_errno) {
    printf("Connect failed: %s\n", $mysqli->connect_error);
    exit();
}

/* check if server is alive */
```

```php
if ($mysqli->ping()) {
    printf ("Our connection is ok!\n");
} else {
    printf ("Error: %s\n", $mysqli->error);
}

/* close connection */
$mysqli->close();
?>
```

Procedural style

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

/* check if server is alive */
if (mysqli_ping($link)) {
    printf ("Our connection is ok!\n");
} else {
    printf ("Error: %s\n", mysqli_error($link));
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Our connection is ok!
```

mysqli::poll

mysqli::poll

mysqli_poll

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli::poll -- mysqli_poll — Poll connections

Description

Object-oriented style

```
public static function mysqli::poll(
    ?array &$read,
    ?array &$error,
    array &$reject,
    int $seconds,
    int $microseconds = 0
): int|false
```

Procedural style

```
function mysqli_poll(
    ?array &$read,
    ?array &$error,
    array &$reject,
    int $seconds,
    int $microseconds = 0
): int|false
```

Poll connections. The method can be used as static.

> **Note** :
>
> Available only with mysqlnd.

Parameters

`read`

List of connections to check for outstanding results that can be read.

`error`

List of connections on which an error occurred, for example, query failure or lost connection.

`reject`

List of connections rejected because no pending asynchronous query exists to poll.

`seconds`

Maximum number of seconds to wait, must be non-negative.

`microseconds`

Maximum number of microseconds to wait, must be non-negative.

Return Values

Returns number of ready connections upon success, `false` otherwise.

Errors/Exceptions

A ValueError is thrown when neither the `read` nor `error` arguments are passed.

Changelog

Version	Description
8.3.0	Now throws a ValueError exception when neither the `read` nor `error` arguments are passed.

Examples

Example #1 A **mysqli_poll()** example

```php
<?php
$link1 = mysqli_connect();
$link1->query("SELECT 'test'", MYSQLI_ASYNC);
$all_links = array($link1);
$processed = 0;
do {
    $links = $errors = $reject = array();
    foreach ($all_links as $link) {
        $links[] = $errors[] = $reject[] = $link;
    }
    if (!mysqli_poll($links, $errors, $reject, 1)) {
        continue;
    }
    foreach ($links as $link) {
        if ($result = $link->reap_async_query()) {
            print_r($result->fetch_row());
            if (is_object($result))
                mysqli_free_result($result);
        } else die(sprintf("MySQLi Error: %s", mysqli_error($link)));
        $processed++;
    }
} while ($processed < count($all_links));
?>
```

The above example will output:

```
Array
(
    [0] => test
)
```

See Also

- mysqli_query() - Performs a query on the database
- mysqli_reap_async_query() - Get result from async query

mysqli::prepare

mysqli::prepare

mysqli_prepare

(PHP 5, PHP 7, PHP 8)

mysqli::prepare -- mysqli_prepare — Prepares an SQL statement for execution

Description

Object-oriented style

public function **mysqli::prepare**(string `$query`): mysqli_stmt|false

Procedural style

function **mysqli_prepare**(mysqli `$mysql`, string `$query`): mysqli_stmt|false

Prepares the SQL query, and returns a statement handle to be used for further operations on the statement. The query must consist of a single SQL statement.

The statement template can contain zero or more question mark (`?`) parameter markers—also called placeholders. The parameter markers must be bound to application variables using mysqli_stmt_bind_param() before executing the statement.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`query`

The query, as a string. It must consist of a single SQL statement.

The SQL statement may contain zero or more parameter markers represented by question mark (`?`) characters at the appropriate positions.

> **Note** :
>
> The markers are legal only in certain places in SQL statements. For example, they are permitted in the `VALUES()` list of an `INSERT` statement (to specify column values for a row), or in a comparison with a column in a `WHERE` clause to specify a comparison value. However, they are not permitted for identifiers (such as table or column names).

Return Values

mysqli_prepare() returns a statement object or **`false`** if an error occurred.

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

Examples

Example #1 mysqli::prepare() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$city = "Amersfoort";

/* create a prepared statement */
$stmt = $mysqli->prepare("SELECT District FROM City WHERE Name=?");

/* bind parameters for markers */
$stmt->bind_param("s", $city);

/* execute query */
$stmt->execute();

/* bind result variables */
$stmt->bind_result($district);

/* fetch value */
$stmt->fetch();

printf("%s is in district %s\n", $city, $district);
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$city = "Amersfoort";

/* create a prepared statement */
$stmt = mysqli_prepare($link, "SELECT District FROM City WHERE Name=?");

/* bind parameters for markers */
mysqli_stmt_bind_param($stmt, "s", $city);

/* execute query */
mysqli_stmt_execute($stmt);

/* bind result variables */
mysqli_stmt_bind_result($stmt, $district);

/* fetch value */
mysqli_stmt_fetch($stmt);

printf("%s is in district %s\n", $city, $district);
```

The above examples will output:

```
Amersfoort is in district Utrecht
```

See Also

- mysqli_stmt_execute() - Executes a prepared statement
- mysqli_stmt_fetch() - Fetch results from a prepared statement into the bound variables
- mysqli_stmt_bind_param() - Binds variables to a prepared statement as parameters
- mysqli_stmt_bind_result() - Binds variables to a prepared statement for result storage
- mysqli_stmt_get_result() - Gets a result set from a prepared statement as a mysqli_result object
- mysqli_stmt_close() - Closes a prepared statement

mysqli::query

mysqli::query

mysqli_query

(PHP 5, PHP 7, PHP 8)

mysqli::query -- mysqli_query — Performs a query on the database

Description

Object-oriented style

public function **mysqli::query**(string `$query`, int `$result_mode` = **`MYSQLI_STORE_RESULT`**): mysqli_result|bool

Procedural style

function **mysqli_query**(mysqli `$mysql`, string `$query`, int `$result_mode` = **`MYSQLI_STORE_RESULT`**): mysqli_result|bool

Performs a `query` against the database.

> **Warning**
>
> **Security warning: SQL injection**
>
> If the query contains any variable input then parameterized prepared statements should be used instead. Alternatively, the data must be properly formatted and all strings must be escaped using the mysqli_real_escape_string() function.

For non-DML queries (not INSERT, UPDATE or DELETE), this function is similar to calling mysqli_real_query() followed by either mysqli_use_result() or mysqli_store_result().

> **Note** :
>
> In the case where a statement is passed to **mysqli_query()** that is longer than `max_allowed_packet` of the server, the returned error codes are different depending on whether you are using MySQL Native Driver (`mysqlnd`) or MySQL Client Library (`libmysqlclient`). The behavior is as follows:
>
> - `mysqlnd` on Linux returns an error code of 1153. The error message means “got a packet bigger than `max_allowed_packet` bytes”.
> - `mysqlnd` on Windows returns an error code 2006. This error message means “server has gone away”.

> - `libmysqlclient` on all platforms returns an error code 2006. This error message means "server has gone away".

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`query`

The query string.

`result_mode`

The result mode can be one of 3 constants indicating how the result will be returned from the MySQL server.

`MYSQLI_STORE_RESULT` (default) - returns a mysqli_result object with buffered result set.

`MYSQLI_USE_RESULT` - returns a mysqli_result object with unbuffered result set. As long as there are pending records waiting to be fetched, the connection line will be busy and all subsequent calls will return error `Commands out of sync`. To avoid the error all records must be fetched from the server or the result set must be discarded by calling mysqli_free_result().

`MYSQLI_ASYNC` (available with mysqlnd) - the query is performed asynchronously and no result set is immediately returned. mysqli_poll() is then used to get results from such queries. Used in combination with either **`MYSQLI_STORE_RESULT`** or **`MYSQLI_USE_RESULT`** constant.

Return Values

Returns **`false`** on failure. For successful queries which produce a result set, such as `SELECT, SHOW, DESCRIBE` or `EXPLAIN`, **mysqli_query()** will return a mysqli_result object. For other successful queries, **mysqli_query()** will return **`true`**.

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

Changelog

Version	Description
8.4.0	When connecting to MySQL 8.0.24 or later, the error code reported for a server wait timeout is now `4031` (`ER_CLIENT_INTERACTION_TIMEOUT`) instead of `2006` (`CR_SERVER_GONE_ERROR`).

Examples

Example #1 **mysqli::query()** example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
```

```php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* Create table doesn't return a resultset */
$mysqli->query("CREATE TEMPORARY TABLE myCity LIKE City");
printf("Table myCity successfully created.\n");

/* Select queries return a resultset */
$result = $mysqli->query("SELECT Name FROM City LIMIT 10");
printf("Select returned %d rows.\n", $result->num_rows);

/* If we have to retrieve large amount of data we use MYSQLI_USE_RESULT */
$result = $mysqli->query("SELECT * FROM City", MYSQLI_USE_RESULT);

/* Note, that we can't execute any functions which interact with the
    server until all records have been fully retrieved or the result
    set was closed. All calls will return an 'out of sync' error */
$mysqli->query("SET @a:='this will not work'");
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* Create table doesn't return a resultset */
mysqli_query($link, "CREATE TEMPORARY TABLE myCity LIKE City");
printf("Table myCity successfully created.\n");

/* Select queries return a resultset */
$result = mysqli_query($link, "SELECT Name FROM City LIMIT 10");
printf("Select returned %d rows.\n", mysqli_num_rows($result));

/* If we have to retrieve large amount of data we use MYSQLI_USE_RESULT */
$result = mysqli_query($link, "SELECT * FROM City", MYSQLI_USE_RESULT);

/* Note, that we can't execute any functions which interact with the
    server until all records have been fully retrieved or the result
    set was closed. All calls will return an 'out of sync' error */
mysqli_query($link, "SET @a:='this will not work'");
```

The above examples will output something similar to:

```
Table myCity successfully created.
Select returned 10 rows.

Fatal error: Uncaught mysqli_sql_exception: Commands out of sync; you can't run this command now
in...
```

See Also

- mysqli_execute_query() - Prepares, binds parameters, and executes SQL statement

- mysqli_real_query() - Execute an SQL query
- mysqli_multi_query() - Performs one or more queries on the database
- mysqli_prepare() - Prepares an SQL statement for execution
- mysqli_free_result() - Frees the memory associated with a result

mysqli::real_connect

mysqli::real_connect

mysqli_real_connect

(PHP 5, PHP 7, PHP 8)

mysqli::real_connect -- mysqli_real_connect — Opens a connection to the MySQL server

Description

Object-oriented style

```
public function mysqli::real_connect(
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null,
    int $flags = 0
): bool
```

Procedural style

```
function mysqli_real_connect(
    mysqli $mysql,
    ?string $hostname = null,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?string $database = null,
    ?int $port = null,
    ?string $socket = null,
    int $flags = 0
): bool
```

Opens a connection to the MySQL database server with optional connection options.

This function differs from mysqli_connect():

- **mysqli_real_connect()** needs a valid object which has to be created by mysqli_init().
- There is a `flags` parameter.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`hostname`

Can be either a host name or an IP address. When passing **`null`**, the value is retrieved from mysqli.default_host. When possible, pipes will be used instead of the TCP/IP protocol. The TCP/IP protocol is used if a host name and port number are provided together e.g. `localhost:3308`.

`username`

The MySQL username or **`null`** to assume the username based on the mysqli.default_user ini option.

`password`

The MySQL password or **`null`** to assume the password based on the mysqli.default_pw ini option.

`database`

The default database to be used when performing queries or **`null`**.

`port`

The port number to attempt to connect to the MySQL server or **`null`** to assume the port based on the mysqli.default_port ini option.

`socket`

The socket or named pipe that should be used or **`null`** to assume the socket based on the mysqli.default_socket ini option.

> **Note** :
>
> Specifying the `socket` parameter will not explicitly determine the type of connection to be used when connecting to the MySQL server. How the connection is made to the MySQL database is determined by the `hostname` parameter.

`flags`

With the parameter `flags` you can set different connection options:

Supported flags

Name	Description
`MYSQLI_CLIENT_COMPRESS`	Use compression protocol
`MYSQLI_CLIENT_FOUND_ROWS`	Return the number of matched rows, not the number of affected rows
`MYSQLI_CLIENT_IGNORE_SPACE`	Allow spaces after function names. Makes all function names reserved words.
`MYSQLI_CLIENT_INTERACTIVE`	Allow `interactive_timeout` seconds (instead of `wait_timeout` seconds) of inactivity before closing the connection
`MYSQLI_CLIENT_SSL`	Use SSL (encryption)

Name	Description
MYSQLI_CLIENT_SSL_DONT_VERIFY_SERVER_CERT	Like MYSQLI_CLIENT_SSL, but disables validation of the provided SSL certificate. This is only for installations using MySQL Native Driver and MySQL 5.6 or later.

> **Note** :
>
> For security reasons the MULTI_STATEMENT flag is not supported in PHP. If you want to execute multiple queries use the mysqli_multi_query() function.

Return Values

Returns true on success or false on failure.

Errors/Exceptions

If mysqli error reporting is enabled (MYSQLI_REPORT_ERROR) and the requested operation fails, a warning is generated. If, in addition, the mode is set to MYSQLI_REPORT_STRICT, a mysqli_sql_exception is thrown instead.

Changelog

Version	Description
7.4.0	All parameters are now nullable.

Examples

Example #1 **mysqli::real_connect()** example

Object-oriented style

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_init();

$mysqli->options(MYSQLI_INIT_COMMAND, 'SET AUTOCOMMIT = 0');
$mysqli->options(MYSQLI_OPT_CONNECT_TIMEOUT, 5);

$mysqli->real_connect('localhost', 'my_user', 'my_password', 'my_db', null, null,
MYSQLI_CLIENT_COMPRESS|MYSQLI_CLIENT_FOUND_ROWS);

echo 'Success... ' . $mysqli->host_info . "\n";
```

Object-oriented style when extending mysqli class

```
<?php

class foo_mysqli extends mysqli {
    public function __construct($host, $user, $pass, $db)
    {
        parent::__construct();

        parent::options(MYSQLI_INIT_COMMAND, 'SET AUTOCOMMIT = 0');
        parent::options(MYSQLI_OPT_CONNECT_TIMEOUT, 5);
```

```
        parent::real_connect($host, $user, $pass, $db, null, null,
MYSQLI_CLIENT_COMPRESS|MYSQLI_CLIENT_FOUND_ROWS);
    }
}

$db = new foo_mysqli('localhost', 'my_user', 'my_password', 'my_db');

echo 'Success... ' . $db->host_info . "\n";
```

Procedural style

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_init();

mysqli_options($link, MYSQLI_INIT_COMMAND, 'SET AUTOCOMMIT = 0');
mysqli_options($link, MYSQLI_OPT_CONNECT_TIMEOUT, 5);

mysqli_real_connect($link, 'localhost', 'my_user', 'my_password', 'my_db', null, null,
MYSQLI_CLIENT_COMPRESS|MYSQLI_CLIENT_FOUND_ROWS);

echo 'Success... ' . mysqli_get_host_info($link) . "\n";
```

The above examples will output:

```
Success... MySQL host info: localhost via TCP/IP
```

Notes

> **Note** :
>
> MySQLnd always assumes the server default charset. This charset is sent during connection hand-shake/authentication, which mysqlnd will use.
>
> Libmysqlclient uses the default charset set in the *my.cnf* or by an explicit call to mysqli_options() prior to calling **mysqli_real_connect()**, but after mysqli_init().

See Also

- mysqli_connect() - Open a new connection to the MySQL server
- mysqli_init() - Initializes MySQLi and returns an object for use with mysqli_real_connect()
- mysqli_options() - Set options
- mysqli_ssl_set() - Used for establishing secure connections using SSL
- mysqli_close() - Closes a previously opened database connection

mysqli::real_escape_string

mysqli::real_escape_string

mysqli_real_escape_string

(PHP 5, PHP 7, PHP 8)

mysqli::real_escape_string -- mysqli_real_escape_string — Escapes special characters in a string for use in an SQL statement, taking into account the current charset of the connection

Description

Object-oriented style

public function **mysqli::real_escape_string**(string `$string`): string

Procedural style

function **mysqli_real_escape_string**(mysqli `$mysql`, string `$string`): string

This function is used to create a legal SQL string that you can use in an SQL statement. The given string is encoded to produce an escaped SQL string, taking into account the current character set of the connection.

> **Caution**
>
> **Security: the default character set**
>
> The character set must be set either at the server level, or with the API function mysqli_set_charset() for it to affect **mysqli_real_escape_string()**. See the concepts section on character sets for more information.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`string`

The string to be escaped.

Characters encoded are `NUL (ASCII 0)`, `\n`, `\r`, `\`, `'`, `"`, and `CTRL+Z`.

Return Values

Returns an escaped string.

Examples

Example #1 mysqli::real_escape_string() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$city = "'s-Hertogenbosch";
```

```php
/* this query with escaped $city will work */
$query = sprintf("SELECT CountryCode FROM City WHERE name='%s'",
    $mysqli->real_escape_string($city));
$result = $mysqli->query($query);
printf("Select returned %d rows.\n", $result->num_rows);

/* this query will fail, because we didn't escape $city */
$query = sprintf("SELECT CountryCode FROM City WHERE name='%s'", $city);
$result = $mysqli->query($query);
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$city = "'s-Hertogenbosch";

/* this query with escaped $city will work */
$query = sprintf("SELECT CountryCode FROM City WHERE name='%s'",
    mysqli_real_escape_string($mysqli, $city));
$result = mysqli_query($mysqli, $query);
printf("Select returned %d rows.\n", mysqli_num_rows($result));

/* this query will fail, because we didn't escape $city */
$query = sprintf("SELECT CountryCode FROM City WHERE name='%s'", $city);
$result = mysqli_query($mysqli, $query);
```

The above examples will output something similar to:

```
Select returned 1 rows.

Fatal error: Uncaught mysqli_sql_exception: You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax to use near 's-Hertogenbosch'' at
line 1 in...
```

See Also

- mysqli_set_charset() - Sets the client character set

mysqli::real_query

mysqli::real_query

mysqli_real_query

(PHP 5, PHP 7, PHP 8)

mysqli::real_query -- mysqli_real_query — Execute an SQL query

Description

Object-oriented style

public function **mysqli::real_query**(string `$query`): bool

Procedural style

function **mysqli_real_query**(mysqli `$mysql`, string `$query`): bool

Executes a single query against the database whose result can then be retrieved or stored using the mysqli_store_result() or mysqli_use_result() functions.

> **Warning**
>
> **Security warning: SQL injection**
>
> If the query contains any variable input then parameterized prepared statements should be used instead. Alternatively, the data must be properly formatted and all strings must be escaped using the mysqli_real_escape_string() function.

In order to determine if a given query should return a result set or not, see mysqli_field_count().

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`query`

The query string.

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

See Also

- mysqli_query() - Performs a query on the database
- mysqli_store_result() - Transfers a result set from the last query
- mysqli_use_result() - Initiate a result set retrieval

mysqli::reap_async_query

mysqli::reap_async_query

mysqli_reap_async_query

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli::reap_async_query -- mysqli_reap_async_query — Get result from async query

Description

Object-oriented style

public function **mysqli::reap_async_query**(): mysqli_result|bool

Procedural style

function **mysqli_reap_async_query**(mysqli `$mysql`): mysqli_result|bool

Get result from async query.

> **Note** :
>
> Available only with mysqlnd.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Returns `false` on failure. For successful queries which produce a result set, such as `SELECT, SHOW, DESCRIBE` or `EXPLAIN`, **mysqli_reap_async_query()** will return a mysqli_result object. For other successful queries, **mysqli_reap_async_query()** will return `true`.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

See Also

- mysqli_poll() - Poll connections

mysqli::refresh

mysqli::refresh

mysqli_refresh

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli::refresh -- mysqli_refresh — Refreshes

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.4.0. Relying on this function is highly discouraged.

Description

Object-oriented style

```
#[\Deprecated(
    since: '8.4',
    message: 'use FLUSH SQL statement instead',
)]
public function mysqli::refresh(int $flags): bool
```

Procedural style

```
#[\Deprecated(
    since: '8.4',
    message: 'use FLUSH SQL statement instead',
)]
function mysqli_refresh(mysqli $mysql, int $flags): bool
```

Flushes tables or caches, or resets the replication server information.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`flags`

The options to refresh, using the MYSQLI_REFRESH_* constants as documented within the MySQLi constants documentation.

See also the official » MySQL Refresh documentation.

Return Values

`true` if the refresh was a success, otherwise `false`

Changelog

Version	Description
8.4.0	Both **mysqli::refresh()** and **mysqli_refresh()** are now deprecated. Use `FLUSH` SQL commands instead.

See Also

- mysqli_poll() - Poll connections

mysqli::release_savepoint

mysqli::release_savepoint

mysqli_release_savepoint

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

mysqli::release_savepoint -- mysqli_release_savepoint — Removes the named savepoint from the set of savepoints of the current transaction

Description

Object-oriented style

public function **mysqli::release_savepoint**(string `$name`): bool

Procedural style:

function **mysqli_release_savepoint**(mysqli `$mysql`, string `$name`): bool

This function is identical to executing `$mysqli->query("RELEASE SAVEPOINT `$name`");`. This function does not trigger commit or rollback.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`name`

The identifier of the savepoint.

Return Values

Returns **`true`** on success or **`false`** on failure.

See Also

- mysqli_savepoint() - Set a named transaction savepoint

mysqli::rollback

mysqli::rollback

mysqli_rollback

(PHP 5, PHP 7, PHP 8)

mysqli::rollback -- mysqli_rollback — Rolls back current transaction

Description

Object-oriented style

public function **mysqli::rollback**(int `$flags` = 0, ?string `$name` = **`null`**): bool

Procedural style

function **mysqli_rollback**(mysqli `$mysql`, int `$flags` = 0, ?string `$name` = **`null`**): bool

Rolls back the current transaction for the database.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`flags`

A bitmask of `MYSQLI_TRANS_COR_*` constants.

`name`

If provided then `ROLLBACK/*name*/` is executed.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

Changelog

Version	Description
8.0.0	`name` is now nullable.

Examples

See the mysqli::begin_transaction() example.

Notes

> **Note** :
>
> This function does not work with non transactional table types (like MyISAM or ISAM).

See Also

- mysqli_begin_transaction() - Starts a transaction
- mysqli_commit() - Commits the current transaction
- mysqli_autocommit() - Turns on or off auto-committing database modifications
- mysqli_release_savepoint() - Removes the named savepoint from the set of savepoints of the current transaction

mysqli::savepoint

mysqli::savepoint

mysqli_savepoint

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

mysqli::savepoint -- mysqli_savepoint — Set a named transaction savepoint

Description

Object-oriented style

public function **mysqli::savepoint**(string `$name`): bool

Procedural style:

function **mysqli_savepoint**(mysqli `$mysql`, string `$name`): bool

This function is identical to executing `$mysqli->query("SAVEPOINT `$name`");`

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`name`

The identifier of the savepoint.

Return Values

Returns **`true`** on success or **`false`** on failure.

See Also

- mysqli_release_savepoint() - Removes the named savepoint from the set of savepoints of the current transaction

mysqli::select_db

mysqli::select_db

mysqli_select_db

(PHP 5, PHP 7, PHP 8)

mysqli::select_db -- mysqli_select_db — Selects the default database for database queries

Description

Object-oriented style

public function **mysqli::select_db**(string `$database`): bool

Procedural style

function **mysqli_select_db**(mysqli `$mysql`, string `$database`): bool

Selects the default database to be used when performing queries against the database connection.

> **Note** :
>
> This function should only be used to change the default database for the connection. You can select the default database with the 4th parameter in mysqli_connect().

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`database`

The database name.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

Examples

Example #1 mysqli::select_db() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "test");

/* get the name of the current default database */
$result = $mysqli->query("SELECT DATABASE()");
$row = $result->fetch_row();
printf("Default database is %s.\n", $row[0]);

/* change default database to "world" */
$mysqli->select_db("world");

/* get the name of the current default database */
$result = $mysqli->query("SELECT DATABASE()");
$row = $result->fetch_row();
printf("Default database is %s.\n", $row[0]);
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "test");

/* get the name of the current default database */
$result = mysqli_query($link, "SELECT DATABASE()");
$row = mysqli_fetch_row($result);
printf("Default database is %s.\n", $row[0]);

/* change default database to "world" */
mysqli_select_db($link, "world");

/* get the name of the current default database */
$result = mysqli_query($link, "SELECT DATABASE()");
$row = mysqli_fetch_row($result);
printf("Default database is %s.\n", $row[0]);
```

The above examples will output:

```
Default database is test.
Default database is world.
```

See Also

- mysqli_connect() - Open a new connection to the MySQL server
- mysqli_real_connect() - Opens a connection to the MySQL server

mysqli::set_charset

mysqli::set_charset

mysqli_set_charset

(PHP 5 >= 5.0.5, PHP 7, PHP 8)

mysqli::set_charset -- mysqli_set_charset — Sets the client character set

Description

Object-oriented style

public function **mysqli::set_charset**(string `$charset`): bool

Procedural style

function **mysqli_set_charset**(mysqli `$mysql`, string `$charset`): bool

Sets the character set to be used when sending data from and to the database server.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`charset`

The desired character set.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

Examples

Example #1 mysqli::set_charset() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "test");

printf("Initial character set: %s\n", $mysqli->character_set_name());

/* change character set to utf8mb4 */
$mysqli->set_charset("utf8mb4");

printf("Current character set: %s\n", $mysqli->character_set_name());
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect('localhost', 'my_user', 'my_password', 'test');

printf("Initial character set: %s\n", mysqli_character_set_name($link));

/* change character set to utf8mb4 */
mysqli_set_charset($link, "utf8mb4");

printf("Current character set: %s\n", mysqli_character_set_name($link));
```

The above examples will output something similar to:

```
Initial character set: latin1
Current character set: utf8mb4
```

Notes

> **Note** :
>
> This is the preferred way to change the charset. Using mysqli_query() to set it (such as `SET NAMES utf8`) is not recommended. See the MySQL character set concepts section for more information.

See Also

- mysqli_character_set_name() - Returns the current character set of the database connection
- mysqli_real_escape_string() - Escapes special characters in a string for use in an SQL statement, taking into account the current charset of the connection
- MySQL character set concepts
- » List of character sets that MySQL supports

mysqli::set_opt

mysqli::set_opt

mysqli_set_opt

(PHP 5, PHP 7, PHP 8)

mysqli::set_opt -- mysqli_set_opt — Alias of mysqli_options()

Description

This function is an alias of: mysqli_options().

mysqli::$sqlstate

mysqli::$sqlstate

mysqli_sqlstate

(PHP 5, PHP 7, PHP 8)

mysqli::$sqlstate -- mysqli_sqlstate — Returns the SQLSTATE error from previous MySQL operation

Description

Object-oriented style

string *$mysqli->sqlstate*;

Procedural style

function **mysqli_sqlstate**(mysqli `$mysql`): string

Returns a string containing the SQLSTATE error code for the last error. The error code consists of five characters. `'00000'` means no error. The values are specified by ANSI SQL and ODBC. For a list of possible values, see » http://dev.mysql.com/doc/mysql/en/error-handling.html.

> **Note** :
>
> Note that not all MySQL errors are yet mapped to SQLSTATEs. The value `HY000` (general error) is used for unmapped errors.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Returns a string containing the SQLSTATE error code for the last error. The error code consists of five characters. `'00000'` means no error.

Examples

Example #1 *$mysqli->sqlstate* example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* Table City already exists, so we should get an error */
try {
    $mysqli->query("CREATE TABLE City (ID INT, Name VARCHAR(30))");
} catch (mysqli_sql_exception) {
    printf("Error - SQLSTATE %s.\n", $mysqli->sqlstate);
}
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* Table City already exists, so we should get an error */
try {
    mysqli_query($link, "CREATE TABLE City (ID INT, Name VARCHAR(30))");
} catch (mysqli_sql_exception) {
    printf("Error - SQLSTATE %s.\n", mysqli_sqlstate($link));
}
```

The above examples will output:

```
Error - SQLSTATE 42S01.
```

See Also

- mysqli_errno() - Returns the error code for the most recent function call
- mysqli_error() - Returns a string description of the last error

mysqli::ssl_set

mysqli::ssl_set

mysqli_ssl_set

(PHP 5, PHP 7, PHP 8)

mysqli::ssl_set -- mysqli_ssl_set — Used for establishing secure connections using SSL

Description

Object-oriented style

public function **mysqli::ssl_set**(
    ?string `$key`,
    ?string `$certificate`,
    ?string `$ca_certificate`,
    ?string `$ca_path`,
    ?string `$cipher_algos`
): true

Procedural style

function **mysqli_ssl_set**(
    mysqli `$mysql`,
    ?string `$key`,
    ?string `$certificate`,
    ?string `$ca_certificate`,
    ?string `$ca_path`,
    ?string `$cipher_algos`
): true

Used for establishing secure connections using SSL. It must be called before mysqli_real_connect(). This function does nothing unless OpenSSL support is enabled.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`key`

The path name to the key file.

`certificate`

The path name to the certificate file.

`ca_certificate`

The path name to the certificate authority file.

`ca_path`

The pathname to a directory that contains trusted SSL CA certificates in PEM format.

`cipher_algos`

A list of allowable ciphers to use for SSL encryption.

Return Values

Always returns **`true`**. If SSL setup is incorrect mysqli_real_connect() will return an error when you attempt to connect.

See Also

- mysqli_options() - Set options

- mysqli_real_connect() - Opens a connection to the MySQL server

mysqli::stat

mysqli::stat

mysqli_stat

(PHP 5, PHP 7, PHP 8)

mysqli::stat -- mysqli_stat — Gets the current system status

Description

Object-oriented style

public function **mysqli::stat**(): string|false

Procedural style

function **mysqli_stat**(mysqli `$mysql`): string|false

mysqli_stat() returns a string containing information similar to that provided by the 'mysqladmin status' command. This includes uptime in seconds and the number of running threads, questions, reloads, and open tables.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

A string describing the server status. `false` if an error occurred.

Examples

Example #1 mysqli::stat() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

printf("System status: %s\n", $mysqli->stat());
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

printf("System status: %s\n", mysqli_stat($link));
```

The above examples will output:

```
System status: Uptime: 272  Threads: 1  Questions: 5340  Slow queries: 0
Opens: 13  Flush tables: 1  Open tables: 0  Queries per second avg: 19.632
Memory in use: 8496K  Max memory used: 8560K
```

See Also

- mysqli_get_server_info() - Returns the version of the MySQL server

mysqli::stmt_init

mysqli::stmt_init

mysqli_stmt_init

(PHP 5, PHP 7, PHP 8)

mysqli::stmt_init -- mysqli_stmt_init — Initializes a statement and returns an object for use with mysqli_stmt_prepare

Description

Object-oriented style

public function **mysqli::stmt_init**(): mysqli_stmt|false

Procedural style

function **mysqli_stmt_init**(mysqli `$mysql`): mysqli_stmt|false

Allocates and initializes a statement object suitable for mysqli_stmt_prepare().

> **Note** :
>
> Any subsequent call to any mysqli_stmt method will fail until mysqli_stmt_prepare() is called.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Returns a mysqli_stmt object.

See Also

- mysqli_stmt_prepare() - Prepares an SQL statement for execution

mysqli::store_result

mysqli::store_result

mysqli_store_result

(PHP 5, PHP 7, PHP 8)

mysqli::store_result -- mysqli_store_result — Transfers a result set from the last query

Description

Object-oriented style

public function **mysqli::store_result**(int `$mode` = 0): mysqli_result|false

Procedural style

function **mysqli_store_result**(mysqli `$mysql`, int `$mode` = 0): mysqli_result|false

Transfers the result set from the last query on the database connection represented by the `mysql` parameter to be used with the mysqli_data_seek() function.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`mode`

The option that you want to set. As of PHP 8.1, this parameter has no effect. It can be one of the following values:

Valid options

Name	Description
`MYSQLI_STORE_RESULT_COPY_DATA`	Copy results from the internal mysqlnd buffer into the PHP variables fetched. By default, mysqlnd will use a reference logic to avoid copying and duplicating results held in memory. For certain result sets, for example, result sets with many small rows, the copy approach can reduce the overall memory usage because PHP variables holding results may be released earlier (available with mysqlnd only)

Return Values

Returns a buffered result object or `false` if an error occurred.

> **Note** :
>
> **mysqli_store_result()** returns `false` in case the query didn't return a result set (if the query was, for example an INSERT statement). This function also returns `false` if the reading of the result set failed. You can check if you have got an error by checking if mysqli_error() doesn't return an empty string, if mysqli_errno() returns a non zero value, or if mysqli_field_count() returns a non zero value. Also possible reason for this function returning `false` after successful call to mysqli_query() can be too large result set (memory for it cannot be

> allocated). If mysqli_field_count() returns a non-zero value, the statement should have produced a non-empty result set.

Errors/Exceptions

If mysqli error reporting is enabled (**MYSQLI_REPORT_ERROR**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **MYSQLI_REPORT_STRICT**, a mysqli_sql_exception is thrown instead.

Changelog

Version	Description
8.4.0	Passing the `mode` parameter is now deprecated. The parameter has had no effect as of PHP 8.1.0.

Examples

See mysqli_multi_query().

Notes

> **Note** :
>
> Although it is always good practice to free the memory used by the result of a query using the mysqli_free_result() function, when transferring large result sets using the **mysqli_store_result()** this becomes particularly important.

See Also

- mysqli_real_query() - Execute an SQL query
- mysqli_use_result() - Initiate a result set retrieval

mysqli::$thread_id

mysqli::$thread_id

mysqli_thread_id

(PHP 5, PHP 7, PHP 8)

mysqli::$thread_id -- mysqli_thread_id — Returns the thread ID for the current connection

Description

Object-oriented style

int *$mysqli->thread_id*;

Procedural style

function **mysqli_thread_id**(mysqli `$mysql`): int

The **mysqli_thread_id()** function returns the thread ID for the current connection which can then be killed using the mysqli_kill() function. If the connection is lost and you reconnect with mysqli_ping(), the thread ID will be different. Therefore you should get the thread ID only when you need it.

> **Note** :
>
> The thread ID is assigned on a connection-by-connection basis. Hence, if the connection is broken and then re-established a new thread ID will be assigned.
>
> To kill a running query you can use the SQL command `KILL QUERY processid`.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Returns the Thread ID for the current connection.

Examples

Example #1 *$mysqli->thread_id* example

Object-oriented style

```
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

/* determine our thread id */
$thread_id = $mysqli->thread_id;

/* Kill connection */
$mysqli->kill($thread_id);

/* This should produce an error */
if (!$mysqli->query("CREATE TABLE myCity LIKE City")) {
    printf("Error: %s\n", $mysqli->error);
    exit;
}

/* close connection */
$mysqli->close();
?>
```

Procedural style

```
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
```

```
    exit();
}

/* determine our thread id */
$thread_id = mysqli_thread_id($link);

/* Kill connection */
mysqli_kill($link, $thread_id);

/* This should produce an error */
if (!mysqli_query($link, "CREATE TABLE myCity LIKE City")) {
    printf("Error: %s\n", mysqli_error($link));
    exit;
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Error: MySQL server has gone away
```

See Also

- mysqli_kill() - Asks the server to kill a MySQL thread

mysqli::thread_safe

mysqli::thread_safe

mysqli_thread_safe

(PHP 5, PHP 7, PHP 8)

mysqli::thread_safe -- mysqli_thread_safe — Returns whether thread safety is given or not

Description

Object-oriented style

public function **mysqli::thread_safe**(): bool

Procedural style

function **mysqli_thread_safe**(): bool

Tells whether the client library is compiled as thread-safe.

Parameters

This function has no parameters.

Return Values

`true` if the client library is thread-safe, otherwise `false`.

mysqli::use_result

mysqli::use_result

mysqli_use_result

(PHP 5, PHP 7, PHP 8)

mysqli::use_result -- mysqli_use_result — Initiate a result set retrieval

Description

Object-oriented style

public function **mysqli::use_result**(): mysqli_result|false

Procedural style

function **mysqli_use_result**(mysqli `$mysql`): mysqli_result|false

Used to initiate the retrieval of a result set from the last query executed using the mysqli_real_query() function on the database connection.

Either this or the mysqli_store_result() function must be called before the results of a query can be retrieved, and one or the other must be called to prevent the next query on that database connection from failing.

> **Note** :
>
> The **mysqli_use_result()** function does not transfer the entire result set from the database and hence cannot be used with functions such as mysqli_data_seek() to move to a particular row within the set. To use this functionality, the result set must be stored using mysqli_store_result(). One should not use **mysqli_use_result()** if a lot of processing on the client side is performed, since this will tie up the server and prevent other threads from updating any tables from which the data is being fetched.

Parameters

This function has no parameters.

Return Values

Returns an unbuffered result object or `false` if an error occurred.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

Examples

Example #1 mysqli::use_result() example

Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query  = "SELECT CURRENT_USER();";
$query .= "SELECT Name FROM City ORDER BY ID LIMIT 20, 5";

/* execute multi query */
if ($mysqli->multi_query($query)) {
    do {
        /* store first result set */
        if ($result = $mysqli->use_result()) {
            while ($row = $result->fetch_row()) {
                printf("%s\n", $row[0]);
            }
            $result->close();
        }
        /* print divider */
        if ($mysqli->more_results()) {
            printf("-----------------\n");
        }
    } while ($mysqli->next_result());
}

/* close connection */
$mysqli->close();
?>
```

Procedural style

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query  = "SELECT CURRENT_USER();";
$query .= "SELECT Name FROM City ORDER BY ID LIMIT 20, 5";

/* execute multi query */
if (mysqli_multi_query($link, $query)) {
    do {
        /* store first result set */
        if ($result = mysqli_use_result($link)) {
            while ($row = mysqli_fetch_row($result)) {
                printf("%s\n", $row[0]);
            }
            mysqli_free_result($result);
```

```
        }
        /* print divider */
        if (mysqli_more_results($link)) {
            printf("-----------------\n");
        }
    } while (mysqli_next_result($link));
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
my_user@localhost
-----------------
Amersfoort
Maastricht
Dordrecht
Leiden
Haarlemmermeer
```

See Also

- mysqli_real_query() - Execute an SQL query
- mysqli_store_result() - Transfers a result set from the last query

mysqli::$warning_count

mysqli::$warning_count

mysqli_warning_count

(PHP 5, PHP 7, PHP 8)

mysqli::$warning_count -- mysqli_warning_count — Returns the number of warnings generated by the most recently executed query

Description

Object-oriented style

int *$mysqli->warning_count*;

Procedural style

function **mysqli_warning_count**(mysqli `$mysql`): int

Returns the number of warnings generated by the most recently executed query.

> **Note** :

> For retrieving warning messages the following SQL command can be used: `SHOW WARNINGS [limit row_count]`.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

Return Values

Number of warnings or zero if there are no warnings.

Examples

Example #1 *$mysqli->warning_count* example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$mysqli->query("SELECT 42/0");

if ($mysqli->warning_count > 0) {
    $result = $mysqli->query("SHOW WARNINGS");
    $row = $result->fetch_row();
    printf("%s (%d): %s\n", $row[0], $row[1], $row[2]);
}
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

mysqli_query($link, "SELECT 42/0");

if (mysqli_warning_count($link) > 0) {
    $result = mysqli_query($link, "SHOW WARNINGS");
    $row = mysqli_fetch_row($result);
    printf("%s (%d): %s\n", $row[0], $row[1], $row[2]);
}
```

The above examples will output:

```
Warning (1365): Division by 0
```

See Also

- mysqli_errno() - Returns the error code for the most recent function call
- mysqli_error() - Returns a string description of the last error
- mysqli_sqlstate() - Returns the SQLSTATE error from previous MySQL operation

mysqli_stmt

(PHP 5, PHP 7, PHP 8)

Introduction

Represents a prepared statement.

Class synopsis

```
class mysqli_stmt {
/* Properties */
public readonly int|string $affected_rows;
public readonly int|string $insert_id;
public readonly int|string $num_rows;
public readonly int $param_count;
public readonly int $field_count;
public readonly int $errno;
public readonly string $error;
public readonly array $error_list;
public readonly string $sqlstate;
public int $id;
/* Methods */
public function __construct(mysqli $mysql, ?string $query = null)
public function attr_get(int $attribute): int
public function attr_set(int $attribute, int $value): bool
public function bind_param(string $types, mixed &$var, mixed &...$vars): bool
public function bind_result(mixed &$var, mixed &...$vars): bool
public function close(): true
public function data_seek(int $offset): void
public function execute(?array $params = null): bool
public function fetch(): ?bool
public function free_result(): void
public function get_result(): mysqli_result|false
public function get_warnings(): mysqli_warning|false
public function more_results(): bool
public function next_result(): bool
public function num_rows(): int|string
public function prepare(string $query): bool
public function reset(): bool
public function result_metadata(): mysqli_result|false
public function send_long_data(int $param_num, string $data): bool
public function store_result(): bool
}
```

Properties

id

Stores the statement ID.

Table of Contents

mysqli_stmt::$affected_rows

mysqli_stmt::$affected_rows

mysqli_stmt_affected_rows

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$affected_rows -- mysqli_stmt_affected_rows — Returns the total number of rows changed, deleted, inserted, or matched by the last statement executed

Description

Object-oriented style

int|string *$mysqli_stmt->affected_rows*;

Procedural style

function **mysqli_stmt_affected_rows**(mysqli_stmt `$statement`): int|string

Returns the number of rows affected by the `INSERT`, `UPDATE`, or `DELETE` query. Works like mysqli_stmt_num_rows() for `SELECT` statements.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

An integer greater than zero indicates the number of rows affected or retrieved. Zero indicates that no records were updated for an `UPDATE` statement, no rows matched the `WHERE` clause in the query or that no query has yet been executed. `-1` indicates that the query returned an error or that, for a `SELECT` query, **mysqli_stmt_affected_rows()** was called prior to calling mysqli_stmt_store_result().

> **Note** :
>
> If the number of affected rows is greater than the maximum PHP int value, the number of affected rows will be returned as a string value.

Examples

Example #1 mysqli_stmt_affected_rows() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* create temp table */
$mysqli->query("CREATE TEMPORARY TABLE myCountry LIKE Country");

$query = "INSERT INTO myCountry SELECT * FROM Country WHERE Code LIKE ?";

/* prepare statement */
$stmt = $mysqli->prepare($query);

/* Bind variable for placeholder */
$code = 'A%';
$stmt->bind_param("s", $code);

/* execute statement */
$stmt->execute();

printf("Rows inserted: %d\n", $stmt->affected_rows);
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* create temp table */
mysqli_query($link, "CREATE TEMPORARY TABLE myCountry LIKE Country");

$query = "INSERT INTO myCountry SELECT * FROM Country WHERE Code LIKE ?";

/* prepare statement */
$stmt = mysqli_prepare($link, $query);

/* Bind variable for placeholder */
$code = 'A%';
mysqli_stmt_bind_param($stmt, "s", $code);

/* execute statement */
mysqli_stmt_execute($stmt);

printf("Rows inserted: %d\n", mysqli_stmt_affected_rows($stmt));
```

The above examples will output:

```
Rows inserted: 17
```

See Also

- mysqli_stmt_num_rows() - Returns the number of rows fetched from the server
- mysqli_stmt_store_result() - Stores a result set in an internal buffer

mysqli_stmt::attr_get

mysqli_stmt::attr_get

mysqli_stmt_attr_get

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::attr_get -- mysqli_stmt_attr_get — Used to get the current value of a statement attribute

Description

Object-oriented style

public function **mysqli_stmt::attr_get**(int `$attribute`): int

Procedural style

function **mysqli_stmt_attr_get**(mysqli_stmt `$statement`, int `$attribute`): int

Gets the current value of a statement attribute.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

`attribute`

The attribute that you want to get.

Return Values

Returns the value of the attribute.

mysqli_stmt::attr_set

mysqli_stmt::attr_set

mysqli_stmt_attr_set

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::attr_set -- mysqli_stmt_attr_set — Used to modify the behavior of a prepared statement

Description

Object-oriented style

public function **mysqli_stmt::attr_set**(int `$attribute`, int `$value`): bool

Procedural style

function **mysqli_stmt_attr_set**(mysqli_stmt `$statement`, int `$attribute`, int `$value`): bool

Used to modify the behavior of a prepared statement. This function may be called multiple times to set several attributes.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

`attribute`

The attribute that you want to set. It can have one of the following values:

Attribute values

Character	Description
MYSQLI_STMT_ATTR_UPDATE_MAX_LENGTH	Setting to `true` causes mysqli_stmt_store_result() to update the metadata `MYSQL_FIELD->max_length` value.
MYSQLI_STMT_ATTR_CURSOR_TYPE	Type of cursor to open for statement when mysqli_stmt_execute() is invoked. `value` can be `MYSQLI_CURSOR_TYPE_NO_CURSOR` (the default) or `MYSQLI_CURSOR_TYPE_READ_ONLY`.
MYSQLI_STMT_ATTR_PREFETCH_ROWS	Number of rows to fetch from server at a time when using a cursor. `value` can be in the range from 1 to the maximum value of unsigned long. The default is 1. Removed as of PHP 8.4.0.

If you use the `MYSQLI_STMT_ATTR_CURSOR_TYPE` option with `MYSQLI_CURSOR_TYPE_READ_ONLY`, a cursor is opened for the statement when you invoke mysqli_stmt_execute(). If there is already an open cursor from a previous mysqli_stmt_execute() call, it closes the cursor before opening a new one. mysqli_stmt_reset() also closes any open cursor before preparing the statement for re-execution. mysqli_stmt_free_result() closes any open cursor.

If you open a cursor for a prepared statement, mysqli_stmt_store_result() is unnecessary.

`value`

The value to assign to the attribute.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

See Also

- » Connector/MySQL mysql_stmt_attr_set()

mysqli_stmt::bind_param

mysqli_stmt::bind_param

mysqli_stmt_bind_param

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::bind_param -- mysqli_stmt_bind_param — Binds variables to a prepared statement as parameters

Description

Object-oriented style

public function **mysqli_stmt::bind_param**(string `$types`, mixed `&$var`, mixed `&...$vars`): bool

Procedural style

function **mysqli_stmt_bind_param**(
  mysqli_stmt `$statement`,
  string `$types`,
  mixed `&$var`,
  mixed `&...$vars`
): bool

Bind variables for the parameter markers in the SQL statement prepared by mysqli_prepare() or mysqli_stmt_prepare().

> **Note** :
>
> If data size of a variable exceeds max. allowed packet size (max_allowed_packet), you have to specify `b` in `types` and use mysqli_stmt_send_long_data() to send the data in packets.

> **Note** :
>
> Care must be taken when using **mysqli_stmt_bind_param()** in conjunction with call_user_func_array(). Note that **mysqli_stmt_bind_param()** requires parameters to be passed by reference, whereas call_user_func_array() can accept as a parameter a list of variables that can represent references or values.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

`types`

A string that contains one or more characters which specify the types for the corresponding bind variables:

Type specification chars

Character	Description
i	corresponding variable has type int
d	corresponding variable has type float
s	corresponding variable has type string
b	corresponding variable is a blob and will be sent in packets

`var`
`vars`

The number of variables and length of string `types` must match the parameters in the statement.

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

Examples

Example #1 **mysqli_stmt::bind_param()** example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli('localhost', 'my_user', 'my_password', 'world');

$stmt = $mysqli->prepare("INSERT INTO CountryLanguage VALUES (?, ?, ?, ?)");
$stmt->bind_param('sssd', $code, $language, $official, $percent);

$code = 'DEU';
$language = 'Bavarian';
$official = "F";
$percent = 11.2;

$stmt->execute();

printf("%d row inserted.\n", $stmt->affected_rows);

/* Clean up table CountryLanguage */
$mysqli->query("DELETE FROM CountryLanguage WHERE Language='Bavarian'");
printf("%d row deleted.\n", $mysqli->affected_rows);
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect('localhost', 'my_user', 'my_password', 'world');

$stmt = mysqli_prepare($link, "INSERT INTO CountryLanguage VALUES (?, ?, ?, ?)");
mysqli_stmt_bind_param($stmt, 'sssd', $code, $language, $official, $percent);

$code = 'DEU';
$language = 'Bavarian';
$official = "F";
$percent = 11.2;

mysqli_stmt_execute($stmt);
```

```php
printf("%d row inserted.\n", mysqli_stmt_affected_rows($stmt));

/* Clean up table CountryLanguage */
mysqli_query($link, "DELETE FROM CountryLanguage WHERE Language='Bavarian'");
printf("%d row deleted.\n", mysqli_affected_rows($link));
```

The above examples will output:

```
1 row inserted.
1 row deleted.
```

Example #2 Using ... to provide arguments

The ... operator can be used to provide variable-length argument list, e.g. in a `WHERE IN` clause.

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli('localhost', 'my_user', 'my_password', 'world');

$stmt = $mysqli->prepare("SELECT Language FROM CountryLanguage WHERE CountryCode IN (?, ?)");
/* Using ... to provide arguments */
$stmt->bind_param('ss', ...['DEU', 'POL']);
$stmt->execute();
$stmt->store_result();

printf("%d rows found.\n", $stmt->num_rows());
```

The above examples will output:

```
10 rows found.
```

See Also

- mysqli_stmt_bind_result() - Binds variables to a prepared statement for result storage
- mysqli_stmt_execute() - Executes a prepared statement
- mysqli_stmt_fetch() - Fetch results from a prepared statement into the bound variables
- mysqli_prepare() - Prepares an SQL statement for execution
- mysqli_stmt_send_long_data() - Send data in blocks
- mysqli_stmt_errno() - Returns the error code for the most recent statement call
- mysqli_stmt_error() - Returns a string description for last statement error

mysqli_stmt::bind_result

mysqli_stmt::bind_result

mysqli_stmt_bind_result

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::bind_result -- mysqli_stmt_bind_result — Binds variables to a prepared statement for result storage

Description

Object-oriented style

public function **mysqli_stmt::bind_result**(mixed `&$var`, mixed `&...$vars`): bool

Procedural style

function **mysqli_stmt_bind_result**(mysqli_stmt `$statement`, mixed `&$var`, mixed `&...$vars`): bool

Binds columns in the result set to variables.

When mysqli_stmt_fetch() is called to fetch data, the MySQL client/server protocol places the data for the bound columns into the specified variables `var`/`vars`.

A column can be bound or rebound at any time, even after a result set has been partially retrieved. The new binding takes effect the next time mysqli_stmt_fetch() is called.

> **Note** :
>
> All columns must be bound after mysqli_stmt_execute() and prior to calling mysqli_stmt_fetch().

> **Note** :
>
> Depending on column types bound variables can silently change to the corresponding PHP type.

> **Tip**
>
> This function is useful for simple results. To retrieve an iterable result set, or fetch each row as an array or object, use mysqli_stmt_get_result().

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

`var`

The first variable to be bound.

`vars`

Further variables to be bound.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* prepare statement */
$stmt = $mysqli->prepare("SELECT Code, Name FROM Country ORDER BY Name LIMIT 5");
$stmt->execute();

/* bind variables to prepared statement */
$stmt->bind_result($col1, $col2);

/* fetch values */
while ($stmt->fetch()) {
    printf("%s %s\n", $col1, $col2);
}
```

Example #2 Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* prepare statement */
$stmt = mysqli_prepare($link, "SELECT Code, Name FROM Country ORDER BY Name LIMIT 5");
mysqli_stmt_execute($stmt);

/* bind variables to prepared statement */
mysqli_stmt_bind_result($stmt, $col1, $col2);

/* fetch values */
while (mysqli_stmt_fetch($stmt)) {
    printf("%s %s\n", $col1, $col2);
}
```

The above examples will output something similar to:

```
AFG Afghanistan
ALB Albania
DZA Algeria
ASM American Samoa
AND Andorra
```

See Also

- mysqli_stmt_get_result() - Gets a result set from a prepared statement as a mysqli_result object
- mysqli_stmt_bind_param() - Binds variables to a prepared statement as parameters
- mysqli_stmt_execute() - Executes a prepared statement
- mysqli_stmt_fetch() - Fetch results from a prepared statement into the bound variables
- mysqli_prepare() - Prepares an SQL statement for execution
- mysqli_stmt_prepare() - Prepares an SQL statement for execution

mysqli_stmt::close

mysqli_stmt::close

mysqli_stmt_close

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::close -- mysqli_stmt_close — Closes a prepared statement

Description

Object-oriented style

public function **mysqli_stmt::close**(): true

Procedural style

function **mysqli_stmt_close**(mysqli_stmt `$statement`): true

Closes a prepared statement. **mysqli_stmt_close()** also deallocates the statement handle. If the current statement has pending or unread results, this function cancels them so that the next query can be executed.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

Always returns `true`.

Changelog

Version	Description
8.0.0	This function now always returns `true`. Previously it returned `false` on failure.

See Also

- mysqli_prepare() - Prepares an SQL statement for execution

mysqli_stmt::__construct

mysqli_stmt::__construct

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::__construct — Constructs a new mysqli_stmt object

Description

public function **mysqli_stmt::__construct**(mysqli `$mysql`, ?string `$query` = **`null`**)

This method constructs a new mysqli_stmt object.

Parameters

`link`

A valid mysqli object.

`query`

The query, as a string. If this parameter is **`null`**, then the constructor behaves identically to mysqli_stmt_init(), otherwise it behaves as per mysqli_prepare().

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

Changelog

Version	Description
8.0.0	`query` is now nullable.

See Also

- mysqli_prepare() - Prepares an SQL statement for execution
- mysqli_stmt_init() - Initializes a statement and returns an object for use with mysqli_stmt_prepare

mysqli_stmt::data_seek

mysqli_stmt::data_seek

mysqli_stmt_data_seek

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::data_seek -- mysqli_stmt_data_seek — Adjusts the result pointer to an arbitrary row in the buffered result

Description

Object-oriented style

public function **mysqli_stmt::data_seek**(int `$offset`): void

Procedural style

function **mysqli_stmt_data_seek**(mysqli_stmt `$statement`, int `$offset`): void

This function moves the result set pointer of the buffered result set to an arbitrary row specified by the `offset` parameter.

This function works only on the buffered internal result set. mysqli_stmt_store_result() must be called prior to **mysqli_stmt_data_seek()**.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

`offset`

Must be between zero and the total number of rows minus one (0.. mysqli_stmt_num_rows() - 1).

Return Values

No value is returned.

Examples

Example #1 Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name";
$stmt = $mysqli->prepare($query);
$stmt->execute();

$stmt->bind_result($name, $code);

$stmt->store_result();

/* seek to row no. 400 */
$stmt->data_seek(399);

$stmt->fetch();

printf("City: %s  Countrycode: %s\n", $name, $code);
```

Example #2 Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name";
$stmt = mysqli_prepare($link, $query);

mysqli_stmt_execute($stmt);

mysqli_stmt_bind_result($stmt, $name, $code);

mysqli_stmt_store_result($stmt);

/* seek to row no. 400 */
```

```
mysqli_stmt_data_seek($stmt, 399);

mysqli_stmt_fetch($stmt);

printf("City: %s  Countrycode: %s\n", $name, $code);
```

The above examples will output:

```
City: Benin City  Countrycode: NGA
```

See Also

- mysqli_prepare() - Prepares an SQL statement for execution
- mysqli_stmt_store_result() - Stores a result set in an internal buffer
- mysqli_stmt_num_rows() - Returns the number of rows fetched from the server
- mysqli_data_seek() - Adjusts the result pointer to an arbitrary row in the result

mysqli_stmt::$errno

mysqli_stmt::$errno

mysqli_stmt_errno

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$errno -- mysqli_stmt_errno — Returns the error code for the most recent statement call

Description

Object-oriented style

int *$mysqli_stmt->errno*;

Procedural style

function **mysqli_stmt_errno**(mysqli_stmt `$statement`): int

Returns the error code for the most recently invoked statement function that can succeed or fail.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

An error code value. Zero means no error occurred.

Examples

Example #1 Object-oriented style

```php
<?php
/* Open a connection */
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$mysqli->query("CREATE TABLE myCountry LIKE Country");
$mysqli->query("INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = $mysqli->prepare($query)) {

    /* drop table */
    $mysqli->query("DROP TABLE myCountry");

    /* execute query */
    $stmt->execute();

    printf("Error: %d.\n", $stmt->errno);

    /* close statement */
    $stmt->close();
}

/* close connection */
$mysqli->close();
?>
```

Example #2 Procedural style

```php
<?php
/* Open a connection */
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

mysqli_query($link, "CREATE TABLE myCountry LIKE Country");
mysqli_query($link, "INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = mysqli_prepare($link, $query)) {

    /* drop table */
    mysqli_query($link, "DROP TABLE myCountry");

    /* execute query */
    mysqli_stmt_execute($stmt);
```

```
    printf("Error: %d.\n", mysqli_stmt_errno($stmt));

    /* close statement */
    mysqli_stmt_close($stmt);
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Error: 1146.
```

See Also

- mysqli_stmt_error() - Returns a string description for last statement error
- mysqli_stmt_sqlstate() - Returns SQLSTATE error from previous statement operation

mysqli_stmt::$error

mysqli_stmt::$error

mysqli_stmt_error

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$error -- mysqli_stmt_error — Returns a string description for last statement error

Description

Object-oriented style

string *$mysqli_stmt->error*;

Procedural style

function **mysqli_stmt_error**(mysqli_stmt `$statement`): string

Returns a string containing the error message for the most recently invoked statement function that can succeed or fail.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

A string that describes the error. An empty string if no error occurred.

Examples

Example #1 Object-oriented style

```php
<?php
/* Open a connection */
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$mysqli->query("CREATE TABLE myCountry LIKE Country");
$mysqli->query("INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = $mysqli->prepare($query)) {

    /* drop table */
    $mysqli->query("DROP TABLE myCountry");

    /* execute query */
    $stmt->execute();

    printf("Error: %s.\n", $stmt->error);

    /* close statement */
    $stmt->close();
}

/* close connection */
$mysqli->close();
?>
```

Example #2 Procedural style

```php
<?php
/* Open a connection */
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

mysqli_query($link, "CREATE TABLE myCountry LIKE Country");
mysqli_query($link, "INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = mysqli_prepare($link, $query)) {

    /* drop table */
    mysqli_query($link, "DROP TABLE myCountry");
```

```
    /* execute query */
    mysqli_stmt_execute($stmt);

    printf("Error: %s.\n", mysqli_stmt_error($stmt));

    /* close statement */
    mysqli_stmt_close($stmt);
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Error: Table 'world.myCountry' doesn't exist.
```

See Also

- mysqli_stmt_errno() - Returns the error code for the most recent statement call
- mysqli_stmt_sqlstate() - Returns SQLSTATE error from previous statement operation

mysqli_stmt::$error_list

mysqli_stmt::$error_list

mysqli_stmt_error_list

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

mysqli_stmt::$error_list -- mysqli_stmt_error_list — Returns a list of errors from the last statement executed

Description

Object-oriented style

array *$mysqli_stmt->error_list*;

Procedural style

function **mysqli_stmt_error_list**(mysqli_stmt `$statement`): array

Returns an array of errors for the most recently invoked statement function that can succeed or fail.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

A list of errors, each as an associative array containing the errno, error, and sqlstate.

Examples

Example #1 Object-oriented style

```php
<?php
/* Open a connection */
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$mysqli->query("CREATE TABLE myCountry LIKE Country");
$mysqli->query("INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = $mysqli->prepare($query)) {

    /* drop table */
    $mysqli->query("DROP TABLE myCountry");

    /* execute query */
    $stmt->execute();

    echo "Error:\n";
    print_r($stmt->error_list);

    /* close statement */
    $stmt->close();
}

/* close connection */
$mysqli->close();
?>
```

Example #2 Procedural style

```php
<?php
/* Open a connection */
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

mysqli_query($link, "CREATE TABLE myCountry LIKE Country");
mysqli_query($link, "INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = mysqli_prepare($link, $query)) {
```

```
    /* drop table */
    mysqli_query($link, "DROP TABLE myCountry");

    /* execute query */
    mysqli_stmt_execute($stmt);

    echo "Error:\n";
    print_r(mysqli_stmt_error_list($stmt));

    /* close statement */
    mysqli_stmt_close($stmt);
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Error:
Array
(
    [0] => Array
        (
            [errno] => 1146
            [sqlstate] => 42S02
            [error] => Table 'world.myCountry' doesn't exist
        )

)
```

See Also

- mysqli_stmt_error() - Returns a string description for last statement error
- mysqli_stmt_errno() - Returns the error code for the most recent statement call
- mysqli_stmt_sqlstate() - Returns SQLSTATE error from previous statement operation

mysqli_stmt::execute

mysqli_stmt::execute

mysqli_stmt_execute

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::execute -- mysqli_stmt_execute — Executes a prepared statement

Description

Object-oriented style

public function **mysqli_stmt::execute**(?array `$params` = `null`): bool

Procedural style

function **mysqli_stmt_execute**(mysqli_stmt `$statement`, ?array `$params` = `null`): bool

Executes a previously prepared statement. The statement must be successfully prepared prior to execution, using either the mysqli_prepare() or mysqli_stmt_prepare() function, or by passing the second argument to mysqli_stmt::__construct().

If the statement is `UPDATE`, `DELETE`, or `INSERT`, the total number of affected rows can be determined by using the mysqli_stmt_affected_rows() function. If the query yields a result set, it can be fetched using mysqli_stmt_get_result() function or by fetching it row by row directly from the statement using mysqli_stmt_fetch() function.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

`params`

An optional list array with as many elements as there are bound parameters in the SQL statement being executed. Each value is treated as a string.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

Changelog

Version	Description
8.1.0	The optional `params` parameter has been added.

Examples

Example #1 Execute a prepared statement with bound variables

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$mysqli->query("CREATE TEMPORARY TABLE myCity LIKE City");

/* Prepare an insert statement */
$stmt = $mysqli->prepare("INSERT INTO myCity (Name, CountryCode, District) VALUES (?,?,?)");

/* Bind variables to parameters */
```

```php
$stmt->bind_param("sss", $val1, $val2, $val3);

$val1 = 'Stuttgart';
$val2 = 'DEU';
$val3 = 'Baden-Wuerttemberg';

/* Execute the statement */
$stmt->execute();

$val1 = 'Bordeaux';
$val2 = 'FRA';
$val3 = 'Aquitaine';

/* Execute the statement */
$stmt->execute();

/* Retrieve all rows from myCity */
$query = "SELECT Name, CountryCode, District FROM myCity";
$result = $mysqli->query($query);
while ($row = $result->fetch_row()) {
    printf("%s (%s,%s)\n", $row[0], $row[1], $row[2]);
}
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

mysqli_query($link, "CREATE TEMPORARY TABLE myCity LIKE City");

/* Prepare an insert statement */
$stmt = mysqli_prepare($link, "INSERT INTO myCity (Name, CountryCode, District) VALUES (?,?,?)");

/* Bind variables to parameters */
mysqli_stmt_bind_param($stmt, "sss", $val1, $val2, $val3);

$val1 = 'Stuttgart';
$val2 = 'DEU';
$val3 = 'Baden-Wuerttemberg';

/* Execute the statement */
mysqli_stmt_execute($stmt);

$val1 = 'Bordeaux';
$val2 = 'FRA';
$val3 = 'Aquitaine';

/* Execute the statement */
mysqli_stmt_execute($stmt);

/* Retrieve all rows from myCity */
$query = "SELECT Name, CountryCode, District FROM myCity";
$result = mysqli_query($link, $query);
while ($row = mysqli_fetch_row($result)) {
    printf("%s (%s,%s)\n", $row[0], $row[1], $row[2]);
}
```

The above examples will output:

```
Stuttgart (DEU,Baden-Wuerttemberg)
Bordeaux (FRA,Aquitaine)
```

Example #2 Execute a prepared statement with an array of values

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli('localhost', 'my_user', 'my_password', 'world');

$mysqli->query('CREATE TEMPORARY TABLE myCity LIKE City');

/* Prepare an insert statement */
$stmt = $mysqli->prepare('INSERT INTO myCity (Name, CountryCode, District) VALUES (?,?,?)');

/* Execute the statement */
$stmt->execute(['Stuttgart', 'DEU', 'Baden-Wuerttemberg']);

/* Retrieve all rows from myCity */
$query = 'SELECT Name, CountryCode, District FROM myCity';
$result = $mysqli->query($query);
while ($row = $result->fetch_row()) {
    printf("%s (%s,%s)\n", $row[0], $row[1], $row[2]);
}
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect('localhost', 'my_user', 'my_password', 'world');

mysqli_query($link, 'CREATE TEMPORARY TABLE myCity LIKE City');

/* Prepare an insert statement */
$stmt = mysqli_prepare($link, 'INSERT INTO myCity (Name, CountryCode, District) VALUES (?,?,?)');

/* Execute the statement */
mysqli_stmt_execute($stmt, ['Stuttgart', 'DEU', 'Baden-Wuerttemberg']);

/* Retrieve all rows from myCity */
$query = 'SELECT Name, CountryCode, District FROM myCity';
$result = mysqli_query($link, $query);
while ($row = mysqli_fetch_row($result)) {
    printf("%s (%s,%s)\n", $row[0], $row[1], $row[2]);
}
```

The above examples will output:

```
Stuttgart (DEU,Baden-Wuerttemberg)
```

See Also

- mysqli_execute_query() - Prepares, binds parameters, and executes SQL statement
- mysqli_prepare() - Prepares an SQL statement for execution
- mysqli_stmt_bind_param() - Binds variables to a prepared statement as parameters
- mysqli_stmt_get_result() - Gets a result set from a prepared statement as a mysqli_result object

mysqli_stmt::fetch

mysqli_stmt::fetch

mysqli_stmt_fetch

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::fetch -- mysqli_stmt_fetch — Fetch results from a prepared statement into the bound variables

Description

Object-oriented style

public function **mysqli_stmt::fetch**(): ?bool

Procedural style

function **mysqli_stmt_fetch**(mysqli_stmt `$statement`): ?bool

Fetch the result from a prepared statement into the variables bound by mysqli_stmt_bind_result().

> **Note** :
>
> Note that all columns must be bound by the application before calling **mysqli_stmt_fetch()**.

> **Note** :
>
> Data is transferred unbuffered without calling mysqli_stmt_store_result() which can decrease performance (but reduces memory cost).

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

Return Values

Value	Description
`true`	Success. Data has been fetched
`false`	Error occurred
`null`	No more rows/data exists or data truncation occurred

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

Examples

Example #1 Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, CountryCode FROM City ORDER by ID DESC LIMIT 150,5";

if ($stmt = $mysqli->prepare($query)) {

    /* execute statement */
    $stmt->execute();

    /* bind result variables */
    $stmt->bind_result($name, $code);

    /* fetch values */
    while ($stmt->fetch()) {
        printf ("%s (%s)\n", $name, $code);
    }

    /* close statement */
    $stmt->close();
}

/* close connection */
$mysqli->close();
?>
```

Example #2 Procedural style

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
```

```php
}

$query = "SELECT Name, CountryCode FROM City ORDER by ID DESC LIMIT 150,5";

if ($stmt = mysqli_prepare($link, $query)) {

    /* execute statement */
    mysqli_stmt_execute($stmt);

    /* bind result variables */
    mysqli_stmt_bind_result($stmt, $name, $code);

    /* fetch values */
    while (mysqli_stmt_fetch($stmt)) {
        printf ("%s (%s)\n", $name, $code);
    }

    /* close statement */
    mysqli_stmt_close($stmt);
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Rockford (USA)
Tallahassee (USA)
Salinas (USA)
Santa Clarita (USA)
Springfield (USA)
```

See Also

- mysqli_prepare() - Prepares an SQL statement for execution
- mysqli_stmt_errno() - Returns the error code for the most recent statement call
- mysqli_stmt_error() - Returns a string description for last statement error
- mysqli_stmt_bind_result() - Binds variables to a prepared statement for result storage

mysqli_stmt::$field_count

mysqli_stmt::$field_count

mysqli_stmt_field_count

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$field_count -- mysqli_stmt_field_count — Returns the number of columns in the given statement

Description

Object-oriented style

int *$mysqli_stmt->field_count*;

Procedural style

function **mysqli_stmt_field_count**(mysqli_stmt `$statement`): int

Returns the number of columns in the prepared statement.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

Returns an integer representing the number of columns.

Examples

Example #1 Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$code = 'FR';

$stmt = $mysqli->prepare("SELECT Name FROM Country WHERE Code=?");
$stmt->bind_param('s', $code);
$stmt->execute();
$row = $stmt->get_result()->fetch_row();
for ($i = 0; $i < $stmt->field_count; $i++) {
    printf("Value of column number %d is %s", $i, $row[$i]);
}
```

Example #2 Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$code = 'FR';

$stmt = mysqli_prepare($mysqli, "SELECT Name FROM Country WHERE Code=?");
mysqli_stmt_bind_param($stmt, 's', $code);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);
$row = mysqli_fetch_row($result);
for ($i = 0; $i < mysqli_stmt_field_count($stmt); $i++) {
    printf("Value of column number %d is %s", $i, $row[$i]);
}
```

The above examples will output something similar to:

```
Value of column number 0 is France
```

See Also

- mysqli_stmt_num_rows() - Returns the number of rows fetched from the server

mysqli_stmt::free_result

mysqli_stmt::free_result

mysqli_stmt_free_result

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::free_result -- mysqli_stmt_free_result — Frees stored result memory for the given statement handle

Description

Object-oriented style

public function **mysqli_stmt::free_result**(): void

Procedural style

function **mysqli_stmt_free_result**(mysqli_stmt `$statement`): void

Frees the result memory associated with the statement, which was allocated by mysqli_stmt_store_result().

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

No value is returned.

See Also

- mysqli_stmt_store_result() - Stores a result set in an internal buffer

mysqli_stmt::get_result

mysqli_stmt::get_result

mysqli_stmt_get_result

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli_stmt::get_result -- mysqli_stmt_get_result — Gets a result set from a prepared statement as a mysqli_result object

Description

Object-oriented style

public function **mysqli_stmt::get_result**(): mysqli_result|false

Procedural style

function **mysqli_stmt_get_result**(mysqli_stmt `$statement`): mysqli_result|false

Retrieves a result set from a prepared statement as a mysqli_result object. The data will be fetched from the MySQL server to PHP. This method should be called only for queries which produce a result set.

> **Note** :
>
> Available only with mysqlnd.

> **Note** :
>
> This function cannot be used together with mysqli_stmt_store_result(). Both of these functions retrieve the full result set from the MySQL server.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

Returns **`false`** on failure. For successful queries which produce a result set, such as `SELECT, SHOW, DESCRIBE` or `EXPLAIN`, **mysqli_stmt_get_result()** will return a mysqli_result object. For other successful queries, **mysqli_stmt_get_result()** will return **`false`**. The mysqli_stmt_errno() function can be used to distinguish between the two reasons for **`false`**; due to a bug, prior to PHP 7.4.13, mysqli_errno() had to be used for this purpose.

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

Examples

Example #1 Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, Population, Continent FROM Country WHERE Continent=? ORDER BY Name LIMIT 1";

$stmt = $mysqli->prepare($query);
$stmt->bind_param("s", $continent);
```

```php
$continentList = array('Europe', 'Africa', 'Asia', 'North America');

foreach ($continentList as $continent) {
    $stmt->execute();
    $result = $stmt->get_result();
    while ($row = $result->fetch_array(MYSQLI_NUM)) {
        foreach ($row as $r) {
            print "$r ";
        }
        print "\n";
    }
}
```

Example #2 Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, Population, Continent FROM Country WHERE Continent=? ORDER BY Name LIMIT 1";

$stmt = mysqli_prepare($link, $query);
mysqli_stmt_bind_param($stmt, "s", $continent);

$continentList= array('Europe', 'Africa', 'Asia', 'North America');

foreach ($continentList as $continent) {
    mysqli_stmt_execute($stmt);
    $result = mysqli_stmt_get_result($stmt);
    while ($row = mysqli_fetch_array($result, MYSQLI_NUM)) {
        foreach ($row as $r) {
            print "$r ";
        }
        print "\n";
    }
}
```

The above examples will output something similar to:

```
Albania 3401200 Europe
Algeria 31471000 Africa
Afghanistan 22720000 Asia
Anguilla 8000 North America
```

See Also

- mysqli_prepare() - Prepares an SQL statement for execution
- mysqli_stmt_result_metadata() - Returns result set metadata from a prepared statement
- mysqli_stmt_fetch() - Fetch results from a prepared statement into the bound variables
- mysqli_fetch_array() - Fetch the next row of a result set as an associative, a numeric array, or both

- mysqli_stmt_store_result() - Stores a result set in an internal buffer

mysqli_stmt::get_warnings

mysqli_stmt::get_warnings

mysqli_stmt_get_warnings

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

mysqli_stmt::get_warnings -- mysqli_stmt_get_warnings — Get result of SHOW WARNINGS

Description

Object-oriented style

public function **mysqli_stmt::get_warnings**(): mysqli_warning|false

Procedural style

function **mysqli_stmt_get_warnings**(mysqli_stmt `$statement`): mysqli_warning|false

> **Warning**
>
> This function is currently not documented; only its argument list is available.

mysqli_stmt::$insert_id

mysqli_stmt::$insert_id

mysqli_stmt_insert_id

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$insert_id -- mysqli_stmt_insert_id — Get the ID generated from the previous INSERT operation

Description

Object-oriented style

int|string *$mysqli_stmt->insert_id*;

Procedural style

function **mysqli_stmt_insert_id**(mysqli_stmt `$statement`): int|string

> **Warning**
>
> This function is currently not documented; only its argument list is available.

mysqli_stmt::more_results

mysqli_stmt::more_results

mysqli_stmt_more_results

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli_stmt::more_results -- mysqli_stmt_more_results — Check if there are more query results from a multiple query

Description

Object-oriented style

public function **mysqli_stmt::more_results**(): bool

Procedural style:

function **mysqli_stmt_more_results**(mysqli_stmt `$statement`): bool

Checks if there are more query results from a multiple query.

> **Note** :
>
> Available only with mysqlnd.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

Returns `true` if more results exist, otherwise `false`.

See Also

- mysqli_stmt::next_result() - Reads the next result from a multiple query
- mysqli::multi_query() - Performs one or more queries on the database

mysqli_stmt::next_result

mysqli_stmt::next_result

mysqli_stmt_next_result

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli_stmt::next_result -- mysqli_stmt_next_result — Reads the next result from a multiple query

Description

Object-oriented style

public function **mysqli_stmt::next_result**(): bool

Procedural style:

function **mysqli_stmt_next_result**(mysqli_stmt `$statement`): bool

Reads the next result from a multiple query.

> **Note** :
>
> Prior to PHP 8.1.0, available only with mysqlnd.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

Changelog

Version	Description
8.1.0	Now also available when linking against libmysqlclient.

See Also

- mysqli_stmt::more_results() - Check if there are more query results from a multiple query
- mysqli::multi_query() - Performs one or more queries on the database

mysqli_stmt::$num_rows

mysqli_stmt::$num_rows

mysqli_stmt::num_rows

mysqli_stmt_num_rows

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$num_rows -- mysqli_stmt::num_rows -- mysqli_stmt_num_rows — Returns the number of rows fetched from the server

Description

Object-oriented style

int|string *$mysqli_stmt->num_rows*;
public function **mysqli_stmt::num_rows**(): int|string

Procedural style

function **mysqli_stmt_num_rows**(mysqli_stmt `$statement`): int|string

Returns the number of rows buffered in the statement. This function will only work after mysqli_stmt_store_result() is called to buffer the entire result set in the statement handle.

This function returns `0` unless all rows have been fetched from the server.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

An int representing the number of buffered rows. Returns `0` in unbuffered mode unless all rows have been fetched from the server.

> **Note** :
>
> If the number of rows is greater than **`PHP_INT_MAX`**, the number will be returned as a string.

Examples

Example #1 Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name LIMIT 20";
$stmt = $mysqli->prepare($query);
$stmt->execute();

/* store the result in an internal buffer */
$stmt->store_result();

printf("Number of rows: %d.\n", $stmt->num_rows);
```

Example #2 Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name LIMIT 20";
$stmt = mysqli_prepare($link, $query);
mysqli_stmt_execute($stmt);

/* store the result in an internal buffer */
mysqli_stmt_store_result($stmt);

printf("Number of rows: %d.\n", mysqli_stmt_num_rows($stmt));
```

The above examples will output:

```
Number of rows: 20.
```

See Also

- mysqli_stmt_store_result() - Stores a result set in an internal buffer
- mysqli_stmt_affected_rows() - Returns the total number of rows changed, deleted, inserted, or matched by the last statement executed
- mysqli_prepare() - Prepares an SQL statement for execution

mysqli_stmt::$param_count

mysqli_stmt::$param_count

mysqli_stmt_param_count

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$param_count -- mysqli_stmt_param_count — Returns the number of parameters for the given statement

Description

Object-oriented style

int *$mysqli_stmt->param_count*;

Procedural style

function **mysqli_stmt_param_count**(mysqli_stmt `$statement`): int

Returns the number of parameter markers present in the prepared statement.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

Returns an integer representing the number of parameters.

Examples

Example #1 Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
```

```
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

if ($stmt = $mysqli->prepare("SELECT Name FROM Country WHERE Name=? OR Code=?")) {

    $marker = $stmt->param_count;
    printf("Statement has %d markers.\n", $marker);

    /* close statement */
    $stmt->close();
}

/* close connection */
$mysqli->close();
?>
```

Example #2 Procedural style

```
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

if ($stmt = mysqli_prepare($link, "SELECT Name FROM Country WHERE Name=? OR Code=?")) {

    $marker = mysqli_stmt_param_count($stmt);
    printf("Statement has %d markers.\n", $marker);

    /* close statement */
    mysqli_stmt_close($stmt);
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Statement has 2 markers.
```

See Also

- mysqli_prepare() - Prepares an SQL statement for execution

mysqli_stmt::prepare

mysqli_stmt::prepare

mysqli_stmt_prepare

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::prepare -- mysqli_stmt_prepare — Prepares an SQL statement for execution

Description

Object-oriented style

public function **mysqli_stmt::prepare**(string `$query`): bool

Procedural style

function **mysqli_stmt_prepare**(mysqli_stmt `$statement`, string `$query`): bool

Prepares a statement for execution. The query must consist of a single SQL statement.

The statement template can contain zero or more question mark (`?`) parameter markers—also called placeholders. The parameter markers must be bound to application variables using mysqli_stmt_bind_param() before executing the statement.

> **Note** :
>
> In the case where a statement is passed to **mysqli_stmt_prepare()** that is longer than `max_allowed_packet` of the server, the returned error codes are different depending on whether you are using MySQL Native Driver (`mysqlnd`) or MySQL Client Library (`libmysqlclient`). The behavior is as follows:
>
> - `mysqlnd` on Linux returns an error code of 1153. The error message means “got a packet bigger than `max_allowed_packet` bytes”.
> - `mysqlnd` on Windows returns an error code 2006. This error message means “server has gone away”.
> - `libmysqlclient` on all platforms returns an error code 2006. This error message means “server has gone away”.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

`query`

The query, as a string. It must consist of a single SQL statement.

The SQL statement may contain zero or more parameter markers represented by question mark (`?`) characters at the appropriate positions.

> **Note** :
>
> The markers are legal only in certain places in SQL statements. For example, they are permitted in the `VALUES()` list of an `INSERT` statement (to specify column values for a row), or in a comparison with a

> column in a `WHERE` clause to specify a comparison value. However, they are not permitted for identifiers (such as table or column names).

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

Examples

Example #1 mysqli_stmt::prepare() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$city = "Amersfoort";

/* create a prepared statement */
$stmt = $mysqli->stmt_init();
$stmt->prepare("SELECT District FROM City WHERE Name=?");

/* bind parameters for markers */
$stmt->bind_param("s", $city);

/* execute query */
$stmt->execute();

/* bind result variables */
$stmt->bind_result($district);

/* fetch value */
$stmt->fetch();

printf("%s is in district %s\n", $city, $district);
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$city = "Amersfoort";

/* create a prepared statement */
$stmt = mysqli_stmt_init($link);
mysqli_stmt_prepare($stmt, "SELECT District FROM City WHERE Name=?");

/* bind parameters for markers */
```

```php
mysqli_stmt_bind_param($stmt, "s", $city);

/* execute query */
mysqli_stmt_execute($stmt);

/* bind result variables */
mysqli_stmt_bind_result($stmt, $district);

/* fetch value */
mysqli_stmt_fetch($stmt);

printf("%s is in district %s\n", $city, $district);
```

The above examples will output:

```
Amersfoort is in district Utrecht
```

See Also

- mysqli_stmt_init() - Initializes a statement and returns an object for use with mysqli_stmt_prepare
- mysqli_stmt_execute() - Executes a prepared statement
- mysqli_stmt_fetch() - Fetch results from a prepared statement into the bound variables
- mysqli_stmt_bind_param() - Binds variables to a prepared statement as parameters
- mysqli_stmt_bind_result() - Binds variables to a prepared statement for result storage
- mysqli_stmt_get_result() - Gets a result set from a prepared statement as a mysqli_result object
- mysqli_stmt_close() - Closes a prepared statement

mysqli_stmt::reset

mysqli_stmt::reset

mysqli_stmt_reset

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::reset -- mysqli_stmt_reset — Resets a prepared statement

Description

Object-oriented style

public function **mysqli_stmt::reset**(): bool

Procedural style

function **mysqli_stmt_reset**(mysqli_stmt `$statement`): bool

Resets a prepared statement on client and server to the state after prepare.

It resets the statement on the server, data sent using mysqli_stmt_send_long_data(), unbuffered result sets and current errors. It does not clear bindings or stored result sets. Stored result sets will be cleared when executing

the prepared statement (or closing it).

To prepare a statement with another query use function mysqli_stmt_prepare().

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

Returns `true` on success or `false` on failure.

See Also

- mysqli_prepare() - Prepares an SQL statement for execution

mysqli_stmt::result_metadata

mysqli_stmt::result_metadata

mysqli_stmt_result_metadata

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::result_metadata -- mysqli_stmt_result_metadata — Returns result set metadata from a prepared statement

Description

Object-oriented style

public function **mysqli_stmt::result_metadata**(): mysqli_result|false

Procedural style

function **mysqli_stmt_result_metadata**(mysqli_stmt `$statement`): mysqli_result|false

If a statement passed to mysqli_prepare() is one that produces a result set, **mysqli_stmt_result_metadata()** returns the result object that can be used to process the meta information such as total number of fields and individual field information.

This function returns an empty mysqli_result object which can be used to access metadata information from the prepared statement without having to fetch the actual rows of data. There is no need to use this function when using mysqli_stmt_get_result() to retrieve the entire result set from a prepared statement as a result object.

> **Note** :
>
> This result set object can be passed as an argument only to the field-based functions that process result set metadata, such as:
>
> - mysqli_num_fields()
> - mysqli_fetch_field()
> - mysqli_fetch_field_direct()

- mysqli_fetch_fields()
- mysqli_field_count()
- mysqli_field_seek()
- mysqli_field_tell()
- mysqli_free_result()

> **Note** :
>
> The result set returned by **mysqli_stmt_result_metadata()** contains only metadata. It does not contain any row results. The rows are obtained by calling mysqli_stmt_get_result() on the statement handle or with mysqli_stmt_fetch().

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

Returns a result object or **`false`** if an error occurred. If the statement does not produce a result set, **`false`** is returned as well.

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

Examples

Example #1 Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "test");

$mysqli->query("DROP TABLE IF EXISTS friends");
$mysqli->query("CREATE TABLE friends (id int, name varchar(20))");

$mysqli->query("INSERT INTO friends VALUES (1,'Hartmut'), (2, 'Ulf')");

$stmt = $mysqli->prepare("SELECT id, name FROM friends");
$stmt->execute();

/* get resultset for metadata */
$result = $stmt->result_metadata();

/* retrieve field information from metadata result set */
$field = $result->fetch_field();

printf("Fieldname: %s\n", $field->name);
?>
```

Example #2 Procedural style

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "test");

mysqli_query($link, "DROP TABLE IF EXISTS friends");
mysqli_query($link, "CREATE TABLE friends (id int, name varchar(20))");

mysqli_query($link, "INSERT INTO friends VALUES (1,'Hartmut'), (2, 'Ulf')");

$stmt = mysqli_prepare($link, "SELECT id, name FROM friends");
mysqli_stmt_execute($stmt);

/* get resultset for metadata */
$result = mysqli_stmt_result_metadata($stmt);

/* retrieve field information from metadata result set */
$field = mysqli_fetch_field($result);

printf("Fieldname: %s\n", $field->name);
?>
```

See Also

- mysqli_prepare() - Prepares an SQL statement for execution
- mysqli_free_result() - Frees the memory associated with a result
- mysqli_stmt_get_result() - Gets a result set from a prepared statement as a mysqli_result object

mysqli_stmt::send_long_data

mysqli_stmt::send_long_data

mysqli_stmt_send_long_data

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::send_long_data -- mysqli_stmt_send_long_data — Send data in blocks

Description

Object-oriented style

public function **mysqli_stmt::send_long_data**(int `$param_num`, string `$data`): bool

Procedural style

function **mysqli_stmt_send_long_data**(mysqli_stmt `$statement`, int `$param_num`, string `$data`): bool

Allows sending parameter data to the server in pieces (or chunks), e.g. if the size of a blob exceeds the size of `max_allowed_packet`. This function can be called multiple times to send the parts of a character or binary data value for a column, which must be one of the TEXT or BLOB datatypes.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

`param_num`

Indicates which parameter to associate the data with. Parameters are numbered beginning with 0.

`data`

A string containing data to be sent.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Object-oriented style

```
<?php
$stmt = $mysqli->prepare("INSERT INTO messages (message) VALUES (?)");
$null = NULL;
$stmt->bind_param("b", $null);
$fp = fopen("messages.txt", "r");
while (!feof($fp)) {
    $stmt->send_long_data(0, fread($fp, 8192));
}
fclose($fp);
$stmt->execute();
?>
```

See Also

- mysqli_prepare() - Prepares an SQL statement for execution
- mysqli_stmt_bind_param() - Binds variables to a prepared statement as parameters

mysqli_stmt::$sqlstate

mysqli_stmt::$sqlstate

mysqli_stmt_sqlstate

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::$sqlstate -- mysqli_stmt_sqlstate — Returns SQLSTATE error from previous statement operation

Description

Object-oriented style

string *$mysqli_stmt->sqlstate*;

Procedural style

function **mysqli_stmt_sqlstate**(mysqli_stmt `$statement`): string

Returns a string containing the SQLSTATE error code for the most recently invoked prepared statement function that can succeed or fail. The error code consists of five characters. `'00000'` means no error. The values are specified by ANSI SQL and ODBC. For a list of possible values, see » http://dev.mysql.com/doc/mysql/en/error-handling.html.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

Returns a string containing the SQLSTATE error code for the last error. The error code consists of five characters. `'00000'` means no error.

Examples

Example #1 Object-oriented style

```php
<?php
/* Open a connection */
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$mysqli->query("CREATE TABLE myCountry LIKE Country");
$mysqli->query("INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = $mysqli->prepare($query)) {

    /* drop table */
    $mysqli->query("DROP TABLE myCountry");

    /* execute query */
    $stmt->execute();

    printf("Error: %s.\n", $stmt->sqlstate);

    /* close statement */
    $stmt->close();
}

/* close connection */
$mysqli->close();
?>
```

Example #2 Procedural style

```php
<?php
/* Open a connection */
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
```

```
}

mysqli_query($link, "CREATE TABLE myCountry LIKE Country");
mysqli_query($link, "INSERT INTO myCountry SELECT * FROM Country");


$query = "SELECT Name, Code FROM myCountry ORDER BY Name";
if ($stmt = mysqli_prepare($link, $query)) {

    /* drop table */
    mysqli_query($link, "DROP TABLE myCountry");

    /* execute query */
    mysqli_stmt_execute($stmt);

    printf("Error: %s.\n", mysqli_stmt_sqlstate($stmt));

    /* close statement */
    mysqli_stmt_close($stmt);
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Error: 42S02.
```

Notes

> **Note** :
>
> Note that not all MySQL errors are yet mapped to SQLSTATEs. The value `HY000` (general error) is used for unmapped errors.

See Also

- mysqli_stmt_errno() - Returns the error code for the most recent statement call
- mysqli_stmt_error() - Returns a string description for last statement error

mysqli_stmt::store_result

mysqli_stmt::store_result

mysqli_stmt_store_result

(PHP 5, PHP 7, PHP 8)

mysqli_stmt::store_result -- mysqli_stmt_store_result — Stores a result set in an internal buffer

Description

Object-oriented style

public function **mysqli_stmt::store_result**(): bool

Procedural style

function **mysqli_stmt_store_result**(mysqli_stmt `$statement`): bool

This function should be called for queries that successfully produce a result set (e.g. `SELECT, SHOW, DESCRIBE, EXPLAIN`) only if the complete result set needs to be buffered in PHP. Each subsequent mysqli_stmt_fetch() call will return buffered data.

> **Note** :
>
> It is unnecessary to call **mysqli_stmt_store_result()** for other queries, but if you do, it will not harm or cause any notable performance loss in all cases. You can detect whether the query produced a result set by checking if mysqli_stmt_result_metadata() returns **`false`**.

Parameters

`statement`

Procedural style only: A mysqli_stmt object returned by mysqli_stmt_init().

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

If mysqli error reporting is enabled (**`MYSQLI_REPORT_ERROR`**) and the requested operation fails, a warning is generated. If, in addition, the mode is set to **`MYSQLI_REPORT_STRICT`**, a mysqli_sql_exception is thrown instead.

Examples

Example #1 Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name LIMIT 20";
$stmt = $mysqli->prepare($query);
$stmt->execute();

/* store the result in an internal buffer */
$stmt->store_result();

printf("Number of rows: %d.\n", $stmt->num_rows);
```

Example #2 Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name LIMIT 20";
$stmt = mysqli_prepare($link, $query);
mysqli_stmt_execute($stmt);

/* store the result in an internal buffer */
mysqli_stmt_store_result($stmt);

printf("Number of rows: %d.\n", mysqli_stmt_num_rows($stmt));
```

The above examples will output:

```
Number of rows: 20.
```

See Also

- mysqli_prepare() - Prepares an SQL statement for execution
- mysqli_stmt_result_metadata() - Returns result set metadata from a prepared statement
- mysqli_stmt_fetch() - Fetch results from a prepared statement into the bound variables

mysqli_result

(PHP 5, PHP 7, PHP 8)

Introduction

Represents the result set obtained from a query against the database.

Class synopsis

class **mysqli_result** implements IteratorAggregate {
/* Properties */
public readonly int *$current_field*;
public readonly int *$field_count*;
public readonly ?array *$lengths*;
public readonly int|string *$num_rows*;
public int *$type*;
/* Methods */
public function __construct(mysqli `$mysql`, int `$result_mode` = **`MYSQLI_STORE_RESULT`**)
public function data_seek(int `$offset`): bool
public function fetch_all(int `$mode` = **`MYSQLI_NUM`**): array
public function fetch_array(int `$mode` = **`MYSQLI_BOTH`**): array|null|false
public function fetch_assoc(): array|null|false
public function fetch_column(int `$column` = 0): null|int|float|string|false
public function fetch_field(): object|false

public function fetch_field_direct(int $index): object|false
public function fetch_fields(): array
public function fetch_object(string $class = "stdClass", array $constructor_args = []): object|null|false
public function fetch_row(): array|null|false
public function field_seek(int $index): true
public function free(): void
public function close(): void
public function free_result(): void
public function getIterator(): Iterator
}

Properties

type

Stores whether the result is buffered or unbuffered as an int (**MYSQLI_STORE_RESULT** or **MYSQLI_USE_RESULT**, respectively).

Changelog

Version	Description
8.0.0	**mysqli_result** implements IteratorAggregate now. Previously, Traversable was implemented instead.

Table of Contents

mysqli_result::__construct

mysqli_result::__construct

(PHP 5, PHP 7, PHP 8)

mysqli_result::__construct — Constructs a mysqli_result object

Description

public function **mysqli_result::__construct**(mysqli $mysql, int $result_mode = **MYSQLI_STORE_RESULT**)

This method constructs a new mysqli_result object.

It can be used to create the mysqli_result object after calling the mysqli_real_query() or mysqli_multi_query() function. Constructing the object manually is equivalent to calling the mysqli_store_result() or mysqli_use_result() function.

Parameters

`mysql`

Procedural style only: A mysqli object returned by mysqli_connect() or mysqli_init()

`result_mode`

The result mode can be one of 2 constants indicating how the result will be returned from the MySQL server.

MYSQLI_STORE_RESULT (default) - creates a mysqli_result object with buffered result set.

`MYSQLI_USE_RESULT` - creates a mysqli_result object with unbuffered result set. As long as there are pending records waiting to be fetched, the connection line will be busy and all subsequent calls will return error `Commands out of sync`. To avoid the error all records must be fetched from the server or the result set must be discarded by calling mysqli_free_result(). The connection must remain open for the rows to be fetched.

Errors/Exceptions

If mysqli error reporting is enabled (`MYSQLI_REPORT_ERROR`) and the requested operation fails, a warning is generated. If, in addition, the mode is set to `MYSQLI_REPORT_STRICT`, a mysqli_sql_exception is thrown instead.

Examples

Example #1 Creation of a mysqli_result object

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* Select queries return a result set */
$mysqli->real_query("SELECT Name FROM City LIMIT 10");

$result = new mysqli_result($mysqli);
printf("Select returned %d rows.\n", $result->num_rows);
```

The above examples will output something similar to:

```
Select returned 10 rows.
```

See Also

- mysqli_multi_query() - Performs one or more queries on the database
- mysqli_real_query() - Execute an SQL query
- mysqli_store_result() - Transfers a result set from the last query
- mysqli_use_result() - Initiate a result set retrieval

mysqli_result::$current_field

mysqli_result::$current_field

mysqli_field_tell

(PHP 5, PHP 7, PHP 8)

mysqli_result::$current_field -- mysqli_field_tell — Get current field offset of a result pointer

Description

Object-oriented style

int *$mysqli_result->current_field*;

Procedural style

function **mysqli_field_tell**(mysqli_result `$result`): int

Returns the position of the field cursor used for the last mysqli_fetch_field() call. This value can be used as an argument to mysqli_field_seek().

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

Return Values

Returns current offset of field cursor.

Examples

Example #1 Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Code LIMIT 5";

if ($result = $mysqli->query($query)) {

    /* Get field information for all columns */
    while ($finfo = $result->fetch_field()) {

        /* get fieldpointer offset */
        $currentfield = $result->current_field;

        printf("Column %d:\n", $currentfield);
        printf("Name:     %s\n", $finfo->name);
        printf("Table:    %s\n", $finfo->table);
        printf("max. Len: %d\n", $finfo->max_length);
        printf("Flags:    %d\n", $finfo->flags);
        printf("Type:     %d\n\n", $finfo->type);
    }
    $result->close();
}

/* close connection */
$mysqli->close();
?>
```

Example #2 Procedural style

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Code LIMIT 5";

if ($result = mysqli_query($link, $query)) {

    /* Get field information for all fields */
    while ($finfo = mysqli_fetch_field($result)) {

        /* get fieldpointer offset */
        $currentfield = mysqli_field_tell($result);

        printf("Column %d:\n", $currentfield);
        printf("Name:     %s\n", $finfo->name);
        printf("Table:    %s\n", $finfo->table);
        printf("max. Len: %d\n", $finfo->max_length);
        printf("Flags:    %d\n", $finfo->flags);
        printf("Type:     %d\n\n", $finfo->type);
    }
    mysqli_free_result($result);
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Column 1:
Name:     Name
Table:    Country
max. Len: 11
Flags:    1
Type:     254

Column 2:
Name:     SurfaceArea
Table:    Country
max. Len: 10
Flags:    32769
Type:     4
```

See Also

- mysqli_fetch_field() - Returns the next field in the result set

- mysqli_field_seek() - Set result pointer to a specified field offset

mysqli_result::data_seek

mysqli_result::data_seek

mysqli_data_seek

(PHP 5, PHP 7, PHP 8)

mysqli_result::data_seek -- mysqli_data_seek — Adjusts the result pointer to an arbitrary row in the result

Description

Object-oriented style

public function **mysqli_result::data_seek**(int `$offset`): bool

Procedural style

function **mysqli_data_seek**(mysqli_result `$result`, int `$offset`): bool

The **mysqli_data_seek()** function seeks to an arbitrary result pointer specified by the `offset` in the result set.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

`offset`

The row offset. Must be between zero and the total number of rows minus one (0..mysqli_num_rows() - 1).

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 mysqli::data_seek() example

Object-oriented style

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name";
$result = $mysqli->query($query);

/* Seek to row no. 401 */
$result->data_seek(400);

/* Fetch single row */
$row = $result->fetch_row();
```

```
printf("City: %s  Countrycode: %s\n", $row[0], $row[1]);
```

Procedural style

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name";

$result = mysqli_query($link, $query);

/* Seek to row no. 401 */
mysqli_data_seek($result, 400);

/* Fetch single row */
$row = mysqli_fetch_row($result);

printf ("City: %s  Countrycode: %s\n", $row[0], $row[1]);
```

The above examples will output:

```
City: Benin City  Countrycode: NGA
```

Example #2 Adjusting the result pointer when iterating

This function can be useful when iterating over the result set to impose a custom order or rewind the result set when iterating multiple times.

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY Name LIMIT 15,4";
$result = $mysqli->query($query);

/* Iterate the result set in reverse order */
for ($row_no = $result->num_rows - 1; $row_no >= 0; $row_no--) {
    $result->data_seek($row_no);

    /* Fetch single row */
    $row = $result->fetch_row();

    printf("City: %s  Countrycode: %s\n", $row[0], $row[1]);
}

/* Reset pointer to the beginning of the result set */
$result->data_seek(0);
```

```
print "\n";

/* Iterate the same result set again */
while ($row = $result->fetch_row()) {
    printf("City: %s  Countrycode: %s\n", $row[0], $row[1]);
}
```

The above examples will output:

```
City: Acmbaro  Countrycode: MEX
City: Abuja  Countrycode: NGA
City: Abu Dhabi  Countrycode: ARE
City: Abottabad  Countrycode: PAK

City: Abottabad  Countrycode: PAK
City: Abu Dhabi  Countrycode: ARE
City: Abuja  Countrycode: NGA
City: Acmbaro  Countrycode: MEX
```

Notes

> **Note** :
>
> This function can only be used with buffered results attained from the use of the mysqli_store_result(), mysqli_query() or mysqli_stmt_get_result() functions.

See Also

- mysqli_store_result() - Transfers a result set from the last query
- mysqli_fetch_row() - Fetch the next row of a result set as an enumerated array
- mysqli_fetch_array() - Fetch the next row of a result set as an associative, a numeric array, or both
- mysqli_fetch_assoc() - Fetch the next row of a result set as an associative array
- mysqli_fetch_object() - Fetch the next row of a result set as an object
- mysqli_query() - Performs a query on the database
- mysqli_num_rows() - Gets the number of rows in the result set

mysqli_result::fetch_all

mysqli_result::fetch_all

mysqli_fetch_all

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli_result::fetch_all -- mysqli_fetch_all — Fetch all result rows as an associative array, a numeric array, or both

Description

Object-oriented style

public function **mysqli_result::fetch_all**(int `$mode` = `MYSQLI_NUM`): array

Procedural style

function **mysqli_fetch_all**(mysqli_result `$result`, int `$mode` = `MYSQLI_NUM`): array

Returns a two-dimensional array of all result rows as an associative array, a numeric array, or both.

> **Note** :
>
> Prior to PHP 8.1.0, available only with mysqlnd.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

`mode`

This optional parameter is a constant indicating what type of array should be produced from the current row data. The possible values for this parameter are the constants `MYSQLI_ASSOC`, `MYSQLI_NUM`, or `MYSQLI_BOTH`.

Return Values

Returns an array of associative or numeric arrays holding result rows.

Changelog

Version	Description
8.1.0	Now also available when linking against libmysqlclient.

Examples

Example #1 **mysqli_result::fetch_all()** example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$result = $mysqli->query("SELECT Name, CountryCode FROM City ORDER BY ID LIMIT 3");

$rows = $result->fetch_all(MYSQLI_ASSOC);
foreach ($rows as $row) {
    printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
}
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$result = mysqli_query($mysqli, "SELECT Name, CountryCode FROM City ORDER BY ID LIMIT 3");

$rows = mysqli_fetch_all($result, MYSQLI_ASSOC);
foreach ($rows as $row) {
    printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
}
```

The above examples will output:

```
Kabul (AFG)
Qandahar (AFG)
Herat (AFG)
```

See Also

- mysqli_fetch_array() - Fetch the next row of a result set as an associative, a numeric array, or both
- mysqli_fetch_column() - Fetch a single column from the next row of a result set
- mysqli_query() - Performs a query on the database

mysqli_result::fetch_array

mysqli_result::fetch_array

mysqli_fetch_array

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_array -- mysqli_fetch_array — Fetch the next row of a result set as an associative, a numeric array, or both

Description

Object-oriented style

public function **mysqli_result::fetch_array**(int `$mode` = **`MYSQLI_BOTH`**): array|null|false

Procedural style

function **mysqli_fetch_array**(mysqli_result `$result`, int `$mode` = **`MYSQLI_BOTH`**): array|null|false

Fetches one row of data from the result set and returns it as an array. Each subsequent call to this function will return the next row within the result set, or **`null`** if there are no more rows.

In addition to storing the data in the numeric indices of the result array, this function can also store the data in associative indices by using the field names of the result set as keys.

If two or more columns of the result have the same name, the last column will take precedence and overwrite any previous data. To access multiple columns with the same name, the numerically indexed version of the row must be used.

> **Note** :
>
> Field names returned by this function are *case-sensitive*.

> **Note** :
>
> This function sets NULL fields to the PHP `null` value.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

`mode`

This optional parameter is a constant indicating what type of array should be produced from the current row data. The possible values for this parameter are the constants `MYSQLI_ASSOC`, `MYSQLI_NUM`, or `MYSQLI_BOTH`.

By using the `MYSQLI_ASSOC` constant this function will behave identically to the mysqli_fetch_assoc(), while `MYSQLI_NUM` will behave identically to the mysqli_fetch_row() function. The final option `MYSQLI_BOTH` will create a single array with the attributes of both.

Return Values

Returns an array representing the fetched row, `null` if there are no more rows in the result set, or `false` on failure.

Examples

Example #1 mysqli_result::fetch_array() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID LIMIT 3";
$result = $mysqli->query($query);

/* numeric array */
$row = $result->fetch_array(MYSQLI_NUM);
printf("%s (%s)\n", $row[0], $row[1]);

/* associative array */
$row = $result->fetch_array(MYSQLI_ASSOC);
printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
```

```
/* associative and numeric array */
$row = $result->fetch_array(MYSQLI_BOTH);
printf("%s (%s)\n", $row[0], $row["CountryCode"]);
```

Procedural style

```
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER by ID LIMIT 3";
$result = mysqli_query($mysqli, $query);

/* numeric array */
$row = mysqli_fetch_array($result, MYSQLI_NUM);
printf("%s (%s)\n", $row[0], $row[1]);

/* associative array */
$row = mysqli_fetch_array($result, MYSQLI_ASSOC);
printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);

/* associative and numeric array */
$row = mysqli_fetch_array($result, MYSQLI_BOTH);
printf("%s (%s)\n", $row[0], $row["CountryCode"]);
```

The above examples will output something similar to:

```
Kabul (AFG)
Qandahar (AFG)
Herat (AFG)
```

See Also

- mysqli_fetch_assoc() - Fetch the next row of a result set as an associative array
- mysqli_fetch_column() - Fetch a single column from the next row of a result set
- mysqli_fetch_row() - Fetch the next row of a result set as an enumerated array
- mysqli_fetch_object() - Fetch the next row of a result set as an object
- mysqli_query() - Performs a query on the database
- mysqli_data_seek() - Adjusts the result pointer to an arbitrary row in the result

mysqli_result::fetch_assoc

mysqli_result::fetch_assoc

mysqli_fetch_assoc

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_assoc -- mysqli_fetch_assoc — Fetch the next row of a result set as an associative array

Description

Object-oriented style

public function **mysqli_result::fetch_assoc**(): array|null|false

Procedural style

function **mysqli_fetch_assoc**(mysqli_result `$result`): array|null|false

Fetches one row of data from the result set and returns it as an associative array. Each subsequent call to this function will return the next row within the result set, or `null` if there are no more rows.

If two or more columns of the result have the same name, the last column will take precedence and overwrite any previous data. To access multiple columns with the same name, mysqli_fetch_row() may be used to fetch the numerically indexed array, or aliases may be used in the SQL query select list to give columns different names.

> **Note** :
>
> Field names returned by this function are *case-sensitive*.

> **Note** :
>
> This function sets NULL fields to the PHP `null` value.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

Return Values

Returns an associative array representing the fetched row, where each key in the array represents the name of one of the result set's columns, `null` if there are no more rows in the result set, or `false` on failure.

Examples

Example #1 mysqli_result::fetch_assoc() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID DESC";

$result = $mysqli->query($query);

/* fetch associative array */
while ($row = $result->fetch_assoc()) {
```

```
    printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
}
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID DESC";

$result = mysqli_query($mysqli, $query);

/* fetch associative array */
while ($row = mysqli_fetch_assoc($result)) {
    printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
}
```

The above examples will output something similar to:

```
Pueblo (USA)
Arvada (USA)
Cape Coral (USA)
Green Bay (USA)
Santa Clara (USA)
```

Example #2 Comparison of mysqli_result iterator and mysqli_result::fetch_assoc() usage

mysqli_result can be iterated using `foreach`. The result set will always be iterated from the first row, regardless of the current position.

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = 'SELECT Name, CountryCode FROM City ORDER BY ID DESC';

// Using iterators
$result = $mysqli->query($query);
foreach ($result as $row) {
    printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
}

echo "\n==================\n";

// Not using iterators
$result = $mysqli->query($query);
while ($row = $result->fetch_assoc()) {
```

```
    printf("%s (%s)\n", $row["Name"], $row["CountryCode"]);
}
```

The above example will output something similar to:

```
Pueblo (USA)
Arvada (USA)
Cape Coral (USA)
Green Bay (USA)
Santa Clara (USA)

==================
Pueblo (USA)
Arvada (USA)
Cape Coral (USA)
Green Bay (USA)
Santa Clara (USA)
```

See Also

- mysqli_fetch_array() - Fetch the next row of a result set as an associative, a numeric array, or both
- mysqli_fetch_column() - Fetch a single column from the next row of a result set
- mysqli_fetch_row() - Fetch the next row of a result set as an enumerated array
- mysqli_fetch_object() - Fetch the next row of a result set as an object
- mysqli_query() - Performs a query on the database
- mysqli_data_seek() - Adjusts the result pointer to an arbitrary row in the result

mysqli_result::fetch_column

mysqli_result::fetch_column

mysqli_fetch_column

(PHP 8 >= 8.1.0)

mysqli_result::fetch_column -- mysqli_fetch_column — Fetch a single column from the next row of a result set

Description

Object-oriented style

public function **mysqli_result::fetch_column**(int `$column` = 0): null|int|float|string|false

Procedural style

function **mysqli_fetch_column**(mysqli_result `$result`, int `$column` = 0): null|int|float|string|false

Fetches one row of data from the result set and returns the 0-indexed column. Each subsequent call to this function will return the value from the next row within the result set, or **`false`** if there are no more rows.

> **Note** :
>
> This function sets NULL fields to the PHP `null` value.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

`column`

0-indexed number of the column you wish to retrieve from the row. If no value is supplied, the first column will be returned.

Return Values

Returns a single column from the next row of a result set or `false` if there are no more rows.

> **Warning**
>
> There is no way to return another column from the same row if you use this function to retrieve data.

Examples

Example #1 mysqli_result::fetch_column() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT CountryCode, Name FROM City ORDER BY ID DESC LIMIT 5";

$result = $mysqli->query($query);

/* fetch a single value from the second column */
while ($Name = $result->fetch_column(1)) {
    printf("%s\n", $Name);
}
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT CountryCode, Name FROM City ORDER BY ID DESC LIMIT 5";

$result = mysqli_query($mysqli, $query);
```

```php
/* fetch a single value from the second column */
while ($Name = mysqli_fetch_column($result, 1)) {
    printf("%s\n", $Name);
}
```

The above examples will output something similar to:

```
Rafah
Nablus
Jabaliya
Hebron
Khan Yunis
```

See Also

- mysqli_fetch_all() - Fetch all result rows as an associative array, a numeric array, or both
- mysqli_fetch_array() - Fetch the next row of a result set as an associative, a numeric array, or both
- mysqli_fetch_assoc() - Fetch the next row of a result set as an associative array
- mysqli_fetch_object() - Fetch the next row of a result set as an object
- mysqli_fetch_row() - Fetch the next row of a result set as an enumerated array
- mysqli_data_seek() - Adjusts the result pointer to an arbitrary row in the result

mysqli_result::fetch_field

mysqli_result::fetch_field

mysqli_fetch_field

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_field -- mysqli_fetch_field — Returns the next field in the result set

Description

Object-oriented style

public function **mysqli_result::fetch_field**(): object|false

Procedural style

function **mysqli_fetch_field**(mysqli_result `$result`): object|false

Returns the definition of one column of a result set as an object. Call this function repeatedly to retrieve information about all columns in the result set.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

Return Values

Returns an object which contains field definition information or `false` if no field information is available.

Object properties

Property	Description
name	The name of the column
orgname	Original column name if an alias was specified
table	The name of the table this field belongs to (if not calculated)
orgtable	Original table name if an alias was specified
def	Unused. Always an empty string
db	The name of the database
catalog	Unused. Always `"def"`
max_length	The maximum width of the field for the result set. As of PHP 8.1, this value is always `0`.
length	The width of the field in bytes. For string columns, the length value depends on the connection character set. For example, if the character set is `latin1`, a single-byte character set, the length value for a `SELECT 'abc'` query is 3. If the character set is `utf8mb4`, a multibyte character set in which characters take up to 4 bytes, the length value is 12.
charsetnr	The character set number for the field.
flags	An integer representing the bit-flags for the field.
type	The data type used for this field
decimals	The number of decimals for numeric fields, and the fractional seconds precision for temporal fields.

Examples

Example #1 Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Code LIMIT 5";

if ($result = $mysqli->query($query)) {

    /* Get field information for all columns */
    while ($finfo = $result->fetch_field()) {

        printf("Name:     %s\n", $finfo->name);
        printf("Table:    %s\n", $finfo->table);
        printf("max. Len: %d\n", $finfo->max_length);
        printf("Flags:    %d\n", $finfo->flags);
        printf("Type:     %d\n\n", $finfo->type);
    }
```

```
    $result->close();
}

/* close connection */
$mysqli->close();
?>
```

Example #2 Procedural style

```
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Code LIMIT 5";

if ($result = mysqli_query($link, $query)) {

    /* Get field information for all fields */
    while ($finfo = mysqli_fetch_field($result)) {

        printf("Name:     %s\n", $finfo->name);
        printf("Table:    %s\n", $finfo->table);
        printf("max. Len: %d\n", $finfo->max_length);
        printf("Flags:    %d\n", $finfo->flags);
        printf("Type:     %d\n\n", $finfo->type);
    }
    mysqli_free_result($result);
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Name:     Name
Table:    Country
max. Len: 11
Flags:    1
Type:     254

Name:     SurfaceArea
Table:    Country
max. Len: 10
Flags:    32769
Type:     4
```

See Also

- mysqli_num_fields() - Gets the number of fields in the result set
- mysqli_fetch_field_direct() - Fetch meta-data for a single field
- mysqli_fetch_fields() - Returns an array of objects representing the fields in a result set
- mysqli_field_seek() - Set result pointer to a specified field offset

mysqli_result::fetch_field_direct

mysqli_result::fetch_field_direct

mysqli_fetch_field_direct

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_field_direct -- mysqli_fetch_field_direct — Fetch meta-data for a single field

Description

Object-oriented style

public function **mysqli_result::fetch_field_direct**(int `$index`): object|false

Procedural style

function **mysqli_fetch_field_direct**(mysqli_result `$result`, int `$index`): object|false

Returns an object which contains field definition information from the specified result set.

Parameters

`result`

> Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

`index`

> The field number. This value must be in the range from `0` to `number of fields - 1`.

Return Values

Returns an object which contains field definition information or **`false`** if no field information for specified `index` is available.

Object properties

Property	Description
name	The name of the column
orgname	Original column name if an alias was specified
table	The name of the table this field belongs to (if not calculated)
orgtable	Original table name if an alias was specified
def	Unused. Always an empty string
db	The name of the database

Property	Description
catalog	Unused. Always `"def"`
max_length	The maximum width of the field for the result set. As of PHP 8.1, this value is always `0`.
length	The width of the field in bytes. For string columns, the length value depends on the connection character set. For example, if the character set is `latin1`, a single-byte character set, the length value for a `SELECT 'abc'` query is 3. If the character set is `utf8mb4`, a multibyte character set in which characters take up to 4 bytes, the length value is 12.
charsetnr	The character set number for the field.
flags	An integer representing the bit-flags for the field.
type	The data type used for this field
decimals	The number of decimals for numeric fields, and the fractional seconds precision for temporal fields.

Examples

Example #1 Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Name LIMIT 5";

if ($result = $mysqli->query($query)) {

    /* Get field information for column 'SurfaceArea' */
    $finfo = $result->fetch_field_direct(1);

    printf("Name:     %s\n", $finfo->name);
    printf("Table:    %s\n", $finfo->table);
    printf("max. Len: %d\n", $finfo->max_length);
    printf("Flags:    %d\n", $finfo->flags);
    printf("Type:     %d\n", $finfo->type);

    $result->close();
}

/* close connection */
$mysqli->close();
?>
```

Example #2 Procedural style

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
```

```
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Name LIMIT 5";

if ($result = mysqli_query($link, $query)) {

    /* Get field information for column 'SurfaceArea' */
    $finfo = mysqli_fetch_field_direct($result, 1);

    printf("Name:     %s\n", $finfo->name);
    printf("Table:    %s\n", $finfo->table);
    printf("max. Len: %d\n", $finfo->max_length);
    printf("Flags:    %d\n", $finfo->flags);
    printf("Type:     %d\n", $finfo->type);

    mysqli_free_result($result);
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Name:     SurfaceArea
Table:    Country
max. Len: 10
Flags:    32769
Type:     4
```

See Also

- mysqli_num_fields() - Gets the number of fields in the result set
- mysqli_fetch_field() - Returns the next field in the result set
- mysqli_fetch_fields() - Returns an array of objects representing the fields in a result set

mysqli_result::fetch_fields

mysqli_result::fetch_fields

mysqli_fetch_fields

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_fields -- mysqli_fetch_fields — Returns an array of objects representing the fields in a result set

Description

Object-oriented style

public function **mysqli_result::fetch_fields**(): array

Procedural style

function **mysqli_fetch_fields**(mysqli_result `$result`): array

This function serves an identical purpose to the mysqli_fetch_field() function with the single difference that, instead of returning one object at a time for each field, the columns are returned as an array of objects.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

Return Values

Returns an array of objects containing field definition information.

Object properties

Property	Description
name	The name of the column
orgname	Original column name if an alias was specified
table	The name of the table this field belongs to (if not calculated)
orgtable	Original table name if an alias was specified
def	Unused. Always an empty string
db	The name of the database
catalog	Unused. Always `"def"`
max_length	The maximum width of the field for the result set. As of PHP 8.1, this value is always `0`.
length	The width of the field in bytes. For string columns, the length value depends on the connection character set. For example, if the character set is `latin1`, a single-byte character set, the length value for a `SELECT 'abc'` query is 3. If the character set is `utf8mb4`, a multibyte character set in which characters take up to 4 bytes, the length value is 12.
charsetnr	The character set number for the field.
flags	An integer representing the bit-flags for the field.
type	The data type used for this field
decimals	The number of decimals for numeric fields, and the fractional seconds precision for temporal fields.

Examples

Example #1 Object-oriented style

```php
<?php
$mysqli = new mysqli("127.0.0.1", "root", "foofoo", "sakila");

/* check connection */
if ($mysqli->connect_errno) {
    printf("Connect failed: %s\n", $mysqli->connect_error);
```

```
    exit();
}

foreach (array('latin1', 'utf8') as $charset) {

    // Set character set, to show its impact on some values (e.g., length in bytes)
    $mysqli->set_charset($charset);

    $query = "SELECT actor_id, last_name from actor ORDER BY actor_id";

    echo "=====================\n";
    echo "Character Set: $charset\n";
    echo "=====================\n";

    if ($result = $mysqli->query($query)) {

        /* Get field information for all columns */
        $finfo = $result->fetch_fields();

        foreach ($finfo as $val) {
            printf("Name:      %s\n",   $val->name);
            printf("Table:     %s\n",   $val->table);
            printf("Max. Len:  %d\n",   $val->max_length);
            printf("Length:    %d\n",   $val->length);
            printf("charsetnr: %d\n",   $val->charsetnr);
            printf("Flags:     %d\n",   $val->flags);
            printf("Type:      %d\n\n", $val->type);
        }
        $result->free();
    }
}
$mysqli->close();
?>
```

Example #2 Procedural style

```
<?php
$link = mysqli_connect("127.0.0.1", "my_user", "my_password", "sakila");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

foreach (array('latin1', 'utf8') as $charset) {

    // Set character set, to show its impact on some values (e.g., length in bytes)
    mysqli_set_charset($link, $charset);

    $query = "SELECT actor_id, last_name from actor ORDER BY actor_id";

    echo "=====================\n";
    echo "Character Set: $charset\n";
    echo "=====================\n";

    if ($result = mysqli_query($link, $query)) {

        /* Get field information for all columns */
```

```
        $finfo = mysqli_fetch_fields($result);

        foreach ($finfo as $val) {
            printf("Name:      %s\n",   $val->name);
            printf("Table:     %s\n",   $val->table);
            printf("Max. Len:  %d\n",   $val->max_length);
            printf("Length:    %d\n",   $val->length);
            printf("charsetnr: %d\n",   $val->charsetnr);
            printf("Flags:     %d\n",   $val->flags);
            printf("Type:      %d\n\n", $val->type);
        }
        mysqli_free_result($result);
    }
}

mysqli_close($link);
?>
```

The above examples will output:

```
======================
Character Set: latin1
======================
Name:      actor_id
Table:     actor
Max. Len:  3
Length:    5
charsetnr: 63
Flags:     49699
Type:      2

Name:      last_name
Table:     actor
Max. Len:  12
Length:    45
charsetnr: 8
Flags:     20489
Type:      253

======================
Character Set: utf8
======================
Name:      actor_id
Table:     actor
Max. Len:  3
Length:    5
charsetnr: 63
Flags:     49699
Type:      2

Name:      last_name
Table:     actor
Max. Len:  12
Length:    135
charsetnr: 33
Flags:     20489
```

See Also

- mysqli_num_fields() - Gets the number of fields in the result set
- mysqli_fetch_field_direct() - Fetch meta-data for a single field
- mysqli_fetch_field() - Returns the next field in the result set

mysqli_result::fetch_object

mysqli_result::fetch_object

mysqli_fetch_object

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_object -- mysqli_fetch_object — Fetch the next row of a result set as an object

Description

Object-oriented style

public function **mysqli_result::fetch_object**(string `$class` = "stdClass", array `$constructor_args` = []): object|null|false

Procedural style

function **mysqli_fetch_object**(mysqli_result `$result`, string `$class` = "stdClass", array `$constructor_args` = []): object|null|false

Fetches one row of data from the result set and returns it as an object, where each property represents the name of the result set's column. Each subsequent call to this function will return the next row within the result set, or `null` if there are no more rows.

If two or more columns of the result have the same name, the last column will take precedence and overwrite any previous data. To access multiple columns with the same name, mysqli_fetch_row() may be used to fetch the numerically indexed array, or aliases may be used in the SQL query select list to give columns different names.

> **Note** :
>
> This function sets the properties of the object before calling the object constructor.

> **Note** :
>
> Field names returned by this function are *case-sensitive*.

> **Note** :
>
> This function sets NULL fields to the PHP `null` value.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

`class`

The name of the class to instantiate, set the properties of and return. If not specified, a stdClass object is returned.

`constructor_args`

An optional array of parameters to pass to the constructor for `class` objects.

Return Values

Returns an object representing the fetched row, where each property represents the name of the result set's column, **`null`** if there are no more rows in the result set, or **`false`** on failure.

Errors/Exceptions

A ValueError is thrown when the `constructor_args` is non-empty with the class not having a constructor.

Changelog

Version	Description
8.3.0	Now throws a ValueError exception when the `constructor_args` is non-empty with the class not having a constructor; previously an Exception was thrown.
8.0.0	`constructor_args` now accepts `[]` for constructors with 0 parameters; previously an exception was thrown.

Examples

Example #1 **mysqli_result::fetch_object()** example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID DESC";

$result = $mysqli->query($query);

while ($obj = $result->fetch_object()) {
    printf("%s (%s)\n", $obj->Name, $obj->CountryCode);
}
```

Procedural style

```php
<?php
```

```php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID DESC";

$result = mysqli_query($link, $query);

while ($obj = mysqli_fetch_object($result)) {
    printf("%s (%s)\n", $obj->Name, $obj->CountryCode);
}
```

The above examples will output something similar to:

```
Pueblo (USA)
Arvada (USA)
Cape Coral (USA)
Green Bay (USA)
Santa Clara (USA)
```

See Also

- mysqli_fetch_array() - Fetch the next row of a result set as an associative, a numeric array, or both
- mysqli_fetch_assoc() - Fetch the next row of a result set as an associative array
- mysqli_fetch_column() - Fetch a single column from the next row of a result set
- mysqli_fetch_row() - Fetch the next row of a result set as an enumerated array
- mysqli_query() - Performs a query on the database
- mysqli_data_seek() - Adjusts the result pointer to an arbitrary row in the result

mysqli_result::fetch_row

mysqli_result::fetch_row

mysqli_fetch_row

(PHP 5, PHP 7, PHP 8)

mysqli_result::fetch_row -- mysqli_fetch_row — Fetch the next row of a result set as an enumerated array

Description

Object-oriented style

public function **mysqli_result::fetch_row**(): array|null|false

Procedural style

function **mysqli_fetch_row**(mysqli_result `$result`): array|null|false

Fetches one row of data from the result set and returns it as an enumerated array, where each column is stored in an array offset starting from 0 (zero). Each subsequent call to this function will return the next row within the result set, or `null` if there are no more rows.

> **Note** :
>
> This function sets NULL fields to the PHP `null` value.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

Return Values

Returns an enumerated array representing the fetched row, `null` if there are no more rows in the result set, or `false` on failure.

Examples

Example #1 mysqli_result::fetch_row() example

Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID DESC";

$result = $mysqli->query($query);

/* fetch object array */
while ($row = $result->fetch_row()) {
    printf("%s (%s)\n", $row[0], $row[1]);
}
```

Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = mysqli_connect("localhost", "my_user", "my_password", "world");

$query = "SELECT Name, CountryCode FROM City ORDER BY ID DESC";

$result = mysqli_query($mysqli, $query);

/* fetch associative array */
while ($row = mysqli_fetch_row($result)) {
    printf("%s (%s)\n", $row[0], $row[1]);
}
```

The above examples will output something similar to:

```
```

```
Pueblo (USA)
Arvada (USA)
Cape Coral (USA)
Green Bay (USA)
Santa Clara (USA)
```

See Also

- mysqli_fetch_array() - Fetch the next row of a result set as an associative, a numeric array, or both
- mysqli_fetch_assoc() - Fetch the next row of a result set as an associative array
- mysqli_fetch_column() - Fetch a single column from the next row of a result set
- mysqli_fetch_object() - Fetch the next row of a result set as an object
- mysqli_query() - Performs a query on the database
- mysqli_data_seek() - Adjusts the result pointer to an arbitrary row in the result

mysqli_result::$field_count

mysqli_result::$field_count

mysqli_num_fields

(PHP 5, PHP 7, PHP 8)

mysqli_result::$field_count -- mysqli_num_fields — Gets the number of fields in the result set

Description

Object-oriented style

int *$mysqli_result->field_count*;

Procedural style

function **mysqli_num_fields**(mysqli_result `$result`): int

Returns the number of fields in the result set.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

Return Values

An int representing the number of fields.

Examples

Example #1 Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$result = $mysqli->query("SELECT Name, CountryCode, District, Population FROM City ORDER BY ID LIMIT 1");

/* Get the number of fields in the result set */
$field_cnt = $result->field_count;

printf("Result set has %d fields.\n", $field_cnt);
```

Example #2 Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$result = mysqli_query($link, "SELECT Name, CountryCode, District, Population FROM City ORDER BY ID LIMIT 1");

/* Get the number of fields in the result set */
$field_cnt = mysqli_num_fields($result);

printf("Result set has %d fields.\n", $field_cnt);
```

The above examples will output:

```
Result set has 4 fields.
```

See Also

- mysqli_fetch_field() - Returns the next field in the result set

mysqli_result::field_seek

mysqli_result::field_seek

mysqli_field_seek

(PHP 5, PHP 7, PHP 8)

mysqli_result::field_seek -- mysqli_field_seek — Set result pointer to a specified field offset

Description

Object-oriented style

public function **mysqli_result::field_seek**(int `$index`): true

Procedural style

function **mysqli_field_seek**(mysqli_result `$result`, int `$index`): true

Sets the field cursor to the given offset. The next call to mysqli_fetch_field() will retrieve the field definition of the column associated with that offset.

> **Note** :
>
> To seek to the beginning of a row, pass an offset value of zero.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

`index`

The field number. This value must be in the range from `0` to `number of fields - 1`.

Return Values

Always returns **`true`**.

Changelog

Version	Description
8.0.0	This function now always returns **`true`**. Previously it returned **`false`** on failure.

Examples

Example #1 Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Code LIMIT 5";

if ($result = $mysqli->query($query)) {

    /* Get field information for 2nd column */
    $result->field_seek(1);
    $finfo = $result->fetch_field();

    printf("Name:     %s\n", $finfo->name);
    printf("Table:    %s\n", $finfo->table);
    printf("max. Len: %d\n", $finfo->max_length);
```

```
    printf("Flags:    %d\n", $finfo->flags);
    printf("Type:     %d\n\n", $finfo->type);

    $result->close();
}

/* close connection */
$mysqli->close();
?>
```

Example #2 Procedural style

```
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT Name, SurfaceArea from Country ORDER BY Code LIMIT 5";

if ($result = mysqli_query($link, $query)) {

    /* Get field information for 2nd column */
    mysqli_field_seek($result, 1);
    $finfo = mysqli_fetch_field($result);

    printf("Name:     %s\n", $finfo->name);
    printf("Table:    %s\n", $finfo->table);
    printf("max. Len: %d\n", $finfo->max_length);
    printf("Flags:    %d\n", $finfo->flags);
    printf("Type:     %d\n\n", $finfo->type);

    mysqli_free_result($result);
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
Name:     SurfaceArea
Table:    Country
max. Len: 10
Flags:    32769
Type:     4
```

See Also

- mysqli_fetch_field() - Returns the next field in the result set

mysqli_result::free

mysqli_result::free

mysqli_result::close

mysqli_result::free_result

mysqli_free_result

(PHP 5, PHP 7, PHP 8)

mysqli_result::free -- mysqli_result::close -- mysqli_result::free_result -- mysqli_free_result — Frees the memory associated with a result

Description

Object-oriented style

public function **mysqli_result::free**(): void
public function **mysqli_result::close**(): void
public function **mysqli_result::free_result**(): void

Procedural style

function **mysqli_free_result**(mysqli_result `$result`): void

Frees the memory associated with the result.

Parameters

`result`

> Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

Return Values

No value is returned.

See Also

- mysqli_query() - Performs a query on the database
- mysqli_stmt_get_result() - Gets a result set from a prepared statement as a mysqli_result object
- mysqli_store_result() - Transfers a result set from the last query
- mysqli_use_result() - Initiate a result set retrieval

mysqli_result::getIterator

mysqli_result::getIterator

(PHP 8)

mysqli_result::getIterator — Retrieve an external iterator

Description

public function **mysqli_result::getIterator**(): Iterator

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

mysqli_result::$lengths

mysqli_result::$lengths

mysqli_fetch_lengths

(PHP 5, PHP 7, PHP 8)

mysqli_result::$lengths -- mysqli_fetch_lengths — Returns the lengths of the columns of the current row in the result set

Description

Object-oriented style

?array *$mysqli_result->lengths*;

Procedural style

function **mysqli_fetch_lengths**(mysqli_result `$result`): array|false

The **mysqli_fetch_lengths()** function returns an array containing the lengths of every column of the current row within the result set.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

Return Values

An array of integers representing the size of each column (not including any terminating null characters). `false` if an error occurred.

mysqli_fetch_lengths() is valid only for the current row of the result set. It returns `false` if you call it before calling mysqli_fetch_row/array/object or after retrieving all rows in the result.

Examples

Example #1 Object-oriented style

```php
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT * from Country ORDER BY Code LIMIT 1";

if ($result = $mysqli->query($query)) {

    $row = $result->fetch_row();

    /* display column lengths */
    foreach ($result->lengths as $i => $val) {
        printf("Field %2d has Length %2d\n", $i+1, $val);
    }
    $result->close();
}

/* close connection */
$mysqli->close();
?>
```

Example #2 Procedural style

```php
<?php
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

/* check connection */
if (mysqli_connect_errno()) {
    printf("Connect failed: %s\n", mysqli_connect_error());
    exit();
}

$query = "SELECT * from Country ORDER BY Code LIMIT 1";

if ($result = mysqli_query($link, $query)) {

    $row = mysqli_fetch_row($result);

    /* display column lengths */
    foreach (mysqli_fetch_lengths($result) as $i => $val) {
        printf("Field %2d has Length %2d\n", $i+1, $val);
    }
    mysqli_free_result($result);
}

/* close connection */
mysqli_close($link);
?>
```

The above examples will output:

```
```

```
Field  1 has Length  3
Field  2 has Length  5
Field  3 has Length 13
Field  4 has Length  9
Field  5 has Length  6
Field  6 has Length  1
Field  7 has Length  6
Field  8 has Length  4
Field  9 has Length  6
Field 10 has Length  6
Field 11 has Length  5
Field 12 has Length 44
Field 13 has Length  7
Field 14 has Length  3
Field 15 has Length  2
```

mysqli_result::$num_rows

mysqli_result::$num_rows

mysqli_num_rows

(PHP 5, PHP 7, PHP 8)

mysqli_result::$num_rows -- mysqli_num_rows — Gets the number of rows in the result set

Description

Object-oriented style

int|string *$mysqli_result->num_rows*;

Procedural style

function **mysqli_num_rows**(mysqli_result `$result`): int|string

Returns the number of rows in the result set.

The behaviour of **mysqli_num_rows()** depends on whether buffered or unbuffered result sets are being used. This function returns `0` for unbuffered result sets unless all rows have been fetched from the server.

Parameters

`result`

Procedural style only: A mysqli_result object returned by mysqli_query(), mysqli_store_result(), mysqli_use_result() or mysqli_stmt_get_result().

Return Values

An int representing the number of fetched rows. Returns `0` in unbuffered mode unless all rows have been fetched from the server.

> **Note** :
>
> If the number of rows is greater than `PHP_INT_MAX`, the number will be returned as a string.

Examples

Example #1 Object-oriented style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$mysqli = new mysqli("localhost", "my_user", "my_password", "world");

$result = $mysqli->query("SELECT Code, Name FROM Country ORDER BY Name");

/* Get the number of rows in the result set */
$row_cnt = $result->num_rows;

printf("Result set has %d rows.\n", $row_cnt);
```

Example #2 Procedural style

```php
<?php

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$link = mysqli_connect("localhost", "my_user", "my_password", "world");

$result = mysqli_query($link, "SELECT Code, Name FROM Country ORDER BY Name");

/* Get the number of rows in the result set */
$row_cnt = mysqli_num_rows($result);

printf("Result set has %d rows.\n", $row_cnt);
```

The above examples will output:

```
Result set has 239 rows.
```

Notes

> **Note** :
>
> In contrast to the mysqli_stmt_num_rows() function, this function doesn't have an object-oriented method variant. In the object-oriented style, use the getter property.

See Also

- mysqli_affected_rows() - Gets the number of affected rows in a previous MySQL operation
- mysqli_store_result() - Transfers a result set from the last query
- mysqli_use_result() - Initiate a result set retrieval
- mysqli_query() - Performs a query on the database
- mysqli_stmt_num_rows() - Returns the number of rows fetched from the server

mysqli_driver

(PHP 5, PHP 7, PHP 8)

Introduction

The **mysqli_driver** class is an instance of the monostate pattern, i.e. there is only one driver which can be accessed through an arbitrary amount of **mysqli_driver** instances.

Class synopsis

final class **mysqli_driver** {
/* Properties */
public readonly string *$client_info*;
public readonly int *$client_version*;
public readonly int *$driver_version*;
public int *$report_mode*;
}

Properties

client_info

The Client API header version

client_version

The Client version

driver_version

The MySQLi Driver version

> **Warning**
>
> This property has been *deprecated* as of PHP 8.1.0. Relying on this property is highly discouraged.

embedded

Whether MySQLi Embedded support is enabled

> **Warning**
>
> This property has been *removed* as of PHP 8.0.0.

reconnect

Allow or prevent reconnect (see the mysqli.reconnect INI directive)

> **Warning**
>
> This property has been *removed* along with the mysqli.reconnect INI directive as of PHP 8.2.0.

report_mode

Set to `MYSQLI_REPORT_OFF`, `MYSQLI_REPORT_ALL` or any combination of `MYSQLI_REPORT_STRICT` (throw Exceptions for errors), `MYSQLI_REPORT_ERROR` (report errors) and `MYSQLI_REPORT_INDEX` (errors regarding indexes). See also mysqli_report().

Changelog

Version	Description
8.2.0	mysqli_driver::$reconnect was removed.
8.1.0	mysqli_driver::$driver_version has been deprecated.
8.0.0	mysqli_driver::$embedded was removed.
7.4.0	mysqli_driver::embedded_server_start() and **mysqli_driver:embedded_server_end()** were removed.

Table of Contents

mysqli_driver::embedded_server_end

mysqli_driver::embedded_server_end

mysqli_embedded_server_end

(PHP 5 >= 5.1.0, PHP 7 < 7.4.0)

mysqli_driver::embedded_server_end -- mysqli_embedded_server_end — Stop embedded server

> **Warning**
>
> This function was *REMOVED* in PHP 7.4.0.

Description

Object-oriented style

public function **mysqli_driver::embedded_server_end**(): void

Procedural style

function **mysqli_embedded_server_end**(): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

mysqli_driver::embedded_server_start

mysqli_driver::embedded_server_start

mysqli_embedded_server_start

(PHP 5 >= 5.1.0, PHP 7 < 7.4.0)

mysqli_driver::embedded_server_start -- mysqli_embedded_server_start — Initialize and start embedded server

> **Warning**
>
> This function was *REMOVED* in PHP 7.4.0.

Description

Object-oriented style

public function **mysqli_driver::embedded_server_start**(int `$start`, array `$arguments`, array `$groups`): bool

Procedural style

function **mysqli_embedded_server_start**(int `$start`, array `$arguments`, array `$groups`): bool

> **Warning**
>
> This function is currently not documented; only its argument list is available.

mysqli_driver::$report_mode

mysqli_driver::$report_mode

mysqli_report

(PHP 5, PHP 7, PHP 8)

mysqli_driver::$report_mode -- mysqli_report — Sets mysqli error reporting mode

Description

Object-oriented style

int *$mysqli_driver->report_mode*;

Procedural style

function **mysqli_report**(int `$flags`): true

Depending on the flags, it sets mysqli error reporting mode to exception, warning or none. When set to `MYSQLI_REPORT_ALL` or `MYSQLI_REPORT_INDEX` it will also inform about queries that don't use an index (or use a bad index).

As of PHP 8.1.0, the default setting is `MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT`. Previously, it was `MYSQLI_REPORT_OFF`.

Parameters

`flags`

Supported flags

Name	Description

Name	Description
MYSQLI_REPORT_OFF	Turns reporting off
MYSQLI_REPORT_ERROR	Report errors from mysqli function calls
MYSQLI_REPORT_STRICT	Throw mysqli_sql_exception for errors instead of warnings
MYSQLI_REPORT_INDEX	Report if no index or bad index was used in a query
MYSQLI_REPORT_ALL	Set all options (report all)

Return Values

Always returns `true`.

Changelog

Version	Description
8.1.0	The default value is now `MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT`. Previously, it was `MYSQLI_REPORT_OFF`.

Examples

Example #1 Object-oriented style

```php
<?php

/* activate reporting */
$driver = new mysqli_driver();
$driver->report_mode = MYSQLI_REPORT_ALL;

try {
    /* if the connection fails, a mysqli_sql_exception will be thrown */
    $mysqli = new mysqli("localhost", "my_user", "my_password", "my_db");

    /* this query should report an error */
    $result = $mysqli->query("SELECT Name FROM Nonexistingtable WHERE population > 50000");

    /* this query should report a bad index if the column population doesn't have an index */
    $result = $mysqli->query("SELECT Name FROM City WHERE population > 50000");
} catch (mysqli_sql_exception $e) {
    error_log($e->__toString());
}
```

Example #2 Procedural style

```php
<?php

/* activate reporting */
mysqli_report(MYSQLI_REPORT_ALL);

try {
    /* if the connection fails, a mysqli_sql_exception will be thrown */
    $link = mysqli_connect("localhost", "my_user", "my_password", "my_db");

    /* this query should report an error */
    $result = mysqli_query($link, "SELECT Name FROM Nonexistingtable WHERE population > 50000");
```

```
    /* this query should report a bad index if the column population doesn't have an index */
    $result = mysqli_query($link, "SELECT Name FROM City WHERE population > 50000");
} catch (mysqli_sql_exception $e) {
    error_log($e->__toString());
}
```

Example #3 Error reporting except bad index errors

```
<?php

/* activate reporting */
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

try {
    /* if the connection fails, a mysqli_sql_exception will be thrown */
    $mysqli = new mysqli("localhost", "my_user", "my_password", "my_db");

    /* this query should report an error */
    $result = $mysqli->query("SELECT Name FROM Nonexistingtable WHERE population > 50000");

    /* this WILL NOT report any errors even if index is not available */
    $result = $mysqli->query("SELECT Name FROM City WHERE population > 50000");
} catch (mysqli_sql_exception $e) {
    error_log($e->__toString());
}
```

See Also

- mysqli_sql_exception
- set_exception_handler() - Sets a user-defined exception handler function
- error_reporting() - Sets which PHP errors are reported

mysqli_warning

(PHP 5, PHP 7, PHP 8)

Introduction

Represents a MySQL warning.

Class synopsis

final class **mysqli_warning** {
/* Properties */
public string *$message*;
public string *$sqlstate*;
public int *$errno*;
/* Methods */
private function __construct()
public function next(): bool
}

Properties

message

Message string

sqlstate

SQL state

errno

Error number

Table of Contents

mysqli_warning::__construct

mysqli_warning::__construct

(PHP 5, PHP 7, PHP 8)

mysqli_warning::__construct — Private constructor to disallow direct instantiation

Description

private function **mysqli_warning::__construct**()

Parameters

This function has no parameters.

mysqli_warning::next

mysqli_warning::next

(PHP 5, PHP 7, PHP 8)

mysqli_warning::next — Fetch next warning

Description

public function **mysqli_warning::next**(): bool

Change warning information to the next warning if possible.

Once the warning has been set to the next warning, new values of properties `message`, `sqlstate` and `errno` of mysqli_warning are available.

Parameters

This function has no parameters.

Return Values

Returns `true` if next warning was fetched successfully. If there are no more warnings, it will return `false`

mysqli_sql_exception

(PHP 5, PHP 7, PHP 8)

Introduction

The mysqli exception handling class.

Class synopsis

final class **mysqli_sql_exception** extends RuntimeException {
/* Properties */
protected string *$sqlstate* = "00000";
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Methods */
public function getSqlState(): string
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

Properties

sqlstate

The sql state with the error.

Table of Contents

mysqli_sql_exception::getSqlState

mysqli_sql_exception::getSqlState

(PHP 8 >= 8.1.2)

mysqli_sql_exception::getSqlState — Returns the SQLSTATE error code

Description

public function **mysqli_sql_exception::getSqlState**(): string

Returns a string containing the SQLSTATE error code for the last error. The error code consists of five characters. The values are specified by ANSI SQL and ODBC. For a list of possible values, see
» http://dev.mysql.com/doc/mysql/en/error-handling.html.

> **Note** :
>
> Note that not all MySQL errors are yet mapped to SQLSTATEs. The value `HY000` (general error) is used for unmapped errors.

Parameters

This function has no parameters.

Return Values

Returns a string containing the SQLSTATE error code for the last error. The error code consists of five characters.

MySQLi Functions

Table of Contents

mysqli_execute

mysqli_execute

(PHP 5, PHP 7, PHP 8)

mysqli_execute — Alias of mysqli_stmt_execute()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

This function is an alias of: mysqli_stmt_execute().

See Also

- mysqli_stmt_execute() - Executes a prepared statement

mysqli_get_client_stats

mysqli_get_client_stats

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mysqli_get_client_stats — Returns client per-process statistics

Description

function **mysqli_get_client_stats**(): array

Returns client per-process statistics.

> **Note** :

> Available only with mysqlnd.

Parameters

This function has no parameters.

Return Values

Returns an array with client stats.

Examples

Example #1 A **mysqli_get_client_stats()** example

```php
<?php
$link = mysqli_connect();
print_r(mysqli_get_client_stats());
?>
```

The above example will output something similar to:

```
Array
(
    [bytes_sent] => 43
    [bytes_received] => 80
    [packets_sent] => 1
    [packets_received] => 2
    [protocol_overhead_in] => 8
    [protocol_overhead_out] => 4
    [bytes_received_ok_packet] => 11
    [bytes_received_eof_packet] => 0
    [bytes_received_rset_header_packet] => 0
    [bytes_received_rset_field_meta_packet] => 0
    [bytes_received_rset_row_packet] => 0
    [bytes_received_prepare_response_packet] => 0
    [bytes_received_change_user_packet] => 0
    [packets_sent_command] => 0
    [packets_received_ok] => 1
    [packets_received_eof] => 0
    [packets_received_rset_header] => 0
    [packets_received_rset_field_meta] => 0
    [packets_received_rset_row] => 0
    [packets_received_prepare_response] => 0
    [packets_received_change_user] => 0
    [result_set_queries] => 0
    [non_result_set_queries] => 0
    [no_index_used] => 0
    [bad_index_used] => 0
    [slow_queries] => 0
    [buffered_sets] => 0
    [unbuffered_sets] => 0
    [ps_buffered_sets] => 0
    [ps_unbuffered_sets] => 0
    [flushed_normal_sets] => 0
    [flushed_ps_sets] => 0
```

```
    [ps_prepared_never_executed] => 0
    [ps_prepared_once_executed] => 0
    [rows_fetched_from_server_normal] => 0
    [rows_fetched_from_server_ps] => 0
    [rows_buffered_from_client_normal] => 0
    [rows_buffered_from_client_ps] => 0
    [rows_fetched_from_client_normal_buffered] => 0
    [rows_fetched_from_client_normal_unbuffered] => 0
    [rows_fetched_from_client_ps_buffered] => 0
    [rows_fetched_from_client_ps_unbuffered] => 0
    [rows_fetched_from_client_ps_cursor] => 0
    [rows_skipped_normal] => 0
    [rows_skipped_ps] => 0
    [copy_on_write_saved] => 0
    [copy_on_write_performed] => 0
    [command_buffer_too_small] => 0
    [connect_success] => 1
    [connect_failure] => 0
    [connection_reused] => 0
    [reconnect] => 0
    [pconnect_success] => 0
    [active_connections] => 1
    [active_persistent_connections] => 0
    [explicit_close] => 0
    [implicit_close] => 0
    [disconnect_close] => 0
    [in_middle_of_command_close] => 0
    [explicit_free_result] => 0
    [implicit_free_result] => 0
    [explicit_stmt_close] => 0
    [implicit_stmt_close] => 0
    [mem_emalloc_count] => 0
    [mem_emalloc_ammount] => 0
    [mem_ecalloc_count] => 0
    [mem_ecalloc_ammount] => 0
    [mem_erealloc_count] => 0
    [mem_erealloc_ammount] => 0
    [mem_efree_count] => 0
    [mem_malloc_count] => 0
    [mem_malloc_ammount] => 0
    [mem_calloc_count] => 0
    [mem_calloc_ammount] => 0
    [mem_realloc_count] => 0
    [mem_realloc_ammount] => 0
    [mem_free_count] => 0
    [proto_text_fetched_null] => 0
    [proto_text_fetched_bit] => 0
    [proto_text_fetched_tinyint] => 0
    [proto_text_fetched_short] => 0
    [proto_text_fetched_int24] => 0
    [proto_text_fetched_int] => 0
    [proto_text_fetched_bigint] => 0
    [proto_text_fetched_decimal] => 0
    [proto_text_fetched_float] => 0
    [proto_text_fetched_double] => 0
    [proto_text_fetched_date] => 0
    [proto_text_fetched_year] => 0
    [proto_text_fetched_time] => 0
    [proto_text_fetched_datetime] => 0
```

```
    [proto_text_fetched_timestamp] => 0
    [proto_text_fetched_string] => 0
    [proto_text_fetched_blob] => 0
    [proto_text_fetched_enum] => 0
    [proto_text_fetched_set] => 0
    [proto_text_fetched_geometry] => 0
    [proto_text_fetched_other] => 0
    [proto_binary_fetched_null] => 0
    [proto_binary_fetched_bit] => 0
    [proto_binary_fetched_tinyint] => 0
    [proto_binary_fetched_short] => 0
    [proto_binary_fetched_int24] => 0
    [proto_binary_fetched_int] => 0
    [proto_binary_fetched_bigint] => 0
    [proto_binary_fetched_decimal] => 0
    [proto_binary_fetched_float] => 0
    [proto_binary_fetched_double] => 0
    [proto_binary_fetched_date] => 0
    [proto_binary_fetched_year] => 0
    [proto_binary_fetched_time] => 0
    [proto_binary_fetched_datetime] => 0
    [proto_binary_fetched_timestamp] => 0
    [proto_binary_fetched_string] => 0
    [proto_binary_fetched_blob] => 0
    [proto_binary_fetched_enum] => 0
    [proto_binary_fetched_set] => 0
    [proto_binary_fetched_geometry] => 0
    [proto_binary_fetched_other] => 0
)
```

See Also

- Stats description

mysqli_get_links_stats

mysqli_get_links_stats

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

mysqli_get_links_stats — Return information about open and cached links

Description

function **mysqli_get_links_stats**(): array

mysqli_get_links_stats() returns information about open and cached MySQL links.

Parameters

This function has no parameters.

Return Values

mysqli_get_links_stats() returns an associative array with three elements, keyed as follows:

`total`

An int indicating the total number of open links in any state.

`active_plinks`

An int representing the number of active persistent connections.

`cached_plinks`

An int representing the number of inactive persistent connections.

Changelog

The following changes have been made to classes/functions/methods of this extension.

Version	Function	Description
8.5.0	mysqli::__construct	Calling the constructor on an already-constructed object now throws an Error.
8.4.0	mysqli::kill	Both mysqli::kill and mysqli_kill are now deprecated. Use the KILL SQL command instead.
	mysqli::ping	Both mysqli::ping and mysqli_ping are now deprecated. As of PHP 8.2.0, the reconnect feature is no longer available, making this function obsolete.
	mysqli::query	When connecting to MySQL 8.0.24 or later, the error code reported for a server wait timeout is now 4031 (ER_CLIENT_INTERACTION_TIMEOUT) instead of 2006 (CR_SERVER_GONE_ERROR).
	mysqli::refresh	Both mysqli::refresh and mysqli_refresh are now deprecated. Use FLUSH SQL commands instead.
	mysqli::store_result	Passing the mode parameter is now deprecated. The parameter has had no effect as of PHP 8.1.0.
8.3.0	mysqli_result::fetch_object	Now throws a ValueError exception when the constructor_args is non-empty with the class not having a constructor; previously an Exception was thrown.
	mysqli::poll	Now throws a ValueError exception when neither the read nor error arguments are passed.
8.1.0	mysqli_driver::$report_mode	The default value is now MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT. Previously, it was MYSQLI_REPORT_OFF.
	mysqli_result::fetch_all	Now also available when linking against libmysqlclient.
	mysqli_stmt::execute	The optional params parameter has been added.
	mysqli_stmt::next_result	Now also available when linking against libmysqlclient.
	mysqli::__construct	mysqli::connect now returns true instead of null on success.
	mysqli::$client_info	Calling mysqli_get_client_info with the mysql argument has been deprecated. This function never required a parameter, but incorrectly allowed it as an optional parameter.
	mysqli::$client_info	The object-oriented style mysqli::get_client_info has been deprecated.

Version	Function	Description
	mysqli::init	The object-oriented style mysqli::init method has been deprecated. Replace calls to parent::init with parent::__construct.
8.0.0	mysqli_result::fetch_object	constructor_args now accepts [] for constructors with 0 parameters; previously an exception was thrown.
	mysqli_result::field_seek	This function now always returns true. Previously it returned false on failure.
	mysqli_stmt::close	This function now always returns true. Previously it returned false on failure.
	mysqli_stmt::__construct	query is now nullable.
	mysqli::begin_transaction	name is now nullable.
	mysqli::close	This function now always returns true. Previously it returned false on failure.
	mysqli::commit	name is now nullable.
	mysqli::debug	This function now always returns true. Previously it returned false on failure.
	mysqli::rollback	name is now nullable.
7.4.0	mysqli::__construct	All parameters are now nullable.
	mysqli::real_connect	All parameters are now nullable.

Mysql_xdevapi

Introduction

This extension provides access to the MySQL Document Store via the X DevAPI. The X DevAPI is a common API provided by multiple MySQL Connectors providing easy access to relational tables as well as collections of documents, which are represented in JSON, from a API with CRUD-style operations.

The X DevAPI uses the X Protocol, the new generation client-server protocol of the MySQL 8.0 server.

For general information about the MySQL Document Store, please refer to the » MySQL Document Store chapter in the MySQL manual.

Installing/Configuring

Table of Contents

Requirements

This extension requires a MySQL 8+ server with the X plugin enabled (default).

Prerequisite libraries for compiling this extension are: Boost (1.53.0 or higher), OpenSSL, and Protobuf.

Installation

This » PECL extension is not bundled with PHP.

An example installation procedure on Ubuntu 18.04 with PHP 7.2:

```
// Dependencies
$ apt install build-essential libprotobuf-dev libboost-dev openssl protobuf-compiler liblz4-tool zstd

// PHP with the desired extensions; php7.2-dev is required to compile
$ apt install php7.2-cli php7.2-dev php7.2-mysql php7.2-pdo php7.2-xml

// Compile the extension
$ pecl install mysql_xdevapi
```

The `pecl install` command does not enable PHP extensions (by default) and enabling PHP extensions can be done in several ways. Another PHP 7.2 on Ubuntu 18.04 example:

```
// Create its own ini file
$ echo "extension=mysql_xdevapi.so" > /etc/php/7.2/mods-available/mysql_xdevapi.ini

// Use the 'phpenmod' command (note: it's Debian/Ubuntu specific)
$ phpenmod -v 7.2 -s ALL mysql_xdevapi

// A 'phpenmod' alternative is to manually symlink it
// $ ln -s /etc/php/7.2/mods-available/mysql_xdevapi.ini /etc/php/7.2/cli/conf.d/20-mysql_xdevapi.ini

// Let's see which MySQL extensions are enabled now
$ php -m |grep mysql

mysql_xdevapi
mysqli
mysqlnd
pdo_mysql
```

Information for installing this PECL extension may be found in the manual chapter titled Installation of PECL extensions. Additional information such as new releases, downloads, source files, maintainer information, and a CHANGELOG, can be located here: » https://pecl.php.net/package/mysql_xdevapi.

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

Mysql_xdevapi Configure Options

Name	Default	Changeable	Changelog
xmysqlnd.collect_memory_statistics	0	INI_SYSTEM	
xmysqlnd.collect_statistics	1	INI_ALL	
xmysqlnd.debug		INI_SYSTEM	
xmysqlnd.mempool_default_size	16000	INI_ALL	
xmysqlnd.net_read_timeout	31536000	INI_SYSTEM	
xmysqlnd.trace_alloc		INI_SYSTEM	

Here's a short explanation of the configuration directives.

`xmysqlnd.collect_memory_statistics` int

`xmysqlnd.collect_statistics` int

`xmysqlnd.debug` string

`xmysqlnd.mempool_default_size` int

`xmysqlnd.net_read_timeout` int

`xmysqlnd.trace_alloc` string

Building / Compiling From Source

Considerations for compiling this extension from source.

- The extension name is 'mysql_xdevapi', so use `--enable-mysql-xdevapi`.
- Boost: required, optionally use the --with-boost=DIR configure option or set the MYSQL_XDEVAPI_BOOST_ROOT environment variable. Only the boost header files are required; not the binaries.
- Google Protocol Buffers (protobuf): required, optionally use the --with-protobuf=DIR configure option or set the MYSQL_XDEVAPI_PROTOBUF_ROOT environment variable.

  Optionally use `make protobufs` to generate protobuf files (*.pb.cc/.h), and `make clean-protobufs` to delete generate protobuf files.

  Windows specific protobuf note: depending on your environment, the static library with a multi-threaded DLL runtime may be needed. To prepare, use the following options: *-Dprotobuf_MSVC_STATIC_RUNTIME=OFF -Dprotobuf_BUILD_SHARED_LIBS=OFF*
- Google Protocol Buffers / protocol compiler (protoc): required, ensure that proper 'protoc' is available in the PATH while building. It is especially important as Windows PHP SDK batch scripts may overwrite the environment.
- Bison: required, and available from the PATH.

  Windows specific bison note: we strongly recommended that bison delivered with the chosen PHP SDKis used else an error similar to "zend_globals_macros.h(39): error C2375: 'zendparse': redefinition; different linkage Zend/zend_language_parser.h(214): note: see declaration of 'zendparse'" may be the result. Also, Windows PHP SDK batch scripts may overwrite the environment.
- Windows Specific Notes: To prepare the environment, see the official Windows build documentation for » the current SDK.

  We recommend using the backslash '\\' instead of a slash '/' for all paths.

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

`MYSQLX_CLIENT_SSL` (int)

`MYSQLX_TYPE_DECIMAL` (int)

`MYSQLX_TYPE_TINY` (int)

`MYSQLX_TYPE_SHORT` (int)

`MYSQLX_TYPE_SMALLINT` (int)

MYSQLX_TYPE_MEDIUMINT (int)

MYSQLX_TYPE_INT (int)

MYSQLX_TYPE_BIGINT (int)

MYSQLX_TYPE_LONG (int)

MYSQLX_TYPE_FLOAT (int)

MYSQLX_TYPE_DOUBLE (int)

MYSQLX_TYPE_NULL (int)

MYSQLX_TYPE_TIMESTAMP (int)

MYSQLX_TYPE_LONGLONG (int)

MYSQLX_TYPE_INT24 (int)

MYSQLX_TYPE_DATE (int)

MYSQLX_TYPE_TIME (int)

MYSQLX_TYPE_DATETIME (int)

MYSQLX_TYPE_YEAR (int)

MYSQLX_TYPE_NEWDATE (int)

MYSQLX_TYPE_ENUM (int)

MYSQLX_TYPE_SET (int)

MYSQLX_TYPE_TINY_BLOB (int)

MYSQLX_TYPE_MEDIUM_BLOB (int)

MYSQLX_TYPE_LONG_BLOB (int)

MYSQLX_TYPE_BLOB (int)

MYSQLX_TYPE_VAR_STRING (int)

MYSQLX_TYPE_STRING (int)

MYSQLX_TYPE_CHAR (int)

MYSQLX_TYPE_BYTES (int)

MYSQLX_TYPE_INTERVAL (int)

MYSQLX_TYPE_GEOMETRY (int)

MYSQLX_TYPE_JSON (int)

MYSQLX_TYPE_NEWDECIMAL (int)

MYSQLX_TYPE_BIT (int)

`MYSQLX_LOCK_DEFAULT` (int)

`MYSQLX_LOCK_NOWAIT` (int)

`MYSQLX_LOCK_SKIP_LOCKED` (int)

Changelog

The following changes have been made to classes/functions/methods of this extension.

General Changelog for the ext/mysql_xdevapi extension

This changelog references the ext/mysql_xdevapi extension.

Global ext/mysql_xdevapi changes

Version	Description
8.0.28	Removed TLSv1 and TLSv1.1 support. The values "TLSv1", "TLSv1.0", and "TLSv1.1" are now considered invalid for the "tls-versions" connection attribute.
8.0.26	Deprecated TLSv1 and TLSv1.1 support.

Examples

The central entry point to the X DevAPI is the **mysql_xdevapi\getSession()** function, which receives a URI to a MySQL 8.0 Server and returns a mysql_xdevapi\Session object.

Example #1 Connecting to a MySQL Server

```
<?php
try {
    $session = mysql_xdevapi\getSession("mysqlx://user:password@host");
} catch(Exception $e) {
    die("Connection could not be established: " . $e->getMessage());
}

// ... use $session
?>
```

The session provides full access to the API. For a new MySQL Server installation, the first step is to create a database schema with a collection to store data:

Example #2 Creating a Schema and Collection on the MySQL Server

```
<?php
$schema = $session->createSchema("test");
$collection = $schema->createCollection("example");
?>
```

When storing data, typically json_encode() is used to encode the data into JSON, which can then be stored inside a collection.

The following example stores data into the collection we created earlier, and then retrieve parts of it again.

Example #3 Storing and Retrieving Data

```php
<?php
$marco = [
  "name" => "Marco",
  "age"  => 19,
  "job"  => "Programmer"
];
$mike = [
  "name" => "Mike",
  "age"  => 39,
  "job"  => "Manager"
];

$schema = $session->getSchema("test");
$collection = $schema->getCollection("example");

$collection->add($marco, $mike)->execute();

var_dump($collection->find("name = 'Mike'")->execute()->fetchOne());
?>
```

The above example will output something similar to:

```
array(4) {
  ["_id"]=>
  string(28) "00005ad66aaf0000000000000003"
  ["age"]=>
  int(39)
  ["job"]=>
  string(7) "Manager"
  ["name"]=>
  string(4) "Mike"
}
```

The example demonstrates that the MySQL Server adds an extra field named `_id`, which serves as primary key to the document.

The example also demonstrates that retrieved data is sorted alphabetically. That specific order comes from the efficient binary storage inside the MySQL server, but it should not be relied upon. Refer to the MySQL JSON datatype documentation for details.

Optionally use PHP's iterators to fetch multiple documents:

Example #4 Fetching and Iterating Multiple Documents

```php
<?php
$result = $collection->find()->execute();
foreach ($result as $doc) {
  echo "{$doc["name"]} is a {$doc["job"]}.\n";
}
?>
```

The above example will output something similar to:

```
Marco is a Programmer.
Mike is a Manager.
```

Mysql_xdevapi Functions

Table of Contents

expression

expression

(No version information available, might only be in Git)

expression — Bind prepared statement variables as parameters

Description

function **mysql_xdevapi\expression**(string `$expression`): object

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`expression`

Return Values

Examples

Example #1 **mysql_xdevapi\Expression()** example

```php
<?php
$expression = mysql_xdevapi\Expression("[age,job]");

$res  = $coll->find("age > 30")->fields($expression)->limit(3)->execute();
$data = $res->fetchAll();

print_r($data);
?>
```

The above example will output something similar to:

```
<?php
```

getSession

getSession

(No version information available, might only be in Git)

getSession — Connect to a MySQL server

Description

function **mysql_xdevapi\getSession**(string `$uri`): mysql_xdevapi\Session

Connects to the MySQL server.

Parameters

`uri`

The URI to the MySQL server, such as `mysqlx://user:password@host`.

URI format:

`scheme://[user[:[password]]@]target[:port][?attribute1=value1&attribute2=value2...`

- `scheme`: required, the connection protocol

  In mysql_xdevapi it is always 'mysqlx' (for X Protocol)

- `user`: optional, the MySQL user account for authentication
- `password`: optional, the MySQL user's password for authentication
- `target`: required, the server instance the connection refers to:

  * TCP connection (host name, IPv4 address, or IPv6 address)

  * Unix socket path (local file path)

  * Windows named-pipe (local file path)

- `port`: optional, network port of MySQL server.

  by default port for X Protocol is 33060

- `?attribute=value`: this element is optional and specifies a data dictionary that contains different options, including:

  - The `auth` (authentication mechanism) attribute as it relates to encrypted connections. For additional information, see » Command Options for Encrypted Connections. The following 'auth' values are supported: `plain`, `mysql41`, `external`, and `sha256_mem`.

  - The `connect-timeout` attribute affects the connection and not subsequent operations. It is set per connection whether on a single or multiple hosts.

    Pass in a positive integer to define the connection timeout in seconds, or pass in 0 (zero) to disable the timeout (infinite). Not defining connect-timeout uses the default value of 10.

Related, the MYSQLX_CONNECTION_TIMEOUT (timeout in seconds) and MYSQLX_TEST_CONNECTION_TIMEOUT (used while running tests) environment variables can be set and used instead of connect-timeout in the URI. The connect-timeout URI option has precedence over these environment variables.

- The optional `compression` attribute accepts these values: `preferred` (client negotiates with server to find a supported algorithm; connection is uncompressed if a mutually supported algorithm is not found), `required` (like "preferred", but connection is terminated if a mutually supported algorithm is not found), or `disabled` (connection is uncompressed). Defaults to `preferred`.

  This option was added in version 8.0.20.

- The optional `compression-algorithms` attribute defines the desired compression algorithms (and their preferred usage order): `zstd_stream` (alias: zstd), `lz4_message` (alias: lz4), or `deflate_stream` (aliases: deflate or zlib). By default, the order used (depending on system availability) is lz4_message, zstd_stream, then deflate_stream. For example, passing in compression-algorithms=[lz4,zstd_stream] uses lz4 if it's available, otherwise zstd_stream is used. If both are unavailable then behavior depends on the compression value e.g., if compression=required then it'll fail with an error.

  This option was added in version 8.0.22.

Example #1 URI examples

```
mysqlx://foobar
mysqlx://root@localhost?socket=%2Ftmp%2Fmysqld.sock%2F
mysqlx://foo:bar@localhost:33060
mysqlx://foo:bar@localhost:33160?ssl-mode=disabled
mysqlx://foo:bar@localhost:33260?ssl-mode=required
mysqlx://foo:bar@localhost:33360?ssl-mode=required&auth=mysql41
mysqlx://foo:bar@(/path/to/socket)
mysqlx://foo:bar@(/path/to/socket)?auth=sha256_mem
mysqlx://foo:bar@[localhost:33060, 127.0.0.1:33061]
mysqlx://foobar?ssl-ca=(/path/to/ca.pem)&ssl-crl=(/path/to/crl.pem)
mysqlx://foo:bar@[localhost:33060, 127.0.0.1:33061]?ssl-mode=disabled
mysqlx://foo:bar@localhost:33160/?connect-timeout=0
mysqlx://foo:bar@localhost:33160/?connect-timeout=10&compression=required
mysqlx://foo:bar@localhost:33160/?connect-timeout=10&compression=required&compression-algorithms=
[lz4,zstd_stream]
```

For related information, see MySQL Shell's » Connecting using a URI String.

Return Values

A **Session** object.

Errors/Exceptions

A connection failure throws an Exception.

Examples

Example #2 mysql_xdevapi\getSession() example

```php
<?php
try {
    $session = mysql_xdevapi\getSession("mysqlx://user:password@host");
} catch(Exception $e) {
    die("Connection could not be established: " . $e->getMessage());
}

$schemas = $session->getSchemas();
print_r($schemas);

$mysql_version = $session->getServerVersion();
print_r($mysql_version);

var_dump($collection->find("name = 'Alfred'")->execute()->fetchOne());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => mysql_xdevapi\Schema Object
        (
            [name] => helloworld
        )
    [1] => mysql_xdevapi\Schema Object
        (
            [name] => information_schema
        )
    [2] => mysql_xdevapi\Schema Object
        (
            [name] => mysql
        )
    [3] => mysql_xdevapi\Schema Object
        (
            [name] => performance_schema
        )
    [4] => mysql_xdevapi\Schema Object
        (
            [name] => sys
        )
)

80012

array(4) {
  ["_id"]=>
  string(28) "00005ad66abf0001000400000003"
  ["age"]=>
  int(42)
  ["job"]=>
  string(7) "Butler"
  ["name"]=>
  string(4) "Alfred"
}
```

mysql_xdevapi\BaseResult

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\BaseResult** {
/* Methods */
abstract public function **getWarnings**(): array
abstract public function **getWarningsCount**(): int
}

Table of Contents

BaseResult::getWarnings

BaseResult::getWarnings

(No version information available, might only be in Git)

BaseResult::getWarnings — Fetch warnings from last operation

Description

abstract public function **mysql_xdevapi\BaseResult::getWarnings**(): array

Fetches warnings generated by MySQL server's last operation.

Parameters

This function has no parameters.

Return Values

An array of Warning objects from the last operation. Each object defines an error 'message', error 'level', and error 'code'. An empty array is returned if no errors are present.

Examples

Example #1 mysql_xdevapi\RowResult::getWarnings() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();

$schema = $session->getSchema("foo");
$table  = $schema->getTable("test_table");

$table->insert(['x'])->values([1])->values([2])->execute();

$res = $table->select(['x/0 as bad_x'])->execute();
$warnings = $res->getWarnings();
```

```
print_r($warnings);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => mysql_xdevapi\Warning Object
        (
            [message] => Division by 0
            [level] => 2
            [code] => 1365
        )
    [1] => mysql_xdevapi\Warning Object
        (
            [message] => Division by 0
            [level] => 2
            [code] => 1365
        )
)
```

BaseResult::getWarningsCount

BaseResult::getWarningsCount

(No version information available, might only be in Git)

BaseResult::getWarningsCount — Fetch warning count from last operation

Description

abstract public function **mysql_xdevapi\BaseResult::getWarningsCount**(): int

Returns the number of warnings raised by the last operation. Specifically, these warnings are raised by the MySQL server.

Parameters

This function has no parameters.

Return Values

The number of warnings from the last operation.

Examples

Example #1 mysql_xdevapi\RowResult::getWarningsCount() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
```

```php
$session->sql("DROP DATABASE IF EXISTS foo")->execute();
$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();

$schema = $session->getSchema("foo");
$table  = $schema->getTable("test_table");

$table->insert(['x'])->values([1])->values([2])->execute();

$res = $table->select(['x/0 as bad_x'])->execute();

echo $res->getWarningsCount();
?>
```

The above example will output something similar to:

```
2
```

mysql_xdevapi\Client

(No version information available, might only be in Git)

Introduction

Provides access to the connection pool.

Class synopsis

class **mysql_xdevapi\Client** {
/* Methods */
public function close(): bool
public function **getSession**(): mysql_xdevapi\Session
}

Table of Contents

mysql_xdevapi\Client::close

mysql_xdevapi\Client::close

(No version information available, might only be in Git)

mysql_xdevapi\Client::close — Close client

Description

public function **mysql_xdevapi\Client::close**(): bool

Close all client connections with the server.

Parameters

This function has no parameters.

Return Values

`true` if connections are closed.

Client::__construct

Client::__construct

(No version information available, might only be in Git)

Client::__construct — Client constructor

Description

private function **mysql_xdevapi\Client::__construct**()

Construct a client object.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\Client::__construct() example

```
<?php
$pooling_options = '{
  "enabled": true,
    "maxSize": 10,
    "maxIdleTime": 3600,
    "queueTimeOut": 1000
}';
$client = mysql_xdevapi\getClient($connection_uri, $pooling_options);
$session = $client->getSession();
```

Client::getClient

Client::getClient

(No version information available, might only be in Git)

Client::getClient — Get client session

Description

public function **mysql_xdevapi\Client::getSession**(): mysql_xdevapi\Session

Get session associated with the client.

Parameters

This function has no parameters.

Return Values

A Session object.

mysql_xdevapi\Collection

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\Collection** implements mysql_xdevapi\SchemaObject {
/* Properties */
public *$name*;
/* Methods */
public function **add**(mixed `$document`): mysql_xdevapi\CollectionAdd
public function **addOrReplaceOne**(string `$id`, string `$doc`): mysql_xdevapi\Result
public function **count**(): int
public function **createIndex**(string `$index_name`, string `$index_desc_json`): void
public function **dropIndex**(string `$index_name`): bool
public function **existsInDatabase**(): bool
public function **find**(string `$search_condition = ?`): mysql_xdevapi\CollectionFind
public function **getName**(): string
public function **getOne**(string `$id`): Document
public function **getSchema**(): Schema Object
public function **getSession**(): Session
public function **modify**(string `$search_condition`): mysql_xdevapi\CollectionModify
public function **remove**(string `$search_condition`): mysql_xdevapi\CollectionRemove
public function **removeOne**(string `$id`): mysql_xdevapi\Result
public function **replaceOne**(string `$id`, string `$doc`): mysql_xdevapi\Result
}

Properties

name

Table of Contents

Collection::add

Collection::add

(No version information available, might only be in Git)

Collection::add — Add collection document

Description

public function **mysql_xdevapi\Collection::add**(mixed `$document`): mysql_xdevapi\CollectionAdd

Triggers the insertion of the given document(s) into the collection, and multiple variants of this method are supported. Options include:

1. Add a single document as a JSON string.
2. Add a single document as an array as: `[ 'field' => 'value', 'field2' => 'value2' ... ]`
3. A mix of both, and multiple documents can be added in the same operation.

Parameters

`document`

> One or multiple documents, and this can be either JSON or an array of fields with their associated values. This cannot be an empty array.
>
> The MySQL server automatically generates unique `_id` values for each document (recommended), although this can be manually added as well. This value must be unique as otherwise the add operation will fail.

Return Values

A CollectionAdd object. Use execute() to return a Result that can be used to query the number of affected items, the number warnings generated by the operation, or to fetch a list of generated IDs for the inserted documents.

Examples

Example #1 mysql_xdevapi\Collection::add() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

// Add two documents
$collection->add('{"name": "Fred",  "age": 21, "job": "Construction"}')->execute();
$collection->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')->execute();

// Add two documents using a single JSON object
$result = $collection->add(
  '{"name": "Bernie",
    "jobs": [{"title":"Cat Herder","Salary":42000}, {"title":"Father","Salary":0}],
    "hobbies": ["Sports","Making cupcakes"]}',
  '{"name": "Jane",
    "jobs": [{"title":"Scientist","Salary":18000}, {"title":"Mother","Salary":0}],
    "hobbies": ["Walking","Making pies"]}')->execute();

// Fetch a list of generated ID's from the last add()
$ids = $result->getGeneratedIds();
print_r($ids);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => 00005b6b53610000000000000056
    [1] => 00005b6b53610000000000000057
)
```

Notes

> **Note** :
>
> A unique _id is generated by MySQL Server 8.0 or higher, as demonstrated in the example. The _id field must be manually defined if using MySQL Server 5.7.

Collection::addOrReplaceOne

Collection::addOrReplaceOne

(No version information available, might only be in Git)

Collection::addOrReplaceOne — Add or replace collection document

Description

public function **mysql_xdevapi\Collection::addOrReplaceOne**(string `$id`, string `$doc`): mysql_xdevapi\Result

Add a new document, or replace a document if it already exists.

Here are several scenarios for this method:

- If neither the id or any unique key values conflict with any document in the collection, then the document is added.
- If the id does not match any document but one or more unique key values conflict with a document in the collection, then an error is raised.
- If id matches an existing document and no unique keys are defined for the collection, then the document is replaced.
- If id matches an existing document, and either all unique keys in the replacement document match that same document or they don't conflict with any other documents in the collection, then the document is replaced.
- If id matches an existing document and one or more unique keys match a different document from the collection, then an error is raised.

Parameters

`id`

This is the filter id. If this id or any other field that has a unique index already exists in the collection, then it will update the matching document instead.

By default, this id is automatically generated by MySQL Server when the record was added, and is referenced as a field named '_id'.

`doc`

This is the document to add or replace, which is a JSON string.

Return Values

A Result object.

Examples

Example #1 **mysql_xdevapi\Collection::addOrReplaceOne()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

// Using add()
$result = $collection->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')->execute();

// Using addOrReplaceOne()
// Note: we're passing in a known _id value here
$result = $collection->addOrReplaceOne('00005b6b536100000000000000056', '{"name": "Fred",  "age": 21,
"job": "Construction"}');

?>
```

Collection::__construct

Collection::__construct

(No version information available, might only be in Git)

Collection::__construct — Collection constructor

Description

private function **mysql_xdevapi\Collection::__construct**()

Construct a Collection object.

Parameters

This function has no parameters.

Examples

Example #1 **mysql_xdevapi\Collection::getOne()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");
```

```php
$result = $collection->add('{"name": "Alfred", "age": 42, "job": "Butler"}')->execute();

// A unique _id is (by default, and recommended) generated by MySQL Server
// This retrieves the generated _id's; only one in this example, so $ids[0]
$ids        = $result->getGeneratedIds();
$alfreds_id = $ids[0];

// ...

print_r($alfreds_id);
print_r($collection->getOne($alfreds_id));
?>
```

The above example will output something similar to:

```
00005b6b536100000000000000b1

Array
(
    [_id] => 00005b6b536100000000000000b1
    [age] => 42
    [job] => Butler
    [name] => Alfred
)
```

Collection::count

Collection::count

(No version information available, might only be in Git)

Collection::count — Get document count

Description

public function **mysql_xdevapi\Collection::count**(): int

This functionality is similar to a `SELECT COUNT(*)` SQL operation against the MySQL server for the current schema and collection. In other words, it counts the number of documents in the collection.

Parameters

This function has no parameters.

Return Values

The number of documents in the collection.

Examples

Example #1 mysql_xdevapi\Collection::count() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

$result = $collection
  ->add(
  '{"name": "Bernie",
    "jobs": [
      {"title":"Cat Herder","Salary":42000},
      {"title":"Father","Salary":0}
    ],
    "hobbies": ["Sports","Making cupcakes"]}',
  '{"name": "Jane",
    "jobs": [
      {"title":"Scientist","Salary":18000},
      {"title":"Mother","Salary":0}
    ],
    "hobbies": ["Walking","Making pies"]}')
  ->execute();

var_dump($collection->count());
?>
```

The above example will output:

```
int(2)
```

Collection::createIndex

Collection::createIndex

(No version information available, might only be in Git)

Collection::createIndex — Create collection index

Description

public function **mysql_xdevapi\Collection::createIndex**(string `$index_name`, string `$index_desc_json`): void

Creates an index on the collection.

An exception is thrown if an index with the same name already exists, or if index definition is not correctly formed.

Parameters

`index_name`

The name of the index that to create. This name must be a valid index name as accepted by the `CREATE INDEX` SQL query.

`index_desc_json`

Definition of the index to create. It contains an array of IndexField objects, and each object describes a single document member to include in the index, and an optional string for the type of index that might be INDEX (default) or SPATIAL.

A single IndexField description consists of the following fields:

- `field`: string, the full document path to the document member or field to be indexed.
- `type`: string, one of the supported SQL column types to map the field into. For numeric types, the optional UNSIGNED keyword may follow. For the TEXT type, the length to consider for indexing may be added.
- `required`: bool, (optional) true if the field is required to exist in the document. Defaults to `false`, except for `GEOJSON` where it defaults to `true`.
- `options`: integer, (optional) special option flags for use when decoding `GEOJSON` data.
- `srid`: integer, (optional) srid value for use when decoding `GEOJSON` data.

It is an error to include other fields not described above in IndexDefinition or IndexField documents.

Return Values

Examples

Example #1 mysql_xdevapi\Collection::createIndex() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

// Creating a text index
$collection->createIndex(
  'myindex1',
  '{"fields": [{
    "field": "$.name",
    "type": "TEXT(25)",
    "required": true}],
    "unique": false}'
);

// A spatial index
$collection->createIndex(
  'myindex2',
  '{"fields": [{
    "field": "$.home",
    "type": "GEOJSON",
```

```
    "required": true}],
    "type": "SPATIAL"}'
);

// Index with multiple fields
$collection->createIndex(
  'myindex3',
  '{"fields": [
    {
      "field": "$.name",
      "type": "TEXT(20)",
      "required": true
    },
    {
      "field": "$.age",
      "type": "INTEGER"
    },
    {
      "field": "$.job",
      "type": "TEXT(30)",
      "required": false
    }
  ],
  "unique": true
  }'
);
```

Collection::dropIndex

Collection::dropIndex

(No version information available, might only be in Git)

Collection::dropIndex — Drop collection index

Description

public function **mysql_xdevapi\Collection::dropIndex**(string `$index_name`): bool

Drop a collection index.

This operation does not yield an error if the index does not exist, but `false` is returned in that case.

Parameters

`index_name`

Name of collection index to drop.

Return Values

`true` if the DROP INDEX operation succeeded, otherwise `false`.

Examples

Example #1 mysql_xdevapi\Collection::dropIndex() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
```

```
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

// ...

$collection = $schema->getCollection("people");

$collection->createIndex(
  'myindex',
  '{"fields": [{"field": "$.name", "type": "TEXT(25)", "required": true}], "unique": false}'
);

// ...

if ($collection->dropIndex('myindex')) {
    echo "An index named 'myindex' was found, and dropped.";
}
?>
```

The above example will output:

```
An index named 'myindex' was found, and dropped.
```

Collection::existsInDatabase

Collection::existsInDatabase

(No version information available, might only be in Git)

Collection::existsInDatabase — Check if collection exists in database

Description

public function **mysql_xdevapi\Collection::existsInDatabase**(): bool

Checks if the Collection object refers to a collection in the database (schema).

Parameters

This function has no parameters.

Return Values

Returns `true` if collection exists in the database, else `false` if it does not.

A table defined with two columns (doc and _id) is considered a collection, and a third _json_schema column as of MySQL 8.0.21. Adding an additional column means existsInDatabase() will no longer see it as a collection.

Examples

Example #1 mysql_xdevapi\Collection::existsInDatabase() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

// ...

$collection = $schema->getCollection("people");

// ...

if (!$collection->existsInDatabase()) {
    echo "The collection no longer exists in the database named addressbook. What happened?";
}
?>
```

Collection::find

Collection::find

(No version information available, might only be in Git)

Collection::find — Search for document

Description

public function **mysql_xdevapi\Collection::find**(string `$search_condition` = ?): mysql_xdevapi\CollectionFind

Search a database collection for a document or set of documents. The found documents are returned as a CollectionFind object is to further modify or fetch results from.

Parameters

`search_condition`

> Although optional, normally a condition is defined to limit the results to a subset of documents.
>
> Multiple elements might build the condition and the syntax supports parameter binding. The expression used as search condition must be a valid SQL expression. If no search condition is provided (field empty) then find('true') is assumed.

Return Values

A CollectionFind object to verify the operation, or fetch the found documents.

Examples

Example #1 mysql_xdevapi\Collection::find() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
```

```php
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$collection->add('{"name": "Alfred",     "age": 18, "job": "Butler"}')->execute();
$collection->add('{"name": "Bob",        "age": 19, "job": "Swimmer"}')->execute();
$collection->add('{"name": "Fred",       "age": 20, "job": "Construction"}')->execute();
$collection->add('{"name": "Wilma",      "age": 21, "job": "Teacher"}')->execute();
$collection->add('{"name": "Suki",       "age": 22, "job": "Teacher"}')->execute();

$find   = $collection->find('job LIKE :job AND age > :age');
$result = $find
  ->bind(['job' => 'Teacher', 'age' => 20])
  ->sort('age DESC')
  ->limit(2)
  ->execute();

print_r($result->fetchAll());
?>
```

The above example will output:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b536100000000000000a8
            [age] => 22
            [job] => Teacher
            [name] => Suki
        )
    [1] => Array
        (
            [_id] => 00005b6b536100000000000000a7
            [age] => 21
            [job] => Teacher
            [name] => Wilma
        )
)
```

Collection::getName

Collection::getName

(No version information available, might only be in Git)

Collection::getName — Get collection name

Description

public function **mysql_xdevapi\Collection::getName**(): string

Retrieve the collection's name.

Parameters

This function has no parameters.

Return Values

The collection name, as a string.

Examples

Example #1 mysql_xdevapi\Collection::getName() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");


// ...

var_dump($collection->getName());
?>
```

The above example will output something similar to:

```
string(6) "people"
```

Collection::getOne

Collection::getOne

(No version information available, might only be in Git)

Collection::getOne — Get one document

Description

public function **mysql_xdevapi\Collection::getOne**(string `$id`): Document

Fetches one document from the collection.

This is a shortcut for: `Collection.find("_id = :id").bind("id", id).execute().fetchOne();`

Parameters

`id`

The document _id in the collection.

Return Values

The collection object, or `null` if the _id does not match a document.

Examples

Example #1 mysql_xdevapi\Collection::getOne() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection->add('{"name": "Alfred", "age": 42, "job": "Butler"}')->execute();

// A unique _id is (by default, and recommended) generated by MySQL Server
// This retrieves the generated _id's; only one in this example, so $ids[0]
$ids        = $result->getGeneratedIds();
$alfreds_id = $ids[0];

// ...

print_r($alfreds_id);
print_r($collection->getOne($alfreds_id));
?>
```

The above example will output something similar to:

```
00005b6b536100000000000000b1

Array
(
    [_id] => 00005b6b536100000000000000b1
    [age] => 42
    [job] => Butler
    [name] => Alfred
)
```

Collection::getSchema

Collection::getSchema

(No version information available, might only be in Git)

Collection::getSchema — Get schema object

Description

public function **mysql_xdevapi\Collection::getSchema**(): Schema Object

Retrieve the schema object that contains the collection.

Parameters

This function has no parameters.

Return Values

The schema object on success, or `null` if the object cannot be retrieved for the given collection.

Examples

Example #1 mysql_xdevapi\Collection::getSchema() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

var_dump($collection->getSchema());
?>
```

The above example will output something similar to:

```
object(mysql_xdevapi\Schema)#9 (1) {
  ["name"]=>
  string(11) "addressbook"
}
```

Collection::getSession

Collection::getSession

(No version information available, might only be in Git)

Collection::getSession — Get session object

Description

public function **mysql_xdevapi\Collection::getSession**(): Session

Get a new Session object from the Collection object.

Parameters

This function has no parameters.

Return Values

A Session object.

Examples

Example #1 **mysql_xdevapi\Collection::getSession()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

// ...

$newsession = $collection->getSession();

var_dump($session);
var_dump($newsession);
?>
```

The above example will output something similar to:

```
object(mysql_xdevapi\Session)#1 (0) {
}
object(mysql_xdevapi\Session)#4 (0) {
}
```

Collection::modify

Collection::modify

(No version information available, might only be in Git)

Collection::modify — Modify collection documents

Description

public function **mysql_xdevapi\Collection::modify**(string `$search_condition`): mysql_xdevapi\CollectionModify

Modify the documents of a collection that meet specific search conditions. Multiple operations are allowed, and parameter binding is supported.

Parameters

`search_condition`

Must be a valid SQL expression used to match the documents to modify. This expression might be as simple as `true`, which matches all documents, or it might use functions and operators such as `'CAST(_id AS SIGNED) >= 10'`, `'age MOD 2 = 0 OR age MOD 3 = 0'`, or `'_id IN ["2","5","7","10"]'`.

Return Values

If the operation is not executed, then the function will return a Modify object that can be used to add additional modify operations.

If the modify operation is executed, then the returned object will contain the result of the operation.

Examples

Example #1 mysql_xdevapi\Collection::modify() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$collection->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$collection->add('{"name": "Bob",    "age": 19, "job": "Painter"}')->execute();

// Add two new jobs for all Painters: Artist and Crafter
$collection
  ->modify("job in ('Butler', 'Painter')")
  ->arrayAppend('job', 'Artist')
  ->arrayAppend('job', 'Crafter')
  ->execute();

// Remove the 'beer' field from all documents with the age 21
$collection
  ->modify('age < 21')
  ->unset(['beer'])
  ->execute();
?>
```

Collection::remove

Collection::remove

(No version information available, might only be in Git)

Collection::remove — Remove collection documents

Description

public function **mysql_xdevapi\Collection::remove**(string `$search_condition`): mysql_xdevapi\CollectionRemove

Removes documents of a collection that meet specific search conditions. Multiple operations are allowed, and parameter binding is supported.

Parameters

`search_condition`

> Must be a valid SQL expression used to match the documents to remove. This expression might be as simple as `true`, which matches all documents, or it might use functions and operators such as `'CAST(_id AS SIGNED) >= 10'`, `'age MOD 2 = 0 OR age MOD 3 = 0'`, or `'_id IN ["2","5","7","10"]'`.

Return Values

If the operation is not executed, then the function will return a Remove object that can be used to add additional remove operations.

If the remove operation is executed, then the returned object will contain the result of the operation.

Examples

Example #1 mysql_xdevapi\Collection::remove() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$collection->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$collection->add('{"name": "Bob",    "age": 19, "job": "Painter"}')->execute();

// Remove all painters
$collection
  ->remove("job in ('Painter')")
  ->execute();

// Remove the oldest butler
$collection
  ->remove("job in ('Butler')")
  ->sort('age desc')
  ->limit(1)
  ->execute();

// Remove record with highest age
$collection
  ->remove('true')
  ->sort('age desc')
  ->limit(1)
  ->execute();
?>
```

Collection::removeOne

Collection::removeOne

(No version information available, might only be in Git)

Collection::removeOne — Remove one collection document

Description

public function **mysql_xdevapi\Collection::removeOne**(string `$id`): mysql_xdevapi\Result

Remove one document from the collection with the corresponding ID. This is a shortcut for `Collection.remove("_id = :id").bind("id", id).execute()`.

Parameters

`id`

The ID of the collection document to remove. Typically this is the _id that was generated by MySQL Server when the record was added.

Return Values

A Result object that can be used to query the number of affected items or the number warnings generated by the operation.

Examples

Example #1 **mysql_xdevapi\Collection::removeOne()** example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();

// Normally the _id is known by other means,
// but for this example let's fetch the generated id and use it
$ids       = $result->getGeneratedIds();
$alfred_id = $ids[0];

$result = $collection->removeOne($alfred_id);

if(!$result->getAffectedItemsCount()) {
    echo "Alfred with id $alfred_id was not removed.";
} else {
    echo "Goodbye, Alfred, you can take _id $alfred_id with you.";
}
?>
```

The above example will output something similar to:

```
Goodbye, Alfred, you can take _id 00005b6b536100000000000000cb with you.
```

Collection::replaceOne

Collection::replaceOne

(No version information available, might only be in Git)

Collection::replaceOne — Replace one collection document

Description

public function **mysql_xdevapi\Collection::replaceOne**(string `$id`, string `$doc`): mysql_xdevapi\Result

Updates (or replaces) the document identified by ID, if it exists.

Parameters

`id`

ID of the document to replace or update. Typically this is the _id that was generated by MySQL Server when the record was added.

`doc`

Collection document to update or replace the document matching the **id** parameter.

This document can be either a document object or a valid JSON string describing the new document.

Return Values

A Result object that can be used to query the number of affected items and the number warnings generated by the operation.

Examples

Example #1 **mysql_xdevapi\Collection::replaceOne()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();

// Normally the _id is known by other means,
// but for this example let's fetch the generated id and use it
$ids       = $result->getGeneratedIds();
$alfred_id = $ids[0];

// ...

$alfred = $collection->getOne($alfred_id);
$alfred['age'] = 81;
$alfred['job'] = 'Guru';

$collection->replaceOne($alfred_id, $alfred);
```

```
?>
```

mysql_xdevapi\CollectionAdd

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\CollectionAdd** implements mysql_xdevapi\Executable {
/* Methods */
public function **execute**(): mysql_xdevapi\Result
}

Table of Contents

CollectionAdd::__construct

CollectionAdd::__construct

(No version information available, might only be in Git)

CollectionAdd::__construct — CollectionAdd constructor

Description

private function **mysql_xdevapi\CollectionAdd::__construct**()

Use to add a document to a collection; called from a Collection object.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\CollectionAdd::__construct() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

// Add two documents
$collection
  ->add('{"name": "Fred",  "age": 21, "job": "Construction"}')
  ->execute();

$collection
  ->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')
  ->execute();
```

```
// Add two documents using a single JSON object
$result = $collection
  ->add(
    '{"name": "Bernie",
      "jobs": [{"title":"Cat Herder","Salary":42000}, {"title":"Father","Salary":0}],
      "hobbies": ["Sports","Making cupcakes"]}',
    '{"name": "Jane",
      "jobs": [{"title":"Scientist","Salary":18000}, {"title":"Mother","Salary":0}],
      "hobbies": ["Walking","Making pies"]}')
  ->execute();

// Fetch a list of generated ID's from the last add()
$ids = $result->getGeneratedIds();
print_r($ids);

?>
```

The above example will output something similar to:

```
Array
(
    [0] => 00005b6b53610000000000000056
    [1] => 00005b6b53610000000000000057
)
```

Notes

> **Note** :
>
> A unique _id is generated by MySQL Server 8.0 or higher, as demonstrated in the example. The _id field must be manually defined if using MySQL Server 5.7.

CollectionAdd::execute

CollectionAdd::execute

(No version information available, might only be in Git)

CollectionAdd::execute — Execute the statement

Description

public function **mysql_xdevapi\CollectionAdd::execute**(): mysql_xdevapi\Result

The execute method is required to send the CRUD operation request to the MySQL server.

Parameters

This function has no parameters.

Return Values

A Result object that can be used to verify the status of the operation, such as the number of affected rows.

Examples

Example #1 **mysql_xdevapi\CollectionAdd::execute()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

// Add two documents
$collection
  ->add('{"name": "Fred",  "age": 21, "job": "Construction"}')
  ->execute();

$collection
  ->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')
  ->execute();

// Add two documents using a single JSON object
$result = $collection
  ->add(
    '{"name": "Bernie",
      "jobs": [{"title":"Cat Herder","Salary":42000}, {"title":"Father","Salary":0}],
      "hobbies": ["Sports","Making cupcakes"]}',
    '{"name": "Jane",
      "jobs": [{"title":"Scientist","Salary":18000}, {"title":"Mother","Salary":0}],
      "hobbies": ["Walking","Making pies"]}')
  ->execute();

// Fetch a list of generated ID's from the last add()
$ids = $result->getGeneratedIds();
print_r($ids);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => 00005b6b53610000000000000056
    [1] => 00005b6b53610000000000000057
)
```

mysql_xdevapi\CollectionFind

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\CollectionFind** implements mysql_xdevapi\Executable, mysql_xdevapi\CrudOperationBindable, mysql_xdevapi\CrudOperationLimitable, mysql_xdevapi\CrudOperationSortable {
/* Methods */
public function **bind**(array `$placeholder_values`): mysql_xdevapi\CollectionFind
public function **execute**(): mysql_xdevapi\DocResult
public function **fields**(string `$projection`): mysql_xdevapi\CollectionFind
public function **groupBy**(string `$sort_expr`): mysql_xdevapi\CollectionFind
public function **having**(string `$sort_expr`): mysql_xdevapi\CollectionFind
public function **limit**(int `$rows`): mysql_xdevapi\CollectionFind
public function **lockExclusive**(int `$lock_waiting_option` = ?): mysql_xdevapi\CollectionFind
public function **lockShared**(int `$lock_waiting_option` = ?): mysql_xdevapi\CollectionFind
public function **offset**(int `$position`): mysql_xdevapi\CollectionFind
public function **sort**(string `$sort_expr`): mysql_xdevapi\CollectionFind
}

Table of Contents

CollectionFind::bind

CollectionFind::bind

(No version information available, might only be in Git)

CollectionFind::bind — Bind value to query placeholder

Description

public function **mysql_xdevapi\CollectionFind::bind**(array `$placeholder_values`): mysql_xdevapi\CollectionFind

It allows the user to bind a parameter to the placeholder in the search condition of the find operation. The placeholder has the form of :NAME where ':' is a common prefix that must always exists before any NAME, NAME is the actual name of the placeholder. The bind function accepts a list of placeholders if multiple entities have to be substituted in the search condition.

Parameters

`placeholder_values`

> Values to substitute in the search condition; multiple values are allowed and are passed as an array where "PLACEHOLDER_NAME => PLACEHOLDER_VALUE".

Return Values

A CollectionFind object, or chain with execute() to return a Result object.

Examples

Example #1 mysql_xdevapi\CollectionFind::bind() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");
$result = $create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();

// ...

$collection = $schema->getCollection("people");

$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->execute();

var_dump($result->fetchAll());
?>
```

The above example will output something similar to:

```
array(1) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000cf"
    ["age"]=>
    int(18)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(6) "Alfred"
  }
}
```

CollectionFind::__construct

CollectionFind::__construct

(No version information available, might only be in Git)

CollectionFind::__construct — CollectionFind constructor

Description

private function **mysql_xdevapi\CollectionFind::__construct**()

> **Warning**

> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Examples

Example #1 CollectionFind example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");
$result = $create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();

// ...

$collection = $schema->getCollection("people");

$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->execute();

var_dump($result->fetchAll());
?>
```

The above example will output something similar to:

```
array(1) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000cf"
    ["age"]=>
    int(18)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(6) "Alfred"
  }
}
```

CollectionFind::execute

CollectionFind::execute

(No version information available, might only be in Git)

CollectionFind::execute — Execute the statement

Description

public function **mysql_xdevapi\CollectionFind::execute**(): mysql_xdevapi\DocResult

Execute the find operation; this functionality allows for method chaining.

Parameters

This function has no parameters.

Return Values

A mysql_xdevapi\DocResult object that can be used to either fetch results from, or to query the status of the operation.

Examples

Example #1 CollectionFind example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();

// ...

$collection = $schema->getCollection("people");

$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->execute();

var_dump($result->fetchAll());
?>
```

The above example will output something similar to:

```
array(1) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000cf"
```

```
    ["age"]=>
    int(18)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(6) "Alfred"
  }
}
```

CollectionFind::fields

CollectionFind::fields

(No version information available, might only be in Git)

CollectionFind::fields — Set document field filter

Description

public function **mysql_xdevapi\CollectionFind::fields**(string `$projection`): mysql_xdevapi\CollectionFind

Defines the columns for the query to return. If not defined, all columns are used.

Parameters

`projection`

> Can either be a single string or an array of strings identifying the columns to be returned for each document that match the search condition.

Return Values

A CollectionFind object that can be used for further processing.

Examples

Example #1 mysql_xdevapi\CollectionFind::fields() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();

// ...

$collection = $schema->getCollection("people");
```

```php
$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->fields('name')
  ->execute();

var_dump($result->fetchAll());
?>
```

The above example will output something similar to:

```
array(1) {
  [0]=>
  array(1) {
    ["name"]=>
    string(6) "Alfred"
  }
}
```

CollectionFind::groupBy

CollectionFind::groupBy

(No version information available, might only be in Git)

CollectionFind::groupBy — Set grouping criteria

Description

public function **mysql_xdevapi\CollectionFind::groupBy**(string `$sort_expr`): mysql_xdevapi\CollectionFind

This function can be used to group the result set by one or more columns. It is often used with aggregate functions such as `COUNT`, `MAX`, `MIN`, `SUM` etc.

Parameters

`sort_expr`

> The column or columns that have to be used for the group operation, this can either be a single string or an array of string arguments, one for each column.

Return Values

A CollectionFind object that can be used for further processing.

Examples

Example #1 mysql_xdevapi\CollectionFind::groupBy() example

```php
<?php

// Assuming $coll is a valid Collection object
```

```php
// Extract all the documents from the Collection and group the results by the 'name' field
$res = $coll->find()->groupBy('name')->execute();

?>
```

CollectionFind::having

CollectionFind::having

(No version information available, might only be in Git)

CollectionFind::having — Set condition for aggregate functions

Description

public function **mysql_xdevapi\CollectionFind::having**(string `$sort_expr`): mysql_xdevapi\CollectionFind

This function can be used after the 'field' operation in order to make a selection on the documents to extract.

Parameters

`sort_expr`

This must be a valid SQL expression, the use of aggreate functions is allowed

Return Values

CollectionFind object that can be used for further processing

Examples

Example #1 mysql_xdevapi\CollectionFind::having() example

```php
<?php

//Assuming $coll is a valid Collection object

//Find all the documents for which the 'age' is greather than 40,
//Only the columns 'name' and 'age' are returned in the Result object
$res = $coll->find()->fields(['name','age'])->having('age > 40')->execute();

?>
```

CollectionFind::limit

CollectionFind::limit

(No version information available, might only be in Git)

CollectionFind::limit — Limit number of returned documents

Description

public function **mysql_xdevapi\CollectionFind::limit**(int `$rows`): mysql_xdevapi\CollectionFind

Set the maximum number of documents to return.

Parameters

`rows`

Maximum number of documents.

Return Values

A CollectionFind object that can be used for additional processing; chain with the execute() method to return a DocResult object.

Examples

Example #1 mysql_xdevapi\CollectionFind::limit() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");
$create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();
$create
  ->add('{"name": "Reginald", "age": 42, "job": "Butler"}')
  ->execute();


// ...

$collection = $schema->getCollection("people");

$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->sort('age desc')
  ->limit(1)
  ->execute();

var_dump($result->fetchAll());
?>
```

The above example will output something similar to:

```
array(1) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000f3"
    ["age"]=>
    int(42)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
```

```
    string(8) "Reginald"
  }
}
```

CollectionFind::lockExclusive

CollectionFind::lockExclusive

(No version information available, might only be in Git)

CollectionFind::lockExclusive — Execute operation with EXCLUSIVE LOCK

Description

public function **mysql_xdevapi\CollectionFind::lockExclusive**(int `$lock_waiting_option = ?`): mysql_xdevapi\CollectionFind

Locks the document exclusively. As long as the document is locked, other transactions can't update the document, use `SELECT ... LOCK IN SHARE MODE`, or read the data in certain transaction isolation levels. Consistent reads ignore any locks set on the records that exist in the read view.

To avoid concurrency problems, it makes sense to use this function with the **mysql_xdevapi\Collection::modify()** method. Essentially, this function uses row locks to serialise access to rows.

Parameters

`lock_waiting_option`

Optional waiting option. By default it is `MYSQLX_LOCK_DEFAULT`. Valid values are these constants:

- `MYSQLX_LOCK_DEFAULT`
- `MYSQLX_LOCK_NOWAIT`
- `MYSQLX_LOCK_SKIP_LOCKED`

Return Values

Returns a CollectionFind object that can be used for further processing.

Examples

Example #1 mysql_xdevapi\CollectionFind::lockExclusive() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$session->startTransaction();

$result = $collection
  ->find("age > 50")
  ->lockExclusive()
```

```
  ->execute();

// ... do an operation on the object

// Complete the transaction and unlock the document
$session->commit();
?>
```

CollectionFind::lockShared

CollectionFind::lockShared

(No version information available, might only be in Git)

CollectionFind::lockShared — Execute operation with SHARED LOCK

Description

public function **mysql_xdevapi\CollectionFind::lockShared**(int `$lock_waiting_option = ?`): mysql_xdevapi\CollectionFind

Allows to share the documents between multiple transactions which are locking in shared mode.

Other sessions can read the rows, but cannot modify them until your transaction commits.

If any of these rows were changed by another transaction that hasn't been committed, the query will wait until that transaction has ended to be able to use the latest values.

Parameters

`lock_waiting_option`

Optional waiting option. By default it is `MYSQLX_LOCK_DEFAULT`. Valid values are these constants:

- `MYSQLX_LOCK_DEFAULT`
- `MYSQLX_LOCK_NOWAIT`
- `MYSQLX_LOCK_SKIP_LOCKED`

Return Values

A CollectionFind object that can be used for further processing

Examples

Example #1 mysql_xdevapi\CollectionFind::lockShared() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$session->startTransaction();

$result = $collection
  ->find("age > 50")
```

```
  ->lockShared()
  ->execute();

// ... read the object in shared mode

// Complete the transaction and unlock the document
$session->commit();
?>
```

CollectionFind::offset

CollectionFind::offset

(No version information available, might only be in Git)

CollectionFind::offset — Skip given number of elements to be returned

Description

public function **mysql_xdevapi\CollectionFind::offset**(int `$position`): mysql_xdevapi\CollectionFind

Skip (offset) these number of elements that otherwise would be returned by the find operation. Use with the limit() method.

Defining an offset larger than the result set size results in an empty set.

Parameters

`position`

Number of elements to skip for the limit() operation.

Return Values

A CollectionFind object that can be used for additional processing.

Examples

Example #1 mysql_xdevapi\CollectionFind::offset() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");
$create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();
$create
  ->add('{"name": "Reginald", "age": 42, "job": "Butler"}')
  ->execute();

// ...

$collection = $schema->getCollection("people");

$result = $collection
```

```
  ->find()
  ->sort('age asc')
  ->offset(1)
  ->limit(1)
  ->execute();

var_dump($result->fetchAll());
?>
```

The above example will output something similar to:

```
array(1) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000f3"
    ["age"]=>
    int(42)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(8) "Reginald"
  }
}
```

CollectionFind::sort

CollectionFind::sort

(No version information available, might only be in Git)

CollectionFind::sort — Set the sorting criteria

Description

public function **mysql_xdevapi\CollectionFind::sort**(string `$sort_expr`): mysql_xdevapi\CollectionFind

Sort the result set by the field selected in the sort_expr argument. The allowed orders are ASC (Ascending) or DESC (Descending). This operation is equivalent to the 'ORDER BY' SQL operation and it follows the same set of rules.

Parameters

`sort_expr`

> One or more sorting expressions can be provided. The evaluation is from left to right, and each expression is separated by a comma.

Return Values

A CollectionFind object that can be used to execute the command, or to add additional operations.

Examples

Example #1 **mysql_xdevapi\CollectionFind::sort()** example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");
$create
  ->add('{"name": "Alfred", "age": 18, "job": "Butler"}')
  ->execute();
$create
  ->add('{"name": "Reginald", "age": 42, "job": "Butler"}')
  ->execute();

// ...

$collection = $schema->getCollection("people");

$result = $collection
  ->find()
  ->sort('job desc', 'age asc')
  ->execute();

var_dump($result->fetchAll());
?>
```

The above example will output something similar to:

```
array(2) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000106"
    ["age"]=>
    int(18)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(6) "Alfred"
  }
  [1]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000107"
    ["age"]=>
    int(42)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(8) "Reginald"
```

```
  }
}
```

mysql_xdevapi\CollectionModify

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\CollectionModify** implements mysql_xdevapi\Executable, mysql_xdevapi\CrudOperationBindable, mysql_xdevapi\CrudOperationLimitable, mysql_xdevapi\CrudOperationSkippable, mysql_xdevapi\CrudOperationSortable {
/* Methods */
public function **arrayAppend**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify
public function **arrayInsert**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify
public function **bind**(array `$placeholder_values`): mysql_xdevapi\CollectionModify
public function **execute**(): mysql_xdevapi\Result
public function **limit**(int `$rows`): mysql_xdevapi\CollectionModify
public function **patch**(string `$document`): mysql_xdevapi\CollectionModify
public function **replace**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify
public function **set**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify
public function **skip**(int `$position`): mysql_xdevapi\CollectionModify
public function **sort**(string `$sort_expr`): mysql_xdevapi\CollectionModify
public function **unset**(array `$fields`): mysql_xdevapi\CollectionModify
}

Table of Contents

CollectionModify::arrayAppend

CollectionModify::arrayAppend

(No version information available, might only be in Git)

CollectionModify::arrayAppend — Append element to an array field

Description

public function **mysql_xdevapi\CollectionModify::arrayAppend**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify

Add an element to a document's field, as multiple elements of a field are represented as an array. Unlike arrayInsert(), arrayAppend() always appends the new element at the end of the array, whereas arrayInsert() can define the location.

Parameters

`collection_field`

The identifier of the field where the new element is inserted.

`expression_or_literal`

The new element to insert at the end of the document field array.

Return Values

A CollectionModify object that can be used to execute the command, or to add additional operations.

Examples

Example #1 **mysql_xdevapi\CollectionModify::arrayAppend()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection
  ->add(
  '{"name":   "Bernie",
    "traits": ["Friend", "Brother", "Human"]}')
  ->execute();

$collection
  ->modify("name in ('Bernie', 'Jane')")
  ->arrayAppend('traits', 'Happy')
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b536100000000000010c
            [name] => Bernie
            [traits] => Array
                (
                    [0] => Friend
                    [1] => Brother
                    [2] => Human
                    [3] => Happy
```

```
                )
        )
)
```

CollectionModify::arrayInsert

CollectionModify::arrayInsert

(No version information available, might only be in Git)

CollectionModify::arrayInsert — Insert element into an array field

Description

public function **mysql_xdevapi\CollectionModify::arrayInsert**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify

Adds an element to a document's field, as multiple elements of a field are represented as an array. Unlike **mysql_xdevapi\CollectionModify::arrayAppend()** this method allows to specify where the new element is inserted by defining which item it is after, whereas **mysql_xdevapi\CollectionModify::arrayAppend()** always appends the new element at the end of the array.

Parameters

`collection_field`

> Identifies the item in the array after which the new element will be inserted. The format of this parameter is `FIELD_NAME[ INDEX ]` where FIELD_NAME is the name of the document field to add the element to, and INDEX is the INDEX of the element within the field.
>
> The INDEX field is zero based, so the first item of the array has an index of 0.

`expression_or_literal`

> The new element to insert after FIELD_NAME[INDEX]

Return Values

A CollectionModify object that can be used to execute the command, or to add additional operations

Examples

Example #1 mysql_xdevapi\CollectionModify::arrayInsert() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection
  ->add(
```

```
  '{"name":   "Bernie",
    "traits": ["Friend", "Brother", "Human"]}')
  ->execute();

$collection
  ->modify("name in ('Bernie', 'Jane')")
  ->arrayInsert('traits[1]', 'Happy')
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b5361000000000000010d
            [name] => Bernie
            [traits] => Array
                (
                    [0] => Friend
                    [1] => Happy
                    [2] => Brother
                    [3] => Human
                )
        )
)
```

CollectionModify::bind

CollectionModify::bind

(No version information available, might only be in Git)

CollectionModify::bind — Bind value to query placeholder

Description

public function **mysql_xdevapi\CollectionModify::bind**(array `$placeholder_values`): mysql_xdevapi\CollectionModify

Bind a parameter to the placeholder in the search condition of the modify operation.

The placeholder has the form of :NAME where ':' is a common prefix that must always exists before any NAME where NAME is the name of the placeholder. The bind method accepts a list of placeholders if multiple entities have to be substituted in the search condition of the modify operation.

Parameters

`placeholder_values`

Placeholder values to substitute in the search condition. Multiple values are allowed and have to be passed as an array of mappings PLACEHOLDER_NAME->PLACEHOLDER_VALUE.

Return Values

A CollectionModify object that can be used to execute the command, or to add additional operations.

Examples

Example #1 **mysql_xdevapi\CollectionModify::bind()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection
  ->add(
  '{"name":   "Bernie",
    "traits": ["Friend", "Brother", "Human"]}')
  ->execute();

$collection
  ->modify("name = :name")
  ->bind(['name' => 'Bernie'])
  ->arrayAppend('traits', 'Happy')
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b536100000000000000110
            [name] => Bernie
            [traits] => Array
                (
                    [0] => Friend
                    [1] => Brother
                    [2] => Human
                    [3] => Happy
```

```
                )
        )
)
```

CollectionModify::__construct

CollectionModify::__construct

(No version information available, might only be in Git)

CollectionModify::__construct — CollectionModify constructor

Description

private function **mysql_xdevapi\CollectionModify::__construct**()

Modifies (updates) a collection and is instantiated by the **mysql_xdevapi\Collection::modify()** method.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\CollectionModify::__construct() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection
  ->add(
  '{"name":   "Bernie",
    "traits": ["Friend", "Brother", "Human"]}')
  ->execute();

$collection
  ->modify("name in ('Bernie', 'Jane')")
  ->arrayAppend('traits', 'Happy')
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

The above example will output something similar to:

```
```

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b5361000000000000010c
            [name] => Bernie
            [traits] => Array
                (
                    [0] => Friend
                    [1] => Brother
                    [2] => Human
                    [3] => Happy
                )
        )
)
```

CollectionModify::execute

CollectionModify::execute

(No version information available, might only be in Git)

CollectionModify::execute — Execute modify operation

Description

public function **mysql_xdevapi\CollectionModify::execute**(): mysql_xdevapi\Result

The execute method is required to send the CRUD operation request to the MySQL server.

Parameters

This function has no parameters.

Return Values

A Result object that can be used to verify the status of the operation, such as the number of affected rows.

Examples

Example #1 **mysql_xdevapi\CollectionModify::execute()** example

```php
<?php

/* ... */

?>
```

CollectionModify::limit

CollectionModify::limit

(No version information available, might only be in Git)

CollectionModify::limit — Limit number of modified documents

Description

public function **mysql_xdevapi\CollectionModify::limit**(int `$rows`): mysql_xdevapi\CollectionModify

Limit the number of documents modified by this operation. Optionally combine with skip() to define an offset value.

Parameters

`rows`

The maximum number of documents to modify.

Return Values

A CollectionModify object.

Examples

Example #1 mysql_xdevapi\CollectionModify::limit() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$collection->add('{"name": "Fred",  "age": 21, "job": "Construction"}')->execute();
$collection->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')->execute();
$collection->add('{"name": "Betty", "age": 24, "job": "Teacher"}')->execute();

$collection
  ->modify("job = :job")
  ->bind(['job' => 'Teacher'])
  ->set('job', 'Principal')
  ->limit(1)
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b536100000000000000118
```

```
            [age] => 21
            [job] => Construction
            [name] => Fred
        )
    [1] => Array
        (
            [_id] => 00005b6b53610000000000000119
            [age] => 23
            [job] => Principal
            [name] => Wilma
        )
    [2] => Array
        (
            [_id] => 00005b6b536100000000000011a
            [age] => 24
            [job] => Teacher
            [name] => Betty
        )
)
```

CollectionModify::patch

CollectionModify::patch

(No version information available, might only be in Git)

CollectionModify::patch — Patch document

Description

public function **mysql_xdevapi\CollectionModify::patch**(string `$document`): mysql_xdevapi\CollectionModify

Takes a patch object and applies it on one or more documents, and can update multiple document properties.

Parameters

`document`

A document with the properties to apply to the matching documents.

Return Values

A CollectionModify object.

Examples

Example #1 mysql_xdevapi\CollectionModify::patch() example

```php
<?php

$res = $coll->modify('"Programmatore" IN job')->patch('{"Hobby" : "Programmare"}')->execute();

?>
```

CollectionModify::replace

CollectionModify::replace

(No version information available, might only be in Git)

CollectionModify::replace — Replace document field

Description

public function **mysql_xdevapi\CollectionModify::replace**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify

Replace (update) a given field value with a new one.

Parameters

`collection_field`

The document path of the item to set.

`expression_or_literal`

The value to set on the specified attribute.

Return Values

A CollectionModify object.

Examples

Example #1 mysql_xdevapi\CollectionModify::replace() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection
  ->add(
  '{"name":   "Bernie",
    "traits": ["Friend", "Brother", "Human"]}')
  ->execute();

$collection
  ->modify("name = :name")
  ->bind(['name' => 'Bernie'])
  ->replace("name", "Bern")
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b5361000000000000011b
            [name] => Bern
            [traits] => Array
                (
                    [0] => Friend
                    [1] => Brother
                    [2] => Human
                )
        )
)
```

CollectionModify::set

CollectionModify::set

(No version information available, might only be in Git)

CollectionModify::set — Set document attribute

Description

public function **mysql_xdevapi\CollectionModify::set**(string `$collection_field`, string `$expression_or_literal`): mysql_xdevapi\CollectionModify

Sets or updates attributes on documents in a collection.

Parameters

`collection_field`

The document path (name) of the item to set.

`expression_or_literal`

The value to set it to.

Return Values

A CollectionModify object.

Examples

Example #1 mysql_xdevapi\CollectionModify::set() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
```

```php
$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$result = $collection
  ->add(
  '{"name":   "Bernie",
    "traits": ["Friend", "Brother", "Human"]}')
  ->execute();

$collection
  ->modify("name = :name")
  ->bind(['name' => 'Bernie'])
  ->set("name", "Bern")
  ->execute();

$result = $collection
  ->find()
  ->execute();

print_r($result->fetchAll());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [_id] => 00005b6b536100000000000000111
            [name] => Bern
            [traits] => Array
                (
                    [0] => Friend
                    [1] => Brother
                    [2] => Human
                )
        )
)
```

CollectionModify::skip

CollectionModify::skip

(No version information available, might only be in Git)

CollectionModify::skip — Skip elements

Description

public function **mysql_xdevapi\CollectionModify::skip**(int `$position`): mysql_xdevapi\CollectionModify

Skips the first N elements that would otherwise be returned by a find operation. If the number of elements skipped is larger than the size of the result set, then the find operation returns an empty set.

Parameters

`position`

Number of elements to skip.

Return Values

A CollectionModify object to use for further processing.

Examples

Example #1 mysql_xdevapi\CollectionModify::skip() example

```
<?php

$coll->modify('age > :age')->sort('age desc')->unset(['age'])->bind(['age' => 20])->limit(4)->skip(1)->execute();

?>
```

CollectionModify::sort

CollectionModify::sort

(No version information available, might only be in Git)

CollectionModify::sort — Set the sorting criteria

Description

public function **mysql_xdevapi\CollectionModify::sort**(string `$sort_expr`): mysql_xdevapi\CollectionModify

Sorts the result set by the field selected in the sort_expr argument. The allowed orders are ASC (Ascending) or DESC (Descending). This operation is equivalent to the 'ORDER BY' SQL operation and it follows the same set of rules.

Parameters

`sort_expr`

One or more sort expressions can be provided. The evaluation is from left to right and each expression must be separated by a comma.

Return Values

CollectionModify object that can be used for further processing.

Examples

Example #1 mysql_xdevapi\CollectionModify::sort() example

```
<?php
```

```
$res = $coll->modify('true')->sort('name desc', 'age asc')->limit(4)->set('Married', 'NO')->execute();

?>
```

CollectionModify::unset

CollectionModify::unset

(No version information available, might only be in Git)

CollectionModify::unset — Unset the value of document fields

Description

public function **mysql_xdevapi\CollectionModify::unset**(array `$fields`): mysql_xdevapi\CollectionModify

Removes attributes from documents in a collection.

Parameters

`fields`

The attributes to remove from documents in a collection.

Return Values

CollectionModify object that can be used for further processing.

Examples

Example #1 mysql_xdevapi\CollectionModify::unset() example

```
<?php

$res = $coll->modify('job like :job_name')->unset(["age", "name"])->bind(['job_name' => 'Plumber'])->execute();

?>
```

mysql_xdevapi\CollectionRemove

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\CollectionRemove** implements mysql_xdevapi\Executable, mysql_xdevapi\CrudOperationBindable, mysql_xdevapi\CrudOperationLimitable, mysql_xdevapi\CrudOperationSortable {
/* Methods */
public function **bind**(array `$placeholder_values`): mysql_xdevapi\CollectionRemove
public function **execute**(): mysql_xdevapi\Result
public function **limit**(int `$rows`): mysql_xdevapi\CollectionRemove
public function **sort**(string `$sort_expr`): mysql_xdevapi\CollectionRemove
}

Table of Contents

CollectionRemove::bind

CollectionRemove::bind

(No version information available, might only be in Git)

CollectionRemove::bind — Bind value to placeholder

Description

public function **mysql_xdevapi\CollectionRemove::bind**(array `$placeholder_values`): mysql_xdevapi\CollectionRemove

Bind a parameter to the placeholder in the search condition of the remove operation.

The placeholder has the form of :NAME where ':' is a common prefix that must always exists before any NAME where NAME is the name of the placeholder. The bind method accepts a list of placeholders if multiple entities have to be substituted in the search condition of the remove operation.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`placeholder_values`

Placeholder values to substitute in the search condition. Multiple values are allowed and have to be passed as an array of mappings PLACEHOLDER_NAME->PLACEHOLDER_VALUE.

Return Values

A CollectionRemove object that can be used to execute the command, or to add additional operations.

Examples

Example #1 mysql_xdevapi\CollectionRemove::bind() example

```
<?php

$res = $coll->remove('age > :age_from and age < :age_to')->bind(['age_from' => 20, 'age_to' => 50])->limit(7)->execute();

?>
```

CollectionRemove::__construct

CollectionRemove::__construct

(No version information available, might only be in Git)

CollectionRemove::__construct — CollectionRemove constructor

Description

private function **mysql_xdevapi\CollectionRemove::__construct**()

Removes collection documents and is instantiated by the **mysql_xdevapi\Collection::remove()** method.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\Collection::remove() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("people");

$collection->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$collection->add('{"name": "Bob",    "age": 19, "job": "Painter"}')->execute();

// Remove all painters
$collection
  ->remove("job in ('Painter')")
  ->execute();

// Remove the oldest butler
$collection
  ->remove("job in ('Butler')")
  ->sort('age desc')
  ->limit(1)
  ->execute();

// Remove record with lowest age
$collection
  ->remove('true')
  ->sort('age desc')
  ->limit(1)
  ->execute();
?>
```

CollectionRemove::execute

CollectionRemove::execute

(No version information available, might only be in Git)

CollectionRemove::execute — Execute remove operation

Description

public function **mysql_xdevapi\CollectionRemove::execute**(): mysql_xdevapi\Result

The execute function needs to be invoked in order to trigger the client to send the CRUD operation request to the server.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Result object.

Examples

Example #1 mysql_xdevapi\CollectionRemove::execute() example

```
<?php

$res = $coll->remove('true')->sort('age desc')->limit(2)->execute();

?>
```

CollectionRemove::limit

CollectionRemove::limit

(No version information available, might only be in Git)

CollectionRemove::limit — Limit number of documents to remove

Description

public function **mysql_xdevapi\CollectionRemove::limit**(int `$rows`): mysql_xdevapi\CollectionRemove

Sets the maximum number of documents to remove.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`rows`

The maximum number of documents to remove.

Return Values

Returns a CollectionRemove object that can be used to execute the command, or to add additional operations.

Examples

Example #1 mysql_xdevapi\CollectionRemove::limit() example

```
<?php
```

```
$res = $coll->remove('job in (\'Barista\', \'Programmatore\', \'Ballerino\', \'Programmatrice\')')-
>limit(5)->sort(['age desc', 'name asc'])->execute();

?>
```

CollectionRemove::sort

CollectionRemove::sort

(No version information available, might only be in Git)

CollectionRemove::sort — Set the sorting criteria

Description

public function **mysql_xdevapi\CollectionRemove::sort**(string `$sort_expr`): mysql_xdevapi\CollectionRemove

Sort the result set by the field selected in the sort_expr argument. The allowed orders are ASC (Ascending) or DESC (Descending). This operation is equivalent to the 'ORDER BY' SQL operation and it follows the same set of rules.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`sort_expr`

One or more sorting expressions can be provided. The evaluation is from left to right, and each expression is separated by a comma.

Return Values

A CollectionRemove object that can be used to execute the command, or to add additional operations.

Examples

Example #1 mysql_xdevapi\CollectionRemove::sort() example

```
<?php

$res = $coll->remove('true')->sort('age desc')->limit(2)->execute();

?>
```

mysql_xdevapi\ColumnResult

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\ColumnResult** {
/* Methods */

public function **getCharacterSetName**(): string
public function **getCollationName**(): string
public function **getColumnLabel**(): string
public function **getColumnName**(): string
public function **getFractionalDigits**(): int
public function **getLength**(): int
public function **getSchemaName**(): string
public function **getTableLabel**(): string
public function **getTableName**(): string
public function **getType**(): int
public function **isNumberSigned**(): int
public function **isPadded**(): int
}

Table of Contents

ColumnResult::__construct

ColumnResult::__construct

(No version information available, might only be in Git)

ColumnResult::__construct — ColumnResult constructor

Description

private function **mysql_xdevapi\ColumnResult::__construct**()

Retrieves column metadata, such as its character set; this is instantiated by the **mysql_xdevapi\RowResult::getColumns()** method.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\ColumnResult::__construct() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS nonsense")->execute();
$session->sql("CREATE DATABASE nonsense")->execute();
$session->sql("CREATE TABLE nonsense.numbers (hello int, world float unsigned)")->execute();
$session->sql("INSERT INTO  nonsense.numbers values (42, 42)")->execute();

$schema = $session->getSchema("nonsense");
$table  = $schema->getTable("numbers");

$result1 = $table->select('hello','world')->execute();

// Returns an array of ColumnResult objects
$columns = $result1->getColumns();

foreach ($columns as $column) {
    echo "\nColumn label " , $column->getColumnLabel();
```

```
    echo " is type "        , $column->getType();
    echo " and is ", ($column->isNumberSigned() === 0) ? "unsigned." : "signed.";
}

// Alternatively
$result2 = $session->sql("SELECT * FROM nonsense.numbers")->execute();

// Returns an array of FieldMetadata objects
print_r($result2->getColumns());
```

The above example will output something similar to:

```
Column label hello is type 19 and is signed.
Column label world is type 4  and is unsigned.

Array
(
    [0] => mysql_xdevapi\FieldMetadata Object
        (
            [type] => 1
            [type_name] => SINT
            [name] => hello
            [original_name] => hello
            [table] => numbers
            [original_table] => numbers
            [schema] => nonsense
            [catalog] => def
            [collation] => 0
            [fractional_digits] => 0
            [length] => 11
            [flags] => 0
            [content_type] => 0
        )
    [1] => mysql_xdevapi\FieldMetadata Object
        (
            [type] => 6
            [type_name] => FLOAT
            [name] => world
            [original_name] => world
            [table] => numbers
            [original_table] => numbers
            [schema] => nonsense
            [catalog] => def
            [collation] => 0
            [fractional_digits] => 31
            [length] => 12
            [flags] => 1
            [content_type] => 0
        )
)
```

ColumnResult::getCharacterSetName

ColumnResult::getCharacterSetName

(No version information available, might only be in Git)

ColumnResult::getCharacterSetName — Get character set

Description

public function **mysql_xdevapi\ColumnResult::getCharacterSetName**(): string

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\ColumnResult::getCharacterSetName() example

```
<?php

/* ... */

?>
```

ColumnResult::getCollationName

ColumnResult::getCollationName

(No version information available, might only be in Git)

ColumnResult::getCollationName — Get collation name

Description

public function **mysql_xdevapi\ColumnResult::getCollationName**(): string

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\ColumnResult::getCollationName() example

```php
<?php

/* ... */

?>
```

ColumnResult::getColumnLabel

ColumnResult::getColumnLabel

(No version information available, might only be in Git)

ColumnResult::getColumnLabel — Get column label

Description

public function **mysql_xdevapi\ColumnResult::getColumnLabel**(): string

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\ColumnResult::getColumnLabel() example

```php
<?php

/* ... */

?>
```

ColumnResult::getColumnName

ColumnResult::getColumnName

(No version information available, might only be in Git)

ColumnResult::getColumnName — Get column name

Description

public function **mysql_xdevapi\ColumnResult::getColumnName**(): string

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\ColumnResult::getColumnName() example

```
<?php

/* ... */

?>
```

ColumnResult::getFractionalDigits

ColumnResult::getFractionalDigits

(No version information available, might only be in Git)

ColumnResult::getFractionalDigits — Get fractional digit length

Description

public function **mysql_xdevapi\ColumnResult::getFractionalDigits**(): int

Fetch the number of fractional digits for column.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\ColumnResult::getFractionalDigits() example

```
<?php

/* ... */

?>
```

ColumnResult::getLength

ColumnResult::getLength

(No version information available, might only be in Git)

ColumnResult::getLength — Get column field length

Description

public function **mysql_xdevapi\ColumnResult::getLength**(): int

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\ColumnResult::getLength() example

```
<?php

/* ... */

?>
```

ColumnResult::getSchemaName

ColumnResult::getSchemaName

(No version information available, might only be in Git)

ColumnResult::getSchemaName — Get schema name

Description

public function **mysql_xdevapi\ColumnResult::getSchemaName**(): string

Fetch the schema name where the column is stored.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\ColumnResult::getSchemaName() example

```
<?php

/* ... */
```

```
?>
```

ColumnResult::getTableLabel

ColumnResult::getTableLabel

(No version information available, might only be in Git)

ColumnResult::getTableLabel — Get table label

Description

public function **mysql_xdevapi\ColumnResult::getTableLabel**(): string

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\ColumnResult::getTableLabel() example

```
<?php

/* ... */

?>
```

ColumnResult::getTableName

ColumnResult::getTableName

(No version information available, might only be in Git)

ColumnResult::getTableName — Get table name

Description

public function **mysql_xdevapi\ColumnResult::getTableName**(): string

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Name of the table for the column.

Examples

Example #1 mysql_xdevapi\ColumnResult::getTableName() example

```
<?php

/* ... */

?>
```

ColumnResult::getType

ColumnResult::getType

(No version information available, might only be in Git)

ColumnResult::getType — Get column type

Description

public function **mysql_xdevapi\ColumnResult::getType**(): int

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\ColumnResult::getType() example

```
<?php

/* ... */

?>
```

ColumnResult::isNumberSigned

ColumnResult::isNumberSigned

(No version information available, might only be in Git)

ColumnResult::isNumberSigned — Check if signed type

Description

public function **mysql_xdevapi\ColumnResult::isNumberSigned**(): int

Retrieves a table's column information and is instantiated by the **mysql_xdevapi\RowResult::getColumns()** method.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

`true` if a given column has a signed type.

Examples

Example #1 mysql_xdevapi\ColumnResult::isNumberSigned() example

```
<?php

/* ... */

?>
```

ColumnResult::isPadded

ColumnResult::isPadded

(No version information available, might only be in Git)

ColumnResult::isPadded — Check if padded

Description

public function **mysql_xdevapi\ColumnResult::isPadded**(): int

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

`true` if a given column is padded.

Examples

Example #1 mysql_xdevapi\ColumnResult::isPadded() example

```
<?php

/* ... */
```

```
?>
```

mysql_xdevapi\CrudOperationBindable

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\CrudOperationBindable** {
/* Methods */
abstract public function **bind**(array `$placeholder_values`): mysql_xdevapi\CrudOperationBindable
}

Table of Contents

CrudOperationBindable::bind

CrudOperationBindable::bind

(No version information available, might only be in Git)

CrudOperationBindable::bind — Bind value to placeholder

Description

abstract public function **mysql_xdevapi\CrudOperationBindable::bind**(array `$placeholder_values`): mysql_xdevapi\CrudOperationBindable

Binds a value to a specific placeholder.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`placeholder_values`

The name of the placeholders and the values to bind.

Return Values

A CrudOperationBindable object.

Examples

Example #1 mysql_xdevapi\CrudOperationBindable::bind() example

```
<?php

$res = $coll->modify('name like :name')->arrayInsert('job[0]', 'Calciatore')->bind(['name' =>
'ENTITY'])->execute();
$res = $table->delete()->orderby('age desc')->where('age < 20 and age > 12 and name != :name')-
>bind(['name' => 'Tierney'])->limit(2)->execute();
```

```
?>
```

mysql_xdevapi\CrudOperationLimitable

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\CrudOperationLimitable** {
/* Methods */
abstract public function **limit**(int `$rows`): mysql_xdevapi\CrudOperationLimitable
}

Table of Contents

CrudOperationLimitable::limit

CrudOperationLimitable::limit

(No version information available, might only be in Git)

CrudOperationLimitable::limit — Set result limit

Description

abstract public function **mysql_xdevapi\CrudOperationLimitable::limit**(int `$rows`): mysql_xdevapi\CrudOperationLimitable

Sets the maximum number of records or documents to return.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`rows`

The maximum number of records or documents.

Return Values

A CrudOperationLimitable object.

Examples

Example #1 mysql_xdevapi\CrudOperationLimitable::limit() example

```
<?php

$res = $coll->find()->fields(['name as n','age as a','job as j'])->groupBy('j')->limit(11)->execute();
$res = $table->update()->set('age',69)->where('age > 15 and age < 22')->limit(4)->orderby(['age
asc','name desc'])->execute();
```

```
?>
```

mysql_xdevapi\CrudOperationSkippable

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\CrudOperationSkippable** {
/* Methods */
abstract public function **skip**(int `$skip`): mysql_xdevapi\CrudOperationSkippable
}

Table of Contents

CrudOperationSkippable::skip

CrudOperationSkippable::skip

(No version information available, might only be in Git)

CrudOperationSkippable::skip — Number of operations to skip

Description

abstract public function **mysql_xdevapi\CrudOperationSkippable::skip**(int `$skip`): mysql_xdevapi\CrudOperationSkippable

Skip this number of records in the returned operation.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`skip`

Number of elements to skip.

Return Values

A CrudOperationSkippable object.

Examples

Example #1 mysql_xdevapi\CrudOperationSkippable::skip() example

```
<?php

$res = $coll->find('job like \'Programmatore\'')->limit(1)->skip(3)->sort('age asc')->execute();

?>
```

mysql_xdevapi\CrudOperationSortable

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\CrudOperationSortable** {
/* Methods */
abstract public function **sort**(string `$sort_expr`): mysql_xdevapi\CrudOperationSortable
}

Table of Contents

CrudOperationSortable::sort

CrudOperationSortable::sort

(No version information available, might only be in Git)

CrudOperationSortable::sort — Sort results

Description

abstract public function **mysql_xdevapi\CrudOperationSortable::sort**(string `$sort_expr`): mysql_xdevapi\CrudOperationSortable

Sort the result set by the field selected in the sort_expr argument. The allowed orders are ASC (Ascending) or DESC (Descending). This operation is equivalent to the 'ORDER BY' SQL operation and it follows the same set of rules.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`sort_expr`

One or more sorting expressions can be provided. The evaluation is from left to right, and each expression is separated by a comma.

Return Values

A CrudOperationSortable object.

Examples

Example #1 mysql_xdevapi\CrudOperationSortable::sort() example

```
<?php

$res = $coll->find('job like \'Cavia\'')->sort('age desc', '_id desc')->execute();

?>
```

mysql_xdevapi\DatabaseObject

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\DatabaseObject** {
/* Methods */
abstract public function **existsInDatabase**(): bool
abstract public function **getName**(): string
abstract public function **getSession**(): mysql_xdevapi\Session
}

Table of Contents

DatabaseObject::existsInDatabase

DatabaseObject::existsInDatabase

(No version information available, might only be in Git)

DatabaseObject::existsInDatabase — Check if object exists in database

Description

abstract public function **mysql_xdevapi\DatabaseObject::existsInDatabase**(): bool

Verifies if the database object refers to an object that exists in the database.

Parameters

This function has no parameters.

Return Values

Returns `true` if object exists in the database, else `false` if it does not.

Examples

Example #1 mysql_xdevapi\DatabaseObject::existsInDatabase() example

```
<?php

$existInDb = $dbObj->existsInDatabase();

?>
```

DatabaseObject::getName

DatabaseObject::getName

(No version information available, might only be in Git)

DatabaseObject::getName — Get object name

Description

abstract public function **mysql_xdevapi\DatabaseObject::getName**(): string

Fetch name of this database object.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The name of this database object.

Examples

Example #1 mysql_xdevapi\DatabaseObject::getName() example

```
<?php

$dbObjName = $dbObj->getName();

?>
```

DatabaseObject::getSession

DatabaseObject::getSession

(No version information available, might only be in Git)

DatabaseObject::getSession — Get session name

Description

abstract public function **mysql_xdevapi\DatabaseObject::getSession**(): mysql_xdevapi\Session

Fetch session associated to the database object.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The Session object.

Examples

Example #1 mysql_xdevapi\DatabaseObject::getSession() example

```php
<?php

$session = $dbObj->getSession();

?>
```

mysql_xdevapi\DocResult

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\DocResult** implements mysql_xdevapi\BaseResult, Traversable {
/* Methods */
public function **fetchAll**(): array
public function **fetchOne**(): array
public function **getWarnings**(): Array
public function **getWarningsCount**(): int
}

Table of Contents

DocResult::__construct

DocResult::__construct

(No version information available, might only be in Git)

DocResult::__construct — DocResult constructor

Description

private function **mysql_xdevapi\DocResult::__construct**()

Fetch document results and warnings, and is instantiated by CollectionFind.

Parameters

This function has no parameters.

Examples

Example #1 A DocResult example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$create->add('{"name": "Reginald", "age": 42, "job": "Butler"}')->execute();
```

```php
// ...

$collection = $schema->getCollection("people");

// Yields a DocResult object
$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->sort('age desc')
  ->limit(1)
  ->execute();

var_dump($result->fetchAll());
?>
```

The above example will output something similar to:

```
array(1) {
  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000f3"
    ["age"]=>
    int(42)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(8) "Reginald"
  }
}
```

DocResult::fetchAll

DocResult::fetchAll

(No version information available, might only be in Git)

DocResult::fetchAll — Get all rows

Description

public function **mysql_xdevapi\DocResult::fetchAll**(): array

Fetch all results from a result set.

Parameters

This function has no parameters.

Return Values

A numerical array with all results from the query; each result is an associative array. An empty array is returned if no rows are present.

Examples

Example #1 mysql_xdevapi\DocResult::fetchAll() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$create->add('{"name": "Reginald", "age": 42, "job": "Butler"}')->execute();

// ...

$collection = $schema->getCollection("people");

// Yields a DocResult object
$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->sort('age desc')
  ->execute();

var_dump($result->fetchAll());
?>
```

The above example will output something similar to:

```
array(2) {

  [0]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000123"
    ["age"]=>
    int(42)
    ["job"]=>
    string(6) "Butler"
    ["name"]=>
    string(8) "Reginald"
  }

  [1]=>
  array(4) {
    ["_id"]=>
    string(28) "00005b6b536100000000000000122"
    ["age"]=>
    int(18)
    ["job"]=>
    string(6) "Butler"
```

```
    ["name"]=>
    string(6) "Alfred"
  }

}
```

DocResult::fetchOne

DocResult::fetchOne

(No version information available, might only be in Git)

DocResult::fetchOne — Get one row

Description

public function **mysql_xdevapi\DocResult::fetchOne**(): array

Fetch one result from a result set.

Parameters

This function has no parameters.

Return Values

The result, as an associative array or `null` if no results are present.

Examples

Example #1 mysql_xdevapi\DocResult::fetchOne() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$create->add('{"name": "Reginald", "age": 42, "job": "Butler"}')->execute();

// ...

$collection = $schema->getCollection("people");

// Yields a DocResult object
$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->sort('age desc')
  ->execute();
```

```
var_dump($result->fetchOne());
?>
```

The above example will output something similar to:

```
array(4) {
  ["_id"]=>
  string(28) "00005b6b536100000000000000125"
  ["age"]=>
  int(42)
  ["job"]=>
  string(6) "Butler"
  ["name"]=>
  string(8) "Reginald"
}
```

DocResult::getWarnings

DocResult::getWarnings

(No version information available, might only be in Git)

DocResult::getWarnings — Get warnings from last operation

Description

public function **mysql_xdevapi\DocResult::getWarnings**(): Array

Fetches warnings generated by MySQL server's last operation.

Parameters

This function has no parameters.

Return Values

An array of Warning objects from the last operation. Each object defines an error 'message', error 'level', and error 'code'. An empty array is returned if no errors are present.

Examples

Example #1 mysql_xdevapi\DocResult::getWarnings() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
```

```
$create->add('{"name": "Reginald", "age": 42, "job": "Butler"}')->execute();

// ...

$collection = $schema->getCollection("people");

// Yields a DocResult object
$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->sort('age desc')
  ->execute();

if (!$result->getWarningsCount()) {
    echo "There was an error:\n";
    print_r($result->getWarnings());
    exit;
}

var_dump($result->fetchOne());
?>
```

The above example will output something similar to:

```
There was an error:

Array
(
    [0] => mysql_xdevapi\Warning Object
        (
            [message] => Something bad and so on
            [level] => 2
            [code] => 1365
        )
    [1] => mysql_xdevapi\Warning Object
        (
            [message] => Something bad and so on
            [level] => 2
            [code] => 1365
        )
)
```

DocResult::getWarningsCount

DocResult::getWarningsCount

(No version information available, might only be in Git)

DocResult::getWarningsCount — Get warning count from last operation

Description

public function **mysql_xdevapi\DocResult::getWarningsCount**(): int

Returns the number of warnings raised by the last operation. Specifically, these warnings are raised by the MySQL server.

Parameters

This function has no parameters.

Return Values

The number of warnings from the last operation.

Examples

Example #1 mysql_xdevapi\DocResult::getWarningsCount() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$create->add('{"name": "Alfred", "age": 18, "job": "Butler"}')->execute();
$create->add('{"name": "Reginald", "age": 42, "job": "Butler"}')->execute();

// ...

$collection = $schema->getCollection("people");

// Yields a DocResult object
$result = $collection
  ->find('job like :job and age > :age')
  ->bind(['job' => 'Butler', 'age' => 16])
  ->sort('age desc')
  ->execute();

if (!$result->getWarningsCount()) {
    echo "There was an error:\n";
    print_r($result->getWarnings());
    exit;
}

var_dump($result->fetchOne());
?>
```

The above example will output something similar to:

```
array(4) {
  ["_id"]=>
  string(28) "00005b6b536100000000000000135"
  ["age"]=>
  int(42)
  ["job"]=>
  string(6) "Butler"
  ["name"]=>
```

```
  string(8) "Reginald"
}
```

mysql_xdevapi\Driver

(No version information available, might only be in Git)

Introduction

Class synopsis

class **mysql_xdevapi\Driver** {
/* Constants */
const string *version* = 8.0.3;
}

Predefined Constants

`mysql_xdevapi\Driver::version`

mysql_xdevapi\Exception

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\Exception** extends RuntimeException implements Throwable {
}

mysql_xdevapi\Executable

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\Executable** {
/* Methods */
abstract public function **execute**(): mysql_xdevapi\Result
}

Table of Contents

Executable::execute

Executable::execute

(No version information available, might only be in Git)

Executable::execute — Execute statement

Description

abstract public function **mysql_xdevapi\Executable::execute**(): mysql_xdevapi\Result

Execute the statement from either a collection operation or a table query; this functionality allows for method chaining.

Parameters

This function has no parameters.

Return Values

One of the Result objects, such as Result or SqlStatementResult.

Examples

Example #1 execute() examples

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$result_sql = $session->sql("CREATE DATABASE addressbook")->execute();

var_dump($result_sql);


$schema     = $session->getSchema("addressbook");
$collection = $schema->createCollection("humans");

$result_collection = $collection->add(
  '{"name": "Jane",
    "jobs": [{"title":"Scientist","Salary":18000}, {"title":"Mother","Salary":0}],
    "hobbies": ["Walking","Making pies"]}');

$result_collection_executed = $result_collection->execute();

var_dump($result_collection);
var_dump($result_collection_executed);
?>
```

The above example will output something similar to:

```
object(mysql_xdevapi\SqlStatementResult)#3 (0) {
}

object(mysql_xdevapi\CollectionAdd)#5 (0) {
}

object(mysql_xdevapi\Result)#7 (0) {
}
```

mysql_xdevapi\ExecutionStatus

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\ExecutionStatus** {
/* Properties */
public *$affectedItems*;
public *$matchedItems*;
public *$foundItems*;
public *$lastInsertId*;
public *$lastDocumentId*;
/* Constructor */
private function **__construct**()
}

Properties

affectedItems

matchedItems

foundItems

lastInsertId

lastDocumentId

Table of Contents

ExecutionStatus::__construct

ExecutionStatus::__construct

(No version information available, might only be in Git)

ExecutionStatus::__construct — ExecutionStatus constructor

Description

private function **mysql_xdevapi\ExecutionStatus::__construct**()

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\ExecutionStatus::__construct() example

```php
<?php

/* ... */

?>
```

mysql_xdevapi\Expression

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\Expression** {
/* Properties */
public *$name*;
/* Constructor */
public function **__construct**(string `$expression`)
}

Properties

name

Table of Contents

Expression::__construct

Expression::__construct

(No version information available, might only be in Git)

Expression::__construct — Expression constructor

Description

public function **mysql_xdevapi\Expression::__construct**(string `$expression`)

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`expression`

Examples

Example #1 mysql_xdevapi\Expression::__construct() example

```php
<?php

/* ... */

?>
```

mysql_xdevapi\FieldMetadata

(No version information available, might only be in Git)

Introduction

Class synopsis

class **mysql_xdevapi\FieldMetadata** {
/* Properties */
public *$type*;
public *$type_name*;
public *$name*;
public *$original_name*;
public *$table*;
public *$original_table*;
public *$schema*;
public *$catalog*;
public *$collation*;
public *$fractional_digits*;
public *$length*;
public *$flags*;
public *$content_type*;
}

Properties

type

type_name

name

original_name

table

original_table

schema

catalog

collation

fractional_digits

length

flags

content_type

mysql_xdevapi\Result

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\Result** implements mysql_xdevapi\BaseResult, Traversable {
/* Methods */
public function **getAffectedItemsCount**(): int
public function **getAutoIncrementValue**(): int
public function **getGeneratedIds**(): array
public function **getWarnings**(): array
public function **getWarningsCount**(): int
}

Table of Contents

Result::__construct

Result::__construct

(No version information available, might only be in Git)

Result::__construct — Result constructor

Description

private function **mysql_xdevapi\Result::__construct**()

An object that retrieves generated IDs, AUTO_INCREMENT values, and warnings, for a Result set.

Parameters

This function has no parameters.

Examples

Example #1 **mysql_xdevapi\Result::__construct()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("
  CREATE TABLE addressbook.names
    (id INT NOT NULL AUTO_INCREMENT, name VARCHAR(30), age INT, PRIMARY KEY (id))
  ")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->insert("name", "age")->values(["Suzanne", 31],["Julie", 43])->execute();
$result = $table->insert("name", "age")->values(["Suki", 34])->execute();

$ai = $result->getAutoIncrementValue();
var_dump($ai);
?>
```

The above example will output:

```
int(3)
```

Result::getAffectedItemsCount

Result::getAffectedItemsCount

(No version information available, might only be in Git)

Result::getAffectedItemsCount — Get affected row count

Description

public function **mysql_xdevapi\Result::getAffectedItemsCount**(): int

Get the number of affected rows by the previous operation.

Parameters

This function has no parameters.

Return Values

The number (as an integer) of affected rows.

Examples

Example #1 mysql_xdevapi\Result::getAffectedItemsCount() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

$result = $collection->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')->execute();

var_dump( $res->getAffectedItemsCount() );
?>
```

The above example will output:

```
int(1)
```

Result::getAutoIncrementValue

Result::getAutoIncrementValue

(No version information available, might only be in Git)

Result::getAutoIncrementValue — Get autoincremented value

Description

public function **mysql_xdevapi\Result::getAutoIncrementValue**(): int

Get the last AUTO_INCREMENT value (last insert id).

Parameters

This function has no parameters.

Return Values

The last AUTO_INCREMENT value.

Examples

Example #1 mysql_xdevapi\Result::getAutoIncrementValue() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("
  CREATE TABLE addressbook.names
    (id INT NOT NULL AUTO_INCREMENT, name VARCHAR(30), age INT, PRIMARY KEY (id))
  ")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->insert("name", "age")->values(["Suzanne", 31],["Julie", 43])->execute();
$result = $table->insert("name", "age")->values(["Suki", 34])->execute();

$ai = $result->getAutoIncrementValue();
var_dump($ai);
?>
```

The above example will output:

```
int(3)
```

Result::getGeneratedIds

Result::getGeneratedIds

(No version information available, might only be in Git)

Result::getGeneratedIds — Get generated ids

Description

public function **mysql_xdevapi\Result::getGeneratedIds**(): array

Fetch the generated _id values from the last operation. The unique _id field is generated by the MySQL server.

Parameters

This function has no parameters.

Return Values

An array of generated _id's from the last operation, or an empty array if there are none.

Examples

Example #1 mysql_xdevapi\Result::getGeneratedIds() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema = $session->getSchema("addressbook");
$create = $schema->createCollection("people");

$collection = $schema->getCollection("people");

$result = $collection->add(
  '{"name": "Bernie",
    "jobs": [{"title":"Cat Herder","Salary":42000}, {"title":"Father","Salary":0}],
    "hobbies": ["Sports","Making cupcakes"]}',
  '{"name": "Jane",
    "jobs": [{"title":"Scientist","Salary":18000}, {"title":"Mother","Salary":0}],
    "hobbies": ["Walking","Making pies"]}')->execute();

$ids = $result->getGeneratedIds();
var_dump($ids);
?>
```

The above example will output something similar to:

```
array(2) {
  [0]=>
  string(28) "00005b6b536100000000000000064"
  [1]=>
  string(28) "00005b6b536100000000000000065"
}
```

Result::getWarnings

Result::getWarnings

(No version information available, might only be in Git)

Result::getWarnings — Get warnings from last operation

Description

public function **mysql_xdevapi\Result::getWarnings**(): array

Retrieve warnings from the last Result operation.

Parameters

This function has no parameters.

Return Values

An array of Warning objects from the last operation. Each object defines an error 'message', error 'level', and error 'code'. An empty array is returned if no errors are present.

Examples

Example #1 mysql_xdevapi\RowResult::getWarnings() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();

$schema = $session->getSchema("foo");
$table  = $schema->getTable("test_table");

$table->insert(['x'])->values([1])->values([2])->execute();

$res = $table->select(['x/0 as bad_x'])->execute();
$warnings = $res->getWarnings();

print_r($warnings);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => mysql_xdevapi\Warning Object
        (
            [message] => Division by 0
            [level] => 2
```

```
            [code] => 1365
        )
    [1] => mysql_xdevapi\Warning Object
        (
            [message] => Division by 0
            [level] => 2
            [code] => 1365
        )
)
```

Result::getWarningsCount

Result::getWarningsCount

(No version information available, might only be in Git)

Result::getWarningsCount — Get warning count from last operation

Description

public function **mysql_xdevapi\Result::getWarningsCount**(): int

Retrieve the number of warnings from the last Result operation.

Parameters

This function has no parameters.

Return Values

The number of warnings generated by the last operation.

Examples

Example #1 mysql_xdevapi\RowResult::getWarningsCount() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS foo")->execute();
$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();

$schema = $session->getSchema("foo");
$table  = $schema->getTable("test_table");

$table->insert(['x'])->values([1])->values([2])->execute();

$res = $table->select(['x/0 as bad_x'])->execute();

echo $res->getWarningsCount();
?>
```

The above example will output something similar to:

```
2
```

mysql_xdevapi\RowResult

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\RowResult** implements mysql_xdevapi\BaseResult, Traversable {
/* Methods */
public function **fetchAll**(): array
public function **fetchOne**(): array
public function **getColumnsCount**(): int
public function **getColumnNames**(): array
public function **getColumns**(): array
public function **getWarnings**(): array
public function **getWarningsCount**(): int
}

Table of Contents

RowResult::__construct

RowResult::__construct

(No version information available, might only be in Git)

RowResult::__construct — RowResult constructor

Description

private function **mysql_xdevapi\RowResult::__construct**()

Represents the result set obtained from querying the database.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\RowResult::__construct() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");
```

```php
$row = $table->select('name', 'age')->where('age > 18')->execute()->fetchAll();

print_r($row);
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 33
        )
)
```

RowResult::fetchAll

RowResult::fetchAll

(No version information available, might only be in Git)

RowResult::fetchAll — Get all rows from result

Description

public function **mysql_xdevapi\RowResult::fetchAll**(): array

Fetch all the rows from the result set.

Parameters

This function has no parameters.

Return Values

A numerical array with all results from the query; each result is an associative array. An empty array is returned if no rows are present.

Examples

Example #1 mysql_xdevapi\RowResult::fetchAll() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
```

```php
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$row = $table->select('name', 'age')->execute()->fetchAll();

print_r($row);
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 33
        )
)
```

RowResult::fetchOne

RowResult::fetchOne

(No version information available, might only be in Git)

RowResult::fetchOne — Get row from result

Description

public function **mysql_xdevapi\RowResult::fetchOne**(): array

Fetch one result from the result set.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The result, as an associative array or `null` if no results are present.

Examples

Example #1 mysql_xdevapi\RowResult::fetchOne() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$row = $table->select('name', 'age')->where('age < 40')->execute()->fetchOne();

print_r($row);
```

The above example will output something similar to:

```
Array
(
    [name] => Sam
    [age] => 33
)
```

RowResult::getColumnsCount

RowResult::getColumnsCount

(No version information available, might only be in Git)

RowResult::getColumnsCount — Get column count

Description

public function **mysql_xdevapi\RowResult::getColumnsCount**(): int

Retrieve the column count for columns present in the result set.

Parameters

This function has no parameters.

Return Values

The number of columns; 0 if there are none.

Changelog

Version	Description
8.0.14	Method renamed from getColumnCount() to getColumnsCount().

Examples

Example #1 **mysql_xdevapi\RowResult::getColumnsCount()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE addressbook")->execute();
$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$sql = $session->sql("SELECT * from addressbook.names")->execute();

echo $sql->getColumnsCount();
```

The above example will output something similar to:

```
2
```

RowResult::getColumnNames

RowResult::getColumnNames

(No version information available, might only be in Git)

RowResult::getColumnNames — Get all column names

Description

public function **mysql_xdevapi\RowResult::getColumnNames**(): array

Retrieve column names for columns present in the result set.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

A numerical array of table columns names, or an empty array if the result set is empty.

Examples

Example #1 mysql_xdevapi\RowResult::getColumnNames() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE addressbook")->execute();
$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$sql = $session->sql("SELECT * from addressbook.names")->execute();

$colnames = $sql->getColumnNames();

print_r($colnames);
```

The above example will output something similar to:

```
Array
(
    [0] => name
    [1] => age
)
```

RowResult::getColumns

RowResult::getColumns

(No version information available, might only be in Git)

RowResult::getColumns — Get column metadata

Description

public function **mysql_xdevapi\RowResult::getColumns**(): array

Retrieve column metadata for columns present in the result set.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

An array of FieldMetadata objects representing the columns in the result, or an empty array if the result set is empty.

Examples

Example #1 **mysql_xdevapi\RowResult::getColumns()** example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE addressbook")->execute();
$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$sql = $session->sql("SELECT * from addressbook.names")->execute();

$cols = $sql->getColumns();

print_r($cols);
```

The above example will output something similar to:

```
Array
(
    [0] => mysql_xdevapi\FieldMetadata Object
        (
            [type] => 7
            [type_name] => BYTES
            [name] => name
            [original_name] => name
            [table] => names
            [original_table] => names
            [schema] => addressbook
            [catalog] => def
            [collation] => 255
            [fractional_digits] => 0
            [length] => 65535
            [flags] => 0
            [content_type] => 0
        )
    [1] => mysql_xdevapi\FieldMetadata Object
        (
            [type] => 1
            [type_name] => SINT
            [name] => age
            [original_name] => age
            [table] => names
            [original_table] => names
            [schema] => addressbook
            [catalog] => def
            [collation] => 0
            [fractional_digits] => 0
            [length] => 11
            [flags] => 0
            [content_type] => 0
        )
)
```

RowResult::getWarnings

RowResult::getWarnings

(No version information available, might only be in Git)

RowResult::getWarnings — Get warnings from last operation

Description

public function **mysql_xdevapi\RowResult::getWarnings**(): array

Retrieve warnings from the last RowResult operation.

Parameters

This function has no parameters.

Return Values

An array of Warning objects from the last operation. Each object defines an error 'message', error 'level', and error 'code'. An empty array is returned if no errors are present.

Examples

Example #1 mysql_xdevapi\RowResult::getWarnings() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();

$schema = $session->getSchema("foo");
$table  = $schema->getTable("test_table");

$table->insert(['x'])->values([1])->values([2])->execute();

$res = $table->select(['x/0 as bad_x'])->execute();
$warnings = $res->getWarnings();

print_r($warnings);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => mysql_xdevapi\Warning Object
        (
            [message] => Division by 0
            [level] => 2
```

```
            [code] => 1365
        )
    [1] => mysql_xdevapi\Warning Object
        (
            [message] => Division by 0
            [level] => 2
            [code] => 1365
        )
)
```

RowResult::getWarningsCount

RowResult::getWarningsCount

(No version information available, might only be in Git)

RowResult::getWarningsCount — Get warning count from last operation

Description

public function **mysql_xdevapi\RowResult::getWarningsCount**(): int

Retrieve the number of warnings from the last RowResult operation.

Parameters

This function has no parameters.

Return Values

The number of warnings generated by the last operation.

Examples

Example #1 mysql_xdevapi\RowResult::getWarningsCount() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS foo")->execute();
$session->sql("CREATE DATABASE foo")->execute();
$session->sql("CREATE TABLE foo.test_table(x int)")->execute();

$schema = $session->getSchema("foo");
$table  = $schema->getTable("test_table");

$table->insert(['x'])->values([1])->values([2])->execute();

$res = $table->select(['x/0 as bad_x'])->execute();

echo $res->getWarningsCount();
?>
```

The above example will output something similar to:

```
2
```

mysql_xdevapi\Schema

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\Schema** implements mysql_xdevapi\DatabaseObject {
/* Properties */
public *$name*;
/* Methods */
public function **createCollection**(string `$name`, string `$validate = ?`): mysql_xdevapi\Collection
public function **dropCollection**(string `$collection_name`): bool
public function **existsInDatabase**(): bool
public function **getCollection**(string `$name`): mysql_xdevapi\Collection
public function **getCollectionAsTable**(string `$name`): mysql_xdevapi\Table
public function **getCollections**(): array
public function **getName**(): string
public function **getSession**(): mysql_xdevapi\Session
public function **getTable**(string `$name`): mysql_xdevapi\Table
public function **getTables**(): array
}

Properties

name

Table of Contents

Schema::__construct

Schema::__construct

(No version information available, might only be in Git)

Schema::__construct — Schema constructor

Description

private function **mysql_xdevapi\Schema::__construct**()

The Schema object provides full access to the schema (database).

Parameters

This function has no parameters.

Examples

Example #1 **mysql_xdevapi\Schema::__construct()** example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS food")->execute();
$session->sql("CREATE DATABASE food")->execute();
$session->sql("CREATE TABLE food.fruit(name text, rating text)")->execute();

$schema = $session->getSchema("food");
$schema->createCollection("trees");

print_r($schema->gettables());
print_r($schema->getcollections());
```

The above example will output something similar to:

```
Array
(
    [fruit] => mysql_xdevapi\Table Object
        (
            [name] => fruit
        )
)
Array
(
    [trees] => mysql_xdevapi\Collection Object
        (
            [name] => trees
        )
)
```

Schema::createCollection

Schema::createCollection

(No version information available, might only be in Git)

Schema::createCollection — Add collection to schema

Description

public function **mysql_xdevapi\Schema::createCollection**(string `$name`, string `$validate = ?`): mysql_xdevapi\Collection

Create a collection within the schema.

Parameters

`name`

Collection name.

`validate`

Validation definition, as a JSON object.

Return Values

The Collection object.

Changelog

Version	Description
8.0.20	Added the optional validate parameter.

Examples

Example #1 mysql_xdevapi\Schema::createCollection() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS food")->execute();
$session->sql("CREATE DATABASE food")->execute();
$session->sql("CREATE TABLE food.fruit(name text, rating text)")->execute();

$schema = $session->getSchema("food");
$schema->createCollection("trees");

print_r($schema->gettables());
print_r($schema->getcollections());
```

The above example will output something similar to:

```
Array
(
    [fruit] => mysql_xdevapi\Table Object
        (
            [name] => fruit
        )
)
Array
(
    [trees] => mysql_xdevapi\Collection Object
        (
            [name] => trees
        )
)
```

Example #2 mysql_xdevapi\Schema::createCollection() example

```php
<?php
 $collection = $schema->createCollection("mycollection", '{
    "validation": {
        "level": "strict",
        "schema": {
            "id": "http://json-schema.org/geo",
            "description": "A geographical coordinate",
            "type": "object",
            "properties": {
                "latitude": {
                    "type": "number"
                },
                "longitude": {
                    "type": "number"
                }
            },
            "required": ["latitude", "longitude"]
        }
    }
}');
// Succeeds
$collection->add('{"latitude": 10, "longitude": 20}')->execute();

// Fails, invalid types (not numbers)
$collection->add('{"latitude": "lat", "longitude": "long"}')->execute();
```

Schema::dropCollection

Schema::dropCollection

(No version information available, might only be in Git)

Schema::dropCollection — Drop collection from schema

Description

public function **mysql_xdevapi\Schema::dropCollection**(string `$collection_name`): bool

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`collection_name`

Return Values

Examples

Example #1 mysql_xdevapi\Schema::dropCollection() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS food")->execute();
$session->sql("CREATE DATABASE food")->execute();
```

```php
$session->sql("CREATE TABLE food.fruit(name text, rating text)")->execute();

$schema = $session->getSchema("food");

$schema->createCollection("trees");
$schema->dropCollection("trees");
$schema->createCollection("buildings");

print_r($schema->gettables());
print_r($schema->getcollections());
```

The above example will output something similar to:

```
Array
(
    [fruit] => mysql_xdevapi\Table Object
        (
            [name] => fruit
        )
)
Array
(
    [buildings] => mysql_xdevapi\Collection Object
        (
            [name] => buildings
        )
)
```

Schema::existsInDatabase

Schema::existsInDatabase

(No version information available, might only be in Git)

Schema::existsInDatabase — Check if exists in database

Description

public function **mysql_xdevapi\Schema::existsInDatabase**(): bool

Checks if the current object (schema, table, collection, or view) exists in the schema object.

Parameters

This function has no parameters.

Return Values

`true` if the schema, table, collection, or view still exists in the schema, else `false`.

Examples

Example #1 mysql_xdevapi\Schema::existsInDatabase() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS food")->execute();
$session->sql("CREATE DATABASE food")->execute();
$session->sql("CREATE TABLE food.fruit(name text, rating text)")->execute();

$schema = $session->getSchema("food");
$schema->createCollection("trees");

// ...

$trees = $schema->getCollection("trees");

// ...

// Is this collection still in the database (schema)?
if ($trees->existsInDatabase()) {
    echo "Yes, the 'trees' collection is still present.";
}
```

The above example will output something similar to:

```
Yes, the 'trees' collection is still present.
```

Schema::getCollection

Schema::getCollection

(No version information available, might only be in Git)

Schema::getCollection — Get collection from schema

Description

public function **mysql_xdevapi\Schema::getCollection**(string `$name`): mysql_xdevapi\Collection

Get a collection from the schema.

Parameters

`name`

Collection name to retrieve.

Return Values

The Collection object for the selected collection.

Examples

Example #1 mysql_xdevapi\Schema::getCollection() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS food")->execute();
$session->sql("CREATE DATABASE food")->execute();

$schema = $session->getSchema("food");
$schema->createCollection("trees");

// ...

$trees = $schema->getCollection("trees");

var_dump($trees);
```

The above example will output something similar to:

```
object(mysql_xdevapi\Collection)#3 (1) {
  ["name"]=>
  string(5) "trees"
}
```

Schema::getCollectionAsTable

Schema::getCollectionAsTable

(No version information available, might only be in Git)

Schema::getCollectionAsTable — Get collection as a Table object

Description

public function **mysql_xdevapi\Schema::getCollectionAsTable**(string `$name`): mysql_xdevapi\Table

Get a collection, but as a Table object instead of a Collection object.

Parameters

`name`

Name of the collection to instantiate a Table object from.

Return Values

A mysql_xdevapi\Table object for the collection.

Examples

Example #1 mysql_xdevapi\Schema::getCollectionAsTable() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
```

```php
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema  = $session->getSchema("addressbook");
$collect = $schema->createCollection("people");
$collect->add('{"name": "Fred",  "age": 21, "job": "Construction"}')->execute();
$collect->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')->execute();

$table      = $schema->getCollectionAsTable("people");
$collection = $schema->getCollection("people");

var_dump($table);
var_dump($collection);
```

The above example will output something similar to:

```
object(mysql_xdevapi\Table)#4 (1) {
  ["name"]=>
  string(6) "people"
}

object(mysql_xdevapi\Collection)#5 (1) {
  ["name"]=>
  string(6) "people"
}
```

Schema::getCollections

Schema::getCollections

(No version information available, might only be in Git)

Schema::getCollections — Get all schema collections

Description

public function **mysql_xdevapi\Schema::getCollections**(): array

Fetch a list of collections for this schema.

Parameters

This function has no parameters.

Return Values

Array of all collections in this schema, where each array element value is a Collection object with the collection name as the key.

Examples

Example #1 mysql_xdevapi\Schema::getCollections() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema  = $session->getSchema("addressbook");
$collect = $schema->createCollection("people");
$collect->add('{"name": "Fred",  "age": 21, "job": "Construction"}')->execute();
$collect->add('{"name": "Wilma", "age": 23, "job": "Teacher"}')->execute();

$collections = $schema->getCollections();
var_dump($collections);
?>
```

The above example will output something similar to:

```
array(1) {
  ["people"]=>
  object(mysql_xdevapi\Collection)#4 (1) {
    ["name"]=>
    string(6) "people"
  }
}
```

Schema::getName

Schema::getName

(No version information available, might only be in Git)

Schema::getName — Get schema name

Description

public function **mysql_xdevapi\Schema::getName**(): string

Get the name of the schema.

Parameters

This function has no parameters.

Return Values

The name of the schema connected to the schema object, as a string.

Examples

Example #1 mysql_xdevapi\Schema::getName() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
```

```php
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema  = $session->getSchema("addressbook");

// ...

var_dump($schema->getName());
?>
```

The above example will output something similar to:

```
string(11) "addressbook"
```

Schema::getSession

Schema::getSession

(No version information available, might only be in Git)

Schema::getSession — Get schema session

Description

public function **mysql_xdevapi\Schema::getSession**(): mysql_xdevapi\Session

Get a new Session object from the Schema object.

Parameters

This function has no parameters.

Return Values

A Session object.

Examples

Example #1 mysql_xdevapi\Schema::getSession() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();

$schema  = $session->getSchema("addressbook");

// ...

$newsession = $schema->getSession();

var_dump($session);
```

```
var_dump($newsession);
?>
```

The above example will output something similar to:

```
object(mysql_xdevapi\Session)#1 (0) {
}

object(mysql_xdevapi\Session)#3 (0) {
}
```

Schema::getTable

Schema::getTable

(No version information available, might only be in Git)

Schema::getTable — Get schema table

Description

public function **mysql_xdevapi\Schema::getTable**(string `$name`): mysql_xdevapi\Table

Fetch a Table object for the provided table in the schema.

Parameters

`name`

Name of the table.

Return Values

A Table object.

Examples

Example #1 mysql_xdevapi\Schema::getTable() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$row = $table->select('name', 'age')->execute()->fetchAll();
```

```
print_r($row);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 33
        )
)
```

Schema::getTables

Schema::getTables

(No version information available, might only be in Git)

Schema::getTables — Get schema tables

Description

public function **mysql_xdevapi\Schema::getTables**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Array of all tables in this schema, where each array element value is a Table object with the table name as the key.

Examples

Example #1 mysql_xdevapi\Schema::getTables() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
```

```php
$session->sql("CREATE DATABASE addressbook")->execute();

$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$session->sql("CREATE TABLE addressbook.cities(name text, population int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('Portland', 639863), ('Seattle', 704352)")->execute();

$schema = $session->getSchema("addressbook");
$tables = $schema->getTables();

var_dump($tables);
?>
```

The above example will output something similar to:

```
array(2) {
  ["cities"]=>
  object(mysql_xdevapi\Table)#3 (1) {
    ["name"]=>
    string(6) "cities"
  }

  ["names"]=>
  object(mysql_xdevapi\Table)#4 (1) {
    ["name"]=>
    string(5) "names"
  }
}
```

mysql_xdevapi\SchemaObject

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\SchemaObject** implements mysql_xdevapi\DatabaseObject {
/* Methods */
abstract function **getSchema**(): mysql_xdevapi\Schema
}

Table of Contents

SchemaObject::getSchema

SchemaObject::getSchema

(No version information available, might only be in Git)

SchemaObject::getSchema — Get schema object

Description

abstract function **mysql_xdevapi\SchemaObject::getSchema**(): mysql_xdevapi\Schema

Used by other objects to retrieve a schema object.

Parameters

This function has no parameters.

Return Values

The current Schema object.

Examples

Example #1 **mysql_xdevapi\Session::getSchema()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$schema  = $session->getSchema("addressbook");

print_r($schema);
```

The above example will output something similar to:

```
mysql_xdevapi\Schema Object
(
    [name] => addressbook
)
```

mysql_xdevapi\Session

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\Session** {
/* Methods */
public function **close**(): bool
public function **commit**(): Object
public function **createSchema**(string `$schema_name`): mysql_xdevapi\Schema
public function **dropSchema**(string `$schema_name`): bool
public function **generateUUID**(): string
public function **getDefaultSchema**(): ?mysql_xdevapi\Schema
public function **getSchema**(string `$schema_name`): mysql_xdevapi\Schema
public function **getSchemas**(): array

public function **getServerVersion**(): int
public function **listClients**(): array
public function **quoteName**(string $name): string
public function **releaseSavepoint**(string $name): void
public function **rollback**(): void
public function **rollbackTo**(string $name): void
public function **setSavepoint**(string $name = ?): string
public function **sql**(string $query): mysql_xdevapi\SqlStatement
public function **startTransaction**(): void
}

Table of Contents

Session::close

Session::close

(No version information available, might only be in Git)

Session::close — Close session

Description

public function **mysql_xdevapi\Session::close**(): bool

Close the session with the server.

Parameters

This function has no parameters.

Return Values

`true` if the session closed.

Examples

Example #1 mysql_xdevapi\Session::close() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$session->close();
```

Session::commit

Session::commit

(PHP 4 >= 4.4.0, PHP 5, PHP 7, PHP 8)

Session::commit — Commit transaction

Description

public function **mysql_xdevapi\Session::commit**(): Object

Commit the transaction.

Parameters

This function has no parameters.

Return Values

An SqlStatementResult object.

Examples

Example #1 mysql_xdevapi\Session::commit() example

```
<?php
$session    = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$collection = $session->getSchema("addressbook")->getCollection("friends");

$session->startTransaction();

$collection->add('{"John":42, "Sam":33}')->execute();
$savepoint = $session->setSavepoint();

$session->commit();
$session->close();
```

Session::__construct

Session::__construct

(No version information available, might only be in Git)

Session::__construct — Description constructor

Description

private function **mysql_xdevapi\Session::__construct**()

A Session object, as initiated by getSession().

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\Session::__construct() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->close();
?>
```

Session::createSchema

Session::createSchema

(No version information available, might only be in Git)

Session::createSchema — Create new schema

Description

public function **mysql_xdevapi\Session::createSchema**(string `$schema_name`): mysql_xdevapi\Schema

Creates a new schema.

Parameters

`schema_name`

Name of the schema to create.

Return Values

A Schema object on success, and emits an exception on failure.

Examples

Example #1 mysql_xdevapi\Session::createSchema() example

```
<?php
$uri  = 'mysqlx://happyuser:password@127.0.0.1:33060/';
$sess = mysql_xdevapi\getSession($uri);

try {

    if ($schema = $sess->createSchema('fruit')) {
        echo "Info: I created a schema named 'fruit'\n";
    }

} catch (Exception $e) {

   echo $e->getMessage();

}
?>
```

The above example will output something similar to:

```
Info: I created a schema named 'fruit'
```

Session::dropSchema

Session::dropSchema

(No version information available, might only be in Git)

Session::dropSchema — Drop a schema

Description

public function **mysql_xdevapi\Session::dropSchema**(string `$schema_name`): bool

Drop a schema (database).

Parameters

`schema_name`

Name of the schema to drop.

Return Values

`true` if the schema is dropped, or `false` if it does not exist or can't be dropped.

An `E_WARNING` level error is generated if the schema does not exist.

Examples

Example #1 mysql_xdevapi\Session::dropSchema() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->dropSchema("addressbook");

$session->close();
?>
```

Session::generateUUID

Session::generateUUID

(No version information available, might only be in Git)

Session::generateUUID — Get new UUID

Description

public function **mysql_xdevapi\Session::generateUUID**(): string

Generate a Universal Unique IDentifier (UUID) generated according to » RFC 4122.

Parameters

This function has no parameters.

Return Values

The UUID; a string with a length of 32.

Examples

Example #1 mysql_xdevapi\Session::generateUuid() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$uuid = $session->generateUuid();
```

```php
var_dump($uuid);
```

The above example will output something similar to:

```
string(32) "484B18AC7980F8D4FE84613CDA5EE84B"
```

Session::getDefaultSchema

Session::getDefaultSchema

(No version information available, might only be in Git)

Session::getDefaultSchema — Get default schema name

Description

public function **mysql_xdevapi\Session::getDefaultSchema**(): ?mysql_xdevapi\Schema

Retrieve the default schema that's typically set in the connection URI.

Parameters

This function has no parameters.

Return Values

The default schema defined by the connection, or `null` if one was not set.

Examples

Example #1 mysql_xdevapi\Session::getSchema() example

```php
<?php
$uri = "mysqlx://testuser:testpasswd@localhost:33160/testx?ssl-mode=disabled";
$session = mysql_xdevapi\getSession($uri);

$schema = $session->getDefaultSchema();
echo $schema->getName();
?>
```

The above example will output:

```
testx
```

Session::getSchema

Session::getSchema

(No version information available, might only be in Git)

Session::getSchema — Get a new schema object

Description

public function **mysql_xdevapi\Session::getSchema**(string `$schema_name`): mysql_xdevapi\Schema

A new Schema object for the provided schema name.

Parameters

`schema_name`

Name of the schema (database) to fetch a Schema object for.

Return Values

A Schema object.

Examples

Example #1 mysql_xdevapi\Session::getSchema() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$schema  = $session->getSchema("addressbook");

print_r($schema);
```

The above example will output something similar to:

```
mysql_xdevapi\Schema Object
(
    [name] => addressbook
)
```

Session::getSchemas

Session::getSchemas

(No version information available, might only be in Git)

Session::getSchemas — Get the schemas

Description

public function **mysql_xdevapi\Session::getSchemas**(): array

Get schema objects for all schemas available to the session.

Parameters

This function has no parameters.

Return Values

An array containing objects that represent all of the schemas available to the session.

Examples

Example #1 **mysql_xdevapi\Session::getSchemas()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$schemas  = $session->getSchemas();

print_r($schemas);
```

The above example will output something similar to:

```
Array
(
    [0] => mysql_xdevapi\Schema Object
        (
            [name] => addressbook
        )
    [1] => mysql_xdevapi\Schema Object
        (
            [name] => information_schema
        )
    ...
```

Session::getServerVersion

Session::getServerVersion

(No version information available, might only be in Git)

Session::getServerVersion — Get server version

Description

public function **mysql_xdevapi\Session::getServerVersion**(): int

Retrieve the MySQL server version for the session.

Parameters

This function has no parameters.

Return Values

The MySQL server version for the session, as an integer such as "80012".

Examples

Example #1 **mysql_xdevapi\Session::getServerVersion()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$version = $session->getServerVersion();

var_dump($version);
```

The above example will output something similar to:

```
int(80012)
```

Session::listClients

Session::listClients

(No version information available, might only be in Git)

Session::listClients — Get client list

Description

public function **mysql_xdevapi\Session::listClients**(): array

Get a list of client connections to the session's MySQL server.

Parameters

This function has no parameters.

Return Values

An array containing the currently logged clients. The array elements are "client_id", "user", "host", and "sql_session".

Examples

Example #1 **mysql_xdevapi\Session::listClients()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$ids = $session->listClients();

var_dump($ids);
?>
```

The above example will output something similar to:

```
array(1) {
  [0]=>
  array(4) {
    ["client_id"]=>
    int(61)
    ["user"]=>
    string(4) "root"
    ["host"]=>
    string(9) "localhost"
    ["sql_session"]=>
    int(72)
  }
}
```

Session::quoteName

Session::quoteName

(No version information available, might only be in Git)

Session::quoteName — Add quotes

Description

public function **mysql_xdevapi\Session::quoteName**(string `$name`): string

A quoting function to escape SQL names and identifiers. It escapes the identifier given in accordance to the settings of the current connection. This escape function should not be used to escape values.

Parameters

`name`

The string to quote.

Return Values

The quoted string.

Examples

Example #1 mysql_xdevapi\Session::quoteName() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$first = "MySQL's test";
var_dump($first);
var_dump($session->quoteName($first));
```

```
$second = 'Another `test` "like" `this`';
var_dump($second);
var_dump($session->quoteName($second));
?>
```

The above example will output something similar to:

```
string(12) "MySQL's test"
string(14) "`MySQL's test`"

string(28) "Another `test` "like" `this`"
string(34) "`Another ``test`` "like" ``this```"
```

Session::releaseSavepoint

Session::releaseSavepoint

(No version information available, might only be in Git)

Session::releaseSavepoint — Release set savepoint

Description

public function **mysql_xdevapi\Session::releaseSavepoint**(string `$name`): void

Release a previously set savepoint.

Parameters

`name`

Name of the savepoint to release.

Return Values

An SqlStatementResult object.

Examples

Example #1 mysql_xdevapi\Session::releaseSavepoint() example

```
<?php
$session    = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$collection = $session->getSchema("addressbook")->getCollection("friends");

$session->startTransaction();
$collection->add( '{"test1":1, "test2":2}' )->execute();

$savepoint = $session->setSavepoint();

$collection->add( '{"test3":3, "test4":4}' )->execute();
```

```
$session->releaseSavepoint($savepoint);
$session->rollback();
?>
```

Session::rollback

Session::rollback

(No version information available, might only be in Git)

Session::rollback — Rollback transaction

Description

public function **mysql_xdevapi\Session::rollback**(): void

Rollback the transaction.

Parameters

This function has no parameters.

Return Values

An SqlStatementResult object.

Examples

Example #1 mysql_xdevapi\Session::rollback() example

```
<?php
$session    = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$collection = $session->getSchema("addressbook")->getCollection("names");

$session->startTransaction();
$collection->add( '{"test1":1, "test2":2}' )->execute();

$savepoint = $session->setSavepoint();

$collection->add( '{"test3":3, "test4":4}' )->execute();

$session->releaseSavepoint($savepoint);
$session->rollback();
?>
```

Session::rollbackTo

Session::rollbackTo

(No version information available, might only be in Git)

Session::rollbackTo — Rollback transaction to savepoint

Description

public function **mysql_xdevapi\Session::rollbackTo**(string `$name`): void

Rollback the transaction back to the savepoint.

Parameters

`name`

Name of the savepoint to rollback to; case-insensitive.

Return Values

An SqlStatementResult object.

Examples

Example #1 **mysql_xdevapi\Session::rollbackTo()** example

```
<?php
$session    = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$collection = $session->getSchema("addressbook")->getCollection("names");

$session->startTransaction();
$collection->add( '{"test1":1, "test2":2}' )->execute();

$savepoint1 = $session->setSavepoint();

$collection->add( '{"test3":3, "test4":4}' )->execute();

$savepoint2 = $session->setSavepoint();

$session->rollbackTo($savepoint1);
?>
```

Session::setSavepoint

Session::setSavepoint

(No version information available, might only be in Git)

Session::setSavepoint — Create savepoint

Description

public function **mysql_xdevapi\Session::setSavepoint**(string `$name` = ?): string

Create a new savepoint for the transaction.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`name`

The name of the savepoint. The name is auto-generated if the optional `name` parameter is not defined as 'SAVEPOINT1', 'SAVEPOINT2', and so on.

Return Values

The name of the save point.

Examples

Example #1 **mysql_xdevapi\Session::setSavepoint()** example

```
<?php
$session    = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$collection = $session->getSchema("addressbook")->getCollection("names");

$session->startTransaction();
$collection->add( '{"test1":1, "test2":2}' )->execute();

$savepoint = $session->setSavepoint();

$collection->add( '{"test3":3, "test4":4}' )->execute();

$session->releaseSavepoint($savepoint);
$session->rollback();
?>
```

Session::sql

Session::sql

(No version information available, might only be in Git)

Session::sql — Create SQL query

Description

public function **mysql_xdevapi\Session::sql**(string `$query`): mysql_xdevapi\SqlStatement

Create a native SQL statement. Placeholders are supported using the native "?" syntax. Use the `execute` method to execute the SQL statement.

Parameters

`query`

SQL statement to execute.

Return Values

An SqlStatement object.

Examples

Example #1 **mysql_xdevapi\Session::sql()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("CREATE DATABASE addressbook")->execute();
?>
```

Session::startTransaction

Session::startTransaction

(No version information available, might only be in Git)

Session::startTransaction — Start transaction

Description

public function **mysql_xdevapi\Session::startTransaction**(): void

Start a new transaction.

Parameters

This function has no parameters.

Return Values

An SqlStatementResult object.

Examples

Example #1 mysql_xdevapi\Session::startTransaction() example

```
<?php
$session    = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$collection = $session->getSchema("addressbook")->getCollection("friends");

$session->startTransaction();
$collection->add( '{"test1":1, "test2":2}' )->execute();

$savepoint = $session->setSavepoint();

$collection->add( '{"test3":3, "test4":4}' )->execute();

$session->releaseSavepoint($savepoint);
$session->rollback();
?>
```

mysql_xdevapi\SqlStatement

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\SqlStatement** {
/* Constants */
const int *EXECUTE_ASYNC* = 1;
const int *BUFFERED* = 2;
/* Properties */
public *$statement*;
/* Methods */
public function **bind**(string `$param`): mysql_xdevapi\SqlStatement
public function **execute**(): mysql_xdevapi\Result
public function **getNextResult**(): mysql_xdevapi\Result

public function **getResult**(): mysql_xdevapi\Result
public function **hasMoreResults**(): bool
}

Properties

statement

Predefined Constants

`mysql_xdevapi\SqlStatement::EXECUTE_ASYNC`

`mysql_xdevapi\SqlStatement::BUFFERED`

Table of Contents

SqlStatement::bind

SqlStatement::bind

(No version information available, might only be in Git)

SqlStatement::bind — Bind statement parameters

Description

public function **mysql_xdevapi\SqlStatement::bind**(string `$param`): mysql_xdevapi\SqlStatement

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`param`

Return Values

Examples

Example #1 mysql_xdevapi\SqlStatement::bind() example

```
<?php

/* ... */

?>
```

SqlStatement::__construct

SqlStatement::__construct

(No version information available, might only be in Git)

SqlStatement::__construct — Description constructor

Description

private function **mysql_xdevapi\SqlStatement::__construct**()

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\SqlStatement::__construct() example

```
<?php

/* ... */

?>
```

SqlStatement::execute

SqlStatement::execute

(No version information available, might only be in Git)

SqlStatement::execute — Execute the operation

Description

public function **mysql_xdevapi\SqlStatement::execute**(): mysql_xdevapi\Result

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\SqlStatement::execute() example

```
<?php

/* ... */

?>
```

SqlStatement::getNextResult

SqlStatement::getNextResult

(No version information available, might only be in Git)

SqlStatement::getNextResult — Get next result

Description

public function **mysql_xdevapi\SqlStatement::getNextResult**(): mysql_xdevapi\Result

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\SqlStatement::getNextResult() example

```
<?php

/* ... */

?>
```

SqlStatement::getResult

SqlStatement::getResult

(No version information available, might only be in Git)

SqlStatement::getResult — Get result

Description

public function **mysql_xdevapi\SqlStatement::getResult**(): mysql_xdevapi\Result

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\SqlStatement::getResult() example

```php
<?php

/* ... */

?>
```

SqlStatement::hasMoreResults

SqlStatement::hasMoreResults

(No version information available, might only be in Git)

SqlStatement::hasMoreResults — Check for more results

Description

public function **mysql_xdevapi\SqlStatement::hasMoreResults**(): bool

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

`true` if the result set has more objects to fetch.

Examples

Example #1 mysql_xdevapi\SqlStatement::hasMoreResults() example

```php
<?php

/* ... */

?>
```

mysql_xdevapi\SqlStatementResult

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\SqlStatementResult** implements mysql_xdevapi\BaseResult, Traversable {
/* Methods */
public function **fetchAll**(): array
public function **fetchOne**(): array
public function **getAffectedItemsCount**(): int
public function **getColumnsCount**(): int
public function **getColumnNames**(): array

public function **getColumns**(): Array
public function **getGeneratedIds**(): array
public function **getLastInsertId**(): String
public function **getWarnings**(): array
public function **getWarningCounts**(): int
public function **hasData**(): bool
public function **nextResult**(): mysql_xdevapi\Result
}

Table of Contents

SqlStatementResult::__construct

SqlStatementResult::__construct

(No version information available, might only be in Git)

SqlStatementResult::__construct — Description constructor

Description

private function **mysql_xdevapi\SqlStatementResult::__construct**()

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\SqlStatementResult::__construct() example

```php
<?php

/* ... */

?>
```

SqlStatementResult::fetchAll

SqlStatementResult::fetchAll

(No version information available, might only be in Git)

SqlStatementResult::fetchAll — Get all rows from result

Description

public function **mysql_xdevapi\SqlStatementResult::fetchAll**(): array

Fetch all the rows from the result set.

> **Warning**

> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

A numerical array with all results from the query; each result is an associative array. An empty array is returned if no rows are present.

Examples

Example #1 mysql_xdevapi\SqlStatementResult::fetchAll() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS dbtest")->execute();
$session->sql("CREATE DATABASE dbtest")->execute();
$session->sql("CREATE TABLE dbtest.workers(name text, age int, job text)")->execute();
$session->sql("INSERT INTO dbtest.workers values ('John', 42, 'bricklayer'), ('Sam', 33,
'carpenter')")->execute();

$schema = $session->getSchema("dbtest");
$table  = $schema->getTable("workers");

$rows = $session->sql("SELECT * FROM dbtest.workers")->execute()->fetchAll();

print_r($rows);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 33
        )
)
```

SqlStatementResult::fetchOne

SqlStatementResult::fetchOne

(No version information available, might only be in Git)

SqlStatementResult::fetchOne — Get single row

Description

public function **mysql_xdevapi\SqlStatementResult::fetchOne**(): array

Fetch one row from the result set.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The result, as an associative array. In case there is not any result, null will be returned.

Examples

Example #1 mysql_xdevapi\SqlStatementResult::fetchOne() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
$session->sql("DROP DATABASE IF EXISTS dbtest")->execute();
$session->sql("CREATE DATABASE dbtest")->execute();
$session->sql("CREATE TABLE dbtest.workers(name text, age int, job text)")->execute();
$session->sql("INSERT INTO dbtest.workers values ('John', 42, 'bricklayer'), ('Sam', 33,
'carpenter')")->execute();

$schema = $session->getSchema("dbtest");
$table  = $schema->getTable("workers");

$rows = $session->sql("SELECT * FROM dbtest.workers")->execute()->fetchOne();

print_r($rows);
?>
```

The above example will output something similar to:

```
Array
(
    [name] => John
    [age] => 42
    [job] => bricklayer
)
```

SqlStatementResult::getAffectedItemsCount

SqlStatementResult::getAffectedItemsCount

(No version information available, might only be in Git)

SqlStatementResult::getAffectedItemsCount — Get affected row count

Description

public function **mysql_xdevapi\SqlStatementResult::getAffectedItemsCount**(): int

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\SqlStatementResult::getAffectedItemsCount() example

```
<?php

/* ... */

?>
```

SqlStatementResult::getColumnsCount

SqlStatementResult::getColumnsCount

(No version information available, might only be in Git)

SqlStatementResult::getColumnsCount — Get column count

Description

public function **mysql_xdevapi\SqlStatementResult::getColumnsCount**(): int

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The number of columns; 0 if there are none.

Changelog

Version	Description
8.0.14	Method renamed from getColumnCount() to getColumnsCount().

Examples

Example #1 mysql_xdevapi\SqlStatementResult::getColumnsCount() example

```
<?php

/* ... */

?>
```

SqlStatementResult::getColumnNames

SqlStatementResult::getColumnNames

(No version information available, might only be in Git)

SqlStatementResult::getColumnNames — Get column names

Description

public function **mysql_xdevapi\SqlStatementResult::getColumnNames**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\SqlStatementResult::getColumnNames() example

```
<?php

/* ... */

?>
```

SqlStatementResult::getColumns

SqlStatementResult::getColumns

(No version information available, might only be in Git)

SqlStatementResult::getColumns — Get columns

Description

public function **mysql_xdevapi\SqlStatementResult::getColumns**(): Array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\SqlStatementResult::getColumns() example

```
<?php

/* ... */

?>
```

SqlStatementResult::getGeneratedIds

SqlStatementResult::getGeneratedIds

(No version information available, might only be in Git)

SqlStatementResult::getGeneratedIds — Get generated ids

Description

public function **mysql_xdevapi\SqlStatementResult::getGeneratedIds**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

An array of generated _id's from the last operation, or an empty array if there are none.

Examples

Example #1 mysql_xdevapi\SqlStatementResult::getGeneratedIds() example

```
<?php

/* ... */

?>
```

SqlStatementResult::getLastInsertId

SqlStatementResult::getLastInsertId

(No version information available, might only be in Git)

SqlStatementResult::getLastInsertId — Get last insert id

Description

public function **mysql_xdevapi\SqlStatementResult::getLastInsertId**(): String

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The ID for the last insert operation.

Examples

Example #1 mysql_xdevapi\SqlStatementResult::getLastInsertId() example

```
<?php

/* ... */

?>
```

SqlStatementResult::getWarnings

SqlStatementResult::getWarnings

(No version information available, might only be in Git)

SqlStatementResult::getWarnings — Get warnings from last operation

Description

public function **mysql_xdevapi\SqlStatementResult::getWarnings**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

An array of Warning objects from the last operation. Each object defines an error 'message', error 'level', and error 'code'. An empty array is returned if no errors are present.

Examples

Example #1 mysql_xdevapi\SqlStatementResult::getWarnings() example

```
<?php

/* ... */

?>
```

SqlStatementResult::getWarningsCount

SqlStatementResult::getWarningsCount

(No version information available, might only be in Git)

SqlStatementResult::getWarningsCount — Get warning count from last operation

Description

public function **mysql_xdevapi\SqlStatementResult::getWarningCounts**(): int

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The number of warnings raised during the last CRUD operation.

Examples

Example #1 mysql_xdevapi\SqlStatementResult::getWarningsCount() example

```
<?php

/* ... */

?>
```

SqlStatementResult::hasData

SqlStatementResult::hasData

(No version information available, might only be in Git)

SqlStatementResult::hasData — Check if result has data

Description

public function **mysql_xdevapi\SqlStatementResult::hasData**(): bool

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

`true` if the result set has data.

Examples

Example #1 mysql_xdevapi\SqlStatementResult::hasData() example

```
<?php

/* ... */

?>
```

SqlStatementResult::nextResult

SqlStatementResult::nextResult

(No version information available, might only be in Git)

SqlStatementResult::nextResult — Get next result

Description

public function **mysql_xdevapi\SqlStatementResult::nextResult**(): mysql_xdevapi\Result

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The next Result object from the result set.

Examples

Example #1 mysql_xdevapi\SqlStatementResult::nextResult() example

```
<?php

/* ... */
```

```
?>
```

mysql_xdevapi\Statement

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\Statement** {
/* Constants */
const int *EXECUTE_ASYNC* = 1;
const int *BUFFERED* = 2;
/* Methods */
public function **getNextResult**(): mysql_xdevapi\Result
public function **getResult**(): mysql_xdevapi\Result
public function **hasMoreResults**(): bool
}

Predefined Constants

`mysql_xdevapi\Statement::EXECUTE_ASYNC`

`mysql_xdevapi\Statement::BUFFERED`

Table of Contents

Statement::__construct

Statement::__construct

(No version information available, might only be in Git)

Statement::__construct — Description constructor

Description

private function **mysql_xdevapi\Statement::__construct**()

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\Statement::__construct() example

```
<?php

/* ... */
```

```
?>
```

Statement::getNextResult

Statement::getNextResult

(No version information available, might only be in Git)

Statement::getNextResult — Get next result

Description

public function **mysql_xdevapi\Statement::getNextResult**(): mysql_xdevapi\Result

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\Statement::getNextResult() example

```
<?php

/* ... */

?>
```

Statement::getResult

Statement::getResult

(No version information available, might only be in Git)

Statement::getResult — Get result

Description

public function **mysql_xdevapi\Statement::getResult**(): mysql_xdevapi\Result

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\Statement::getResult() example

```
<?php

/* ... */

?>
```

Statement::hasMoreResults

Statement::hasMoreResults

(No version information available, might only be in Git)

Statement::hasMoreResults — Check if more results

Description

public function **mysql_xdevapi\Statement::hasMoreResults**(): bool

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 mysql_xdevapi\Statement::hasMoreResults() example

```
<?php

/* ... */

?>
```

mysql_xdevapi\Table

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Provides access to the table through INSERT/SELECT/UPDATE/DELETE statements.

Class synopsis

class **mysql_xdevapi\Table** implements mysql_xdevapi\SchemaObject {
/* Properties */
public *$name*;
/* Methods */

public function **count**(): int
public function **delete**(): mysql_xdevapi\TableDelete
public function **existsInDatabase**(): bool
public function **getName**(): string
public function **getSchema**(): mysql_xdevapi\Schema
public function **getSession**(): mysql_xdevapi\Session
public function **insert**(mixed `$columns`, mixed `...$more_columns`): mysql_xdevapi\TableInsert
public function **isView**(): bool
public function **select**(mixed `$columns`, mixed `...$more_columns`): mysql_xdevapi\TableSelect
public function **update**(): mysql_xdevapi\TableUpdate
}

Properties

name

Table of Contents

Table::__construct

Table::__construct

(No version information available, might only be in Git)

Table::__construct — Table constructor

Description

private function **mysql_xdevapi\Table::__construct**()

Construct a table object.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\Table::__construct() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");
?>
```

Table::count

Table::count

(No version information available, might only be in Git)

Table::count — Get row count

Description

public function **mysql_xdevapi\Table::count**(): int

Fetch the number of rows in the table.

Parameters

This function has no parameters.

Return Values

The total number of rows in the table.

Examples

Example #1 mysql_xdevapi\Table::count() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

var_dump($table->count());
?>
```

The above example will output:

```
int(2)
```

Table::delete

Table::delete

(No version information available, might only be in Git)

Table::delete — Delete rows from table

Description

public function **mysql_xdevapi\Table::delete**(): mysql_xdevapi\TableDelete

Deletes rows from a table.

Parameters

This function has no parameters.

Return Values

A TableDelete object; use the execute() method to execute the delete query.

Examples

Example #1 **mysql_xdevapi\Table::delete()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()->where("name = :name")->orderby("age DESC")->limit(1)->bind(['name' => 'John'])-
>execute();
?>
```

Table::existsInDatabase

Table::existsInDatabase

(No version information available, might only be in Git)

Table::existsInDatabase — Check if table exists in database

Description

public function **mysql_xdevapi\Table::existsInDatabase**(): bool

Verifies if this table exists in the database.

Parameters

This function has no parameters.

Return Values

Returns `true` if table exists in the database, else `false` if it does not.

Examples

Example #1 **mysql_xdevapi\Table::existsInDatabase()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
```

```php
$table  = $schema->getTable("names");

if ($table->existsInDatabase()) {
  echo "Yes, this table still exists in the session's schema.";
}
?>
```

The above example will output something similar to:

```
Yes, this table still exists in the session's schema.
```

Table::getName

Table::getName

(No version information available, might only be in Git)

Table::getName — Get table name

Description

public function **mysql_xdevapi\Table::getName**(): string

Returns the name of this database object.

Parameters

This function has no parameters.

Return Values

The name of this database object.

Examples

Example #1 mysql_xdevapi\Table::getName() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

var_dump($table->getName());
?>
```

The above example will output something similar to:

```
string(5) "names"
```

Table::getSchema

Table::getSchema

(No version information available, might only be in Git)

Table::getSchema — Get table schema

Description

public function **mysql_xdevapi\Table::getSchema**(): mysql_xdevapi\Schema

Fetch the schema associated with the table.

Parameters

This function has no parameters.

Return Values

A Schema object.

Examples

Example #1 **mysql_xdevapi\Table::getSchema()** example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

var_dump($table->getSchema());
?>
```

The above example will output something similar to:

```
object(mysql_xdevapi\Schema)#9 (1) {
  ["name"]=>
  string(11) "addressbook"
}
```

Table::getSession

Table::getSession

(No version information available, might only be in Git)

Table::getSession — Get table session

Description

public function **mysql_xdevapi\Table::getSession**(): mysql_xdevapi\Session

Get session associated with the table.

Parameters

This function has no parameters.

Return Values

A Session object.

Examples

Example #1 **mysql_xdevapi\Table::getSession()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

var_dump($table->getSession());
?>
```

The above example will output something similar to:

```
object(mysql_xdevapi\Session)#9 (0) {
}
```

Table::insert

Table::insert

(No version information available, might only be in Git)

Table::insert — Insert table rows

Description

public function **mysql_xdevapi\Table::insert**(mixed `$columns`, mixed `...$more_columns`): mysql_xdevapi\TableInsert

Inserts rows into a table.

Parameters

`columns`

The columns to insert data into. Can be an array with one or more values, or a string.

`more_columns`

Additional columns definitions.

Return Values

A TableInsert object; use the execute() method to execute the insert statement.

Examples

Example #1 mysql_xdevapi\Table::insert() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table ->insert("name", "age")
  ->values(["Suzanne", 31],["Julie", 43])
  ->execute();
?>
```

Table::isView

Table::isView

(No version information available, might only be in Git)

Table::isView — Check if table is view

Description

public function **mysql_xdevapi\Table::isView**(): bool

Determine if the underlying object is a view or not.

Parameters

This function has no parameters.

Return Values

`true` if the underlying object is a view, otherwise `false`.

Examples

Example #1 mysql_xdevapi\Table::isView() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

if ($table->isView()) {
    echo "This is a view.";
} else {
    echo "This is not a view.";
}
?>
```

The above example will output:

```
This is not a view.
```

Table::select

Table::select

(No version information available, might only be in Git)

Table::select — Select rows from table

Description

public function **mysql_xdevapi\Table::select**(mixed `$columns`, mixed `...$more_columns`): mysql_xdevapi\TableSelect

Fetches data from a table.

Parameters

`columns`

The columns to select data from. Can be an array with one or more values, or a string.

`more_columns`

Additional columns parameter definitions.

Return Values

A TableSelect object; use the execute() method to execute the select and return a RowResult object.

Examples

Example #1 **mysql_xdevapi\Table::count()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$row = $table->select('name', 'age')->execute()->fetchAll();

print_r($row);
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 33
        )
)
```

Table::update

Table::update

(No version information available, might only be in Git)

Table::update — Update rows in table

Description

public function **mysql_xdevapi\Table::update**(): mysql_xdevapi\TableUpdate

Updates columns in a table.

Parameters

This function has no parameters.

Return Values

A TableUpdate object; use the execute() method to execute the update statement.

Examples

Example #1 mysql_xdevapi\Table::update() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->update()->set('age',34)->where('name = "Sam"')->limit(1)->execute();
?>
```

mysql_xdevapi\TableDelete

(PECL mysql-xdevapi >= 8.0.11)

Introduction

A statement for delete operations on Table.

Class synopsis

class **mysql_xdevapi\TableDelete** implements mysql_xdevapi\Executable {
/* Methods */
public function **bind**(array `$placeholder_values`): mysql_xdevapi\TableDelete
public function **execute**(): mysql_xdevapi\Result
public function **limit**(int `$rows`): mysql_xdevapi\TableDelete
public function **orderby**(string `$orderby_expr`): mysql_xdevapi\TableDelete
public function **where**(string `$where_expr`): mysql_xdevapi\TableDelete
}

Table of Contents

TableDelete::bind

TableDelete::bind

(No version information available, might only be in Git)

TableDelete::bind — Bind delete query parameters

Description

public function **mysql_xdevapi\TableDelete::bind**(array `$placeholder_values`): mysql_xdevapi\TableDelete

Binds a value to a specific placeholder.

Parameters

`placeholder_values`

The name of the placeholder and the value to bind.

Return Values

A TableDelete object.

Examples

Example #1 mysql_xdevapi\TableDelete::bind() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()
  ->where("name = :name")
  ->bind(['name' => 'John'])
  ->orderby("age DESC")
  ->limit(1)
  ->execute();

?>
```

TableDelete::__construct

TableDelete::__construct

(No version information available, might only be in Git)

TableDelete::__construct — TableDelete constructor

Description

private function **mysql_xdevapi\TableDelete::__construct**()

Initiated by using the delete() method.

Parameters

This function has no parameters.

Examples

Example #1 **mysql_xdevapi\TableDelete::__construct()** example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()
  ->where("name = :name")
  ->bind(['name' => 'John'])
  ->orderby("age DESC")
  ->limit(1)
  ->execute();

?>
```

TableDelete::execute

TableDelete::execute

(No version information available, might only be in Git)

TableDelete::execute — Execute delete query

Description

public function **mysql_xdevapi\TableDelete::execute**(): mysql_xdevapi\Result

Execute the delete query.

Parameters

This function has no parameters.

Return Values

A Result object.

Examples

Example #1 **mysql_xdevapi\TableDelete::execute()** example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
```

```
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()
  ->where("name = :name")
  ->bind(['name' => 'John'])
  ->orderby("age DESC")
  ->limit(1)
  ->execute();

?>
```

TableDelete::limit

TableDelete::limit

(No version information available, might only be in Git)

TableDelete::limit — Limit deleted rows

Description

public function **mysql_xdevapi\TableDelete::limit**(int `$rows`): mysql_xdevapi\TableDelete

Sets the maximum number of records or documents to delete.

Parameters

`rows`

The maximum number of records or documents to delete.

Return Values

TableDelete object.

Examples

Example #1 mysql_xdevapi\TableDelete::limit() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()
  ->where("name = :name")
  ->bind(['name' => 'John'])
  ->orderby("age DESC")
  ->limit(1)
  ->execute();
```

```
?>
```

TableDelete::orderby

TableDelete::orderby

(No version information available, might only be in Git)

TableDelete::orderby — Set delete sort criteria

Description

public function **mysql_xdevapi\TableDelete::orderby**(string `$orderby_expr`): mysql_xdevapi\TableDelete

Set the order options for a result set.

Parameters

`orderby_expr`

The sort definition.

Return Values

A TableDelete object.

Examples

Example #1 mysql_xdevapi\TableDelete::orderBy() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()
  ->where("age = :age")
  ->bind(['age' => 42])
  ->orderby("name DESC")
  ->limit(1)
  ->execute();

?>
```

TableDelete::where

TableDelete::where

(No version information available, might only be in Git)

TableDelete::where — Set delete search condition

Description

public function **mysql_xdevapi\TableDelete::where**(string `$where_expr`): mysql_xdevapi\TableDelete

Sets the search condition to filter.

Parameters

`where_expr`

Define the search condition to filter documents or records.

Return Values

TableDelete object.

Examples

Example #1 mysql_xdevapi\TableDelete::where() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->delete()
  ->where("id = :id")
  ->bind(['id' => 42])
  ->limit(1)
  ->execute();

?>
```

mysql_xdevapi\TableInsert

(PECL mysql-xdevapi >= 8.0.11)

Introduction

A statement for insert operations on Table.

Class synopsis

class **mysql_xdevapi\TableInsert** implements mysql_xdevapi\Executable {
/* Methods */
public function **execute**(): mysql_xdevapi\Result
public function **values**(array `$row_values`): mysql_xdevapi\TableInsert
}

Table of Contents

TableInsert::__construct

TableInsert::__construct

(No version information available, might only be in Git)

TableInsert::__construct — TableInsert constructor

Description

private function **mysql_xdevapi\TableInsert::__construct**()

Initiated by using the insert() method.

Parameters

This function has no parameters.

Examples

Example #1 **mysql_xdevapi\TableInsert::__construct()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table
  ->insert("name", "age")
  ->values(["Suzanne", 31],["Julie", 43])
  ->execute();
?>
```

TableInsert::execute

TableInsert::execute

(No version information available, might only be in Git)

TableInsert::execute — Execute insert query

Description

public function **mysql_xdevapi\TableInsert::execute**(): mysql_xdevapi\Result

Execute the statement.

Parameters

This function has no parameters.

Return Values

A Result object.

Examples

Example #1 **mysql_xdevapi\TableInsert::execute()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
```

```php
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table
  ->insert("name", "age")
  ->values(["Suzanne", 31],["Julie", 43])
  ->execute();
?>
```

TableInsert::values

TableInsert::values

(No version information available, might only be in Git)

TableInsert::values — Add insert row values

Description

public function **mysql_xdevapi\TableInsert::values**(array `$row_values`): mysql_xdevapi\TableInsert

Set the values to be inserted.

Parameters

`row_values`

Values (an array) of columns to insert.

Return Values

A TableInsert object.

Examples

Example #1 mysql_xdevapi\TableInsert::values() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table
  ->insert("name", "age")
  ->values(["Suzanne", 31],["Julie", 43])
  ->execute();
?>
```

mysql_xdevapi\TableSelect

(PECL mysql-xdevapi >= 8.0.11)

Introduction

A statement for record retrieval operations on a Table.

Class synopsis

class **mysql_xdevapi\TableSelect** implements mysql_xdevapi\Executable {
/* Methods */
public function **bind**(array `$placeholder_values`): mysql_xdevapi\TableSelect
public function **execute**(): mysql_xdevapi\RowResult
public function **groupBy**(mixed `$sort_expr`): mysql_xdevapi\TableSelect
public function **having**(string `$sort_expr`): mysql_xdevapi\TableSelect
public function **limit**(int `$rows`): mysql_xdevapi\TableSelect
public function **lockExclusive**(int `$lock_waiting_option = ?`): mysql_xdevapi\TableSelect
public function **lockShared**(int `$lock_waiting_option = ?`): mysql_xdevapi\TableSelect
public function **offset**(int `$position`): mysql_xdevapi\TableSelect
public function **orderby**(mixed `$sort_expr`, mixed `...$sort_exprs`): mysql_xdevapi\TableSelect
public function **where**(string `$where_expr`): mysql_xdevapi\TableSelect
}

Table of Contents

TableSelect::bind

TableSelect::bind

(No version information available, might only be in Git)

TableSelect::bind — Bind select query parameters

Description

public function **mysql_xdevapi\TableSelect::bind**(array `$placeholder_values`): mysql_xdevapi\TableSelect

Binds a value to a specific placeholder.

Parameters

`placeholder_values`

The name of the placeholder, and the value to bind.

Return Values

A TableSelect object.

Examples

Example #1 mysql_xdevapi\TableSelect::bind() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");
```

```php
$result = $table->select('name','age')
  ->where('name like :name and age > :age')
  ->bind(['name' => 'John', 'age' => 42])
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
)
```

TableSelect::__construct

TableSelect::__construct

(No version information available, might only be in Git)

TableSelect::__construct — TableSelect constructor

Description

private function **mysql_xdevapi\TableSelect::__construct**()

An object returned by the select() method; use execute() to execute the query.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\TableSelect::__construct() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 33)")->execute();

$schema = $session->getSchema("addressbook");
```

```php
$table  = $schema->getTable("names");

$result = $table->select('name','age')
  ->where('name like :name and age > :age')
  ->bind(['name' => 'John', 'age' => 42])
  ->orderBy('age desc')
  ->execute();

$row = $result->fetchAll();
print_r($row);

?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
)
```

TableSelect::execute

TableSelect::execute

(No version information available, might only be in Git)

TableSelect::execute — Execute select statement

Description

public function **mysql_xdevapi\TableSelect::execute**(): mysql_xdevapi\RowResult

Execute the select statement by chaining it with the execute() method.

Parameters

This function has no parameters.

Return Values

A RowResult object.

Examples

Example #1 mysql_xdevapi\TableSelect::execute() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
```

```
$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('name','age')
  ->where('name like :name and age > :age')
  ->bind(['name' => 'John', 'age' => 42])
  ->orderBy('age desc')
  ->execute();

$row = $result->fetchAll();
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
)
```

TableSelect::groupBy

TableSelect::groupBy

(No version information available, might only be in Git)

TableSelect::groupBy — Set select grouping criteria

Description

public function **mysql_xdevapi\TableSelect::groupBy**(mixed `$sort_expr`): mysql_xdevapi\TableSelect

Sets a grouping criteria for the result set.

Parameters

`sort_expr`

The grouping criteria.

Return Values

A TableSelect object.

Examples

Example #1 mysql_xdevapi\TableSelect::groupBy() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 42)")->execute();
$session->sql("INSERT INTO addressbook.names values ('Suki', 31)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('count(*) as count', 'age')
  ->groupBy('age')->orderBy('age asc')
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [count] => 1
            [age] => 31
        )
    [1] => Array
        (
            [count] => 2
            [age] => 42
        )
)
```

TableSelect::having

TableSelect::having

(No version information available, might only be in Git)

TableSelect::having — Set select having condition

Description

public function **mysql_xdevapi\TableSelect::having**(string `$sort_expr`): mysql_xdevapi\TableSelect

Sets a condition for records to consider in aggregate function operations.

Parameters

`sort_expr`

A condition on the aggregate functions used on the grouping criteria.

Return Values

A TableSelect object.

Examples

Example #1 **mysql_xdevapi\TableSelect::having()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 42)")->execute();
$session->sql("INSERT INTO addressbook.names values ('Suki', 31)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('count(*) as count', 'age')
  ->groupBy('age')->orderBy('age asc')
  ->having('count > 1')
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [count] => 2
            [age] => 42
        )
)
```

TableSelect::limit

TableSelect::limit

(No version information available, might only be in Git)

TableSelect::limit — Limit selected rows

Description

public function **mysql_xdevapi\TableSelect::limit**(int `$rows`): mysql_xdevapi\TableSelect

Sets the maximum number of records or documents to return.

Parameters

`rows`

The maximum number of records or documents.

Return Values

A TableSelect object.

Examples

Example #1 mysql_xdevapi\TableSelect::limit() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('name', 'age')
  ->limit(1)
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
)
```

TableSelect::lockExclusive

TableSelect::lockExclusive

(No version information available, might only be in Git)

TableSelect::lockExclusive — Execute EXCLUSIVE LOCK

Description

public function **mysql_xdevapi\TableSelect::lockExclusive**(int `$lock_waiting_option = ?`): mysql_xdevapi\TableSelect

Execute a read operation with EXCLUSIVE LOCK. Only one lock can be active at a time.

Parameters

`lock_waiting_option`

The optional waiting option that defaults to `MYSQLX_LOCK_DEFAULT`. Valid values are:

- `MYSQLX_LOCK_DEFAULT`
- `MYSQLX_LOCK_NOWAIT`
- `MYSQLX_LOCK_SKIP_LOCKED`

Return Values

TableSelect object.

Examples

Example #1 mysql_xdevapi\TableSelect::lockExclusive() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$session->startTransaction();

$result = $table->select('name', 'age')
  ->lockExclusive(MYSQLX_LOCK_NOWAIT)
  ->execute();

$session->commit();

$row = $result->fetchAll();
print_r($row);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
```

```
            [name] => Sam
            [age] => 42
        )
)
```

TableSelect::lockShared

TableSelect::lockShared

(No version information available, might only be in Git)

TableSelect::lockShared — Execute SHARED LOCK

Description

public function **mysql_xdevapi\TableSelect::lockShared**(int `$lock_waiting_option` = ?): mysql_xdevapi\TableSelect

Execute a read operation with SHARED LOCK. Only one lock can be active at a time.

Parameters

`lock_waiting_option`

The optional waiting option that defaults to `MYSQLX_LOCK_DEFAULT`. Valid values are:

- `MYSQLX_LOCK_DEFAULT`
- `MYSQLX_LOCK_NOWAIT`
- `MYSQLX_LOCK_SKIP_LOCKED`

Return Values

A TableSelect object.

Examples

Example #1 mysql_xdevapi\TableSelect::lockShared() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$session->startTransaction();

$result = $table->select('name', 'age')
  ->lockShared(MYSQLX_LOCK_NOWAIT)
  ->execute();

$session->commit();

$row = $result->fetchAll();
```

```
print_r($row);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
    [1] => Array
        (
            [name] => Sam
            [age] => 42
        )
)
```

TableSelect::offset

TableSelect::offset

(No version information available, might only be in Git)

TableSelect::offset — Set limit offset

Description

public function **mysql_xdevapi\TableSelect::offset**(int `$position`): mysql_xdevapi\TableSelect

Skip given number of rows in result.

Parameters

`position`

The limit offset.

Return Values

A TableSelect object.

Examples

Example #1 mysql_xdevapi\TableSelect::offset() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$session->sql("DROP DATABASE IF EXISTS addressbook")->execute();
$session->sql("CREATE DATABASE addressbook")->execute();
```

```php
$session->sql("CREATE TABLE addressbook.names(name text, age int)")->execute();
$session->sql("INSERT INTO addressbook.names values ('John', 42), ('Sam', 42)")->execute();

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('name', 'age')
  ->limit(1)
  ->offset(1)
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => Sam
            [age] => 42
        )
)
```

TableSelect::orderby

TableSelect::orderby

(No version information available, might only be in Git)

TableSelect::orderby — Set select sort criteria

Description

public function **mysql_xdevapi\TableSelect::orderby**(mixed `$sort_expr`, mixed `...$sort_exprs`): mysql_xdevapi\TableSelect

Sets the order by criteria.

Parameters

`sort_expr`

The expressions that define the order by criteria. Can be an array with one or more expressions, or a string.

`sort_exprs`

Additional sort_expr parameters.

Return Values

A TableSelect object.

Examples

Example #1 **mysql_xdevapi\TableSelect::orderBy()** example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('name', 'age')
  ->orderBy('name desc')
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => Sam
            [age] => 42
        )
    [1] => Array
        (
            [name] => John
            [age] => 42
        )
)
```

TableSelect::where

TableSelect::where

(No version information available, might only be in Git)

TableSelect::where — Set select search condition

Description

public function **mysql_xdevapi\TableSelect::where**(string `$where_expr`): mysql_xdevapi\TableSelect

Sets the search condition to filter.

Parameters

`where_expr`

Define the search condition to filter documents or records.

Return Values

A TableSelect object.

Examples

Example #1 **mysql_xdevapi\TableSelect::where()** example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$result = $table->select('name','age')
  ->where('name like :name and age > :age')
  ->bind(['name' => 'John', 'age' => 42])
  ->execute();

$row = $result->fetchAll();
print_r($row);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [name] => John
            [age] => 42
        )
)
```

mysql_xdevapi\TableUpdate

(PECL mysql-xdevapi >= 8.0.11)

Introduction

A statement for record update operations on a Table.

Class synopsis

class **mysql_xdevapi\TableUpdate** implements mysql_xdevapi\Executable {
/* Methods */
public function **bind**(array `$placeholder_values`): mysql_xdevapi\TableUpdate

public function **execute**(): mysql_xdevapi\TableUpdate
public function **limit**(int `$rows`): mysql_xdevapi\TableUpdate
public function **orderby**(mixed `$orderby_expr`, mixed `...$orderby_exprs`): mysql_xdevapi\TableUpdate
public function **set**(string `$table_field`, string `$expression_or_literal`): mysql_xdevapi\TableUpdate
public function **where**(string `$where_expr`): mysql_xdevapi\TableUpdate
}

Table of Contents

TableUpdate::bind

TableUpdate::bind

(No version information available, might only be in Git)

TableUpdate::bind — Bind update query parameters

Description

public function **mysql_xdevapi\TableUpdate::bind**(array `$placeholder_values`): mysql_xdevapi\TableUpdate

Binds a value to a specific placeholder.

Parameters

`placeholder_values`

The name of the placeholder, and the value to bind, defined as a JSON array.

Return Values

A TableUpdate object.

Examples

Example #1 mysql_xdevapi\TableUpdate::bind() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$table->update()
  ->set('status', 'admin')
  ->where('name = :name and age > :age')
  ->bind(['name' => 'Bernie', 'age' => 2000])
  ->execute();

?>
```

TableUpdate::__construct

TableUpdate::__construct

(No version information available, might only be in Git)

TableUpdate::__construct — TableUpdate constructor

Description

private function **mysql_xdevapi\TableUpdate::__construct**()

Initiated by using the update() method.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\TableUpdate::__construct() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$res = $table->update()
  ->set('level', 3)
  ->where('age > 15 and age < 22')
  ->limit(4)
  ->orderby(['age asc','name desc'])
  ->execute();

?>
```

TableUpdate::execute

TableUpdate::execute

(No version information available, might only be in Git)

TableUpdate::execute — Execute update query

Description

public function **mysql_xdevapi\TableUpdate::execute**(): mysql_xdevapi\TableUpdate

Executes the update statement.

Parameters

This function has no parameters.

Return Values

A TableUpdate object.

Examples

Example #1 mysql_xdevapi\TableUpdate::execute() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");
```

```
$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$res = $table->update()
  ->set('level', 3)
  ->where('age > 15 and age < 22')
  ->limit(4)
  ->orderby(['age asc','name desc'])
  ->execute();

?>
```

TableUpdate::limit

TableUpdate::limit

(No version information available, might only be in Git)

TableUpdate::limit — Limit update row count

Description

public function **mysql_xdevapi\TableUpdate::limit**(int `$rows`): mysql_xdevapi\TableUpdate

Set the maximum number of records or documents update.

Parameters

`rows`

The maximum number of records or documents to update.

Return Values

A TableUpdate object.

Examples

Example #1 mysql_xdevapi\TableUpdate::limit() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$res = $table->update()
  ->set('level', 3)
  ->where('age > 15 and age < 22')
  ->limit(4)
  ->orderby(['age asc','name desc'])
  ->execute();

?>
```

TableUpdate::orderby

TableUpdate::orderby

(No version information available, might only be in Git)

TableUpdate::orderby — Set sorting criteria

Description

public function **mysql_xdevapi\TableUpdate::orderby**(mixed `$orderby_expr`, mixed `...$orderby_exprs`): mysql_xdevapi\TableUpdate

Sets the sorting criteria.

Parameters

`orderby_expr`

The expressions that define the order by criteria. Can be an array with one or more expressions, or a string.

`orderby_exprs`

Additional sort_expr parameters.

Return Values

TableUpdate object.

Examples

Example #1 mysql_xdevapi\TableUpdate::orderby() example

```php
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$res = $table->update()
  ->set('level', 3)
  ->where('age > 15 and age < 22')
  ->limit(4)
  ->orderby(['age asc','name desc'])
  ->execute();
?>
```

TableUpdate::set

TableUpdate::set

(No version information available, might only be in Git)

TableUpdate::set — Add field to be updated

Description

public function **mysql_xdevapi\TableUpdate::set**(string `$table_field`, string `$expression_or_literal`): mysql_xdevapi\TableUpdate

Updates the column value on records in a table.

Parameters

`table_field`

The column name to be updated.

`expression_or_literal`

The value to be set on the specified column.

Return Values

TableUpdate object.

Examples

Example #1 mysql_xdevapi\TableUpdate::set() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$res = $table->update()
  ->set('level', 3)
  ->where('age > 15 and age < 22')
  ->limit(4)
  ->orderby(['age asc','name desc'])
  ->execute();

?>
```

TableUpdate::where

TableUpdate::where

(No version information available, might only be in Git)

TableUpdate::where — Set search filter

Description

public function **mysql_xdevapi\TableUpdate::where**(string `$where_expr`): mysql_xdevapi\TableUpdate

Set the search condition to filter.

Parameters

`where_expr`

The search condition to filter documents or records.

Return Values

A TableUpdate object.

Examples

Example #1 mysql_xdevapi\TableUpdate::where() example

```
<?php
$session = mysql_xdevapi\getSession("mysqlx://user:password@localhost");

$schema = $session->getSchema("addressbook");
$table  = $schema->getTable("names");

$res = $table->update()
  ->set('level', 3)
  ->where('age > 15 and age < 22')
  ->limit(4)
  ->orderby(['age asc','name desc'])
  ->execute();

?>
```

mysql_xdevapi\Warning

(PECL mysql-xdevapi >= 8.0.11)

Introduction

Class synopsis

class **mysql_xdevapi\Warning** {
/* Properties */
public *$message*;
public *$level*;
public *$code*;
/* Constructor */
private function **__construct**()
}

Properties

message

level

code

Table of Contents

Warning::__construct

Warning::__construct

(No version information available, might only be in Git)

Warning::__construct — Warning constructor

Description

private function **mysql_xdevapi\Warning::__construct**()

> **Warning**

> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Examples

Example #1 mysql_xdevapi\Warning::__construct() example

```php
<?php

/* ... */

?>
```

MySQL (Original)

Introduction

This extension is deprecated as of PHP 5.5.0, and has been removed as of PHP 7.0.0. Instead, either the mysqli or PDO_MySQL extension should be used. See also the MySQL API Overview for further help while choosing a MySQL API.

These functions allow you to access MySQL database servers. More information about MySQL can be found at » http://www.mysql.com/.

Documentation for MySQL can be found at » http://dev.mysql.com/doc/.

Installing/Configuring

Table of Contents

Requirements

In order to have these functions available, you must compile PHP with MySQL support.

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:

Installation

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:

For compiling, simply use the **--with-mysql[=DIR]** configuration option where the optional `[DIR]` points to the MySQL installation directory.

Although this MySQL extension is compatible with MySQL 4.1.0 and greater, it doesn't support the extra functionality that these versions provide. For that, use the MySQLi extension.

If you would like to install the mysql extension along with the mysqli extension you have to use the same client library to avoid any conflicts.

Installation on Linux Systems

Note: `[DIR]` is the path to the MySQL client library files (*headers and libraries*), which can be downloaded from » MySQL.

ext/mysql compile time support matrix

PHP Version	Default	Configure Options: mysqlnd	Configure Options: `libmysqlclient`	Changelog
4.x.x	libmysqlclient	Not Available	**--without-mysql** to disable	MySQL enabled by default, MySQL client libraries are bundled
5.0.x, 5.1.x, 5.2.x	libmysqlclient	Not Available	**--with-mysql=[DIR]**	MySQL is no longer enabled by default, and the MySQL client libraries are no longer bundled
5.3.x	libmysqlclient	**--with-mysql=mysqlnd**	**--with-mysql=[DIR]**	mysqlnd is now available
5.4.x	mysqlnd	**--with-mysql**	**--with-mysql=[DIR]**	mysqlnd is now the default

Installation on Windows Systems

PHP 5.0.x, 5.1.x, 5.2.x

MySQL is no longer enabled by default, so the *php_mysql.dll* DLL must be enabled inside of *php.ini*. Also, PHP needs access to the MySQL client library. A file named *libmysql.dll* is included in the Windows PHP distribution and in order for PHP to talk to MySQL this file needs to be available to the Windows systems *PATH*. See the FAQ titled "How do I add my PHP directory to the *PATH* on Windows" for information on how to do this. Although copying *libmysql.dll* to the Windows system directory also works (because the system directory is by default in the system's *PATH*), it's not recommended.

As with enabling any PHP extension (such as *php_mysql.dll*), the PHP directive extension_dir should be set to the directory where the PHP extensions are located. See also the Manual Windows Installation Instructions. An example extension_dir value for PHP 5 is `c:\php\ext`

> **Note** :
>
> If when starting the web server an error similar to the following occurs: `"Unable to load dynamic library './php_mysql.dll'"`, this is because *php_mysql.dll* and/or *libmysql.dll* cannot be found by the system.

PHP 5.3.0+

The MySQL Native Driver is enabled by default. Include *php_mysql.dll*, but *libmysql.dll* is no longer required or used.

MySQL Installation Notes

> **Warning**
>
> Crashes and startup problems of PHP may be encountered when loading this extension in conjunction with the recode extension. See the recode extension for more information.

> **Note** :
>
> If you need charsets other than *latin* (default), you have to install external (not bundled) libmysqlclient with compiled charset support.

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

MySQL Configuration Options

Name	Default	Changeable	Changelog
mysql.allow_local_infile	"1"	INI_SYSTEM	
mysql.allow_persistent	"1"	INI_SYSTEM	
mysql.max_persistent	"-1"	INI_SYSTEM	
mysql.max_links	"-1"	INI_SYSTEM	
mysql.trace_mode	"0"	INI_ALL	
mysql.default_port	NULL	INI_ALL	
mysql.default_socket	NULL	INI_ALL	
mysql.default_host	NULL	INI_ALL	
mysql.default_user	NULL	INI_ALL	
mysql.default_password	NULL	INI_ALL	
mysql.connect_timeout	"60"	INI_ALL	

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`mysql.allow_local_infile` int

Allow accessing, from PHP's perspective, local files with LOAD DATA statements

`mysql.allow_persistent` bool

Whether to allow persistent connections to MySQL.

`mysql.max_persistent` int

The maximum number of persistent MySQL connections per process.

`mysql.max_links` int

The maximum number of MySQL connections per process, including persistent connections.

`mysql.trace_mode` bool

Trace mode. When `mysql.trace_mode` is enabled, warnings for table/index scans, non free result sets, and SQL-Errors will be displayed. (Introduced in PHP 4.3.0)

`mysql.default_port` string

The default TCP port number to use when connecting to the database server if no other port is specified. If no default is specified, the port will be obtained from the *MYSQL_TCP_PORT* environment variable, the `mysql-tcp` entry in */etc/services* or the compile-time **`MYSQL_PORT`** constant, in that order. Win32 will only use the **`MYSQL_PORT`** constant.

`mysql.default_socket` string

The default socket name to use when connecting to a local database server if no other socket name is specified.

`mysql.default_host` string

The default server host to use when connecting to the database server if no other host is specified. Doesn't apply in SQL safe mode.

`mysql.default_user` string

The default user name to use when connecting to the database server if no other name is specified. Doesn't apply in SQL safe mode.

`mysql.default_password` string

The default password to use when connecting to the database server if no other password is specified. Doesn't apply in SQL safe mode.

`mysql.connect_timeout` int

Connect timeout in seconds. On Linux this timeout is also used for waiting for the first answer from the server.

Resource Types

There are two resource types used in the MySQL module. The first one is the link identifier for a database connection, the second a resource which holds the result of a query.

Changelog

The following changes have been made to classes/functions/methods of this extension.

General Changelog for the ext/mysql extension

This changelog references the ext/mysql extension.

Global ext/mysql changes

The following is a list of changes to the entire ext/mysql extension.

Version	Description
7.0.0	This extension was removed from PHP. For details, see Choosing an API.

Changes to existing functions

The following list is a compilation of changelog entries from the ext/mysql functions.

Version	Function	Description

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

It is possible to specify additional client flags for the mysql_connect() and mysql_pconnect() functions. The following constants are defined:

MySQL client constants

Constant	Description
`MYSQL_CLIENT_COMPRESS`	Use compression protocol
`MYSQL_CLIENT_IGNORE_SPACE`	Allow space after function names
`MYSQL_CLIENT_INTERACTIVE`	Allow interactive_timeout seconds (instead of **wait_timeout**) of inactivity before closing the connection.
`MYSQL_CLIENT_SSL`	Use SSL encryption. This flag is only available with version 4.x of the MySQL client library or newer. Version 3.23.x is bundled both with PHP 4 and Windows binaries of PHP 5.

The function mysql_fetch_array() uses a constant for the different types of result arrays. The following constants are defined:

MySQL fetch constants

Constant	Description
`MYSQL_ASSOC`	Columns are returned into the array having the fieldname as the array index.
`MYSQL_BOTH`	Columns are returned into the array having both a numerical index and the fieldname as the array index.
`MYSQL_NUM`	Columns are returned into the array having a numerical index to the fields. This index starts with 0, the first field in the result.

Examples

Table of Contents

MySQL extension overview example

This simple example shows how to connect, execute a query, print resulting rows and disconnect from a MySQL database.

Example #1 MySQL extension overview example

```
<?php
// Connecting, selecting database
$link = mysql_connect('mysql_host', 'mysql_user', 'mysql_password')
    or die('Could not connect: ' . mysql_error());
echo 'Connected successfully';
```

```php
mysql_select_db('my_database') or die('Could not select database');

// Performing SQL query
$query = 'SELECT * FROM my_table';
$result = mysql_query($query) or die('Query failed: ' . mysql_error());

// Printing results in HTML
echo "<table>\n";
while ($line = mysql_fetch_array($result, MYSQL_ASSOC)) {
    echo "\t<tr>\n";
    foreach ($line as $col_value) {
        echo "\t\t<td>$col_value</td>\n";
    }
    echo "\t</tr>\n";
}
echo "</table>\n";

// Free resultset
mysql_free_result($result);

// Closing connection
mysql_close($link);
?>
```

MySQL Functions

Notes

> **Note** :
>
> Most MySQL functions accept `link_identifier` as the last optional parameter. If it is not provided, last opened connection is used. If it doesn't exist, connection is tried to establish with default parameters defined in *php.ini*. If it is not successful, functions return `false`.

Table of Contents

mysql_affected_rows

mysql_affected_rows

(PHP 4, PHP 5)

mysql_affected_rows — Get number of affected rows in previous MySQL operation

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_affected_rows()
> - PDOStatement::rowCount()

Description

function **mysql_affected_rows**(resource `$link_identifier` = NULL): int

Get the number of affected rows by the last INSERT, UPDATE, REPLACE or DELETE query associated with `link_identifier`.

Parameters

`link_identifier`

> The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **E_WARNING** level error is generated.

Return Values

Returns the number of affected rows on success, and -1 if the last query failed.

If the last query was a DELETE query with no WHERE clause, all of the records will have been deleted from the table but this function will return zero with MySQL versions prior to 4.1.2.

When using UPDATE, MySQL will not update columns where the new value is the same as the old value. This creates the possibility that **mysql_affected_rows()** may not actually equal the number of rows matched, only the number of rows that were literally affected by the query.

The REPLACE statement first deletes the record with the same primary key and then inserts the new record. This function returns the number of deleted records plus the number of inserted records.

In the case of "INSERT ... ON DUPLICATE KEY UPDATE" queries, the return value will be `1` if an insert was performed, or `2` for an update of an existing row.

Examples

Example #1 mysql_affected_rows() example

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
mysql_select_db('mydb');

/* this should return the correct numbers of deleted records */
mysql_query('DELETE FROM mytable WHERE id < 10');
printf("Records deleted: %d\n", mysql_affected_rows());

/* with a where clause that is never true, it should return 0 */
mysql_query('DELETE FROM mytable WHERE 0');
printf("Records deleted: %d\n", mysql_affected_rows());
?>
```

The above example will output something similar to:

```
Records deleted: 10
Records deleted: 0
```

Example #2 mysql_affected_rows() example using transactions

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
mysql_select_db('mydb');

/* Update records */
mysql_query("UPDATE mytable SET used=1 WHERE id < 10");
printf ("Updated records: %d\n", mysql_affected_rows());
mysql_query("COMMIT");
?>
```

The above example will output something similar to:

```
Updated Records: 10
```

Notes

> **Note** : **Transactions**
>
> If you are using transactions, you need to call **mysql_affected_rows()** after your INSERT, UPDATE, or DELETE query, not after the COMMIT.

> **Note** : **SELECT Statements**
>
> To retrieve the number of rows returned by a SELECT, it is possible to use mysql_num_rows().

> **Note** : **Cascaded Foreign Keys**
>
> **mysql_affected_rows()** does not count rows affected implicitly through the use of ON DELETE CASCADE and/or ON UPDATE CASCADE in foreign key constraints.

See Also

- mysql_num_rows() - Get number of rows in result
- mysql_info() - Get information about the most recent query

mysql_client_encoding

mysql_client_encoding

(PHP 4 >= 4.3.0, PHP 5)

mysql_client_encoding — Returns the name of the character set

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_character_set_name()

Description

function **mysql_client_encoding**(resource `$link_identifier` = NULL): string

Retrieves the `character_set` variable from MySQL.

Parameters

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an `E_WARNING` level error is generated.

Return Values

Returns the default character set name for the current connection.

Examples

Example #1 mysql_client_encoding() example

```php
<?php
$link    = mysql_connect('localhost', 'mysql_user', 'mysql_password');
$charset = mysql_client_encoding($link);

echo "The current character set is: $charset\n";
?>
```

The above example will output something similar to:

```
The current character set is: latin1
```

See Also

- mysql_set_charset() - Sets the client character set
- mysql_real_escape_string() - Escapes special characters in a string for use in an SQL statement

mysql_close

mysql_close

(PHP 4, PHP 5)

mysql_close — Close MySQL connection

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_close()
> - PDO: Assign the value of `null` to the PDO object

Description

function **mysql_close**(resource `$link_identifier` = NULL): bool

mysql_close() closes the non-persistent connection to the MySQL server that's associated with the specified link identifier. If `link_identifier` isn't specified, the last opened link is used.

Open non-persistent MySQL connections and result sets are automatically destroyed when a PHP script finishes its execution. So, while explicitly closing open connections and freeing result sets is optional, doing so is recommended. This will immediately return resources to PHP and MySQL, which can improve performance. For related information, see freeing resources

Parameters

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no connection is found or established, an `E_WARNING` level error is generated.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 mysql_close() example

```
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
echo 'Connected successfully';
mysql_close($link);
?>
```

The above example will output:

```
```

```
Connected successfully
```

Notes

> **Note** :
>
> **mysql_close()** will not close persistent links created by mysql_pconnect(). For additional details, see the manual page on persistent connections.

See Also

- mysql_connect() - Open a connection to a MySQL Server
- mysql_free_result() - Free result memory

mysql_connect

mysql_connect

(PHP 4, PHP 5)

mysql_connect — Open a connection to a MySQL Server

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_connect()
> - PDO::__construct()

Description

```
function mysql_connect(
    string $server = ini_get("mysql.default_host"),
    string $username = ini_get("mysql.default_user"),
    string $password = ini_get("mysql.default_password"),
    bool $new_link = false,
    int $client_flags = 0
): resource|false
```

Opens or reuses a connection to a MySQL server.

Parameters

`server`

The MySQL server. It can also include a port number. e.g. "hostname:port" or a path to a local socket e.g. ":/path/to/socket" for the localhost.

If the PHP directive mysql.default_host is undefined (default), then the default value is 'localhost:3306'. In SQL safe mode, this parameter is ignored and value 'localhost:3306' is always used.

`username`

The username. Default value is defined by mysql.default_user. In SQL safe mode, this parameter is ignored and the name of the user that owns the server process is used.

`password`

The password. Default value is defined by mysql.default_password. In SQL safe mode, this parameter is ignored and empty password is used.

`new_link`

If a second call is made to **mysql_connect()** with the same arguments, no new link will be established, but instead, the link identifier of the already opened link will be returned. The `new_link` parameter modifies this behavior and makes **mysql_connect()** always open a new link, even if **mysql_connect()** was called before with the same parameters. In SQL safe mode, this parameter is ignored.

`client_flags`

The `client_flags` parameter can be a combination of the following constants: 128 (enable `LOAD DATA LOCAL` handling), **`MYSQL_CLIENT_SSL`**, **`MYSQL_CLIENT_COMPRESS`**, **`MYSQL_CLIENT_IGNORE_SPACE`** or **`MYSQL_CLIENT_INTERACTIVE`**. Read the section about MySQL client constants for further information. In SQL safe mode, this parameter is ignored.

Return Values

Returns a MySQL link identifier on success or **`false`** on failure.

Examples

Example #1 **mysql_connect()** example

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
echo 'Connected successfully';
mysql_close($link);
?>
```

Example #2 **mysql_connect()** example using `hostname:port` syntax

```php
<?php
// we connect to example.com and port 3307
$link = mysql_connect('example.com:3307', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
echo 'Connected successfully';
mysql_close($link);

// we connect to localhost at port 3307
$link = mysql_connect('127.0.0.1:3307', 'mysql_user', 'mysql_password');
```

```php
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
echo 'Connected successfully';
mysql_close($link);
?>
```

Example #3 mysql_connect() example using ":/path/to/socket" syntax

```php
<?php
// we connect to localhost and socket e.g. /tmp/mysql.sock

// variant 1: omit localhost
$link = mysql_connect(':/tmp/mysql', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
echo 'Connected successfully';
mysql_close($link);


// variant 2: with localhost
$link = mysql_connect('localhost:/tmp/mysql.sock', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
echo 'Connected successfully';
mysql_close($link);
?>
```

Notes

> **Note** :
>
> Whenever you specify "localhost" or "localhost:port" as server, the MySQL client library will override this and try to connect to a local socket (named pipe on Windows). If you want to use TCP/IP, use "127.0.0.1" instead of "localhost". If the MySQL client library tries to connect to the wrong local socket, the correct path should be set as mysql.default_host in *php.ini* and the server field left blank.

> **Note** :
>
> The link to the server will be closed as soon as the execution of the script ends, unless it's closed earlier by explicitly calling mysql_close().

> **Note** :
>
> Error "Can't create TCP/IP socket (10106)" usually means that the variables_order configure directive doesn't contain character `E`. On Windows, if the environment is not copied the `SYSTEMROOT` environment variable won't be available and PHP will have problems loading Winsock.

See Also

- mysql_pconnect() - Open a persistent connection to a MySQL server
- mysql_close() - Close MySQL connection

mysql_create_db

mysql_create_db

(PHP 4, PHP 5)

mysql_create_db — Create a MySQL database

> **Warning**
>
> This function was deprecated in PHP 4.3.0, and it and the entire original MySQL extension was removed in PHP 7.0.0. Instead, use either the actively developed MySQLi or PDO_MySQL extensions. See also the MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_query()
> - PDO::query()

Description

function **mysql_create_db**(string `$database_name`, resource `$link_identifier` = NULL): bool

mysql_create_db() attempts to create a new database on the server associated with the specified link identifier.

Parameters

`database_name`

The name of the database being created.

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **E_WARNING** level error is generated.

Return Values

Returns **true** on success or **false** on failure.

Examples

Example #1 mysql_create_db() alternative example

The function **mysql_create_db()** is deprecated. It is preferable to use mysql_query() to issue an sql CREATE DATABASE statement instead.

```
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}

$sql = 'CREATE DATABASE my_db';
if (mysql_query($sql, $link)) {
```

```
    echo "Database my_db created successfully\n";
} else {
    echo 'Error creating database: ' . mysql_error() . "\n";
}
?>
```

The above example will output something similar to:

```
Database my_db created successfully
```

Notes

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_createdb()**

> **Note** :
>
> This function will not be available if the MySQL extension was built against a MySQL 4.x client library.

See Also

- mysql_query() - Send a MySQL query
- mysql_select_db() - Select a MySQL database

mysql_data_seek

mysql_data_seek

(PHP 4, PHP 5)

mysql_data_seek — Move internal result pointer

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_data_seek()
> - **PDO::FETCH_ORI_ABS**

Description

function **mysql_data_seek**(resource `$result`, int `$row_number`): bool

mysql_data_seek() moves the internal row pointer of the MySQL result associated with the specified result identifier to point to the specified row number. The next call to a MySQL fetch function, such as mysql_fetch_assoc(), would return that row.

`row_number` starts at 0. The `row_number` should be a value in the range from 0 to mysql_num_rows() - 1. However if the result set is empty (mysql_num_rows() == 0), a seek to 0 will fail with an `E_WARNING` and **mysql_data_seek()** will return `false`.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

`row_number`

The desired row number of the new result pointer.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 mysql_data_seek() example

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
$db_selected = mysql_select_db('sample_db');
if (!$db_selected) {
    die('Could not select database: ' . mysql_error());
}
$query = 'SELECT last_name, first_name FROM friends';
$result = mysql_query($query);
if (!$result) {
    die('Query failed: ' . mysql_error());
}
/* fetch rows in reverse order */
for ($i = mysql_num_rows($result) - 1; $i >= 0; $i--) {
    if (!mysql_data_seek($result, $i)) {
        echo "Cannot seek to row $i: " . mysql_error() . "\n";
        continue;
    }

    if (!($row = mysql_fetch_assoc($result))) {
        continue;
    }

    echo $row['last_name'] . ' ' . $row['first_name'] . "<br />\n";
}

mysql_free_result($result);
?>
```

Notes

> **Note** :
>
> The function **mysql_data_seek()** can be used in conjunction only with mysql_query(), not with mysql_unbuffered_query().

See Also

- mysql_query() - Send a MySQL query
- mysql_num_rows() - Get number of rows in result
- mysql_fetch_row() - Get a result row as an enumerated array
- mysql_fetch_assoc() - Fetch a result row as an associative array
- mysql_fetch_array() - Fetch a result row as an associative array, a numeric array, or both
- mysql_fetch_object() - Fetch a result row as an object

mysql_db_name

mysql_db_name

(PHP 4, PHP 5)

mysql_db_name — Retrieves database name from the call to mysql_list_dbs()

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - Query: `SELECT DATABASE()`

Description

function **mysql_db_name**(resource `$result`, int `$row`, mixed `$field` = NULL): string

Retrieve the database name from a call to mysql_list_dbs().

Parameters

`result`

The result pointer from a call to mysql_list_dbs().

`row`

The index into the result set.

`field`

The field name.

Return Values

Returns the database name on success, and **`false`** on failure. If **`false`** is returned, use mysql_error() to determine the nature of the error.

Examples

Example #1 mysql_db_name() example

```php
<?php
error_reporting(E_ALL);

$link = mysql_connect('dbhost', 'username', 'password');
$db_list = mysql_list_dbs($link);

$i = 0;
$cnt = mysql_num_rows($db_list);
while ($i < $cnt) {
    echo mysql_db_name($db_list, $i) . "\n";
    $i++;
}
?>
```

Notes

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_dbname()**

See Also

- mysql_list_dbs() - List databases available on a MySQL server
- mysql_tablename() - Get table name of field

mysql_db_query

mysql_db_query

(PHP 4, PHP 5)

mysql_db_query — Selects a database and executes a query on it

> **Warning**
>
> This function was deprecated in PHP 5.3.0, and it and the entire original MySQL extension was removed in PHP 7.0.0. Instead, use either the actively developed MySQLi or PDO_MySQL extensions. See also the MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_select_db() then the query
> - PDO::__construct()

Description

function **mysql_db_query**(string `$database`, string `$query`, resource `$link_identifier` = NULL): resource|bool

mysql_db_query() selects a database, and executes a query on it.

Parameters

`database`

The name of the database that will be selected.

`query`

The MySQL query.

Data inside the query should be properly escaped.

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **`E_WARNING`** level error is generated.

Return Values

Returns a positive MySQL result resource to the query result, or **`false`** on error. The function also returns **`true`**/**`false`** for `INSERT`/`UPDATE`/`DELETE` queries to indicate success/failure.

Examples

Example #1 **mysql_db_query()** alternative example

```php
<?php

if (!$link = mysql_connect('mysql_host', 'mysql_user', 'mysql_password')) {
    echo 'Could not connect to mysql';
    exit;
}

if (!mysql_select_db('mysql_dbname', $link)) {
    echo 'Could not select database';
    exit;
}

$sql    = 'SELECT foo FROM bar WHERE id = 42';
$result = mysql_query($sql, $link);

if (!$result) {
    echo "DB Error, could not query the database\n";
    echo 'MySQL Error: ' . mysql_error();
    exit;
}

while ($row = mysql_fetch_assoc($result)) {
    echo $row['foo'];
}

mysql_free_result($result);

?>
```

Notes

> **Note** :

> Be aware that this function does **NOT** switch back to the database you were connected before. In other words, you can't use this function to *temporarily* run a sql query on another database, you would have to manually switch back. Users are strongly encouraged to use the `database.table` syntax in their sql queries or mysql_select_db() instead of this function.

See Also

- mysql_query() - Send a MySQL query
- mysql_select_db() - Select a MySQL database

mysql_drop_db

mysql_drop_db

(PHP 4, PHP 5)

mysql_drop_db — Drop (delete) a MySQL database

> **Warning**
>
> This function was deprecated in PHP 4.3.0, and it and the entire original MySQL extension was removed in PHP 7.0.0. Instead, use either the actively developed MySQLi or PDO_MySQL extensions. See also the MySQL: choosing an API guide. Alternatives to this function include:
>
> - Execute a `DROP DATABASE` query

Description

function **mysql_drop_db**(string `$database_name`, resource `$link_identifier` = NULL): bool

mysql_drop_db() attempts to drop (remove) an entire database from the server associated with the specified link identifier. This function is deprecated, it is preferable to use mysql_query() to issue an sql `DROP DATABASE` statement instead.

Parameters

`database_name`

The name of the database that will be deleted.

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an `E_WARNING` level error is generated.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 mysql_drop_db() alternative example

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}

$sql = 'DROP DATABASE my_db';
if (mysql_query($sql, $link)) {
    echo "Database my_db was successfully dropped\n";
} else {
    echo 'Error dropping database: ' . mysql_error() . "\n";
}
?>
```

Notes

> **Warning**
>
> This function will not be available if the MySQL extension was built against a MySQL 4.x client library.

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_dropdb()**

See Also

- mysql_query() - Send a MySQL query

mysql_errno

mysql_errno

(PHP 4, PHP 5)

mysql_errno — Returns the numerical value of the error message from previous MySQL operation

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_errno()
> - PDO::errorCode()

Description

function **mysql_errno**(resource `$link_identifier` = NULL): int

Returns the error number from the last MySQL function.

Errors coming back from the MySQL database backend no longer issue warnings. Instead, use **mysql_errno()** to retrieve the error code. Note that this function only returns the error code from the most recently executed

MySQL function (not including mysql_error() and **mysql_errno()**), so if you want to use it, make sure you check the value before calling another MySQL function.

Parameters

`link_identifier`

> The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an `E_WARNING` level error is generated.

Return Values

Returns the error number from the last MySQL function, or `0` (zero) if no error occurred.

Examples

Example #1 mysql_errno() example

```
<?php
$link = mysql_connect("localhost", "mysql_user", "mysql_password");

if (!mysql_select_db("nonexistentdb", $link)) {
    echo mysql_errno($link) . ": " . mysql_error($link). "\n";
}

mysql_select_db("kossu", $link);
if (!mysql_query("SELECT * FROM nonexistenttable", $link)) {
    echo mysql_errno($link) . ": " . mysql_error($link) . "\n";
}
?>
```

The above example will output something similar to:

```
1049: Unknown database 'nonexistentdb'
1146: Table 'kossu.nonexistenttable' doesn't exist
```

See Also

- mysql_error() - Returns the text of the error message from previous MySQL operation
- » MySQL error codes

mysql_error

mysql_error

(PHP 4, PHP 5)

mysql_error — Returns the text of the error message from previous MySQL operation

> **Warning**

> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_error()
> - PDO::errorInfo()

Description

function **mysql_error**(resource `$link_identifier` = NULL): string

Returns the error text from the last MySQL function. Errors coming back from the MySQL database backend no longer issue warnings. Instead, use **mysql_error()** to retrieve the error text. Note that this function only returns the error text from the most recently executed MySQL function (not including **mysql_error()** and mysql_errno()), so if you want to use it, make sure you check the value before calling another MySQL function.

Parameters

`link_identifier`

> The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **E_WARNING** level error is generated.

Return Values

Returns the error text from the last MySQL function, or `''` (empty string) if no error occurred.

Examples

Example #1 mysql_error() example

```php
<?php
$link = mysql_connect("localhost", "mysql_user", "mysql_password");

mysql_select_db("nonexistentdb", $link);
echo mysql_errno($link) . ": " . mysql_error($link). "\n";

mysql_select_db("kossu", $link);
mysql_query("SELECT * FROM nonexistenttable", $link);
echo mysql_errno($link) . ": " . mysql_error($link) . "\n";
?>
```

The above example will output something similar to:

```
1049: Unknown database 'nonexistentdb'
1146: Table 'kossu.nonexistenttable' doesn't exist
```

See Also

- mysql_errno() - Returns the numerical value of the error message from previous MySQL operation
- » MySQL error codes

mysql_escape_string

mysql_escape_string

(PHP 4 >= 4.0.3, PHP 5)

mysql_escape_string — Escapes a string for use in a mysql_query

> **Warning**
>
> This function was deprecated in PHP 4.3.0, and it and the entire original MySQL extension was removed in PHP 7.0.0. Instead, use either the actively developed MySQLi or PDO_MySQL extensions. See also the MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_escape_string()
> - PDO::quote()

Description

function **mysql_escape_string**(string `$unescaped_string`): string

This function will escape the `unescaped_string`, so that it is safe to place it in a mysql_query(). This function is deprecated.

This function is identical to mysql_real_escape_string() except that mysql_real_escape_string() takes a connection handler and escapes the string according to the current character set. **mysql_escape_string()** does not take a connection argument and does not respect the current charset setting.

Parameters

`unescaped_string`

The string that is to be escaped.

Return Values

Returns the escaped string.

Examples

Example #1 mysql_escape_string() example

```
<?php
$item = "Zak's Laptop";
$escaped_item = mysql_escape_string($item);
printf("Escaped string: %s\n", $escaped_item);
?>
```

The above example will output:

```
```

```
Escaped string: Zak\'s Laptop
```

Notes

> **Note** :
>
> **mysql_escape_string()** does not escape % and _.

See Also

- mysql_real_escape_string() - Escapes special characters in a string for use in an SQL statement

mysql_fetch_array

mysql_fetch_array

(PHP 4, PHP 5)

mysql_fetch_array — Fetch a result row as an associative array, a numeric array, or both

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_fetch_array()
> - PDOStatement::fetch()

Description

function **mysql_fetch_array**(resource `$result`, int `$result_type` = MYSQL_BOTH): array

Returns an array that corresponds to the fetched row and moves the internal data pointer ahead.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

`result_type`

The type of array that is to be fetched. It's a constant and can take the following values: **`MYSQL_ASSOC`**, **`MYSQL_NUM`**, and **`MYSQL_BOTH`**.

Return Values

Returns an array of strings that corresponds to the fetched row, or **`false`** if there are no more rows. The type of returned array depends on how `result_type` is defined. By using **`MYSQL_BOTH`** (default), you'll get an array with

both associative and number indices. Using **MYSQL_ASSOC**, you only get associative indices (as mysql_fetch_assoc() works), using **MYSQL_NUM**, you only get number indices (as mysql_fetch_row() works).

If two or more columns of the result have the same field names, the last column will take precedence. To access the other column(s) of the same name, you must use the numeric index of the column or make an alias for the column. For aliased columns, you cannot access the contents with the original column name.

Examples

Example #1 Query with aliased duplicate field names

```
SELECT table1.field AS foo, table2.field AS bar FROM table1, table2
```

Example #2 mysql_fetch_array() with MYSQL_NUM

```
<?php
mysql_connect("localhost", "mysql_user", "mysql_password") or
    die("Could not connect: " . mysql_error());
mysql_select_db("mydb");

$result = mysql_query("SELECT id, name FROM mytable");

while ($row = mysql_fetch_array($result, MYSQL_NUM)) {
    printf("ID: %s  Name: %s", $row[0], $row[1]);
}

mysql_free_result($result);
?>
```

Example #3 mysql_fetch_array() with MYSQL_ASSOC

```
<?php
mysql_connect("localhost", "mysql_user", "mysql_password") or
    die("Could not connect: " . mysql_error());
mysql_select_db("mydb");

$result = mysql_query("SELECT id, name FROM mytable");

while ($row = mysql_fetch_array($result, MYSQL_ASSOC)) {
    printf("ID: %s  Name: %s", $row["id"], $row["name"]);
}

mysql_free_result($result);
?>
```

Example #4 mysql_fetch_array() with MYSQL_BOTH

```
<?php
mysql_connect("localhost", "mysql_user", "mysql_password") or
    die("Could not connect: " . mysql_error());
mysql_select_db("mydb");

$result = mysql_query("SELECT id, name FROM mytable");

while ($row = mysql_fetch_array($result, MYSQL_BOTH)) {
    printf ("ID: %s  Name: %s", $row[0], $row["name"]);
```

```
}

mysql_free_result($result);
?>
```

Notes

> **Note** : **Performance**
>
> An important thing to note is that using **mysql_fetch_array()** is *not significantly* slower than using mysql_fetch_row(), while it provides a significant added value.

> **Note** :
>
> Field names returned by this function are *case-sensitive*.

> **Note** :
>
> This function sets NULL fields to the PHP `null` value.

See Also

- mysql_fetch_row() - Get a result row as an enumerated array
- mysql_fetch_assoc() - Fetch a result row as an associative array
- mysql_data_seek() - Move internal result pointer
- mysql_query() - Send a MySQL query

mysql_fetch_assoc

mysql_fetch_assoc

(PHP 4 >= 4.0.3, PHP 5)

mysql_fetch_assoc — Fetch a result row as an associative array

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_fetch_assoc()
> - PDOStatement::fetch() with `mode` as `PDO::FETCH_ASSOC`

Description

function **mysql_fetch_assoc**(resource `$result`): array

Returns an associative array that corresponds to the fetched row and moves the internal data pointer ahead. **mysql_fetch_assoc()** is equivalent to calling mysql_fetch_array() with MYSQL_ASSOC for the optional second parameter. It only returns an associative array.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

Return Values

Returns an associative array of strings that corresponds to the fetched row, or `false` if there are no more rows.

If two or more columns of the result have the same field names, the last column will take precedence. To access the other column(s) of the same name, you either need to access the result with numeric indices by using mysql_fetch_row() or add alias names. See the example at the mysql_fetch_array() description about aliases.

Examples

Example #1 An expanded mysql_fetch_assoc() example

```php
<?php

$conn = mysql_connect("localhost", "mysql_user", "mysql_password");

if (!$conn) {
    echo "Unable to connect to DB: " . mysql_error();
    exit;
}

if (!mysql_select_db("mydbname")) {
    echo "Unable to select mydbname: " . mysql_error();
    exit;
}

$sql = "SELECT id as userid, fullname, userstatus
        FROM   sometable
        WHERE  userstatus = 1";

$result = mysql_query($sql);

if (!$result) {
    echo "Could not successfully run query ($sql) from DB: " . mysql_error();
    exit;
}

if (mysql_num_rows($result) == 0) {
    echo "No rows found, nothing to print so am exiting";
    exit;
}

// While a row of data exists, put that row in $row as an associative array
// Note: If you're expecting just one row, no need to use a loop
// Note: If you put extract($row); inside the following loop, you'll
//       then create $userid, $fullname, and $userstatus
while ($row = mysql_fetch_assoc($result)) {
    echo $row["userid"];
    echo $row["fullname"];
    echo $row["userstatus"];
}
```

```
mysql_free_result($result);

?>
```

Notes

> **Note** : **Performance**
>
> An important thing to note is that using **mysql_fetch_assoc()** is *not significantly* slower than using mysql_fetch_row(), while it provides a significant added value.

> **Note** :
>
> Field names returned by this function are *case-sensitive*.

> **Note** :
>
> This function sets NULL fields to the PHP `null` value.

See Also

- mysql_fetch_row() - Get a result row as an enumerated array
- mysql_fetch_array() - Fetch a result row as an associative array, a numeric array, or both
- mysql_data_seek() - Move internal result pointer
- mysql_query() - Send a MySQL query
- mysql_error() - Returns the text of the error message from previous MySQL operation

mysql_fetch_field

mysql_fetch_field

(PHP 4, PHP 5)

mysql_fetch_field — Get column information from a result and return as an object

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_fetch_field()
> - PDOStatement::getColumnMeta()

Description

function **mysql_fetch_field**(resource `$result`, int `$field_offset` = 0): object

Returns an object containing field information. This function can be used to obtain information about fields in the provided query result.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

`field_offset`

The numerical field offset. If the field offset is not specified, the next field that was not yet retrieved by this function is retrieved. The `field_offset` starts at `0`.

Return Values

Returns an object containing field information. The properties of the object are:

- name - column name
- table - name of the table the column belongs to, which is the alias name if one is defined
- max_length - maximum length of the column
- not_null - 1 if the column cannot be `null`
- primary_key - 1 if the column is a primary key
- unique_key - 1 if the column is a unique key
- multiple_key - 1 if the column is a non-unique key
- numeric - 1 if the column is numeric
- blob - 1 if the column is a BLOB
- type - the type of the column
- unsigned - 1 if the column is unsigned
- zerofill - 1 if the column is zero-filled

Examples

Example #1 **mysql_fetch_field() example**

```
<?php
$conn = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$conn) {
    die('Could not connect: ' . mysql_error());
}
mysql_select_db('database');
$result = mysql_query('select * from table');
if (!$result) {
    die('Query failed: ' . mysql_error());
}
/* get column metadata */
$i = 0;
while ($i < mysql_num_fields($result)) {
    echo "Information for column $i:<br />\n";
    $meta = mysql_fetch_field($result, $i);
    if (!$meta) {
```

```
        echo "No information available<br />\n";
    }
    echo "<pre>
blob:         $meta->blob
max_length:   $meta->max_length
multiple_key: $meta->multiple_key
name:         $meta->name
not_null:     $meta->not_null
numeric:      $meta->numeric
primary_key:  $meta->primary_key
table:        $meta->table
type:         $meta->type
unique_key:   $meta->unique_key
unsigned:     $meta->unsigned
zerofill:     $meta->zerofill
</pre>";
    $i++;
}
mysql_free_result($result);
?>
```

Notes

> **Note** :
>
> Field names returned by this function are *case-sensitive*.

> **Note** :
>
> If field or tablenames are aliased in the SQL query the aliased name will be returned. The original name can be retrieved for instance by using mysqli_result::fetch_field().

See Also

- mysql_field_seek() - Set result pointer to a specified field offset

mysql_fetch_lengths

mysql_fetch_lengths

(PHP 4, PHP 5)

mysql_fetch_lengths — Get the length of each output in a result

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_fetch_lengths()
> - PDOStatement::getColumnMeta()

Description

function **mysql_fetch_lengths**(resource `$result`): array|false

Returns an array that corresponds to the lengths of each field in the last row fetched by MySQL.

mysql_fetch_lengths() stores the lengths of each result column in the last row returned by mysql_fetch_row(), mysql_fetch_assoc(), mysql_fetch_array(), and mysql_fetch_object() in an array, starting at offset 0.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

Return Values

An array of lengths on success or `false` on failure.

Examples

Example #1 A mysql_fetch_lengths() example

```php
<?php
$result = mysql_query("SELECT id,email FROM people WHERE id = '42'");
if (!$result) {
    echo 'Could not run query: ' . mysql_error();
    exit;
}
$row     = mysql_fetch_assoc($result);
$lengths = mysql_fetch_lengths($result);

print_r($row);
print_r($lengths);
?>
```

The above example will output something similar to:

```
Array
(
    [id] => 42
    [email] => user@example.com
)
Array
(
    [0] => 2
    [1] => 16
)
```

See Also

- mysql_field_len() - Returns the length of the specified field

- mysql_fetch_row() - Get a result row as an enumerated array
- strlen() - Get string length

mysql_fetch_object

mysql_fetch_object

(PHP 4, PHP 5)

mysql_fetch_object — Fetch a result row as an object

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_fetch_object()
> - PDOStatement::fetch() with `mode` as **`PDO::FETCH_OBJ`**

Description

function **mysql_fetch_object**(resource `$result`, string `$class_name = ?`, array `$params = ?`): object

Returns an object with properties that correspond to the fetched row and moves the internal data pointer ahead.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

`class_name`

The name of the class to instantiate, set the properties of and return. If not specified, a stdClass object is returned.

`params`

An optional array of parameters to pass to the constructor for `class_name` objects.

Return Values

Returns an object with string properties that correspond to the fetched row, or **`false`** if there are no more rows.

Examples

Example #1 **mysql_fetch_object()** example

```
<?php
mysql_connect("hostname", "user", "password");
mysql_select_db("mydb");
$result = mysql_query("select * from mytable");
while ($row = mysql_fetch_object($result)) {
    echo $row->user_id;
    echo $row->fullname;
}
```

```
mysql_free_result($result);
?>
```

Example #2 mysql_fetch_object() example

```
<?php
class foo {
    public $name;
}

mysql_connect("hostname", "user", "password");
mysql_select_db("mydb");

$result = mysql_query("select name from mytable limit 1");
$obj = mysql_fetch_object($result, 'foo');
var_dump($obj);
?>
```

Notes

> **Note** : **Performance**
>
> Speed-wise, the function is identical to mysql_fetch_array(), and almost as quick as mysql_fetch_row() (the difference is insignificant).

> **Note** :
>
> **mysql_fetch_object()** is similar to mysql_fetch_array(), with one difference - an object is returned, instead of an array. Indirectly, that means that you can only access the data by the field names, and not by their offsets (numbers are illegal property names).

> **Note** :
>
> Field names returned by this function are *case-sensitive*.

> **Note** :
>
> This function sets NULL fields to the PHP `null` value.

See Also

- mysql_fetch_array() - Fetch a result row as an associative array, a numeric array, or both
- mysql_fetch_assoc() - Fetch a result row as an associative array
- mysql_fetch_row() - Get a result row as an enumerated array
- mysql_data_seek() - Move internal result pointer
- mysql_query() - Send a MySQL query

mysql_fetch_row

mysql_fetch_row

(PHP 4, PHP 5)

mysql_fetch_row — Get a result row as an enumerated array

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_fetch_row()
> - PDOStatement::fetch() with `mode` as **`PDO::FETCH_NUM`**

Description

function **mysql_fetch_row**(resource `$result`): array

Returns a numerical array that corresponds to the fetched row and moves the internal data pointer ahead.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

Return Values

Returns an numerical array of strings that corresponds to the fetched row, or **`false`** if there are no more rows.

mysql_fetch_row() fetches one row of data from the result associated with the specified result identifier. The row is returned as an array. Each result column is stored in an array offset, starting at offset 0.

Examples

Example #1 Fetching one row with mysql_fetch_row()

```
<?php
$result = mysql_query("SELECT id,email FROM people WHERE id = '42'");
if (!$result) {
    echo 'Could not run query: ' . mysql_error();
    exit;
}
$row = mysql_fetch_row($result);

echo $row[0]; // 42
echo $row[1]; // the email value
?>
```

Notes

> **Note** :
>
> This function sets NULL fields to the PHP **`null`** value.

See Also

- mysql_fetch_array() - Fetch a result row as an associative array, a numeric array, or both
- mysql_fetch_assoc() - Fetch a result row as an associative array
- mysql_fetch_object() - Fetch a result row as an object
- mysql_data_seek() - Move internal result pointer
- mysql_fetch_lengths() - Get the length of each output in a result
- mysql_result() - Get result data

mysql_field_flags

mysql_field_flags

(PHP 4, PHP 5)

mysql_field_flags — Get the flags associated with the specified field in a result

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_fetch_field_direct() [flags]
> - PDOStatement::getColumnMeta() [flags]

Description

function **mysql_field_flags**(resource `$result`, int `$field_offset`): string|false

mysql_field_flags() returns the field flags of the specified field. The flags are reported as a single word per flag separated by a single space, so that you can split the returned value using explode().

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

`field_offset`

The numerical field offset. The `field_offset` starts at `0`. If `field_offset` does not exist, an error of level `E_WARNING` is also issued.

Return Values

Returns a string of flags associated with the result or `false` on failure.

The following flags are reported, if your version of MySQL is current enough to support them: `"not_null"`, `"primary_key"`, `"unique_key"`, `"multiple_key"`, `"blob"`, `"unsigned"`, `"zerofill"`, `"binary"`, `"enum"`, `"auto_increment"` and `"timestamp"`.

Examples

Example #1 A mysql_field_flags() example

```
<?php
$result = mysql_query("SELECT id,email FROM people WHERE id = '42'");
```

```php
if (!$result) {
    echo 'Could not run query: ' . mysql_error();
    exit;
}
$flags = mysql_field_flags($result, 0);

echo $flags;
print_r(explode(' ', $flags));
?>
```

The above example will output something similar to:

```
not_null primary_key auto_increment
Array
(
    [0] => not_null
    [1] => primary_key
    [2] => auto_increment
)
```

Notes

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_fieldflags()**

See Also

- mysql_field_type() - Get the type of the specified field in a result
- mysql_field_len() - Returns the length of the specified field

mysql_field_len

mysql_field_len

(PHP 4, PHP 5)

mysql_field_len — Returns the length of the specified field

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_fetch_field_direct() [length]
> - PDOStatement::getColumnMeta() [len]

Description

function **mysql_field_len**(resource `$result`, int `$field_offset`): int|false

mysql_field_len() returns the length of the specified field.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

`field_offset`

The numerical field offset. The `field_offset` starts at `0`. If `field_offset` does not exist, an error of level `E_WARNING` is also issued.

Return Values

The length of the specified field index on success or `false` on failure.

Examples

Example #1 mysql_field_len() example

```php
<?php
$result = mysql_query("SELECT id,email FROM people WHERE id = '42'");
if (!$result) {
    echo 'Could not run query: ' . mysql_error();
    exit;
}

// Will get the length of the id field as specified in the database
// schema.
$length = mysql_field_len($result, 0);
echo $length;
?>
```

Notes

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_fieldlen()**

See Also

- mysql_fetch_lengths() - Get the length of each output in a result
- strlen() - Get string length

mysql_field_name

mysql_field_name

(PHP 4, PHP 5)

mysql_field_name — Get the name of the specified field in a result

> **Warning**

> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_fetch_field_direct() [name] or [orgname]
> - PDOStatement::getColumnMeta() [name]

Description

function **mysql_field_name**(resource `$result`, int `$field_offset`): string|false

mysql_field_name() returns the name of the specified field index.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

`field_offset`

The numerical field offset. The `field_offset` starts at `0`. If `field_offset` does not exist, an error of level `E_WARNING` is also issued.

Return Values

The name of the specified field index on success or `false` on failure.

Examples

Example #1 mysql_field_name() example

```php
<?php
/* The users table consists of three fields:
 *   user_id
 *   username
 *   password.
 */
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect to MySQL server: ' . mysql_error());
}
$dbname = 'mydb';
$db_selected = mysql_select_db($dbname, $link);
if (!$db_selected) {
    die("Could not set $dbname: " . mysql_error());
}
$res = mysql_query('select * from users', $link);

echo mysql_field_name($res, 0) . "\n";
echo mysql_field_name($res, 2);
?>
```

The above example will output:

```
user_id
password
```

Notes

> **Note** :
>
> Field names returned by this function are *case-sensitive*.

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_fieldname()**

See Also

- mysql_field_type() - Get the type of the specified field in a result
- mysql_field_len() - Returns the length of the specified field

mysql_field_seek

mysql_field_seek

(PHP 4, PHP 5)

mysql_field_seek — Set result pointer to a specified field offset

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_field_seek()
> - PDOStatement::fetch() using the `cursor_orientation` and `offset` parameters

Description

function **mysql_field_seek**(resource `$result`, int `$field_offset`): bool

Seeks to the specified field offset. If the next call to mysql_fetch_field() doesn't include a field offset, the field offset specified in **mysql_field_seek()** will be returned.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

`field_offset`

The numerical field offset. The `field_offset` starts at `0`. If `field_offset` does not exist, an error of level `E_WARNING` is also issued.

Return Values

Returns `true` on success or `false` on failure.

See Also

- mysql_fetch_field() - Get column information from a result and return as an object

mysql_field_table

mysql_field_table

(PHP 4, PHP 5)

mysql_field_table — Get name of the table the specified field is in

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_fetch_field_direct() [table] or [orgtable]
> - PDOStatement::getColumnMeta() [table]

Description

function **mysql_field_table**(resource `$result`, int `$field_offset`): string

Returns the name of the table that the specified field is in.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

`field_offset`

The numerical field offset. The `field_offset` starts at `0`. If `field_offset` does not exist, an error of level `E_WARNING` is also issued.

Return Values

The name of the table on success.

Examples

Example #1 A **mysql_field_table()** example

```
<?php

$query = "SELECT account.*, country.* FROM account, country WHERE country.name = 'Portugal' AND
account.country_id = country.id";
```

```php
// get the result from the DB
$result = mysql_query($query);

// Lists the table name and then the field name
for ($i = 0; $i < mysql_num_fields($result); ++$i) {
    $table = mysql_field_table($result, $i);
    $field = mysql_field_name($result, $i);

    echo  "$table: $field\n";
}

?>
```

Notes

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_fieldtable()**

See Also

- mysql_list_tables() - List tables in a MySQL database

mysql_field_type

mysql_field_type

(PHP 4, PHP 5)

mysql_field_type — Get the type of the specified field in a result

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_fetch_field_direct() [type]
> - PDOStatement::getColumnMeta() [driver:decl_type] or [pdo_type]

Description

function **mysql_field_type**(resource `$result`, int `$field_offset`): string

mysql_field_type() is similar to the mysql_field_name() function. The arguments are identical, but the field type is returned instead.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

`field_offset`

The numerical field offset. The `field_offset` starts at `0`. If `field_offset` does not exist, an error of level `E_WARNING` is also issued.

Return Values

The returned field type will be one of `"int"`, `"real"`, `"string"`, `"blob"`, and others as detailed in the » MySQL documentation.

Examples

Example #1 **mysql_field_type()** example

```
<?php
mysql_connect("localhost", "mysql_username", "mysql_password");
mysql_select_db("mysql");
$result = mysql_query("SELECT * FROM func");
$fields = mysql_num_fields($result);
$rows   = mysql_num_rows($result);
$table  = mysql_field_table($result, 0);
echo "Your '" . $table . "' table has " . $fields . " fields and " . $rows . " record(s)\n";
echo "The table has the following fields:\n";
for ($i=0; $i < $fields; $i++) {
    $type  = mysql_field_type($result, $i);
    $name  = mysql_field_name($result, $i);
    $len   = mysql_field_len($result, $i);
    $flags = mysql_field_flags($result, $i);
    echo $type . " " . $name . " " . $len . " " . $flags . "\n";
}
mysql_free_result($result);
mysql_close();
?>
```

The above example will output something similar to:

```
Your 'func' table has 4 fields and 1 record(s)
The table has the following fields:
string name 64 not_null primary_key binary
int ret 1 not_null
string dl 128 not_null
string type 9 not_null enum
```

Notes

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_fieldtype()**

See Also

- mysql_field_name() - Get the name of the specified field in a result

- mysql_field_len() - Returns the length of the specified field

mysql_free_result

mysql_free_result

(PHP 4, PHP 5)

mysql_free_result — Free result memory

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_free_result()
> - Assign the value of `null` to the PDO object, or PDOStatement::closeCursor()

Description

function **mysql_free_result**(resource `$result`): bool

mysql_free_result() will free all memory associated with the result identifier `result`.

mysql_free_result() only needs to be called if you are concerned about how much memory is being used for queries that return large result sets. All associated result memory is automatically freed at the end of the script's execution.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

Return Values

Returns `true` on success or `false` on failure.

If a non-resource is used for the `result`, an error of level E_WARNING will be emitted. It's worth noting that mysql_query() only returns a resource for SELECT, SHOW, EXPLAIN, and DESCRIBE queries.

Examples

Example #1 A **mysql_free_result()** example

```
<?php
$result = mysql_query("SELECT id,email FROM people WHERE id = '42'");
if (!$result) {
    echo 'Could not run query: ' . mysql_error();
    exit;
}
/* Use the result, assuming we're done with it afterwards */
$row = mysql_fetch_assoc($result);

/* Now we free up the result and continue on with our script */
mysql_free_result($result);
```

```
echo $row['id'];
echo $row['email'];
?>
```

Notes

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_freeresult()**

See Also

- mysql_query() - Send a MySQL query
- is_resource() - Finds whether a variable is a resource

mysql_get_client_info

mysql_get_client_info

(PHP 4 >= 4.0.5, PHP 5)

mysql_get_client_info — Get MySQL client info

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_get_client_info()
> - PDO::getAttribute() with `attribute` as **`PDO::ATTR_CLIENT_VERSION`**

Description

function **mysql_get_client_info**(): string

mysql_get_client_info() returns a string that represents the client library version.

Parameters

This function has no parameters.

Return Values

The MySQL client version.

Examples

Example #1 mysql_get_client_info() example

```
<?php
printf("MySQL client info: %s\n", mysql_get_client_info());
?>
```

The above example will output something similar to:

```
MySQL client info: 3.23.39
```

See Also

- mysql_get_host_info() - Get MySQL host info
- mysql_get_proto_info() - Get MySQL protocol info
- mysql_get_server_info() - Get MySQL server info

mysql_get_host_info

mysql_get_host_info

(PHP 4 >= 4.0.5, PHP 5)

mysql_get_host_info — Get MySQL host info

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_get_host_info()
> - PDO::getAttribute() with `attribute` as **`PDO::ATTR_CONNECTION_STATUS`**

Description

function **mysql_get_host_info**(resource `$link_identifier` = NULL): string|false

Describes the type of connection in use for the connection, including the server host name.

Parameters

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **`E_WARNING`** level error is generated.

Return Values

Returns a string describing the type of MySQL connection in use for the connection or **`false`** on failure.

Examples

Example #1 mysql_get_host_info() example

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
printf("MySQL host info: %s\n", mysql_get_host_info());
?>
```

The above example will output something similar to:

```
MySQL host info: Localhost via UNIX socket
```

See Also

- mysql_get_client_info() - Get MySQL client info
- mysql_get_proto_info() - Get MySQL protocol info
- mysql_get_server_info() - Get MySQL server info

mysql_get_proto_info

mysql_get_proto_info

(PHP 4 >= 4.0.5, PHP 5)

mysql_get_proto_info — Get MySQL protocol info

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_get_proto_info()

Description

function **mysql_get_proto_info**(resource `$link_identifier` = NULL): int|false

Retrieves the MySQL protocol.

Parameters

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **E_WARNING** level error is generated.

Return Values

Returns the MySQL protocol on success or `false` on failure.

Examples

Example #1 **mysql_get_proto_info()** example

```
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
printf("MySQL protocol version: %s\n", mysql_get_proto_info());
?>
```

The above example will output something similar to:

```
MySQL protocol version: 10
```

See Also

- mysql_get_client_info() - Get MySQL client info
- mysql_get_host_info() - Get MySQL host info
- mysql_get_server_info() - Get MySQL server info

mysql_get_server_info

mysql_get_server_info

(PHP 4 >= 4.0.5, PHP 5)

mysql_get_server_info — Get MySQL server info

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_get_server_info()
> - PDO::getAttribute() with `attribute` as **`PDO::ATTR_SERVER_VERSION`**

Description

function **mysql_get_server_info**(resource `$link_identifier` = NULL): string|false

Retrieves the MySQL server version.

Parameters

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an `E_WARNING` level error is generated.

Return Values

Returns the MySQL server version on success or `false` on failure.

Examples

Example #1 mysql_get_server_info() example

```
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
printf("MySQL server version: %s\n", mysql_get_server_info());
?>
```

The above example will output something similar to:

```
MySQL server version: 4.0.1-alpha
```

See Also

- mysql_get_client_info() - Get MySQL client info
- mysql_get_host_info() - Get MySQL host info
- mysql_get_proto_info() - Get MySQL protocol info
- phpversion() - Gets the current PHP version

mysql_info

mysql_info

(PHP 4 >= 4.3.0, PHP 5)

mysql_info — Get information about the most recent query

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_info()

Description

function **mysql_info**(resource `$link_identifier` = NULL): string

Returns detailed information about the last query.

Parameters

`link_identifier`

> The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an `E_WARNING` level error is generated.

Return Values

Returns information about the statement on success, or `false` on failure. See the example below for which statements provide information, and what the returned value may look like. Statements that are not listed will return `false`.

Examples

Example #1 Relevant MySQL Statements

Statements that return string values. The numbers are only for illustrating purpose; their values will correspond to the query.

```
INSERT INTO ... SELECT ...
String format: Records: 23 Duplicates: 0 Warnings: 0
INSERT INTO ... VALUES (...),(...),(...)...
String format: Records: 37 Duplicates: 0 Warnings: 0
LOAD DATA INFILE ...
String format: Records: 42 Deleted: 0 Skipped: 0 Warnings: 0
ALTER TABLE
String format: Records: 60 Duplicates: 0 Warnings: 0
UPDATE
String format: Rows matched: 65 Changed: 65 Warnings: 0
```

Notes

> **Note** :
>
> **mysql_info()** returns a non-`false` value for the INSERT ... VALUES statement only if multiple value lists are specified in the statement.

See Also

- mysql_affected_rows() - Get number of affected rows in previous MySQL operation
- mysql_insert_id() - Get the ID generated in the last query
- mysql_stat() - Get current system status

mysql_insert_id

mysql_insert_id

(PHP 4, PHP 5)

mysql_insert_id — Get the ID generated in the last query

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_insert_id()
> - PDO::lastInsertId()

Description

function **mysql_insert_id**(resource `$link_identifier` = NULL): int

Retrieves the ID generated for an AUTO_INCREMENT column by the previous query (usually INSERT).

Parameters

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **`E_WARNING`** level error is generated.

Return Values

The ID generated for an AUTO_INCREMENT column by the previous query on success, `0` if the previous query does not generate an AUTO_INCREMENT value, or **`false`** if no MySQL connection was established.

Examples

Example #1 mysql_insert_id() example

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
mysql_select_db('mydb');

mysql_query("INSERT INTO mytable (product) values ('kossu')");
printf("Last inserted record has id %d\n", mysql_insert_id());
?>
```

Notes

> **Caution**
>
> **mysql_insert_id()** will convert the return type of the native MySQL C API function `mysql_insert_id()` to a type of `long` (named int in PHP). If your AUTO_INCREMENT column has a column type of BIGINT (64 bits) the conversion may result in an incorrect value. Instead, use the internal MySQL SQL function LAST_INSERT_ID() in an SQL query. For more information about PHP's maximum integer values, please see the integer documentation.

> **Note** :

> Because **mysql_insert_id()** acts on the last performed query, be sure to call **mysql_insert_id()** immediately after the query that generates the value.

> **Note** :
>
> The value of the MySQL SQL function `LAST_INSERT_ID()` always contains the most recently generated AUTO_INCREMENT value, and is not reset between queries.

See Also

- mysql_query() - Send a MySQL query
- mysql_info() - Get information about the most recent query

mysql_list_dbs

mysql_list_dbs

(PHP 4, PHP 5)

mysql_list_dbs — List databases available on a MySQL server

> **Warning**
>
> This function was deprecated in PHP 5.4.0, and it and the entire original MySQL extension was removed in PHP 7.0.0. Instead, use either the actively developed MySQLi or PDO_MySQL extensions. See also the MySQL: choosing an API guide. Alternatives to this function include:
>
> - SQL Query: `SHOW DATABASES`

Description

function **mysql_list_dbs**(resource `$link_identifier` = NULL): resource

Returns a result pointer containing the databases available from the current mysql daemon.

Parameters

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **`E_WARNING`** level error is generated.

Return Values

Returns a result pointer resource on success, or **`false`** on failure. Use the mysql_tablename() function to traverse this result pointer, or any function for result tables, such as mysql_fetch_array().

Examples

Example #1 mysql_list_dbs() example

```
<?php
// Usage without mysql_list_dbs()
```

```php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
$res = mysql_query("SHOW DATABASES");

while ($row = mysql_fetch_assoc($res)) {
    echo $row['Database'] . "\n";
}

// Deprecated as of PHP 5.4.0
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
$db_list = mysql_list_dbs($link);

while ($row = mysql_fetch_object($db_list)) {
     echo $row->Database . "\n";
}
?>
```

The above example will output something similar to:

```
database1
database2
database3
```

Notes

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_listdbs()**

See Also

- mysql_db_name() - Retrieves database name from the call to mysql_list_dbs
- mysql_select_db() - Select a MySQL database

mysql_list_fields

mysql_list_fields

(PHP 4, PHP 5)

mysql_list_fields — List MySQL table fields

> **Warning**
>
> This function was deprecated in PHP 5.4.0, and it and the entire original MySQL extension was removed in PHP 7.0.0. Instead, use either the actively developed MySQLi or PDO_MySQL extensions. See also the MySQL: choosing an API guide. Alternatives to this function include:
>
> - SQL Query: `SHOW COLUMNS FROM sometable`

Description

function **mysql_list_fields**(string `$database_name`, string `$table_name`, resource `$link_identifier` = NULL): resource

Retrieves information about the given table name.

This function is deprecated. It is preferable to use mysql_query() to issue an SQL `SHOW COLUMNS FROM table [LIKE 'name']` statement instead.

Parameters

`database_name`

The name of the database that's being queried.

`table_name`

The name of the table that's being queried.

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **`E_WARNING`** level error is generated.

Return Values

A result pointer resource on success, or **`false`** on failure.

The returned result can be used with mysql_field_flags(), mysql_field_len(), mysql_field_name() and mysql_field_type().

Examples

Example #1 Alternate to deprecated mysql_list_fields()

```php
<?php
$result = mysql_query("SHOW COLUMNS FROM sometable");
if (!$result) {
    echo 'Could not run query: ' . mysql_error();
    exit;
}
if (mysql_num_rows($result) > 0) {
    while ($row = mysql_fetch_assoc($result)) {
        print_r($row);
    }
}
?>
```

The above example will output something similar to:

```
Array
(
    [Field] => id
    [Type] => int(7)
    [Null] =>
```

```
    [Key] => PRI
    [Default] =>
    [Extra] => auto_increment
)
Array
(
    [Field] => email
    [Type] => varchar(100)
    [Null] =>
    [Key] =>
    [Default] =>
    [Extra] =>
)
```

Notes

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_listfields()**

See Also

- mysql_field_flags() - Get the flags associated with the specified field in a result
- mysql_info() - Get information about the most recent query

mysql_list_processes

mysql_list_processes

(PHP 4 >= 4.3.0, PHP 5)

mysql_list_processes — List MySQL processes

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_thread_id()

Description

function **mysql_list_processes**(resource `$link_identifier` = NULL): resource|false

Retrieves the current MySQL server threads.

Parameters

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an `E_WARNING` level error is generated.

Return Values

A result pointer resource on success or `false` on failure.

Examples

Example #1 mysql_list_processes() example

```
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');

$result = mysql_list_processes($link);
while ($row = mysql_fetch_assoc($result)){
    printf("%s %s %s %s %s\n", $row["Id"], $row["Host"], $row["db"],
        $row["Command"], $row["Time"]);
}
mysql_free_result($result);
?>
```

The above example will output something similar to:

```
1 localhost test Processlist 0
4 localhost mysql sleep 5
```

See Also

- mysql_thread_id() - Return the current thread ID
- mysql_stat() - Get current system status

mysql_list_tables

mysql_list_tables

(PHP 4, PHP 5)

mysql_list_tables — List tables in a MySQL database

> **Warning**
>
> This function was deprecated in PHP 4.3.0, and it and the entire original MySQL extension was removed in PHP 7.0.0. Instead, use either the actively developed MySQLi or PDO_MySQL extensions. See also the MySQL: choosing an API guide. Alternatives to this function include:
>
> - SQL Query: `SHOW TABLES FROM dbname`

Description

function **mysql_list_tables**(string `$database`, resource `$link_identifier` = NULL): resource|false

Retrieves a list of table names from a MySQL database.

This function is deprecated. It is preferable to use mysql_query() to issue an SQL `SHOW TABLES [FROM db_name] [LIKE 'pattern']` statement instead.

Parameters

`database`

The name of the database

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **`E_WARNING`** level error is generated.

Return Values

A result pointer resource on success or **`false`** on failure.

Use the mysql_tablename() function to traverse this result pointer, or any function for result tables, such as mysql_fetch_array().

Examples

Example #1 mysql_list_tables() alternative example

```php
<?php
$dbname = 'mysql_dbname';

if (!mysql_connect('mysql_host', 'mysql_user', 'mysql_password')) {
    echo 'Could not connect to mysql';
    exit;
}

$sql = "SHOW TABLES FROM $dbname";
$result = mysql_query($sql);

if (!$result) {
    echo "DB Error, could not list tables\n";
    echo 'MySQL Error: ' . mysql_error();
    exit;
}

while ($row = mysql_fetch_row($result)) {
    echo "Table: {$row[0]}\n";
}

mysql_free_result($result);
?>
```

Notes

> **Note** :

> For backward compatibility, the following deprecated alias may be used: **mysql_listtables()**

See Also

- mysql_list_dbs() - List databases available on a MySQL server
- mysql_tablename() - Get table name of field

mysql_num_fields

mysql_num_fields

(PHP 4, PHP 5)

mysql_num_fields — Get number of fields in result

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_num_fields()
> - PDOStatement::columnCount()

Description

function **mysql_num_fields**(resource `$result`): int|false

Retrieves the number of fields from a query.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

Return Values

Returns the number of fields in the result set resource on success or **`false`** on failure.

Examples

Example #1 A mysql_num_fields() example

```
<?php
$result = mysql_query("SELECT id,email FROM people WHERE id = '42'");
if (!$result) {
    echo 'Could not run query: ' . mysql_error();
    exit;
}

/* returns 2 because id,email === two fields */
echo mysql_num_fields($result);
?>
```

Notes

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_numfields()**

See Also

- mysql_select_db() - Select a MySQL database
- mysql_query() - Send a MySQL query
- mysql_fetch_field() - Get column information from a result and return as an object
- mysql_num_rows() - Get number of rows in result

mysql_num_rows

mysql_num_rows

(PHP 4, PHP 5)

mysql_num_rows — Get number of rows in result

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_num_rows()
> - mysqli_stmt_num_rows()
> - PDOStatement::rowCount()

Description

function **mysql_num_rows**(resource `$result`): int|false

Retrieves the number of rows from a result set. This command is only valid for statements like SELECT or SHOW that return an actual result set. To retrieve the number of rows affected by a INSERT, UPDATE, REPLACE or DELETE query, use mysql_affected_rows().

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

Return Values

The number of rows in a result set on success or `false` on failure.

Examples

Example #1 mysql_num_rows() example

```
<?php

$link = mysql_connect("localhost", "mysql_user", "mysql_password");
```

```php
mysql_select_db("database", $link);

$result = mysql_query("SELECT * FROM table1", $link);
$num_rows = mysql_num_rows($result);

echo "$num_rows Rows\n";

?>
```

Notes

> **Note** :
>
> If you use mysql_unbuffered_query(), **mysql_num_rows()** will not return the correct value until all the rows in the result set have been retrieved.

> **Note** :
>
> For backward compatibility, the following deprecated alias may be used: **mysql_numrows()**

See Also

- mysql_affected_rows() - Get number of affected rows in previous MySQL operation
- mysql_connect() - Open a connection to a MySQL Server
- mysql_data_seek() - Move internal result pointer
- mysql_select_db() - Select a MySQL database
- mysql_query() - Send a MySQL query

mysql_pconnect

mysql_pconnect

(PHP 4, PHP 5)

mysql_pconnect — Open a persistent connection to a MySQL server

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_connect() with `p:` host prefix
> - PDO::__construct() with **`PDO::ATTR_PERSISTENT`** as a driver option

Description

function **mysql_pconnect**(
  string `$server` = ini_get("mysql.default_host"),
  string `$username` = ini_get("mysql.default_user"),
  string `$password` = ini_get("mysql.default_password"),

```
    int $client_flags = 0
): resource
```

Establishes a persistent connection to a MySQL server.

mysql_pconnect() acts very much like mysql_connect() with two major differences.

First, when connecting, the function would first try to find a (persistent) link that's already open with the same host, username and password. If one is found, an identifier for it will be returned instead of opening a new connection.

Second, the connection to the SQL server will not be closed when the execution of the script ends. Instead, the link will remain open for future use (mysql_close() will not close links established by **mysql_pconnect()**).

This type of link is therefore called 'persistent'.

Parameters

`server`

The MySQL server. It can also include a port number. e.g. "hostname:port" or a path to a local socket e.g. ":/path/to/socket" for the localhost.

If the PHP directive mysql.default_host is undefined (default), then the default value is 'localhost:3306'

`username`

The username. Default value is the name of the user that owns the server process.

`password`

The password. Default value is an empty password.

`client_flags`

The `client_flags` parameter can be a combination of the following constants: 128 (enable `LOAD DATA LOCAL` handling), **`MYSQL_CLIENT_SSL`**, **`MYSQL_CLIENT_COMPRESS`**, **`MYSQL_CLIENT_IGNORE_SPACE`** or **`MYSQL_CLIENT_INTERACTIVE`**.

Return Values

Returns a MySQL persistent link identifier on success, or **`false`** on failure.

Notes

> **Note** :
>
> Note, that these kind of links only work if you are using a module version of PHP. See the Persistent Database Connections section for more information.

> **Warning**
>
> Using persistent connections can require a bit of tuning of your Apache and MySQL configurations to ensure that you do not exceed the number of connections allowed by MySQL.

See Also

- mysql_connect() - Open a connection to a MySQL Server
- Persistent Database Connections

mysql_ping

mysql_ping

(PHP 4 >= 4.3.0, PHP 5)

mysql_ping — Ping a server connection or reconnect if there is no connection

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_ping()

Description

function **mysql_ping**(resource `$link_identifier` = NULL): bool

Checks whether or not the connection to the server is working. If it has gone down, an automatic reconnection is attempted. This function can be used by scripts that remain idle for a long while, to check whether or not the server has closed the connection and reconnect if necessary.

> **Note** :
>
> Automatic reconnection is disabled by default in versions of MySQL >= 5.0.3.

Parameters

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **`E_WARNING`** level error is generated.

Return Values

Returns **`true`** if the connection to the server MySQL server is working, otherwise **`false`**.

Examples

Example #1 A mysql_ping() example

```
<?php
set_time_limit(0);

$conn = mysql_connect('localhost', 'mysqluser', 'mypass');
$db   = mysql_select_db('mydb');
```

```php
/* Assuming this query will take a long time */
$result = mysql_query($sql);
if (!$result) {
    echo 'Query #1 failed, exiting.';
    exit;
}

/* Make sure the connection is still alive, if not, try to reconnect */
if (!mysql_ping($conn)) {
    echo 'Lost connection, exiting after query #1';
    exit;
}
mysql_free_result($result);

/* So the connection is still alive, let's run another query */
$result2 = mysql_query($sql2);
?>
```

See Also

- mysql_thread_id() - Return the current thread ID
- mysql_list_processes() - List MySQL processes

mysql_query

mysql_query

(PHP 4, PHP 5)

mysql_query — Send a MySQL query

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_query()
> - PDO::query()

Description

function **mysql_query**(string `$query`, resource `$link_identifier` = NULL): mixed

mysql_query() sends a unique query (multiple queries are not supported) to the currently active database on the server that's associated with the specified `link_identifier`.

Parameters

`query`

An SQL query

The query string should not end with a semicolon. Data inside the query should be properly escaped.

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an `E_WARNING` level error is generated.

Return Values

For SELECT, SHOW, DESCRIBE, EXPLAIN and other statements returning resultset, **mysql_query()** returns a resource on success, or `false` on error.

For other type of SQL statements, INSERT, UPDATE, DELETE, DROP, etc, **mysql_query()** returns `true` on success or `false` on error.

The returned result resource should be passed to mysql_fetch_array(), and other functions for dealing with result tables, to access the returned data.

Use mysql_num_rows() to find out how many rows were returned for a SELECT statement or mysql_affected_rows() to find out how many rows were affected by a DELETE, INSERT, REPLACE, or UPDATE statement.

mysql_query() will also fail and return `false` if the user does not have permission to access the table(s) referenced by the query.

Examples

Example #1 Invalid Query

The following query is syntactically invalid, so **mysql_query()** fails and returns `false`.

```
<?php
$result = mysql_query('SELECT * WHERE 1=1');
if (!$result) {
    die('Invalid query: ' . mysql_error());
}

?>
```

Example #2 Valid Query

The following query is valid, so **mysql_query()** returns a resource.

```
<?php
// This could be supplied by a user, for example
$firstname = 'fred';
$lastname  = 'fox';

// Formulate Query
// This is the best way to perform an SQL query
// For more examples, see mysql_real_escape_string()
$query = sprintf("SELECT firstname, lastname, address, age FROM friends
    WHERE firstname='%s' AND lastname='%s'",
    mysql_real_escape_string($firstname),
    mysql_real_escape_string($lastname));

// Perform Query
$result = mysql_query($query);

// Check result
```

```php
// This shows the actual query sent to MySQL, and the error. Useful for debugging.
if (!$result) {
    $message  = 'Invalid query: ' . mysql_error() . "\n";
    $message .= 'Whole query: ' . $query;
    die($message);
}

// Use result
// Attempting to print $result won't allow access to information in the resource
// One of the mysql result functions must be used
// See also mysql_result(), mysql_fetch_array(), mysql_fetch_row(), etc.
while ($row = mysql_fetch_assoc($result)) {
    echo $row['firstname'];
    echo $row['lastname'];
    echo $row['address'];
    echo $row['age'];
}

// Free the resources associated with the result set
// This is done automatically at the end of the script
mysql_free_result($result);
?>
```

See Also

- mysql_connect() - Open a connection to a MySQL Server
- mysql_error() - Returns the text of the error message from previous MySQL operation
- mysql_real_escape_string() - Escapes special characters in a string for use in an SQL statement
- mysql_result() - Get result data
- mysql_fetch_assoc() - Fetch a result row as an associative array
- mysql_unbuffered_query() - Send an SQL query to MySQL without fetching and buffering the result rows

mysql_real_escape_string

mysql_real_escape_string

(PHP 4 >= 4.3.0, PHP 5)

mysql_real_escape_string — Escapes special characters in a string for use in an SQL statement

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_real_escape_string()
> - PDO::quote()

Description

function **mysql_real_escape_string**(string `$unescaped_string`, resource `$link_identifier` = NULL): string

Escapes special characters in the `unescaped_string`, taking into account the current character set of the connection so that it is safe to place it in a mysql_query(). If binary data is to be inserted, this function must be used.

mysql_real_escape_string() calls MySQL's library function mysql_real_escape_string, which prepends backslashes to the following characters: `\x00`, `\n`, `\r`, `\`, `'`, `"` and `\x1a`.

This function must always (with few exceptions) be used to make data safe before sending a query to MySQL.

> **Caution**
>
> **Security: the default character set**
>
> The character set must be set either at the server level, or with the API function mysql_set_charset() for it to affect **mysql_real_escape_string()**. See the concepts section on character sets for more information.

Parameters

`unescaped_string`

The string that is to be escaped.

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an `E_WARNING` level error is generated.

Return Values

Returns the escaped string, or `false` on error.

Errors/Exceptions

Executing this function without a MySQL connection present will also emit `E_WARNING` level PHP errors. Only execute this function with a valid MySQL connection present.

Examples

Example #1 Simple **mysql_real_escape_string()** example

```php
<?php
// Connect
$link = mysql_connect('mysql_host', 'mysql_user', 'mysql_password')
    OR die(mysql_error());

// Query
$query = sprintf("SELECT * FROM users WHERE user='%s' AND password='%s'",
            mysql_real_escape_string($user),
            mysql_real_escape_string($password));
?>
```

Example #2 **mysql_real_escape_string()** requires a connection example

This example demonstrates what happens if a MySQL connection is not present when calling this function.

```php
<?php
// We have not connected to MySQL

$lastname  = "O'Reilly";
```

```php
$_lastname = mysql_real_escape_string($lastname);

$query = "SELECT * FROM actors WHERE last_name = '$_lastname'";

var_dump($_lastname);
var_dump($query);
?>
```

The above example will output something similar to:

```
Warning: mysql_real_escape_string(): No such file or directory in /this/test/script.php on line 5
Warning: mysql_real_escape_string(): A link to the server could not be established in
/this/test/script.php on line 5

bool(false)
string(41) "SELECT * FROM actors WHERE last_name = '"
```

Example #3 An example SQL Injection Attack

```php
<?php
// We didn't check $_POST['password'], it could be anything the user wanted! For example:
$_POST['username'] = 'aidan';
$_POST['password'] = "' OR ''='";

// Query database to check if there are any matching users
$query = "SELECT * FROM users WHERE user='{$_POST['username']}' AND password='{$_POST['password']}'";
mysql_query($query);

// This means the query sent to MySQL would be:
echo $query;
?>
```

The query sent to MySQL:

```
SELECT * FROM users WHERE user='aidan' AND password='' OR ''=''
```

This would allow anyone to log in without a valid password.

Notes

> **Note** :

> A MySQL connection is required before using **mysql_real_escape_string()** otherwise an error of level `E_WARNING` is generated, and `false` is returned. If `link_identifier` isn't defined, the last MySQL connection is used.

> **Note** :
>
> If this function is not used to escape data, the query is vulnerable to SQL Injection Attacks.

> **Note** :
>
> **mysql_real_escape_string()** does not escape `%` and _. These are wildcards in MySQL if combined with `LIKE`, `GRANT`, or `REVOKE`.

See Also

- mysql_set_charset() - Sets the client character set
- mysql_client_encoding() - Returns the name of the character set

mysql_result

mysql_result

(PHP 4, PHP 5)

mysql_result — Get result data

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_data_seek() in conjunction with mysqli_field_seek() and mysqli_fetch_field()
> - PDOStatement::fetchColumn()

Description

function **mysql_result**(resource `$result`, int `$row`, mixed `$field` = 0): string

Retrieves the contents of one cell from a MySQL result set.

When working on large result sets, you should consider using one of the functions that fetch an entire row (specified below). As these functions return the contents of multiple cells in one function call, they're MUCH quicker than **mysql_result()**. Also, note that specifying a numeric offset for the field argument is much quicker than specifying a fieldname or tablename.fieldname argument.

Parameters

`result`

The result resource that is being evaluated. This result comes from a call to mysql_query().

`row`

The row number from the result that's being retrieved. Row numbers start at `0`.

`field`

The name or offset of the field being retrieved.

It can be the field's offset, the field's name, or the field's table dot field name (tablename.fieldname). If the column name has been aliased ('select foo as bar from...'), use the alias instead of the column name. If undefined, the first field is retrieved.

Return Values

The contents of one cell from a MySQL result set on success, or `false` on failure.

Examples

Example #1 **mysql_result()** example

```php
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Could not connect: ' . mysql_error());
}
if (!mysql_select_db('database_name')) {
    die('Could not select database: ' . mysql_error());
}
$result = mysql_query('SELECT name FROM work.employee');
if (!$result) {
    die('Could not query:' . mysql_error());
}
echo mysql_result($result, 2); // outputs third employee's name

mysql_close($link);
?>
```

Notes

> **Note** :
>
> Calls to **mysql_result()** should not be mixed with calls to other functions that deal with the result set.

See Also

- mysql_fetch_row() - Get a result row as an enumerated array
- mysql_fetch_array() - Fetch a result row as an associative array, a numeric array, or both
- mysql_fetch_assoc() - Fetch a result row as an associative array
- mysql_fetch_object() - Fetch a result row as an object

mysql_select_db

mysql_select_db

(PHP 4, PHP 5)

mysql_select_db — Select a MySQL database

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_select_db()
> - PDO::__construct() (part of dsn)

Description

function **mysql_select_db**(string `$database_name`, resource `$link_identifier` = NULL): bool

Sets the current active database on the server that's associated with the specified link identifier. Every subsequent call to mysql_query() will be made on the active database.

Parameters

`database_name`

The name of the database that is to be selected.

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **`E_WARNING`** level error is generated.

Return Values

Returns **`true`** on success or **`false`** on failure.

Examples

Example #1 mysql_select_db() example

```php
<?php

$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
if (!$link) {
    die('Not connected : ' . mysql_error());
}

// make foo the current db
$db_selected = mysql_select_db('foo', $link);
if (!$db_selected) {
    die ('Can\'t use foo : ' . mysql_error());
}
?>
```

Notes

> **Note** :

> For backward compatibility, the following deprecated alias may be used: **mysql_selectdb()**

See Also

- mysql_connect() - Open a connection to a MySQL Server
- mysql_pconnect() - Open a persistent connection to a MySQL server
- mysql_query() - Send a MySQL query

mysql_set_charset

mysql_set_charset

(PHP 5 >= 5.2.3)

mysql_set_charset — Sets the client character set

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_set_charset()
> - PDO: Add `charset` to the connection string, such as `charset=utf8`

Description

function **mysql_set_charset**(string `$charset`, resource `$link_identifier` = NULL): bool

Sets the default character set for the current connection.

Parameters

`charset`

A valid character set name.

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **`E_WARNING`** level error is generated.

Return Values

Returns **`true`** on success or **`false`** on failure.

Notes

> **Note** :
>
> This function requires MySQL 5.0.7 or later.

> **Note** :
>
> This is the preferred way to change the charset. Using mysql_query() to set it (such as `SET NAMES utf8`) is not recommended. See the MySQL character set concepts section for more information.

See Also

- Setting character sets in MySQL
- » List of character sets that MySQL supports
- mysql_client_encoding() - Returns the name of the character set

mysql_stat

mysql_stat

(PHP 4 >= 4.3.0, PHP 5)

mysql_stat — Get current system status

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_stat()
> - PDO::getAttribute() with `attribute` as **`PDO::ATTR_SERVER_INFO`**

Description

function **mysql_stat**(resource `$link_identifier` = NULL): string

mysql_stat() returns the current server status.

Parameters

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **`E_WARNING`** level error is generated.

Return Values

Returns a string with the status for uptime, threads, queries, open tables, flush tables and queries per second. For a complete list of other status variables, you have to use the `SHOW STATUS` SQL command. If `link_identifier` is invalid, **`null`** is returned.

Examples

Example #1 mysql_stat() example

```
<?php
$link   = mysql_connect('localhost', 'mysql_user', 'mysql_password');
```

```php
$status = explode('  ', mysql_stat($link));
print_r($status);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Uptime: 5380
    [1] => Threads: 2
    [2] => Questions: 1321299
    [3] => Slow queries: 0
    [4] => Opens: 26
    [5] => Flush tables: 1
    [6] => Open tables: 17
    [7] => Queries per second avg: 245.595
)
```

Example #2 Alternative mysql_stat() example

```php
<?php
$link   = mysql_connect('localhost', 'mysql_user', 'mysql_password');
$result = mysql_query('SHOW STATUS', $link);
while ($row = mysql_fetch_assoc($result)) {
    echo $row['Variable_name'] . ' = ' . $row['Value'] . "\n";
}
?>
```

The above example will output something similar to:

```
back_log = 50
basedir = /usr/local/
bdb_cache_size = 8388600
bdb_log_buffer_size = 32768
bdb_home = /var/db/mysql/
bdb_max_lock = 10000
bdb_logdir =
bdb_shared_data = OFF
bdb_tmpdir = /var/tmp/
...
```

See Also

- mysql_get_server_info() - Get MySQL server info
- mysql_list_processes() - List MySQL processes

mysql_tablename

mysql_tablename

(PHP 4, PHP 5)

mysql_tablename — Get table name of field

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - SQL Query: `SHOW TABLES`

Description

function **mysql_tablename**(resource `$result`, int `$i`): string|false

Retrieves the table name from a `result`.

This function is deprecated. It is preferable to use mysql_query() to issue an SQL `SHOW TABLES [FROM db_name] [LIKE 'pattern']` statement instead.

Parameters

`result`

A result pointer resource that's returned from mysql_list_tables().

`i`

The integer index (row/table number)

Return Values

The name of the table on success or `false` on failure.

Use the **mysql_tablename()** function to traverse this result pointer, or any function for result tables, such as mysql_fetch_array().

Examples

Example #1 mysql_tablename() example

```php
<?php
mysql_connect("localhost", "mysql_user", "mysql_password");
$result = mysql_list_tables("mydb");
$num_rows = mysql_num_rows($result);
for ($i = 0; $i < $num_rows; $i++) {
    echo "Table: ", mysql_tablename($result, $i), "\n";
}

mysql_free_result($result);
?>
```

Notes

> **Note** :
>
> The mysql_num_rows() function may be used to determine the number of tables in the result pointer.

See Also

- mysql_list_tables() - List tables in a MySQL database
- mysql_field_table() - Get name of the table the specified field is in
- mysql_db_name() - Retrieves database name from the call to mysql_list_dbs

mysql_thread_id

mysql_thread_id

(PHP 4 >= 4.3.0, PHP 5)

mysql_thread_id — Return the current thread ID

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - mysqli_thread_id()

Description

function **mysql_thread_id**(resource `$link_identifier` = NULL): int|false

Retrieves the current thread ID. If the connection is lost, and a reconnect with mysql_ping() is executed, the thread ID will change. This means only retrieve the thread ID when needed.

Parameters

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an **`E_WARNING`** level error is generated.

Return Values

The thread ID on success or **`false`** on failure.

Examples

Example #1 mysql_thread_id() example

```
<?php
$link = mysql_connect('localhost', 'mysql_user', 'mysql_password');
$thread_id = mysql_thread_id($link);
if ($thread_id){
```

```
    printf("current thread id is %d\n", $thread_id);
}
?>
```

The above example will output something similar to:

```
current thread id is 73
```

See Also

- mysql_ping() - Ping a server connection or reconnect if there is no connection
- mysql_list_processes() - List MySQL processes

mysql_unbuffered_query

mysql_unbuffered_query

(PHP 4 >= 4.0.6, PHP 5)

mysql_unbuffered_query — Send an SQL query to MySQL without fetching and buffering the result rows

> **Warning**
>
> This extension was deprecated in PHP 5.5.0, and it was removed in PHP 7.0.0. Instead, the MySQLi or PDO_MySQL extension should be used. See also MySQL: choosing an API guide. Alternatives to this function include:
>
> - See: Buffered and Unbuffered queries

Description

function **mysql_unbuffered_query**(string `$query`, resource `$link_identifier` = NULL): resource

mysql_unbuffered_query() sends the SQL query `query` to MySQL without automatically fetching and buffering the result rows as mysql_query() does. This saves a considerable amount of memory with SQL queries that produce large result sets, and you can start working on the result set immediately after the first row has been retrieved as you don't have to wait until the complete SQL query has been performed. To use **mysql_unbuffered_query()** while multiple database connections are open, you must specify the optional parameter `link_identifier` to identify which connection you want to use.

Parameters

`query`

The SQL query to execute.

Data inside the query should be properly escaped.

`link_identifier`

The MySQL connection. If the link identifier is not specified, the last link opened by mysql_connect() is assumed. If no such link is found, it will try to create one as if mysql_connect() had been called with no arguments. If no connection is found or established, an `E_WARNING` level error is generated.

Return Values

For SELECT, SHOW, DESCRIBE or EXPLAIN statements, **mysql_unbuffered_query()** returns a resource on success, or `false` on error.

For other type of SQL statements, UPDATE, DELETE, DROP, etc, **mysql_unbuffered_query()** returns `true` on success or `false` on error.

Notes

> **Note** :
>
> The benefits of **mysql_unbuffered_query()** come at a cost: you cannot use mysql_num_rows() and mysql_data_seek() on a result set returned from **mysql_unbuffered_query()**, until all rows are fetched. You also have to fetch all result rows from an unbuffered SQL query before you can send a new SQL query to MySQL, using the same `link_identifier`.

See Also

- mysql_query() - Send a MySQL query

Mysqlnd

Introduction

MySQL Native Driver is a replacement for the MySQL Client Library (libmysqlclient). MySQL Native Driver is part of the official PHP sources as of PHP 5.3.0.

The MySQL database extensions MySQL extension, `mysqli` and PDO MYSQL all communicate with the MySQL server. In the past, this was done by the extension using the services provided by the MySQL Client Library. The extensions were compiled against the MySQL Client Library in order to use its client-server protocol.

With MySQL Native Driver there is now an alternative, as the MySQL database extensions can be compiled to use MySQL Native Driver instead of the MySQL Client Library.

MySQL Native Driver is written in C as a PHP extension.

Overview

What it is not

Although MySQL Native Driver is written as a PHP extension, it is important to note that it does not provide a new API to the PHP programmer. The programmer APIs for MySQL database connectivity are provided by the MySQL extension, `mysqli` and PDO MYSQL. These extensions can now use the services of MySQL Native Driver to communicate with the MySQL Server. Therefore, you should not think of MySQL Native Driver as an API.

Why use it?

Using the MySQL Native Driver offers a number of advantages over using the MySQL Client Library.

The older MySQL Client Library was written by MySQL AB (now Oracle Corporation) and so was released under the MySQL license. This ultimately led to MySQL support being disabled by default in PHP. However, the MySQL Native Driver has been developed as part of the PHP project, and is therefore released under the PHP license. This removes licensing issues that have been problematic in the past.

Also, in the past, you needed to build the MySQL database extensions against a copy of the MySQL Client Library. This typically meant you needed to have MySQL installed on a machine where you were building the PHP source code. Also, when your PHP application was running, the MySQL database extensions would call down to the MySQL Client library file at run time, so the file needed to be installed on your system. With MySQL Native Driver that is no longer the case as it is included as part of the standard distribution. So you do not need MySQL installed in order to build PHP or run PHP database applications.

Because MySQL Native Driver is written as a PHP extension, it is tightly coupled to the workings of PHP. This leads to gains in efficiency, especially when it comes to memory usage, as the driver uses the PHP memory management system. It also supports the PHP memory limit. Using MySQL Native Driver leads to comparable or better performance than using MySQL Client Library, it always ensures the most efficient use of memory. One example of the memory efficiency is the fact that when using the MySQL Client Library, each row is stored in memory twice, whereas with the MySQL Native Driver each row is only stored once in memory.

> **Note** : **Reporting memory usage**
>
> Because MySQL Native Driver uses the PHP memory management system, its memory usage can be tracked with memory_get_usage(). This is not possible with libmysqlclient because it uses the C function malloc() instead.

Special features

MySQL Native Driver also provides some special features not available when the MySQL database extensions use MySQL Client Library. These special features are listed below:

- Improved persistent connections
- The special function mysqli_fetch_all()
- Performance statistics calls: mysqli_get_client_stats(), mysqli_get_connection_stats()

The performance statistics facility can prove to be very useful in identifying performance bottlenecks.

MySQL Native Driver also allows for persistent connections when used with the `mysqli` extension.

SSL Support

MySQL Native Driver supports SSL.

Compressed Protocol Support

MySQL Native Driver supports the compressed client server protocol. Extension `ext/mysqli`, if configured to use MySQL Native Driver, can also take advantage of this feature. Note that `PDO_MYSQL` does *NOT* support compression when used together with mysqlnd.

Named Pipes Support

Named pipes can be used to connect on Windows environments.

Installation

Installation on Unix

In order to use the MySQL Native Driver, PHP needs to be built specifying that the MySQL database extensions are compiled with MySQL Native Driver support. This is done through configuration options prior to building the PHP source code.

For example, to build the MySQL extension, `mysqli` and PDO MYSQL using the MySQL Native Driver, the following command would be given:

```
./configure --with-mysql=mysqlnd \
--with-mysqli=mysqlnd \
--with-pdo-mysql=mysqlnd \
[other options]
```

Installation on Windows

In the official PHP Windows distributions, MySQL Native Driver is enabled by default, so no additional configuration is required to use it. All MySQL database extensions will use MySQL Native Driver in this case.

SHA-256 Authentication Plugin support

The MySQL Native Driver requires the OpenSSL functionality of PHP to be loaded and enabled to connect to MySQL through accounts that use the MySQL SHA-256 Authentication Plugin. For example, PHP could be configured using:

```
./configure --with-mysql=mysqlnd \
--with-mysqli=mysqlnd \
--with-pdo-mysql=mysqlnd \
--with-openssl
[other options]
```

In Autotools, the extended SSL support in `mysqlnd` is enabled implicitly when building together with the `openssl` extension using the **--with-openssl** configure option. When building without the `openssl` extension, the **--with-mysqlnd-ssl** configure option can be used to enable extended SSL support explicitly.

Changelog

Version	Description
8.4.0	The **--with-mysqlnd-ssl** Autotools configure option was added to enable the extended SSL support explicitly when building without the `openssl` extension.

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

MySQL Native Driver Configuration Options

Name	Default	Changeable	Changelog
mysqlnd.collect_statistics	"1"	`INI_SYSTEM`	
mysqlnd.collect_memory_statistics	"0"	`INI_SYSTEM`	
mysqlnd.debug	""	`INI_SYSTEM`	

Name	Default	Changeable	Changelog
mysqlnd.log_mask	0	INI_ALL	
mysqlnd.mempool_default_size	16000	INI_ALL	
mysqlnd.net_read_timeout	"86400"	INI_ALL	Before PHP 7.2.0 the default value was "31536000" and the changeability was INI_SYSTEM
mysqlnd.net_cmd_buffer_size	"4096"	INI_SYSTEM	
mysqlnd.net_read_buffer_size	"32768"	INI_SYSTEM	
mysqlnd.sha256_server_public_key	""	INI_PERDIR	
mysqlnd.trace_alloc	""	INI_SYSTEM	
mysqlnd.fetch_data_copy	0	INI_ALL	Removed as of PHP 8.1.0

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`mysqlnd.collect_statistics` bool

Enables the collection of various client statistics which can be accessed through mysqli_get_client_stats(), mysqli_get_connection_stats(), and are shown in `mysqlnd` section of the output of the phpinfo() function as well.

This configuration setting enables all MySQL Native Driver statistics except those relating to memory management.

`mysqlnd.collect_memory_statistics` bool

Enable the collection of various memory statistics which can be accessed through mysqli_get_client_stats(), mysqli_get_connection_stats(), and are shown in `mysqlnd` section of the output of the phpinfo() function as well.

This configuration setting enables the memory management statistics within the overall set of MySQL Native Driver statistics.

`mysqlnd.debug` string

Records communication from all extensions using `mysqlnd` to the specified log file.

The format of the directive is `mysqlnd.debug = "option1[,parameter_option1][:option2[,parameter_option2]]"`.

The options for the format string are as follows:

- A[,file] - Appends trace output to specified file. Also ensures that data is written after each write. This is done by closing and reopening the trace file (this is slow). It helps ensure a complete log file should the application crash.
- a[,file] - Appends trace output to the specified file.
- d - Enables output from DBUG_<N> macros for the current state. May be followed by a list of keywords which selects output only for the DBUG macros with that keyword. An empty list of keywords implies output for all macros.

- f[,functions] - Limits debugger actions to the specified list of functions. An empty list of functions implies that all functions are selected.
- F - Marks each debugger output line with the name of the source file containing the macro causing the output.
- i - Marks each debugger output line with the PID of the current process.
- L - Marks each debugger output line with the name of the source file line number of the macro causing the output.
- n - Marks each debugger output line with the current function nesting depth
- o[,file] - Similar to a[,file] but overwrites old file, and does not append.
- O[,file] - Similar to A[,file] but overwrites old file, and does not append.
- t[,N] - Enables function control flow tracing. The maximum nesting depth is specified by N, and defaults to 200.
- x - This option activates profiling.
- m - Trace memory allocation and deallocation related calls.

Example:

```
d:t:x:O,/tmp/mysqlnd.trace
```

> **Note** :
>
> This feature is only available with a debug build of PHP.

`mysqlnd.log_mask` int

Defines which queries will be logged. The default 0, which disables logging. Define using an integer, and not with PHP constants. For example, a value of 48 (16 + 32) will log slow queries which either use 'no good index' (SERVER_QUERY_NO_GOOD_INDEX_USED = 16) or no index at all (SERVER_QUERY_NO_INDEX_USED = 32). A value of 2043 (1 + 2 + 8 + ... + 1024) will log all slow query types.

The types are as follows: SERVER_STATUS_IN_TRANS=1, SERVER_STATUS_AUTOCOMMIT=2, SERVER_MORE_RESULTS_EXISTS=8, SERVER_QUERY_NO_GOOD_INDEX_USED=16, SERVER_QUERY_NO_INDEX_USED=32, SERVER_STATUS_CURSOR_EXISTS=64, SERVER_STATUS_LAST_ROW_SENT=128, SERVER_STATUS_DB_DROPPED=256, SERVER_STATUS_NO_BACKSLASH_ESCAPES=512, and SERVER_QUERY_WAS_SLOW=1024.

`mysqlnd.mempool_default_size` int

Default size of the mysqlnd memory pool, which is used by result sets.

`mysqlnd.net_read_timeout` int

`mysqlnd` and the MySQL Client Library, `libmysqlclient` use different networking APIs. `mysqlnd` uses PHP streams, whereas `libmysqlclient` uses its own wrapper around the operating level network calls. PHP, by default, sets a read timeout of 60s for streams. This is set via *php.ini*, `default_socket_timeout`. This default

applies to all streams that set no other timeout value. `mysqlnd` does not set any other value and therefore connections of long running queries can be disconnected after `default_socket_timeout` seconds resulting in an error message "2006 - MySQL Server has gone away". The MySQL Client Library sets a default timeout of 24 * 3600 seconds (1 day) and waits for other timeouts to occur, such as TCP/IP timeouts. `mysqlnd` now uses the same very long timeout. The value is configurable through a new *php.ini* setting: `mysqlnd.net_read_timeout`. `mysqlnd.net_read_timeout` gets used by any extension (`ext/mysql`, `ext/mysqli`, `PDO_MySQL`) that uses `mysqlnd`. `mysqlnd` tells PHP Streams to use `mysqlnd.net_read_timeout`. Please note that there may be subtle differences between `MYSQL_OPT_READ_TIMEOUT` from the MySQL Client Library and PHP Streams, for example `MYSQL_OPT_READ_TIMEOUT` is documented to work only for TCP/IP connections and, prior to MySQL 5.1.2, only for Windows. PHP streams may not have this limitation. Please check the streams documentation, if in doubt.

`mysqlnd.net_cmd_buffer_size` int

`mysqlnd` allocates an internal command/network buffer of `mysqlnd.net_cmd_buffer_size` (in *php.ini*) bytes for every connection. If a MySQL Client Server protocol command, for example, `COM_QUERY` ("normal" query), does not fit into the buffer, `mysqlnd` will grow the buffer to the size required for sending the command. Whenever the buffer gets extended for one connection, `command_buffer_too_small` will be incremented by one.

If `mysqlnd` has to grow the buffer beyond its initial size of `mysqlnd.net_cmd_buffer_size` bytes for almost every connection, you should consider increasing the default size to avoid re-allocations.

The default buffer size is 4096 bytes, which is the smallest value possible.

The value can also be set using `mysqli_options(link, MYSQLI_OPT_NET_CMD_BUFFER_SIZE, size)`.

`mysqlnd.net_read_buffer_size` int

Maximum read chunk size in bytes when reading the body of a MySQL command packet. The MySQL client server protocol encapsulates all its commands in packets. The packets consist of a small header and a body with the actual payload. The size of the body is encoded in the header. `mysqlnd` reads the body in chunks of `MIN(header.size, mysqlnd.net_read_buffer_size)` bytes. If a packet body is larger than `mysqlnd.net_read_buffer_size` bytes, `mysqlnd` has to call `read()` multiple times.

The value can also be set using `mysqli_options(link, MYSQLI_OPT_NET_READ_BUFFER_SIZE, size)`.

`mysqlnd.sha256_server_public_key` string

SHA-256 Authentication Plugin related. File with the MySQL server public RSA key.

Clients can either omit setting a public RSA key, specify the key through this PHP configuration setting or set the key at runtime using mysqli_options(). If not public RSA key file is given by the client, then the key will be exchanged as part of the standard SHA-256 Authentication Plugin authentication procedure.

`mysqlnd.trace_alloc` string

`mysqlnd.fetch_data_copy` int

Enforce copying result sets from the internal result set buffers into PHP variables instead of using the default reference and copy-on-write logic. Please, see the memory management implementation notes for further details.

Copying result sets instead of having PHP variables reference them allows releasing the memory occupied for the PHP variables earlier. Depending on the user API code, the actual database queries and the size of

their result sets this may reduce the memory footprint of mysqlnd.

Do not set if using PDO_MySQL. PDO_MySQL has not yet been updated to support the new fetch mode.

> **Note** :
>
> Removed as of PHP 8.1.0

Incompatibilities

MySQL Native Driver is in most cases compatible with MySQL Client Library (`libmysql`). This section documents incompatibilities between these libraries.

- Values of `bit` data type are returned as binary strings (e.g. "\0" or "\x1F") with `libmysql` and as decimal strings (e.g. "0" or "31") with `mysqlnd`. If you want the code to be compatible with both libraries then always return bit fields as numbers from MySQL with a query like this: `SELECT bit + 0 FROM table`.

Persistent Connections

Using Persistent Connections

If `mysqli` is used with `mysqlnd`, when a persistent connection is created it generates a `COM_CHANGE_USER` (`mysql_change_user()`) call on the server. This ensures that re-authentication of the connection takes place.

As there is some overhead associated with the `COM_CHANGE_USER` call, it is possible to switch this off at compile time. Reusing a persistent connection will then generate a `COM_PING` (`mysql_ping`) call to simply test the connection is reusable.

Generation of `COM_CHANGE_USER` can be switched off with the compile flag `MYSQLI_NO_CHANGE_USER_ON_PCONNECT`. For example:

```
shell# CFLAGS="-DMYSQLI_NO_CHANGE_USER_ON_PCONNECT" ./configure --with-mysql=/usr/local/mysql/ --with-
mysqli=/usr/local/mysql/bin/mysql_config --with-pdo-mysql=/usr/local/mysql/bin/mysql_config --enable-
debug && make clean && make -j6
```

Or alternatively:

```
shell# export CFLAGS="-DMYSQLI_NO_CHANGE_USER_ON_PCONNECT"
shell# configure --whatever-option
shell# make clean
shell# make
```

Note that only `mysqli` on `mysqlnd` uses `COM_CHANGE_USER`. Other extension-driver combinations use `COM_PING` on initial use of a persistent connection.

Statistics

MySQL Native Driver contains support for gathering statistics on the communication between the client and the server. The statistics gathered are of two main types:

- Client statistics
- Connection statistics

When using the mysqli extension, these statistics can be obtained through two API calls:

- mysqli_get_client_stats()
- mysqli_get_connection_stats()

> **Note** :
>
> Statistics are aggregated among all extensions that use the MySQL Native Driver. For example, if the mysqli extension and the PDO MySQL driver are both set-up to use MySQLnd, then function calls from mysqli and method calls from PDO will affect the statistics.
>
> There is no way to find out how much a certain API call of any extension that has been compiled against MySQL Native Driver has impacted a certain statistic.

Retrieving statistics

Client statistics can be retrieved by calling the mysqli_get_client_stats() function.

Connection statistics can be retrieved by calling the mysqli_get_connection_stats() function.

Both functions return an associative array, where the name of a statistic is the key for the corresponding statistical data.

MySQL Native Driver Statistics

Most statistics are associated to a connection, but some are associated to the process in which case this will be mentioned.

The following statistics are produced by the MySQL Native Driver:

Network Related Statistics

`bytes_sent`

: Number of bytes sent from PHP to the MySQL server.

`bytes_received`

: Number of bytes received from the MySQL server.

`packets_sent`

: Number of packets sent by the MySQL Client Server protocol.

`packets_received`

: Number of packets received from the MySQL Client Server protocol.

`protocol_overhead_in`

: MySQL Client Server protocol overhead in bytes for incoming traffic. Currently only the Packet Header (4 bytes) is considered as overhead. `protocol_overhead_in = packets_received * 4`

`protocol_overhead_out`

MySQL Client Server protocol overhead in bytes for outgoing traffic. Currently only the Packet Header (4 bytes) is considered as overhead. `protocol_overhead_out = packets_received * 4`

`bytes_received_ok_packet`

Total size of bytes of MySQL Client Server protocol OK packets received. OK packets can contain a status message. The length of the status message can vary and thus the size of an OK packet is not fixed.

> **Note** :
>
> The total size in bytes includes the size of the header packet (4 bytes, see protocol overhead).

`packets_received_ok`

Number of MySQL Client Server protocol OK packets received.

`bytes_received_eof_packet`

Total size in bytes of MySQL Client Server protocol EOF packets received. EOF can vary in size depending on the server version. Also, EOF can transport an error message.

> **Note** :
>
> The total size in bytes includes the size of the header packet (4 bytes, see protocol overhead).

`packets_received_eof`

Number of MySQL Client Server protocol EOF packets.

Like with other packet statistics the number of packets will be increased even if PHP does not receive the expected packet but, for example, an error message.

`bytes_received_rset_header_packet`

Total size in bytes of MySQL Client Server protocol result set header packets. The size of the packets varies depending on the payload (`LOAD LOCAL INFILE`, `INSERT`, `UPDATE`, `SELECT`, error message).

> **Note** :
>
> The total size in bytes includes the size of the header packet (4 bytes, see protocol overhead).

`packets_received_rset_header`

Number of MySQL Client Server protocol result set header packets.

`bytes_received_rset_field_meta_packet`

Total size in bytes of MySQL Client Server protocol result set metadata (field information) packets. Of course the size varies with the fields in the result set. The packet may also transport an error or an EOF packet in case of COM_LIST_FIELDS.

> **Note** :

> The total size in bytes includes the size of the header packet (4 bytes, see protocol overhead).

`packets_received_rset_field_meta`

Number of MySQL Client Server protocol result set metadata (field information) packets.

`bytes_received_rset_row_packet`

Total size in bytes of MySQL Client Server protocol result set row data packets. The packet may also transport an error or an EOF packet. One can compute the number of error and EOF packets by subtracting `rows_fetched_from_server_normal` and `rows_fetched_from_server_ps` from `bytes_received_rset_row_packet`.

> **Note** :
>
> The total size in bytes includes the size of the header packet (4 bytes, see protocol overhead).

`packets_received_rset_row`

Number of MySQL Client Server protocol result set row data packets.

`bytes_received_prepare_response_packet`

Total size in bytes of MySQL Client Server protocol OK for Prepared Statement Initialization packets (prepared statement init packets). The packet may also transport an error. The packet size depends on the MySQL version.

> **Note** :
>
> The total size in bytes includes the size of the header packet (4 bytes, see protocol overhead).

`packets_received_prepare_response`

Number of MySQL Client Server protocol OK for Prepared Statement Initialization packets (prepared statement init packets).

`bytes_received_change_user_packet`

Total size in bytes of MySQL Client Server protocol COM_CHANGE_USER packets. The packet may also transport an error or EOF.

> **Note** :
>
> The total size in bytes includes the size of the header packet (4 bytes, see protocol overhead).

`packets_received_change_user`

Number of MySQL Client Server protocol COM_CHANGE_USER packets.

`packets_sent_command`

Number of MySQL Client Server protocol commands sent from PHP to MySQL.

There is no way to know which specific commands and how many of them have been sent.

`bytes_received_real_data_normal`

Number of bytes of payload fetched by the PHP client from `mysqlnd` using the text protocol.

This is the size of the actual data contained in result sets that do not originate from prepared statements and which have been fetched by the PHP client.

Note that although a full result set may have been pulled from MySQL by `mysqlnd`, this statistic only counts actual data pulled from `mysqlnd` by the PHP client.

An example of a code sequence that will increase the value is as follows:

```
$mysqli = new mysqli();
$res = $mysqli->query("SELECT 'abc'");
$res->fetch_assoc();
$res->close();
```

Every fetch operation will increase the value.

However, the statistic will not be increased if the result set is only buffered on the client, but not fetched, such as in the following example:

```
$mysqli = new mysqli();
$res = $mysqli->query("SELECT 'abc'");
$res->close();
```

`bytes_received_real_data_ps`

Number of bytes of the payload fetched by the PHP client from `mysqlnd` using the prepared statement protocol.

This is the size of the actual data contained in result sets that originate from prepared statements and which have been fetched by the PHP client.

The value will not be increased if the result set is not subsequently read by the PHP client.

Note that although a full result set may have been pulled from MySQL by `mysqlnd`, this statistic only counts actual data pulled from `mysqlnd` by the PHP client.

See also `bytes_received_real_data_normal`.

Result Set Related Statistics

`result_set_queries`

Number of queries that have generated a result set. Examples of queries that generate a result set: `SELECT`, `SHOW`.

The statistic will not be incremented if there is an error reading the result set header packet from the line.

> **Note** :
>
> This statistic can be used as an indirect measure for the number of queries PHP has sent to MySQL. This could help identifying a client that causes a high database load.

`non_result_set_queries`

Number of queries that did not generate a result set. Examples of queries that do not generate a result set: `INSERT`, `UPDATE`, `LOAD DATA`.

The statistic will not be incremented if there is an error reading the result set header packet from the line.

> **Note** :
>
> This statistic can be used as an indirect measure for the number of queries PHP has sent to MySQL. This could help identifying a client that causes a high database load.

`no_index_used`

Number of queries that have generated a result set but did not use an index. (See also the mysqld start option `--log-queries-not-using-indexes`).

> **Note** :
>
> Those queries can be reported via an exception by calling `mysqli_report(MYSQLI_REPORT_INDEX);`. It is possible to have them be reported via a warning instead by calling `mysqli_report(MYSQLI_REPORT_INDEX ^ MYSQLI_REPORT_STRICT);`.

`bad_index_used`

Number of queries that have generated a result set and did not use a good index. (See also the mysqld start option `--log-slow-queries`).

> **Note** :
>
> Those queries can be reported via an exception by calling `mysqli_report(MYSQLI_REPORT_INDEX);`. It is possible to have them be reported via a warning instead by calling `mysqli_report(MYSQLI_REPORT_INDEX ^ MYSQLI_REPORT_STRICT);`.

`slow_queries`

SQL statements that took more than `long_query_time` seconds to execute and required at least `min_examined_row_limit` rows to be examined.

> **Caution**
>
> Not reported through mysqli_report().

`buffered_sets`

Number of buffered result sets returned by normal (i.e. not via a prepared statement) queries.

Examples of API calls that will buffer result sets on the client: mysqli_query(), mysqli_store_result(), mysqli_stmt_get_result()

`unbuffered_sets`

Number of unbuffered result sets returned by normal (i.e. not via a prepared statement) queries.

Examples of API calls that will not buffer result sets on the client: mysqli_use_result()

`ps_buffered_sets`

Number of buffered result sets returned by prepared statements.

Examples of API calls that will buffer result sets on the client: mysqli_stmt_store_result()

`ps_unbuffered_sets`

Number of unbuffered result sets returned by prepared statements.

By default prepared statements are unbuffered, thus most prepared statements will be accounted in this statistic.

`flushed_normal_sets`

Number of result sets returned by normal (i.e. not via a prepared statement) queries with unread data that have been silently flushed.

> **Note** :
>
> Flushing happens only with unbuffered result sets.
>
> Unbuffered result sets must be fetched completely before a new query can be run on the connection otherwise MySQL will throw an error. If the application does not fetch all rows from an unbuffered result set, mysqlnd does implicitly fetch the result set to clear the line.
>
> See also `rows_skipped_normal`, `rows_skipped_ps`.
>
> Some possible causes for an implicit flush:
>
> - Faulty client application
> - Client stopped reading after it found what it was looking for but has made MySQL calculate more records than needed
> - Client application has stopped unexpectedly

`flushed_ps_sets`

Number of result sets from prepared statements with unread data that have been silently flushed.

> **Note** :
>
> Flushing happens only with unbuffered result sets.
>
> Unbuffered result sets must be fetched completely before a new query can be run on the connection otherwise MySQL will throw an error. If the application does not fetch all rows from an unbuffered result set, mysqlnd does implicitly fetch the result set to clear the line.
>
> See also `rows_skipped_normal`, `rows_skipped_ps`.
>
> Some possible causes for an implicit flush:

> - Faulty client application
> - Client stopped reading after it found what it was looking for but has made MySQL calculate more records than needed
> - Client application has stopped unexpectedly

`ps_prepared_never_executed`

Number of statements prepared but never executed.

`ps_prepared_once_executed`

Number of prepared statements executed only once.

`rows_fetched_from_server_normal`
`rows_fetched_from_server_ps`

Total number of result set rows fetched from the server. This includes the rows which were not read by the client but had been implicitly fetched due to flushed unbuffered result sets.

See also `packets_received_rset_row`.

`rows_buffered_from_client_normal`

Total number of buffered rows originating from a normal query.

This is the number of rows that have been fetched from MySQL and buffered on client.

Examples of queries that will buffer results:

- mysqli_query()
- mysqli_store_result()

`rows_buffered_from_client_ps`

Same as `rows_buffered_from_client_normal` but for prepared statements.

`rows_fetched_from_client_normal_buffered`

Total number of rows fetched by the client from a buffered result set created by a normal query.

`rows_fetched_from_client_ps_buffered`

Total number of rows fetched by the client from a buffered result set created by a prepared statement.

`rows_fetched_from_client_normal_unbuffered`

Total number of rows fetched by the client from an unbuffered result set created by a normal query.

`rows_fetched_from_client_ps_unbuffered`

Total number of rows fetched by the client from an unbuffered result set created by a prepared statement.

`rows_fetched_from_client_ps_cursor`

Total number of rows fetch by the client from a cursor created by a prepared statement.

`rows_skipped_normal`

`rows_skipped_ps`

Reserved for future use (currently not supported).

`copy_on_write_saved`
`copy_on_write_performed`

This is a process level scope statistic.

With mysqlnd, variables returned by the extensions point into mysqlnd internal network result buffers. If the data are not changed, the fetched data is kept only once in memory. However, any modification to the data will require mysqlnd to perform a copy-on-write operation.

`explicit_free_result`
`implicit_free_result`

This is a connection and process level scope statistic.

Total number of freed result sets.

`proto_text_fetched_null`

Total number of columns of type `MYSQL_TYPE_NULL` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_null`

Total number of columns of type `MYSQL_TYPE_NULL` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_bit`

Total number of columns of type `MYSQL_TYPE_BIT` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_bit`

Total number of columns of type `MYSQL_TYPE_BIT` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_tinyint`

Total number of columns of type `MYSQL_TYPE_TINY` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_tinyint`

Total number of columns of type `MYSQL_TYPE_TINY` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_short`

Total number of columns of type `MYSQL_TYPE_SHORT` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_short`

Total number of columns of type `MYSQL_TYPE_SHORT` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_int24`

Total number of columns of type `MYSQL_TYPE_INT24` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_int24`

Total number of columns of type `MYSQL_TYPE_INT24` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_int`

Total number of columns of type `MYSQL_TYPE_LONG` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_int`

Total number of columns of type `MYSQL_TYPE_LONG` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_bigint`

Total number of columns of type `MYSQL_TYPE_LONGLONG` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_bigint`

Total number of columns of type `MYSQL_TYPE_LONGLONG` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_decimal`

Total number of columns of type `MYSQL_TYPE_DECIMAL`, or `MYSQL_TYPE_NEWDECIMAL` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_decimal`

Total number of columns of type `MYSQL_TYPE_DECIMAL`, or `MYSQL_TYPE_NEWDECIMAL` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_float`

Total number of columns of type `MYSQL_TYPE_FLOAT` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_float`

Total number of columns of type `MYSQL_TYPE_FLOAT` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_double`

Total number of columns of type `MYSQL_TYPE_DOUBLE` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_double`

Total number of columns of type `MYSQL_TYPE_DOUBLE` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_date`

Total number of columns of type `MYSQL_TYPE_DATE`, or `MYSQL_TYPE_NEWDATE` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_date`

Total number of columns of type `MYSQL_TYPE_DATE`, or `MYSQL_TYPE_NEWDATE` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_year`

Total number of columns of type `MYSQL_TYPE_YEAR` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_year`

Total number of columns of type `MYSQL_TYPE_YEAR` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_time`

Total number of columns of type `MYSQL_TYPE_TIME` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_time`

Total number of columns of type `MYSQL_TYPE_TIME` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_datetime`

Total number of columns of type `MYSQL_TYPE_DATETIME` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_datetime`

Total number of columns of type `MYSQL_TYPE_DATETIME` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_timestamp`

Total number of columns of type `MYSQL_TYPE_TIMESTAMP` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_timestamp`

Total number of columns of type `MYSQL_TYPE_TIMESTAMP` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_string`

Total number of columns of type `MYSQL_TYPE_STRING`, `MYSQL_TYPE_VARSTRING`, or `MYSQL_TYPE_VARCHAR` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_string`

Total number of columns of type `MYSQL_TYPE_STRING`, `MYSQL_TYPE_VARSTRING`, or `MYSQL_TYPE_VARCHAR` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_blob`

Total number of columns of type `MYSQL_TYPE_TINY_BLOB`, `MYSQL_TYPE_MEDIUM_BLOB`, `MYSQL_TYPE_LONG_BLOB`, or `MYSQL_TYPE_BLOB` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_blob`

Total number of columns of type `MYSQL_TYPE_TINY_BLOB`, `MYSQL_TYPE_MEDIUM_BLOB`, `MYSQL_TYPE_LONG_BLOB`, or `MYSQL_TYPE_BLOB` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_enum`

Total number of columns of type `MYSQL_TYPE_ENUM` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_enum`

Total number of columns of type `MYSQL_TYPE_ENUM` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_set`

Total number of columns of type `MYSQL_TYPE_SET` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_set`

Total number of columns of type `MYSQL_TYPE_SET` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_geometry`

Total number of columns of type `MYSQL_TYPE_GEOMETRY` fetched from a normal query (MySQL text protocol).

`proto_binary_fetched_geometry`

Total number of columns of type `MYSQL_TYPE_GEOMETRY` fetched from a prepared statement (MySQL binary protocol).

`proto_text_fetched_other`

Total number of columns of types `MYSQL_TYPE_*` not listed previously fetched from a normal query (MySQL text protocol).

> **Note** :
>
> In theory, this should always be `0`.

`proto_binary_fetched_other`

Total number of columns of type `MYSQL_TYPE_*` not listed previously fetched from a prepared statement (MySQL binary protocol).

> **Note** :
>
> In theory, this should always be `0`.

Connection Related Statistics

`connect_success`

Total number of successful connection attempts.

> **Note** :
>
> `connect_success` holds the sum of successful persistent and non-persistent connection attempts. Therefore, the number of successful non-persistent connection attempts is `connect_success - pconnect_success`.

`pconnect_success`

Total number of successful persistent connection attempts.

`connect_failure`

Total number of failed connection attempts.

`reconnect`

This is a process level scope statistic.

`active_connections`

This is a process level scope statistic.

Total number of active persistent and non-persistent connections.

> **Note** :
>
> The total number of active non-persistent connections is `active_connections - active_persistent_connections`.

`active_persistent_connections`

This is a process level scope statistic.

Total number of active persistent connections.

`explicit_close`

Total number of explicitly closed connections.

Example #1 Examples of code snippets that cause an explicit close

- ```
  $link = new mysqli(/* ... */);
  $link->close(/* ... */);
  ```
- ```
  $link = new mysqli(/* ... */);
  $link->connect(/* ... */);
  ```

`implicit_close`

Total number of implicitly closed connections.

Example #2 Examples of code snippets that cause an implicit close

- ```
  $link = new mysqli(/* ... */);
  $link->real_connect(/* ... */);
  ```
- `unset($link)`
- Persistent connection: pooled connection has been created with real_connect and there may be unknown options set - close implicitly to avoid returning a connection with unknown options
- Persistent connection: ping/change_user fails and ext/mysqli closes the connection
- End of script execution: close connections that have not been closed by the user

`disconnect_close`

Connection failures indicated by the C API call `mysql_real_connect` during an attempt to establish a connection.

`in_middle_of_command_close`

This is a process level scope statistic.

A connection has been closed in the middle of a command execution (outstanding result sets not fetched, after sending a query and before retrieving an answer, while fetching data, while transferring data with LOAD DATA).

> **Warning**
>
> Unless asynchronous queries are used, this should only happen if the PHP application terminated unexpectedly, and PHP shuts down the connection automatically.

`init_command_executed_count`

Total number of init command executions. For example: `mysqli_options(MYSQLI_INIT_COMMAND , $value)`.

The number of successful executions is `init_command_executed_count - init_command_failed_count`.

`init_command_failed_count`

Total number of failed init commands.

`COM_*` Command Related Statistics

`com_quit`
`com_init_db`
`com_query`
`com_field_list`
`com_create_db`
`com_drop_db`
`com_refresh`
`com_shutdown`
`com_statistics`
`com_process_info`
`com_connect`
`com_process_kill`
`com_debug`
`com_ping`
  ```

com_time
com_delayed_insert
com_change_user
com_binlog_dump
com_table_dump
com_connect_out
com_register_slave
com_stmt_prepare
com_stmt_execute
com_stmt_send_long_data
com_stmt_close
com_stmt_reset
com_stmt_set_option
com_stmt_fetch
com_daemon

Total number of attempts to send a certain COM_* command from PHP to MySQL.

The statistics are incremented after checking the line and immediately before sending the corresponding MySQL client server protocol packet.

> **Caution**
>
> If MySQLnd fails to send the packet over the wire the statistics will not be decremented. In case of a failure MySQLnd emits a PHP warning "Error while sending %s packet. PID=%d."

Example #3 Usage examples

- Check if PHP sends certain commands to MySQL, for example, check if a client sends COM_PROCESS_KILL
- Calculate the average number of prepared statement executions by comparing COM_EXECUTE with COM_PREPARE
- Check if PHP has run any non-prepared SQL statements by checking if COM_QUERY is zero
- Identify PHP scripts that run an excessive number of SQL statements by checking COM_QUERY and COM_EXECUTE

Miscellaneous Statistics

explicit_stmt_close
implicit_stmt_close

This is a process level scope statistic.

Total number of closed prepared statements.

> **Note** :
>
> A prepared statement is always explicitly closed. The only time it's closed implicitly is when preparing it fails.

mem_emalloc_count

`mem_emalloc_ammount`
`mem_ecalloc_count`
`mem_ecalloc_ammount`
`mem_realloc_count`
`mem_realloc_ammount`
`mem_efree_count`
`mem_malloc_count`
`mem_malloc_ammount`
`mem_calloc_count`
`mem_calloc_ammount`
`mem_ealloc_count`
`mem_ealloc_ammount`
`mem_free_count`

This is a process level scope statistic.

Memory management calls.

`command_buffer_too_small`

Number of network command buffer extensions while sending commands from PHP to MySQL.

MySQLnd allocates an internal command/network buffer of mysqlnd.net_cmd_buffer_size bytes for every connection.

If a MySQL Client Server protocol command, e.g. `COM_QUERY` (normal query), does not fit into the buffer, MySQLnd will grow the buffer to what is needed for sending the command. Whenever the buffer gets extended for one connection `command_buffer_too_small` will be incremented by one.

If MySQLnd has to grow the buffer beyond its initial size of mysqlnd.net_cmd_buffer_size bytes for almost every connection, considerations to increase the default size should be made to avoid re-allocations.

`connection_reused`

The total number of times a persistent connection has been reused.

Notes

This section provides a collection of miscellaneous notes on MySQL Native Driver usage.

- Using `mysqlnd` means using PHP streams for underlying connectivity. For `mysqlnd`, the PHP streams documentation (Streams) should be consulted on such details as timeout settings, not the documentation for the MySQL Client Library.

Memory management

Introduction

The MySQL Native Driver manages memory different than the MySQL Client Library. The libraries differ in the way memory is allocated and released, how memory is allocated in chunks while reading results from MySQL, which debug and development options exist, and how results read from MySQL are linked to PHP user variables.

The following notes are intended as an introduction and summary to users interested at understanding the MySQL Native Driver at the C code level.

Memory management functions used

All memory allocation and deallocation is done using the PHP memory management functions. Therefore, the memory consumption of mysqlnd can be tracked using PHP API calls, such as memory_get_usage(). Because memory is allocated and released using the PHP memory management, the changes may not immediately become visible at the operating system level. The PHP memory management acts as a proxy which may delay releasing memory towards the system. Due to this, comparing the memory usage of the MySQL Native Driver and the MySQL Client Library is difficult. The MySQL Client Library is using the operating system memory management calls directly, hence the effects can be observed immediately at the operating system level.

Any memory limit enforced by PHP also affects the MySQL Native Driver. This may cause out of memory errors when fetching large result sets that exceed the size of the remaining memory made available by PHP. Because the MySQL Client Library is not using PHP memory management functions, it does not comply to any PHP memory limit set. If using the MySQL Client Library, depending on the deployment model, the memory footprint of the PHP process may grow beyond the PHP memory limit. But also PHP scripts may be able to process larger result sets as parts of the memory allocated to hold the result sets are beyond the control of the PHP engine.

PHP memory management functions are invoked by the MySQL Native Driver through a lightweight wrapper. Among others, the wrapper makes debugging easier.

Handling of result sets

The various MySQL Server and the various client APIs differentiate between buffered and unbuffered result sets. Unbuffered result sets are transferred row-by-row from MySQL to the client as the client iterates over the results. Buffered results are fetched in their entirety by the client library before passing them on to the client.

The MySQL Native Driver is using PHP Streams for the network communication with the MySQL Server. Results sent by MySQL are fetched from the PHP Streams network buffers into the result buffer of mysqlnd. The result buffer is made of zvals. In a second step the results are made available to the PHP script. This final transfer from the result buffer into PHP variables impacts the memory consumption and is mostly noticeable when using buffered result sets.

By default the MySQL Native Driver tries to avoid holding buffered results twice in memory. Results are kept only once in the internal result buffers and their zvals. When results are fetched into PHP variables by the PHP script, the variables will reference the internal result buffers. Database query results are not copied and kept in memory only once. Should the user modify the contents of a variable holding the database results a copy-on-write must be performed to avoid changing the referenced internal result buffer. The contents of the buffer must not be modified because the user may decide to read the result set a second time. The copy-on-write mechanism is implemented using an additional reference management list and the use of standard zval reference counters. Copy-on-write must also be done if the user reads a result set into PHP variables and frees a result set before the variables are unset.

Generally speaking, this pattern works well for scripts that read a result set once and do not modify variables holding results. Its major drawback is the memory overhead caused by the additional reference management which comes primarily from the fact that user variables holding results cannot be entirely released until the mysqlnd reference management stops referencing them. The MySQL Native driver removes the reference to the user variables when the result set is freed or a copy-on-write is performed. An observer will see the total memory consumption grow until the result set is released. Use the statistics to check whether a script does release result sets explicitly or the driver does implicit releases and thus memory is used for a time longer than necessary. Statistics also help to see how many copy-on-write operations happened.

A PHP script reading many small rows of a buffered result set using a code snippet equal or equivalent to `while ($row = $res->fetch_assoc()) { ... }` may optimize memory consumption by requesting copies instead of

references. Albeit requesting copies means keeping results twice in memory, it allows PHP to free the copy contained in `$row` as the result set is being iterated and prior to releasing the result set itself. On a loaded server optimizing peak memory usage may help improving the overall system performance although for an individual script the copy approach may be slower due to additional allocations and memory copy operations.

Monitoring and debugging

There are multiple ways of tracking the memory usage of the MySQL Native Driver. If the goal is to get a quick high level overview or to verify the memory efficiency of PHP scripts, then check the statistics collected by the library. The statistics allow you, for example, to catch SQL statements which generate more results than are processed by a PHP script.

The debug trace log can be configured to record memory management calls. This helps to see when memory is allocated or free'd. However, the size of the requested memory chunks may not be listed.

Some, recent versions of the MySQL Native Driver feature the emulation of random out of memory situations. This feature is meant to be used by the C developers of the library or mysqlnd plugin authors only. Please, search the source code for corresponding PHP configuration settings and further details. The feature is considered private and may be modified at any time without prior notice.

MySQL Native Driver Plugin API

Table of Contents

The MySQL Native Driver Plugin API is a feature of MySQL Native Driver, or `mysqlnd`. `Mysqlnd` plugins operate in the layer between PHP applications and the MySQL server. This is comparable to MySQL Proxy. MySQL Proxy operates on a layer between any MySQL client application, for example, a PHP application and, the MySQL server. `Mysqlnd` plugins can undertake typical MySQL Proxy tasks such as load balancing, monitoring and performance optimizations. Due to the different architecture and location, `mysqlnd` plugins do not have some of MySQL Proxy's disadvantages. For example, with plugins, there is no single point of failure, no dedicated proxy server to deploy, and no new programming language to learn (Lua).

A `mysqlnd` plugin can be thought of as an extension to `mysqlnd`. Plugins can intercept the majority of `mysqlnd` functions. The `mysqlnd` functions are called by the PHP MySQL extensions such as `ext/mysql`, `ext/mysqli`, and `PDO_MYSQL`. As a result, it is possible for a `mysqlnd` plugin to intercept all calls made to these extensions from the client application.

Internal `mysqlnd` function calls can also be intercepted, or replaced. There are no restrictions on manipulating `mysqlnd` internal function tables. It is possible to set things up so that when certain `mysqlnd` functions are called by the extensions that use `mysqlnd`, the call is directed to the appropriate function in the `mysqlnd` plugin. The ability to manipulate `mysqlnd` internal function tables in this way allows maximum flexibility for plugins.

`Mysqlnd` plugins are in fact PHP Extensions, written in C, that use the `mysqlnd` plugin API (which is built into MySQL Native Driver, `mysqlnd`). Plugins can be made 100% transparent to PHP applications. No application changes are needed because plugins operate on a different layer. The `mysqlnd` plugin can be thought of as operating in a layer below `mysqlnd`.

The following list represents some possible applications of `mysqlnd` plugins.

- Load Balancing
    - Read/Write Splitting. An example of this is the PECL/mysqlnd_ms (Master Slave) extension. This extension splits read/write queries for a replication setup.

    - Failover
    - Round-Robin, least loaded
- Monitoring
    - Query Logging
    - Query Analysis
    - Query Auditing. An example of this is the PECL/mysqlnd_sip (SQL Injection Protection) extension. This extension inspects queries and executes only those that are allowed according to a ruleset.
- Performance
    - Caching. An example of this is the PECL/mysqlnd_qc (Query Cache) extension.
    - Throttling
    - Sharding. An example of this is the PECL/mysqlnd_mc (Multi Connect) extension. This extension will attempt to split a SELECT statement into n-parts, using SELECT ... LIMIT part_1, SELECT LIMIT part_n. It sends the queries to distinct MySQL servers and merges the result at the client.

MySQL Native Driver Plugins Available

There are a number of mysqlnd plugins already available. These include:

- **PECL/mysqlnd_mc** - Multi Connect plugin.
- **PECL/mysqlnd_ms** - Master Slave plugin.
- **PECL/mysqlnd_qc** - Query Cache plugin.
- **PECL/mysqlnd_pscache** - Prepared Statement Handle Cache plugin.
- **PECL/mysqlnd_sip** - SQL Injection Protection plugin.
- **PECL/mysqlnd_uh** - User Handler plugin.

A comparison of mysqlnd plugins with MySQL Proxy

`Mysqlnd` plugins and MySQL Proxy are different technologies using different approaches. Both are valid tools for solving a variety of common tasks such as load balancing, monitoring, and performance enhancements. An important difference is that MySQL Proxy works with all MySQL clients, whereas `mysqlnd` plugins are specific to PHP applications.

As a PHP Extension, a `mysqlnd` plugin gets installed on the PHP application server, along with the rest of PHP. MySQL Proxy can either be run on the PHP application server or can be installed on a dedicated machine to handle multiple PHP application servers.

Deploying MySQL Proxy on the application server has two advantages:

1. No single point of failure
2. Easy to scale out (horizontal scale out, scale by client)

MySQL Proxy (and `mysqlnd` plugins) can solve problems easily which otherwise would have required changes to existing applications.

However, MySQL Proxy does have some disadvantages:

- MySQL Proxy is a new component and technology to master and deploy.
- MySQL Proxy requires knowledge of the Lua scripting language.

MySQL Proxy can be customized with C and Lua programming. Lua is the preferred scripting language of MySQL Proxy. For most PHP experts Lua is a new language to learn. A `mysqlnd` plugin can be written in C. It is also possible to write plugins in PHP using » PECL/mysqlnd_uh.

MySQL Proxy runs as a daemon - a background process. MySQL Proxy can recall earlier decisions, as all state can be retained. However, a `mysqlnd` plugin is bound to the request-based lifecycle of PHP. MySQL Proxy can also share one-time computed results among multiple application servers. A `mysqlnd` plugin would need to store data in a persistent medium to be able to do this. Another daemon would need to be used for this purpose, such as Memcache. This gives MySQL Proxy an advantage in this case.

MySQL Proxy works on top of the wire protocol. With MySQL Proxy you have to parse and reverse engineer the MySQL Client Server Protocol. Actions are limited to those that can be achieved by manipulating the communication protocol. If the wire protocol changes (which happens very rarely) MySQL Proxy scripts would need to be changed as well.

`Mysqlnd` plugins work on top of the C API, which mirrors the `libmysqlclient` client. This C API is basically a wrapper around the MySQL Client Server protocol, or wire protocol, as it is sometimes called. You can intercept all C API calls. PHP makes use of the C API, therefore you can hook all PHP calls, without the need to program at the level of the wire protocol.

`Mysqlnd` implements the wire protocol. Plugins can therefore parse, reverse engineer, manipulate and even replace the communication protocol. However, this is usually not required.

As plugins allow you to create implementations that use two levels (C API and wire protocol), they have greater flexibility than MySQL Proxy. If a `mysqlnd` plugin is implemented using the C API, any subsequent changes to the wire protocol do not require changes to the plugin itself.

Obtaining the mysqlnd plugin API

The `mysqlnd` plugin API is simply part of the MySQL Native Driver PHP extension, `ext/mysqlnd`. Development started on the `mysqlnd` plugin API in December 2009. It is developed as part of the PHP source repository, and as such is available to the public either via Git, or through source snapshot downloads.

Plugin developers can determine the `mysqlnd` version through accessing `MYSQLND_VERSION`, which is a string of the format “mysqlnd 8.3.17”, or through `MYSQLND_VERSION_ID`, which is an integer such as 50007. Developers can calculate the version number as follows:

MYSQLND_VERSION_ID calculation table

Version (part)	Example
Major*10000	5*10000 = 50000
Minor*100	0*100 = 0
Patch	7 = 7
MYSQLND_VERSION_ID	50007

During development, developers should refer to the `mysqlnd` version number for compatibility and version tests, as several iterations of `mysqlnd` could occur during the lifetime of a PHP development branch with a single PHP version number.

MySQL Native Driver Plugin Architecture

This section provides an overview of the `mysqlnd` plugin architecture.

MySQL Native Driver Overview

Before developing `mysqlnd` plugins, it is useful to know a little of how `mysqlnd` itself is organized. `Mysqlnd` consists of the following modules:

The mysqlnd organization chart, per module

Modules Statistics	mysqlnd_statistics.c
Connection	mysqlnd.c
Resultset	mysqlnd_result.c
Resultset Metadata	mysqlnd_result_meta.c
Statement	mysqlnd_ps.c
Network	mysqlnd_net.c
Wire protocol	mysqlnd_wireprotocol.c

C Object-Oriented Paradigm

At the code level, `mysqlnd` uses a C pattern for implementing object orientation.

In C you use a `struct` to represent an object. Members of the struct represent object properties. Struct members pointing to functions represent methods.

Unlike with other languages such as C++ or Java, there are no fixed rules on inheritance in the C object-oriented paradigm. However, there are some conventions that need to be followed that will be discussed later.

The PHP Life Cycle

When considering the PHP life cycle there are two basic cycles:

- PHP engine startup and shutdown cycle
- Request cycle

When the PHP engine starts up it will call the module initialization (MINIT) function of each registered extension. This allows each module to setup variables and allocate resources that will exist for the lifetime of the PHP engine process. When the PHP engine shuts down it will call the module shutdown (MSHUTDOWN) function of each extension.

During the lifetime of the PHP engine it will receive a number of requests. Each request constitutes another life cycle. On each request the PHP engine will call the request initialization function of each extension. The extension can perform any variable setup and resource allocation required for request processing. As the request cycle ends the engine calls the request shutdown (RSHUTDOWN) function of each extension so the extension can perform any cleanup required.

How a plugin works

A `mysqlnd` plugin works by intercepting calls made to `mysqlnd` by extensions that use `mysqlnd`. This is achieved by obtaining the `mysqlnd` function table, backing it up, and replacing it by a custom function table, which calls the functions of the plugin as required.

The following code shows how the `mysqlnd` function table is replaced:

```
/* a place to store original function table */
struct st_mysqlnd_conn_methods org_methods;

void minit_register_hooks(TSRMLS_D) {
  /* active function table */
  struct st_mysqlnd_conn_methods * current_methods
    = mysqlnd_conn_get_methods();

  /* backup original function table */
  memcpy(&org_methods, current_methods,
    sizeof(struct st_mysqlnd_conn_methods);

  /* install new methods */
  current_methods->query = MYSQLND_METHOD(my_conn_class, query);
}
```

Connection function table manipulations must be done during Module Initialization (MINIT). The function table is a global shared resource. In an multi-threaded environment, with a TSRM build, the manipulation of a global shared resource during the request processing will almost certainly result in conflicts.

> **Note** :
>
> Do not use any fixed-size logic when manipulating the `mysqlnd` function table: new methods may be added at the end of the function table. The function table may change at any time in the future.

Calling parent methods

If the original function table entries are backed up, it is still possible to call the original function table entries - the parent methods.

In some cases, such as for `Connection::stmt_init()`, it is vital to call the parent method prior to any other activity in the derived method.

```
MYSQLND_METHOD(my_conn_class, query)(MYSQLND *conn,
  const char *query, unsigned int query_len TSRMLS_DC) {

  php_printf("my_conn_class::query(query = %s)\n", query);

  query = "SELECT 'query rewritten' FROM DUAL";
  query_len = strlen(query);

  return org_methods.query(conn, query, query_len); /* return with call to parent */
}
```

Extending properties

A `mysqlnd` object is represented by a C struct. It is not possible to add a member to a C struct at run time. Users of `mysqlnd` objects cannot simply add properties to the objects.

Arbitrary data (properties) can be added to a `mysqlnd` objects using an appropriate function of the `mysqlnd_plugin_get_plugin_<object>_data()` family. When allocating an object `mysqlnd` reserves space at the end of the object to hold a `void *` pointer to arbitrary data. `mysqlnd` reserves space for one `void *` pointer per plugin.

The following table shows how to calculate the position of the pointer for a specific plugin:

Pointer calculations for mysqlnd

Memory address	Contents
0	Beginning of the mysqlnd object C struct
n	End of the mysqlnd object C struct
n + (m x sizeof(void*))	void* to object data of the m-th plugin

If you plan to subclass any of the `mysqlnd` object constructors, which is allowed, you must keep this in mind!

The following code shows extending properties:

```
/* any data we want to associate */
typedef struct my_conn_properties {
  unsigned long query_counter;
} MY_CONN_PROPERTIES;

/* plugin id */
unsigned int my_plugin_id;

void minit_register_hooks(TSRMLS_D) {
  /* obtain unique plugin ID */
  my_plugin_id = mysqlnd_plugin_register();
  /* snip - see Extending Connection: methods */
}

static MY_CONN_PROPERTIES** get_conn_properties(const MYSQLND *conn TSRMLS_DC) {
  MY_CONN_PROPERTIES** props;
  props = (MY_CONN_PROPERTIES**)mysqlnd_plugin_get_plugin_connection_data(
    conn, my_plugin_id);
  if (!props || !(*props)) {
    *props = mnd_pecalloc(1, sizeof(MY_CONN_PROPERTIES), conn->persistent);
    (*props)->query_counter = 0;
  }
  return props;
}
```

The plugin developer is responsible for the management of plugin data memory.

Use of the `mysqlnd` memory allocator is recommended for plugin data. These functions are named using the convention: `mnd_*loc()`. The `mysqlnd` allocator has some useful features, such as the ability to use a debug allocator in a non-debug build.

When and how to subclass

	When to subclass?	Each instance has its own private function table?	How to subclass?
Connection (MYSQLND)	MINIT	No	mysqlnd_conn_get_methods()
Resultset (MYSQLND_RES)	MINIT or later	Yes	mysqlnd_result_get_methods() or object method function table manipulation

	When to subclass?	Each instance has its own private function table?	How to subclass?
Resultset Meta (MYSQLND_RES_METADATA)	MINIT	No	mysqlnd_result_metadata_get_methods()
Statement (MYSQLND_STMT)	MINIT	No	mysqlnd_stmt_get_methods()
Network (MYSQLND_NET)	MINIT or later	Yes	mysqlnd_net_get_methods() or object method function table manipulation
Wire protocol (MYSQLND_PROTOCOL)	MINIT or later	Yes	mysqlnd_protocol_get_methods() or object method function table manipulation

You must not manipulate function tables at any time later than MINIT if it is not allowed according to the above table.

Some classes contain a pointer to the method function table. All instances of such a class will share the same function table. To avoid chaos, in particular in threaded environments, such function tables must only be manipulated during MINIT.

Other classes use copies of a globally shared function table. The class function table copy is created together with the object. Each object uses its own function table. This gives you two options: you can manipulate the default function table of an object at MINIT, and you can additionally refine methods of an object without impacting other instances of the same class.

The advantage of the shared function table approach is performance. There is no need to copy a function table for each and every object.

Constructor status

Type	Allocation, construction, reset	Can be modified?	Caller
Connection (MYSQLND)	mysqlnd_init()	No	mysqlnd_connect()
Resultset(MYSQLND_RES)	Allocation: • Connection::result_init() Reset and re-initialized during: • Result::use_result() • Result::store_result	Yes, but call parent!	• Connection::list_fields() • Statement::get_result() • Statement::prepare() (Metadata only) • Statement::resultMetaData()
Resultset Meta (MYSQLND_RES_METADATA)	Connection::result_meta_init()	Yes, but call parent!	Result::read_result_metadata()
Statement (MYSQLND_STMT)	Connection::stmt_init()	Yes, but call parent!	Connection::stmt_init()
Network (MYSQLND_NET)	mysqlnd_net_init()	No	Connection::init()

Type	Allocation, construction, reset	Can be modified?	Caller
Wire protocol (MYSQLND_PROTOCOL)	mysqlnd_protocol_init()	No	Connection::init()

It is strongly recommended that you do not entirely replace a constructor. The constructors perform memory allocations. The memory allocations are vital for the `mysqlnd` plugin API and the object logic of `mysqlnd`. If you do not care about warnings and insist on hooking the constructors, you should at least call the parent constructor before doing anything in your constructor.

Regardless of all warnings, it can be useful to subclass constructors. Constructors are the perfect place for modifying the function tables of objects with non-shared object tables, such as Resultset, Network, Wire Protocol.

Destruction status

Type	Derived method must call parent?	Destructor
Connection	yes, after method execution	free_contents(), end_psession()
Resultset	yes, after method execution	free_result()
Resultset Meta	yes, after method execution	free()
Statement	yes, after method execution	dtor(), free_stmt_content()
Network	yes, after method execution	free()
Wire protocol	yes, after method execution	free()

The destructors are the appropriate place to free properties, `mysqlnd_plugin_get_plugin_<object>_data()`.

The listed destructors may not be equivalent to the actual `mysqlnd` method freeing the object itself. However, they are the best possible place for you to hook in and free your plugin data. As with constructors you may replace the methods entirely but this is not recommended. If multiple methods are listed in the above table you will need to hook all of the listed methods and free your plugin data in whichever method is called first by `mysqlnd`.

The recommended method for plugins is to simply hook the methods, free your memory and call the parent implementation immediately following this.

The mysqlnd plugin API

The following is a list of functions provided in the `mysqlnd` plugin API:

- mysqlnd_plugin_register()
- mysqlnd_plugin_count()
- mysqlnd_plugin_get_plugin_connection_data()
- mysqlnd_plugin_get_plugin_result_data()
- mysqlnd_plugin_get_plugin_stmt_data()
- mysqlnd_plugin_get_plugin_net_data()
- mysqlnd_plugin_get_plugin_protocol_data()

- mysqlnd_conn_get_methods()
- mysqlnd_result_get_methods()
- mysqlnd_result_meta_get_methods()
- mysqlnd_stmt_get_methods()
- mysqlnd_net_get_methods()
- mysqlnd_protocol_get_methods()

There is no formal definition of what a plugin is and how a plugin mechanism works.

Components often found in plugins mechanisms are:

- A plugin manager
- A plugin API
- Application services (or modules)
- Application service APIs (or module APIs)

The `mysqlnd` plugin concept employs these features, and additionally enjoys an open architecture.

No Restrictions

A plugin has full access to the inner workings of `mysqlnd`. There are no security limits or restrictions. Everything can be overwritten to implement friendly or hostile algorithms. It is recommended you only deploy plugins from a trusted source.

As discussed previously, plugins can use pointers freely. These pointers are not restricted in any way, and can point into another plugin's data. Simple offset arithmetic can be used to read another plugin's data.

It is recommended that you write cooperative plugins, and that you always call the parent method. The plugins should always cooperate with `mysqlnd` itself.

Issues: an example of chaining and cooperation

Extension	mysqlnd.query() pointer	call stack if calling parent
ext/mysqlnd	mysqlnd.query()	mysqlnd.query
ext/mysqlnd_cache	mysqlnd_cache.query()	1. mysqlnd_cache.query() 2. mysqlnd.query
ext/mysqlnd_monitor	mysqlnd_monitor.query()	1. mysqlnd_monitor.query() 2. mysqlnd_cache.query() 3. mysqlnd.query

In this scenario, a cache (`ext/mysqlnd_cache`) and a monitor (`ext/mysqlnd_monitor`) plugin are loaded. Both subclass `Connection::query()`. Plugin registration happens at `MINIT` using the logic shown previously. PHP calls

extensions in alphabetical order by default. Plugins are not aware of each other and do not set extension dependencies.

By default the plugins call the parent implementation of the query method in their derived version of the method.

PHP Extension Recap

This is a recap of what happens when using an example plugin, `ext/mysqlnd_plugin`, which exposes the `mysqlnd` C plugin API to PHP:

- Any PHP MySQL application tries to establish a connection to 192.168.2.29
- The PHP application will either use `ext/mysql`, `ext/mysqli` or `PDO_MYSQL`. All three PHP MySQL extensions use `mysqlnd` to establish the connection to 192.168.2.29.
- `Mysqlnd` calls its connect method, which has been subclassed by `ext/mysqlnd_plugin`.
- `ext/mysqlnd_plugin` calls the userspace hook `proxy::connect()` registered by the user.
- The userspace hook changes the connection host IP from 192.168.2.29 to 127.0.0.1 and returns the connection established by `parent::connect()`.
- `ext/mysqlnd_plugin` performs the equivalent of `parent::connect(127.0.0.1)` by calling the original `mysqlnd` method for establishing a connection.
- `ext/mysqlnd` establishes a connection and returns to `ext/mysqlnd_plugin`. `ext/mysqlnd_plugin` returns as well.
- Whatever PHP MySQL extension had been used by the application, it receives a connection to 127.0.0.1. The PHP MySQL extension itself returns to the PHP application. The circle is closed.

Getting started building a mysqlnd plugin

It is important to remember that a `mysqlnd` plugin is itself a PHP extension.

The following code shows the basic structure of the MINIT function that will be used in the typical `mysqlnd` plugin:

```
/* my_php_mysqlnd_plugin.c */

 static PHP_MINIT_FUNCTION(mysqlnd_plugin) {
  /* globals, ini entries, resources, classes */

  /* register mysqlnd plugin */
  mysqlnd_plugin_id = mysqlnd_plugin_register();

  conn_m = mysqlnd_get_conn_methods();
  memcpy(org_conn_m, conn_m,
    sizeof(struct st_mysqlnd_conn_methods));

  conn_m->query = MYSQLND_METHOD(mysqlnd_plugin_conn, query);
  conn_m->connect = MYSQLND_METHOD(mysqlnd_plugin_conn, connect);
}
```

```
/* my_mysqlnd_plugin.c */

 enum_func_status MYSQLND_METHOD(mysqlnd_plugin_conn, query)(/* ... */) {
```

```
  /* ... */
}
enum_func_status MYSQLND_METHOD(mysqlnd_plugin_conn, connect)(/* ... */) {
  /* ... */
}
```

Task analysis: from C to userspace

```
 class proxy extends mysqlnd_plugin_connection {
  public function connect($host, ...) { .. }
}
mysqlnd_plugin_set_conn_proxy(new proxy());
```

Process:

1. PHP: user registers plugin callback
2. PHP: user calls any PHP MySQL API to connect to MySQL
3. C: ext/*mysql* calls mysqlnd method
4. C: mysqlnd ends up in ext/mysqlnd_plugin
5. C: ext/mysqlnd_plugin
    1. Calls userspace callback
    2. Or original `mysqlnd` method, if userspace callback not set

You need to carry out the following:

1. Write a class "mysqlnd_plugin_connection" in C
2. Accept and register proxy object through "mysqlnd_plugin_set_conn_proxy()"
3. Call userspace proxy methods from C (optimization - zend_interfaces.h)

Userspace object methods can either be called using `call_user_function()` or you can operate at a level closer to the Zend Engine and use `zend_call_method()`.

Optimization: calling methods from C using zend_call_method

The following code snippet shows the prototype for the `zend_call_method` function, taken from *zend_interfaces.h*.

```
 ZEND_API zval* zend_call_method(
  zval **object_pp, zend_class_entry *obj_ce,
  zend_function **fn_proxy, char *function_name,
  int function_name_len, zval **retval_ptr_ptr,
  int param_count, zval* arg1, zval* arg2 TSRMLS_DC
);
```

Zend API supports only two arguments. You may need more, for example:

```
 enum_func_status (*func_mysqlnd_conn__connect)(
  MYSQLND *conn, const char *host,
  const char * user, const char * passwd,
  unsigned int passwd_len, const char * db,
  unsigned int db_len, unsigned int port,
```

```
  const char * socket, unsigned int mysql_flags TSRMLS_DC
);
```

To get around this problem you will need to make a copy of `zend_call_method()` and add a facility for additional parameters. You can do this by creating a set of `MY_ZEND_CALL_METHOD_WRAPPER` macros.

Calling PHP userspace

This code snippet shows the optimized method for calling a userspace function from C:

```
/* my_mysqlnd_plugin.c */

MYSQLND_METHOD(my_conn_class,connect)(
  MYSQLND *conn, const char *host /* ... */ TSRMLS_DC) {
  enum_func_status ret = FAIL;
  zval * global_user_conn_proxy = fetch_userspace_proxy();
  if (global_user_conn_proxy) {
    /* call userspace proxy */
    ret = MY_ZEND_CALL_METHOD_WRAPPER(global_user_conn_proxy, host, /*...*/);
  } else {
    /* or original mysqlnd method = do nothing, be transparent */
    ret = org_methods.connect(conn, host, user, passwd,
          passwd_len, db, db_len, port,
          socket, mysql_flags TSRMLS_CC);
  }
  return ret;
}
```

Calling userspace: simple arguments

```
/* my_mysqlnd_plugin.c */

 MYSQLND_METHOD(my_conn_class,connect)(
  /* ... */, const char *host, /* ...*/) {
  /* ... */
  if (global_user_conn_proxy) {
    /* ... */
    zval* zv_host;
    MAKE_STD_ZVAL(zv_host);
    ZVAL_STRING(zv_host, host, 1);
    MY_ZEND_CALL_METHOD_WRAPPER(global_user_conn_proxy, zv_retval, zv_host /*, ...*/);
    zval_ptr_dtor(&zv_host);
    /* ... */
  }
  /* ... */
}
```

Calling userspace: structs as arguments

```
/* my_mysqlnd_plugin.c */

MYSQLND_METHOD(my_conn_class, connect)(
  MYSQLND *conn, /* ...*/) {
  /* ... */
  if (global_user_conn_proxy) {
    /* ... */
    zval* zv_conn;
```

```
    ZEND_REGISTER_RESOURCE(zv_conn, (void *)conn, le_mysqlnd_plugin_conn);
    MY_ZEND_CALL_METHOD_WRAPPER(global_user_conn_proxy, zv_retval, zv_conn, zv_host /*, ...*/);
    zval_ptr_dtor(&zv_conn);
    /* ... */
  }
  /* ... */
}
```

The first argument of many `mysqlnd` methods is a C "object". For example, the first argument of the connect() method is a pointer to `MYSQLND`. The struct MYSQLND represents a `mysqlnd` connection object.

The `mysqlnd` connection object pointer can be compared to a standard I/O file handle. Like a standard I/O file handle a `mysqlnd` connection object shall be linked to the userspace using the PHP resource variable type.

From C to userspace and back

```
 class proxy extends mysqlnd_plugin_connection {
  public function connect($conn, $host, ...) {
    /* "pre" hook */
    printf("Connecting to host = '%s'\n", $host);
    debug_print_backtrace();
    return parent::connect($conn);
  }

  public function query($conn, $query) {
    /* "post" hook */
    $ret = parent::query($conn, $query);
    printf("Query = '%s'\n", $query);
    return $ret;
  }
}
mysqlnd_plugin_set_conn_proxy(new proxy());
```

PHP users must be able to call the parent implementation of an overwritten method.

As a result of subclassing it is possible to refine only selected methods and you can choose to have "pre" or "post" hooks.

Built-in class: mysqlnd_plugin_connection::connect()

```
/*  my_mysqlnd_plugin_classes.c */

 PHP_METHOD("mysqlnd_plugin_connection", connect) {
  /* ... simplified! ... */
  zval* mysqlnd_rsrc;
  MYSQLND* conn;
  char* host; int host_len;
  if (zend_parse_parameters(ZEND_NUM_ARGS() TSRMLS_CC, "rs",
    &mysqlnd_rsrc, &host, &host_len) == FAILURE) {
    RETURN_NULL();
  }
  ZEND_FETCH_RESOURCE(conn, MYSQLND* conn, &mysqlnd_rsrc, -1,
    "Mysqlnd Connection", le_mysqlnd_plugin_conn);
  if (PASS == org_methods.connect(conn, host, /* simplified! */ TSRMLS_CC))
    RETVAL_TRUE;
  else
```

```
    RETVAL_FALSE;
}
```

PHP Data Objects

Introduction

The `PHP Data Objects` (PDO) extension defines a lightweight, consistent interface for accessing databases in PHP. Each database driver that implements the PDO interface can expose database-specific features as regular extension functions. Note that you cannot perform any database functions using the PDO extension by itself; you must use a database-specific PDO driver to access a database server.

PDO provides a *data-access* abstraction layer, which means that, regardless of which database you're using, you use the same functions to issue queries and fetch data. PDO does *not* provide a *database* abstraction; it doesn't rewrite SQL or emulate missing features. You should use a full-blown abstraction layer if you need that facility.

PDO ships with PHP.

Installing/Configuring

Table of Contents

Installation

This extension is enabled by default. It may be disabled by using the following option at compile time: **--disable-pdo**

Installing PDO on Unix systems

1. PDO and the PDO_SQLITE driver are enabled by default. You may need to enable the PDO driver for your database of choice; consult the documentation for database-specific PDO drivers to find out more about that.

   > **Note** :
   >
   > When building PDO as a shared extension (*not recommended*) then all PDO drivers *must* be loaded *after* PDO itself.

2. When installing PDO as a shared module, the php.ini file needs to be updated so that the PDO extension will be loaded automatically when PHP runs. You will also need to enable any database specific drivers there too; make sure that they are listed after the extension=pdo line, as PDO must be initialized before the database-specific extensions can be loaded. If you built PDO and the database-specific extensions statically, you can skip this step.

   ```
   extension=pdo
   ```

Windows users

1. PDO is enabled by default. Enable the database-specific drivers in *php.ini* as needed. For example, this loads the PDO_SQLITE driver but leaves the PDO_ODBC driver commented out:

```
;extension=pdo_odbc
extension=pdo_sqlite
```

   The corresponding extension files should exist in the extension_dir.

> **Note** :
>
> Remember that after making changes to your *php.ini* file you will need to restart PHP for your new configuration directives to take effect.

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

PDO Configuration Options

Name	Default	Changeable	Changelog
pdo.dsn.*		*php.ini* only	

Here's a short explanation of the configuration directives.

`pdo.dsn.*` string

Defines a DSN alias. See PDO::__construct() for thorough explanation.

Predefined Constants

Table of Contents

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

Cursors

See also **`PDO::ATTR_CURSOR_NAME`**.

`PDO::FETCH_ORI_NEXT` (int)

Fetch the next row in the result set. Valid only for scrollable cursors.

`PDO::FETCH_ORI_PRIOR` (int)

Fetch the previous row in the result set. Valid only for scrollable cursors.

`PDO::FETCH_ORI_FIRST` (int)

Fetch the first row in the result set. Valid only for scrollable cursors.

PDO::FETCH_ORI_LAST (int)

Fetch the last row in the result set. Valid only for scrollable cursors.

PDO::FETCH_ORI_ABS (int)

Fetch the requested row by row number from the result set. Valid only for scrollable cursors.

PDO::FETCH_ORI_REL (int)

Fetch the requested row by relative position from the current position of the cursor in the result set. Valid only for scrollable cursors.

PDO::CURSOR_FWDONLY (int)

Create a PDOStatement object with a forward-only cursor. This is the default cursor choice, as it is the fastest and most common data access pattern in PHP.

PDO::CURSOR_SCROLL (int)

Create a PDOStatement object with a scrollable cursor. Pass the **PDO::FETCH_ORI_*** constants to control the rows fetched from the result set.

Other Constants

PDO::PARAM_BOOL (int)

Represents a boolean data type.

PDO::PARAM_NULL (int)

Represents the SQL NULL data type.

PDO::PARAM_INT (int)

Represents the SQL INTEGER data type.

PDO::PARAM_STR (int)

Represents the SQL CHAR, VARCHAR, or other string data type.

PDO::PARAM_STR_NATL (int)

Flag to denote a string uses the national character set.

Available as of PHP 7.2.0

PDO::PARAM_STR_CHAR (int)

Flag to denote a string uses the regular character set.

Available as of PHP 7.2.0

PDO::PARAM_LOB (int)

Represents the SQL large object data type.

PDO::PARAM_STMT (int)

Represents a recordset type. Not currently supported by any drivers.

PDO::PARAM_INPUT_OUTPUT (int)

Specifies that the parameter is an INOUT parameter for a stored procedure. This constant must be bitwise-ORed with one of the **PDO::PARAM_*** constants.

PDO::ATTR_AUTOCOMMIT (bool)

If this value is **false**, PDO attempts to disable autocommit so that the connection begins a transaction.

As of PHP 8.4.0 this constant is typed as bool; previously, it was typed as int.

PDO::ATTR_PREFETCH (int)

Setting the prefetch size allows balancing speed against memory usage for an application. Not all database/driver combinations support setting of the prefetch size. A larger prefetch size results in increased performance at the cost of higher memory usage.

The PDO_PGSQL driver instead treats this attribute as a toggle: as of PHP 8.5.0, a value of `0` enables lazy (single-row) fetching; see the PDO_PGSQL driver docs for details.

PDO::ATTR_TIMEOUT (int)

Sets the timeout value in seconds for communications with the database.

PDO::ATTR_ERRMODE (int)

See the Errors and error handling section for more information about this attribute.

PDO::ATTR_SERVER_VERSION (int)

This is a read only attribute; it will return information about the version of the database server to which PDO is connected.

PDO::ATTR_CLIENT_VERSION (int)

This is a read only attribute; it will return information about the version of the client libraries that the PDO driver is using.

PDO::ATTR_SERVER_INFO (int)

This is a read only attribute; it will return some meta information about the database server to which PDO is connected.

PDO::ATTR_CONNECTION_STATUS (int)

PDO::ATTR_CASE (int)

Force column names to a specific case specified by the **PDO::CASE_*** constants.

PDO::ATTR_CURSOR_NAME (int)

Get or set the name to use for a cursor. Most useful when using scrollable cursors and positioned updates.

PDO::ATTR_CURSOR (int)

Selects the cursor type. PDO currently supports either `PDO::CURSOR_FWDONLY` or `PDO::CURSOR_SCROLL`. Unless one has a need for scrollable cursors, one should use the `PDO::CURSOR_FWDONLY` cursor mode.

`PDO::ATTR_DRIVER_NAME` (int)

Returns the name of the driver.

Example #1 using `PDO::ATTR_DRIVER_NAME`

```
<?php
 if ($db->getAttribute(PDO::ATTR_DRIVER_NAME) == 'mysql') {
   echo "Running on mysql; doing something mysql specific here\n";
 }
 ?>
```

`PDO::ATTR_ORACLE_NULLS` (int)

Convert empty strings to SQL NULL values on data fetches.

`PDO::ATTR_PERSISTENT` (int)

Request a persistent connection, rather than creating a new connection. See Connections and Connection management for more information on this attribute.

`PDO::ATTR_STATEMENT_CLASS` (int)

Sets the class name and constructor arguments for the extension of the PDOStatement object. The first element of the array is the class name, and the second element is an array of constructor arguments.

`PDO::ATTR_FETCH_CATALOG_NAMES` (int)

Prepend the containing catalog name to each column name returned in the result set. The catalog name and column name are separated by a decimal (.) character. Support of this attribute is at the driver level; it may not be supported by the driver in use.

`PDO::ATTR_FETCH_TABLE_NAMES` (int)

Prepend the containing table name to each column name returned in the result set. The table name and column name are separated by a decimal (.) character. Support of this attribute is at the driver level; it may not be supported by the driver in use.

`PDO::ATTR_STRINGIFY_FETCHES` (int)

Forces all fetched values (except `null`) to be treated as strings. `null` values remain unchanged unless `PDO::ATTR_ORACLE_NULLS` is set to `PDO::NULL_TO_STRING`.

`PDO::ATTR_MAX_COLUMN_LEN` (int)

Sets the maximum column name length.

`PDO::ATTR_DEFAULT_FETCH_MODE` (int)

`PDO::ATTR_EMULATE_PREPARES` (bool)

Whether to enable or disable emulation of prepared statements. Some drivers either do not support native prepared statements or have only limited support for them. When set to `true`, PDO always emulates prepared statements; when set to `false`, it uses the driver's native prepared statements.

As of PHP 8.4.0 this constant is typed as bool; previously, it was typed as int.

PDO::ATTR_DEFAULT_STR_PARAM (int)

Sets the default string parameter type, which can be either **PDO::PARAM_STR_NATL** or **PDO::PARAM_STR_CHAR**.

Available as of PHP 7.2.0.

PDO::ERRMODE_SILENT (int)

Do not raise an error or exception if an error occurs. The developer is expected to explicitly check for errors. Prior to PHP 8.0.0, this was the default mode. See Errors and error handling for more information about this attribute.

PDO::ERRMODE_WARNING (int)

Issue a PHP **E_WARNING** message if an error occurs. See Errors and error handling for more information about this attribute.

PDO::ERRMODE_EXCEPTION (int)

Throw a PDOException if an error occurs. This is the default mode as of PHP 8.0.0. See Errors and error handling for more information about this attribute.

PDO::CASE_NATURAL (int)

Leave column names as returned by the database driver.

PDO::CASE_LOWER (int)

Force column names to lower case.

PDO::CASE_UPPER (int)

Force column names to upper case.

PDO::NULL_NATURAL (int)

PDO::NULL_EMPTY_STRING (int)

PDO::NULL_TO_STRING (int)

PDO::ERR_NONE (string)

Corresponds to SQLSTATE `'00000'`, meaning that the SQL statement was successfully issued with no errors or warnings. This constant is a convenience constant to help when checking PDO::errorCode() or PDOStatement::errorCode() to determine if an error occurred. This is usually known by examining the return code from the method that raised the error condition anyway.

PDO::PARAM_EVT_ALLOC (int)

Allocation event

PDO::PARAM_EVT_FREE (int)

Deallocation event

PDO::PARAM_EVT_EXEC_PRE (int)

Event triggered prior to execution of a prepared statement.

`PDO::PARAM_EVT_EXEC_POST` (int)

Event triggered subsequent to execution of a prepared statement.

`PDO::PARAM_EVT_FETCH_PRE` (int)

Event triggered prior to fetching a result from a resultset.

`PDO::PARAM_EVT_FETCH_POST` (int)

Event triggered subsequent to fetching a result from a resultset.

`PDO::PARAM_EVT_NORMALIZE` (int)

Event triggered during bound parameter registration allowing the driver to normalize the parameter name.

`PDO::SQLITE_DETERMINISTIC` (int)

> **Warning**
>
> This constant has been *DEPRECATED* as of PHP 8.5.0. Use `Pdo\Sqlite::DETERMINISTIC` instead.

Specifies that a function created with PDO::sqliteCreateFunction() is deterministic, i.e. it always returns the same result given the same inputs within a single SQL statement. (Available as of PHP 7.1.4.)

Fetch Modes

See cursor constants for the `PDO::FETCH_ORI_*` cursor constants.

Basic Fetch Modes

Fetch Mode	Summary
`PDO::FETCH_DEFAULT`	Special value for using the current default fetch mode.
`PDO::FETCH_ASSOC`	Array indexed by column name only.
`PDO::FETCH_BOTH` (Default)	Array indexed by both column number and name.
`PDO::FETCH_NAMED`	Variant of `PDO::FETCH_ASSOC` that retains duplicated columns.
`PDO::FETCH_NUM`	Array indexed by column number only.
`PDO::FETCH_COLUMN`	A single column.
`PDO::FETCH_KEY_PAIR`	Key-value pairs, indexed by the first column.
`PDO::FETCH_FUNC`	Use a function to create the return value. (PDOStatement::fetchAll() only)
`PDO::FETCH_OBJ`	Anonymous (stdClass) object.
`PDO::FETCH_CLASS`	An object of a specified class.

PDO::FETCH_CLASS options

These modes are used to implement options when using `PDO::FETCH_CLASS`.

Fetch Mode	Summary
`PDO::FETCH_CLASSTYPE`	Use the first column as the class name.

Fetch Mode	Summary
`PDO::FETCH_PROPS_LATE`	Call the constructor before setting properties.
`PDO::FETCH_SERIALIZE`	Use PHP serialized data. Deprecated as of PHP 8.1.0.

Single Result Modes

The following modes cannot be used with PDOStatement::fetchAll().

Fetch Mode	Summary
`PDO::FETCH_BOUND`	Bind values to specified variables.
`PDO::FETCH_INTO`	Update an existing object.
`PDO::FETCH_LAZY`	Lazy fetch via PDORow for array- and object-like access.

Special Behavior Flags for PDOStatement::fetchAll()

The following special modes for multiple results only work with PDOStatement::fetchAll() and do not work with some other fetch modes. Check the full documentation for details.

Fetch Mode	Summary
`PDO::FETCH_GROUP`	Results are grouped by the first column.
`PDO::FETCH_UNIQUE`	Results are (uniquely) indexed by the first column.

Handling of Duplicated Column Names

It's possible for results to contain multiple columns that use the same name. For example, when joining 2 tables that both contain a column with the same name.

Because PHP structures such as arrays and objects don't support multiple keys or properties that use the same name, the returned array or object will contain only 1 of the values using the same name.

Which value is returned for a given duplicated name should be considered undefined.

To avoid this issue, explicitly name columns using an alias. For example:

```
SELECT table1.created_at AS t1_created_at,
       table2.created_at AS t2_created_at
FROM table1
JOIN table2 ON table1.table2id = table2.id
```

See also `PDO::FETCH_NAMED`, `PDO::ATTR_FETCH_TABLE_NAMES` and `PDO::ATTR_FETCH_CATALOG_NAMES`.

Setting the Default Fetch Mode

It is possible to set default fetch mode for all queries using `PDO::ATTR_DEFAULT_FETCH_MODE` with PDO::__construct() or PDO::setAttribute().

The default fetch mode for a specific statement can be set using PDOStatement::setFetchMode(). This affects reuse as a prepared statement and iteration (using `foreach`).

> **Caution**

> PDOStatement::setAttribute() cannot be used to set the default fetch mode. It only accepts driver specific attributes and silently ignores attributes that are not recognized.

PDO::FETCH_DEFAULT (int)

Available as of PHP 8.0.7.

This is a special value that uses the current default fetch mode for a PDOStatement. It's specifically useful as the default value for method parameters when extending PDOStatement for use with `PDO::ATTR_STATEMENT_CLASS`.

This value cannot be used with `PDO::ATTR_DEFAULT_FETCH_MODE`.

PDO::FETCH_ASSOC (int)

`PDO::FETCH_ASSOC` returns an array indexed by column name only.

```php
<?php
$stmt = $pdo->query("SELECT userid, name, country FROM users");
$row = $stmt->fetch(\PDO::FETCH_ASSOC);
print_r($row);
```

The above example will output:

```
Array
(
    [userid] => 104
    [name] => Chris
    [country] => Ukraine
)
```

PDO::FETCH_BOTH (int)

This is the default fetch mode.

`PDO::FETCH_BOTH` returns an array indexed by both column number and name. This means that every returned value is duplicated for each result row.

The column number starts at 0 and is determined by the result column order in the query, not (for example) the order columns are defined in the table.

> **Note** :
>
> Using the numeric column index is not recommended as this may change when the query is changed, or when the table schema is changed when using `SELECT *`.

> **Note** :

> The number of entries indexed by name may not match the number of entries indexed by number in cases where multiple returned columns use the same name.

```php
<?php
$stmt = $pdo->query("SELECT userid, name, country FROM users");
$row = $stmt->fetch(\PDO::FETCH_BOTH);
print_r($row);
```

The above example will output:

```
Array
(
    [id] => 104,
    [0] => 104,
    [name] => Chris,
    [1] => Chris,
    [country] => Ukraine,
    [2] => Ukraine
)
```

PDO::FETCH_NAMED (int)

`PDO::FETCH_NAMED` returns results in the same format as `PDO::FETCH_ASSOC` except that where multiple columns use the same name, all values are returned as a list.

For more information on handling of duplicated column names and alternatives, see the handling of duplicated names section above.

The order in which duplicated values are returned should be considered undefined. There's no way to tell where each value came from.

```php
<?php
$stmt = $pdo->query(
    "SELECT users.*, referrer.name
     FROM users
     LEFT JOIN users AS referrer ON users.referred_by = referrer.userid
     WHERE userid = 109"
);
$row = $stmt->fetch(\PDO::FETCH_NUM);
print_r($row);
```

The above example will output:

```
Array
(
    [userid] => 109
    [name] => Array
        (
            [0] => Toni
```

```
            [1] => Chris
        )
    [country] => Germany
    [referred_by] = 104
)
```

PDO::FETCH_NUM (int)

`PDO::FETCH_NUM` returns an array indexed by column number only. The column number starts at 0 and is determined by the result column order in the query, not (for example) the order columns are defined in the table.

> **Note** :
>
> Using the numeric column index is not recommended as this may change when the query is changed, or when the table schema is changed when using `SELECT *`.

```php
<?php
$stmt = $pdo->query("SELECT userid, name, country FROM users");
$row = $stmt->fetch(\PDO::FETCH_NUM);
print_r($row);
```

The above example will output:

```
Array
(
    [0] => 104
    [1] => Chris
    [2] => Ukraine
)
```

PDO::FETCH_COLUMN (int)

`PDO::FETCH_COLUMN` returns values from a single column. Use the second argument for PDOStatement::setFetchMode() or PDOStatement::fetchAll() to specify which column is returned.

If the specified column does not exist a ValueError will be thrown.

```php
<?php
$stmt = $pdo->query("SELECT name, country FROM users LIMIT 3");
$row = $stmt->fetchAll(\PDO::FETCH_COLUMN);
print_r($row);

$stmt = $pdo->query("SELECT name, country FROM users LIMIT 3");
$row = $stmt->fetchAll(\PDO::FETCH_COLUMN, 1);
print_r($row);
```

The above example will output:

```
Array
(
    [0] => Chris
    [1] => Jamie
    [2] => Robin
)

Array
(
    [0] => Ukraine
    [1] => England
    [2] => Germany
)
```

PDO::FETCH_KEY_PAIR (int)

`PDO::FETCH_KEY_PAIR` returns pairs of values, indexed by the first column. The results must contain only 2 columns. This fetch mode only makes sense with PDOStatement::fetchAll().

> **Note** :
>
> If the first column is not unique, values will be lost. Which value(s) are lost should be considered undefined.

```php
<?php
$stmt = $pdo->query("SELECT name, country FROM users LIMIT 3");
$row = $stmt->fetchAll(\PDO::FETCH_KEY_PAIR);
print_r($row);
```

The above example will output:

```
Array
(
    [Chris] => Ukraine
    [Jamie] => England
    [Robin] => Germany
)
```

PDO::FETCH_FUNC (int)

Specify a function to create the returned value. This mode can only be used with PDOStatement::fetchAll().

The function receives the values as parameters. There's no way to retrieve the column name a given value was associated with. It is crucial to ensure that the column order in the query matches the order of parameters of the function.

> **Note** :
>
> The effects of `PDO::FETCH_GROUP` and `PDO::FETCH_UNIQUE` are applied to results before the function is called.

```php
<?php
function valueCreator($col1, $col2, $col3)
{
    return [
        'col1' => $col1,
        'col2' => strtoupper($col2),
        'col3' => $col3,
        'customKey' => 'customValue',
    ];
}

$stmt = $pdo->query("SELECT userid, name, country FROM users LIMIT 3");
$row = $stmt->fetchAll(\PDO::FETCH_FUNC, valueCreator(...));
print_r($row);
```

The above example will output:

```
Array
(
    [0] => Array
        (
            [col1] => 104
            [col2] => SAM
            [col3] => Ukraine
            [customKey] => customValue
        )

    [1] => Array
        (
            [col1] => 105
            [col2] => JAMIE
            [col3] => England
            [customKey] => customValue
        )

    [2] => Array
        (
            [col1] => 107
            [col2] => ROBIN
            [col3] => Germany
            [customKey] => customValue
        )

)
```

PDO::FETCH_OBJ (int)

PDO::FETCH_OBJ returns a stdClass object.

See also PDOStatement::fetchObject() and **PDO::FETCH_CLASS**.

```
<?php
$stmt = $pdo->query("SELECT userid, name, country FROM users");
$row = $stmt->fetch(\PDO::FETCH_OBJ);
print_r($row);
```

The above example will output:

```
stdClass Object
(
    [userid] => 104
    [name] => Chris
    [country] => Ukraine
)
```

PDO::FETCH_CLASS (int)

Returns an object of a specified class. For additional behaviors see the option flags.

If a property does not exist with the name of a returned column, it will be dynamically declared. This behavior is deprecated and will cause an error from PHP 9.0.

See also PDOStatement::fetchObject().

```
<?php
class TestEntity
{
    public $userid;

    public $name;

    public $country;

    public $referred_by_userid;

    public function __construct()
    {
        print "Constructor called with ". count(func_get_args()) ." args\n";
        print "Properties set when constructor called? "
            . (isset($this->name) ? 'Yes' : 'No') . "\n";
    }
}

$stmt = $db->query(
    "SELECT userid, name, country, referred_by_userid FROM users"
);
$stmt->setFetchMode(PDO::FETCH_CLASS, TestEntity::class);
$result = $stmt->fetch();
var_dump($result);
```

The above example will output something similar to:

```
Constructor called with 0 args
Properties set when constructor called? Yes
object(TestEntity)#3 (4) {
  ["userid"]=>
  int(104)
  ["name"]=>
  string(5) "Chris"
  ["country"]=>
  string(7) "Ukraine"
  ["referred_by_userid"]=>
  NULL
}
```

PDO::FETCH_CLASSTYPE (int)

This fetch mode can only be used combined with **PDO::FETCH_CLASS** (and its other options).

When this fetch mode is used, PDO will use the first returned column as the name of the class to return.

If the specified class cannot be found, a stdClass object will be returned, without warning or error.

```php
<?php
class TestEntity
{
    public $userid;

    public $name;

    public $country;

    public $referred_by_userid;

    public function __construct()
    {
        print "Constructor called with ". count(func_get_args()) ." args\n";
        print "Properties set when constructor called? "
            . (isset($this->name) ? 'Yes' : 'No') . "\n";
    }
}

$stmt = $db->query(
    "SELECT 'TestEntity', userid, name, country, referred_by_userid FROM users"
);
$stmt->setFetchMode(PDO::FETCH_CLASS | PDO::FETCH_CLASSTYPE);
$result = $stmt->fetch();
var_dump($result);
```

The above example will output something similar to:

```
Constructor called with 0 args
Properties set when constructor called? Yes
object(TestEntity)#3 (4) {
  ["userid"]=>
  int(104)
  ["name"]=>
  string(5) "Chris"
  ["country"]=>
  string(7) "Ukraine"
  ["referred_by_userid"]=>
  NULL
}
```

PDO::FETCH_PROPS_LATE (int)

This fetch mode can only be used combined with **PDO::FETCH_CLASS** (and its other options).

When this fetch mode is used, the constructor will be called before the properties are set.

```php
<?php
class TestEntity
{
    public $userid;

    public $name;

    public $country;

    public $referred_by_userid;

    public function __construct()
    {
        print "Constructor called with ". count(func_get_args()) ." args\n";
        print "Properties set when constructor called? "
            . (isset($this->name) ? 'Yes' : 'No') . "\n";
    }
}

$stmt = $db->query(
    "SELECT userid, name, country, referred_by_userid FROM users"
);
$stmt->setFetchMode(PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE, TestEntity::class);
$result = $stmt->fetch();
var_dump($result);
```

The above example will output something similar to:

```
Constructor called with 0 args
Properties set when constructor called? No
object(TestEntity)#3 (4) {
  ["userid"]=>
  int(104)
  ["name"]=>
```

```
  string(5) "Chris"
  ["country"]=>
  string(7) "Ukraine"
  ["referred_by_userid"]=>
  NULL
}
```

PDO::FETCH_SERIALIZE (int)

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 8.1.0. Relying on this feature is highly discouraged.

This fetch mode can only be used combined with **PDO::FETCH_CLASS** (and its other options).

When this fetch mode is used, the specified class must be Serializable.

> **Caution**
>
> This feature does not support a string that contains a complete serialized object (with serialize()).

> **Caution**
>
> This fetch mode does not call the constructor.

```php
<?php
class TestEntity implements Serializable
{
    public $userid;

    public $name;

    public $country;

    public $referred_by_userid;

    public function __construct()
    {
        print "Constructor called with " . count(func_get_args()) . " args\n";
        print "Properties set when constructor called? "
            . (isset($this->name) ? 'Yes' : 'No') . "\n";
    }

    public function serialize()
    {
        return join(
            "|",
            [$this->userid, $this->name, $this->country, $this->referred_by_userid]
        );
    }

    public function unserialize(string $data)
    {
        $parts = explode("|", $data);
```

```
        $this->userid = (int) $parts[0];
        $this->name = $parts[1];
        $this->country = $parts[2];

        $refId = $parts[3];
        $this->referred_by_userid = ($refId === "" ? null : (int) $refId);
    }
}

print "Set up record (constructor called manually):\n";
$db->exec(
    "CREATE TABLE serialize (
        sdata TEXT
    )"
);

$origObj = new TestEntity();
$origObj->userid = 200;
$origObj->name = 'Seri';
$origObj->country = 'Syria';
$origObj->referred_by_userid = null;

$insert = $db->prepare("INSERT INTO serialize (sdata) VALUES (:sdata)");
$insert->execute(['sdata' => $origObj->serialize()]);

print "\nRetrieve result:\n";
$query = "SELECT sdata FROM serialize";
$stmt = $db->query($query);
// NOTE: Constructor is never called!
$stmt->setFetchMode(PDO::FETCH_CLASS | PDO::FETCH_SERIALIZE, TestEntity::class);
$result = $stmt->fetch();
var_dump($result);
```

The above example will output something similar to:

```
Deprecated: TestEntity implements the Serializable interface, which is deprecated. Implement
__serialize() and __unserialize() instead (or in addition, if support for old PHP versions is
necessary) in Standard input code on line 2
Set up record (constructor called manually):
Constructor called with 0 args
Properties set when constructor called? No

Retrieve result:
Deprecated: PDOStatement::setFetchMode(): The PDO::FETCH_SERIALIZE mode is deprecated in Standard
input code on line 58

Deprecated: PDOStatement::fetch(): The PDO::FETCH_SERIALIZE mode is deprecated in Standard input
code on line 59
object(TestEntity)#5 (4) {
  ["userid"]=>
  int(200)
  ["name"]=>
  string(4) "Seri"
  ["country"]=>
  string(5) "Syria"
  ["referred_by_userid"]=>
```

```
  NULL
}
```

PDO::FETCH_BOUND (int)

This fetch mode cannot be used with PDOStatement::fetchAll().

This fetch mode does not directly return a result, but binds values to variables specified with PDOStatement::bindColumn(). The called fetch method returns `true`.

> **Note** :
>
> When using prepared statements, to work correctly, variables must be bound after the query is executed.

```php
<?php
$query = "SELECT users.userid, users.name, users.country, referrer.name
    FROM users
    LEFT JOIN users AS referrer ON users.referred_by_userid = referrer.userid";
$stmt = $db->prepare($query);
$stmt->execute();

$stmt->bindColumn('userid', $userId);
$stmt->bindColumn('name', $name);
$stmt->bindColumn('country', $country);
// Bind by column position to resolve duplicated column name
// To avoid this breaking if the query is changed, use an SQL alias instead
// For example: referrer.name AS referrer_name
$stmt->bindColumn(4, $referrerName);

while ($stmt->fetch(\PDO::FETCH_BOUND)) {
    print join("\t", [$userId, $name, $country, ($referrerName ?? 'NULL')]) . "\n";
}
```

The above example will output:

```
104     Chris   Ukraine NULL
105     Jamie   England NULL
107     Robin   Germany Chris
108     Sean    Ukraine NULL
109     Toni    Germany NULL
110     Toni    Germany NULL
```

PDO::FETCH_INTO (int)

This fetch mode cannot be used with PDOStatement::fetchAll().

This fetch mode updates properties in the specified object. The object is returned on success.

If a property does not exist with the name of a returned column, it will be dynamically declared. This behavior is deprecated and will cause an error from PHP 9.0.

Properties must be `public` and cannot be `readonly`.

> **Caution**
>
> There's no way to change the object to be updated without using PDOStatement::setFetchMode() between retrieving each record.

```php
<?php

class TestEntity
{
    public $userid;

    public $name;

    public $country;

    public $referred_by_userid;
}

$obj = new TestEntity();

$stmt = $db->query("SELECT userid, name, country, referred_by_userid FROM users");

$stmt->setFetchMode(\PDO::FETCH_INTO, $obj);
$result = $stmt->fetch();

var_dump($result);
```

The above example will output something similar to:

```
object(TestEntity)#3 (4) {
  ["userid"]=>
  int(104)
  ["name"]=>
  string(5) "Chris"
  ["country"]=>
  string(7) "Ukraine"
  ["referred_by_userid"]=>
  NULL
}
```

PDO::FETCH_LAZY (int)

This fetch mode cannot be used with PDOStatement::fetchAll().

This fetch mode returns a PDORow object which provides both array- and object-like access to values (i.e. combines the behavior of **`PDO::FETCH_BOTH`** and **`PDO::FETCH_OBJ`**), retrieved in a lazy manner.

This can provide memory efficient access (on the PHP side) to unbuffered results on the database server. Whether PDO uses client-side buffering for results depends on the database-specific driver used (and its configuration).

> **Caution**
>
> PDORow will return `null` without any error or warning when accessing properties or keys that are not defined. This can make errors such as typos or queries not returning expected data harder to spot and debug.

> **Caution**
>
> The returned PDORow object is updated each time a result is retrieved.

```php
<?php
$stmt = $db->query("SELECT userid, name, country, referred_by_userid FROM users");
$result = $stmt->fetch(\PDO::FETCH_LAZY);

print "ID: ". $result[0] ."\n";
print "Name: {$result->name}\n";
print "Country: " . $result['country'] ."\n";
// Returns NULL. No warning or error is raised.
print "Does not exist: " . var_export($result->does_not_exist, true) . "\n";

$differentResult = $stmt->fetch(\PDO::FETCH_LAZY);
// The previously retrieved PDORow now points to the newly retrieved result
print "ID: ". $result[0] ."\n";
```

The above example will output:

```
ID: 104
Name: Chris
Country: Ukraine
Does not exist: NULL
ID: 105
```

PDO::FETCH_GROUP (int)

`PDO::FETCH_GROUP` returns lists of associative arrays, indexed by a (non-unique) column. This fetch mode only works with PDOStatement::fetchAll().

When combined with `PDO::FETCH_UNIQUE`, both modes will use the same column, rendering the combination of these modes useless.

This fetch should be combined with one of `PDO::FETCH_ASSOC`, `PDO::FETCH_BOTH`, `PDO::FETCH_NAMED`, `PDO::FETCH_NUM`, `PDO::FETCH_COLUMN` or `PDO::FETCH_FUNC`.

If no fetch mode from the above list is given, the current default fetch mode for the PDOStatement will be used.

```php
<?php
$stmt = $pdo->query("SELECT country, userid, name FROM users");
$row = $stmt->fetchAll(\PDO::FETCH_GROUP | \PDO::FETCH_ASSOC);
print_r($row);
```

The above example will output:

```
Array
(
    [Ukraine] => Array
        (
            [0] => Array
                (
                    [userid] => 104
                    [name] => Chris
                )

            [1] => Array
                (
                    [userid] => 108
                    [name] => Sean
                )

        )
    [England] => Array
        (
            [0] => Array
                (
                    [userid] => 105
                    [name] => Jamie
                )

        )

    [Germany] => Array
        (
            [0] => Array
                (
                    [userid] => 107
                    [name] => Robin
                )

            [1] => Array
                (
                    [userid] => 109
                    [name] => Toni
                )
        )
)
```

In the above example one should note that the first column is omitted from the array for each row, only available as the key. It can be included by repeating the column, as in the following example:

```php
<?php
$stmt = $pdo->query("SELECT country, userid, name, country FROM users");
$row = $stmt->fetchAll(\PDO::FETCH_GROUP | \PDO::FETCH_ASSOC);
print_r($row);
```

The above example will output:

```
Array
(
    [Ukraine] => Array
        (
            [0] => Array
                (
                    [userid] => 104
                    [name] => Chris
                    [country] => Ukraine
                )

            [1] => Array
                (
                    [userid] => 108
                    [name] => Sean
                    [country] => Ukraine
                )

        )
    [England] => Array
        (
            [0] => Array
                (
                    [userid] => 105
                    [name] => Jamie
                    [country] => England
                )

        )

    [Germany] => Array
        (
            [0] => Array
                (
                    [userid] => 107
                    [name] => Robin
                    [country] => Germany
                )

            [1] => Array
                (
                    [userid] => 109
                    [name] => Toni
                    [country] => Germany
                )
        )
)
```

PDO::FETCH_UNIQUE (int)

PDO::FETCH_UNIQUE uses the first column to index records, returning 1 record per index value. This fetch mode only works with PDOStatement::fetchAll().

When combined with **PDO::FETCH_GROUP**, both modes will use the same column, rendering the combination of these modes useless.

This fetch should be combined with one of **PDO::FETCH_ASSOC**, **PDO::FETCH_BOTH**, **PDO::FETCH_NAMED**, **PDO::FETCH_NUM**, **PDO::FETCH_COLUMN** or **PDO::FETCH_FUNC**.

If no fetch mode from the above list is given, the current default fetch mode for the PDOStatement will be used.

When used with a column that is known to be unique (such as record ID), this mode provides the ability to quickly return results indexed by that value.

> **Note** :
>
> If the first column is not unique, values will be lost. Which value(s) are lost should be considered undefined.

> **Caution**
>
> Filtering records should be done in SQL where possible. The database will use indexes to optimize this process and return only the required records. Selecting more records than required from the database may significantly increase memory usage and query time for larger result sets.

```php
<?php
$stmt = $pdo->query("SELECT userid, name, country FROM users LIMIT 3");
$row = $stmt->fetchAll(\PDO::FETCH_UNIQUE | \PDO::FETCH_ASSOC);
print_r($row);
```

The above example will output:

```
Array
(
    [104] => Array
        (
            [name] => Chris
            [country] => Ukraine
        )

    [105] => Array
        (
            [name] => Jamie
            [country] => England
        )

    [107] => Array
        (
            [name] => Robin
```

```
            [country] => Germany
        )

)
```

In the above example one should note that the first column is omitted from the array for each row, only available as the key. It can be included by repeating the column, as in the following example:

```php
<?php
$stmt = $pdo->query("SELECT userid, userid, name, country FROM users LIMIT 3");
$row = $stmt->fetchAll(\PDO::FETCH_UNIQUE | \PDO::FETCH_ASSOC);
print_r($row);
```

The above example will output:

```
Array
(
    [104] => Array
        (
            [userid] => 104
            [name] => Chris
            [country] => Ukraine
        )

    [105] => Array
        (
            [userid] => 105
            [name] => Jamie
            [country] => England
        )

    [107] => Array
        (
            [userid] => 107
            [name] => Robin
            [country] => Germany
        )

)
```

Connections and Connection management

Connections are established by creating instances of the PDO base class. It doesn't matter which driver you want to use; you always use the PDO class name. The constructor accepts parameters for specifying the database source (known as the DSN) and optionally for the username and password (if any).

Example #1 Connecting to MySQL

```php
<?php
$dbh = new PDO('mysql:host=localhost;dbname=test', $user, $pass);
?>
```

If there are any connection errors, a `PDOException` object will be thrown. You may catch the exception if you want to handle the error condition, or you may opt to leave it for an application global exception handler that you set up via set_exception_handler().

Example #2 Handling connection errors

```php
<?php
try {
    $dbh = new PDO('mysql:host=localhost;dbname=test', $user, $pass);
} catch (PDOException $e) {
    // attempt to retry the connection after some timeout for example
}
```

> **Warning**
>
> Just like any other exception, PDOException could be caught either explicitly, via a `catch` statement, or implicitly via set_exception_handler(). Otherwise, the default behaviour of converting an uncaught exception to a **`E_FATAL_ERROR`** will occur. The fatal error will contain a backtrace that can leak connection details. As such, the *php.ini* option `display_errors` should be set to `0` on a production server.

Upon successful connection to the database, an instance of the PDO class is returned to your script. The connection remains active for the lifetime of that PDO object. To close the connection, you need to destroy the object by ensuring that all remaining references to it are deleted—you do this by assigning `null` to the variable that holds the object. If you don't do this explicitly, PHP will automatically close the connection when your script ends.

> **Note** :
>
> If there are still other references to this PDO instance (such as from a PDOStatement instance, or from other variables referencing the same PDO instance), these have to be removed also (for instance, by assigning `null` to the variable that references the PDOStatement).

Example #3 Closing a connection

```php
<?php
$dbh = new PDO('mysql:host=localhost;dbname=test', $user, $pass);
// use the connection here
$sth = $dbh->query('SELECT * FROM foo');

// and now we're done; close it
$sth = null;
$dbh = null;
?>
```

Many web applications will benefit from making persistent connections to database servers. Persistent connections are not closed at the end of the script, but are cached and re-used when another script requests a connection using the same credentials. The persistent connection cache allows you to avoid the overhead of

establishing a new connection every time a script needs to talk to a database, resulting in a faster web application.

Example #4 Persistent connections

```php
<?php
$dbh = new PDO('mysql:host=localhost;dbname=test', $user, $pass, array(
    PDO::ATTR_PERSISTENT => true
));
?>
```

The value of the `PDO::ATTR_PERSISTENT` option is converted to bool (enable/disable persistent connections), unless it is a non-numeric string, in which case it allows the use of multiple persistent connection pools. This is useful if different connections use incompatible settings, for instance, different values of `PDO::MYSQL_ATTR_USE_BUFFERED_QUERY`.

> **Note** :
>
> If you wish to use persistent connections, you must set `PDO::ATTR_PERSISTENT` in the array of driver options passed to the PDO constructor. If setting this attribute with PDO::setAttribute() after instantiation of the object, the driver will not use persistent connections.

> **Warning**
>
> PDO does not perform any cleanup of persistent connections. Temporary tables, locks, transactions and other stateful changes may remain from previous usage of the connection, causing unexpected problems. See the Persistent Database Connections section for more information.

> **Note** :
>
> If you're using the PDO ODBC driver and your ODBC libraries support ODBC Connection Pooling (unixODBC and Windows are two that do; there may be more), then it's recommended that you don't use persistent PDO connections, and instead leave the connection caching to the ODBC Connection Pooling layer. The ODBC Connection Pool is shared with other modules in the process; if PDO is told to cache the connection, then that connection would never be returned to the ODBC connection pool, resulting in additional connections being created to service those other modules.

Transactions and auto-commit

Now that you're connected via PDO, you must understand how PDO manages transactions before you start issuing queries. If you've never encountered transactions before, they offer 4 major features: Atomicity, Consistency, Isolation and Durability (ACID). In layman's terms, any work carried out in a transaction, even if it is carried out in stages, is guaranteed to be applied to the database safely, and without interference from other connections, when it is committed. Transactional work can also be automatically undone at your request (provided you haven't already committed it), which makes error handling in your scripts easier.

Transactions are typically implemented by "saving-up" your batch of changes to be applied all at once; this has the nice side effect of drastically improving the efficiency of those updates. In other words, transactions can make your scripts faster and potentially more robust (you still need to use them correctly to reap that benefit).

Unfortunately, not every database supports transactions, so PDO needs to run in what is known as "auto-commit" mode when you first open the connection. Auto-commit mode means that every query that you run has its own implicit transaction, if the database supports it, or no transaction if the database doesn't support transactions. If you need a transaction, you must use the PDO::beginTransaction() method to initiate one. If the underlying driver does not support transactions, a PDOException will be thrown (regardless of your error handling settings: this is always a serious error condition). Once you are in a transaction, you may use PDO::commit() or PDO::rollBack() to finish it, depending on the success of the code you run during the transaction.

> **Warning**
>
> PDO only checks for transaction capabilities on driver level. If certain runtime conditions mean that transactions are unavailable, PDO::beginTransaction() will still return **`true`** without error if the database server accepts the request to start a transaction.
>
> An example of this would be trying to use transactions on MyISAM tables on a MySQL database.

> **Warning**
>
> *Implicit Commits with DDL Statements:* Some databases automatically issue an implicit `COMMIT` when a database definition language (DDL) statement, such as `DROP TABLE` or `CREATE TABLE`, is executed within a transaction. This means that any prior changes made within the same transaction will be *automatically committed* and cannot be rolled back.
>
> `MySQL` and `Oracle` are example databases that exhibit this behavior.

Example #1 Implicit Commit Example

```php
<?php
$pdo->beginTransaction();
$pdo->exec("INSERT INTO users (name) VALUES ('Rasmus')");
$pdo->exec("CREATE TABLE test (id INT PRIMARY KEY)"); // Implicit COMMIT occurs here
$pdo->rollBack(); // This will NOT undo the INSERT/CREATE for MySQL or Oracle
?>
```

Best Practice: Avoid executing DDL statements inside transactions if using databases that enforce this behavior. If necessary, separate DDL operations from transactional logic.

When the script ends or when a connection is about to be closed, if you have an outstanding transaction, PDO will automatically roll it back. This is a safety measure to help avoid inconsistency in the cases where the script terminates unexpectedly--if you didn't explicitly commit the transaction, then it is assumed that something went awry, so the rollback is performed for the safety of your data.

> **Warning**
>
> The automatic rollback only happens if you initiate the transaction via PDO::beginTransaction(). If you manually issue a query that begins a transaction PDO has no way of knowing about it and thus cannot roll it back if something bad happens.

Example #2 Executing a batch in a transaction

In the following sample, let's assume that we are creating a set of entries for a new employee, who has been assigned an ID number of 23. In addition to entering the basic data for that person, we also need to record their salary. It's pretty simple to make two separate updates, but by enclosing them within the PDO::beginTransaction() and PDO::commit() calls, we are guaranteeing that no one else will be able to see those changes until they are complete. If something goes wrong, the catch block rolls back all changes made since the transaction was started, and then prints out an error message.

```
<?php
try {
  $dbh = new PDO('odbc:SAMPLE', 'db2inst1', 'ibmdb2',
      array(PDO::ATTR_PERSISTENT => true));
  echo "Connected\n";
} catch (Exception $e) {
  die("Unable to connect: " . $e->getMessage());
}

try {
  $dbh->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

  $dbh->beginTransaction();
  $dbh->exec("insert into staff (id, first, last) values (23, 'Joe', 'Bloggs')");
  $dbh->exec("insert into salarychange (id, amount, changedate)
      values (23, 50000, NOW())");
  $dbh->commit();

} catch (Exception $e) {
  $dbh->rollBack();
  echo "Failed: " . $e->getMessage();
}
?>
```

You're not limited to making updates in a transaction; you can also issue complex queries to extract data, and possibly use that information to build up more updates and queries; while the transaction is active, you are guaranteed that no one else can make changes while you are in the middle of your work. For further reading on transactions, refer to the documentation provided by your database server.

Prepared statements and stored procedures

Many of the more mature databases support the concept of prepared statements. What are they? They can be thought of as a kind of compiled template for the SQL that an application wants to run, that can be customized using variable parameters. Prepared statements offer two major benefits:

- The query only needs to be parsed (or prepared) once, but can be executed multiple times with the same or different parameters. When the query is prepared, the database will analyze, compile and optimize its plan for executing the query. For complex queries this process can take up enough time that it will noticeably slow down an application if there is a need to repeat the same query many times with different parameters. By using a prepared statement the application avoids repeating the analyze/compile/optimize cycle. This means that prepared statements use fewer resources and thus run faster.

- The parameters to prepared statements don't need to be quoted; the driver automatically handles this. If an application exclusively uses prepared statements, the developer can be sure that no SQL injection will occur (however, if other portions of the query are being built up with unescaped input, SQL injection is still possible).

Prepared statements are so useful that they are the only feature that PDO will emulate for drivers that don't support them. This ensures that an application will be able to use the same data access paradigm regardless of the capabilities of the database.

Example #1 Repeated inserts using prepared statements

This example performs an INSERT query by substituting a `name` and a `value` for the named placeholders.

```php
<?php
$stmt = $dbh->prepare("INSERT INTO REGISTRY (name, value) VALUES (:name, :value)");
$stmt->bindParam(':name', $name);
$stmt->bindParam(':value', $value);

// insert one row
$name = 'one';
$value = 1;
$stmt->execute();

// insert another row with different values
$name = 'two';
$value = 2;
$stmt->execute();
?>
```

Example #2 Repeated inserts using prepared statements

This example performs an INSERT query by substituting a `name` and a `value` for the positional `?` placeholders.

```php
<?php
$stmt = $dbh->prepare("INSERT INTO REGISTRY (name, value) VALUES (?, ?)");
$stmt->bindParam(1, $name);
$stmt->bindParam(2, $value);

// insert one row
$name = 'one';
$value = 1;
$stmt->execute();

// insert another row with different values
$name = 'two';
$value = 2;
$stmt->execute();
?>
```

Example #3 Fetching data using prepared statements

This example fetches data based on a key value supplied by a form. The user input is automatically quoted, so there is no risk of a SQL injection attack.

```php
<?php
$stmt = $dbh->prepare("SELECT * FROM REGISTRY where name = ?");
$stmt->execute([$_GET['name']]);
foreach ($stmt as $row) {
  print_r($row);
}
?>
```

Example #4 Calling a stored procedure with an output parameter

If the database driver supports it, an application may also bind parameters for output as well as input. Output parameters are typically used to retrieve values from stored procedures. Output parameters are slightly more complex to use than input parameters, in that a developer must know how large a given parameter might be when they bind it. If the value turns out to be larger than the size they suggested, an error is raised.

```
<?php
$stmt = $dbh->prepare("CALL sp_returns_string(?)");
$stmt->bindParam(1, $return_value, PDO::PARAM_STR, 4000);

// call the stored procedure
$stmt->execute();

print "procedure returned $return_value\n";
?>
```

Example #5 Calling a stored procedure with an input/output parameter

Developers may also specify parameters that hold values both input and output; the syntax is similar to output parameters. In this next example, the string 'hello' is passed into the stored procedure, and when it returns, hello is replaced with the return value of the procedure.

```
<?php
$stmt = $dbh->prepare("CALL sp_takes_string_returns_string(?)");
$value = 'hello';
$stmt->bindParam(1, $value, PDO::PARAM_STR|PDO::PARAM_INPUT_OUTPUT, 4000);

// call the stored procedure
$stmt->execute();

print "procedure returned $value\n";
?>
```

Example #6 Invalid use of placeholder

```
<?php
$stmt = $dbh->prepare("SELECT * FROM REGISTRY where name LIKE '%?%'");
$stmt->execute([$_GET['name']]);

// placeholder must be used in the place of the whole value
$stmt = $dbh->prepare("SELECT * FROM REGISTRY where name LIKE ?");
$stmt->execute(["%$_GET[name]%"]);
?>
```

Errors and error handling

PDO offers you a choice of 3 different error handling strategies, to fit your style of application development.

- **PDO::ERRMODE_SILENT**

  Prior to PHP 8.0.0, this was the default mode. PDO will simply set the error code for you to inspect using the PDO::errorCode() and PDO::errorInfo() methods on both the statement and database objects; if the error resulted from a call on a statement object, you would invoke the PDOStatement::errorCode() or

PDOStatement::errorInfo() method on that object. If the error resulted from a call on the database object, you would invoke those methods on the database object instead.

- **PDO::ERRMODE_WARNING**

  In addition to setting the error code, PDO will emit a traditional E_WARNING message. This setting is useful during debugging/testing, if you just want to see what problems occurred without interrupting the flow of the application.

- **PDO::ERRMODE_EXCEPTION**

  As of PHP 8.0.0, this is the default mode. In addition to setting the error code, PDO will throw a PDOException and set its properties to reflect the error code and error information. This setting is also useful during debugging, as it will effectively "blow up" the script at the point of the error, very quickly pointing a finger at potential problem areas in your code (remember: transactions are automatically rolled back if the exception causes the script to terminate).

  Exception mode is also useful because you can structure your error handling more clearly than with traditional PHP-style warnings, and with less code/nesting than by running in silent mode and explicitly checking the return value of each database call.

  See Exceptions for more information about Exceptions in PHP.

PDO standardizes on using SQL-92 SQLSTATE error code strings; individual PDO drivers are responsible for mapping their native codes to the appropriate SQLSTATE codes. The PDO::errorCode() method returns a single SQLSTATE code. If you need more specific information about an error, PDO also offers an PDO::errorInfo() method which returns an array containing the SQLSTATE code, the driver specific error code and driver specific error string.

Example #1 Create a PDO instance and set the error mode

```php
<?php
$dsn = 'mysql:dbname=testdb;host=127.0.0.1';
$user = 'dbuser';
$password = 'dbpass';

$dbh = new PDO($dsn, $user, $password);
$dbh->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

// This will cause PDO to throw a PDOException (when the table doesn't exist)
$dbh->query("SELECT wrongcolumn FROM wrongtable");
```

The above example will output:

```
Fatal error: Uncaught PDOException: SQLSTATE[42S02]: Base table or view not found: 1146 Table
'testdb.wrongtable' doesn't exist in /tmp/pdo_test.php:10
Stack trace:
#0 /tmp/pdo_test.php(10): PDO->query('SELECT wrongcol...')
#1 {main}
  thrown in /tmp/pdo_test.php on line 10
```

> **Note** :
>
> PDO::__construct() will always throw a PDOException if the connection fails regardless of which `PDO::ATTR_ERRMODE` is currently set.

Example #2 Create a PDO instance and set the error mode from the constructor

```php
<?php
$dsn = 'mysql:dbname=test;host=127.0.0.1';
$user = 'googleguy';
$password = 'googleguy';

$dbh = new PDO($dsn, $user, $password, array(PDO::ATTR_ERRMODE => PDO::ERRMODE_WARNING));

// This will cause PDO to throw an error of level E_WARNING instead of an exception (when the table doesn't exist)
$dbh->query("SELECT wrongcolumn FROM wrongtable");
```

The above example will output:

```
Warning: PDO::query(): SQLSTATE[42S02]: Base table or view not found: 1146 Table 'test.wrongtable' doesn't exist in
/tmp/pdo_test.php on line 9
```

Large Objects (LOBs)

At some point in your application, you might find that you need to store "large" data in your database. Large typically means "around 4kb or more", although some databases can happily handle up to 32kb before data becomes "large". Large objects can be either textual or binary in nature. PDO allows you to work with this large data type by using the `PDO::PARAM_LOB` type code in your PDOStatement::bindParam() or PDOStatement::bindColumn() calls. `PDO::PARAM_LOB` tells PDO to map the data as a stream, so that you can manipulate it using the PHP Streams API.

Example #1 Displaying an image from a database

This example binds the LOB into the variable named $lob and then sends it to the browser using fpassthru(). Since the LOB is represented as a stream, functions such as fgets(), fread() and stream_get_contents() can be used on it.

```php
<?php
$db = new PDO('odbc:SAMPLE', 'db2inst1', 'ibmdb2');
$stmt = $db->prepare("select contenttype, imagedata from images where id=?");
$stmt->execute(array($_GET['id']));
$stmt->bindColumn(1, $type, PDO::PARAM_STR, 256);
$stmt->bindColumn(2, $lob, PDO::PARAM_LOB);
$stmt->fetch(PDO::FETCH_BOUND);

header("Content-Type: $type");
```

```
fpassthru($lob);
?>
```

Example #2 Inserting an image into a database

This example opens up a file and passes the file handle to PDO to insert it as a LOB. PDO will do its best to get the contents of the file up to the database in the most efficient manner possible.

```
<?php
$db = new PDO('odbc:SAMPLE', 'db2inst1', 'ibmdb2');
$stmt = $db->prepare("insert into images (id, contenttype, imagedata) values (?, ?, ?)");
$id = get_new_id(); // some function to allocate a new ID

// assume that we are running as part of a file upload form
// You can find more information in the PHP documentation

$fp = fopen($_FILES['file']['tmp_name'], 'rb');

$stmt->bindParam(1, $id);
$stmt->bindParam(2, $_FILES['file']['type']);
$stmt->bindParam(3, $fp, PDO::PARAM_LOB);

$db->beginTransaction();
$stmt->execute();
$db->commit();
?>
```

Example #3 Inserting an image into a database: Oracle

Oracle requires a slightly different syntax for inserting a lob from a file. It's also essential that you perform the insert under a transaction, otherwise your newly inserted LOB will be committed with a zero-length as part of the implicit commit that happens when the query is executed:

```
<?php
$db = new PDO('oci:', 'scott', 'tiger');
$stmt = $db->prepare("insert into images (id, contenttype, imagedata) " .
"VALUES (?, ?, EMPTY_BLOB()) RETURNING imagedata INTO ?");
$id = get_new_id(); // some function to allocate a new ID

// assume that we are running as part of a file upload form
// You can find more information in the PHP documentation

$fp = fopen($_FILES['file']['tmp_name'], 'rb');

$stmt->bindParam(1, $id);
$stmt->bindParam(2, $_FILES['file']['type']);
$stmt->bindParam(3, $fp, PDO::PARAM_LOB);

$db->beginTransaction();
$stmt->execute();
$db->commit();
?>
```

PDO

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

Introduction

Represents a connection between PHP and a database server.

Class synopsis

```
class PDO {
/* Constants */
public const int PARAM_NULL;
public const int PARAM_BOOL = 5;
public const int PARAM_INT = 1;
public const int PARAM_STR = 2;
public const int PARAM_LOB = 3;
public const int PARAM_STMT = 4;
public const int PARAM_INPUT_OUTPUT;
public const int PARAM_STR_NATL;
public const int PARAM_STR_CHAR;
public const int PARAM_EVT_ALLOC;
public const int PARAM_EVT_FREE;
public const int PARAM_EVT_EXEC_PRE;
public const int PARAM_EVT_EXEC_POST;
public const int PARAM_EVT_FETCH_PRE;
public const int PARAM_EVT_FETCH_POST;
public const int PARAM_EVT_NORMALIZE;
public const int FETCH_DEFAULT;
public const int FETCH_LAZY;
public const int FETCH_ASSOC;
public const int FETCH_NUM;
public const int FETCH_BOTH;
public const int FETCH_OBJ;
public const int FETCH_BOUND;
public const int FETCH_COLUMN;
public const int FETCH_CLASS;
public const int FETCH_INTO;
public const int FETCH_FUNC;
public const int FETCH_GROUP;
public const int FETCH_UNIQUE;
public const int FETCH_KEY_PAIR;
public const int FETCH_CLASSTYPE;
public const int FETCH_SERIALIZE;
public const int FETCH_PROPS_LATE;
public const int FETCH_NAMED;
public const int ATTR_AUTOCOMMIT;
public const int ATTR_PREFETCH;
public const int ATTR_TIMEOUT;
public const int ATTR_ERRMODE;
public const int ATTR_SERVER_VERSION;
public const int ATTR_CLIENT_VERSION;
public const int ATTR_SERVER_INFO;
public const int ATTR_CONNECTION_STATUS;
public const int ATTR_CASE;
public const int ATTR_CURSOR_NAME;
```

```
public const int ATTR_CURSOR;
public const int ATTR_ORACLE_NULLS;
public const int ATTR_PERSISTENT;
public const int ATTR_STATEMENT_CLASS;
public const int ATTR_FETCH_TABLE_NAMES;
public const int ATTR_FETCH_CATALOG_NAMES;
public const int ATTR_DRIVER_NAME;
public const int ATTR_STRINGIFY_FETCHES;
public const int ATTR_MAX_COLUMN_LEN;
public const int ATTR_EMULATE_PREPARES;
public const int ATTR_DEFAULT_FETCH_MODE;
public const int ATTR_DEFAULT_STR_PARAM;
public const int ERRMODE_SILENT;
public const int ERRMODE_WARNING;
public const int ERRMODE_EXCEPTION;
public const int CASE_NATURAL;
public const int CASE_LOWER;
public const int CASE_UPPER;
public const int NULL_NATURAL;
public const int NULL_EMPTY_STRING;
public const int NULL_TO_STRING;
public const string ERR_NONE;
public const int FETCH_ORI_NEXT;
public const int FETCH_ORI_PRIOR;
public const int FETCH_ORI_FIRST;
public const int FETCH_ORI_LAST;
public const int FETCH_ORI_ABS;
public const int FETCH_ORI_REL;
public const int CURSOR_FWDONLY;
public const int CURSOR_SCROLL;
/* Methods */
public function __construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function beginTransaction(): bool
public function commit(): bool
public static function connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function errorCode(): ?string
public function errorInfo(): array
public function exec(string $statement): int|false
public function getAttribute(int $attribute): mixed
public static function getAvailableDrivers(): array
public function inTransaction(): bool
```

```
public function lastInsertId(?string $name = null): string|false
public function prepare(string $query, array $options = []): PDOStatement|false
public function query(string $query, ?int $fetchMode = null): PDOStatement|false
public function query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno): PDOStatement|false
public function query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object): PDOStatement|false
public function quote(string $string, int $type = PDO::PARAM_STR): string|false
public function rollBack(): bool
public function setAttribute(int $attribute, mixed $value): bool
}
```

Changelog

Version	Description
8.5.0	Driver-specific constants of the **PDO** class have been deprecated in favor of the equivalent constants of the driver-specific subclasses (Pdo\Dblib, Pdo\Firebird, Pdo\Mysql, Pdo\Odbc, Pdo\Pgsql, Pdo\Sqlite).
8.4.0	The class constants are now typed.

Table of Contents

PDO::beginTransaction

PDO::beginTransaction

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::beginTransaction — Initiates a transaction

Description

public function **PDO::beginTransaction**(): bool

Turns off autocommit mode. While autocommit mode is turned off, changes made to the database via the PDO object instance are not committed until you end the transaction by calling PDO::commit(). Calling PDO::rollBack() will roll back all changes to the database and return the connection to autocommit mode.

Some databases, including MySQL, automatically issue an implicit COMMIT when a database definition language (DDL) statement such as DROP TABLE or CREATE TABLE is issued within a transaction. The implicit COMMIT will prevent you from rolling back any other changes within the transaction boundary.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

Throws a PDOException if there is already a transaction started or the driver does not support transactions.

> **Note** :
>
> An exception is raised even when the `PDO::ATTR_ERRMODE` attribute is not `PDO::ERRMODE_EXCEPTION`.

Examples

Example #1 Roll back a transaction

The following example begins a transaction and issues two statements that modify the database before rolling back the changes. On MySQL, however, the DROP TABLE statement automatically commits the transaction so that none of the changes in the transaction are rolled back.

```
<?php
/* Begin a transaction, turning off autocommit */
$dbh->beginTransaction();

/* Change the database schema and data */
$sth = $dbh->exec("DROP TABLE fruit");
$sth = $dbh->exec("UPDATE dessert
    SET name = 'hamburger'");

/* Recognize mistake and roll back changes */
$dbh->rollBack();

/* Database connection is now back in autocommit mode */
?>
```

See Also

- PDO::commit() - Commits a transaction
- PDO::rollBack() - Rolls back a transaction
- Transactions and auto-commit

PDO::commit

PDO::commit

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::commit — Commits a transaction

Description

public function **PDO::commit**(): bool

Commits a transaction, returning the database connection to autocommit mode until the next call to PDO::beginTransaction() starts a new transaction.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

Throws a PDOException if there is no active transaction.

> **Note** :
>
> An exception is raised even when the `PDO::ATTR_ERRMODE` attribute is not `PDO::ERRMODE_EXCEPTION`.

Examples

Example #1 Committing a basic transaction

```php
<?php
/* Begin a transaction, turning off autocommit */
$dbh->beginTransaction();

/* Insert multiple records on an all-or-nothing basis */
$sql = 'INSERT INTO fruit
    (name, colour, calories)
    VALUES (?, ?, ?)';

$sth = $dbh->prepare($sql);

foreach ($fruits as $fruit) {
    $sth->execute(array(
        $fruit->name,
        $fruit->colour,
        $fruit->calories,
    ));
}

/* Commit the changes */
$dbh->commit();

/* Database connection is now back in autocommit mode */
?>
```

Example #2 Committing a DDL transaction

```php
<?php
/* Begin a transaction, turning off autocommit */
$dbh->beginTransaction();

/* Change the database schema */
$sth = $dbh->exec("DROP TABLE fruit");

/* Commit the changes */
$dbh->commit();

/* Database connection is now back in autocommit mode */
?>
```

> **Note** :
>
> Not all databases will allow transactions to operate on DDL statements: some will generate errors, whereas others (including MySQL) will automatically commit the transaction after the first DDL statement has been encountered.

See Also

- PDO::beginTransaction() - Initiates a transaction
- PDO::rollBack() - Rolls back a transaction
- Transactions and auto-commit

PDO::connect

PDO::connect

(PHP 8 >= 8.4.0)

PDO::connect — Connect to a database and return a PDO subclass for drivers that support it

Description

```
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
```

Creates an instance of a PDO subclass for the database being connected to, if it exists, otherwise returns a generic PDO instance.

Parameters

`dsn`

The Data Source Name, or DSN, contains the information required to connect to the database.

In general, a DSN consists of the PDO driver name, followed by a colon, followed by the PDO driver-specific connection syntax. Further information is available from the PDO driver-specific documentation.

The `dsn` parameter supports three different methods of specifying the arguments required to create a database connection:

Driver invocation

`dsn` contains the full DSN.

URI invocation

`dsn` consists of **`uri:`** followed by a URI that defines the location of a file containing the DSN string. The URI can specify a local file or a remote URL.

```
uri:file:///path/to/dsnfile
```

> **Warning**
>
> The **`uri:`** DSN scheme is deprecated as of PHP 8.5.0 due to security concerns with DSNs coming from remote URIs.

Aliasing

`dsn` consists of a name `name` that maps to `pdo.dsn.name` in *php.ini* defining the DSN string.

> **Note** :
>
> The alias must be defined in *php.ini*, and not *.htaccess* or *httpd.conf*

`username`

The user name for the DSN string. This parameter is optional for some PDO drivers.

`password`

The password for the DSN string. This parameter is optional for some PDO drivers.

`options`

A key=>value array of driver-specific connection options.

Return Values

Returns an instance of a PDO subclass for the corresponding PDO driver if it exists, or a generic PDO instance.

Errors/Exceptions

A PDOException is thrown if the attempt to connect to the requested database fails, regardless of which **`PDO::ATTR_ERRMODE`** is currently set.

See Also

- Pdo\Dblib
- Pdo\Firebird
- Pdo\Mysql
- Pdo\Odbc
- Pdo\Pgsql
- Pdo\Sqlite
- PDO::__construct() - Creates a PDO instance representing a connection to a database

PDO::__construct

PDO::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::__construct — Creates a PDO instance representing a connection to a database

Description

```
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
```

Creates a PDO instance to represent a connection to the requested database.

Parameters

`dsn`

The Data Source Name, or DSN, contains the information required to connect to the database.

In general, a DSN consists of the PDO driver name, followed by a colon, followed by the PDO driver-specific connection syntax. Further information is available from the PDO driver-specific documentation.

The `dsn` parameter supports three different methods of specifying the arguments required to create a database connection:

Driver invocation

`dsn` contains the full DSN.

URI invocation

`dsn` consists of **`uri:`** followed by a URI that defines the location of a file containing the DSN string. The URI can specify a local file or a remote URL.

```
uri:file:///path/to/dsnfile
```

> **Warning**
>
> The **`uri:`** DSN scheme is deprecated as of PHP 8.5.0 due to security concerns with DSNs coming from remote URIs.

Aliasing

`dsn` consists of a name `name` that maps to `pdo.dsn.name` in *php.ini* defining the DSN string.

> **Note** :
>
> The alias must be defined in *php.ini*, and not *.htaccess* or *httpd.conf*

`username`

The user name for the DSN string. This parameter is optional for some PDO drivers.

`password`

The password for the DSN string. This parameter is optional for some PDO drivers.

`options`

A key=>value array of driver-specific connection options.

Errors/Exceptions

A PDOException is thrown if the attempt to connect to the requested database fails, regardless of which `PDO::ATTR_ERRMODE` is currently set.

Changelog

Version	Description
8.5.0	The `uri:` DSN scheme is now deprecated.

Examples

Example #1 Create a PDO instance via driver invocation

```php
<?php

$dsn = 'mysql:dbname=testdb;host=127.0.0.1';
$user = 'dbuser';
$password = 'dbpass';

$dbh = new PDO($dsn, $user, $password);

?>
```

Example #2 Create a PDO instance via URI invocation

The following example assumes that the file */usr/local/dbconnect* exists with file permissions that enable PHP to read the file. The file contains the PDO DSN to connect to a DB2 database through the PDO_ODBC driver:

```
odbc:DSN=SAMPLE;UID=john;PWD=mypass
```

The PHP script can then create a database connection by simply passing the `uri:` parameter and pointing to the file URI:

```php
<?php

$dsn = 'uri:file:///usr/local/dbconnect';
$user = '';
$password = '';

$dbh = new PDO($dsn, $user, $password);

?>
```

Example #3 Create a PDO instance using an alias

The following example assumes that *php.ini* contains the following entry to enable a connection to a MySQL database using only the alias `mydb`:

```
[PDO]
pdo.dsn.mydb="mysql:dbname=testdb;host=localhost"
```

```php
<?php

$dsn = 'mydb';
$user = '';
$password = '';

$dbh = new PDO($dsn, $user, $password);

?>
```

PDO::errorCode

PDO::errorCode

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::errorCode — Fetch the SQLSTATE associated with the last operation on the database handle

Description

public function **PDO::errorCode**(): ?string

Parameters

This function has no parameters.

Return Values

Returns an SQLSTATE, a five-character alphanumeric identifier defined in the ANSI SQL-92 standard. Briefly, an SQLSTATE consists of a two-character class value followed by a three-character subclass value. A class value of 01 indicates a warning and is accompanied by a return code of SQL_SUCCESS_WITH_INFO. Class values other than '01', except for the class 'IM', indicate an error. The class 'IM' is specific to warnings and errors that derive from the implementation of PDO (or perhaps ODBC, if you're using the ODBC driver) itself. The subclass value '000' in any class indicates that there is no subclass for that SQLSTATE.

PDO::errorCode() only retrieves error codes for operations performed directly on the database handle. If you create a PDOStatement object through PDO::prepare() or PDO::query() and invoke an error on the statement handle, **PDO::errorCode()** will not reflect that error. You must call PDOStatement::errorCode() to return the error code for an operation performed on a particular statement handle.

Returns `null` if no operation has been run on the database handle.

Examples

Example #1 Retrieving an SQLSTATE code

```php
<?php
/* Provoke an error -- the BONES table does not exist */
$dbh->exec("INSERT INTO bones(skull) VALUES ('lucy')");

echo "\nPDO::errorCode(): ", $dbh->errorCode();
?>
```

The above example will output:

```
PDO::errorCode(): 42S02
```

See Also

- PDO::errorInfo() - Fetch extended error information associated with the last operation on the database handle
- PDOStatement::errorCode() - Fetch the SQLSTATE associated with the last operation on the statement handle
- PDOStatement::errorInfo() - Fetch extended error information associated with the last operation on the statement handle

PDO::errorInfo

PDO::errorInfo

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::errorInfo — Fetch extended error information associated with the last operation on the database handle

Description

public function **PDO::errorInfo**(): array

Parameters

This function has no parameters.

Return Values

PDO::errorInfo() returns an array of error information about the last operation performed by this database handle. The array consists of at least the following fields:

Element	Information
0	SQLSTATE error code (a five-character alphanumeric identifier defined in the ANSI SQL standard).
1	Driver-specific error code.
2	Driver-specific error message.

> **Note** :
>
> If the SQLSTATE error code is not set or there is no driver-specific error, the elements following element 0 will be set to `null`.

PDO::errorInfo() only retrieves error information for operations performed directly on the database handle. If you create a PDOStatement object through PDO::prepare() or PDO::query() and invoke an error on the statement handle, **PDO::errorInfo()** will not reflect the error from the statement handle. You must call

PDOStatement::errorInfo() to return the error information for an operation performed on a particular statement handle.

Examples

Example #1 Displaying errorInfo() fields for a PDO_ODBC connection to a DB2 database

```php
<?php
/* Provoke an error -- bogus SQL syntax */
$stmt = $dbh->prepare('bogus sql');
if (!$stmt) {
    echo "\nPDO::errorInfo():\n";
    print_r($dbh->errorInfo());
}
?>
```

The above example will output:

```
PDO::errorInfo():
Array
(
    [0] => HY000
    [1] => 1
    [2] => near "bogus": syntax error
)
```

See Also

- PDO::errorCode() - Fetch the SQLSTATE associated with the last operation on the database handle
- PDOStatement::errorCode() - Fetch the SQLSTATE associated with the last operation on the statement handle
- PDOStatement::errorInfo() - Fetch extended error information associated with the last operation on the statement handle

PDO::exec

PDO::exec

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::exec — Execute an SQL statement and return the number of affected rows

Description

public function **PDO::exec**(string `$statement`): int|false

PDO::exec() executes an SQL statement in a single function call, returning the number of rows affected by the statement.

PDO::exec() does not return results from a SELECT statement. For a SELECT statement that you only need to issue once during your program, consider issuing PDO::query(). For a statement that you need to issue multiple

times, prepare a PDOStatement object with PDO::prepare() and issue the statement with PDOStatement::execute().

Parameters

`statement`

The SQL statement to prepare and execute.

Data inside the query should be properly escaped.

Return Values

PDO::exec() returns the number of rows that were modified or deleted by the SQL statement you issued. If no rows were affected, **PDO::exec()** returns `0`.

> **Warning**
>
> This function may return Boolean **`false`**, but may also return a non-Boolean value which evaluates to **`false`**. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

The following example incorrectly relies on the return value of **PDO::exec()**, wherein a statement that affected 0 rows results in a call to die():

```
<?php
$db->exec() or die(print_r($db->errorInfo(), true)); // incorrect
?>
```

Errors/Exceptions

Emits an error with level **`E_WARNING`** if the attribute **`PDO::ATTR_ERRMODE`** is set to **`PDO::ERRMODE_WARNING`**.

Throws a PDOException if the attribute **`PDO::ATTR_ERRMODE`** is set to **`PDO::ERRMODE_EXCEPTION`**.

Examples

Example #1 Issuing a DELETE statement

Count the number of rows deleted by a DELETE statement with no WHERE clause.

```
<?php
$dbh = new PDO('odbc:sample', 'db2inst1', 'ibmdb2');

/* Delete all rows from the FRUIT table */
$count = $dbh->exec("DELETE FROM fruit");

/* Return number of rows that were deleted */
print "Deleted $count rows.\n";
?>
```

The above example will output:

```

```

```
Deleted 1 rows.
```

See Also

- PDO::prepare() - Prepares a statement for execution and returns a statement object
- PDO::query() - Prepares and executes an SQL statement without placeholders
- PDOStatement::execute() - Executes a prepared statement

PDO::getAttribute

PDO::getAttribute

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDO::getAttribute — Retrieve a database connection attribute

Description

public function **PDO::getAttribute**(int `$attribute`): mixed

This function returns the value of a database connection attribute. To retrieve PDOStatement attributes, refer to PDOStatement::getAttribute().

Note that some database/driver combinations may not support all of the database connection attributes.

Parameters

`attribute`

One of the `PDO::ATTR_*` constants. The generic attributes that apply to database connections are as follows:

- `PDO::ATTR_CASE`
- `PDO::ATTR_DEFAULT_FETCH_MODE`
- `PDO::ATTR_DRIVER_NAME`
- `PDO::ATTR_ERRMODE`
- `PDO::ATTR_ORACLE_NULLS`
- `PDO::ATTR_PERSISTENT`
- `PDO::ATTR_STATEMENT_CLASS`
- `PDO::ATTR_STRINGIFY_FETCHES`

Some drivers may make use of additional driver specific attributes. Note that driver specific attributes *must not* be used with other drivers.

Return Values

A successful call returns the value of the requested PDO attribute. An unsuccessful call returns **`null`**.

Errors/Exceptions

PDO::getAttribute() may throw a PDOException when the underlying driver does not support the requested `attribute`.

Changelog

Version	Description
8.4.0	It is now possible to fetch the value of the `PDO::ATTR_STRINGIFY_FETCHES` attribute.

Examples

Example #1 Retrieving database connection attributes

```php
<?php
$conn = new PDO('odbc:sample', 'db2inst1', 'ibmdb2');
$attributes = array(
    "AUTOCOMMIT", "ERRMODE", "CASE", "CLIENT_VERSION", "CONNECTION_STATUS",
    "ORACLE_NULLS", "PERSISTENT", "PREFETCH", "SERVER_INFO", "SERVER_VERSION",
    "TIMEOUT"
);

foreach ($attributes as $val) {
    echo "PDO::ATTR_$val: ";
    echo $conn->getAttribute(constant("PDO::ATTR_$val")) . "\n";
}
?>
```

See Also

- PDO::setAttribute() - Set an attribute
- PDOStatement::getAttribute() - Retrieve a statement attribute
- PDOStatement::setAttribute() - Set a statement attribute

PDO::getAvailableDrivers

PDO::getAvailableDrivers

pdo_drivers

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 1.0.3)

PDO::getAvailableDrivers -- pdo_drivers — Return an array of available PDO drivers

Description

public static function **PDO::getAvailableDrivers**(): array
function **pdo_drivers**(): array

This function returns all currently available PDO drivers which can be used in the `DSN` parameter of PDO::__construct().

Parameters

This function has no parameters.

Return Values

PDO::getAvailableDrivers() returns an array of PDO driver names. If no drivers are available, it returns an empty array.

Examples

Example #1 A PDO::getAvailableDrivers() example

```php
<?php
print_r(PDO::getAvailableDrivers());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => mysql
    [1] => sqlite
)
```

PDO::inTransaction

PDO::inTransaction

(PHP 5 >= 5.3.3, Bundled pdo_pgsql, PHP 7, PHP 8)

PDO::inTransaction — Checks if inside a transaction

Description

public function **PDO::inTransaction**(): bool

Checks if a transaction is currently active within the driver. This method only works for database drivers that support transactions.

Parameters

This function has no parameters.

Return Values

Returns `true` if a transaction is currently active, and `false` if not.

PDO::lastInsertId

PDO::lastInsertId

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::lastInsertId — Returns the ID of the last inserted row or sequence value

Description

public function **PDO::lastInsertId**(?string `$name` = `null`): string|false

Returns the ID of the last inserted row, or the last value from a sequence object, depending on the underlying driver. For example, PDO_PGSQL allows the name of any sequence object to be specified for the `name` parameter.

> **Note** :
>
> This method may not return a meaningful or consistent result across different PDO drivers, because the underlying database may not even support the notion of auto-increment fields or sequences.

Parameters

`name`

Name of the sequence object from which the ID should be returned.

Return Values

If a sequence name was not specified for the `name` parameter, **PDO::lastInsertId()** returns a string representing the row ID of the last row that was inserted into the database.

If a sequence name was specified for the `name` parameter, **PDO::lastInsertId()** returns a string representing the last value retrieved from the specified sequence object.

If the PDO driver does not support this capability, **PDO::lastInsertId()** triggers an `IM001` SQLSTATE.

Errors/Exceptions

Emits an error with level `E_WARNING` if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_WARNING`.

Throws a PDOException if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_EXCEPTION`.

PDO::prepare

PDO::prepare

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PHP 8,PECL pdo >= 0.1.0)

PDO::prepare — Prepares a statement for execution and returns a statement object

Description

public function **PDO::prepare**(string `$query`, array `$options = []`): PDOStatement|false

Prepares an SQL statement to be executed by the PDOStatement::execute() method. The statement template can contain zero or more named (:name) or question mark (?) parameter markers for which real values will be substituted when the statement is executed. Both named and question mark parameter markers cannot be used within the same statement template; only one or the other parameter style. Use these parameters to bind any user-input, do not include the user-input directly in the query.

You must include a unique parameter marker for each value you wish to pass in to the statement when you call PDOStatement::execute(). You cannot use a named parameter marker of the same name more than once in a prepared statement, unless emulation mode is on.

> **Note** :

> Parameter markers can represent a complete data literal only. Neither part of a literal, nor keyword, nor identifier, nor whatever arbitrary query part can be bound using parameters. For example, you cannot bind multiple values to a single parameter in the IN() clause of an SQL statement.

Calling **PDO::prepare()** and PDOStatement::execute() for statements that will be issued multiple times with different parameter values optimizes the performance of your application by allowing the driver to negotiate client and/or server side caching of the query plan and meta information. Also, calling **PDO::prepare()** and PDOStatement::execute() helps to prevent SQL injection attacks by eliminating the need to manually quote and escape the parameters.

PDO will emulate prepared statements/bound parameters for drivers that do not natively support them, and can also rewrite named or question mark style parameter markers to something more appropriate, if the driver supports one style but not the other.

> **Note** :
>
> Before PHP 8.4.0, the parser used for emulated prepared statements and for rewriting named or question mark style parameters supported the non standard backslash escapes for single- and double quotes. That meant that terminating quotes immediately preceded by a backslash were not recognized as such, which could result in wrong detection of parameters causing the prepared statement to fail when it was executed. A work-around was to not use emulated prepares for such SQL queries, and to avoid rewriting of parameters by using a parameter style which is natively supported by the driver.

As of PHP 7.4.0, question marks can be escaped by doubling them. That means that the `??` string will be translated to `?` when sending the query to the database.

As of PHP 8.4.0, PDO uses driver-specific SQL parsers to locate parameter markers, so that markers appearing inside string literals or comments are not mistaken for placeholders. The default parser recognizes single- and double-quoted string literals, as well as two-dashes and non-nested C-style comments. Individual drivers may provide a parser that recognizes additional or different syntax; refer to the driver-specific documentation for details.

Parameters

`query`

This must be a valid SQL statement template for the target database server.

`options`

This array holds one or more key=>value pairs to set attribute values for the PDOStatement object that this method returns. You would most commonly use this to set the `PDO::ATTR_CURSOR` value to `PDO::CURSOR_SCROLL` to request a scrollable cursor. Some drivers have driver-specific options that may be set at prepare-time.

Return Values

If the database server successfully prepares the statement, **PDO::prepare()** returns a PDOStatement object. If the database server cannot successfully prepare the statement, **PDO::prepare()** returns **`false`** or emits PDOException (depending on error handling).

> **Note** :
>
> Emulated prepared statements do not communicate with the database server so **PDO::prepare()** does not check the statement.

Errors/Exceptions

Emits an error with level `E_WARNING` if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_WARNING`.

Throws a PDOException if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_EXCEPTION`.

Examples

Example #1 SQL statement template with named parameters

```php
<?php
/* Execute a prepared statement by passing an array of values */
$sql = 'SELECT name, colour, calories
    FROM fruit
    WHERE calories < :calories AND colour = :colour';
$sth = $dbh->prepare($sql, [PDO::ATTR_CURSOR => PDO::CURSOR_FWDONLY]);
$sth->execute(['calories' => 150, 'colour' => 'red']);
$red = $sth->fetchAll();
/* Array keys can be prefixed with colons ":" too (optional) */
$sth->execute([':calories' => 175, ':colour' => 'yellow']);
$yellow = $sth->fetchAll();
?>
```

Example #2 SQL statement template with question mark parameters

```php
<?php
/* Execute a prepared statement by passing an array of values */
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < ? AND colour = ?');
$sth->execute([150, 'red']);
$red = $sth->fetchAll();
$sth->execute([175, 'yellow']);
$yellow = $sth->fetchAll();
?>
```

Example #3 SQL statement template with question mark escaped

```php
<?php
/* note: this is only valid on PostgreSQL databases */
$sth = $dbh->prepare('SELECT * FROM issues WHERE tag::jsonb ?? ?');
$sth->execute(['feature']);
$featureIssues = $sth->fetchAll();
$sth->execute(['performance']);
$performanceIssues = $sth->fetchAll();
?>
```

See Also

- PDO::exec() - Execute an SQL statement and return the number of affected rows

- PDO::query() - Prepares and executes an SQL statement without placeholders
- PDOStatement::execute() - Executes a prepared statement

PDO::query

PDO::query

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDO::query — Prepares and executes an SQL statement without placeholders

Description

public function **PDO::query**(string `$query`, ?int `$fetchMode` = `null`): PDOStatement|false
public function **PDO::query**(string `$query`, ?int `$fetchMode` = PDO::FETCH_COLUMN, int `$colno`): PDOStatement|false
public function **PDO::query**(
    string `$query`,
    ?int `$fetchMode` = PDO::FETCH_CLASS,
    string `$classname`,
    array `$constructorArgs`
): PDOStatement|false
public function **PDO::query**(string `$query`, ?int `$fetchMode` = PDO::FETCH_INTO, object `$object`): PDOStatement|false

PDO::query() prepares and executes an SQL statement in a single function call, returning the statement as a PDOStatement object.

For a query that you need to issue multiple times, you will realize better performance if you prepare a PDOStatement object using PDO::prepare() and issue the statement with multiple calls to PDOStatement::execute().

If you do not fetch all of the data in a result set before issuing your next call to **PDO::query()**, your call may fail. Call PDOStatement::closeCursor() to release the database resources associated with the PDOStatement object before issuing your next call to **PDO::query()**.

> **Note** :
>
> If the `query` contains placeholders, the statement must be prepared and executed separately using PDO::prepare() and PDOStatement::execute() methods.

Parameters

`query`

The SQL statement to prepare and execute.

If the SQL contains placeholders, PDO::prepare() and PDOStatement::execute() must be used instead. Alternatively, the SQL can be prepared manually before calling **PDO::query()**, with the data properly formatted using PDO::quote() if the driver supports it.

`fetchMode`

The default fetch mode for the returned PDOStatement. It must be one of the `PDO::FETCH_*` constants.

If this argument is passed to the function, the remaining arguments will be treated as though PDOStatement::setFetchMode() was called on the resultant statement object. The subsequent arguments vary depending on the selected fetch mode.

Return Values

Returns a PDOStatement object or `false` on failure.

Errors/Exceptions

Emits an error with level `E_WARNING` if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_WARNING`.

Throws a PDOException if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_EXCEPTION`.

Examples

Example #1 SQL with no placeholders can be executed using PDO::query()

```php
<?php
$sql = 'SELECT name, color, calories FROM fruit ORDER BY name';
foreach ($conn->query($sql) as $row) {
    print $row['name'] . "\t";
    print $row['color'] . "\t";
    print $row['calories'] . "\n";
}
?>
```

The above example will output:

```
apple   red     150
banana  yellow  250
kiwi    brown   75
lemon   yellow  25
orange  orange  300
pear    green   150
watermelon      pink    90
```

See Also

- PDO::exec() - Execute an SQL statement and return the number of affected rows
- PDO::prepare() - Prepares a statement for execution and returns a statement object
- PDOStatement::execute() - Executes a prepared statement

PDO::quote

PDO::quote

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.1)

PDO::quote — Quotes a string for use in a query

Description

public function **PDO::quote**(string `$string`, int `$type` = **`PDO::PARAM_STR`**): string|false

PDO::quote() places quotes around the input string and escapes special characters within the input string, using a quoting style appropriate to the underlying driver.

If you are using this function to build SQL statements, you are *strongly* recommended to use PDO::prepare() to prepare SQL statements with bound parameters instead of using **PDO::quote()** to interpolate user input into an SQL statement. Prepared statements with bound parameters are not only more portable, more convenient, immune to SQL injection, but are often much faster to execute than interpolated queries, as both the server and client side can cache a compiled form of the query.

Not all PDO drivers implement this method (notably PDO_ODBC). Consider using prepared statements instead.

> **Caution**
>
> **Security: the default character set**
>
> The character set must be set either on the server level, or within the database connection itself (depending on the driver) for it to affect **PDO::quote()**. See the driver-specific documentation for more information.

Parameters

`string`

The string to be quoted.

`type`

Provides a hint as to the type of data for drivers that have alternate quoting styles. For example **`PDO_PARAM_LOB`** will tell the driver to escape binary data.

Return Values

Returns a quoted string that is theoretically safe to pass into an SQL statement. Returns **`false`** if the driver does not support quoting in this way.

Changelog

Version	Description
8.5.0	When using the SQLite driver, a PDOException is now thrown or a warning is emitted, depending on the error mode, if the `string` contains a null byte; previously the string was silently truncated at the null byte.

Examples

Example #1 Quoting a normal string

```php
<?php
$conn = new PDO('sqlite:/home/lynn/music.sql3');

/* Simple string */
$string = 'Nice';
print "Unquoted string: $string\n";
```

```
print "Quoted string: " . $conn->quote($string) . "\n";
?>
```

The above example will output:

```
Unquoted string: Nice
Quoted string: 'Nice'
```

Example #2 Quoting a dangerous string

```
<?php
$conn = new PDO('sqlite:/home/lynn/music.sql3');

/* Dangerous string */
$string = 'Naughty \' string';
print "Unquoted string: $string\n";
print "Quoted string:" . $conn->quote($string) . "\n";
?>
```

The above example will output:

```
Unquoted string: Naughty ' string
Quoted string: 'Naughty '' string'
```

Example #3 Quoting a complex string

```
<?php
$conn = new PDO('sqlite:/home/lynn/music.sql3');

/* Complex string */
$string = "Co'mpl''ex \"st'\"ring";
print "Unquoted string: $string\n";
print "Quoted string: " . $conn->quote($string) . "\n";
?>
```

The above example will output:

```
Unquoted string: Co'mpl''ex "st'"ring
Quoted string: 'Co''mpl''''ex "st'""ring'
```

See Also

- PDO::prepare() - Prepares a statement for execution and returns a statement object
- PDOStatement::execute() - Executes a prepared statement

PDO::rollBack

PDO::rollBack

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::rollBack — Rolls back a transaction

Description

public function **PDO::rollBack**(): bool

Rolls back the current transaction, as initiated by PDO::beginTransaction().

If the database was set to autocommit mode, this function will restore autocommit mode after it has rolled back the transaction.

Some databases, including MySQL, automatically issue an implicit COMMIT when a database definition language (DDL) statement such as DROP TABLE or CREATE TABLE is issued within a transaction. The implicit COMMIT will prevent you from rolling back any other changes within the transaction boundary.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

Throws a PDOException if there is no active transaction.

> **Note** :
>
> An exception is raised even when the `PDO::ATTR_ERRMODE` attribute is not `PDO::ERRMODE_EXCEPTION`.

Examples

Example #1 Roll back a transaction

The following example begins a transaction and issues two statements that modify the database before rolling back the changes. On MySQL, however, the DROP TABLE statement automatically commits the transaction so that none of the changes in the transaction are rolled back.

```
<?php
/* Begin a transaction, turning off autocommit */
$dbh->beginTransaction();

/* Change the database schema and data */
$sth = $dbh->exec("DROP TABLE fruit");
```

```php
$sth = $dbh->exec("UPDATE dessert
    SET name = 'hamburger'");

/* Recognize mistake and roll back changes */
$dbh->rollBack();

/* Database connection is now back in autocommit mode */
?>
```

See Also

- PDO::beginTransaction() - Initiates a transaction
- PDO::commit() - Commits a transaction
- Transactions and auto-commit

PDO::setAttribute

PDO::setAttribute

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDO::setAttribute — Set an attribute

Description

public function **PDO::setAttribute**(int `$attribute`, mixed `$value`): bool

Sets an attribute on the database handle. Some available generic attributes are listed below; some drivers may make use of additional driver specific attributes. Note that driver specific attributes *must not* be used with other drivers.

`PDO::ATTR_CASE`

Force column names to a specific case. Can take one of the following values:

`PDO::CASE_LOWER`

Force column names to lower case.

`PDO::CASE_NATURAL`

Leave column names as returned by the database driver.

`PDO::CASE_UPPER`

Force column names to upper case.

`PDO::ATTR_ERRMODE`

Error reporting mode of PDO. Can take one of the following values:

`PDO::ERRMODE_SILENT`

Only sets error codes.

`PDO::ERRMODE_WARNING`

Raises **`E_WARNING`** diagnostics.

`PDO::ERRMODE_EXCEPTION`

Throws PDOExceptions.

`PDO::ATTR_ORACLE_NULLS`

> **Note** :
>
> This attribute is available with all drivers, not just Oracle.

Determines if and how `null` and empty strings should be converted. Can take one of the following values:

`PDO::NULL_NATURAL`

No conversion takes place.

`PDO::NULL_EMPTY_STRING`

Empty strings get converted to `null`.

`PDO::NULL_TO_STRING`

`null` gets converted to an empty string.

`PDO::ATTR_STRINGIFY_FETCHES`

Controls whether fetched values (except `null`) are converted to strings. Takes a value of type bool: `true` to enable and `false` to disable (default). `null` values remain unchanged unless `PDO::ATTR_ORACLE_NULLS` is set to `PDO::NULL_TO_STRING`.

`PDO::ATTR_STATEMENT_CLASS`

Set user-supplied statement class derived from PDOStatement. Requires `array(string classname, array(mixed constructor_args))`.

> **Caution**
>
> Cannot be used with persistent PDO instances.

`PDO::ATTR_TIMEOUT`

Specifies the timeout duration in seconds. Takes a value of type int.

> **Note** :
>
> Not all drivers support this option, and its meaning may differ from driver to driver. For example, SQLite will wait for up to this time value before giving up on obtaining a writable lock, but other drivers may interpret this as a connection or a read timeout interval.

`PDO::ATTR_AUTOCOMMIT`

> **Note** :

> Only available for the OCI, Firebird, and MySQL drivers.

Whether to autocommit every single statement. Takes a value of type bool: `true` to enable and `false` to disable. By default, `true`.

`PDO::ATTR_EMULATE_PREPARES`

> **Note** :
>
> Only available for the OCI, Firebird, and MySQL drivers.

Whether enable or disable emulation of prepared statements. Some drivers do not support prepared statements natively or have limited support for them. If set to `true` PDO will always emulate prepared statements, otherwise PDO will attempt to use native prepared statements. In case the driver cannot successfully prepare the current query, PDO will always fall back to emulating the prepared statement.

`PDO::MYSQL_ATTR_USE_BUFFERED_QUERY`

> **Note** :
>
> Only available for the MySQL driver.

Whether to use buffered queries. Takes a value of type bool: `true` to enable and `false` to disable. By default, `true`.

`PDO::ATTR_DEFAULT_FETCH_MODE`

Set the default fetch mode. A description of the modes and how to use them is available in the PDOStatement::fetch() documentation.

Parameters

`attribute`

The attribute to modify.

`value`

The value to set the `attribute`, might require a specific type depending on the attribute.

Return Values

Returns `true` on success or `false` on failure.

See Also

- PDO::getAttribute() - Retrieve a database connection attribute
- PDOStatement::getAttribute() - Retrieve a statement attribute
- PDOStatement::setAttribute() - Set a statement attribute

PDOStatement

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 1.0.0)

Introduction

Represents a prepared statement and, after the statement is executed, an associated result set.

Class synopsis

```
class PDOStatement implements IteratorAggregate {
/* Properties */
public string $queryString;
/* Methods */
public function bindColumn(
    string|int $column,
    mixed &$var,
    int $type = PDO::PARAM_STR,
    int $maxLength = 0,
    mixed $driverOptions = null
): bool
public function bindParam(
    string|int $param,
    mixed &$var,
    int $type = PDO::PARAM_STR,
    int $maxLength = 0,
    mixed $driverOptions = null
): bool
public function bindValue(string|int $param, mixed $value, int $type = PDO::PARAM_STR): bool
public function closeCursor(): bool
public function columnCount(): int
public function debugDumpParams(): ?bool
public function errorCode(): ?string
public function errorInfo(): array
public function execute(?array $params = null): bool
public function fetch(int $mode = PDO::FETCH_DEFAULT, int $cursorOrientation = PDO::FETCH_ORI_NEXT, int
$cursorOffset = 0): mixed
public function fetchAll(int $mode = PDO::FETCH_DEFAULT): array
public function fetchAll(int $mode = PDO::FETCH_COLUMN, int $column): array
public function fetchAll(int $mode = PDO::FETCH_CLASS, string $class, ?array $constructorArgs): array
public function fetchAll(int $mode = PDO::FETCH_FUNC, callable $callback): array
public function fetchColumn(int $column = 0): mixed
public function fetchObject(?string $class = "stdClass", array $constructorArgs = []): object|false
public function getAttribute(int $name): mixed
public function getColumnMeta(int $column): array|false
public function getIterator(): Iterator
public function nextRowset(): bool
public function rowCount(): int
public function setAttribute(int $attribute, mixed $value): bool
public function setFetchMode(int $mode): true
public function setFetchMode(int $mode = PDO::FETCH_COLUMN, int $colno): true
public function setFetchMode(int $mode = PDO::FETCH_CLASS, string $class, ?array $constructorArgs = null): true
public function setFetchMode(int $mode = PDO::FETCH_INTO, object $object): true
}
```

Properties

queryString

Used query string.

Changelog

Version	Description
8.0.0	**PDOStatement** implements IteratorAggregate now instead of Traversable.

Table of Contents

PDOStatement::bindColumn

PDOStatement::bindColumn

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::bindColumn — Bind a column to a PHP variable

Description

```
public function PDOStatement::bindColumn(
    string|int $column,
    mixed &$var,
    int $type = PDO::PARAM_STR,
    int $maxLength = 0,
    mixed $driverOptions = null
): bool
```

PDOStatement::bindColumn() arranges to have a particular variable bound to a given column in the result-set from a query. Each call to PDOStatement::fetch() or PDOStatement::fetchAll() will update all the variables that are bound to columns.

> **Note** :
>
> Since information about the columns is not always available to PDO until the statement is executed, portable applications should call this function *after* PDOStatement::execute().
>
> However, to be able to bind a LOB column as a stream when using the *PgSQL driver*, applications should call this method *before* calling PDOStatement::execute(), otherwise the large object OID will be returned as an integer.

Parameters

`column`

Number of the column (1-indexed) or name of the column in the result set. If using the column name, be aware that the name should match the case of the column, as returned by the driver.

`var`

Name of the PHP variable to which the column will be bound.

`type`

Data type of the parameter, specified by the `PDO::PARAM_*` constants.

`maxLength`

A hint for pre-allocation.

`driverOptions`

Optional parameter(s) for the driver.

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

Emits an error with level **`E_WARNING`** if the attribute **`PDO::ATTR_ERRMODE`** is set to **`PDO::ERRMODE_WARNING`**.

Throws a PDOException if the attribute **`PDO::ATTR_ERRMODE`** is set to **`PDO::ERRMODE_EXCEPTION`**.

Examples

Example #1 Binding result set output to PHP variables

Binding columns in the result set to PHP variables is an effective way to make the data contained in each row immediately available to your application. The following example demonstrates how PDO allows you to bind and retrieve columns with a variety of options and with intelligent defaults.

```php
<?php
$stmt = $dbh->prepare('SELECT name, colour, calories FROM fruit');
$stmt->execute();

/* Bind by column number */
$stmt->bindColumn(1, $name);
$stmt->bindColumn(2, $colour);

/* Bind by column name */
$stmt->bindColumn('calories', $cals);

while ($stmt->fetch(PDO::FETCH_BOUND)) {
    print $name . "\t" . $colour . "\t" . $cals . "\n";
}
```

The above example will output something similar to:

```
apple   red     150
banana  yellow  175
kiwi    green   75
orange  orange  150
mango   red     200
strawberry      red     25
```

See Also

- PDOStatement::execute() - Executes a prepared statement
- PDOStatement::fetch() - Fetches the next row from a result set
- PDOStatement::fetchAll() - Fetches the remaining rows from a result set
- PDOStatement::fetchColumn() - Returns a single column from the next row of a result set

PDOStatement::bindParam

PDOStatement::bindParam

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::bindParam — Binds a parameter to the specified variable name

Description

```
public function PDOStatement::bindParam(
    string|int $param,
    mixed &$var,
    int $type = PDO::PARAM_STR,
    int $maxLength = 0,
    mixed $driverOptions = null
): bool
```

Binds a PHP variable to a corresponding named or question mark placeholder in the SQL statement that was used to prepare the statement. Unlike PDOStatement::bindValue(), the variable is bound as a reference and will only be evaluated at the time that PDOStatement::execute() is called.

Most parameters are input parameters, that is, parameters that are used in a read-only fashion to build up the query (but may nonetheless be cast according to `type`). Some drivers support the invocation of stored procedures that return data as output parameters, and some also as input/output parameters that both send in data and are updated to receive it.

Parameters

`param`

Parameter identifier. For a prepared statement using named placeholders, this will be a parameter name of the form *:name*. For a prepared statement using question mark placeholders, this will be the 1-indexed position of the parameter.

`var`

Name of the PHP variable to bind to the SQL statement parameter.

`type`

Explicit data type for the parameter using the `PDO::PARAM_*` constants. To return an INOUT parameter from a stored procedure, use the bitwise OR operator to set the `PDO::PARAM_INPUT_OUTPUT` bits for the `type` parameter.

`maxLength`

Length of the data type. To indicate that a parameter is an OUT parameter from a stored procedure, you must explicitly set the length. Meaningful only when `type` parameter is `PDO::PARAM_INPUT_OUTPUT`.

`driverOptions`

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

Emits an error with level `E_WARNING` if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_WARNING`.

Throws a PDOException if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_EXCEPTION`.

Examples

Example #1 Execute a prepared statement with named placeholders

```php
<?php
/* Execute a prepared statement by binding PHP variables */
$calories = 150;
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < :calories AND colour = :colour');
$sth->bindParam('calories', $calories, PDO::PARAM_INT);
/* Names can be prefixed with colons ":" too (optional) */
$sth->bindParam(':colour', $colour, PDO::PARAM_STR);
$sth->execute();
?>
```

Example #2 Execute a prepared statement with question mark placeholders

```php
<?php
/* Execute a prepared statement by binding PHP variables */
$calories = 150;
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < ? AND colour = ?');
$sth->bindParam(1, $calories, PDO::PARAM_INT);
$sth->bindParam(2, $colour, PDO::PARAM_STR);
$sth->execute();
?>
```

Example #3 Call a stored procedure with an INOUT parameter

```php
<?php
/* Call a stored procedure with an INOUT parameter */
$colour = 'red';
$sth = $dbh->prepare('CALL puree_fruit(?)');
$sth->bindParam(1, $colour, PDO::PARAM_STR|PDO::PARAM_INPUT_OUTPUT, 12);
$sth->execute();
print "After pureeing fruit, the colour is: $colour";
?>
```

See Also

- PDO::prepare() - Prepares a statement for execution and returns a statement object
- PDOStatement::execute() - Executes a prepared statement
- PDOStatement::bindValue() - Binds a value to a parameter

PDOStatement::bindValue

PDOStatement::bindValue

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 1.0.0)

PDOStatement::bindValue — Binds a value to a parameter

Description

public function **PDOStatement::bindValue**(string|int `$param`, mixed `$value`, int `$type` = **`PDO::PARAM_STR`**): bool

Binds a value to a corresponding named or question mark placeholder in the SQL statement that was used to prepare the statement.

Parameters

`param`

Parameter identifier. For a prepared statement using named placeholders, this will be a parameter name of the form *:name*. For a prepared statement using question mark placeholders, this will be the 1-indexed position of the parameter.

`value`

The value to bind to the parameter.

`type`

Explicit data type for the parameter using the `PDO::PARAM_*` constants.

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

Emits an error with level **`E_WARNING`** if the attribute **`PDO::ATTR_ERRMODE`** is set to **`PDO::ERRMODE_WARNING`**.

Throws a PDOException if the attribute **`PDO::ATTR_ERRMODE`** is set to **`PDO::ERRMODE_EXCEPTION`**.

Examples

Example #1 Execute a prepared statement with named placeholders

```php
<?php
/* Execute a prepared statement by binding PHP variables */
$calories = 150;
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < :calories AND colour = :colour');
```

```
/* Sets a parameter value using its name */
$sth->bindValue('calories', $calories, PDO::PARAM_INT);
/* Optionally, parameter names can also be prefixed with colons ":" */
$sth->bindValue(':colour', $colour, PDO::PARAM_STR);
$sth->execute();
?>
```

Example #2 Execute a prepared statement with question mark placeholders

```
<?php
/* Execute a prepared statement by binding PHP variables */
$calories = 150;
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < ? AND colour = ?');
$sth->bindValue(1, $calories, PDO::PARAM_INT);
$sth->bindValue(2, $colour, PDO::PARAM_STR);
$sth->execute();
?>
```

See Also

- PDO::prepare() - Prepares a statement for execution and returns a statement object
- PDOStatement::execute() - Executes a prepared statement
- PDOStatement::bindParam() - Binds a parameter to the specified variable name

PDOStatement::closeCursor

PDOStatement::closeCursor

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.9.0)

PDOStatement::closeCursor — Closes the cursor, enabling the statement to be executed again

Description

public function **PDOStatement::closeCursor**(): bool

PDOStatement::closeCursor() frees up the connection to the server so that other SQL statements may be issued, but leaves the statement in a state that enables it to be executed again.

This method is useful for database drivers that do not support executing a PDOStatement object when a previously executed PDOStatement object still has unfetched rows. If your database driver suffers from this limitation, the problem may manifest itself in an out-of-sequence error.

PDOStatement::closeCursor() is implemented either as an optional driver specific method (allowing for maximum efficiency), or as the generic PDO fallback if no driver specific function is installed. The PDO generic fallback is semantically the same as writing the following code in your PHP script:

```
<?php
do {
    while ($stmt->fetch())
        ;
    if (!$stmt->nextRowset())
        break;
```

```
} while (true);
?>
```

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

Emits an error with level `E_WARNING` if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_WARNING`.

Throws a PDOException if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_EXCEPTION`.

Examples

Example #1 A PDOStatement::closeCursor() example

In the following example, the *$stmt* PDOStatement object returns multiple rows but the application fetches only the first row, leaving the PDOStatement object in a state of having unfetched rows. To ensure that the application will work with all database drivers, the author inserts a call to **PDOStatement::closeCursor()** on *$stmt* before executing the *$otherStmt* PDOStatement object.

```
<?php
/* Create a PDOStatement object */
$stmt = $dbh->prepare('SELECT foo FROM bar');

/* Create a second PDOStatement object */
$otherStmt = $dbh->prepare('SELECT foobaz FROM foobar');

/* Execute the first statement */
$stmt->execute();

/* Fetch only the first row from the results */
$stmt->fetch();

/* The following call to closeCursor() may be required by some drivers */
$stmt->closeCursor();

/* Now we can execute the second statement */
$otherStmt->execute();
?>
```

See Also

- PDOStatement::execute() - Executes a prepared statement

PDOStatement::columnCount

PDOStatement::columnCount

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDOStatement::columnCount — Returns the number of columns in the result set

Description

public function **PDOStatement::columnCount**(): int

Use **PDOStatement::columnCount()** to return the number of columns in the result set represented by the PDOStatement object.

If the PDOStatement object was returned from PDO::query(), the column count is immediately available.

If the PDOStatement object was returned from PDO::prepare(), an accurate column count will not be available until you invoke PDOStatement::execute().

Parameters

This function has no parameters.

Return Values

Returns the number of columns in the result set represented by the PDOStatement object, even if the result set is empty. If there is no result set, **PDOStatement::columnCount()** returns `0`.

Errors/Exceptions

Emits an error with level `E_WARNING` if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_WARNING`.

Throws a PDOException if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_EXCEPTION`.

Examples

Example #1 Counting columns

This example demonstrates how **PDOStatement::columnCount()** operates with and without a result set.

```php
<?php
$dbh = new PDO('odbc:sample', 'db2inst1', 'ibmdb2');

$sth = $dbh->prepare("SELECT name, colour FROM fruit");

/* Count the number of columns in the (non-existent) result set */
$colcount = $sth->columnCount();
print "Before execute(), result set has $colcount columns (should be 0)\n";

$sth->execute();

/* Count the number of columns in the result set */
$colcount = $sth->columnCount();
print "After execute(), result set has $colcount columns (should be 2)\n";

?>
```

The above example will output:

```
Before execute(), result set has 0 columns (should be 0)
After execute(), result set has 2 columns (should be 2)
```

See Also

- PDO::prepare() - Prepares a statement for execution and returns a statement object
- PDOStatement::execute() - Executes a prepared statement
- PDOStatement::rowCount() - Returns the number of rows affected by the last SQL statement

PDOStatement::debugDumpParams

PDOStatement::debugDumpParams

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.9.0)

PDOStatement::debugDumpParams — Dump an SQL prepared command

Description

public function **PDOStatement::debugDumpParams**(): ?bool

Dumps the information contained by a prepared statement directly on the output. It will provide the `SQL` query in use, the number of parameters used (`Params`), the list of parameters with their key name or position, their name, their position in the query (if this is supported by the PDO driver, otherwise, it will be -1), type (`param_type`) as an integer, and a boolean value `is_param`.

This is a debug function, which dumps the data directly to the normal output.

> **Tip**
>
> As with anything that outputs its result directly to the browser, the output-control functions can be used to capture the output of this function, and save it in a string (for example).

This will only dump the parameters in the statement at the moment of the dump. Extra parameters are not stored in the statement, and not displayed.

Parameters

This function has no parameters.

Return Values

Returns `null`, or `false` in case of an error.

Changelog

Version	Description
7.2.0	**PDOStatement::debugDumpParams()** now returns the SQL sent to the database, including the full, raw query (including the replaced placeholders with their bound values). Note, that this will only be available if emulated prepared statements are turned on.

Examples

Example #1 PDOStatement::debugDumpParams() example with named parameters

```php
<?php
/* Execute a prepared statement by binding PHP variables */
$calories = 150;
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < :calories AND colour = :colour');
$sth->bindParam(':calories', $calories, PDO::PARAM_INT);
$sth->bindValue(':colour', $colour, PDO::PARAM_STR, 12);
$sth->execute();

$sth->debugDumpParams();

?>
```

The above example will output:

```
SQL: [96] SELECT name, colour, calories
    FROM fruit
    WHERE calories < :calories AND colour = :colour
Params:  2
Key: Name: [9] :calories
paramno=-1
name=[9] ":calories"
is_param=1
param_type=1
Key: Name: [7] :colour
paramno=-1
name=[7] ":colour"
is_param=1
param_type=2
```

Example #2 PDOStatement::debugDumpParams() example with unnamed parameters

```php
<?php

/* Execute a prepared statement by binding PHP variables */
$calories = 150;
$colour = 'red';
$name = 'apple';

$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < ? AND colour = ?');
$sth->bindParam(1, $calories, PDO::PARAM_INT);
$sth->bindValue(2, $colour, PDO::PARAM_STR);
$sth->execute();

$sth->debugDumpParams();

?>
```

The above example will output:

```
SQL: [82] SELECT name, colour, calories
    FROM fruit
    WHERE calories < ? AND colour = ?
Params:  2
Key: Position #0:
paramno=0
name=[0] ""
is_param=1
param_type=1
Key: Position #1:
paramno=1
name=[0] ""
is_param=1
param_type=2
```

See Also

- PDO::prepare() - Prepares a statement for execution and returns a statement object
- PDOStatement::bindParam() - Binds a parameter to the specified variable name
- PDOStatement::bindValue() - Binds a value to a parameter

PDOStatement::errorCode

PDOStatement::errorCode

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::errorCode — Fetch the SQLSTATE associated with the last operation on the statement handle

Description

public function **PDOStatement::errorCode**(): ?string

Parameters

This function has no parameters.

Return Values

Identical to PDO::errorCode(), except that **PDOStatement::errorCode()** only retrieves error codes for operations performed with PDOStatement objects.

Examples

Example #1 Retrieving an SQLSTATE code

```php
<?php
/* Provoke an error -- the BONES table does not exist */
$err = $dbh->prepare('SELECT skull FROM bones');
$err->execute();
```

```
echo "\nPDOStatement::errorCode(): ";
print $err->errorCode();
?>
```

The above example will output:

```
PDOStatement::errorCode(): 42S02
```

See Also

- PDO::errorCode() - Fetch the SQLSTATE associated with the last operation on the database handle
- PDO::errorInfo() - Fetch extended error information associated with the last operation on the database handle
- PDOStatement::errorInfo() - Fetch extended error information associated with the last operation on the statement handle

PDOStatement::errorInfo

PDOStatement::errorInfo

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::errorInfo — Fetch extended error information associated with the last operation on the statement handle

Description

public function **PDOStatement::errorInfo**(): array

Parameters

This function has no parameters.

Return Values

PDOStatement::errorInfo() returns an array of error information about the last operation performed by this statement handle. The array consists of at least the following fields:

Element	Information
0	SQLSTATE error code (a five-character alphanumeric identifier defined in the ANSI SQL standard).
1	Driver specific error code.
2	Driver specific error message.

Examples

Example #1 Displaying errorInfo() fields for a PDO_ODBC connection to a DB2 database

```php
<?php
/* Provoke an error -- the BONES table does not exist */
$sth = $dbh->prepare('SELECT skull FROM bones');
$sth->execute();

echo "\nPDOStatement::errorInfo():\n";
$arr = $sth->errorInfo();
print_r($arr);
?>
```

The above example will output:

```
PDOStatement::errorInfo():
Array
(
    [0] => 42S02
    [1] => -204
    [2] => [IBM][CLI Driver][DB2/LINUX] SQL0204N  "DANIELS.BONES" is an undefined name.
SQLSTATE=42704
)
```

See Also

- PDO::errorCode() - Fetch the SQLSTATE associated with the last operation on the database handle
- PDO::errorInfo() - Fetch extended error information associated with the last operation on the database handle
- PDOStatement::errorCode() - Fetch the SQLSTATE associated with the last operation on the statement handle

PDOStatement::execute

PDOStatement::execute

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::execute — Executes a prepared statement

Description

public function **PDOStatement::execute**(?array `$params = null`): bool

Execute the prepared statement. If the prepared statement included parameter markers, either:

- PDOStatement::bindParam() and/or PDOStatement::bindValue() has to be called to bind either variables or values (respectively) to the parameter markers. Bound variables pass their value as input and receive the output value, if any, of their associated parameter markers
- or an array of input-only parameter values has to be passed

Parameters

`params`

An array of values with as many elements as there are bound parameters in the SQL statement being executed. All values are treated as **`PDO::PARAM_STR`**.

Multiple values cannot be bound to a single parameter; for example, it is not allowed to bind two values to a single named parameter in an IN() clause.

Binding more values than specified is not possible; if more keys exist in `params` than in the SQL specified in the PDO::prepare(), then the statement will fail and an error is emitted.

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

Emits an error with level **`E_WARNING`** if the attribute **`PDO::ATTR_ERRMODE`** is set to **`PDO::ERRMODE_WARNING`**.

Throws a PDOException if the attribute **`PDO::ATTR_ERRMODE`** is set to **`PDO::ERRMODE_EXCEPTION`**.

Examples

Example #1 Execute a prepared statement with a bound variable and value

```php
<?php
/* Execute a prepared statement by binding a variable and value */
$calories = 150;
$colour = 'gre';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < :calories AND colour LIKE :colour');
$sth->bindParam('calories', $calories, PDO::PARAM_INT);
/* Names can be prefixed with colons ":" too (optional) */
$sth->bindValue(':colour', "%$colour%");
$sth->execute();
?>
```

Example #2 Execute a prepared statement with an array of named values

```php
<?php
/* Execute a prepared statement by passing an array of insert values */
$calories = 150;
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < :calories AND colour = :colour');
$sth->execute(array('calories' => $calories, 'colour' => $colour));
/* Array keys can be prefixed with colons ":" too (optional) */
$sth->execute(array(':calories' => $calories, ':colour' => $colour));
?>
```

Example #3 Execute a prepared statement with an array of positional values

```php
<?php
/* Execute a prepared statement by passing an array of insert values */
$calories = 150;
```

```php
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < ? AND colour = ?');
$sth->execute(array($calories, $colour));
?>
```

Example #4 Execute a prepared statement with variables bound to positional placeholders

```php
<?php
/* Execute a prepared statement by binding PHP variables */
$calories = 150;
$colour = 'red';
$sth = $dbh->prepare('SELECT name, colour, calories
    FROM fruit
    WHERE calories < ? AND colour = ?');
$sth->bindParam(1, $calories, PDO::PARAM_INT);
$sth->bindParam(2, $colour, PDO::PARAM_STR, 12);
$sth->execute();
?>
```

Example #5 Execute a prepared statement using array for IN clause

```php
<?php
/* Execute a prepared statement using an array of values for an IN clause */
$params = array(1, 21, 63, 171);
/* Create a string for the parameter placeholders filled to the number of params */
$place_holders = '?' . str_repeat(', ?', count($params) - 1);

/*
    This prepares the statement with enough unnamed placeholders for every value
    in our $params array. The values of the $params array are then bound to the
    placeholders in the prepared statement when the statement is executed.
    This is not the same thing as using PDOStatement::bindParam() since this
    requires a reference to the variable. PDOStatement::execute() only binds
    by value instead.
*/
$sth = $dbh->prepare("SELECT id, name FROM contacts WHERE id IN ($place_holders)");
$sth->execute($params);
?>
```

Notes

> **Note** :
>
> Some drivers require the cursor to be closed before executing the next statement.

See Also

- PDO::prepare() - Prepares a statement for execution and returns a statement object
- PDOStatement::bindParam() - Binds a parameter to the specified variable name
- PDOStatement::fetch() - Fetches the next row from a result set
- PDOStatement::fetchAll() - Fetches the remaining rows from a result set
- PDOStatement::fetchColumn() - Returns a single column from the next row of a result set

PDOStatement::fetch

PDOStatement::fetch

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::fetch — Fetches the next row from a result set

Description

public function **PDOStatement::fetch**(int `$mode` = **`PDO::FETCH_DEFAULT`**, int `$cursorOrientation` = **`PDO::FETCH_ORI_NEXT`**, int `$cursorOffset` = 0): mixed

Fetches a row from a result set associated with a PDOStatement object. The `mode` parameter determines how PDO returns the row.

Parameters

`mode`

Controls how the next row will be returned to the caller. This value must be one of the `PDO::FETCH_*` constants, defaulting to value of `PDO::ATTR_DEFAULT_FETCH_MODE` (which defaults to `PDO::FETCH_BOTH`).

- `PDO::FETCH_ASSOC`: returns an array indexed by column name as returned in your result set
- `PDO::FETCH_BOTH` (default): returns an array indexed by both column name and 0-indexed column number as returned in your result set
- `PDO::FETCH_BOUND`: returns **`true`** and assigns the values of the columns in your result set to the PHP variables to which they were bound with the PDOStatement::bindColumn() method
- `PDO::FETCH_CLASS`: returns a new instance of the requested class. The object is initialized by mapping the columns of the result set to properties in the class. This occurs before the constructor is called, allowing properties to be populated regardless of their visibility or whether they are marked as `readonly`. If a property does not exist in the class, the magic __set() method will be invoked if it exists; otherwise, a dynamic public property will be created. However, when **`PDO::FETCH_PROPS_LATE`** is also given, the constructor is called *before* the properties are populated. If `mode` includes **`PDO::FETCH_CLASSTYPE`** (e.g. `PDO::FETCH_CLASS | PDO::FETCH_CLASSTYPE`), the name of the class is determined from the value of the first column.
- `PDO::FETCH_INTO`: updates an existing instance of the requested class, mapping the columns of the result set to named properties in the class
- `PDO::FETCH_LAZY`: combines `PDO::FETCH_BOTH` and `PDO::FETCH_OBJ`, and returns a PDORow object which creates the object property names as they are accessed.
- `PDO::FETCH_NAMED`: returns an array with the same form as `PDO::FETCH_ASSOC`, except that if there are multiple columns with the same name, the value referred to by that key will be an array of all the values in the row that had that column name
- `PDO::FETCH_NUM`: returns an array indexed by column number as returned in your result set, starting at column 0
- `PDO::FETCH_OBJ`: returns an anonymous object with property names that correspond to the column names returned in your result set

- `PDO::FETCH_PROPS_LATE`: when used with `PDO::FETCH_CLASS`, the constructor of the class is called before the properties are assigned from the respective column values.

`cursorOrientation`

For a PDOStatement object representing a scrollable cursor, this value determines which row will be returned to the caller. This value must be one of the `PDO::FETCH_ORI_*` constants, defaulting to `PDO::FETCH_ORI_NEXT`. To request a scrollable cursor for your PDOStatement object, you must set the `PDO::ATTR_CURSOR` attribute to `PDO::CURSOR_SCROLL` when you prepare the SQL statement with PDO::prepare().

`cursorOffset`

For a PDOStatement object representing a scrollable cursor for which the `cursorOrientation` parameter is set to `PDO::FETCH_ORI_ABS`, this value specifies the absolute number of the row in the result set that shall be fetched.

For a PDOStatement object representing a scrollable cursor for which the `cursorOrientation` parameter is set to `PDO::FETCH_ORI_REL`, this value specifies the row to fetch relative to the cursor position before **PDOStatement::fetch()** was called.

Return Values

The return value of this function on success depends on the fetch type. In all cases, `false` is returned on failure or if there are no more rows.

Errors/Exceptions

Emits an error with level `E_WARNING` if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_WARNING`.

Throws a PDOException if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_EXCEPTION`.

Examples

Example #1 Fetching rows using different fetch styles

```php
<?php
$sth = $dbh->prepare("SELECT name, colour FROM fruit");
$sth->execute();

/* Exercise PDOStatement::fetch styles */
print "PDO::FETCH_ASSOC: ";
print "Return next row as an array indexed by column name\n";
$result = $sth->fetch(PDO::FETCH_ASSOC);
print_r($result);
print "\n";

print "PDO::FETCH_BOTH: ";
print "Return next row as an array indexed by both column name and number\n";
$result = $sth->fetch(PDO::FETCH_BOTH);
print_r($result);
print "\n";

print "PDO::FETCH_LAZY: ";
print "Return next row as a PDORow object with column names as properties\n";
$result = $sth->fetch(PDO::FETCH_LAZY);
print_r($result);
```

```php
print "\n";

print "PDO::FETCH_OBJ: ";
print "Return next row as an anonymous object with column names as properties\n";
$result = $sth->fetch(PDO::FETCH_OBJ);
print $result->name;
print "\n";
?>
```

The above example will output:

```
PDO::FETCH_ASSOC: Return next row as an array indexed by column name
Array
(
    [name] => apple
    [colour] => red
)

PDO::FETCH_BOTH: Return next row as an array indexed by both column name and number
Array
(
    [name] => banana
    [0] => banana
    [colour] => yellow
    [1] => yellow
)

PDO::FETCH_LAZY: Return next row as a PDORow object with column names as properties
PDORow Object
(
    [name] => orange
    [colour] => orange
)

PDO::FETCH_OBJ: Return next row as an anonymous object with column names as properties
kiwi
```

Example #2 Fetching rows with a scrollable cursor

```php
<?php
function readDataForwards($dbh) {
    $sql = 'SELECT hand, won, bet FROM mynumbers ORDER BY BET';
    $stmt = $dbh->prepare($sql, array(PDO::ATTR_CURSOR => PDO::CURSOR_SCROLL));
    $stmt->execute();
    while ($row = $stmt->fetch(PDO::FETCH_NUM, PDO::FETCH_ORI_NEXT)) {
        $data = $row[0] . "\t" . $row[1] . "\t" . $row[2] . "\n";
        print $data;
    }
}
function readDataBackwards($dbh) {
    $sql = 'SELECT hand, won, bet FROM mynumbers ORDER BY bet';
    $stmt = $dbh->prepare($sql, array(PDO::ATTR_CURSOR => PDO::CURSOR_SCROLL));
```

```
    $stmt->execute();
    $row = $stmt->fetch(PDO::FETCH_NUM, PDO::FETCH_ORI_LAST);
    do {
        $data = $row[0] . "\t" . $row[1] . "\t" . $row[2] . "\n";
        print $data;
    } while ($row = $stmt->fetch(PDO::FETCH_NUM, PDO::FETCH_ORI_PRIOR));
}

print "Reading forwards:\n";
readDataForwards($conn);

print "Reading backwards:\n";
readDataBackwards($conn);
?>
```

The above example will output:

```
Reading forwards:
21    10    5
16    0     5
19    20    10

Reading backwards:
19    20    10
16    0     5
21    10    5
```

Example #3 Construction order

When objects are fetched via **PDO::FETCH_CLASS**, the object properties are assigned first, and then the constructor of the class is invoked. However, when **PDO::FETCH_PROPS_LATE** is also given, this order is reversed, i.e. first the constructor is called, and afterwards the properties are assigned.

```
<?php
class Person
{
    private $name;

    public function __construct()
    {
        $this->tell();
    }

    public function tell()
    {
        if (isset($this->name)) {
            echo "I am {$this->name}.\n";
        } else {
            echo "I don't have a name yet.\n";
        }
    }
}
```

```php
$sth = $dbh->query("SELECT * FROM people");
$sth->setFetchMode(PDO::FETCH_CLASS, 'Person');
$person = $sth->fetch();
$person->tell();
$sth->setFetchMode(PDO::FETCH_CLASS|PDO::FETCH_PROPS_LATE, 'Person');
$person = $sth->fetch();
$person->tell();
?>
```

The above example will output something similar to:

```
I am Alice.
I am Alice.
I don't have a name yet.
I am Bob.
```

See Also

- PDO::prepare() - Prepares a statement for execution and returns a statement object
- PDOStatement::execute() - Executes a prepared statement
- PDOStatement::fetchAll() - Fetches the remaining rows from a result set
- PDOStatement::fetchColumn() - Returns a single column from the next row of a result set
- PDOStatement::fetchObject() - Fetches the next row and returns it as an object
- PDOStatement::setFetchMode() - Set the default fetch mode for this statement

PDOStatement::fetchAll

PDOStatement::fetchAll

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::fetchAll — Fetches the remaining rows from a result set

Description

public function **PDOStatement::fetchAll**(int `$mode` = PDO::FETCH_DEFAULT): array
public function **PDOStatement::fetchAll**(int `$mode` = PDO::FETCH_COLUMN, int `$column`): array
public function **PDOStatement::fetchAll**(int `$mode` = PDO::FETCH_CLASS, string `$class`, ?array `$constructorArgs`): array
public function **PDOStatement::fetchAll**(int `$mode` = PDO::FETCH_FUNC, callable `$callback`): array

Parameters

`mode`

Controls the contents of the returned array as documented in PDOStatement::fetch(). Defaults to value of **`PDO::ATTR_DEFAULT_FETCH_MODE`** (which defaults to **`PDO::FETCH_BOTH`**)

To return an array consisting of all values of a single column from the result set, specify **`PDO::FETCH_COLUMN`**. You can specify which column you want with the `column` parameter.

To index the resulting array by a certain column's value (instead of consecutive numbers), put this column's name first in the column list in SQL, and use `PDO::FETCH_UNIQUE`. This column must contain only unique values or some data will be lost.

To group results in the form of a 3-dimensional array indexed by values of a specified column, put this column's name first in the column list in SQL and use `PDO::FETCH_GROUP`.

To group results in the form of a 2-dimensional array use bitwise-OR `PDO::FETCH_GROUP` with `PDO::FETCH_COLUMN`. The results will be grouped by the first column, with the array element's value being a list array of the corresponding entries from the second column.

The following are dynamic parameters that depend on the fetch mode. They can't be used with named parameters.

`column`

Used with `PDO::FETCH_COLUMN`. Returns the indicated 0-indexed column.

`class`

Used with `PDO::FETCH_CLASS`. Returns instances of the specified class, mapping the columns of each row to named properties in the class.

`constructorArgs`

Arguments of custom class constructor when the `mode` parameter is `PDO::FETCH_CLASS`.

`callback`

Used with `PDO::FETCH_FUNC`. Returns the results of calling the specified function, using each row's columns as parameters in the call.

Return Values

PDOStatement::fetchAll() returns an array containing all of the remaining rows in the result set. The array represents each row as either an array of column values or an object with properties corresponding to each column name. An empty array is returned if there are zero results to fetch.

Using this method to fetch large result sets will result in a heavy demand on system and possibly network resources. Rather than retrieving all of the data and manipulating it in PHP, consider using the database server to manipulate the result sets. For example, use the WHERE and ORDER BY clauses in SQL to restrict results before retrieving and processing them with PHP.

Errors/Exceptions

Emits an error with level `E_WARNING` if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_WARNING`.

Throws a PDOException if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_EXCEPTION`.

Changelog

Version	Description
8.5.0	Using `PDO::FETCH_INTO` as a fetch mode now throws a ValueError, similar to `PDO::FETCH_LAZY`. Using `PDO::FETCH_PROPS_LATE` with a fetch mode other than `PDO::FETCH_CLASS` now throws a ValueError.
8.0.0	This method always returns an array now, while previously `false` may have been returned on failure.

Examples

Example #1 Fetch all remaining rows in a result set

```php
<?php
$sth = $dbh->prepare("SELECT name, colour FROM fruit");
$sth->execute();

/* Fetch all of the remaining rows in the result set */
print "Fetch all of the remaining rows in the result set:\n";
$result = $sth->fetchAll();
print_r($result);
?>
```

The above example will output something similar to:

```
Fetch all of the remaining rows in the result set:
Array
(
    [0] => Array
        (
            [name] => apple
            [0] => apple
            [colour] => red
            [1] => red
        )

    [1] => Array
        (
            [name] => pear
            [0] => pear
            [colour] => green
            [1] => green
        )

    [2] => Array
        (
            [name] => watermelon
            [0] => watermelon
            [colour] => pink
            [1] => pink
        )

)
```

Example #2 Fetching all values of a single column from a result set

The following example demonstrates how to return all of the values of a single column from a result set, even though the SQL statement itself may return multiple columns per row.

```php
<?php
$sth = $dbh->prepare("SELECT name, colour FROM fruit");
```

```php
$sth->execute();

/* Fetch all of the values of the first column */
$result = $sth->fetchAll(PDO::FETCH_COLUMN, 0);
var_dump($result);
?>
```

The above example will output something similar to:

```
Array(3)
(
    [0] =>
    string(5) => apple
    [1] =>
    string(4) => pear
    [2] =>
    string(10) => watermelon
)
```

Example #3 Grouping all values by a single column

The following example demonstrates how to return an associative array grouped by the values of the specified column in the result set. The array contains three keys: values `apple` and `pear` are returned as arrays that contain two different colours, while `watermelon` is returned as an array that contains only one colour.

```php
<?php
$insert = $dbh->prepare("INSERT INTO fruit(name, colour) VALUES (?, ?)");
$insert->execute(array('apple', 'green'));
$insert->execute(array('pear', 'yellow'));

$sth = $dbh->prepare("SELECT name, colour FROM fruit");
$sth->execute();

/* Group values by the first column */
var_dump($sth->fetchAll(PDO::FETCH_COLUMN|PDO::FETCH_GROUP));
?>
```

The above example will output something similar to:

```
array(3) {
  ["apple"]=>
  array(2) {
    [0]=>
    string(5) "green"
    [1]=>
    string(3) "red"
  }
  ["pear"]=>
  array(2) {
```

```
    [0]=>
    string(5) "green"
    [1]=>
    string(6) "yellow"
  }
  ["watermelon"]=>
  array(1) {
    [0]=>
    string(5) "pink"
  }
}
```

Example #4 Instantiating a class for each result

The following example demonstrates the behaviour of the `PDO::FETCH_CLASS` fetch style.

```php
<?php
class fruit {
    public $name;
    public $colour;
}

$sth = $dbh->prepare("SELECT name, colour FROM fruit");
$sth->execute();

$result = $sth->fetchAll(PDO::FETCH_CLASS, "fruit");
var_dump($result);
?>
```

The above example will output something similar to:

```
array(3) {
  [0]=>
  object(fruit)#1 (2) {
    ["name"]=>
    string(5) "apple"
    ["colour"]=>
    string(5) "green"
  }
  [1]=>
  object(fruit)#2 (2) {
    ["name"]=>
    string(4) "pear"
    ["colour"]=>
    string(6) "yellow"
  }
  [2]=>
  object(fruit)#3 (2) {
    ["name"]=>
    string(10) "watermelon"
    ["colour"]=>
    string(4) "pink"
```

```
  }
  [3]=>
  object(fruit)#4 (2) {
    ["name"]=>
    string(5) "apple"
    ["colour"]=>
    string(3) "red"
  }
  [4]=>
  object(fruit)#5 (2) {
    ["name"]=>
    string(4) "pear"
    ["colour"]=>
    string(5) "green"
  }
}
```

Example #5 Calling a function for each result

The following example demonstrates the behaviour of the `PDO::FETCH_FUNC` fetch style.

```php
<?php
function fruit($name, $colour) {
    return "{$name}: {$colour}";
}

$sth = $dbh->prepare("SELECT name, colour FROM fruit");
$sth->execute();

$result = $sth->fetchAll(PDO::FETCH_FUNC, "fruit");
var_dump($result);
?>
```

The above example will output something similar to:

```
array(3) {
  [0]=>
  string(12) "apple: green"
  [1]=>
  string(12) "pear: yellow"
  [2]=>
  string(16) "watermelon: pink"
  [3]=>
  string(10) "apple: red"
  [4]=>
  string(11) "pear: green"
}
```

See Also

- PDO::query() - Prepares and executes an SQL statement without placeholders
- PDOStatement::fetch() - Fetches the next row from a result set
- PDOStatement::fetchColumn() - Returns a single column from the next row of a result set
- PDO::prepare() - Prepares a statement for execution and returns a statement object
- PDOStatement::setFetchMode() - Set the default fetch mode for this statement

PDOStatement::fetchColumn

PDOStatement::fetchColumn

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.9.0)

PDOStatement::fetchColumn — Returns a single column from the next row of a result set

Description

public function **PDOStatement::fetchColumn**(int `$column` = 0): mixed

Returns a single column from the next row of a result set or `false` if there are no more rows.

> **Note** :
>
> **PDOStatement::fetchColumn()** should not be used to retrieve boolean columns, as it is impossible to distinguish a value of `false` from there being no more rows to retrieve. Use PDOStatement::fetch() instead.

Parameters

`column`

0-indexed number of the column you wish to retrieve from the row. If no value is supplied, **PDOStatement::fetchColumn()** fetches the first column.

Return Values

PDOStatement::fetchColumn() returns a single column from the next row of a result set or `false` if there are no more rows.

> **Warning**
>
> There is no way to return another column from the same row if you use **PDOStatement::fetchColumn()** to retrieve data.

Errors/Exceptions

Emits an error with level `E_WARNING` if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_WARNING`.

Throws a PDOException if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_EXCEPTION`.

Examples

Example #1 Return first column of the next row

```php
<?php
$sth = $dbh->prepare("SELECT name, colour FROM fruit");
$sth->execute();

print "Fetch the first column from the first row in the result set:\n";
$result = $sth->fetchColumn();
print "name = $result\n";

print "Fetch the second column from the second row in the result set:\n";
$result = $sth->fetchColumn(1);
print "colour = $result\n";
?>
```

The above example will output:

```
Fetch the first column from the first row in the result set:
name = lemon
Fetch the second column from the second row in the result set:
colour = red
```

See Also

- PDO::query() - Prepares and executes an SQL statement without placeholders
- PDOStatement::fetch() - Fetches the next row from a result set
- PDOStatement::fetchAll() - Fetches the remaining rows from a result set
- PDO::prepare() - Prepares a statement for execution and returns a statement object
- PDOStatement::setFetchMode() - Set the default fetch mode for this statement

PDOStatement::fetchObject

PDOStatement::fetchObject

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.4)

PDOStatement::fetchObject — Fetches the next row and returns it as an object

Description

public function **PDOStatement::fetchObject**(?string `$class` = "stdClass", array `$constructorArgs` = []): object|false

Fetches the next row and returns it as an object. This function is an alternative to PDOStatement::fetch() with `PDO::FETCH_CLASS` or `PDO::FETCH_OBJ` style.

When an object is fetched, its properties are assigned from respective column values, and afterwards its constructor is invoked.

Parameters

`class`

Name of the created class.

`constructorArgs`

Elements of this array are passed to the constructor.

Return Values

Returns an instance of the required class with property names that correspond to the column names or `false` on failure.

Errors/Exceptions

Emits an error with level `E_WARNING` if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_WARNING`.

Throws a PDOException if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_EXCEPTION`.

See Also

- PDOStatement::fetch() - Fetches the next row from a result set

PDOStatement::getAttribute

PDOStatement::getAttribute

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDOStatement::getAttribute — Retrieve a statement attribute

Description

public function **PDOStatement::getAttribute**(int `$name`): mixed

Gets an attribute of the statement. Currently, no generic attributes exist but only driver specific:

- `PDO::ATTR_CURSOR_NAME` (Firebird and ODBC specific): Get the name of the cursor for `UPDATE ... WHERE CURRENT OF`.
- `Pdo\Pgsql::ATTR_RESULT_MEMORY_SIZE` (PostgreSQL specific, available as of PHP 8.4.0): Get the amount of memory, in bytes, allocated to the query result.

Note that driver specific attributes *must not* be used with other drivers.

Parameters

`name`

The attribute to query.

Return Values

Returns the attribute value.

See Also

- PDO::getAttribute() - Retrieve a database connection attribute
- PDO::setAttribute() - Set an attribute
- PDOStatement::setAttribute() - Set a statement attribute

PDOStatement::getColumnMeta

PDOStatement::getColumnMeta

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDOStatement::getColumnMeta — Returns metadata for a column in a result set

Description

public function **PDOStatement::getColumnMeta**(int `$column`): array|false

Retrieves the metadata for a 0-indexed column in a result set as an associative array.

> **Warning**
>
> Some drivers may not implement **PDOStatement::getColumnMeta()**, as it is optional. However, all PDO drivers documented in the manual implement this function.

Parameters

`column`

The 0-indexed column in the result set.

Return Values

Returns an associative array containing the following values representing the metadata for a single column:

Column metadata

Name	Value
`native_type`	The PHP native type used to represent the column value.
`driver:decl_type`	The SQL type used to represent the column value in the database. If the column in the result set is the result of a function, this value is not returned by **PDOStatement::getColumnMeta()**.
`flags`	Any flags set for this column.
`name`	The name of this column as returned by the database.
`table`	The name of this column's table as returned by the database.
`len`	The length of this column. Normally `-1` for types other than floating point decimals.
`precision`	The numeric precision of this column. Normally `0` for types other than floating point decimals.
`pdo_type`	The type of this column as represented by the `PDO::PARAM_*` constants.

Returns `false` if the requested column does not exist in the result set, or if no result set exists.

Examples

Example #1 Retrieving column metadata

The following example shows the results of retrieving the metadata for a single column generated by a function (COUNT) in a PDO_SQLITE driver.

```
<?php
$select = $DB->query('SELECT COUNT(*) FROM fruit');
```

```
$meta = $select->getColumnMeta(0);
var_dump($meta);
?>
```

The above example will output:

```
array(6) {
  ["native_type"]=>
  string(7) "integer"
  ["flags"]=>
  array(0) {
  }
  ["name"]=>
  string(8) "COUNT(*)"
  ["len"]=>
  int(-1)
  ["precision"]=>
  int(0)
  ["pdo_type"]=>
  int(2)
}
```

See Also

- PDOStatement::columnCount() - Returns the number of columns in the result set
- PDOStatement::rowCount() - Returns the number of rows affected by the last SQL statement

PDOStatement::getIterator

PDOStatement::getIterator

(PHP 8)

PDOStatement::getIterator — Gets result set iterator

Description

public function **PDOStatement::getIterator**(): Iterator

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

PDOStatement::nextRowset

PDOStatement::nextRowset

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDOStatement::nextRowset — Advances to the next rowset in a multi-rowset statement handle

Description

public function **PDOStatement::nextRowset**(): bool

Some database servers support stored procedures that return more than one rowset (also known as a result set). **PDOStatement::nextRowset()** enables you to access the second and subsequent rowsets associated with a PDOStatement object. Each rowset can have a different set of columns from the preceding rowset.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Fetching multiple rowsets returned from a stored procedure

The following example shows how to call a stored procedure, `MULTIPLE_ROWSETS`, which returns three rowsets. We use a do-while loop to call the **PDOStatement::nextRowset()** method until it returns `false` and terminates the loop when no more rowsets are available.

```
<?php
$sql = 'CALL multiple_rowsets()';
$stmt = $conn->query($sql);
$i = 1;
do {
    $rowset = $stmt->fetchAll(PDO::FETCH_NUM);
    if ($rowset) {
        printResultSet($rowset, $i);
    }
    $i++;
} while ($stmt->nextRowset());

function printResultSet(&$rowset, $i) {
    print "Result set $i:\n";
    foreach ($rowset as $row) {
        foreach ($row as $col) {
            print $col . "\t";
        }
        print "\n";
    }
    print "\n";
}
?>
```

The above example will output:

```
Result set 1:
apple    red
banana   yellow

Result set 2:
orange   orange    150
banana   yellow    175

Result set 3:
lime     green
apple    red
banana   yellow
```

See Also

- PDOStatement::columnCount() - Returns the number of columns in the result set
- PDOStatement::execute() - Executes a prepared statement
- PDOStatement::getColumnMeta() - Returns metadata for a column in a result set
- PDO::query() - Prepares and executes an SQL statement without placeholders

PDOStatement::rowCount

PDOStatement::rowCount

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.1.0)

PDOStatement::rowCount — Returns the number of rows affected by the last SQL statement

Description

public function **PDOStatement::rowCount**(): int

PDOStatement::rowCount() returns the number of rows affected by the last DELETE, INSERT, or UPDATE statement executed by the corresponding `PDOStatement` object.

For statements that produce result sets, such as `SELECT`, the behavior is undefined and can be different for each driver. Some databases may return the number of rows produced by that statement (e.g. MySQL in buffered mode), but this behaviour is not guaranteed for all databases and should not be relied on for portable applications.

> **Note** :
>
> This method returns "0" (zero) with the PostgreSQL driver when setting the `PDO::ATTR_CURSOR` statement attribute to `PDO::CURSOR_SCROLL`.

Parameters

This function has no parameters.

Return Values

Returns the number of rows.

Errors/Exceptions

Emits an error with level `E_WARNING` if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_WARNING`.

Throws a PDOException if the attribute `PDO::ATTR_ERRMODE` is set to `PDO::ERRMODE_EXCEPTION`.

Examples

Example #1 Return the number of deleted rows

PDOStatement::rowCount() returns the number of rows affected by a DELETE, INSERT, or UPDATE statement.

```php
<?php
/* Delete all rows from the FRUIT table */
$del = $dbh->prepare('DELETE FROM fruit');
$del->execute();

/* Return number of rows that were deleted */
print "Return number of rows that were deleted:\n";
$count = $del->rowCount();
print "Deleted $count rows.\n";
?>
```

The above example will output something similar to:

```
Return number of rows that were deleted:
Deleted 9 rows.
```

Example #2 Counting rows returned by a SELECT statement

For most databases, **PDOStatement::rowCount()** does not return the number of rows affected by a SELECT statement. Instead, use PDO::query() to issue a SELECT COUNT(*) statement with the same predicates as your intended SELECT statement, then use PDOStatement::fetchColumn() to retrieve the number of matching rows.

```php
<?php
$sql = "SELECT COUNT(*) FROM fruit WHERE calories > 100";
$res = $conn->query($sql);
$count = $res->fetchColumn();

print "There are " .  $count . " matching records.";
```

The above example will output something similar to:

```
There are 2 matching records.
```

See Also

- PDOStatement::columnCount() - Returns the number of columns in the result set
- PDOStatement::fetchColumn() - Returns a single column from the next row of a result set
- PDO::query() - Prepares and executes an SQL statement without placeholders

PDOStatement::setAttribute

PDOStatement::setAttribute

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDOStatement::setAttribute — Set a statement attribute

Description

public function **PDOStatement::setAttribute**(int `$attribute`, mixed `$value`): bool

Sets an attribute on the statement. Currently, no generic attributes are set but only driver specific:

- `PDO::ATTR_CURSOR_NAME` (Firebird and ODBC specific): Set the name of the cursor for `UPDATE ... WHERE CURRENT OF`.

Note that driver specific attributes *must not* be used with other drivers.

Parameters

`attribute`

The attribute to modify.

`value`

The value to set the `attribute`, might require a specific type depending on the attribute.

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

As of PHP 8.5.0, when using the Firebird driver, a ValueError is thrown if the cursor name set via **`PDO::ATTR_CURSOR_NAME`** is too long.

Changelog

Version	Description
8.5.0	When using the Firebird driver, a ValueError is now thrown if the cursor name set via **`PDO::ATTR_CURSOR_NAME`** is too long.

See Also

- PDO::getAttribute() - Retrieve a database connection attribute
- PDO::setAttribute() - Set an attribute
- PDOStatement::getAttribute() - Retrieve a statement attribute

PDOStatement::setFetchMode

PDOStatement::setFetchMode

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 0.2.0)

PDOStatement::setFetchMode — Set the default fetch mode for this statement

Description

public function **PDOStatement::setFetchMode**(int `$mode`): true
public function **PDOStatement::setFetchMode**(int `$mode` = PDO::FETCH_COLUMN, int `$colno`): true
public function **PDOStatement::setFetchMode**(int `$mode` = PDO::FETCH_CLASS, string `$class`, ?array `$constructorArgs` = **`null`**): true
public function **PDOStatement::setFetchMode**(int `$mode` = PDO::FETCH_INTO, object `$object`): true

Parameters

`mode`

The fetch mode must be one of the **`PDO::FETCH_*`** constants.

`colno`

Column number.

`class`

Class name.

`constructorArgs`

Constructor arguments. As of PHP 8.5.0, string keys act as named arguments. To pass a variable by reference, use a reference element: `$constructorArgs = [&$valByRef]`.

`object`

Object.

Return Values

Always returns **`true`**.

Errors/Exceptions

As of PHP 8.5.0, an Error is thrown if this method is called during a call to PDOStatement::fetch(), PDOStatement::fetchObject(), or PDOStatement::fetchAll().

Changelog

Version	Description
8.5.0	The `constructorArgs` for **`PDO::FETCH_CLASS`** now follow CUFA (call_user_func_array()) semantics: string keys act as named arguments, and automatic wrapping of by-value arguments for by-reference parameters has been removed.
8.5.0	Calling this method during a call to PDOStatement::fetch(), PDOStatement::fetchObject(), or PDOStatement::fetchAll() now throws an Error.

Version	Description
8.4.0	The return type is `true` now; previously, it was bool.

Examples

Example #1 Setting the fetch mode

The following example demonstrates how **PDOStatement::setFetchMode()** changes the default fetch mode for a PDOStatement object.

```
<?php
$stmt = $dbh->query('SELECT name, colour, calories FROM fruit');
$stmt->setFetchMode(PDO::FETCH_NUM);
foreach ($stmt as $row) {
    print $row[0] . "\t" . $row[1] . "\t" . $row[2] . "\n";
}
```

The above example will output something similar to:

```
apple   red     150
banana  yellow  250
orange  orange  300
kiwi    brown   75
lemon   yellow  25
pear    green   150
```

PDORow

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo >= 1.0.0)

Introduction

Represents a row from a result set returned by PDOStatement::fetch() called with `PDO::FETCH_LAZY` fetch mode.

Objects of this class cannot be instantiated and are not serializable.

The **PDORow** object allows access to the returned data as if both `PDO::FETCH_OBJ` and `PDO::FETCH_BOTH` mode were used. This means that the returned data can be accessed as object properties, and as an array both indexed by the column name and a column offset number.

> **Caution**
>
> Accessing an undefined property returns `null` without emitting a warning.

Class synopsis

final class **PDORow** {
/* Properties */

public string *$queryString*;
}

Properties

queryString

Query string used by the PDOStatement that returned the **PDORow** object.

Errors/Exceptions

Throws an Error when trying to write to or unset() any property.

PDOException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Represents an error raised by PDO. You should not throw a **PDOException** from your own code. See Exceptions for more information about Exceptions in PHP.

Class synopsis

class **PDOException** extends RuntimeException {
/* Properties */
protected int|string *$code*;
public ?array *$errorInfo* = null;
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

Properties

errorInfo

Corresponds to PDO::errorInfo() or PDOStatement::errorInfo()

code

SQLSTATE error code. Use Exception::getCode() to access it.

PDO Drivers

Table of Contents

The following drivers currently implement the PDO interface:

Driver name	Supported databases
PDO_CUBRID	Cubrid
PDO_DBLIB	FreeTDS / Microsoft SQL Server / Sybase
PDO_FIREBIRD	Firebird
PDO_IBM	IBM DB2
PDO_INFORMIX	IBM Informix Dynamic Server
PDO_MYSQL	MySQL 3.x/4.x/5.x/8.x
PDO_OCI	Oracle Call Interface
PDO_ODBC	ODBC v3 (IBM DB2, unixODBC and win32 ODBC)
PDO_PGSQL	PostgreSQL
PDO_SQLITE	SQLite 3 and SQLite 2
PDO_SQLSRV	Microsoft SQL Server / SQL Azure

CUBRID PDO Driver

Introduction

PDO_CUBRID is a driver that implements the PHP Data Objects (PDO) interface to enable access from PHP to CUBRID databases.

> **Note** :
>
> Current version of PDO_CUBRID doesn't support persistent connection now.

Installation

To build the PDO_CUBRID extension, the CUBRID DBMS must be installed on the same system as PHP. PDO_CUBRID is a » PECL extension, so follow the instructions in Installation of PECL extensions to install the PDO_CUBRID extension. Issue the **configure** command to point to the location of the CUBRID base dir as follows:

```
$ ./configure --with-pdo-cubrid=/path/to/CUBRID[,shared]
```

The **configure** command defaults to the value of the *CUBRID* environment variable.

A DLL for this PECL extension is currently unavailable. See also the building on Windows section. Detailed information about installation on Linux and Windows manually, please read build-guide.html in PECL package CUBRID for reference.

PDO_CUBRID Features

Scrollable cursors

PDO_CUBRID supports scrollable cursors. The default cursor type is forward only, and you can use parameter driver_options in PDO::prepare() to change cursor type.

Timeout

PDO_CUBRID supports sql statement execution timeout setting; You can use PDO::setAttribute() to set timeout value.

Autocommit_mode and Transaction

PDO_CUBRID supports both autocommit_mode and transaction, and autocommit_mode is enabled by default. You can use PDO::setAttribute() to change its state.

If you use PDO::beginTransaction() to begin a transaction, it will disable autocommit_mode automatically and restore it after PDO::commit() or PDO::rollBack().

> **Note** :
>
> Prior do disabling autocommit_mode any pending work is automatically committed.

Multiple SQL Statements

PDO_CUBRID supports Multiple SQL statements. Multiple SQL statements are separated by semicolons (;).

Schema Information

PDO_CUBRID implements PDO::cubrid_schema() to get schema information.

LOBs

PDO_CUBRID supports BLOB/CLOB data type. The LOB in PDO is represented as a stream, so you can insert LOBs by binding a stream, and get LOBs by reading a stream returned by CUBRID PDO. For example:

Example #1 Insert LOBs in CUBRID PDO

```php
<?php
$fp = fopen('lob_test.png', 'rb');

$sql_stmt = "INSERT INTO lob_test(name, content) VALUES('lob_test.png', ?)";

$stmt = $dbh->prepare($sql_stmt);
$ret = $stmt->bindParam(1, $fp, PDO::PARAM_LOB);
$ret = $stmt->execute();
?>
```

Example #2 Fetch LOBs in CUBRID PDO

```php
<?php
$sql_stmt = "SELECT content FROM lob_test WHERE name='lob_test.png'";

$stmt = $dbh->prepare($sql_stmt);
$stmt->execute();
$result = $stmt->fetch(PDO::FETCH_NUM);

header("Content-Type: image/png");
fpassthru($result[0]);
?>
```

Column meta

The PDOStatement::getColumnMeta() in CUBRID PDO will return an associative array containing the following values:

- type
- name
- table
- def
- precision
- scale
- not_null
- auto_increment
- unique_key
- multiple_key
- primary_key
- foreign_key
- reverse_index
- reverse_unique

Collection Data Type

PDO_CUBRID supports SET/MULTISET/SEQUENCE data type. If you don't specify data type, the default data type is char. For example:

Example #3 Insert set in CUBRID PDO with default data type.

```php
<?php
$conn_str ="cubrid:dbname=demodb;host=localhost;port=33000";
$cubrid_pdo = new PDO($conn_str, 'dba', '');

$cubrid_pdo->exec("DROP TABLE if exists test_tbl");
$cubrid_pdo->exec("CREATE TABLE test_tbl (col_1 SET(VARCHAR))");

$sql_stmt_insert = "INSERT INTO test_tbl VALUES (?);";
$stmt = $cubrid_pdo->prepare($sql_stmt_insert);
$data = array("abc","def","ghi");
$ret = $stmt->bindParam(1, $data, PDO::PARAM_NULL);
$ret = $stmt->execute();
var_Dump($ret);
?>
```

Example #4 Specify data type when insert set in CUBRID PDO

```php
<?php
$conn_str ="cubrid:dbname=demodb;host=localhost;port=33000";
$cubrid_pdo = new PDO($conn_str, 'dba', '');

$cubrid_pdo->exec("DROP TABLE if exists test_tbl");
$cubrid_pdo->exec("CREATE TABLE test_tbl (col_1 SET(int))");

$sql_stmt_insert = "INSERT INTO test_tbl VALUES (?);";
$stmt = $cubrid_pdo->prepare($sql_stmt_insert);
$data = array(1,2,3,4);
$ret = $stmt->bindParam(1, $data, 0,0,"int");
$ret = $stmt->execute();
var_Dump($ret);
?>
```

CUBRID Bind Data Types for the fifth parameter of PDOStatement::bindParam():

- CHAR
- STRING
- NCHAR
- VARNCHAR
- BIT
- VARBIT
- NUMERIC
- NUMBER
- INT
- SHORT
- BIGINT
- MONETARY
- FLOAT
- DOUBLE
- DATE
- TIME
- DATETIME
- TIMESTAMP

Predefined Constants

The constants below are defined by this driver, and will only be available when the extension has been either compiled into PHP or dynamically loaded at runtime. In addition, these driver-specific constants should only be used if you are using this driver. Using driver-specific attributes with another driver may result in unexpected behaviour. PDO::getAttribute() may be used to obtain the **PDO::ATTR_DRIVER_NAME** attribute to check the driver, if your code can run against multiple drivers.

The following constants can be used when setting the database attribute. They can be passed to PDO::getAttribute() or PDO::setAttribute().

PDO::CUBRID attribute flags

Constant	Description
PDO::CUBRID_ATTR_ISOLATION_LEVEL	Transaction isolation level for the database connection.
PDO::CUBRID_ATTR_LOCK_TIMEOUT	Transaction timeout in seconds.

Constant	Description
PDO::CUBRID_ATTR_MAX_STRING_LENGTH	Read only. The maximum string length for bit, varbit, char, varchar, nchar, nchar varying data types when using CUBRID PDO API.

The following constants can be used when setting the transaction isolation level. They can be passed to PDO::getAttribute() or returned by PDO::setAttribute().

PDO::CUBRID isolation level flags

Constant	Description
PDO::TRAN_COMMIT_CLASS_UNCOMMIT_INSTANCE	The lowest isolation level (1). A dirty, non-repeatable or phantom read may occur for the tuple and a non-repeatable read may occur for the table as well.
PDO::TRAN_COMMIT_CLASS_COMMIT_INSTANCE	A relatively low isolation level (2). A dirty read does not occur, but non-repeatable or phantom read may occur.
PDO::TRAN_REP_CLASS_UNCOMMIT_INSTANCE	The default isolation of CUBRID (3). A dirty, non-repeatable or phantom read may occur for the tuple, but repeatable read is ensured for the table.
PDO::TRAN_REP_CLASS_COMMIT_INSTANCE	A relatively low isolation level (4). A dirty read does not occur, but non-repeatable or phantom read may.
PDO::TRAN_REP_CLASS_REP_INSTANCE	A relatively high isolation level (5). A dirty or non-repeatable read does not occur, but a phantom read may.
PDO::TRAN_SERIALIZABLE	The highest isolation level (6). Problems concerning concurrency (e.g. dirty read, non-repeatable read, phantom read, etc.) do not occur.

The following constants can be used when getting schema information. They can be passed to PDO::cubrid_schema().

PDO::CUBRID schema flags

Constant	Description
PDO::CUBRID_SCH_TABLE	Get name and type of table in CUBRID.
PDO::CUBRID_SCH_VIEW	Get name and type of view in CUBRID.
PDO::CUBRID_SCH_QUERY_SPEC	Get the query definition of view.
PDO::CUBRID_SCH_ATTRIBUTE	Get the attributes of table column.
PDO::CUBRID_SCH_TABLE_ATTRIBUTE	Get the attributes of table.
PDO::CUBRID_SCH_METHOD	Get the instance method. The instance method is a method called by a class instance. It is used more often than the class method because most operations are executed in the instance.
PDO::CUBRID_SCH_TABLE_METHOD	Get the class method. The class method is a method called by a class object. It is usually used to create a new class instance or to initialize it. It is also used to access or update class attributes.
PDO::CUBRID_SCH_METHOD_FILE	Get the information of the file where the method of the table is defined.
PDO::CUBRID_SCH_SUPER_TABLE	Get the name and type of table which table inherites attributes from.
PDO::CUBRID_SCH_SUB_TABLE	Get the name and type of table which inherites attributes from this table.

Constant	Description
PDO::CUBRID_SCH_CONSTRAINT	Get the table constraints.
PDO::CUBRID_SCH_TRIGGER	Get the table triggers.
PDO::CUBRID_SCH_TABLE_PRIVILEGE	Get the privilege information of table.
PDO::CUBRID_SCH_COL_PRIVILEGE	Get the privilege information of column.
PDO::CUBRID_SCH_DIRECT_SUPER_TABLE	Get the direct super table of table.
PDO::CUBRID_SCH_PRIMARY_KEY	Get the table primary key.
PDO::CUBRID_SCH_IMPORTED_KEYS	Get imported keys of table.
PDO::CUBRID_SCH_EXPORTED_KEYS	Get exported keys of table.
PDO::CUBRID_SCH_CROSS_REFERENCE	Get reference relationship of two tables.

Table of Contents

PDO_CUBRID DSN

PDO_CUBRID DSN

(PECL PDO_CUBRID >= 8.3.0.0001)

PDO_CUBRID DSN — Connecting to CUBRID databases

Description

The PDO_CUBRID Data Source Name (DSN) is composed of the following elements, delimited by semicolons:

DSN prefix

The DSN prefix is **cubrid:**.

`host`

The hostname on which the database server resides.

`port`

The port on which the database server is running.

`dbname`

The name of the database.

Notes

> **Note** :
>
> When you establish the connection to CUBRID, you should give username and password except DSN.

Examples

Example #1 PDO_CUBRID DSN examples

The following example shows a PDO_CUBRID DSN for connecting to a CUBRID database:

```
cubrid:host=localhost;port=33000;dbname=demodb
```

PDO::cubrid_schema

PDO::cubrid_schema

(PECL PDO_CUBRID >= 8.3.0.0001)

PDO::cubrid_schema — Get the requested schema information

Description

public function **PDO::cubrid_schema**(int `$schema_type`, string `$table_name = ?`, string `$col_name = ?`): array

This function is used to get the requested schema information from database. You have to designate `table_name`, if you want to get information on certain table, `col_name`, if you want to get information on certain column (can be used only with PDO::CUBRID_SCH_COL_PRIVILEGE).

The result of this function is returned as a two-dimensional array (column (associative array) * row (numeric array)). The following tables shows types of schema and the column structure of the result array to be returned based on the schema type.

Result Composition of Each Type

Schema	Column Number	Column Name	Value
PDO::CUBRID_SCH_TABLE	1	NAME	
	2	TYPE	0:system table 1:view 2:table
PDO::CUBRID_SCH_VIEW	1	NAME	
	2	TYPE	1:view
PDO::CUBRID_SCH_QUERY_SPEC	1	QUERY_SPEC	
PDO::CUBRID_SCH_ATTRIBUTE / PDO::CUBRID_SCH_TABLE_ATTRIBUTE	1	ATTR_NAME	
	2	DOMAIN	
	3	SCALE	
	4	PRECISION	
	5	INDEXED	1:indexed
	6	NOT NULL	1:not null
	7	SHARED	1:shared
	8	UNIQUE	1:unique
	9	DEFAULT	
	10	ATTR_ORDER	base:1
	11	CLASS_NAME	
	12	SOURCE_CLASS	
	13	IS_KEY	1:key

Schema	Column Number	Column Name	Value
PDO::CUBRID_SCH_METHOD / PDO::CUBRID_SCH_TABLE_METHOD	1	NAME	
	2	RET_DOMAIN	
	3	ARG_DOMAIN	
PDO::CUBRID_SCH_METHOD_FILE	1	METHOD_FILE	
PDO::CUBRID_SCH_SUPER_TABLE / PDO::CUBRID_SCH_DIRECT_SUPER_TABLE / PDO::CUBRID_SCH_SUB_TABLE	1	CLASS_NAME	
	2	TYPE	0:system table 1:view 2:table
PDO::CUBRID_SCH_CONSTRAINT	1	TYPE	0:unique 1:index 2:reverse unique 3:reverse index
	2	NAME	
	3	ATTR_NAME	
	4	NUM_PAGES	
	5	NUM_KEYS	
	6	PRIMARY_KEY	1:primary key
	7	KEY_ORDER	base:1
PDO::CUBRID_SCH_TRIGGER	1	NAME	
	2	STATUS	
	3	EVENT	
	4	TARGET_CLASS	
	5	TARGET_ATTR	
	6	ACTION_TIME	
	7	ACTION	
	8	PRIORITY	
	9	CONDITION_TIME	
	10	CONDITION	
PDO::CUBRID_SCH_TABLE_PRIVILEGE / PDO::CUBRID_SCH_COL_PRIVILEGE	1	CLASS_NAME / ATTR_NAME	
	2	PRIVILEGE	
	3	GRANTABLE	
PDO::CUBRID_SCH_PRIMARY_KEY	1	CLASS_NAME	
	2	ATTR_NAME	
	3	KEY_SEQ	base:1
	4	KEY_NAME	

Schema	Column Number	Column Name	Value
PDO::CUBRID_SCH_IMPORTED_KEYS / PDO::CUBRID_SCH_EXPORTED_KEYS / PDO::CUBRID_SCH_CROSS_REFERENCE	1	PKTABLE_NAME	
	2	PKCOLUMN_NAME	
	3	FKTABLE_NAME	base:1
	4	FKCOLUMN_NAME	
	5	KEY_SEQ	base:1
	6	UPDATE_ACTION	0:cascade 1:restrict 2:no action 3:set null
	7	DELETE_ACTION	0:cascade 1:restrict 2:no action 3:set null
	8	FK_NAME	
	9	PK_NAME	

Parameters

`schema_type`

Schema type that you want to know.

`table_name`

Table you want to know the schema of.

`col_name`

Column you want to know the schema of.

Return Values

Array containing the schema information, when process is successful;

FALSE, when process is unsuccessful

Examples

Example #1 A PDO::cubrid_schema() example

This example shows how to get primary key and foreign keys of table game.

```
<?php
$pk_list = $dbh->cubrid_schema(PDO::CUBRID_SCH_PRIMARY_KEY, "game");
print_r($pk_list);

$fk_list = $dbh->cubrid_schema(PDO::CUBRID_SCH_IMPORTED_KEYS, "game");
print_r($fk_list);
?>
```

The above example will output:

```
Result:
Array
(
    [0] => Array
        (
            [CLASS_NAME] => game
            [ATTR_NAME] => athlete_code
            [KEY_SEQ] => 3
            [KEY_NAME] => pk_game_host_year_event_code_athlete_code
        )

    [1] => Array
        (
            [CLASS_NAME] => game
            [ATTR_NAME] => event_code
            [KEY_SEQ] => 2
            [KEY_NAME] => pk_game_host_year_event_code_athlete_code
        )

    [2] => Array
        (
            [CLASS_NAME] => game
            [ATTR_NAME] => host_year
            [KEY_SEQ] => 1
            [KEY_NAME] => pk_game_host_year_event_code_athlete_code
        )

)
Array
(
    [0] => Array
        (
            [PKTABLE_NAME] => athlete
            [PKCOLUMN_NAME] => code
            [FKTABLE_NAME] => game
            [FKCOLUMN_NAME] => athlete_code
            [KEY_SEQ] => 1
            [UPDATE_RULE] => 1
            [DELETE_RULE] => 1
            [FK_NAME] => fk_game_athlete_code
            [PK_NAME] => pk_athlete_code
        )

    [1] => Array
        (
            [PKTABLE_NAME] => event
            [PKCOLUMN_NAME] => code
            [FKTABLE_NAME] => game
            [FKCOLUMN_NAME] => event_code
            [KEY_SEQ] => 1
            [UPDATE_RULE] => 1
            [DELETE_RULE] => 1
            [FK_NAME] => fk_game_event_code
            [PK_NAME] => pk_event_code
```

```
        )

)
```

MS SQL Server PDO Driver

Introduction

PDO_DBLIB is a driver that implements the PHP Data Objects (PDO) interface to enable access from PHP to Microsoft SQL Server and Sybase databases through the FreeTDS library.

This extension is not available anymore on Windows.

On Windows, you should use SqlSrv, an alternative driver for MS SQL is available from Microsoft:
» http://msdn.microsoft.com/en-us/sqlserver/ff657782.aspx .

If it is not possible to use SqlSrv, you can use the PDO_ODBC driver to connect to Microsoft SQL Server and Sybase databases, as the native Windows DB-LIB is ancient, thread un-safe and no longer supported by Microsoft.

Table of Contents

PDO_DBLIB DSN

PDO_DBLIB DSN

(PECL PDO_DBLIB >= 0.9.0)

PDO_DBLIB DSN — Connecting to Microsoft SQL Server and Sybase databases

Description

The PDO_DBLIB Data Source Name (DSN) is composed of the following elements:

DSN prefix

> The DSN prefix is **`sybase:`** if PDO_DBLIB was linked against the Sybase ct-lib libraries, **`mssql:`** if PDO_DBLIB was linked against the Microsoft SQL Server libraries, or **`dblib:`** if PDO_DBLIB was linked against the FreeTDS libraries.

`host`

> The hostname on which the database server resides. Defaults to 127.0.0.1.

`dbname`

> The name of the database.

`charset`

> The client character set.

`appname`

The application name (used in sysprocesses). Defaults to "PHP Generic DB-lib" or "PHP freetds".

`secure`

Currently unused.

Examples

Example #1 PDO_DBLIB DSN examples

The following examples show a PDO_DBLIB DSN for connecting to Microsoft SQL Server and Sybase databases:

```
mssql:host=localhost;dbname=testdb
sybase:host=localhost;dbname=testdb
dblib:host=localhost;dbname=testdb
```

Pdo\Dblib

(PHP 8 >= 8.4.0)

Introduction

A PDO subclass representing a connection using the DBLib PDO driver.

Class synopsis

namespace **Pdo**;
class **Dblib** extends PDO {
/* Inherited constants */
public const int *PDO::PARAM_NULL*;
public const int *PDO::PARAM_BOOL* = 5;
public const int *PDO::PARAM_INT* = 1;
public const int *PDO::PARAM_STR* = 2;
public const int *PDO::PARAM_LOB* = 3;
public const int *PDO::PARAM_STMT* = 4;
public const int *PDO::PARAM_INPUT_OUTPUT*;
public const int *PDO::PARAM_STR_NATL*;
public const int *PDO::PARAM_STR_CHAR*;
public const int *PDO::PARAM_EVT_ALLOC*;
public const int *PDO::PARAM_EVT_FREE*;
public const int *PDO::PARAM_EVT_EXEC_PRE*;
public const int *PDO::PARAM_EVT_EXEC_POST*;
public const int *PDO::PARAM_EVT_FETCH_PRE*;
public const int *PDO::PARAM_EVT_FETCH_POST*;
public const int *PDO::PARAM_EVT_NORMALIZE*;
public const int *PDO::FETCH_DEFAULT*;
public const int *PDO::FETCH_LAZY*;
public const int *PDO::FETCH_ASSOC*;
public const int *PDO::FETCH_NUM*;
public const int *PDO::FETCH_BOTH*;
public const int *PDO::FETCH_OBJ*;
public const int *PDO::FETCH_BOUND*;
public const int *PDO::FETCH_COLUMN*;
public const int *PDO::FETCH_CLASS*;

```
public const int PDO::FETCH_INTO;
public const int PDO::FETCH_FUNC;
public const int PDO::FETCH_GROUP;
public const int PDO::FETCH_UNIQUE;
public const int PDO::FETCH_KEY_PAIR;
public const int PDO::FETCH_CLASSTYPE;
public const int PDO::FETCH_SERIALIZE;
public const int PDO::FETCH_PROPS_LATE;
public const int PDO::FETCH_NAMED;
public const int PDO::ATTR_AUTOCOMMIT;
public const int PDO::ATTR_PREFETCH;
public const int PDO::ATTR_TIMEOUT;
public const int PDO::ATTR_ERRMODE;
public const int PDO::ATTR_SERVER_VERSION;
public const int PDO::ATTR_CLIENT_VERSION;
public const int PDO::ATTR_SERVER_INFO;
public const int PDO::ATTR_CONNECTION_STATUS;
public const int PDO::ATTR_CASE;
public const int PDO::ATTR_CURSOR_NAME;
public const int PDO::ATTR_CURSOR;
public const int PDO::ATTR_ORACLE_NULLS;
public const int PDO::ATTR_PERSISTENT;
public const int PDO::ATTR_STATEMENT_CLASS;
public const int PDO::ATTR_FETCH_TABLE_NAMES;
public const int PDO::ATTR_FETCH_CATALOG_NAMES;
public const int PDO::ATTR_DRIVER_NAME;
public const int PDO::ATTR_STRINGIFY_FETCHES;
public const int PDO::ATTR_MAX_COLUMN_LEN;
public const int PDO::ATTR_EMULATE_PREPARES;
public const int PDO::ATTR_DEFAULT_FETCH_MODE;
public const int PDO::ATTR_DEFAULT_STR_PARAM;
public const int PDO::ERRMODE_SILENT;
public const int PDO::ERRMODE_WARNING;
public const int PDO::ERRMODE_EXCEPTION;
public const int PDO::CASE_NATURAL;
public const int PDO::CASE_LOWER;
public const int PDO::CASE_UPPER;
public const int PDO::NULL_NATURAL;
public const int PDO::NULL_EMPTY_STRING;
public const int PDO::NULL_TO_STRING;
public const string PDO::ERR_NONE;
public const int PDO::FETCH_ORI_NEXT;
public const int PDO::FETCH_ORI_PRIOR;
public const int PDO::FETCH_ORI_FIRST;
public const int PDO::FETCH_ORI_LAST;
public const int PDO::FETCH_ORI_ABS;
public const int PDO::FETCH_ORI_REL;
public const int PDO::CURSOR_FWDONLY;
public const int PDO::CURSOR_SCROLL;
/* Constants */
public const int ATTR_CONNECTION_TIMEOUT;
```

```
public const int ATTR_QUERY_TIMEOUT;
public const bool ATTR_STRINGIFY_UNIQUEIDENTIFIER;
public const int ATTR_VERSION;
public const int ATTR_TDS_VERSION;
public const int ATTR_SKIP_EMPTY_ROWSETS;
public const bool ATTR_DATETIME_CONVERT;
/* Inherited methods */
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function PDO::beginTransaction(): bool
public function PDO::commit(): bool
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function PDO::errorCode(): ?string
public function PDO::errorInfo(): array
public function PDO::exec(string $statement): int|false
public function PDO::getAttribute(int $attribute): mixed
public static function PDO::getAvailableDrivers(): array
public function PDO::inTransaction(): bool
public function PDO::lastInsertId(?string $name = null): string|false
public function PDO::prepare(string $query, array $options = []): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = null): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno):
PDOStatement|false
public function PDO::query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object):
PDOStatement|false
public function PDO::quote(string $string, int $type = PDO::PARAM_STR): string|false
public function PDO::rollBack(): bool
public function PDO::setAttribute(int $attribute, mixed $value): bool
}
```

Predefined Constants

Pdo\Dblib::ATTR_CONNECTION_TIMEOUT

Pdo\Dblib::ATTR_QUERY_TIMEOUT

Pdo\Dblib::ATTR_STRINGIFY_UNIQUEIDENTIFIER

As of PHP 8.4.0 this constant is typed as bool; previously, it was typed as int.

`Pdo\Dblib::ATTR_VERSION`

`Pdo\Dblib::ATTR_TDS_VERSION`

`Pdo\Dblib::ATTR_SKIP_EMPTY_ROWSETS`

`Pdo\Dblib::ATTR_DATETIME_CONVERT`

This connection attribute controls the format of strings for datetime types. When this is `false`, PDO_DBLIB will return a datetime type as a string in the format that SQL Server returns it in (i.e. `"2017-10-27 10:22:44"`). When `true`, PDO_DBLIB will convert the datetime type into a string using a user-defined or locale format, as specified in the FreeTDS *locales.conf* file. By default, this attribute is `false`.

As of PHP 8.4.0 this constant is typed as bool; previously, it was typed as int.

Firebird PDO Driver

Introduction

PDO_FIREBIRD is a driver that implements the PHP Data Objects (PDO) interface to enable access from PHP to Firebird database.

Installation

Use **--with-pdo-firebird[=DIR]** to install the PDO Firebird extension, where the optional `[=DIR]` is the Firebird base install directory.

```
$ ./configure --with-pdo-firebird
```

As of PHP 8.4.0, this extension uses Firebird C++ APIs and therefore requires a C++ compiler, and must be built against fbclient 3.0 or higher.

Predefined Constants

The constants below are defined by this driver, and will only be available when the extension has been either compiled into PHP or dynamically loaded at runtime. In addition, these driver-specific constants should only be used if you are using this driver. Using driver-specific attributes with another driver may result in unexpected behaviour. PDO::getAttribute() may be used to obtain the `PDO::ATTR_DRIVER_NAME` attribute to check the driver, if your code can run against multiple drivers.

> **Warning**
>
> The constants listed below have been *DEPRECATED* as of PHP 8.5.0. Use the corresponding Pdo\Firebird constants instead.

`PDO::FB_ATTR_DATE_FORMAT` (int)

Alias of `Pdo\Firebird::ATTR_DATE_FORMAT`.

`PDO::FB_ATTR_TIME_FORMAT` (int)

Alias of `Pdo\Firebird::ATTR_TIME_FORMAT`.

`PDO::FB_ATTR_TIMESTAMP_FORMAT` (int)

Alias of `Pdo\Firebird::ATTR_TIMESTAMP_FORMAT`.

Table of Contents

PDO_FIREBIRD DSN

PDO_FIREBIRD DSN

(PECL PDO_FIREBIRD >= 0.1.0)

PDO_FIREBIRD DSN — Connecting to Firebird databases

Description

The PDO_FIREBIRD Data Source Name (DSN) is composed of the following elements:

DSN prefix

The DSN prefix is **`firebird:`**.

`dbname`

The name of the database.

`charset`

The character set.

`role`

The SQL role name.

`dialect`

The dialect of the database; either `1` or `3`. If not specified, the dialect defaults to `3`. Available as of PHP 7.4.0.

Examples

Example #1 PDO_FIREBIRD DSN example with path

The following example shows a PDO_FIREBIRD DSN for connecting to Firebird databases:

```
firebird:dbname=/path/to/DATABASE.FDB
```

Example #2 PDO_FIREBIRD DSN example with port and path

The following example shows a PDO_FIREBIRD DSN for connecting to a Firebird database using hostname port and path:

```
firebird:dbname=hostname/port:/path/to/DATABASE.FDB
```

Example #3 PDO_FIREBIRD DSN example with localhost and path to employee.fdb on Debian system

The following example shows a PDO_FIREBIRD DSN for connecting to a Firebird database employee.fdb using localhost:

```
firebird:dbname=localhost:/var/lib/firebird/2.5/data/employee.fdb
```

Example #4 PDO_FIREBIRD DSN to connect to a dialect 1 database

The following example shows a PDO_FIREBIRD DSN for connecting to a Firebird database test.fdb which has been created using dialect 1. This is only supported as of PHP 7.4.0.

```
firebird:dbname=localhost:/var/lib/firebird/2.5/data/test.fdb;charset=utf-8;dialect=1
```

Pdo\Firebird

(PHP 8 >= 8.4.0)

Introduction

A PDO subclass representing a connection using the Firebird PDO driver.

Class synopsis

namespace **Pdo**;
class **Firebird** extends PDO {
/* Inherited constants */
public const int *PDO::PARAM_NULL*;
public const int *PDO::PARAM_BOOL* = 5;
public const int *PDO::PARAM_INT* = 1;
public const int *PDO::PARAM_STR* = 2;
public const int *PDO::PARAM_LOB* = 3;
public const int *PDO::PARAM_STMT* = 4;
public const int *PDO::PARAM_INPUT_OUTPUT*;
public const int *PDO::PARAM_STR_NATL*;
public const int *PDO::PARAM_STR_CHAR*;
public const int *PDO::PARAM_EVT_ALLOC*;
public const int *PDO::PARAM_EVT_FREE*;
public const int *PDO::PARAM_EVT_EXEC_PRE*;
public const int *PDO::PARAM_EVT_EXEC_POST*;
public const int *PDO::PARAM_EVT_FETCH_PRE*;
public const int *PDO::PARAM_EVT_FETCH_POST*;
public const int *PDO::PARAM_EVT_NORMALIZE*;
public const int *PDO::FETCH_DEFAULT*;
public const int *PDO::FETCH_LAZY*;
public const int *PDO::FETCH_ASSOC*;
public const int *PDO::FETCH_NUM*;
public const int *PDO::FETCH_BOTH*;
public const int *PDO::FETCH_OBJ*;
public const int *PDO::FETCH_BOUND*;
public const int *PDO::FETCH_COLUMN*;
public const int *PDO::FETCH_CLASS*;
public const int *PDO::FETCH_INTO*;
public const int *PDO::FETCH_FUNC*;

```
public const int PDO::FETCH_GROUP;
public const int PDO::FETCH_UNIQUE;
public const int PDO::FETCH_KEY_PAIR;
public const int PDO::FETCH_CLASSTYPE;
public const int PDO::FETCH_SERIALIZE;
public const int PDO::FETCH_PROPS_LATE;
public const int PDO::FETCH_NAMED;
public const int PDO::ATTR_AUTOCOMMIT;
public const int PDO::ATTR_PREFETCH;
public const int PDO::ATTR_TIMEOUT;
public const int PDO::ATTR_ERRMODE;
public const int PDO::ATTR_SERVER_VERSION;
public const int PDO::ATTR_CLIENT_VERSION;
public const int PDO::ATTR_SERVER_INFO;
public const int PDO::ATTR_CONNECTION_STATUS;
public const int PDO::ATTR_CASE;
public const int PDO::ATTR_CURSOR_NAME;
public const int PDO::ATTR_CURSOR;
public const int PDO::ATTR_ORACLE_NULLS;
public const int PDO::ATTR_PERSISTENT;
public const int PDO::ATTR_STATEMENT_CLASS;
public const int PDO::ATTR_FETCH_TABLE_NAMES;
public const int PDO::ATTR_FETCH_CATALOG_NAMES;
public const int PDO::ATTR_DRIVER_NAME;
public const int PDO::ATTR_STRINGIFY_FETCHES;
public const int PDO::ATTR_MAX_COLUMN_LEN;
public const int PDO::ATTR_EMULATE_PREPARES;
public const int PDO::ATTR_DEFAULT_FETCH_MODE;
public const int PDO::ATTR_DEFAULT_STR_PARAM;
public const int PDO::ERRMODE_SILENT;
public const int PDO::ERRMODE_WARNING;
public const int PDO::ERRMODE_EXCEPTION;
public const int PDO::CASE_NATURAL;
public const int PDO::CASE_LOWER;
public const int PDO::CASE_UPPER;
public const int PDO::NULL_NATURAL;
public const int PDO::NULL_EMPTY_STRING;
public const int PDO::NULL_TO_STRING;
public const string PDO::ERR_NONE;
public const int PDO::FETCH_ORI_NEXT;
public const int PDO::FETCH_ORI_PRIOR;
public const int PDO::FETCH_ORI_FIRST;
public const int PDO::FETCH_ORI_LAST;
public const int PDO::FETCH_ORI_ABS;
public const int PDO::FETCH_ORI_REL;
public const int PDO::CURSOR_FWDONLY;
public const int PDO::CURSOR_SCROLL;
/* Constants */
public const int ATTR_DATE_FORMAT;
public const int ATTR_TIME_FORMAT;
public const int ATTR_TIMESTAMP_FORMAT;
```

```
public const int TRANSACTION_ISOLATION_LEVEL;
public const int READ_COMMITTED;
public const int REPEATABLE_READ;
public const int SERIALIZABLE;
public const int WRITABLE_TRANSACTION;
/* Methods */
public static function getApiVersion(): int
/* Inherited methods */
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function PDO::beginTransaction(): bool
public function PDO::commit(): bool
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function PDO::errorCode(): ?string
public function PDO::errorInfo(): array
public function PDO::exec(string $statement): int|false
public function PDO::getAttribute(int $attribute): mixed
public static function PDO::getAvailableDrivers(): array
public function PDO::inTransaction(): bool
public function PDO::lastInsertId(?string $name = null): string|false
public function PDO::prepare(string $query, array $options = []): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = null): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno):
PDOStatement|false
public function PDO::query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object):
PDOStatement|false
public function PDO::quote(string $string, int $type = PDO::PARAM_STR): string|false
public function PDO::rollBack(): bool
public function PDO::setAttribute(int $attribute, mixed $value): bool
}
```

Predefined Constants

`Pdo\Firebird::ATTR_DATE_FORMAT`

Sets the date format.

`Pdo\Firebird::ATTR_TIME_FORMAT`

Sets the time format.

`Pdo\Firebird::ATTR_TIMESTAMP_FORMAT`

Sets the timestamp format.

`Pdo\Firebird::TRANSACTION_ISOLATION_LEVEL`

Attribute to sets the transaction isolation level. This can be one of `Pdo\Firebird::READ_COMMITTED`, `Pdo\Firebird::REPEATABLE_READ`, or `Pdo\Firebird::SERIALIZABLE`.

`Pdo\Firebird::READ_COMMITTED`

Flag denoting the ANSI transaction isolation level is read committed. This is the default behavior.

`Pdo\Firebird::REPEATABLE_READ`

Flag denoting the ANSI transaction isolation level is repeatable read. This corresponds to Firebird's "snapshot" isolation level.

`Pdo\Firebird::SERIALIZABLE`

Flag denoting the ANSI transaction isolation level is serializable. This corresponds to Firebird's "snapshot table stability" isolation level.

`Pdo\Firebird::WRITABLE_TRANSACTION`

Boolean attribute used to toggle the transaction access mode between `READ ONLY` and `READ WRITE`. By default, it is `true` indicating `READ WRITE`.

Table of Contents

Pdo\Firebird::getApiVersion

Pdo\Firebird::getApiVersion

(PHP 8 >= 8.4.0)

Pdo\Firebird::getApiVersion — Get the API version

Description

public static function **Pdo\Firebird::getApiVersion**(): int

Returns the Firebird API version as defined by the C constant **`FB_API_VER`** in *ibase.h*.

Parameters

This function has no parameters.

Return Values

Returns the FireBird API as an int.

IBM PDO Driver

Introduction

PDO_IBM is a driver that implements the PHP Data Objects (PDO) interface to enable access from PHP to IBM databases.

Installation

To build the PDO_IBM extension, the DB2 Client v9.1 or later must be installed on the same system as PHP. The DB2 Client can be downloaded from the IBM » Application Development Site.

> **Note** : **Note**
>
> The DB2 Client v9.1 or later supports direct access to DB2 for Linux, UNIX, and Windows v8 and v9.1 servers.
>
> The DB2 Client v9.1 also supports access to DB2 UDB for i5 and DB2 UDB for z/OS servers using the separately purchased » DB2 Connect product.

PDO_IBM is a » PECL extension, so follow the instructions in Installation of PECL extensions to install the PDO_IBM extension. Issue the **configure** command to point to the location of the DB2 Client header files and libraries as follows:

```
bash$ ./configure --with-pdo-ibm=/path/to/sqllib[,shared]
```

The **configure** command defaults to the value of the *DB2DIR* environment variable.

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

PDO_IBM Configuration Options

Name	Default	Changeable	Changelog
pdo_ibm.i5_dbcs_alloc	"0"	INI_SYSTEM	Added in PDO_IBM 1.5.0
pdo_ibm.i5_override_ccsid	"0"	INI_SYSTEM	Added in PDO_IBM 1.5.0

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`pdo_ibm.i5_dbcs_alloc` int

This options affects the internal buffer allocation strategy on IBM i. By default, this option is 0. When this option is set, buffers are allocated with a much larger size, in case the database is misleading about character size when converting between encodings. This option uses six times as much memory for buffers (to account for the largest possible UTF-8 sequences), but may be needed if truncated data is returned.

- 0 - Minimum size buffers are allocated.
- 1 - Larger size buffers are allocated.

`pdo_ibm.i5_override_ccsid` int

The ASCII CCSID to use for conversion from EBCDIC on IBM i. Setting this to 1208 will use UTF-8. By default, this is 0, which will select the default ASCII job CCSID.

To learn more about CCSIDs on IBM i, consult the » IBM documentation.

Table of Contents

PDO_IBM DSN

PDO_IBM DSN

(PECL PDO_IBM >= 0.9.0)

PDO_IBM DSN — Connecting to IBM databases

Description

The PDO_IBM Data Source Name (DSN) is based on the IBM CLI DSN. The major components of the PDO_IBM DSN are:

DSN prefix

The DSN prefix is **`ibm:`**.

DSN

The DSN can be any of the following:

- a) Data source setup using *db2cli.ini* or *odbc.ini*
- b) Catalogued database name i.e. database alias in the DB2 client catalog
- c) Complete connection string in the following format: `DRIVER={IBM DB2 ODBC DRIVER};DATABASE=database;HOSTNAME=hostname;PORT=port;PROTOCOL=TCPIP;UID=username;PWD=password;` where the parameters represent the following values:

  `database`

  The name of the database.

  `hostname`

  The hostname or IP address of the database server.

  `port`

  The TCP/IP port on which the database is listening for requests.

  `username`

  The username with which you are connecting to the database.

  `password`

  The password with which you are connecting to the database.

Examples

Example #1 PDO_IBM DSN example using *db2cli.ini*

The following example shows a PDO_IBM DSN for connecting to an DB2 database cataloged as DB2_9 in *db2cli.ini*:

```
$db = new PDO("ibm:DSN=DB2_9", "", "");

[DB2_9]
Database=testdb
Protocol=tcpip
Hostname=11.22.33.444
Servicename=56789
```

Example #2 PDO_IBM DSN example using a connection string

The following example shows a PDO_IBM DSN for connecting to an DB2 database named **`testdb`** using the DB2 CLI connection string syntax.

```
$db = new PDO("ibm:DRIVER={IBM DB2 ODBC DRIVER};DATABASE=testdb;" .
  "HOSTNAME=11.22.33.444;PORT=56789;PROTOCOL=TCPIP;", "testuser", "tespass");
```

Informix PDO Driver

Introduction

PDO_INFORMIX is a driver that implements the PHP Data Objects (PDO) interface to enable access from PHP to Informix databases.

Installation

To build the PDO_INFORMIX extension, the Informix Client SDK 2.81 UC1 or higher must be installed on the same system as PHP. The Informix Client SDK is available from the » IBM Informix Support Site.

PDO_INFORMIX is a » PECL extension, so follow the instructions in Installation of PECL extensions to install the PDO_INFORMIX extension. Issue the **configure** command to point to the location of the Informix Client SDK header files and libraries as follows:

```
bash$ ./configure --with-pdo-informix=/path/to/SDK[,shared]
```

The **configure** command defaults to the value of the *INFORMIXDIR* environment variable.

Scrollable cursors

PDO_INFORMIX supports scrollable cursors; however, they are not enabled by default. To enable scrollable cursor support, you must either set **`ENABLESCROLLABLECURSORS=1`** in the corresponding ODBC connection settings in *odbc.ini* or pass the **`EnableScrollableCursors=1`** clause in the DSN connection string.

Table of Contents

PDO_INFORMIX DSN

PDO_INFORMIX DSN

(PECL PDO_INFORMIX >= 0.1.0)

PDO_INFORMIX DSN — Connecting to Informix databases

Description

The PDO_INFORMIX Data Source Name (DSN) is based on the Informix ODBC DSN string. Details on configuring an Informix ODBC DSN are available from the » Informix Dynamic Server Information Center. The major components of the PDO_INFORMIX DSN are:

DSN prefix

The DSN prefix is **`informix:`**.

DSN

The DSN can be either a data source setup using *odbc.ini* or a complete » connection string.

Examples

Example #1 PDO_INFORMIX DSN example using *odbc.ini*

The following example shows a PDO_INFORMIX DSN for connecting to an Informix database cataloged as Infdrv33 in *odbc.ini*:

```
$db = new PDO("informix:DSN=Infdrv33", "", "");
```

```
[ODBC Data Sources]
Infdrv33=INFORMIX 3.3 32-BIT

[Infdrv33]
Driver=/opt/informix/csdk_2.81.UC1G2/lib/cli/iclis09b.so
Description=INFORMIX 3.3 32-BIT
Database=common_db
LogonID=testuser
pwd=testpass
Servername=ids_server
DB_LOCALE=en_US.819
OPTIMIZEAUTOCOMMIT=1
ENABLESCROLLABLECURSORS=1
```

Example #2 PDO_INFORMIX DSN example using a connection string

The following example shows a PDO_INFORMIX DSN for connecting to an Informix database named **`common_db`** using the Informix connection string syntax.

```
$db = new PDO("informix:host=host.domain.com; service=9800;
    database=common_db; server=ids_server; protocol=onsoctcp;
    EnableScrollableCursors=1", "testuser", "tespass");
```

MySQL PDO Driver

Introduction

PDO_MYSQL is a driver that implements the PHP Data Objects (PDO) interface to enable access from PHP to MySQL databases.

PDO_MYSQL uses emulated prepares by default.

MySQL 8

When running a PHP version before 7.1.16, or PHP 7.2 before 7.2.4, set MySQL 8 Server's default password plugin to *mysql_native_password* or else you will see errors similar to *The server requested authentication method unknown to the client [caching_sha2_password]* even when *caching_sha2_password* is not used.

This is because MySQL 8 defaults to caching_sha2_password, a plugin that is not recognized by the older PHP (mysqlnd) releases. Instead, change it by setting `default_authentication_plugin=mysql_native_password` in *my.cnf*. The *caching_sha2_password* plugin is fully supported as of PHP 7.4.4. For older releases, the mysql_xdevapi extension does support it.

> **Warning**
>
> Beware: Some MySQL table types (storage engines) do not support transactions. When writing transactional database code using a table type that does not support transactions, MySQL will pretend that a transaction was initiated successfully. In addition, any DDL queries issued will implicitly commit any pending transactions.

> **Note** :
>
> The MySQL driver does not properly support **`PDO::PARAM_INPUT_OUTPUT`** via PDOStatement::bindParam(); while such parameters can be used, they are not updated (i.e. the actual output is ignored).

Installation

The common Unix distributions include binary versions of PHP that can be installed. Although these binary versions are typically built with support for the MySQL extensions, the extension libraries themselves may need to be installed using an additional package. Check the package manager that comes with your chosen distribution for availability.

For example, on Ubuntu the `php5-mysql` package installs the ext/mysql, ext/mysqli, and PDO_MYSQL PHP extensions. On CentOS, the `php-mysql` package also installs these three PHP extensions.

Alternatively, you can compile this extension yourself. Building PHP from source allows you to specify the MySQL extensions you want to use, as well as your choice of client library for each extension.

When compiling, use **--with-pdo-mysql[=DIR]** to install the PDO MySQL extension, where the optional `[=DIR]` is the MySQL base library. Mysqlnd is the default library. For details about choosing a library, see Choosing a MySQL library.

Optionally, the **--with-mysql-sock[=DIR]** sets to location to the MySQL unix socket pointer for all MySQL extensions, including PDO_MYSQL. If unspecified, the default locations are searched.

Optionally, the **--with-zlib-dir[=DIR]** is used to set the path to the libz install prefix.

```
$ ./configure --with-pdo-mysql --with-mysql-sock=/var/mysql/mysql.sock
```

SSL support is enabled using the appropriate `Pdo\Mysql::ATTR_SSL_*`, which is equivalent to calling the » MySQL C API function mysql_ssl_set(). Also, SSL cannot be enabled with PDO::setAttribute() because the connection already exists. See also the MySQL documentation about » connecting to MySQL with SSL.

Predefined Constants

The constants below are defined by this driver, and will only be available when the extension has been either compiled into PHP or dynamically loaded at runtime. In addition, these driver-specific constants should only be used if you are using this driver. Using driver-specific attributes with another driver may result in unexpected behaviour. PDO::getAttribute() may be used to obtain the `PDO::ATTR_DRIVER_NAME` attribute to check the driver, if your code can run against multiple drivers.

> **Warning**
>
> The constants listed below have been *DEPRECATED* as of PHP 8.5.0. Use the corresponding Pdo\Mysql constants instead.

`PDO::MYSQL_ATTR_USE_BUFFERED_QUERY` (int)

Alias of `Pdo\Mysql::ATTR_USE_BUFFERED_QUERY`

`PDO::MYSQL_ATTR_LOCAL_INFILE` (int)

Alias of `Pdo\Mysql::ATTR_LOCAL_INFILE`

`PDO::MYSQL_ATTR_LOCAL_INFILE_DIRECTORY` (int)

Alias of `Pdo\Mysql::ATTR_LOCAL_INFILE_DIRECTORY`. Available as of PHP 8.1.0.

`PDO::MYSQL_ATTR_INIT_COMMAND` (int)

Alias of `Pdo\Mysql::ATTR_INIT_COMMAND`

`PDO::MYSQL_ATTR_READ_DEFAULT_FILE` (int)

Alias of `Pdo\Mysql::ATTR_READ_DEFAULT_FILE`

`PDO::MYSQL_ATTR_READ_DEFAULT_GROUP` (int)

Alias of `Pdo\Mysql::ATTR_READ_DEFAULT_GROUP`

`PDO::MYSQL_ATTR_MAX_BUFFER_SIZE` (int)

Alias of `Pdo\Mysql::ATTR_MAX_BUFFER_SIZE`

`PDO::MYSQL_ATTR_DIRECT_QUERY` (bool)

Alias of `PDO::ATTR_EMULATE_PREPARES`

As of PHP 8.4.0 this constant is typed as bool; previously, it was typed as int.

`PDO::MYSQL_ATTR_FOUND_ROWS` (int)

Alias of `Pdo\Mysql::ATTR_FOUND_ROWS`

`PDO::MYSQL_ATTR_IGNORE_SPACE` (int)

Alias of `Pdo\Mysql::ATTR_IGNORE_SPACE`

`PDO::MYSQL_ATTR_COMPRESS` (int)

Alias of `Pdo\Mysql::ATTR_COMPRESS`

`PDO::MYSQL_ATTR_SERVER_PUBLIC_KEY` (int)

Alias of `Pdo\Mysql::ATTR_SERVER_PUBLIC_KEY`

`PDO::MYSQL_ATTR_SSL_CA` (int)

Alias of `Pdo\Mysql::ATTR_SSL_CA`

`PDO::MYSQL_ATTR_SSL_CAPATH` (int)

Alias of `Pdo\Mysql::ATTR_SSL_CAPATH`

`PDO::MYSQL_ATTR_SSL_CERT` (int)

Alias of `Pdo\Mysql::ATTR_SSL_CERT`

`PDO::MYSQL_ATTR_SSL_CIPHER` (int)

Alias of `Pdo\Mysql::ATTR_SSL_CIPHER`

`PDO::MYSQL_ATTR_SSL_KEY` (int)

Alias of `Pdo\Mysql::ATTR_SSL_KEY`

`PDO::MYSQL_ATTR_SSL_VERIFY_SERVER_CERT` (int)

Alias of `Pdo\Mysql::ATTR_SSL_VERIFY_SERVER_CERT` Available as of PHP 7.0.18 and PHP 7.1.4.

`PDO::MYSQL_ATTR_MULTI_STATEMENTS` (int)

Alias of `Pdo\Mysql::ATTR_MULTI_STATEMENTS`

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

PDO_MYSQL Configuration Options

Name	Default	Changeable
pdo_mysql.default_socket	"/tmp/mysql.sock"	`INI_SYSTEM`
pdo_mysql.debug	NULL	`INI_SYSTEM`

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`pdo_mysql.default_socket` string

Sets a Unix domain socket. This value can either be set at compile time if a domain socket is found at configure. This ini setting is Unix only.

`pdo_mysql.debug` bool

Enables debugging for PDO_MYSQL. This setting is only available when PDO_MYSQL is compiled against mysqlnd and in PDO debug mode.

Table of Contents

PDO_MYSQL DSN

PDO_MYSQL DSN

(PECL PDO_MYSQL >= 0.1.0)

PDO_MYSQL DSN — Connecting to MySQL databases

Description

The PDO_MYSQL Data Source Name (DSN) is composed of the following elements:

DSN prefix

The DSN prefix is **`mysql:`**.

`host`

The hostname on which the database server resides.

`port`

The port number where the database server is listening.

`dbname`

The name of the database.

`user`

The name of the user for the connection. A user name given as the second argument to the PDO constructor takes precedence over one specified in the DSN.

`password`

The password of the user for the connection. A password given as the third argument to the PDO constructor takes precedence over one specified in the DSN.

`unix_socket`

The MySQL Unix socket (shouldn't be used with `host` or `port`).

`charset`

The character set. See the character set concepts documentation for more information.

Examples

Example #1 PDO_MYSQL DSN examples

The following example shows a PDO_MYSQL DSN for connecting to MySQL databases:

```
mysql:host=localhost;dbname=testdb
```

More complete examples:

```
mysql:host=localhost;port=3307;dbname=testdb
mysql:unix_socket=/tmp/mysql.sock;dbname=testdb
```

Notes

> **Note** : **Unix only:**
>
> When the host name is set to `"localhost"`, then the connection to the server is made through a domain socket. If PDO_MYSQL is compiled against libmysqlclient then the location of the socket file is at libmysqlclient's compiled in location. If PDO_MYSQL is compiled against mysqlnd a default socket can be set through the pdo_mysql.default_socket setting.

Pdo\Mysql

(PHP 8 >= 8.4.0)

Introduction

A PDO subclass representing a connection using the MySQL PDO driver.

This driver supports a dedicated SQL query parser for the MySQL dialect. It can handle the following:

- Single and double-quoted literals with both doubling and backslash as escaping mechanisms
- Backtick literals with doubling as escaping mechanism
- Two-dashes, C-style comments, and Hash-comments.

Class synopsis

class **Pdo\Mysql** extends PDO {
/* Inherited constants */
public const int *PDO::PARAM_NULL*;
public const int *PDO::PARAM_BOOL* = 5;
public const int *PDO::PARAM_INT* = 1;
public const int *PDO::PARAM_STR* = 2;
public const int *PDO::PARAM_LOB* = 3;
public const int *PDO::PARAM_STMT* = 4;
public const int *PDO::PARAM_INPUT_OUTPUT*;
public const int *PDO::PARAM_STR_NATL*;
public const int *PDO::PARAM_STR_CHAR*;
public const int *PDO::PARAM_EVT_ALLOC*;
public const int *PDO::PARAM_EVT_FREE*;
public const int *PDO::PARAM_EVT_EXEC_PRE*;
public const int *PDO::PARAM_EVT_EXEC_POST*;
public const int *PDO::PARAM_EVT_FETCH_PRE*;
public const int *PDO::PARAM_EVT_FETCH_POST*;
public const int *PDO::PARAM_EVT_NORMALIZE*;
public const int *PDO::FETCH_DEFAULT*;

public const int *PDO::FETCH_LAZY*;
public const int *PDO::FETCH_ASSOC*;
public const int *PDO::FETCH_NUM*;
public const int *PDO::FETCH_BOTH*;
public const int *PDO::FETCH_OBJ*;
public const int *PDO::FETCH_BOUND*;
public const int *PDO::FETCH_COLUMN*;
public const int *PDO::FETCH_CLASS*;
public const int *PDO::FETCH_INTO*;
public const int *PDO::FETCH_FUNC*;
public const int *PDO::FETCH_GROUP*;
public const int *PDO::FETCH_UNIQUE*;
public const int *PDO::FETCH_KEY_PAIR*;
public const int *PDO::FETCH_CLASSTYPE*;
public const int *PDO::FETCH_SERIALIZE*;
public const int *PDO::FETCH_PROPS_LATE*;
public const int *PDO::FETCH_NAMED*;
public const int *PDO::ATTR_AUTOCOMMIT*;
public const int *PDO::ATTR_PREFETCH*;
public const int *PDO::ATTR_TIMEOUT*;
public const int *PDO::ATTR_ERRMODE*;
public const int *PDO::ATTR_SERVER_VERSION*;
public const int *PDO::ATTR_CLIENT_VERSION*;
public const int *PDO::ATTR_SERVER_INFO*;
public const int *PDO::ATTR_CONNECTION_STATUS*;
public const int *PDO::ATTR_CASE*;
public const int *PDO::ATTR_CURSOR_NAME*;
public const int *PDO::ATTR_CURSOR*;
public const int *PDO::ATTR_ORACLE_NULLS*;
public const int *PDO::ATTR_PERSISTENT*;
public const int *PDO::ATTR_STATEMENT_CLASS*;
public const int *PDO::ATTR_FETCH_TABLE_NAMES*;
public const int *PDO::ATTR_FETCH_CATALOG_NAMES*;
public const int *PDO::ATTR_DRIVER_NAME*;
public const int *PDO::ATTR_STRINGIFY_FETCHES*;
public const int *PDO::ATTR_MAX_COLUMN_LEN*;
public const int *PDO::ATTR_EMULATE_PREPARES*;
public const int *PDO::ATTR_DEFAULT_FETCH_MODE*;
public const int *PDO::ATTR_DEFAULT_STR_PARAM*;
public const int *PDO::ERRMODE_SILENT*;
public const int *PDO::ERRMODE_WARNING*;
public const int *PDO::ERRMODE_EXCEPTION*;
public const int *PDO::CASE_NATURAL*;
public const int *PDO::CASE_LOWER*;
public const int *PDO::CASE_UPPER*;
public const int *PDO::NULL_NATURAL*;
public const int *PDO::NULL_EMPTY_STRING*;
public const int *PDO::NULL_TO_STRING*;
public const string *PDO::ERR_NONE*;
public const int *PDO::FETCH_ORI_NEXT*;
public const int *PDO::FETCH_ORI_PRIOR*;

```
public const int PDO::FETCH_ORI_FIRST;
public const int PDO::FETCH_ORI_LAST;
public const int PDO::FETCH_ORI_ABS;
public const int PDO::FETCH_ORI_REL;
public const int PDO::CURSOR_FWDONLY;
public const int PDO::CURSOR_SCROLL;
/* Constants */
public const int ATTR_USE_BUFFERED_QUERY;
public const int ATTR_LOCAL_INFILE;
public const int ATTR_LOCAL_INFILE_DIRECTORY;
public const int ATTR_INIT_COMMAND;
public const int ATTR_MAX_BUFFER_SIZE;
public const int ATTR_READ_DEFAULT_FILE;
public const int ATTR_READ_DEFAULT_GROUP;
public const int ATTR_COMPRESS;
public const int ATTR_DIRECT_QUERY;
public const int ATTR_FOUND_ROWS;
public const int ATTR_IGNORE_SPACE;
public const int ATTR_MULTI_STATEMENTS;
public const int ATTR_SERVER_PUBLIC_KEY;
public const int ATTR_SSL_KEY;
public const int ATTR_SSL_CERT;
public const int ATTR_SSL_CA;
public const int ATTR_SSL_CAPATH;
public const int ATTR_SSL_CIPHER;
public const int ATTR_SSL_VERIFY_SERVER_CERT;
/* Methods */
public function getWarningCount(): int
/* Inherited methods */
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function PDO::beginTransaction(): bool
public function PDO::commit(): bool
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function PDO::errorCode(): ?string
public function PDO::errorInfo(): array
public function PDO::exec(string $statement): int|false
public function PDO::getAttribute(int $attribute): mixed
public static function PDO::getAvailableDrivers(): array
public function PDO::inTransaction(): bool
public function PDO::lastInsertId(?string $name = null): string|false
public function PDO::prepare(string $query, array $options = []): PDOStatement|false
```

```
public function PDO::query(string $query, ?int $fetchMode = null): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno): PDOStatement|false
public function PDO::query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object): PDOStatement|false
public function PDO::quote(string $string, int $type = PDO::PARAM_STR): string|false
public function PDO::rollBack(): bool
public function PDO::setAttribute(int $attribute, mixed $value): bool
}
```

Predefined Constants

Pdo\Mysql::ATTR_USE_BUFFERED_QUERY

By default all statements are executed in buffered mode. If this attribute is set to `false` on a **Pdo\Mysql** object, the MySQL driver will use the unbuffered mode.

Example #1 Setting MySQL unbuffered mode

```php
<?php
$pdo = new Pdo\Mysql("mysql:host=localhost;dbname=world", 'my_user', 'my_password');
$pdo->setAttribute(PDO::MYSQL_ATTR_USE_BUFFERED_QUERY, false);

$unbufferedResult = $pdo->query("SELECT Name FROM City");
foreach ($unbufferedResult as $row) {
    echo $row['Name'] . PHP_EOL;
}
?>
```

Pdo\Mysql::ATTR_LOCAL_INFILE

Enable `LOAD LOCAL INFILE`.

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

Pdo\Mysql::ATTR_LOCAL_INFILE_DIRECTORY

Allows restricting LOCAL DATA loading to files located in this designated directory.

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

Pdo\Mysql::ATTR_INIT_COMMAND

Command to execute when connecting to the MySQL server. Will automatically be re-executed when reconnecting.

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

`Pdo\Mysql::ATTR_READ_DEFAULT_FILE`

Read options from the named option file instead of from *my.cnf*.

> **Note** :
>
> This option is not available if mysqlnd is used, because mysqlnd does not read the mysql configuration files.

`Pdo\Mysql::ATTR_READ_DEFAULT_GROUP`

Read options from the named group from *my.cnf* or the file specified with **`Pdo\Mysql::ATTR_READ_DEFAULT_FILE`**.

> **Note** :
>
> This option is not available if mysqlnd is used, because mysqlnd does not read the mysql configuration files.

`Pdo\Mysql::ATTR_COMPRESS`

Enable network communication compression.

`Pdo\Mysql::ATTR_DIRECT_QUERY`

Alias of **`PDO::ATTR_EMULATE_PREPARES`**.

`Pdo\Mysql::ATTR_FOUND_ROWS`

Return the number of found (matched) rows, not the number of changed rows.

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

`Pdo\Mysql::ATTR_IGNORE_SPACE`

Permit spaces after SQL function names. Makes all SQL functions names reserved words.

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

Pdo\Mysql::ATTR_MAX_BUFFER_SIZE

Maximum buffer size. Defaults to 1 MiB.

> **Note** :
>
> This constant is not supported when compiled against mysqlnd.

Pdo\Mysql::ATTR_MULTI_STATEMENTS

Disables multi query execution in both PDO::prepare() and PDO::query() when set to **false**.

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

Pdo\Mysql::ATTR_SERVER_PUBLIC_KEY

RSA public key file used with the SHA-256 based authentication.

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

Pdo\Mysql::ATTR_SSL_KEY

The file path to the SSL key.

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

Pdo\Mysql::ATTR_SSL_CERT

The file path to the SSL certificate.

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

Pdo\Mysql::ATTR_SSL_CA

The file path to the SSL certificate authority.

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

Pdo\Mysql::ATTR_SSL_CAPATH

The file path to the directory that contains the trusted SSL CA certificates, which are stored in PEM format.

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

`Pdo\Mysql::ATTR_SSL_CIPHER`

A list of one or more permissible ciphers to use for SSL encryption, in a format understood by OpenSSL. For example: `DHE-RSA-AES256-SHA:AES128-SHA`

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

`Pdo\Mysql::ATTR_SSL_VERIFY_SERVER_CERT`

Provides a way to disable verification of the server SSL certificate.

> **Note** :
>
> This option is available only with mysqlnd.

> **Note** :
>
> Can only be used in the `driver_options` array when constructing a new database handle.

Table of Contents

Pdo\Mysql::getWarningCount

Pdo\Mysql::getWarningCount

(PHP 8 >= 8.4.0)

Pdo\Mysql::getWarningCount — Returns the number of warnings from the last executed query

Description

public function **Pdo\Mysql::getWarningCount**(): int

Returns the number of warnings from the last executed query.

> **Note** :
>
> For retrieving warning messages the following SQL command can be used: `SHOW WARNINGS [limit row_count]`.

Parameters

This function has no parameters.

Return Values

Returns an int representing the number of warnings generated by the last query.

Examples

Example #1 **Pdo\Mysql::getWarningCount()** example

```php
<?php

$conn = PDO::connect("mysql:host=localhost;dbname=test;charset=utf8mb4", 'user', 'password');

$conn->query('SELECT 42/0');
if ($conn->getWarningCount() > 0) {
    $result = $conn->query("SHOW WARNINGS");
    $row = $result->fetch();
    printf("%s (%d): %s\n", $row[0], $row[1], $row[2]);
}

?>
```

The above example will output:

```
Warning (1365): Division by 0
```

MS SQL Server PDO Driver

Introduction

PDO_SQLSRV is a driver that implements the PHP Data Objects (PDO) interface to enable access from PHP to MS SQL Server (starting with SQL Server 2005) and SQL Azure databases.

Installation

The most recent version of the driver is available for download here: » SQLSRV download. The driver sources are hosted in a » public repository.

For more information about system requirements, see » SQLSRV System Requirements.

On Windows the PDO_SQLSRV extension is enabled by downloading and adding appropriate DLL files to the PHP extension directory and the corresponding entry to the *php.ini* file.

On Linux and macOS, the PDO_SQLSRV extension can be installed using » PECL. See the » installation tutorial for details.

Predefined Constants

The constants below are defined by this driver, and will only be available when the extension has been either compiled into PHP or dynamically loaded at runtime. In addition, these driver-specific constants should only be used if you are using this driver. Using driver-specific attributes with another driver may result in unexpected

behaviour. PDO::getAttribute() may be used to obtain the **PDO::ATTR_DRIVER_NAME** attribute to check the driver, if your code can run against multiple drivers.

PDO::SQLSRV_TXN_READ_UNCOMMITTED (int)

This constant is an acceptable value for the SQLSRV DSN key TransactionIsolation. This constant sets the transaction isolation level for the connection to Read Uncommitted.

PDO::SQLSRV_TXN_READ_COMMITTED (int)

This constant is an acceptable value for the SQLSRV DSN key TransactionIsolation. This constant sets the transaction isolation level for the connection to Read Committed.

PDO::SQLSRV_TXN_REPEATABLE_READ (int)

This constant is an acceptable value for the SQLSRV DSN key TransactionIsolation. This constant sets the transaction isolation level for the connection to Repeateable Read.

PDO::SQLSRV_TXN_SNAPSHOT (int)

This constant is an acceptable value for the SQLSRV DSN key TransactionIsolation. This constant sets the transaction isolation level for the connection to Snapshot.

PDO::SQLSRV_TXN_SERIALIZABLE (int)

This constant is an acceptable value for the SQLSRV DSN key TransactionIsolation. This constant sets the transaction isolation level for the connection to Serializable.

PDO::SQLSRV_ENCODING_BINARY (int)

Specifies that data is sent/retrieved as a raw byte stream to/from the server without performing encoding or translation. This constant can be passed to PDOStatement::setAttribute, PDO::prepare, PDOStatement::bindColumn, and PDOStatement::bindParam.

PDO::SQLSRV_ENCODING_SYSTEM (int)

Specifies that data is sent/retrieved to/from the server as 8-bit characters as specified in the code page of the Windows locale that is set on the system. Any multi-byte characters or characters that do not map into this code page are substituted with a single byte question mark (?) character. This constant can be passed to PDOStatement::setAttribute, PDO::setAttribute, PDO::prepare, PDOStatement::bindColumn, and PDOStatement::bindParam.

PDO::SQLSRV_ENCODING_UTF8 (int)

Specifies that data is sent/retrieved to/from the server in UTF-8 encoding. This is the default encoding. This constant can be passed to PDOStatement::setAttribute, PDO::setAttribute, PDO::prepare, PDOStatement::bindColumn, and PDOStatement::bindParam.

PDO::SQLSRV_ENCODING_DEFAULT (int)

Specifies that data is sent/retrieved to/from the server according to PDO::SQLSRV_ENCODING_SYSTEM if specified during connection. The connection's encoding is used if specified in a prepare statement. This constant can be passed to PDOStatement::setAttribute, PDO::setAttribute, PDO::prepare, PDOStatement::bindColumn, and PDOStatement::bindParam.

PDO::SQLSRV_ATTR_QUERY_TIMEOUT (int)

A non-negative integer representing the timeout period, in seconds. Zero (0) is the default and means no timeout. This constant can be passed to PDOStatement::setAttribute, PDO::setAttribute, and PDO::prepare.

PDO::SQLSRV_ATTR_DIRECT_QUERY (int)

Indicates that a query should be executed directly, without being prepared. This constant can be passed to PDO::setAttribute, and PDO::prepare. For more information, see » Direct and Prepared Statement Execution.

Table of Contents

PDO_SQLSRV DSN

PDO_SQLSRV DSN

(PECL pdo_sqlsrv >= 2.0.1)

PDO_SQLSRV DSN — Connecting to MS SQL Server and SQL Azure databases

Description

The PDO_SQLSRV Data Source Name (DSN) is composed of the following elements:

DSN prefix

The DSN prefix is **sqlsrv:**.

APP

The application name used in tracing.

ConnectionPooling

Specifies whether the connection is assigned from a connection pool (1 or **true**) or not (0 or **false**).

Database

The name of the database.

Encrypt

Specifies whether the communication with SQL Server is encrypted (1 or **true**) or unencrypted (0 or **false**).

Failover_Partner

Specifies the server and instance of the database's mirror (if enabled and configured) to use when the primary server is unavailable.

LoginTimeout

Specifies the number of seconds to wait before failing the connection attempt.

MultipleActiveResultSets

Disables or explicitly enables support for multiple active Result sets (MARS).

QuotedId

Specifies whether to use SQL-92 rules for quoted identifiers (1 or `true`) or to use legacy Transact-SQL rules (0 or `false`).

`Server`

The name of the database server.

`TraceFile`

Specifies the path for the file used for trace data.

`TraceOn`

Specifies whether ODBC tracing is enabled (1 or `true`) or disabled (0 or `false`) for the connection being established.

`TransactionIsolation`

Specifies the transaction isolation level. The accepted values for this option are PDO::SQLSRV_TXN_READ_UNCOMMITTED, PDO::SQLSRV_TXN_READ_COMMITTED, PDO::SQLSRV_TXN_REPEATABLE_READ, PDO::SQLSRV_TXN_SNAPSHOT, and PDO::SQLSRV_TXN_SERIALIZABLE.

`TrustServerCertificate`

Specifies whether the client should trust (1 or `true`) or reject (0 or `false`) a self-signed server certificate.

`WSID`

Specifies the name of the computer for tracing.

Examples

Example #1 PDO_SQLSRV DSN examples

The following example shows how to connect to a specified MS SQL Server database:

```
$c = new PDO("sqlsrv:Server=localhost;Database=testdb", "UserName", "Password");
```

The following example shows how to connect to a MS SQL Server database on a specified port:

```
$c = new PDO("sqlsrv:Server=localhost,1521;Database=testdb", "UserName", "Password");
```

The following example shows how to connect to a SQL Azure database with server ID 12345abcde. Note that when you connect to SQL Azure with PDO, your username will be UserName@12345abcde (UserName@ServerId).

```
$c = new PDO("sqlsrv:Server=12345abcde.database.windows.net;Database=testdb", "UserName@12345abcde",
"Password");
```

Oracle PDO Driver

Installation

If the Oracle Database is on the same machine as PHP, the database software already contains the necessary libraries. When PHP is on a different machine, use the free » Oracle Instant Client libraries. For details refer to the

OCI8 Requirements section.

PHP 8.4

This extension has been moved to the » PECL repository and is no longer bundled with PHP as of PHP 8.4.0

Information for installing this PECL extension may be found in the manual chapter titled Installation of PECL extensions. Additional information such as new releases, downloads, source files, maintainer information, and a CHANGELOG, can be located here: » https://pecl.php.net/package/PDO_OCI.

PHP < 8.4

Use **--with-pdo-oci[=DIR]** to install the PDO Oracle OCI extension, where the optional `[=DIR]` is the Oracle Home directory. `[=DIR]` defaults to the *$ORACLE_HOME* environment variable.

Use **--with-pdo-oci=instantclient,prefix,version** for an Oracle Instant Client SDK, where prefix and version are configured.

```
// Using $ORACLE_HOME
$ ./configure --with-pdo-oci

// Using OIC for Linux with 10.2.0.3 RPMs with a /usr prefix
$ ./configure --with-pdo-oci=instantclient,/usr,10.2.0.3
```

Predefined Constants

The constants below are defined by this driver, and will only be available when the extension has been either compiled into PHP or dynamically loaded at runtime. In addition, these driver-specific constants should only be used if you are using this driver. Using driver-specific attributes with another driver may result in unexpected behaviour. PDO::getAttribute() may be used to obtain the **PDO::ATTR_DRIVER_NAME** attribute to check the driver, if your code can run against multiple drivers.

PDO::OCI_ATTR_ACTION (int)

Provides a way to specify the action on the database session.

This exists as of PHP 7.2.16 and 7.3.3

PDO::OCI_ATTR_CLIENT_INFO (int)

Provides a way to specify the client info on the database session.

This exists as of PHP 7.2.16 and 7.3.3

PDO::OCI_ATTR_CLIENT_IDENTIFIER (int)

Provides a way to specify the client identifier on the database session.

This exists as of PHP 7.2.16 and 7.3.3

PDO::OCI_ATTR_MODULE (int)

Provides a way to specify the module on the database session.

This exists as of PHP 7.2.16 and 7.3.3

Table of Contents

PDO_OCI DSN

PDO_OCI DSN

(PECL PDO_OCI >= 0.1.0)

PDO_OCI DSN — Connecting to Oracle databases

Description

The PDO_OCI Data Source Name (DSN) is composed of the following elements:

DSN prefix

The DSN prefix is **`oci:`**.

`dbname` (Oracle Instant Client)

The URI for the Oracle Instant Client connection takes the form of `dbname=//`*`hostname`*`:`*`port-number`*`/`*`database`*. If you are connecting to a database defined in *tnsnames.ora*, use only the name of the database: `dbname=`*`database`*.

`charset`

The client-side character set for the current environment handle.

Examples

Example #1 PDO_OCI DSN examples

The following examples show a PDO_OCI DSN for connecting to Oracle databases:

```
// Connect to a database defined in tnsnames.ora
oci:dbname=mydb

// Connect using the Oracle Instant Client
oci:dbname=//localhost:1521/mydb
```

ODBC and DB2 PDO Driver

Introduction

PDO_ODBC is a driver that implements the PHP Data Objects (PDO) interface to enable access from PHP to databases through ODBC drivers or through the IBM DB2 Call Level Interface (DB2 CLI) library. PDO_ODBC currently supports three different "flavours" of database drivers:

ibm-db2

Supports access to IBM DB2 Universal Database, Cloudscape, and Apache Derby servers through the free DB2 express-C client.

unixODBC

Supports access to database servers through the unixODBC driver manager and the database's own ODBC drivers.

generic

Offers a compile option for ODBC driver managers that are not explicitly supported by PDO_ODBC.

On Windows, *php_pdo_odbc.dll* has to be enabled as extension in *php.ini*. It is linked against the Windows ODBC Driver Manager so that PHP can connect to any database cataloged as a System DSN.

Installation

PDO_ODBC on UNIX systems

1. PDO_ODBC is included in the PHP source. You can compile the PDO_ODBC extension as either a static or shared module using the following **configure** commands.

   ibm_db2

   ```
   ./configure --with-pdo-odbc=ibm-db2,/opt/IBM/db2/V8.1/
   ```

   To build PDO_ODBC with the ibm-db2 flavour, you have to have previously installed the DB2 application development headers on the same machine on which you are compiling PDO_ODBC. The DB2 application development headers are an installable option in the DB2 servers, and are also available as part of the DB2 Application Development Client freely available for download from the IBM developerWorks » website.

   If you do not supply a location for the DB2 libraries and headers to the **configure** command, PDO_ODBC defaults to */home/db2inst1/sqllib*.

   unixODBC

   ```
   ./configure --with-pdo-odbc=unixODBC,/usr/local
   ```

   If you do not supply a location for the unixODBC libraries and headers to the **configure** command, PDO_ODBC defaults to */usr/local*.

   generic

   ```
   ./configure --with-pdo-odbc=generic,/usr/local,libname,ldflags,cflags
   ```

Predefined Constants

The constants below are defined by this driver, and will only be available when the extension has been either compiled into PHP or dynamically loaded at runtime. In addition, these driver-specific constants should only be used if you are using this driver. Using driver-specific attributes with another driver may result in unexpected behaviour. PDO::getAttribute() may be used to obtain the `PDO::ATTR_DRIVER_NAME` attribute to check the driver, if your code can run against multiple drivers.

`PDO_ODBC_TYPE` (string)

Describes the ODBC library linked against the PDO_ODBC extension. Possible values include `unixODBC`, `iODBC`, or `generic`.

> **Warning**
>
> The constants listed below have been *DEPRECATED* as of PHP 8.5.0. Use the corresponding Pdo\Odbc constants instead.

PDO::ODBC_ATTR_USE_CURSOR_LIBRARY (int)

Alias of **Pdo\Odbc::ATTR_USE_CURSOR_LIBRARY**.

PDO::ODBC_SQL_USE_IF_NEEDED (int)

Alias of **Pdo\Odbc::SQL_USE_IF_NEEDED**.

PDO::ODBC_SQL_USE_DRIVER (int)

Alias of **Pdo\Odbc::SQL_USE_DRIVER**.

PDO::ODBC_SQL_USE_ODBC (int)

Alias of **Pdo\Odbc::SQL_USE_ODBC**.

PDO::ODBC_ATTR_ASSUME_UTF8 (bool)

Alias of **Pdo\Odbc::ATTR_ASSUME_UTF8**.

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

PDO_ODBC Configuration Options

Name	Default	Changeable	Changelog
pdo_odbc.connection_pooling	"strict"	**INI_ALL**	
pdo_odbc.db2_instance_name	NULL	**INI_SYSTEM**	This deprecated feature *will* certainly be *removed* in the future.

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`pdo_odbc.connection_pooling` string

Whether to pool ODBC connections. Can be one of `"strict"`, `"relaxed"` or `"off"` (equals to `""`). The parameter describes how strict the connection manager should be when matching connection parameters to existing pooled connections. **`strict`** is the recommend default, and will result in the use of cached connections only when all the connection parameters match exactly. **`relaxed`** will result in the use of cached connections when similar connection parameters are used. This can result in increased use of the cache, at the risk of bleeding connection information between (for example) virtual hosts.

This setting can only be changed from the *php.ini* file, and affects the entire process; any other modules loaded into the process that use the same ODBC libraries will be affected too, including the Unified ODBC extension.

> **Warning**
>
> **`relaxed`** matching should not be used on a shared server, for security reasons.

> **Tip**
>
> Leave this setting at the default **`strict`** setting unless you have good reason to change it.

`pdo_odbc.db2_instance_name` string

If you compile PDO_ODBC using the `db2` flavour, this setting sets the value of the DB2INSTANCE environment variable on Linux and UNIX operating systems to the specified name of the DB2 instance. This enables PDO_ODBC to resolve the location of the DB2 libraries and make cataloged connections to DB2 databases.

This setting can only be changed from the *php.ini* file, and affects the entire process; any other modules loaded into the process that use the same ODBC libraries will be affected too, including the Unified ODBC extension.

This setting has no effect on Windows.

Table of Contents

PDO_ODBC DSN

PDO_ODBC DSN

(PECL PDO_ODBC >= 0.1.0)

PDO_ODBC DSN — Connecting to ODBC or DB2 databases

Description

The PDO_ODBC Data Source Name (DSN) is composed of the following elements:

DSN prefix

The DSN prefix is **`odbc:`**. If you are connecting to a database cataloged in the ODBC driver manager or the DB2 catalog, you can append the cataloged name of the database to the DSN.

DSN

The name of the database as cataloged in the ODBC driver manager or the DB2 catalog. Alternately, you can provide a complete ODBC connection string to connect to a database as described at » http://www.connectionstrings.com/.

`UID`

The name of the user for the connection. If you specify the user name in the DSN, PDO ignores the value of the user name argument in the PDO constructor.

`PWD`

The password of the user for the connection. If you specify the password in the DSN, PDO ignores the value of the password argument in the PDO constructor.

Changelog

Version	Description
8.4.0	When passing an empty string to the password argument in the PDO constructor, `pwd` was not included in the connection string created until now, but the behavior has been changed to include it as an empty string. Passing `null` for the password argument in the PDO constructor results in the same behavior as before.
8.4.0	Changed the behavior to ignore the user name argument and the password argument in the PDO constructor separately when the DSN contains `uid` or `pwd`. Previously, if included only either `uid` or `pwd` in the DSN, both the user name argument and the password argument in the PDO constructor were ignored.

Examples

Example #1 PDO_ODBC DSN example (ODBC driver manager)

The following example shows a PDO_ODBC DSN for connecting to an ODBC database cataloged as testdb in the ODBC driver manager:

```
odbc:testdb
```

Example #2 PDO_ODBC DSN example (IBM DB2 uncataloged connection)

The following example shows a PDO_ODBC DSN for connecting to an IBM DB2 database named **`SAMPLE`** using the full ODBC DSN syntax:

```
odbc:DRIVER={IBM DB2 ODBC
DRIVER};HOSTNAME=localhost;PORT=50000;DATABASE=SAMPLE;PROTOCOL=TCPIP;UID=db2inst1;PWD=ibmdb2;
```

Example #3 PDO_ODBC DSN example (Microsoft Access uncataloged connection)

The following example shows a PDO_ODBC DSN for connecting to a Microsoft Access database stored at **`C:\db.mdb`** using the full ODBC DSN syntax:

```
odbc:Driver={Microsoft Access Driver (*.mdb)};Dbq=C:\\db.mdb;Uid=Admin
```

Pdo\Odbc

(PHP 8 >= 8.4.0)

Introduction

A PDO subclass representing a connection using the ODBC PDO driver.

Class synopsis

namespace **Pdo**;
class **Odbc** extends PDO {
/* Inherited constants */
public const int *PDO::PARAM_NULL*;
public const int *PDO::PARAM_BOOL* = 5;
public const int *PDO::PARAM_INT* = 1;
public const int *PDO::PARAM_STR* = 2;

public const int *PDO::PARAM_LOB* = 3;
public const int *PDO::PARAM_STMT* = 4;
public const int *PDO::PARAM_INPUT_OUTPUT*;
public const int *PDO::PARAM_STR_NATL*;
public const int *PDO::PARAM_STR_CHAR*;
public const int *PDO::PARAM_EVT_ALLOC*;
public const int *PDO::PARAM_EVT_FREE*;
public const int *PDO::PARAM_EVT_EXEC_PRE*;
public const int *PDO::PARAM_EVT_EXEC_POST*;
public const int *PDO::PARAM_EVT_FETCH_PRE*;
public const int *PDO::PARAM_EVT_FETCH_POST*;
public const int *PDO::PARAM_EVT_NORMALIZE*;
public const int *PDO::FETCH_DEFAULT*;
public const int *PDO::FETCH_LAZY*;
public const int *PDO::FETCH_ASSOC*;
public const int *PDO::FETCH_NUM*;
public const int *PDO::FETCH_BOTH*;
public const int *PDO::FETCH_OBJ*;
public const int *PDO::FETCH_BOUND*;
public const int *PDO::FETCH_COLUMN*;
public const int *PDO::FETCH_CLASS*;
public const int *PDO::FETCH_INTO*;
public const int *PDO::FETCH_FUNC*;
public const int *PDO::FETCH_GROUP*;
public const int *PDO::FETCH_UNIQUE*;
public const int *PDO::FETCH_KEY_PAIR*;
public const int *PDO::FETCH_CLASSTYPE*;
public const int *PDO::FETCH_SERIALIZE*;
public const int *PDO::FETCH_PROPS_LATE*;
public const int *PDO::FETCH_NAMED*;
public const int *PDO::ATTR_AUTOCOMMIT*;
public const int *PDO::ATTR_PREFETCH*;
public const int *PDO::ATTR_TIMEOUT*;
public const int *PDO::ATTR_ERRMODE*;
public const int *PDO::ATTR_SERVER_VERSION*;
public const int *PDO::ATTR_CLIENT_VERSION*;
public const int *PDO::ATTR_SERVER_INFO*;
public const int *PDO::ATTR_CONNECTION_STATUS*;
public const int *PDO::ATTR_CASE*;
public const int *PDO::ATTR_CURSOR_NAME*;
public const int *PDO::ATTR_CURSOR*;
public const int *PDO::ATTR_ORACLE_NULLS*;
public const int *PDO::ATTR_PERSISTENT*;
public const int *PDO::ATTR_STATEMENT_CLASS*;
public const int *PDO::ATTR_FETCH_TABLE_NAMES*;
public const int *PDO::ATTR_FETCH_CATALOG_NAMES*;
public const int *PDO::ATTR_DRIVER_NAME*;
public const int *PDO::ATTR_STRINGIFY_FETCHES*;
public const int *PDO::ATTR_MAX_COLUMN_LEN*;
public const int *PDO::ATTR_EMULATE_PREPARES*;
public const int *PDO::ATTR_DEFAULT_FETCH_MODE*;

```
public const int PDO::ATTR_DEFAULT_STR_PARAM;
public const int PDO::ERRMODE_SILENT;
public const int PDO::ERRMODE_WARNING;
public const int PDO::ERRMODE_EXCEPTION;
public const int PDO::CASE_NATURAL;
public const int PDO::CASE_LOWER;
public const int PDO::CASE_UPPER;
public const int PDO::NULL_NATURAL;
public const int PDO::NULL_EMPTY_STRING;
public const int PDO::NULL_TO_STRING;
public const string PDO::ERR_NONE;
public const int PDO::FETCH_ORI_NEXT;
public const int PDO::FETCH_ORI_PRIOR;
public const int PDO::FETCH_ORI_FIRST;
public const int PDO::FETCH_ORI_LAST;
public const int PDO::FETCH_ORI_ABS;
public const int PDO::FETCH_ORI_REL;
public const int PDO::CURSOR_FWDONLY;
public const int PDO::CURSOR_SCROLL;
/* Constants */
public const int ATTR_USE_CURSOR_LIBRARY;
public const int ATTR_ASSUME_UTF8;
public const int SQL_USE_IF_NEEDED;
public const int SQL_USE_DRIVER;
public const int SQL_USE_ODBC;
/* Inherited methods */
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function PDO::beginTransaction(): bool
public function PDO::commit(): bool
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function PDO::errorCode(): ?string
public function PDO::errorInfo(): array
public function PDO::exec(string $statement): int|false
public function PDO::getAttribute(int $attribute): mixed
public static function PDO::getAvailableDrivers(): array
public function PDO::inTransaction(): bool
public function PDO::lastInsertId(?string $name = null): string|false
public function PDO::prepare(string $query, array $options = []): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = null): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno):
PDOStatement|false
```

```
public function PDO::query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object):
PDOStatement|false
public function PDO::quote(string $string, int $type = PDO::PARAM_STR): string|false
public function PDO::rollBack(): bool
public function PDO::setAttribute(int $attribute, mixed $value): bool
}
```

Predefined Constants

`Pdo\Odbc::ATTR_USE_CURSOR_LIBRARY`

This option controls whether the ODBC cursor library is used. The ODBC cursor library supports some advanced ODBC features (e.g. block scrollable cursors), which may not be implemented by the driver. The following values are supported:

`Pdo\Odbc::SQL_USE_IF_NEEDED`

Use the ODBC cursor library when needed. This is the default.

`Pdo\Odbc::SQL_USE_DRIVER`

Never use the ODBC cursor library.

`Pdo\Odbc::SQL_USE_ODBC`

Always use the ODBC cursor library.

`Pdo\Odbc::ATTR_ASSUME_UTF8`

Windows only. If **`true`**, UTF-16 encoded character data (`CHAR`, `VARCHAR` and `LONGVARCHAR`) is converted to UTF-8 when reading from or writing data to the database. If **`false`** (the default), character encoding conversion may be done by the driver.

PostgreSQL PDO Driver

Introduction

PDO_PGSQL is a driver that implements the PHP Data Objects (PDO) interface to enable access from PHP to PostgreSQL databases.

Resource Types

This extension defines a stream resource returned by Pdo\Pgsql::lobOpen().

Installation

Use **--with-pdo-pgsql[=DIR]** to install the PDO PostgreSQL extension, where the optional `[=DIR]` is the PostgreSQL base install directory, or the path to *pg_config*.

```
$ ./configure --with-pdo-pgsql
```

This extension requires libpq (the PostgreSQL C client library). As of PHP 8.4.0, libpq 10.0 or later is required.

Predefined Constants

The constants below are defined by this driver, and will only be available when the extension has been either compiled into PHP or dynamically loaded at runtime. In addition, these driver-specific constants should only be used if you are using this driver. Using driver-specific attributes with another driver may result in unexpected behaviour. PDO::getAttribute() may be used to obtain the **PDO::ATTR_DRIVER_NAME** attribute to check the driver, if your code can run against multiple drivers.

PDO::PGSQL_ATTR_DISABLE_PREPARES (int)

Alias of **Pdo\Pgsql::ATTR_DISABLE_PREPARES**. As of PHP 8.5.0, this constant is deprecated.

PDO::PGSQL_TRANSACTION_IDLE (int)

Equivalent to **Pdo\Pgsql::TRANSACTION_IDLE**. As of PHP 8.5.0, this constant is deprecated, as it has no effect.

PDO::PGSQL_TRANSACTION_ACTIVE (int)

Equivalent to **Pdo\Pgsql::TRANSACTION_ACTIVE**. As of PHP 8.5.0, this constant is deprecated, as it has no effect.

PDO::PGSQL_TRANSACTION_INTRANS (int)

Equivalent to **Pdo\Pgsql::TRANSACTION_INTRANS**. As of PHP 8.5.0, this constant is deprecated, as it has no effect.

PDO::PGSQL_TRANSACTION_INERROR (int)

Equivalent to **Pdo\Pgsql::TRANSACTION_INERROR**. As of PHP 8.5.0, this constant is deprecated, as it has no effect.

PDO::PGSQL_TRANSACTION_UNKNOWN (int)

Equivalent to **Pdo\Pgsql::TRANSACTION_UNKNOWN**. As of PHP 8.5.0, this constant is deprecated, as it has no effect.

General notes

> **Note** :
>
> `bytea` columns are returned as stream resources. See Large Objects (LOBs) for how to read these values and how to bind data with **PDO::PARAM_LOB**.

Table of Contents

PDO_PGSQL DSN

PDO_PGSQL DSN

(PHP 5 >= 5.1.0, PHP 7, PECL PDO_PGSQL >= 0.1.0)

PDO_PGSQL DSN — Connecting to PostgreSQL databases

Description

The PDO_PGSQL Data Source Name (DSN) is composed of the following elements, delimited by spaces or semicolons:

DSN prefix

The DSN prefix is **`pgsql:`**.

`host`

The hostname on which the database server resides.

`port`

The port on which the database server is running.

`dbname`

The name of the database.

`user`

The name of the user for the connection. If you specify the user name in the DSN, PDO ignores the value of the user name argument in the PDO constructor.

`password`

The password of the user for the connection. If you specify the password in the DSN, PDO ignores the value of the password argument in the PDO constructor.

`sslmode`

The SSL mode. Accepted values are `disable`, `allow`, `prefer`, `require`, `verify-ca`, and `verify-full`; their meaning is described in the » PostgreSQL Documentation. Many hosted PostgreSQL services require an encrypted connection, so `require` or stricter is needed to connect to them.

> **Note** :
>
> As of PHP 8.4.0, credentials specified in the DSN (`user` and `password`) take priority over the corresponding arguments passed to the PDO constructor. In earlier versions, the PDO constructor arguments took precedence.

> **Note** :
>
> All semicolons in the DSN string are replaced by spaces, because PostgreSQL expects this format. This implies that semicolons in any of the components (e.g. `password` or `dbname`) are not supported.

Examples

Example #1 PDO_PGSQL DSN examples

The following example shows a PDO_PGSQL DSN for connecting to a PostgreSQL database:

```
pgsql:host=localhost;port=5432;dbname=testdb;user=bruce;password=mypass
```

The following example shows a PDO_PGSQL DSN for connecting to a PostgreSQL database via unix socket */tmp/.s.PGSQL.5432*:

```
pgsql:host=/tmp;port=5432;dbname=testdb;user=bruce;password=mypass
```

PDO::pgsqlCopyFromArray

PDO::pgsqlCopyFromArray

(PHP 5 >= 5.3.3, PHP 7, PHP 8)

PDO::pgsqlCopyFromArray — Alias of Pdo\Pgsql::copyFromArray()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated]
public function PDO::pgsqlCopyFromArray(
    string $tableName,
    array|Traversable $rows,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
```

This method is an alias of: Pdo\Pgsql::copyFromArray().

Changelog

Version	Description
8.5.0	The `rows` parameter now also accepts a Traversable; previously only an array was accepted.

PDO::pgsqlCopyFromFile

PDO::pgsqlCopyFromFile

(PHP 5 >= 5.3.3, PHP 7, PHP 8)

PDO::pgsqlCopyFromFile — Alias of Pdo\Pgsql::copyFromFile()

> **Warning**

> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated]
public function PDO::pgsqlCopyFromFile(
    string $tableName,
    string $filename,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
```

This method is an alias of: Pdo\Pgsql::copyFromFile().

PDO::pgsqlCopyToArray

PDO::pgsqlCopyToArray

(PHP 5 >= 5.3.3, PHP 7, PHP 8)

PDO::pgsqlCopyToArray — Alias of Pdo\Pgsql::copyToArray()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated]
public function PDO::pgsqlCopyToArray(
    string $tableName,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): array|false
```

This method is an alias of: Pdo\Pgsql::copyToArray().

PDO::pgsqlCopyToFile

PDO::pgsqlCopyToFile

(PHP 5 >= 5.3.3, PHP 7, PHP 8)

PDO::pgsqlCopyToFile — Alias of Pdo\Pgsql::copyToFile()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated]
public function PDO::pgsqlCopyToFile(
    string $tableName,
    string $filename,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
```

This method is an alias of: Pdo\Pgsql::copyToFile().

PDO::pgsqlGetNotify

PDO::pgsqlGetNotify

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

PDO::pgsqlGetNotify — Alias of Pdo\Pgsql::getNotify()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated]
public function PDO::pgsqlGetNotify(int $fetchMode = PDO::FETCH_DEFAULT, int $timeoutMilliseconds = 0): array|false
```

This method is an alias of: Pdo\Pgsql::getNotify().

PDO::pgsqlGetPid

PDO::pgsqlGetPid

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

PDO::pgsqlGetPid — Alias of Pdo\Pgsql::getPid()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated]
public function PDO::pgsqlGetPid(): int
```

This method is an alias of: Pdo\Pgsql::getPid().

PDO::pgsqlLOBCreate

PDO::pgsqlLOBCreate

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL pdo_pgsql >= 1.0.2)

PDO::pgsqlLOBCreate — Alias of Pdo\Pgsql::lobCreate()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

#[\Deprecated]
public function **PDO::pgsqlLOBCreate**(): string

This method is an alias of: Pdo\Pgsql::lobCreate().

PDO::pgsqlLOBOpen

PDO::pgsqlLOBOpen

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL pdo_pgsql >= 1.0.2)

PDO::pgsqlLOBOpen — Alias of Pdo\Pgsql::lobOpen()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

#[\Deprecated]
public function **PDO::pgsqlLOBOpen**(string `$oid`, string `$mode` = "rb"): resource|false

This method is an alias of: Pdo\Pgsql::lobOpen().

PDO::pgsqlLOBUnlink

PDO::pgsqlLOBUnlink

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL pdo_pgsql >= 1.0.2)

PDO::pgsqlLOBUnlink — Alias of Pdo\Pgsql::lobUnlink()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

#[\Deprecated]
public function **PDO::pgsqlLOBUnlink**(string `$oid`): bool

This method is an alias of: Pdo\Pgsql::lobUnlink().

Pdo\Pgsql

(PHP 8 >= 8.4.0)

Introduction

A PDO subclass representing a connection using the PostgreSQL PDO driver.

This driver supports a dedicated SQL query parser for the PostgreSQL dialect. It can handle the following:

- Single and double-quoted literals, with doubling as escaping mechanism
- C-style “escape” string literals
- Dollar-quoted string literals
- Two-dashes and C-style comments (non-nested).
- Support for `??` as escape sequence for the `?` operator.

> **Note** :
>
> Because this parser recognizes dollar-quoted string literals, question marks inside them are not treated as parameter placeholders and do not need to be escaped. As of PHP 8.4.0, escaping question marks as `??` inside a dollar-quoted string is deprecated.

Class synopsis

```
namespace Pdo;
class Pgsql extends PDO {
/* Inherited constants */
public const int PDO::PARAM_NULL;
public const int PDO::PARAM_BOOL = 5;
public const int PDO::PARAM_INT = 1;
public const int PDO::PARAM_STR = 2;
public const int PDO::PARAM_LOB = 3;
public const int PDO::PARAM_STMT = 4;
public const int PDO::PARAM_INPUT_OUTPUT;
public const int PDO::PARAM_STR_NATL;
public const int PDO::PARAM_STR_CHAR;
public const int PDO::PARAM_EVT_ALLOC;
public const int PDO::PARAM_EVT_FREE;
public const int PDO::PARAM_EVT_EXEC_PRE;
public const int PDO::PARAM_EVT_EXEC_POST;
public const int PDO::PARAM_EVT_FETCH_PRE;
public const int PDO::PARAM_EVT_FETCH_POST;
public const int PDO::PARAM_EVT_NORMALIZE;
public const int PDO::FETCH_DEFAULT;
public const int PDO::FETCH_LAZY;
public const int PDO::FETCH_ASSOC;
public const int PDO::FETCH_NUM;
public const int PDO::FETCH_BOTH;
public const int PDO::FETCH_OBJ;
public const int PDO::FETCH_BOUND;
public const int PDO::FETCH_COLUMN;
public const int PDO::FETCH_CLASS;
```

```
public const int PDO::FETCH_INTO;
public const int PDO::FETCH_FUNC;
public const int PDO::FETCH_GROUP;
public const int PDO::FETCH_UNIQUE;
public const int PDO::FETCH_KEY_PAIR;
public const int PDO::FETCH_CLASSTYPE;
public const int PDO::FETCH_SERIALIZE;
public const int PDO::FETCH_PROPS_LATE;
public const int PDO::FETCH_NAMED;
public const int PDO::ATTR_AUTOCOMMIT;
public const int PDO::ATTR_PREFETCH;
public const int PDO::ATTR_TIMEOUT;
public const int PDO::ATTR_ERRMODE;
public const int PDO::ATTR_SERVER_VERSION;
public const int PDO::ATTR_CLIENT_VERSION;
public const int PDO::ATTR_SERVER_INFO;
public const int PDO::ATTR_CONNECTION_STATUS;
public const int PDO::ATTR_CASE;
public const int PDO::ATTR_CURSOR_NAME;
public const int PDO::ATTR_CURSOR;
public const int PDO::ATTR_ORACLE_NULLS;
public const int PDO::ATTR_PERSISTENT;
public const int PDO::ATTR_STATEMENT_CLASS;
public const int PDO::ATTR_FETCH_TABLE_NAMES;
public const int PDO::ATTR_FETCH_CATALOG_NAMES;
public const int PDO::ATTR_DRIVER_NAME;
public const int PDO::ATTR_STRINGIFY_FETCHES;
public const int PDO::ATTR_MAX_COLUMN_LEN;
public const int PDO::ATTR_EMULATE_PREPARES;
public const int PDO::ATTR_DEFAULT_FETCH_MODE;
public const int PDO::ATTR_DEFAULT_STR_PARAM;
public const int PDO::ERRMODE_SILENT;
public const int PDO::ERRMODE_WARNING;
public const int PDO::ERRMODE_EXCEPTION;
public const int PDO::CASE_NATURAL;
public const int PDO::CASE_LOWER;
public const int PDO::CASE_UPPER;
public const int PDO::NULL_NATURAL;
public const int PDO::NULL_EMPTY_STRING;
public const int PDO::NULL_TO_STRING;
public const string PDO::ERR_NONE;
public const int PDO::FETCH_ORI_NEXT;
public const int PDO::FETCH_ORI_PRIOR;
public const int PDO::FETCH_ORI_FIRST;
public const int PDO::FETCH_ORI_LAST;
public const int PDO::FETCH_ORI_ABS;
public const int PDO::FETCH_ORI_REL;
public const int PDO::CURSOR_FWDONLY;
public const int PDO::CURSOR_SCROLL;
/* Constants */
public const int ATTR_DISABLE_PREPARES;
```

```
public const int ATTR_RESULT_MEMORY_SIZE;
public const int TRANSACTION_IDLE;
public const int TRANSACTION_ACTIVE;
public const int TRANSACTION_INTRANS;
public const int TRANSACTION_INERROR;
public const int TRANSACTION_UNKNOWN;
/* Methods */
public function copyFromArray(
    string $tableName,
    array|Traversable $rows,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
public function copyFromFile(
    string $tableName,
    string $filename,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
public function copyToArray(
    string $tableName,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): array|false
public function copyToFile(
    string $tableName,
    string $filename,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
public function escapeIdentifier(string $input): string
public function getNotify(int $fetchMode = PDO::FETCH_DEFAULT, int $timeoutMilliseconds = 0): array|false
public function getPid(): int
public function lobCreate(): string|false
public function lobOpen(string $oid, string $mode = "rb"): resource|false
public function lobUnlink(string $oid): bool
public function setNoticeCallback(?callable $callback): void
/* Inherited methods */
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function PDO::beginTransaction(): bool
public function PDO::commit(): bool
```

```
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function PDO::errorCode(): ?string
public function PDO::errorInfo(): array
public function PDO::exec(string $statement): int|false
public function PDO::getAttribute(int $attribute): mixed
public static function PDO::getAvailableDrivers(): array
public function PDO::inTransaction(): bool
public function PDO::lastInsertId(?string $name = null): string|false
public function PDO::prepare(string $query, array $options = []): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = null): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno):
PDOStatement|false
public function PDO::query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object):
PDOStatement|false
public function PDO::quote(string $string, int $type = PDO::PARAM_STR): string|false
public function PDO::rollBack(): bool
public function PDO::setAttribute(int $attribute, mixed $value): bool
}
```

Predefined Constants

`Pdo\Pgsql::ATTR_DISABLE_PREPARES`

Send the query and the parameters to the server together in a single call, avoiding the need to create a named prepared statement separately. If the query is only going to be executed once this can reduce latency by avoiding an unnecessary server round-trip.

`Pdo\Pgsql::ATTR_RESULT_MEMORY_SIZE`

Returns the amount of memory, in bytes, allocated to the specified query result PDOStatement instance, or `null` if no results exist before the query is executed.

`PDO::ATTR_PREFETCH`

As of PHP 8.5.0, setting this attribute to `0` enables lazy (single-row) fetching: rows are retrieved from the server one at a time as they are fetched, instead of buffering the whole result set in memory before the first PDOStatement::fetch() call. This reduces memory usage for large result sets. Any other value keeps the default buffered behavior.

It can be set per connection with PDO::setAttribute(), or per statement via the PDO::prepare() or PDO::query() driver options.

> **Caution**
>
> In lazy mode, a connection can have only one active statement at a time. Running another statement silently discards any unread rows of the previous one; no error is raised.

`Pdo\Pgsql::TRANSACTION_IDLE`

> **Warning**
>
> This constant has no effect, and is deprecated as of PHP 8.5.0.

`Pdo\Pgsql::TRANSACTION_ACTIVE`

> **Warning**
>
> This constant has no effect, and is deprecated as of PHP 8.5.0.

`Pdo\Pgsql::TRANSACTION_INTRANS`

> **Warning**
>
> This constant has no effect, and is deprecated as of PHP 8.5.0.

`Pdo\Pgsql::TRANSACTION_INERROR`

> **Warning**
>
> This constant has no effect, and is deprecated as of PHP 8.5.0.

`Pdo\Pgsql::TRANSACTION_UNKNOWN`

> **Warning**
>
> This constant has no effect, and is deprecated as of PHP 8.5.0.

Table of Contents

Pdo\Pgsql::copyFromArray

Pdo\Pgsql::copyFromArray

(PHP 8 >= 8.4.0)

Pdo\Pgsql::copyFromArray — Copy data from a PHP array into a table

Description

```
public function Pdo\Pgsql::copyFromArray(
    string $tableName,
    array|Traversable $rows,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
```

Copies data from `rows` array to table `tableName` using `separator` as fields delimiter and `fields` list.

Parameters

`tableName`

String containing table name.

`rows`

An indexed array (or Traversable) of strings with fields separated by `separator`.

`separator`

Delimiter used to separate fields in an entry of the `rows` array.

`nullAs`

How to interpret SQL `NULL` values.

`fields`

List of fields to insert.

Return Values

Returns **`true`** on success or **`false`** on failure.

Changelog

Version	Description
8.5.0	`rows` now also accepts a Traversable; previously only an array was accepted.

Examples

Example #1 **Pdo\Pgsql::copyFromArray()** example

Each element of `rows` is one record whose fields are joined by `separator` (a tab by default).

```
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->exec('CREATE TABLE fruits (id int, name text, qty int)');

$rows = [
    "1\tapple\t10",
    "2\tbanana\t20",
    "3\tcherry\t30",
];
$db->copyFromArray('fruits', $rows);

foreach ($db->query('SELECT * FROM fruits ORDER BY id') as $row) {
    echo "{$row['id']} {$row['name']} {$row['qty']}\n";
}
?>
```

The above example will output:

```
1 apple 10
2 banana 20
3 cherry 30
```

See Also

- Pdo\Pgsql::copyToArray() - Copy data from database table into PHP array
- Pdo\Pgsql::copyFromFile() - Copy data from file into table
- Pdo\Pgsql::copyToFile() - Copy data from table into file

Pdo\Pgsql::copyFromFile

Pdo\Pgsql::copyFromFile

(PHP 8 >= 8.4.0)

Pdo\Pgsql::copyFromFile — Copy data from file into table

Description

public function **Pdo\Pgsql::copyFromFile**(
    string `$tableName`,
    string `$filename`,
    string `$separator` = "\t",
    string `$nullAs` = "\\\\N",
    ?string `$fields` = **`null`**
): bool

Copies data from file specified by `filename` into table `tableName` using `separator` as fields delimiter and `fields` list

Parameters

`tableName`

String containing table name.

`filename`

Filename containing the data to import.

`separator`

Delimiter used to separate fields in an entry of the `rows` array.

`nullAs`

How to interpret SQL `NULL` values.

`fields`

List of fields to insert.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If `filename` cannot be opened for reading, the failure is reported through the connection's error handling (see `PDO::ATTR_ERRMODE`); with `PDO::ERRMODE_EXCEPTION` a PDOException is thrown.

Examples

Example #1 **Pdo\Pgsql::copyFromFile()** example

The file holds one record per line, with fields joined by `separator` (a tab by default).

```
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->exec('CREATE TABLE fruits (id int, name text, qty int)');

file_put_contents('/tmp/fruits.tsv', "1\tapple\t10\n2\tbanana\t20\n");
$db->copyFromFile('fruits', '/tmp/fruits.tsv');

echo $db->query('SELECT count(*) FROM fruits')->fetchColumn(), "\n";
?>
```

The above example will output:

```
2
```

See Also

- Pdo\Pgsql::copyToFile() - Copy data from table into file
- Pdo\Pgsql::copyFromArray() - Copy data from a PHP array into a table
- Pdo\Pgsql::copyToArray() - Copy data from database table into PHP array

Pdo\Pgsql::copyToArray

Pdo\Pgsql::copyToArray

(PHP 8 >= 8.4.0)

Pdo\Pgsql::copyToArray — Copy data from database table into PHP array

Description

public function **Pdo\Pgsql::copyToArray**(
  string `$tableName`,
  string `$separator` = "\t",
  string `$nullAs` = "\\\\N",

```
    ?string $fields = null
): array|false
```

Copies data from `tableName` into array using `separator` as fields delimiter and `fields` list

Parameters

`tableName`

String containing table name.

`separator`

Delimiter used to separate fields in an entry of the `rows` array.

`nullAs`

How to interpret SQL `NULL` values.

`fields`

List of fields to export.

Return Values

Returns an array of rows, or `false` on failure.

Examples

Example #1 Pdo\Pgsql::copyToArray() example

Each returned element is one record, with fields joined by `separator` and a trailing newline.

```php
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->exec('CREATE TABLE fruits (id int, name text, qty int)');
$db->exec("INSERT INTO fruits VALUES (1, 'apple', 10), (2, 'banana', 20)");

$rows = $db->copyToArray('fruits');
var_export($rows);
?>
```

The above example will output:

```
array (
  0 => '1	apple	10
',
  1 => '2	banana	20
',
)
```

See Also

- Pdo\Pgsql::copyFromArray() - Copy data from a PHP array into a table
- Pdo\Pgsql::copyFromFile() - Copy data from file into table
- Pdo\Pgsql::copyToFile() - Copy data from table into file

Pdo\Pgsql::copyToFile

Pdo\Pgsql::copyToFile

(PHP 8 >= 8.4.0)

Pdo\Pgsql::copyToFile — Copy data from table into file

Description

```
public function Pdo\Pgsql::copyToFile(
    string $tableName,
    string $filename,
    string $separator = "\t",
    string $nullAs = "\\\\N",
    ?string $fields = null
): bool
```

Copies data from table into file specified by `filename` using `separator` as fields delimiter and `fields` list.

Parameters

`tableName`

String containing table name.

`filename`

Filename to export data.

`separator`

Delimiter used to separate fields in an entry of the `rows` array.

`nullAs`

How to interpret SQL `NULL` values.

`fields`

List of fields to export.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If `filename` cannot be opened for writing, or cannot be written to, the failure is reported through the connection's error handling (see `PDO::ATTR_ERRMODE`); with `PDO::ERRMODE_EXCEPTION` a PDOException is thrown.

Examples

Example #1 Pdo\Pgsql::copyToFile() example

The table is written to `filename`, one record per line, with fields joined by `separator`.

```
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->exec('CREATE TABLE fruits (id int, name text, qty int)');
$db->exec("INSERT INTO fruits VALUES (1, 'apple', 10), (2, 'banana', 20)");

$db->copyToFile('fruits', '/tmp/fruits.tsv');
echo file_get_contents('/tmp/fruits.tsv');
?>
```

The above example will output:

```
1       apple   10
2       banana  20
```

See Also

- Pdo\Pgsql::copyFromFile() - Copy data from file into table
- Pdo\Pgsql::copyFromArray() - Copy data from a PHP array into a table
- Pdo\Pgsql::copyToArray() - Copy data from database table into PHP array

Pdo\Pgsql::escapeIdentifier

Pdo\Pgsql::escapeIdentifier

(PHP 8 >= 8.4.0)

Pdo\Pgsql::escapeIdentifier — Escapes a string for use as an SQL identifier

Description

public function **Pdo\Pgsql::escapeIdentifier**(string `$input`): string

Escapes a string for use as an SQL identifier, such as a table, column, or function name. This is useful when a user-supplied identifier might contain special characters that would otherwise not be interpreted as part of the identifier by the SQL parser, or when the identifier might contain upper case characters whose case should be preserved.

Parameters

`input`

A string containing text to be escaped.

Return Values

A string containing the escaped data.

Examples

Example #1 Pdo\Pgsql::escapeIdentifier() example

```php
<?php
$pdo = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);

$unescapedTableName = 'UnescapedTableName';
$pdo->exec("CREATE TABLE $unescapedTableName ()");

$escapedTableName = $pdo->escapeIdentifier('EscapedTableName');
$pdo->exec("CREATE TABLE $escapedTableName ()");

$statement = $pdo->query(
  "SELECT relname FROM pg_stat_user_tables WHERE relname ilike '%tablename'"
);

var_export($statement->fetchAll(PDO::FETCH_COLUMN, 0));

$tableNameWithSymbols = 'Table-Name-With-Symbols';
$pdo->exec("CREATE TABLE $tableNameWithSymbols ()");
?>
```

The above example will output something similar to:

```
array (
  0 => 'unescapedtablename',
  1 => 'EscapedTableName',
)
Fatal error: Uncaught PDOException: SQLSTATE[42601]: Syntax error: 7 ERROR:  syntax error at or near
"Table"
LINE 1: CREATE TABLE Table-Name-With-Symbols ()
```

See Also

- PDO::quote() - Quotes a string for use in a query

Pdo\Pgsql::getNotify

Pdo\Pgsql::getNotify

(PHP 8 >= 8.4.0)

Pdo\Pgsql::getNotify — Get asynchronous notification

Description

public function **Pdo\Pgsql::getNotify**(int `$fetchMode` = **`PDO::FETCH_DEFAULT`**, int `$timeoutMilliseconds` = 0): array|false

Returns a result set representing a pending asynchronous notification.

Parameters

`fetchMode`

The format the result set should be returned as, one of the following constants:

- `PDO::FETCH_DEFAULT`
- `PDO::FETCH_BOTH`
- `PDO::FETCH_ASSOC`
- `PDO::FETCH_NUM`

`timeoutMilliseconds`

The length of time to wait for a response, in milliseconds.

Return Values

If a notification is pending, returns a single row, otherwise returns `false`. The row has a `message` field (the channel name) and a `pid` field (the process ID of the notifying backend). If the notification carries a non-empty payload, the row also has a `payload` field. With `PDO::FETCH_NUM`, these fields are at indexes `0`, `1`, and `2`.

Errors/Exceptions

A ValueError is thrown if `fetchMode` is not one of the valid `PDO::FETCH_*` constants.

A ValueError is thrown if `timeoutMilliseconds` is less than `0`.

A `E_WARNING` is raised when `timeoutMilliseconds` is greater than the value that can be contained in a signed 32-bit integer, in which case it will be the maximum value of a signed 32-bit integer.

Examples

Example #1 **Pdo\Pgsql::getNotify()** example

Subscribe to a channel with `LISTEN`, then read the next pending notification with a one-second timeout.

```php
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$db->exec('LISTEN messages');
$db->exec("NOTIFY messages, 'hello'");

$notification = $db->getNotify(PDO::FETCH_ASSOC, 1000);
var_export($notification);
?>
```

The above example will output something similar to:

```
array (
  'message' => 'messages',
  'pid' => 1928,
  'payload' => 'hello',
)
```

See Also

- PDO::query() - Prepares and executes an SQL statement without placeholders
- PDOStatement::fetch() - Fetches the next row from a result set
- PDOStatement::fetchAll() - Fetches the remaining rows from a result set

Pdo\Pgsql::getPid

Pdo\Pgsql::getPid

(PHP 8 >= 8.4.0)

Pdo\Pgsql::getPid — Get the PID of the backend process handling this connection

Description

public function **Pdo\Pgsql::getPid**(): int

Returns the PID of the backend process handling this connection. Note that the PID belongs to a process executing on the database server host, not the local host.

Parameters

This function has no parameters.

Return Values

Returns the PID as an int.

Examples

Example #1 **Pdo\Pgsql::getPid()** example

```php
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
echo $db->getPid();
?>
```

The above example will output something similar to:

```
12345
```

See Also

- pg_get_pid() - Gets the backend's process ID

Pdo\Pgsql::lobCreate

Pdo\Pgsql::lobCreate

(PHP 8 >= 8.4.0)

Pdo\Pgsql::lobCreate — Creates a new large object

Description

public function **Pdo\Pgsql::lobCreate**(): string|false

Pdo\Pgsql::lobCreate() creates a large object and returns the OID which refers to it. It can be opened to read or write data with Pdo\Pgsql::lobOpen().

The OID can be stored in columns of type OID and be used to reference the large object, without causing the row to grow arbitrarily large. The large object will continue to live in the database until it is removed by calling Pdo\Pgsql::lobUnlink().

Large objects are cumbersome to use. Indeed, it is required that Pdo\Pgsql::lobUnlink() is called prior to deleting the last row referencing the OID in the entire database; otherwise, unreferenced large objects will remain on the server indefinitely. Moreover, large objects have no access controls. An alternative is the bytea column type, which can be up to 1GB in size, and this column type transparently manages the storage for optimal row size.

> **Note** :
>
> This function, and all manipulations of the large object, must be called and carried out within a transaction.

Parameters

This function has no parameters.

Return Values

Returns the OID of the newly created large object on success, or `false` on failure.

Examples

Example #1 **Pdo\Pgsql::lobCreate()** example

This example creates a new large object and copies the contents of a file into it. The OID is then stored into a table.

```
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->beginTransaction();
$oid = $db->lobCreate();
$stream = $db->lobOpen($oid, 'w');
$local = fopen($filename, 'rb');
stream_copy_to_stream($local, $stream);
$local = null;
$stream = null;
$stmt = $db->prepare("INSERT INTO BLOBS (ident, oid) VALUES (?, ?)");
$stmt->execute([$some_id, $oid]);
$db->commit();
?>
```

See Also

- Pdo\Pgsql::lobOpen() - Opens an existing large object stream
- Pdo\Pgsql::lobUnlink() - Deletes the large object
- pg_lo_create() - Create a large object
- pg_lo_open() - Open a large object

Pdo\Pgsql::lobOpen

Pdo\Pgsql::lobOpen

(PHP 8 >= 8.4.0)

Pdo\Pgsql::lobOpen — Opens an existing large object stream

Description

public function **Pdo\Pgsql::lobOpen**(string `$oid`, string `$mode` = "rb"): resource|false

Pdo\Pgsql::lobOpen() opens a stream to access the data referenced by `oid`. All usual filesystem functions, such as fread(), fwrite() or fgets() can be used to manipulate the contents of the stream.

> **Note** :
>
> This function, and all manipulations of the large object, must be called and carried out within a transaction.

Parameters

`oid`

A large object identifier.

`mode`

The access mode. If `mode` contains `w` or `+`, the stream is opened for reading and writing; otherwise it is opened for reading only. A `b` (binary) flag has no effect. The default `"rb"` opens the stream for reading.

Return Values

Returns a stream resource on success, or `false` on failure.

Examples

Example #1 Pdo\Pgsql::lobOpen() example

Following on from the Pdo\Pgsql::lobCreate() example, this code snippet retrieves the large object from the database and outputs it to the browser.

```
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->beginTransaction();
$stmt = $db->prepare("SELECT oid FROM BLOBS WHERE ident = ?");
$stmt->execute([$some_id]);
$stmt->bindColumn('oid', $oid, PDO::PARAM_STR);
$stmt->fetch(PDO::FETCH_BOUND);
```

```php
$stream = $db->lobOpen($oid, 'r');
header("Content-type: application/octet-stream");
fpassthru($stream);
?>
```

See Also

- Pdo\Pgsql::lobCreate() - Creates a new large object
- Pdo\Pgsql::lobUnlink() - Deletes the large object
- pg_lo_create() - Create a large object
- pg_lo_open() - Open a large object

Pdo\Pgsql::lobUnlink

Pdo\Pgsql::lobUnlink

(PHP 8 >= 8.4.0)

Pdo\Pgsql::lobUnlink — Deletes the large object

Description

public function **Pdo\Pgsql::lobUnlink**(string `$oid`): bool

Deletes a large object from the database identified by OID.

> **Note** :
>
> This function, and all manipulations of the large object, must be called and carried out within a transaction.

Parameters

`oid`

A large object identifier.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Pdo\Pgsql::lobUnlink() example

This example unlinks a large object from the database before deleting the row that references it. It uses the blobs table from the Pdo\Pgsql::lobCreate() and Pdo\Pgsql::lobOpen() examples.

```php
<?php
$db = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->beginTransaction();
$db->lobUnlink($oid);
$stmt = $db->prepare("DELETE FROM BLOBS where ident = ?");
$stmt->execute([$some_id]);
$db->commit();
?>
```

See Also

- Pdo\Pgsql::lobCreate() - Creates a new large object
- Pdo\Pgsql::lobOpen() - Opens an existing large object stream
- pg_lo_create() - Create a large object
- pg_lo_open() - Open a large object

Pdo\Pgsql::setNoticeCallback

Pdo\Pgsql::setNoticeCallback

(PHP 8 >= 8.4.0)

Pdo\Pgsql::setNoticeCallback — Set a callback to handle notice and warning messages generated by the backend

Description

public function **Pdo\Pgsql::setNoticeCallback**(?callable `$callback`): void

Set a callback to handle notice and warning messages generated by the backend. This includes messages emitted by PostgreSQL itself, as well as those raised by user-defined SQL functions using `RAISE`. Please note that the actual receipt of these messages depends on the backend setting `client_min_messages`.

Parameters

`callback`

If `null` is passed, the handler is reset to its default state.

Otherwise, the handler is a callback with the following signature:

function handler(string `$message`): void

`message`

A message generated by the backend.

Return Values

No value is returned.

Examples

Example #1 **Pdo\Pgsql::setNoticeCallback()** example

```php
<?php
$pdo = new Pdo\Pgsql('pgsql:dbname=test host=localhost', $user, $pass);

$pdo->exec('CREATE TABLE parent(id int primary key)');
$pdo->exec('CREATE TABLE child(id int references parent)');

$pdo->setNoticeCallback(function ($message) {
    echo $message;
});

$pdo->exec('TRUNCATE parent CASCADE');
?>
```

The above example will output something similar to:

```
NOTICE:  truncate cascades to table "child"
```

SQLite PDO Driver

Introduction

PDO_SQLITE is a driver that implements the PHP Data Objects (PDO) interface to enable access to SQLite 3 databases.

> **Note** :
>
> PDO_SQLITE allows using strings apart from streams together with `PDO::PARAM_LOB`.

Installation

The PDO_SQLITE PDO driver is enabled by default. To disable, **--without-pdo-sqlite[=DIR]** may be used, where the optional `[=DIR]` is the sqlite base install directory. As of PHP 7.4.0 » libsqlite ≥ 3.5.0 is required, and as of PHP 8.5.0 libsqlite ≥ 3.7.17 is required. Formerly, the bundled libsqlite could have been used instead, and was the default, if `[=DIR]` has been omitted.

> **Note** : **Additional setup on Windows as of PHP 7.4.0**
>
> In order for this extension to work, there are DLL files that must be available to the Windows system *PATH*. For information on how to do this, see the FAQ entitled "How do I add my PHP directory to the PATH on Windows". Although copying DLL files from the PHP folder into the Windows system directory also works (because the system directory is by default in the system's *PATH*), this is not recommended. *This extension requires the following files to be in the PATH: libsqlite3.dll.*

Table of Contents

PDO_SQLITE DSN

PDO_SQLITE DSN

(PHP 5 >= 5.1.0, PHP 7, PECL PDO_SQLITE >= 0.2.0)

PDO_SQLITE DSN — Connecting to SQLite databases

Description

The PDO_SQLITE Data Source Name (DSN) is composed of the following elements:

DSN prefix (SQLite 3)

The DSN prefix is **`sqlite:`**.

- To access a database on disk, the absolute path has to be appended to the DSN prefix.
- To create a database in memory, `:memory:` has to be appended to the DSN prefix.
- If the DSN consists of the DSN prefix only, a temporary database is used, which is deleted when the connection is closed.

Examples

Example #1 PDO_SQLITE DSN examples

The following examples show PDO_SQLITE DSN for connecting to SQLite databases:

```
sqlite:/opt/databases/mydb.sq3
sqlite::memory:
sqlite:
```

PDO::sqliteCreateAggregate

PDO::sqliteCreateAggregate

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo_sqlite >= 1.0.0)

PDO::sqliteCreateAggregate — Alias of Pdo\Sqlite::createAggregate()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

public function **PDO::sqliteCreateAggregate**(
  string `$name`,
  callable `$step`,
  callable `$finalize`,
  int `$numArgs = -1`
): bool

This method is an alias of: Pdo\Sqlite::createAggregate().

PDO::sqliteCreateCollation

PDO::sqliteCreateCollation

(PHP 5 >= 5.3.11, PHP 7, PHP 8)

PDO::sqliteCreateCollation — Alias of Pdo\Sqlite::createCollation()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

public function **PDO::sqliteCreateCollation**(string `$name`, callable `$callback`): bool

This method is an alias of: Pdo\Sqlite::createCollation().

PDO::sqliteCreateFunction

PDO::sqliteCreateFunction

(PHP 5 >= 5.1.0, PHP 7, PHP 8, PECL pdo_sqlite >= 1.0.0)

PDO::sqliteCreateFunction — Alias of Pdo\Sqlite::createFunction()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

public function **PDO::sqliteCreateFunction**(
    string `$function_name`,
    callable `$callback`,
    int `$num_args` = -1,
    int `$flags` = 0
): bool

This method is an alias of: Pdo\Sqlite::createFunction().

Changelog

Version	Description
7.1.4	The `flags` parameter has been added.

Pdo\Sqlite

(PHP 8 >= 8.4.0)

Introduction

A PDO subclass representing a connection using the SQLite PDO driver.

This driver supports a dedicated SQL query parser for the SQLite dialect. It can handle the following:

- Single, double-quoted, and backtick literals, with doubling as escaping mechanism.
- Square brackets quoting for identifiers.
- Two-dashes and C-style comments (non-nested).

Class synopsis

class **Pdo\Sqlite** extends PDO {
/* Inherited constants */
public const int *PDO::PARAM_NULL*;
public const int *PDO::PARAM_BOOL* = 5;
public const int *PDO::PARAM_INT* = 1;
public const int *PDO::PARAM_STR* = 2;
public const int *PDO::PARAM_LOB* = 3;

public const int *PDO::PARAM_STMT* = 4;
public const int *PDO::PARAM_INPUT_OUTPUT*;
public const int *PDO::PARAM_STR_NATL*;
public const int *PDO::PARAM_STR_CHAR*;
public const int *PDO::PARAM_EVT_ALLOC*;
public const int *PDO::PARAM_EVT_FREE*;
public const int *PDO::PARAM_EVT_EXEC_PRE*;
public const int *PDO::PARAM_EVT_EXEC_POST*;
public const int *PDO::PARAM_EVT_FETCH_PRE*;
public const int *PDO::PARAM_EVT_FETCH_POST*;
public const int *PDO::PARAM_EVT_NORMALIZE*;
public const int *PDO::FETCH_DEFAULT*;
public const int *PDO::FETCH_LAZY*;
public const int *PDO::FETCH_ASSOC*;
public const int *PDO::FETCH_NUM*;
public const int *PDO::FETCH_BOTH*;
public const int *PDO::FETCH_OBJ*;
public const int *PDO::FETCH_BOUND*;
public const int *PDO::FETCH_COLUMN*;
public const int *PDO::FETCH_CLASS*;
public const int *PDO::FETCH_INTO*;
public const int *PDO::FETCH_FUNC*;
public const int *PDO::FETCH_GROUP*;
public const int *PDO::FETCH_UNIQUE*;
public const int *PDO::FETCH_KEY_PAIR*;
public const int *PDO::FETCH_CLASSTYPE*;
public const int *PDO::FETCH_SERIALIZE*;
public const int *PDO::FETCH_PROPS_LATE*;
public const int *PDO::FETCH_NAMED*;
public const int *PDO::ATTR_AUTOCOMMIT*;
public const int *PDO::ATTR_PREFETCH*;
public const int *PDO::ATTR_TIMEOUT*;
public const int *PDO::ATTR_ERRMODE*;
public const int *PDO::ATTR_SERVER_VERSION*;
public const int *PDO::ATTR_CLIENT_VERSION*;
public const int *PDO::ATTR_SERVER_INFO*;
public const int *PDO::ATTR_CONNECTION_STATUS*;
public const int *PDO::ATTR_CASE*;
public const int *PDO::ATTR_CURSOR_NAME*;
public const int *PDO::ATTR_CURSOR*;
public const int *PDO::ATTR_ORACLE_NULLS*;
public const int *PDO::ATTR_PERSISTENT*;
public const int *PDO::ATTR_STATEMENT_CLASS*;
public const int *PDO::ATTR_FETCH_TABLE_NAMES*;
public const int *PDO::ATTR_FETCH_CATALOG_NAMES*;
public const int *PDO::ATTR_DRIVER_NAME*;
public const int *PDO::ATTR_STRINGIFY_FETCHES*;
public const int *PDO::ATTR_MAX_COLUMN_LEN*;
public const int *PDO::ATTR_EMULATE_PREPARES*;
public const int *PDO::ATTR_DEFAULT_FETCH_MODE*;
public const int *PDO::ATTR_DEFAULT_STR_PARAM*;

```
public const int PDO::ERRMODE_SILENT;
public const int PDO::ERRMODE_WARNING;
public const int PDO::ERRMODE_EXCEPTION;
public const int PDO::CASE_NATURAL;
public const int PDO::CASE_LOWER;
public const int PDO::CASE_UPPER;
public const int PDO::NULL_NATURAL;
public const int PDO::NULL_EMPTY_STRING;
public const int PDO::NULL_TO_STRING;
public const string PDO::ERR_NONE;
public const int PDO::FETCH_ORI_NEXT;
public const int PDO::FETCH_ORI_PRIOR;
public const int PDO::FETCH_ORI_FIRST;
public const int PDO::FETCH_ORI_LAST;
public const int PDO::FETCH_ORI_ABS;
public const int PDO::FETCH_ORI_REL;
public const int PDO::CURSOR_FWDONLY;
public const int PDO::CURSOR_SCROLL;
/* Constants */
public const int DETERMINISTIC;
public const int OPEN_READONLY;
public const int OPEN_READWRITE;
public const int OPEN_CREATE;
public const int ATTR_OPEN_FLAGS;
public const int ATTR_READONLY_STATEMENT;
public const int ATTR_EXTENDED_RESULT_CODES;
public const int ATTR_BUSY_STATEMENT;
public const int ATTR_EXPLAIN_STATEMENT;
public const int EXPLAIN_MODE_PREPARED;
public const int EXPLAIN_MODE_EXPLAIN;
public const int EXPLAIN_MODE_EXPLAIN_QUERY_PLAN;
public const int ATTR_TRANSACTION_MODE;
public const int TRANSACTION_MODE_DEFERRED;
public const int TRANSACTION_MODE_IMMEDIATE;
public const int TRANSACTION_MODE_EXCLUSIVE;
public const int OK;
public const int DENY;
public const int IGNORE;
/* Methods */
public function createAggregate(
    string $name,
    callable $step,
    callable $finalize,
    int $numArgs = -1
): bool
public function createCollation(string $name, callable $callback): bool
public function createFunction(
    string $function_name,
    callable $callback,
    int $num_args = -1,
```

```
    int $flags = 0
): bool
public function loadExtension(string $name): void
public function openBlob(
    string $table,
    string $column,
    int $rowid,
    ?string $dbname = "main",
    int $flags = Pdo\Sqlite::OPEN_READONLY
): resource|false
public function setAuthorizer(?callable $callback): void
/* Inherited methods */
public function PDO::__construct(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
)
public function PDO::beginTransaction(): bool
public function PDO::commit(): bool
public static function PDO::connect(
    string $dsn,
    ?string $username = null,
    #[\SensitiveParameter] ?string $password = null,
    ?array $options = null
): static
public function PDO::errorCode(): ?string
public function PDO::errorInfo(): array
public function PDO::exec(string $statement): int|false
public function PDO::getAttribute(int $attribute): mixed
public static function PDO::getAvailableDrivers(): array
public function PDO::inTransaction(): bool
public function PDO::lastInsertId(?string $name = null): string|false
public function PDO::prepare(string $query, array $options = []): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = null): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_COLUMN, int $colno):
PDOStatement|false
public function PDO::query(
    string $query,
    ?int $fetchMode = PDO::FETCH_CLASS,
    string $classname,
    array $constructorArgs
): PDOStatement|false
public function PDO::query(string $query, ?int $fetchMode = PDO::FETCH_INTO, object $object):
PDOStatement|false
public function PDO::quote(string $string, int $type = PDO::PARAM_STR): string|false
public function PDO::rollBack(): bool
public function PDO::setAttribute(int $attribute, mixed $value): bool
}
```

Predefined Constants

Pdo\Sqlite::DETERMINISTIC

Pdo\Sqlite::OPEN_READONLY

Pdo\Sqlite::OPEN_READWRITE

Pdo\Sqlite::OPEN_CREATE

Pdo\Sqlite::ATTR_OPEN_FLAGS

Pdo\Sqlite::ATTR_READONLY_STATEMENT

Pdo\Sqlite::ATTR_EXTENDED_RESULT_CODES

Pdo\Sqlite::ATTR_BUSY_STATEMENT

A read-only statement attribute, retrieved with PDOStatement::getAttribute(), which is **true** if the statement has been executed and still has a pending row, and **false** otherwise. Available as of PHP 8.5.0.

Pdo\Sqlite::ATTR_EXPLAIN_STATEMENT

A statement attribute controlling the explain mode, set with PDOStatement::setAttribute(). Possible values are **Pdo\Sqlite::EXPLAIN_MODE_PREPARED**, **Pdo\Sqlite::EXPLAIN_MODE_EXPLAIN**, and **Pdo\Sqlite::EXPLAIN_MODE_EXPLAIN_QUERY_PLAN**. A ValueError is thrown if PHP has been compiled against a libsqlite older than 3.43.0. Available as of PHP 8.5.0.

Pdo\Sqlite::EXPLAIN_MODE_PREPARED

Value for **Pdo\Sqlite::ATTR_EXPLAIN_STATEMENT**: returns the statement as prepared (no explanation). Available as of PHP 8.5.0, when compiled against libsqlite >= 3.43.0.

Pdo\Sqlite::EXPLAIN_MODE_EXPLAIN

Value for **Pdo\Sqlite::ATTR_EXPLAIN_STATEMENT**: returns the opcodes of the virtual machine used to execute the statement (equivalent to SQLite's `EXPLAIN`). Available as of PHP 8.5.0, when compiled against libsqlite >= 3.43.0.

Pdo\Sqlite::EXPLAIN_MODE_EXPLAIN_QUERY_PLAN

Value for **Pdo\Sqlite::ATTR_EXPLAIN_STATEMENT**: returns the query plan (equivalent to SQLite's `EXPLAIN QUERY PLAN`). Available as of PHP 8.5.0, when compiled against libsqlite >= 3.43.0.

Pdo\Sqlite::ATTR_TRANSACTION_MODE

A connection attribute configuring the transaction mode used by PDO::beginTransaction(). Possible values are **Pdo\Sqlite::TRANSACTION_MODE_DEFERRED**, **Pdo\Sqlite::TRANSACTION_MODE_IMMEDIATE**, and **Pdo\Sqlite::TRANSACTION_MODE_EXCLUSIVE**. Available as of PHP 8.5.0.

Pdo\Sqlite::TRANSACTION_MODE_DEFERRED

Value for **Pdo\Sqlite::ATTR_TRANSACTION_MODE**: uses SQLite's DEFERRED transaction mode (the default). Available as of PHP 8.5.0.

Pdo\Sqlite::TRANSACTION_MODE_IMMEDIATE

Value for **Pdo\Sqlite::ATTR_TRANSACTION_MODE**: uses SQLite's IMMEDIATE transaction mode. Available as of PHP 8.5.0.

`Pdo\Sqlite::TRANSACTION_MODE_EXCLUSIVE`

Value for `Pdo\Sqlite::ATTR_TRANSACTION_MODE`: uses SQLite's EXCLUSIVE transaction mode. Available as of PHP 8.5.0.

`Pdo\Sqlite::OK`

Return value of an authorizer callback set with Pdo\Sqlite::setAuthorizer(), allowing the action. Available as of PHP 8.5.0.

`Pdo\Sqlite::DENY`

Return value of an authorizer callback set with Pdo\Sqlite::setAuthorizer(), denying the action. Available as of PHP 8.5.0.

`Pdo\Sqlite::IGNORE`

Return value of an authorizer callback set with Pdo\Sqlite::setAuthorizer(), substituting a `null` value for the column that would have been read. Available as of PHP 8.5.0.

Changelog

Version	Description
8.5.0	Added constants `Pdo\Sqlite::ATTR_BUSY_STATEMENT`, `Pdo\Sqlite::ATTR_EXPLAIN_STATEMENT`, `Pdo\Sqlite::EXPLAIN_MODE_PREPARED`, `Pdo\Sqlite::EXPLAIN_MODE_EXPLAIN`, `Pdo\Sqlite::EXPLAIN_MODE_EXPLAIN_QUERY_PLAN`, `Pdo\Sqlite::ATTR_TRANSACTION_MODE`, `Pdo\Sqlite::TRANSACTION_MODE_DEFERRED`, `Pdo\Sqlite::TRANSACTION_MODE_IMMEDIATE`, and `Pdo\Sqlite::TRANSACTION_MODE_EXCLUSIVE`, `Pdo\Sqlite::OK`, `Pdo\Sqlite::DENY`, and `Pdo\Sqlite::IGNORE`.

Table of Contents

Pdo\Sqlite::createAggregate

Pdo\Sqlite::createAggregate

(PHP 8 >= 8.4.0)

Pdo\Sqlite::createAggregate — Registers an aggregating user-defined function for use in SQL statements

Description

```
public function Pdo\Sqlite::createAggregate(
    string $name,
    callable $step,
    callable $finalize,
    int $numArgs = -1
): bool
```

This method is similar to Pdo\Sqlite::createFunction() except that it registers functions that can be used to calculate a result aggregated across all the rows of a query.

The key difference between this method and Pdo\Sqlite::createFunction() is that two functions are required to manage the aggregate.

> **Tip**

> By using this method it is possible to override native SQL functions.

Parameters

`name`

The name of the function used in SQL statements.

`step`

Callback function called for each row of the result set. The callback should accumulate the result and store it in the aggregation context.

This function need to be defined as:

```
function step(
    mixed $context,
    int $rownumber,
    mixed $value,
    mixed ...$values
): mixed
```

`context`

`null` for the first row; on subsequent rows it will have the value that was previously returned from the step function; you should use this to maintain the aggregate state.

`rownumber`

The current row number.

`value`

The first argument passed to the aggregate.

`values`

Further arguments passed to the aggregate.

The return value of this function will be used as the `context` argument in the next call of the step or finalize functions.

`finalize`

Callback function to aggregate the "stepped" data from each row. Once all the rows have been processed, this function will be called, and it should then take the data from the aggregation context and return the result. This callback function should return a type understood by SQLite (i.e. scalar type).

This function need to be defined as:

```
function fini(mixed $context, int $rowcount): mixed
```

`context`

Holds the return value from the very last call to the step function.

`rowcount`

Holds the number of rows over which the aggregate was performed.

The return value of this function will be used as the return value for the aggregate.

`numArgs`

Hint to the SQLite parser if the callback function accepts a predetermined number of arguments.

Return Values

Returns **`true`** on success or **`false`** on failure.

Examples

Example #1 **Pdo\Sqlite::createAggregate()** example

In this example we will create a custom aggregate function named `max_length` that can be used in SQL queries.

In this example, we are creating an aggregating function, named `max_length`, that will calculate the length of the longest string in one of the columns of the table. For each row, the `max_len_step` function is called and passed a `$context` parameter. The context parameter is just like any other PHP variable and be set to hold an array or even an object. In this example, we are using it to hold the maximum length we have seen so far; if the *$string* has a length longer than the current maximum, we update the context to hold this new maximum length.

After all the rows have been processed, SQLite calls the `max_len_finalize` function to determine the aggregate result. It is possible to perform some kind of calculation based on the data in `$context`. In this basic example the result was calculated as the query progressed, thus so the context value can be directly returned.

```php
<?php
$data = [
   'one',
   'two',
   'three',
   'four',
   'five',
   'six',
   'seven',
   'eight',
   'nine',
   'ten',
];
$db = new Pdo\Sqlite('sqlite::memory:');
$db->exec("CREATE TABLE strings(a)");
$insert = $db->prepare('INSERT INTO strings VALUES (?)');
foreach ($data as $str) {
    $insert->execute(array($str));
}
$insert = null;

function max_len_step($context, $row_number, $string)
{
    if (strlen($string) > $context) {
        $context = strlen($string);
    }
    return $context;
}

function max_len_finalize($context, $row_count)
{
    return $context === null ? 0 : $context;
}
```

```
$db->createAggregate('max_len', 'max_len_step', 'max_len_finalize');

var_dump($db->query('SELECT max_len(a) from strings')->fetchAll());

?>
```

> **Tip**
>
> It is NOT recommended for you to store a copy of the values in the context and then process them at the end, as you would cause SQLite to use a lot of memory to process the query - just think of how much memory you would need if a million rows were stored in memory, each containing a string 32 bytes in length.

See Also

- Pdo\Sqlite::createFunction() - Registers a user-defined function for use in SQL statements
- Pdo\Sqlite::createCollation() - Registers a user-defined function for use as a collating function in SQL statements
- **sqlite_create_function()**
- **sqlite_create_aggregate()**

Pdo\Sqlite::createCollation

Pdo\Sqlite::createCollation

(PHP 8 >= 8.4.0)

Pdo\Sqlite::createCollation — Registers a user-defined function for use as a collating function in SQL statements

Description

public function **Pdo\Sqlite::createCollation**(string `$name`, callable `$callback`): bool

This method is similar to Pdo\Sqlite::createFunction() except that it registers functions that are used to collate strings.

Parameters

`name`

Name of the SQL collating function to be created or redefined.

`callback`

Callback function that defines the behaviour of a collation. It must accept two strings and return `-1`, `0`, or `1` if the first string sorts before, sorts identically, or sorts after the second string respectively. An internal function that behaves like this is strcmp().

This function need to be defined as:

function collation(string `$string1`, string `$string2`): int

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

A TypeError is thrown if the `callback` does not return an int.

Changelog

Version	Description
8.4.0	A TypeError is now thrown if the `callback` does not return an int; previously the returned value was cast to int.

Examples

Example #1 **Pdo\Sqlite::createCollation()** example

```php
<?php
$db = new Pdo\Sqlite('sqlite::memory:');
$db->exec("CREATE TABLE test (col1 string)");
$db->exec("INSERT INTO test VALUES ('a1')");
$db->exec("INSERT INTO test VALUES ('a10')");
$db->exec("INSERT INTO test VALUES ('a2')");

$db->sqliteCreateCollation('NATURAL_CMP', 'strnatcmp');
foreach ($db->query("SELECT col1 FROM test ORDER BY col1") as $row) {
  echo $row['col1'] . "\n";
}
echo "\n";
foreach ($db->query("SELECT col1 FROM test ORDER BY col1 COLLATE NATURAL_CMP") as $row) {
  echo $row['col1'] . "\n";
}
?>
```

The above example will output:

```
a1
a10
a2

a1
a2
a10
```

See Also

- Pdo\Sqlite::createFunction() - Registers a user-defined function for use in SQL statements
- Pdo\Sqlite::createAggregate() - Registers an aggregating user-defined function for use in SQL statements
- **sqlite_create_function()**
- **sqlite_create_aggregate()**

Pdo\Sqlite::createFunction

Pdo\Sqlite::createFunction

(PHP 8 >= 8.4.0)

Pdo\Sqlite::createFunction — Registers a user-defined function for use in SQL statements

Description

```
public function Pdo\Sqlite::createFunction(
    string $function_name,
    callable $callback,
    int $num_args = -1,
    int $flags = 0
): bool
```

This method allows PHP function to be registered with SQLite as a user-defined function, so that it can be called within SQL queries. The defined function can be used in any SQL query that allows function calls, for example `SELECT`, `UPDATE`, or triggers.

> **Tip**
>
> By using this method it is possible to override native SQL functions.

Parameters

`function_name`

The name of the function used in SQL statements.

`callback`

Callback function to handle the defined SQL function.

> **Note** :
>
> Callback functions should return a type understood by SQLite (i.e. scalar type).

This function need to be defined as:

function callback(mixed `$value`, mixed `...$values`): mixed

`value`

The first argument passed to the SQL function.

`values`

Further arguments passed to the SQL function.

`num_args`

The number of arguments that the SQL function takes. If this parameter is `-1`, then the SQL function may take any number of arguments.

`flags`

A bitmask of flags. Currently, only **`Pdo\Sqlite::DETERMINISTIC`** is supported, which specifies that the function always returns the same result given the same inputs within a single SQL statement.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Pdo\Sqlite::createFunction() example

In this example, we have a function that calculates the SHA256 sum of a string, and then reverses it. When the SQL statement executes, it returns the value of the filename transformed by our function. The data returned in *$rows* contains the processed result.

The beauty of this technique is that there is no need to process the result using a `foreach` loop after the query.

```
<?php
function sha256_and_reverse($string)
{
    return strrev(hash('sha256', $string));
}

$db = new Pdo\Sqlite('sqlite::sqlitedb');
$db->createFunction('sha256rev', 'sha256_and_reverse', 1);
$rows = $db->query('SELECT sha256rev(filename) FROM files')->fetchAll();
?>
```

See Also

- Pdo\Sqlite::createAggregate() - Registers an aggregating user-defined function for use in SQL statements
- Pdo\Sqlite::createCollation() - Registers a user-defined function for use as a collating function in SQL statements
- **sqlite_create_function()**
- **sqlite_create_aggregate()**

Pdo\Sqlite::loadExtension

Pdo\Sqlite::loadExtension

(PHP 8 >= 8.4.0)

Pdo\Sqlite::loadExtension — Description

Description

public function **Pdo\Sqlite::loadExtension**(string `$name`): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Description.

Parameters

`name`

Description.

Return Values

No value is returned.

Errors/Exceptions

When does this function issue `E_*` level errors, and/or throw Exceptions.

Examples

Example #1 **Pdo\Sqlite::loadExtension()** example

Description.

```
<?php
echo "Code example";
?>
```

The above example will output:

```
Code example
```

Notes

> **Note** :
>
> Any notes that don't fit anywhere else should go here.

See Also

- **ClassName::otherMethodName()**
- **some_function()**

Pdo\Sqlite::openBlob

Pdo\Sqlite::openBlob

(PHP 8 >= 8.4.0)

Pdo\Sqlite::openBlob — Description

Description

public function **Pdo\Sqlite::openBlob**(
  string `$table`,
  string `$column`,
  int `$rowid`,
  ?string `$dbname` = "main",

```
    int $flags = Pdo\Sqlite::OPEN_READONLY
): resource|false
```

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Description.

Parameters

`table`

Description.

`column`

Description.

`rowid`

Description.

`dbname`

Description.

`flags`

One of the `Pdo\Sqlite::OPEN_*`

Return Values

Description.

Errors/Exceptions

When does this function issue `E_*` level errors, and/or throw Exceptions.

Examples

Example #1 Pdo\Sqlite::openBlob() example

Description.

```php
<?php
echo "Code example";
?>
```

The above example will output:

```
Code example
```

Notes

> **Note** :
>
> Any notes that don't fit anywhere else should go here.

See Also

- **ClassName::otherMethodName()**
- **some_function()**

Pdo\Sqlite::setAuthorizer

Pdo\Sqlite::setAuthorizer

(PHP 8 >= 8.5.0)

Pdo\Sqlite::setAuthorizer — Configures a callback to be used as an authorizer to limit what a statement can do

Description

public function **Pdo\Sqlite::setAuthorizer**(?callable `$callback`): void

Sets a callback that will be called by SQLite every time an action is performed (reading, deleting, updating, etc.). This is used when preparing an SQL statement from an untrusted source to ensure that the statement does not access data it is not allowed to see or execute malicious statements that damage the database.

The authorizer is used only during the statement preparation phase. It may be called several times for a single statement: a `SELECT` or `UPDATE` query calls it for every column that would be read or updated. It runs again whenever SQLite re-prepares a statement, which may happen while the statement is being executed, for instance after the schema has changed.

The authorizer is called with up to five arguments. The arguments received are described at SQLite3::setAuthorizer().

Only a single authorizer can be in place on a database connection at a time. Each call to this method overrides the previous one. The authorizer is disabled by default, and can be disabled again by setting a `null` callback.

The callback must not modify the database connection that invoked it.

More details can be found in the » SQLite documentation.

> **Note** :
>
> This method is the equivalent of SQLite3::setAuthorizer(), except that it returns void instead of bool.

Parameters

`callback`

The callable to be invoked, or `null` to disable the current authorizer callback.

It must return one of `Pdo\Sqlite::OK`, `Pdo\Sqlite::DENY`, or `Pdo\Sqlite::IGNORE`. When `Pdo\Sqlite::DENY` is returned, the statement that triggered the authorizer fails with an error stating that access is denied. When `Pdo\Sqlite::IGNORE` is returned for a read action, the statement is prepared so that a `null` value is substituted for the column that would have been read.

Return Values

No value is returned.

Errors/Exceptions

This method itself does not throw, but if the authorizer callback does not return an int, or returns an int which is not one of `Pdo\Sqlite::OK`, `Pdo\Sqlite::DENY`, or `Pdo\Sqlite::IGNORE`, the statement preparation throws a TypeError or a ValueError respectively.

Examples

Example #1 Pdo\Sqlite::setAuthorizer() example

Only read actions are allowed on the connection. The action codes are exposed as SQLite3 class constants, which requires the SQLite3 extension to be available; their integer values may be used directly otherwise.

```php
<?php
$db = new Pdo\Sqlite('sqlite::memory:');
$db->exec('CREATE TABLE users (id, name)');

$db->setAuthorizer(function (int $action, ...$args) {
    return match ($action) {
        SQLite3::SELECT, SQLite3::READ => Pdo\Sqlite::OK,
        default => Pdo\Sqlite::DENY,
    };
});

var_dump($db->query('SELECT name FROM users') instanceof PDOStatement);

try {
    $db->exec('DROP TABLE users');
} catch (PDOException $e) {
    echo $e->getMessage(), "\n";
}
?>
```

The above example will output something similar to:

```
bool(true)
SQLSTATE[HY000]: General error: 23 not authorized
```

See Also

- SQLite3::setAuthorizer() - Configures a callback to be used as an authorizer to limit what a statement can do